

Metodologías de desarrollo de software

Breve descripción:

Las metodologías de desarrollo de software proponen un conjunto de procesos y actividades que deben ser desarrolladas por el equipo de desarrollo de software para realizar un trabajo organizado que sea fácil hacerle seguimiento y de esta forma establecer planes de mejora en busca de una mejor calidad de los productos y servicios que se desarrollan.

Mayo 2024

Tabla de contenido

Introducción	3
1. Metodologías de desarrollo de software	5
2. Marcos de trabajos tradicionales.....	6
2.1. Cascada.....	6
2.2. Proceso Racional Unificado - RUP	9
3. Marcos de trabajo ágiles.....	15
3.1. Programación Extrema - XP	17
3.2. Desarrollo rápido de aplicaciones - RAD	20
3.3. Scrum	24
4. Planeación de proyectos de software	32
Síntesis	36
Material complementario.....	37
Glosario	38
Referencias bibliográficas	39
Créditos	41

Introducción

El mundo del desarrollo de software es complejo y variado, con una amplia gama de metodologías y marcos de trabajo diseñados para enfrentar los retos que supone la creación de sistemas efectivos y eficientes. En este componente, se exploran las principales metodologías y marcos de trabajo que han moldeado la ingeniería de software moderna.

- **Metodologías de desarrollo de software**

Aquí se examina el enfoque estructurado para el desarrollo de software, incluyendo la comprensión de requisitos, diseño, implementación, pruebas y mantenimiento. Discutiremos cómo estas metodologías ayudan a gestionar la complejidad y asegurar la calidad en la entrega de productos de software.

- **Marcos de trabajos tradicionales**

Este apartado se sumerge en las metodologías clásicas como el modelo en Cascada y el Proceso Racional Unificado (RUP). Analizaremos cómo estos enfoques secuenciales y predecibles pueden proporcionar una estructura sólida para el desarrollo de software, a la vez que consideramos sus limitaciones en entornos de cambio rápido.

- **Marcos de trabajo ágiles**

La agilidad es la clave de las metodologías modernas, permitiendo a los equipos responder a la imprevisibilidad y al cambio constante.

Exploraremos las prácticas y principios de marcos ágiles como la Programación Extrema (XP), el Desarrollo Rápido de Aplicaciones (RAD) y Scrum, que promueven la colaboración, adaptabilidad y entrega continua.

- **Planeación de proyectos de software**

Desde la asignación de recursos hasta la gestión de riesgos y la definición de cronogramas, la planificación es vital para el éxito y la sostenibilidad de cualquier proyecto de software.

Con estas bases establecidas, se adentrará en el corazón de cada metodología y marco de trabajo, buscando no solo entender su propósito y prácticas, sino también cómo y cuándo implementarlas de manera efectiva en los proyectos de software.

1. Metodologías de desarrollo de software

Es importante aclarar que existe un gran número de definiciones sobre lo que es una metodología, para evitar cualquier confusión en este material se utilizará la definición dada por Maida y Pacienza (2015), quienes indican que:

Una metodología hace referencia a un conjunto de procedimientos genéricos y lógicos que se usan para alcanzar un objetivo particular utilizando un conjunto de habilidades y conocimientos.

Las metodologías de desarrollo de software siempre parten de un componente teórico y, cuando son usadas por los equipos de trabajo, conllevan a la utilización de un conjunto de técnicas y métodos que al final determinarán las tareas generales y específicas que se deberían realizar para alcanzar un objetivo.

Existen diferentes tipos de metodologías de desarrollo de software que fueron ideadas pensando en problemas particulares presentados en la industria en contextos específicos, por lo cual es importante conocer sus diferentes características y contrastarlas con las necesidades particulares a las que se enfrenta a la hora de desarrollar un producto o servicio. Es decir, cada una de estas tiene ventajas y enfoques que pueden ser reutilizados en diferentes momentos.

Existen dos grandes clasificaciones de metodologías de desarrollo de software que se agrupan generalmente como marcos de trabajo tradicionales o marcos de trabajo ágiles que se presentan a continuación.

2. Marcos de trabajos tradicionales

Para el desarrollo de un buen producto de software se debe iniciar con un excelente proceso de planificación y gestión durante todas las etapas y actividades que involucran transformar una idea o requerimiento en un producto o servicio que será utilizado por un cliente particular.

“Los marcos de trabajo o metodologías tradicionales se caracterizan por centrar la mayor parte de su esfuerzo en la planificación y control del proceso, lo que conlleva a una documentación exhaustiva y precisa de los artefactos que describen los requisitos y los modelos del sistema en las etapas iniciales del desarrollo del proyecto” (Maida y Pacienza, 2015).

Lo anterior supone que este tipo de enfoques son óptimos en proyectos en los cuales los requisitos están plenamente identificados y delimitados, y donde no se espera que se produzca ningún cambio en lo establecido mientras el proyecto es finalizado.

A continuación, se describen algunas metodologías que se enmarcan en los marcos tradicionales de desarrollo de software.

2.1. Cascada

Este es uno de los modelos genéricos más ampliamente conocidos en la ingeniería de software y se deriva de procesos de ingeniería de sistemas más generales (Royce, 1970). Este modelo plantea un proceso lineal donde las actividades de desarrollo de un producto o servicios de software se agrupan en un conjunto de fases sucesivas. Estas son desarrolladas una única vez y los resultados de cada fase son la entrada requerida para la siguiente, ninguna fase puede iniciar si la fase anterior no

ha sido finalizada, generalmente mediante un formalismo que puede ser un documento.

Según Sommerville (2005), el modelo en cascada se compone de cinco etapas principales que se asocian con actividades fundamentales en el proceso de desarrollo de software, las cuales son:

- **Requerimientos**

Se lleva a cabo la recopilación y análisis detallado de las necesidades del sistema o software. Se definen los objetivos, funciones, interfaces de usuario y demás necesidades del cliente o mercado.

- **Diseño**

Se establece cómo será construido el sistema para cumplir con los requerimientos identificados. Incluye la arquitectura del sistema, la selección de patrones de diseño, y la planificación de interfaces y experiencias de usuario.

- **Implementación**

En esta fase, se escribe el código fuente del software, basado en el diseño previamente establecido. Cada componente es desarrollado y probado individualmente antes de ser integrado en el sistema mayor.

- **Verificación**

Esta etapa implica probar el sistema completo para validar que cumple con los requerimientos especificados. Incluye pruebas unitarias, de integración, de sistema y de aceptación del usuario.

- **Mantenimiento**

Una vez que el software está en operación, esta fase aborda la corrección de errores, la actualización de funciones y la optimización del rendimiento. También se pueden incorporar nuevos requerimientos según sea necesario.

Continuando con el tema, se presenta un comparativo que resume las principales ventajas y desventajas de este modelo.

Ventaja

- La definición clara de fases permite el desarrollo de una estructura sencilla que es ideal para proyectos sencillos y cortos.
- Siguiendo este modelo se genera una muy buena documentación del proceso y es posible definir hitos claros.
- Es más fácil realizar la estimación de costos al inicio del proyecto.
- Es fácil elaborar cronogramas de trabajo con base al desarrollo de las actividades de cada fase.

Desventaja

- No se acopla muy bien a proyectos complejos con múltiples equipos trabajando en paralelo, ya que las fases normalmente se traslapan y es difícil diferenciarlas.
- Es difícil introducir cambios en el transcurso del proyecto.
- Los usuarios finales y clientes normalmente son integrados al final del proceso, lo que impide tener realimentación y ajustes en etapas tempranas.

- Existen fallos que solo son detectados cuando el sistema entra en funcionamiento, lo que puede ser desastroso para un proyecto que sigue este modelo.

El modelo en cascada define cuatro grupos de roles típicos, los cuales se mencionan a continuación:

- **Desarrolladores**

Es el rol más importante en la metodología cascada, ya que son los encargados directos de la creación de código.

- **Testers**

Encargados de encontrar fallas en los productos finales y retornar el software a los desarrolladores para arreglar todos los defectos.

- **Analista del negocio**

Encargado de la realización de estrategias de negocio que le permitan al producto software alcanzar popularidad en el mercado digital.

- **Administrador del proyecto**

Es responsable de la calidad final del software. Administra el proyecto y lo subdivide en tareas entre los miembros del equipo.

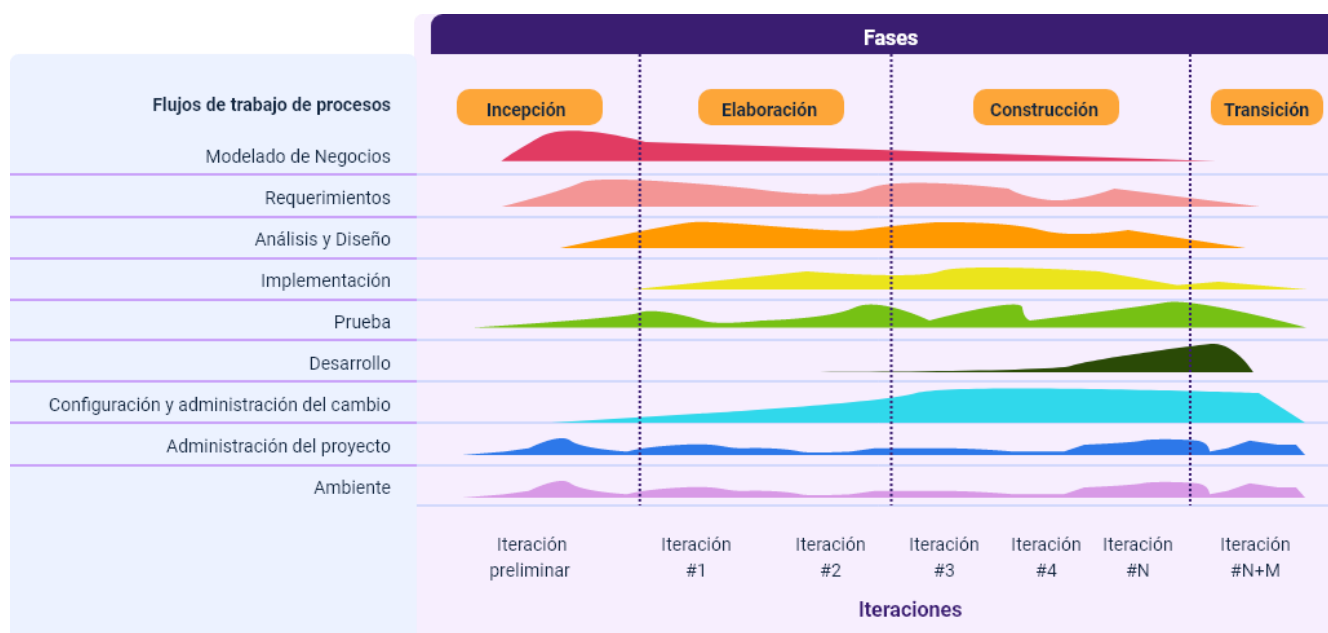
2.2. Proceso Racional Unificado - RUP

RUP es una sigla en inglés que corresponde a Rational Unified Process, el cual es un proceso de desarrollo de software tradicional que se basa en el modelo en cascada y fue desarrollado por la empresa Rational Software, actualmente propiedad de IBM. Esta metodología se enfoca en la arquitectura y es guiada por casos de uso (requerimientos) (Kruchten, 2003).

RUP divide el proceso de desarrollo en cuatro grandes fases. Dentro de ellas, se realizan iteraciones donde se desglosan, con mayor o menor intensidad, un conjunto de disciplinas según la fase en la que se está.

A continuación, se presentarán las fases y disciplinas propuestas por RUP.

Figura 1. Fases y disciplinas de RUP



Nota. Tomada de Man, Hafriz, Nural, Maizura, Noor, Wan, Bakar y Man (2008).

A continuación, se describen las fases y su impacto en el proceso, por lo que se le invita a revisar el siguiente recurso de aprendizaje.

- **Inicio**

La primera gran fase definida por RUP es la de inicio, en la cual se abordan actividades principalmente enfocadas en la comprensión del problema y el tipo de tecnología a utilizar, por lo que hay una gran carga en actividades

relacionadas con la disciplina de modelado del negocio y especificación de requisitos.

- **Elaboración**

La segunda fase es la elaboración, donde se centra la mayor parte del esfuerzo en la definición general de la arquitectura del sistema y el refinamiento de requisitos y modelado del negocio. RUP generalmente se apoya en el lenguaje de modelado UML para el modelado del negocio y descripción de la arquitectura.

- **Construcción**

La tercera fase corresponde a la construcción, donde se centran en las actividades relacionadas con la construcción del producto. Normalmente, esta fase está constituida por varias iteraciones, donde en cada una de ellas se desarrolla un subconjunto de requerimientos que normalmente están especificados como casos de uso.

- **Transición**

La cuarta fase se desarrolla principalmente las disciplinas de pruebas y despliegue, es decir, las actividades encaminadas a garantizar que el producto esté listo para entrega a sus usuarios.

Es importante señalar que, dentro de cada una de las fases del RUP, se desarrollan varias disciplinas, y el esfuerzo y tiempo invertidos en cada una tendrán un enfoque diferente dependiendo de la fase del proyecto en la que se esté.

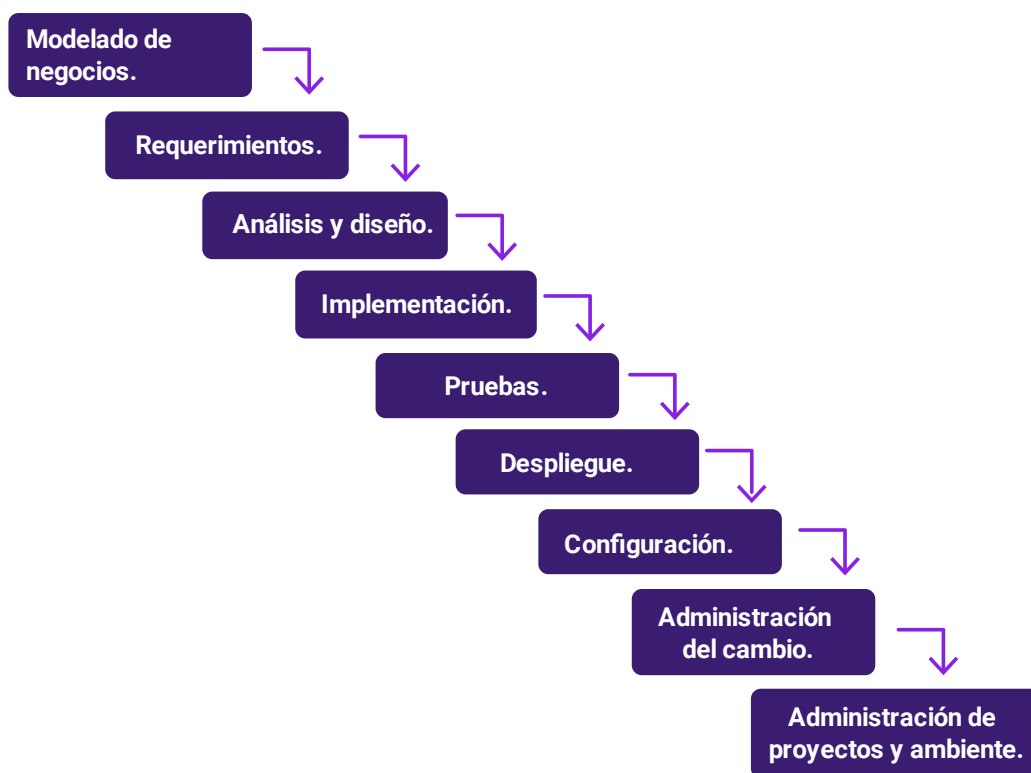
Otro aspecto relevante de esta metodología es que se sustenta en un conjunto de artefactos que se elaboran con el objetivo de especificar el proceso de análisis y diseño

que respalda la construcción del software. Entre los artefactos más comunes se incluyen:

- Documento de visión.
- Documento de especificación de requisitos.
- Diagramas de casos de uso.
- Modelos conceptuales (clases y entidad relación).
- Diagramas que representan la vista de implementación como:
 - Diagramas de secuencia.
 - Diagramas de estados.
 - Diagramas de colaboración, entre otros.

Las disciplinas, por otra parte, representan un conjunto de actividades relacionadas con un área específica del proyecto y están inspiradas en el modelo en cascada. RUP establece las siguientes disciplinas según Kruchten (2003):

Figura 2. Fases en el desarrollo de proyectos



A continuación, se describen cada una de las disciplinas mencionadas y cómo estás aportan en el proceso.

RUP propone una categorización de roles encargados de la realización de actividades dentro de cada una de las disciplinas que son:

- **Analistas**
 - Analistas de procesos de negocio.
 - Diseñadores del negocio.
 - Analistas del sistema.
 - Especificador de requisitos.
 - Diseñadores de interfaces de usuario o similares.

- **Desarrolladores**

- Arquitectos de software.
- Diseñador de bases de datos.
- Desarrollador backend y frontend.
- Cualquier otro rol relacionado con procesos de codificación o integración de código.

- **Testers**

- Diseñadores de pruebas.
- Implementadores de pruebas.

- **Encargados y otros actores**

Incluye a todos los roles mencionados anteriormente en las categorías de Analistas, Desarrolladores y Testers, reflejando su importancia en la gestión y operación de proyectos.

- Artistas gráficos.
- Administradores de sistemas.
- Especialista en herramientas.
- Stakeholders.
- Cualquier otro rol no especificado anteriormente.

Existen también metodologías catalogadas dentro de los marcos de trabajo tradicionales. Sin embargo, el propósito de este texto es resaltar las características principales de las más destacadas en la industria del software. Para aquellos interesados en profundizar sobre las ventajas y desventajas de estas y otras metodologías, se recomienda revisar la sección de materiales complementarios.

3. Marcos de trabajo ágiles

Los marcos de trabajo ágiles, también conocidos como metodologías ágiles para el desarrollo de software, surgen como alternativa para gestionar proyectos en los que no es posible definir completamente los requerimientos y sus estimaciones en la fase inicial, o cuando se requiere adaptabilidad a lo largo del desarrollo del software. (Maida y Pacienza, 2015).

Estas metodologías ofrecen un conjunto de pautas y principios diseñados para facilitar y dar prioridad a la entrega de productos por encima de los procesos de documentación exhaustiva, simplificándolos y promoviendo la participación activa del cliente desde las primeras etapas del proyecto.

El origen de las metodologías ágiles se remonta al año 2001 con la publicación del Manifiesto Ágil para el Desarrollo de Software, que establece cuatro valores fundamentales. (Manifiesto Ágil, 2001):

- Individuos e interacciones sobre procesos y herramientas.
- Software funcionando sobre documentación extensiva.
- Colaboración con el cliente sobre negociación contractual.
- Respuesta ante el cambio sobre seguir un plan.

Adicionalmente a los valores ágiles anteriormente listados, el manifiesto ágil establece 12 principios ágiles para materializar los valores definidos (Manifiesto Ágil, 2001):

- a) Nuestra principal prioridad es satisfacer al cliente a través de la entrega temprana y continua de software de valor.

- b) Son bienvenidos los requisitos cambiantes, incluso si llegan tarde al desarrollo. Los procesos ágiles se doblegan al cambio como ventaja competitiva para el cliente.
- c) Entregar con frecuencia software que funcione, en periodos de un par de semanas hasta un par de meses, con preferencia en los períodos breves.
- d) Las personas del negocio y los desarrolladores deben trabajar juntos de forma cotidiana a través del proyecto.
- e) Construcción de proyectos en torno a individuos motivados, dándoles la oportunidad y el respaldo que necesitan y procurándoles confianza para que realicen la tarea.
- f) La forma más eficiente y efectiva de comunicar información de ida y vuelta dentro de un equipo de desarrollo es mediante la conversación cara a cara.
- g) El software que funciona es la principal medida del progreso.
- h) Los procesos ágiles promueven el desarrollo sostenido. Los patrocinadores, desarrolladores y usuarios deben mantener un ritmo constante de forma indefinida.
- i) La atención continua a la excelencia técnica enaltece la agilidad.
- j) La simplicidad como arte de maximizar la cantidad de trabajo que se hace es esencial.
- k) Las mejores arquitecturas, requisitos y diseños emergen de equipos que se autoorganizan.
- l) En intervalos regulares, el equipo reflexiona sobre la forma de ser más efectivo y ajusta su conducta en consecuencia.

3.1. Programación Extrema - XP

XP, abreviatura de Extreme Programming, es un marco de desarrollo de software ágil diseñado para producir software de alta calidad en entornos con requisitos muy cambiantes. Se enfoca en contextos que presentan riesgos asociados a plazos fijos, la implementación de tecnologías emergentes y equipos de trabajo reducidos localizados en el mismo lugar.

En XP se definen cinco valores según Beck y Andrés (2004):

- **Comunicación**

El desarrollo de software depende del trabajo en equipo, haciendo crucial la transferencia de conocimientos y el uso de medios de comunicación adecuados. Se sugiere la discusión cara a cara con herramientas que faciliten dibujar o escribir, como los tableros.

- **Simplicidad**

Se refiere a realizar solo lo estrictamente necesario, evitando desperdicios. Las tareas deben ser ejecutadas de manera que resulten fáciles de entender para otros, concentrándose únicamente en los requisitos conocidos.

- **Retroalimentación**

Permite identificar áreas de mejora y la revisión constante de las prácticas actuales, de modo que se puedan establecer procesos de mejora continua.

- **Coraje**

Implica enfrentarse a situaciones desafiantes para el equipo, como problemas organizacionales o intentar implementar funcionalidades de

formas innovadoras cuando las convencionales no funcionan, y aceptar comentarios constructivos.

- **Respeto**

Es fundamental que los miembros del equipo se respeten mutuamente para facilitar una comunicación efectiva y un trabajo en equipo armónico.

Cada integrante contribuye significativamente al logro del objetivo común.

Además de sus valores, XP se distingue por establecer un conjunto de 12 prácticas de desarrollo de software. Aunque estas prácticas pueden adoptarse de manera independiente, su importancia se magnifica cuando se implementan conjuntamente (Jeffries, 2011).

- **El juego de la planificación**

Consiste en una reunión de planificación donde el equipo de desarrollo y el cliente discuten y priorizan las características a desarrollar, promoviendo una comprensión mutua de los requisitos.

- **Pequeños lanzamientos**

Se recomienda realizar iteraciones cortas con funcionalidades reducidas para facilitar la prueba con el cliente, permitiendo obtener retroalimentación temprana y frecuente.

- **Metáfora**

Esta práctica enfatiza la importancia de utilizar nombres coherentes y comprensibles para cualquier identificador en el sistema, facilitando así la comprensión global del proyecto.

- **Diseño simple**

Implica desarrollar un código que solamente haga lo que es necesario en la forma más sencilla posible, evitando duplicaciones y reduciendo el número de métodos y clases.

- **Pruebas**

Sugiere el empleo de técnicas de prueba constantes antes de proceder con la escritura del código, como por ejemplo el TDD (Desarrollo Dirigido por Pruebas).

- **Refactorización**

Se centra en la eliminación de elementos innecesarios, desacoplando componentes y reduciendo redundancias para mantener el código limpio, entendible y fácil de modificar.

Los roles fundamentales establecidos por este marco de trabajo ágil son los siguientes: cliente, programador, coach, tester y manager.

- Los clientes son los encargados del establecimiento de las prioridades del proyecto y las necesidades puntuales.
- Los programadores se encargan de transformar esos requerimientos en bloques de código funcional, son el centro del marco de Extreme Programming.
- Los testers se encargan de la aplicación de pruebas para garantizar la calidad de los productos y servicios desarrollados.
- El coach es una figura encargada de brindar asesoría a los miembros del equipo y son los que definen el rumbo del proyecto.

- El manager se encarga de la coordinación de actividades, del aseguramiento de los recursos requeridos para el proyecto y quien tiene la responsabilidad de la comunicación externa.

3.2. Desarrollo rápido de aplicaciones - RAD

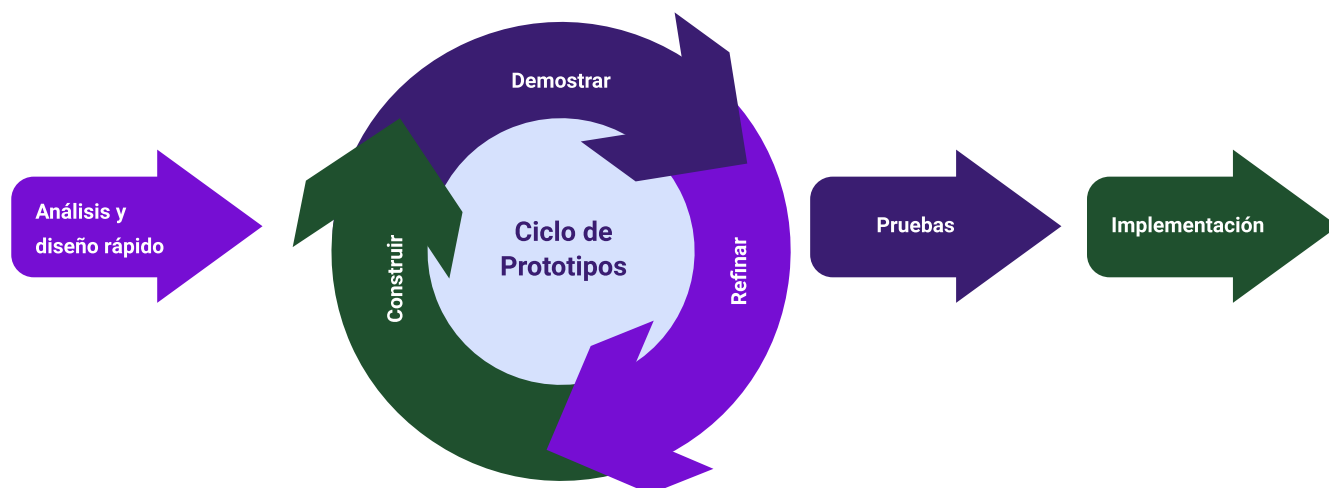
RAD (Desarrollo Rápido de Aplicaciones) es una metodología ágil de desarrollo de software que se enfoca en la rápida creación de aplicaciones a través de iteraciones frecuentes y retroalimentación constante. Fue propuesta por James Martin en 1991.

Algunas características fundamentales de RAD son:

- Mayor flexibilidad y adaptabilidad a cualquier ajuste que deba realizarse durante el proceso de desarrollo.
- Iteraciones rápidas que reducen el tiempo de desarrollo y mantienen un ritmo de entrega acelerado.
- Se fomenta la reutilización de código.
- Mejor gestión del riesgo, ya que las partes interesadas discuten y abordan cualquier vulnerabilidad mientras se construyen los productos.

A continuación, se describen las fases propuestas en RAD.

Figura 3. Fases definidas en RAD



Nota: Tomada de Blog Capterra (2019).

El proceso de desarrollo en cinco fases clave: desde la definición precisa de los requisitos, pasando por la creación y afinamiento de prototipos, hasta las pruebas rigurosas y el lanzamiento cuidadosamente preparado del producto final.

- **Primera fase**

Definición y finalización de requisitos:

- Definir los objetivos del proyecto.
- Establecer las expectativas.
- Acordar los plazos.
- Presupuestar el proyecto.

- **Segunda fase**

Construcción de prototipos:

- Construir prototipos.

- Validar y refinar prototipos con la retroalimentación de los usuarios.
- Aprobar los prototipos para pasar a la siguiente fase.
- **Tercera fase**
Transformación en modelos funcionales:
 - Desarrollar modelos funcionales a partir de los prototipos aprobados.
- **Cuarta fase**
Pruebas:
 - Realizar pruebas exhaustivas para asegurar que cada elemento funciona de manera individual y en conjunto.
- **Quinta fase**
Lanzamiento:
 - Ejecutar todas las actividades necesarias para el lanzamiento del producto, incluyendo la carga inicial de datos y la capacitación de usuarios.

Según HKSAR (2009), los roles más importantes en la metodología RAD incluyen: el facilitador, el escriba, el equipo SWAT, el administrador del modelo, el administrador de la base de datos, el equipo de planificación del taller, el equipo de diseño de usuario, el equipo de soporte de construcción y el equipo de transición. A continuación, se describen las características más destacadas de cada rol.

- **Facilitador**
Encargado de asegurar los objetivos y de los talleres de captura de requerimientos, prepara materiales, y actúa como mediador en la resolución de conflictos.

- **Escriba**

Responsable de documentar los resultados de los talleres de captura de requerimientos y registrar la información de todo el proceso.

- **Equipo SWAT**

Equipo encargado del diseño y construcción del sistema.

- **Administrador del modelo**

Coordinador del desarrollo y mantenimiento de las arquitecturas y modelos resultantes del proyecto.

- **Administrador de bases de datos**

Responsable del rendimiento, integridad y seguridad de la base de datos de la organización.

- **Equipo de planificación de los workshops**

Asisten en la definición de requerimientos y el alcance del sistema propuesto.

- **Equipo de diseño de usuario**

Proveen información detallada sobre las funciones del negocio y procesos afectados por el sistema propuesto.

- **Equipo de soporte de construcción**

Se aseguran de que las necesidades del usuario final se cumplan en el sistema completo, ofreciendo conocimiento detallado de las actividades del negocio y retroalimentación sobre la usabilidad del sistema al equipo SWAT.

- **Equipo de transición**

Desarrollan las tareas necesarias para preparar y llevar el sistema a entornos de producción.

3.3. Scrum

Scrum es un marco de trabajo ágil de muy amplio uso en la industria del software que se fundamenta en los valores y principios ágiles definidos en (Manifiesto Ágil, 2001) y donde se definen tres pilares fundamentales según (SCRUMstudy, 2013) los cuales se describen a continuación:

Transparencia

- Hace referencia a que cualquier proceso de Scrum puede ser conocido por cualquiera. Esto es posible por medio de eventos como:
- Las reuniones de revisión y reuniones diarias.
- Artefactos como la pila de producto.
- Cronogramas de lanzamiento.
- Documentos de visión del proyecto.
- Instrumentos de seguimiento, como: el burndown chart o el tablero de Scrum (Scrum board).

Inspección

- Permite que cualquiera pueda estar enterado de las actividades realizadas por otros y en general conocer el estado actual de los procesos.

Adaptación

- Por medio de la transparencia y la inspección es posible fijar actividades de mejoras que permitan modificar todo tipo de proceso en pro de lograr más altos estándares de calidad.

Adicionalmente, este marco de trabajo ágil está estructurado por un conjunto de roles, eventos y artefactos que pueden ser presentados a continuación:

Figura 4. Marco de trabajo Scrum



Nota. Adaptada de ITMadrid Digital School.

Dentro de los roles hay una división en dos categorías fundamentales:

Los roles centrales

- Hacen referencia a los requeridos obligatoriamente para la creación de un producto están altamente comprometidos y de los cuales depende el éxito o no de un proyecto.

Los roles no centrales

- Refieren a todo el personal interesado en el proyecto pueden interactuar con el equipo, pero no son los responsables del éxito del mismo, dentro de esta categoría entran los stakeholders, directivos, gerentes, marketing, asesores, etc.

Por otro lado, hay tres roles centrales dentro del marco de trabajo de Scrum (SCRUMstudy, 2013) que se describen a continuación:

- **Dueño del producto (Product Owner)**

Persona con amplio conocimiento en el negocio del cliente, sus necesidades y las tendencias del mercado para el área específica.

Encargado de maximizar el valor de negocio entregado al cliente y es el único responsable del control del Product Backlog (requerimientos) y su priorización. Representa al cliente en algunos procesos de demostración de avances y determina cuándo aprobar o no una entrega.

- **Scrum Master**

Rol que se encarga de facilitar los procesos al interior del equipo de trabajo removiendo cualquier impedimento y apoyando procesos de empoderamiento personal. Debe velar porque los elementos propios del marco de trabajo Scrum se apliquen de manera correcta.

- **Equipo de desarrollo (Developer Team)**

Responsables de la transformación de los requerimientos en código ejecutable a ser usado por el cliente. También responsables de la planificación de las iteraciones, establecimiento de características para tener en cuenta en la verificación de un requerimiento terminado y presentación de avances a los clientes. Generalmente es un equipo autoorganizado y autogestionado.

Además de los roles, scrum define un conjunto de eventos con participantes y objetivos claros que se desarrollan en momentos particulares del flujo general de scrum, a continuación, se detalla cada uno de estos:

- **Sprint**

Es el corazón de scrum, una iteración acotada entre 2 y 4 semanas donde se realiza un ciclo completo de actividades de análisis, diseño, construcción y pruebas para desarrollar una versión del producto potencialmente.

- **Planeación del sprint**

Reunión realizada antes del inicio del sprint donde se definen los requerimientos a desarrollar, se detallan en tareas concretas, se estima el tiempo/esfuerzo y se distribuyen las responsabilidades.

- **Reunión diaria (Daily Meeting)**

Reunión corta al inicio de cada día donde el equipo informa sobre su trabajo previo, las actividades del día y los problemas encontrados. Debe ser breve y alinearse con los pilares de transparencia e inspección.

- **Revisión del sprint**

Reunión al final del sprint donde el equipo de desarrollo muestra los resultados. Para sprints de 4 semanas, se usan 4 horas para la revisión.

- **Reunión de retrospectiva**

Última reunión después de la revisión del sprint para autoevaluar el desempeño del equipo durante el sprint, identificando y documentando aprendizajes y respondiendo a preguntas clave sobre lo que se debe continuar, detener o mejorar.

Finalmente, el marco de trabajo scrum define un conjunto de artefactos que permiten registrar y gestionar información clave para asegurar los tres pilares fundamentales y proveen información valiosa durante todo el proceso de desarrollo de software. Entre los artefactos representativos de este se encuentran los siguientes:

- **Pila de producto (Product Backlog):**

Lista priorizada de requerimientos generalmente descritos en formato de historias de usuarios que representa todas las características del sistema a construir.

- **Pila del sprint (Sprint Backlog)**

Lista de requerimientos seleccionados desde el product backlog por el equipo de trabajo para ser desarrollados durante un sprint particular. este es uno de los artefactos generados en la reunión de planeación del sprint.

- **Burndown Chart**

Es un gráfico visual de dos ejes que muestra a los equipos la cantidad de trabajo pendiente por completar (eje Y) y el tiempo disponible para hacerlo (eje X). Este generalmente se realiza por cada sprint ubicando la

cantidad de trabajo a realizar del sprint backlog (usualmente medido por puntos de historia u de horas de trabajo) en un tiempo 0 y por cada día finalizado se resta la cantidad de puntos de historia u horas de cada tarea completada.

Por otro lado, también es posible usar este mismo gráfico para representar el avance general del proyecto, ubicando en el eje Y la cantidad total de horas o esfuerzo del Product Backlog y en el eje X el número de Sprints proyectados. Cada punto representa el total acumulado hasta ese Sprint y se une mediante una línea, lo que permite determinar visualmente si el flujo de trabajo está en una situación óptima o no respecto al tiempo restante para completar el Sprint actual.

Figura 5. Burndown Chart



Nota. Tomada de intl-blog.imgix.net (2019).

Tablero de Scrum (Scrumboard): es un elemento visual donde se integra la mayor parte de los elementos del marco de trabajo Scrum, en él se indica la carga de trabajo, el estado actual de cada una de las actividades y sus respectivos responsables. Este es un elemento que se sincroniza de manera permanente y facilita la implementación de los pilares de transparencia, inspección y adaptabilidad. Si bien se aconseja el uso de un tablero, existen diferentes tipos de herramientas digitales que permiten la implementación de un tablero de Scrum.

Figura 6. Tablero de Scrum



Adicionalmente es de vital importancia mencionar que entre los principales beneficios del marco de trabajo Scrum se encuentran los siguientes elementos:

- a) Es posible gestionar las expectativas del cliente de manera regular, ya que, este puede y debe participar en las reuniones de revisión, por lo que, está enterado todo el tiempo del estado actual del proyecto.
- b) El cliente puede obtener resultados importantes y utilizables desde las primeras iteraciones, ya que, la lista de producto está priorizada para ofrecer mayor valor en el menor tiempo posible y porque cada finalización de Sprint debe tener como resultado una versión totalmente funcional.
- c) El proyecto puede iniciar con requerimientos de muy alto nivel y es fácil administrar los cambios.
- d) La participación constante del cliente permite mitigar riesgos del proyecto desde sus primeras etapas.
- e) Los procesos de retrospectiva permiten establecer actividades permanentes de mejora continua en función de las experiencias del equipo.

Un elemento clave en su formación es la planeación de proyectos de software que aunado con los contenidos ya vistos le permite tener un panorama amplio sobre el tema.

4. Planeación de proyectos de software

La planificación de proyectos de software comprende varias actividades destinadas a obtener una visión preliminar del software a desarrollar, lo que permite realizar estimaciones para determinar la viabilidad del proyecto en función de los recursos asignados.

De acuerdo con SCRUMstudy (2013), las actividades de planificación estipuladas en el marco de trabajo de Scrum se detallan en el siguiente recurso de aprendizaje:

Video 1. Planeación de proyectos de software



[Enlace de reproducción del video](#)

Síntesis del video: Planeación de proyectos de software

A continuación, se presentan los pasos que debe seguir para la planeación de proyectos de software. Creación de historias de usuario: En este proceso, que es responsabilidad del product owner, se crean las historias de usuarios con sus respectivos criterios de aceptación de forma que queden representados los requerimientos del sistema y sean comprendidos por los clientes y stakeholders. Estimación de historias de usuario: Este es un proceso realizado por el equipo de desarrollo, donde se valora el esfuerzo necesario para la realización de cada historia de usuario. Generalmente, hay un proceso previo en el cual el product owner explica y presenta cada una de las historias de usuario y aclara cualquier duda que surja. Así, cuando se realiza el proceso de estimación, todos son conscientes de las actividades que implican cada historia de usuario.

Comprometer historias de usuario: En este proceso, el equipo de desarrollo identifica las historias de usuario a ser desarrolladas en el sprint. Esto se hace mediante la reunión de planificación del sprint. Identificar tareas: En este proceso, cada una de las historias de usuario se desagrega más detalladamente por medio de tareas. Estimar tareas: En este proceso, el equipo de desarrollo realiza una estimación de esfuerzo para la realización de la tarea de forma que es más sencillo asignar responsabilidades puntuales de acuerdo con las diferentes cargas del equipo.

Crear el Sprint Backlog: El equipo de trabajo agrupa cada una de las tareas de las historias de usuario comprometidas en un artefacto llamado Sprint Backlog o pila del Sprint. Además, existen diferentes técnicas para la estimación de historias de usuario. Entre las más destacadas se encuentran las siguientes: Wideband Delphi,

esta es una técnica de estimación basada en consenso para determinar la cantidad de trabajo necesario y el tiempo requerido para el mismo. En esta, cada individuo realiza estimaciones individuales de las historias y luego son socializadas al grupo para analizar los factores que influyen en cada estimación. Posterior a esto, se realiza una nueva ronda de estimación individual, siguiendo el mismo procedimiento hasta que los valores de estimaciones se acercan lo suficiente o se logre un consenso.

Planning Poker es una técnica derivada de Wideband Delphi pero para la realización de las estimaciones se apoya en un conjunto de cartas especiales que vienen marcadas con números semejantes a la serie Fibonacci. Para el proceso de estimación, se lee una historia de usuario, cada miembro del equipo selecciona una de las cartas y, cuando se indica, todos al tiempo muestran sus cartas. Por otro lado, en el caso de haber diversidad muy representativa en los valores, se realiza una ronda donde los puntajes más altos y más bajos justifican sus decisiones. Después, se hace una segunda ronda de estimación. Luego, la estimación de cada historia se puede determinar por medio de consenso, democracia o promedio, según se haya definido en el grupo de trabajo.

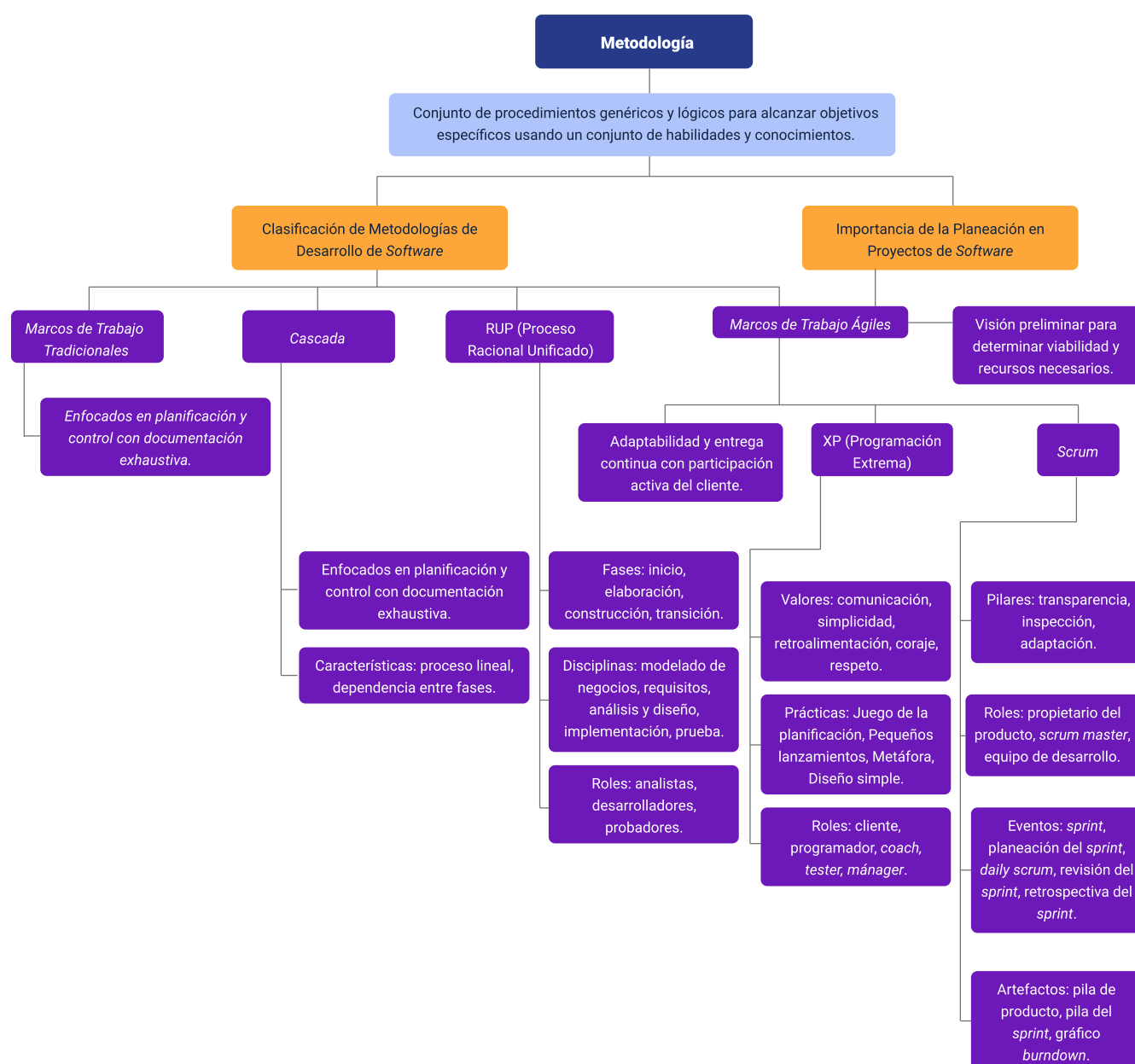
Puño de cinco: En esta técnica, se usan los dedos de la mano para expresar la estimación de cada miembro del equipo. Al igual que en el Planning Poker, se parte de una explicación inicial de la historia y luego se procede a realizar la votación. En caso de presentarse valores atípicos, se realiza una sesión corta donde se justifican los valores asignados para, posteriormente, realizar una nueva votación.

Como se puede ver, la planeación de proyectos de desarrollo del software requiere diseñar estrategias activas de participación de todos y cada uno de los

miembros del equipo para alcanzar los objetivos y los requerimientos del cliente en un plazo predefinido y acordado con el mismo.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
¿Qué son las metodologías tradicionales en el desarrollo de software?	Henao, C. (2018). #1. ¿Qué son las metodologías tradicionales en el desarrollo de software?	Video	https://youtu.be/i8CPD1dW88k?si=n_wD0LM5WHg59CoO
¿Qué son las metodologías ágiles en el desarrollo de Software?	Henao, C. (2018b). #2. ¿Qué son las metodologías ágiles en el desarrollo de Software?	Video	https://youtu.be/fHKsufzM7qQ?si=9HPATMJdNFGJeYHW
SCRUM en 6 minutos Metodologías Ágiles	Henao, C. (2018d). #3. SCRUM en 6 minutos Metodologías Ágiles.	Video	https://youtu.be/HhC75lonpOU?si=9zcH8SvcdPyHmgwJ

Glosario

Iteración: hace referencia a un ciclo limitado por tiempo donde se ejecutan actividades de análisis, diseño, construcción y pruebas.

Product Owner: rol central de Scrum encargado de la gestión de la pila de producto y representante del cliente dentro del grupo de trabajo.

Scrum Master: rol central de Scrum encargado de facilitar el trabajo del equipo de desarrollo y de mantener la aplicación del marco de trabajo SCRUM.

Stakeholders: persona, organización o empresa interesada en el desarrollo del proyecto.

Referencias bibliográficas

Agilemanifesto.org. (2021). Manifiesto por el desarrollo ágil de software.

Agilemanifesto.org. <https://agilemanifesto.org/iso/es/manifesto.html>

Beck, K., & Andrés, C. (2004b). Extreme Programming Explained: Embrace Change, 2nd Edition (The XP Series). Addison-Wesley.

Intl-blog.imgix.net. (2019). Burndown-chart.

Jeffries, R. (2011). What is Extreme Programming? Ronjeffries.com.

<https://ronjeffries.com/xprog/what-is-extreme-programming/>

Kruchten, P. (2003). The Rational Unified Process: An Introduction. Addison-Wesley Professional.

Maida, E, G., Pacienza, J. (2015). Metodologías de desarrollo de software [en línea]. Tesis de Licenciatura en Sistemas y Computación. Facultad de Química e Ingeniería Fray Rogelio Bacon. Universidad Católica Argentina.

<https://repositorio.uca.edu.ar/bitstream/123456789/522/1/metodologias-desarrollo-software.pdf>

Man, M., Hafriz, M., Nural, A., Mohd, H., Maizura, N., Noor, M., Wan, W., Bakar, A., & Man, M. (2008). eWorks: Development of a Web Based Site Assessment Software for Construction Progress Project. Communications of the IBIMA, (5), 93-99.

Martin, J. (1991). Rapid Application Development. Macmillan Coll. Pngwing.com

(s.f.) Marco de trabajo Scrum. <https://www.pngwing.com/es/free-png-xqgijv>

Royce, W. W. (1970). Managing the Development of Large Software Systems. Proceedings of IEEE WESCON, 26, 328-388.

SCRUMstudy. (2013). A Guide to the Scrum Body of Knowledge (SBOK Guide).
VMedu Inc.

Sommerville, I., Galipienso, M. I. A., y Martínez, A. B. (2005). Ingeniería del
software. Pearson Educación.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suárez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Especificaciones y gestión de proyectos de software

Breve descripción:

Este componente se enfoca en la ingeniería de requisitos, la cual es esencial para definir las necesidades de software. Los aprendices aprenderán a formular propuestas técnicas para servicios de software, tomando en cuenta elementos clave como los conceptos, la estructura, los términos, la presentación, las normativas, los costos y la validación, todos ellos cruciales para las empresas y organizaciones.

Junio 2024

Tabla de contenido

Introducción	4
1. Teoría de sistemas	5
Tecnología e innovación	6
Clasificación de tecnología	8
1.1. Diagnósticos	9
1.2. Soluciones	10
2. Licencias de software	12
2.1. Normativa.....	14
2.2. Tipos de licencia	15
Licencia de Software Freeware.....	16
Licencias de software libre o software de código abierto.....	16
Licencias de software shareware	17
Licencias de software propietario	18
2.3. Derechos de autor	18
3. Fichas técnicas	20
Elaboración de la ficha técnica	20
Diligenciamiento.....	20
4. Generalidades de la propuesta técnica.....	24

5.	Costos	28
6.	La propuesta técnica.....	29
	Estructura	29
	Términos de Referencia (TdR)	29
	Elaboración de la propuesta técnica.....	31
6.1.	Validación de la propuesta técnica.....	34
	Aspectos de la contratación por licitación.....	36
6.2.	Normatividad	38
	Acuerdos.....	38
	Síntesis	40
	Material complementario	41
	Glosario	43
	Referencias bibliográficas	45
	Créditos	47

Introducción

La importancia de llevar a cabo los procesos en la ingeniería de software de manera flexible es fundamental para permitir que el equipo de desarrolladores realice su trabajo de forma efectiva. Debido a la diversidad de tipos de software, no existe un proceso universal. En cambio, todo proceso debe incorporar las cuatro actividades principales de la ingeniería de software: especificación, desarrollo, validación y evolución del software. Además, debe contar con:

- **Propuesta de software**

Una propuesta adecuada es esencial al ofrecer nuestros servicios, dado que el éxito del proyecto depende en gran medida de la calidad técnica de la misma. La oferta de servicios debe convencer al cliente de que la solución propuesta es la más adecuada para satisfacer las necesidades específicas de su empresa.

- **Documento técnico**

La propuesta técnica debe cumplir con las normativas aplicables y proporcionar información detallada sobre la solución de software propuesta para resolver un problema de negocio. Este documento puede variar según la naturaleza de la empresa solicitante, ya sea privada, gubernamental, una fundación o una ONG.

- **Adaptación y claridad en las propuestas**

Es fundamental adaptar la propuesta a los formatos requeridos por las empresas. Esta debe ser clara, concreta, explícita y fácil de entender, asegurando así que se transmitan los puntos clave de manera efectiva.

1. Teoría de sistemas

Es el estudio de las características que tienen en común los sistemas; su aplicación se enfoca en los sistemas automatizados. Los siguientes son algunos de los principios generales de la Teoría General de Sistemas (TGS):

- Entre más especializado sea un sistema, tiene menos capacidad de adaptarse a circunstancias diferentes.
- Cuanto mayor sea el sistema, mayor es el número de recursos que deben dedicarse a su mantenimiento diario.
- Los sistemas siempre forman parte de sistemas mayores y siempre pueden dividirse en sistemas menores.
- Los sistemas crecen.

La Teoría General de Sistemas (TGS) se presenta como un ejemplo de representación científica. En sus selecciones conceptuales, no hay definiciones o relaciones con contenidos preestablecidos, pero la observación sí se puede dirigir con referencia a ellas, haciéndola aplicar en contextos identificables. (Arnold, 1989).

Los objetivos originales de la Teoría General de Sistemas son los siguientes:

- Impulsar el desarrollo de una terminología general que permita describir las características, funciones y comportamientos sistémicos.
- Desarrollar un conjunto de leyes aplicables a todos estos comportamientos.
- Promover una formalización (matemática) de estas leyes.

Conceptos Teoría General de Sistemas

Lo invitamos a consultar el PDF **“Conceptos”**, donde se describen los principales conceptos de la Teoría General de Sistemas, según Arnold (1989) y otros autores, el cual se encuentra en la carpeta Anexos.

Tecnología e innovación

Para definir tecnología e innovación, se presentan los conceptos de varios autores:

- Pavón y Goodman (1976) definen la innovación como un conjunto de actividades localizadas en un tiempo y lugar específicos, que conducen a la introducción exitosa en el mercado de una idea por primera vez. Esta idea generalmente se asocia con la creación de productos o servicios nuevos y de calidad, así como con la gestión y organización.
- Coque (2002) describe la innovación tecnológica como algo atribuible al mercado y que se aplica de forma permanente. Su principal objetivo es contribuir al mejoramiento del desarrollo del sector productivo de bienes y servicios, además de incrementar la calidad de vida mediante el abastecimiento de productos excelentes a los consumidores.
- Oberto (2007) sostiene que solo se puede considerar "innovación" a aquellas ideas que se utilizan para satisfacer una necesidad concreta.

Desde otra perspectiva, la innovación tecnológica en las empresas se caracteriza por tener especificidades y limitaciones relacionadas con el origen natural de su fabricación y las características de su consumo. Además, la innovación trasciende la investigación y el desarrollo, abarcando las fases científicas, técnicas, comerciales y

financieras necesarias para el éxito de productos nuevos o mejorados, o la introducción de un nuevo servicio. En el siguiente video, se presenta el concepto:

Video 1. Concepto de tecnología



[Enlace de reproducción del video](#)

Síntesis del video: Concepto de tecnología

Un concepto inicial dice que la tecnología es: el resultado de la evolución de la técnica, mediante la aplicación de métodos sistemáticos desarrollados a partir del conocimiento científico. Pérez y Fernández, 2003, página 9. De manera general, la tecnología se entiende como la capacidad de desarrollo de artefactos, instrucciones y reglas que permiten la interacción entre los hombres y el entorno en el que se

realizan diferentes actividades, para asegurar su persistencia y adaptación frente a situaciones adversas.

Clasificación de tecnología

La clasificación de la tecnología está compuesta por diferentes formas y tecnologías que se pueden encontrar en una organización, y que se agrupan según sus atributos.

- Naturaleza.
- Forma de obtención.
- Aplicación.
- Grado de desarrollo.
- Injerencia.
- Pertinencia.

El concepto de gestión de la tecnología se entiende como el proceso de administración, compra, aplicación y propagación de esta, en diferentes sectores, entre ellos:

- Industrial
- Servicios
- Público y privado

Comprende, desde luego, la administración del proceso de innovación, investigación y desarrollo (I+D). Se constituye como la mejor forma de integrar y utilizar la tecnología para facilitar el desarrollo de productos, así como para definir y mejorar procesos de índole industrial.

Uno de los objetivos de la gestión de la tecnología es evitar prolongar innecesariamente las inversiones, optimizar la productividad, aumentar la calidad, proponer nuevos productos y servicios al mercado, así como promover el desarrollo de nuevo conocimiento. Esta gestión se establece como la forma más efectiva de administrar el conocimiento para solucionar diversos problemas sociales, humanos y ambientales.

Según Gaynor (1999), existe una diferencia entre los conceptos de gestión y administración. La administración implica supervisar las principales tareas asignadas para mantener una organización en funcionamiento, mientras que la gestión incorpora grados de creatividad, liderazgo, preocupación y riesgo por el desempeño futuro. (p. 5)

Es importante aclarar que, aunque la mayoría de las empresas administran sus recursos tecnológicos, no todas logran gestionarlos adecuadamente, principalmente porque no fomentan constantemente la innovación.

1.1. Diagnósticos

El diagnóstico tecnológico se define como una actividad de observación colaborativa entre la empresa y expertos (tanto internos como externos), implementada mediante una metodología previamente establecida. Esta metodología busca definir tanto las necesidades como el potencial tecnológico de la empresa, resultando en la identificación de sus fortalezas y debilidades. Se analizan aspectos tanto de orden interno como externo.

- **Internamente**

Se pretende inventariar y evaluar los recursos tecnológicos con los que cuenta la empresa.

- **Externamente**

Se examina lo que hacen los competidores o la brecha que los separa de los líderes del sector.

Un diagnóstico facilita la identificación, determinación y evaluación del potencial tecnológico que posee una empresa dentro del ambiente en el que opera, como parte de una estrategia global.

Como objetivo principal, el diagnóstico tecnológico identifica diversas capacidades de la empresa para un desarrollo eficiente de innovaciones. Funciona esencialmente como una herramienta que distingue entre las capacidades que son necesarias y las que actualmente posee la empresa (brechas).

Matriz de evaluación de las capacidades tecnológicas

Lo invitamos a consultar el PDF “**Matriz de evaluación de las capacidades tecnológicas**”, para evaluar una empresa, el cual se encuentra en la carpeta Anexos.

1.2. Soluciones

Uno de los grandes desafíos del proceso de innovación es el abordaje unidireccional de los problemas dentro de un sistema. Una manera de mejorar esta situación y adoptar una perspectiva más amplia, que ofrezca diversas opciones para potenciar soluciones, es utilizar una técnica desarrollada por Alex F. Osborn, la cual ha sido además enriquecida por varios autores. Esta técnica consiste en plantear una serie de preguntas que exploran múltiples posibilidades, estimulando así las capacidades creativas de las personas.

De esta forma, las soluciones creativas tienen como objetivo investigar alternativas para resolver un problema mediante la aplicación metodológica de un

conjunto de preguntas que promueven la consideración de opciones distintas a las ya existentes. Como se presenta a continuación:

a) Describir la situación

- Exponer la situación que se abordará.
- Presentar el contexto de la situación actual.

b) Definir el problema

- Identificar el problema a tratar.
- Formular el problema identificado.

c) Analizar el problema

- ¿Qué se puede sustituir?
- ¿Qué se puede combinar?
- ¿Qué se puede adaptar?
- ¿Qué se puede modificar?
- ¿Qué se puede intercambiar?
- ¿Qué se puede eliminar?
- ¿Qué se puede invertir?

d) Tomar decisiones

- Evaluar ideas de posibles soluciones.
- Análisis de ideas a partir de criterios definidos.
- Definir criterios de evaluación de posibles soluciones.
- Seleccionar la solución.

2. Licencias de software

Una licencia de software es un acuerdo esencial que regula el uso y distribución de obras intelectuales en la tecnología. Este contrato establece las condiciones clave para el uso de software, resaltando la importancia de los derechos de autor y la propiedad intelectual. En el siguiente video, se presenta una explicación:

Video 2. Licencias de software



[Enlace de reproducción del video](#)

Síntesis del video: Licencias de software

Un contrato de licencia, es aquel cuyo objetivo es permitir el uso, la explotación, y el aprovechamiento de una obra protegida; en este proceso actúan el licitante y el licenciataria, donde el primero es quien realiza la autorización y el

segundo es quien se beneficia de ella. En el evento en que se incluyan los derechos de autor y propiedad intelectual, la licencia de software se convierte en un contrato, donde participa el desarrollador de software, quien se somete a la propiedad intelectual y a derechos de autor por el usuario, en él se definen explícitamente deberes y derechos de ambas partes, quien elige la licencia es el desarrollador, quien cedió los derechos de explotación; en pocas palabras, consiste en una serie de permisos que un desarrollador otorga para la distribución, uso y/o modificación del desarrollo que construyó, a esta licencia se le pueden agregar datos como los plazos de duración, territorio, etcétera.

Algunos términos importantes son:

- **Licencia**

Este término se refiere a un acuerdo o contrato entre un desarrollador y un usuario, en el que el software está sujeto a propiedad intelectual y derechos de autor. La licencia detalla los derechos y obligaciones de ambas partes. La elección de la licencia de distribución del software depende del desarrollador.

- **Patente**

Consiste en un derecho exclusivo otorgado por el Estado para la protección de una invención, ya sea un producto material o inmaterial que se pueda explotar comercialmente. Este derecho se concede por un período de tiempo limitado y a conveniencia del solicitante.

Derecho de autor o copyright: es una forma de protección otorgada por las leyes vigentes en la mayoría de los países para los autores de obras originales. Esta

protección incluye obras literarias, dramáticas, musicales, artísticas e intelectuales, ya sean publicadas o inéditas.

Software libre: este tipo de software ofrece libertades específicas que incluyen:

- Ejecutar el programa para cualquier propósito.
- Estudiar y modificar el programa según las necesidades del usuario.
- Redistribuir copias del programa.
- Mejorar el programa y compartir las mejoras con la comunidad.

Software de fuente abierta: este software se caracteriza por criterios específicos de distribución:

- Libre distribución del software.
- Acceso al código fuente para su intervención.
- Permitir modificaciones y derivaciones a las mismas condiciones del software original.
- No discriminar a personas o grupos en su uso.
- La licencia no debe imponer restricciones tecnológicas.

Software de dominio público: se refiere al software que no está protegido por derechos de autor y, por tanto, está libre de restricciones de uso asociadas a tales derechos.

2.1. Normativa

Cuando se implementa un software, su uso debe regirse por una normativa que previene futuras sanciones o multas.

¿Cumple con la Ley 603?

En Colombia, la regulación de las licencias de software está a cargo de la Ley 603 de 2000, también conocida como Ley para el Cumplimiento de las Licencias de Software. Esta ley obliga a las empresas a presentar un informe de gestión que debe incluir detalles sobre el progreso de los negocios y su estado financiero, administrativo y jurídico. Dicho informe debe demostrar el cumplimiento de las normas relacionadas con la propiedad intelectual y los derechos de autor.

La DIAN (Dirección de Impuestos y Aduanas Nacionales) tiene la autoridad para realizar auditorías durante las visitas a las empresas, asegurando así el cumplimiento legal del software instalado en los equipos de cómputo.

Se enumeran a continuación las penalidades por incumplimiento de la Ley 603 de 2000:

- Para los administradores de la empresa, sanciones de hasta 200 SMMLV (Salarios Mínimos Mensuales Legales Vigentes).
- Delito de desfalco de derechos patrimoniales, punible con prisión de entre 2 y 5 años.
- Infracción a los elementos de protección de los derechos patrimoniales de autor, punible con prisión de 4 a 8 años.
- La DIAN también puede imponer sanciones por evasión de impuestos derivados del uso de software ilegal.

2.2. Tipos de licencia

Exploramos a continuación los distintos tipos de licencias de software y sus características principales:

Licencia de Software Freeware

Las licencias freeware permiten descargar, instalar y/o usar gratuitamente un programa de computadora. Con este tipo de licencia, no es necesario que el usuario se suscriba, se registre o compre alguna licencia. Estas licencias permiten el acceso al código objeto, pero no al código fuente.

Ejemplo: el antivirus Avast, que se puede descargar gratuitamente de Internet.

Licencias de software libre o software de código abierto

Este tipo de licencia se caracteriza porque los desarrolladores generalmente ofrecen acceso abierto al código fuente de su software. Existen diversas licencias creadas por organizaciones, entre las que destaca la GNU GPL (Licencia Pública General de GNU). Esta licencia, creada por la Fundación de Software Libre (Free Software Foundation), es ampliamente utilizada en el mundo del software libre y código abierto. Garantiza a los usuarios finales, ya sean personas, organizaciones o empresas, la libertad de utilizar, estudiar, compartir (copiar) y modificar el software.

Ejemplos: Android, Mozilla, Linux, WordPress y Chromium.

Este tipo de licencias se subdivide en otras que contienen cláusulas más específicas, como el copyleft.

- **Copyleft**

Es una versión del copyright diseñada para regular las licencias de software libre. Impide que el material sea distribuido o reformado por alguien no autorizado y establece condiciones específicas de uso del código.

- **Licencia de software libre sin protección copyleft**

Permite a terceros modificar el producto y licenciarlo bajo sus propios términos, sin imponer restricciones sobre cómo se puede redistribuir el software modificado.

- **Licencia de software libre con protección copyleft**

Obliga a los programadores que redistribuyan su código a adherirse a ciertos lineamientos y restricciones, asegurando que las modificaciones permanezcan bajo las mismas condiciones de licencia del original.

Licencias de software shareware

Este tipo de licencia permite la descarga gratuita de una versión del software con funcionalidades limitadas. Tras un período determinado, para continuar utilizando todas las funcionalidades, el usuario debe realizar un pago. Estas licencias no permiten modificaciones, ya que no incluyen el código fuente. Se subdividen en tres categorías: trial, demo y adware.

- **Trial**

Licencia que activa todas las funcionalidades del software, pero solo por un periodo de tiempo limitado.

- **Demo**

Permite el uso del software de forma indefinida, pero con la limitante de no tener habilitadas todas sus funciones. Comúnmente utilizado en videojuegos.

- **Adware**

Licencia que permite utilizar los programas de forma gratuita; sin embargo, a cambio, los programas descargan y muestran publicidad de forma continua.

Licencias de software propietario

Esta licencia implica que el autor del software restringe los derechos de copia, redistribución y modificación de su obra. En consecuencia, si un usuario final desea realizar alguna de estas acciones, deberá pagar una cantidad de dinero al autor para obtener el derecho legal de hacerlo.

2.3. Derechos de autor

Se refiere a la protección que se otorga a todas las obras artísticas, literarias o científicas por el derecho de autor durante un período determinado, a partir del momento de su creación. En Colombia, la Dirección Nacional de Derechos de Autor (DNDA) es la entidad encargada de proteger estas obras.

Cuando el software está protegido por derechos de autor, es necesario establecer claramente en la licencia los usos que serán permitidos al usuario. Comúnmente, los usos autorizados incluyen:

- La ejecución del código del software, ya sea en dispositivos de almacenamiento o en el disco local del equipo donde se instalará el software.
- Es fundamental que la licencia especifique el plazo y el territorio de validez de la concesión.

- La posibilidad de distribuir el software, 10 que incluye realizar nuevas copias y comercializarlas a terceros.

Finalmente, es importante tener en cuenta que los usuarios solo podrán hacer uso del software para los fines expresamente autorizados en la licencia.

3. Fichas técnicas

Una ficha técnica es un documento que detalla las características o funciones de un software desarrollado, funcionando como una herramienta esencial para transmitir información relevante sobre un tema específico. Este documento contiene datos útiles y concisos, cuyo contenido varía según la finalidad y el tipo de ficha.

La ficha técnica es especialmente útil para comercializar o promocionar un producto o servicio. Es crucial que contenga información veraz, ofreciendo datos claros y precisos sobre las características de los productos de software desarrollados.

Elaboración de la ficha técnica

Para la elaboración de una ficha técnica, es recomendable apoyarse en normativas establecidas. En este caso, la ficha técnica se elaborará tomando como referencia la norma NTC ISO 9000, Sistema de Gestión de Calidad.

Generalidades sobre ISO-9000

Para obtener más información sobre la norma ISO, incluyendo sus objetivos, finalidades y terminología, puede consultar el siguiente sitio web.

<https://www.iso.org/obp/ui/es/#iso:std:iso:9000:ed-4:v1:es>

Diligenciamiento

A continuación, se realiza la descripción para el diligenciamiento de un ejemplo de una ficha técnica de productos de software.

Tabla 1. Ejemplo ficha técnica

Registro del formato de ficha técnica	Responsable (coordinador encargado)
Para realizar el diligenciamiento de la ficha técnica, el coordinador técnico de desarrollo se apoya en el coordinador de base de datos y el coordinador de infraestructura. Se deben llenar los campos de acuerdo con:	
CARACTERÍSTICAS DEL PRODUCTO Nombre del producto: nombre con el que será identificado el producto. Línea de producción: a qué sistema pertenece. Versiones anteriores: número y nombre del último desarrollo del producto. Versión actual: número con el que se identificará el nuevo producto. Módulo: nombre técnico del módulo.	
DESCRIPCIÓN DEL PRODUCTO Descripción general del producto: descripción breve del nombre del producto. Objetivo: definir el propósito principal del producto, especificando el problema que resuelve o la necesidad que satisface.	
ARQUITECTURA Descripción: explica el tipo de arquitectura de desarrollo del aplicativo.	
REQUERIMIENTOS DEL PRODUCTO Requisitos del sistema (servidor) Hardware: se describen los requerimientos físicos mínimos que debe tener el equipo.	

Registro del formato de ficha técnica	Responsable (coordinador encargado)
Software: se describen los requerimientos lógicos necesarios para la ejecución del aplicativo. Otros: recomendaciones para que el aplicativo funcione mejor. Requisitos del sistema (cliente) Hardware: se describen los requerimientos físicos mínimos que debe tener el equipo. Software: se describen los requerimientos lógicos necesarios para la ejecución del aplicativo. Otros: recomendaciones para que el aplicativo funcione mejor.	
REQUERIMIENTOS Requerimientos funcionales generales: definición de funciones que el sistema realizará. Requerimientos adicionales: funcionalidades adicionales para el correcto funcionamiento del sistema. Cientes del producto: características del cliente o usuarios finales que puedan usar el producto.	

Tabla 2. Historial de modificaciones

Versión	Naturaleza del cambio	Fecha de aprobación	Fecha validación
00	Actualización del documento	25/05/2021	25/06/2021
01	Actualización del documento	25/07/2021	25/08/2021

Tabla 3. Firmas

Elaboró	Aprobó	Validó
Firma	Firma	Firma
Fecha	Fecha	Fecha

4. Generalidades de la propuesta técnica

Para el diseño, construcción y presentación de una propuesta técnica de servicios de software, es esencial considerar aspectos y elementos importantes que definen su carácter y proporcionan seguridad y confianza tanto al oferente como al cliente.

Los aspectos clave para el éxito de una propuesta técnica son:

- **Características y necesidades del cliente**

Entender profundamente lo que el cliente necesita y espera.

- **Contratación**

Definir claramente los términos y condiciones de la contratación.

- **Costos y presupuestos**

Establecer un presupuesto detallado y realista.

- **Estructura lógica de la propuesta**

Organizar la propuesta de manera coherente y lógica.

- **Características del software**

Detallar las funcionalidades y especificaciones del software propuesto.

A continuación, se presentan las generalidades que debe conocer, comprender y examinar detenidamente.

- **Acercamiento**

Indagar sobre la capacidad económica de la empresa y el presupuesto disponible para el proyecto, con el fin de amoldarse a él o presentar una propuesta adecuada a sus recursos.

- **Consulta**

Ingresar a la página <https://www.rues.org.co/> para adquirir un certificado de la empresa, evitando tratar con intermediarios y obteniendo un concepto claro del tipo y tamaño de la empresa.

- **Contrato**

Documento legal en el que dos o más partes acuerdan cumplir condiciones, teniendo como contraprestación un fin económico.

- **Adición de contrato**

Adición que se puede hacer en tiempo y/o valor. Si es en valor, solo se puede adicionar hasta el 50 % del valor inicial del contrato y nunca exceder este tope.

- **Activos**

Conjunto de bienes y derechos con valor monetario que son propiedad de una empresa, institución o individuo.

- **Pasivos**

Conjunto de obligaciones pendientes, tales como deudas no satisfechas.

- **CDP (Certificado de Disponibilidad Presupuestal)**

Documento que garantiza o respalda la existencia de recursos para atender un gasto específico.

- **Monetización**

Proceso por medio del cual una determinada acción es convertida en dinero. Término usado, generalmente, en redes sociales y plataformas con streaming.

- **Gasto**

Consumo de un bien o servicio a cambio de una contraprestación.

- **Licitación**

Procedimiento formal por el cual un organismo público demanda obras, servicios, bienes, entre otros, al sector privado.

- **Freelance**

Trabajo o trabajador independiente.

Cuando se trabaja con clientes en proyectos, especialmente en áreas como el desarrollo de software, es esencial estructurar la interacción y comprensión de las necesidades y expectativas del cliente de manera organizada. A continuación, se presentan los aspectos clave para gestionar y entender eficazmente las relaciones y requisitos del cliente, facilitando así una comunicación efectiva y la toma de decisiones informadas.

- **Relación con el cliente**

Establecer una relación cordial para generar una conexión cercana y comprender lo que busca el cliente.

- **Identificar stakeholders**

Considerar las expectativas y necesidades de cada funcionario, que varían según su cargo y rol dentro de la empresa.

- **Identificar necesidades**

Solicitar información sobre interacciones de la compañía, modelo de negocios, procesos y procedimientos. Sugerir soluciones a problemas no detectados.

- **Conocer el presupuesto**

Solicitar detalles del presupuesto destinado al proyecto, adaptarse o negociar según las necesidades y recursos disponibles.

- **Planificación presupuestaria**

El presupuesto se presenta en términos financieros y debe cumplir metas específicas en un tiempo y condiciones pactadas.

5. Costos

El costo, también llamado coste, representa el valor económico que se incurre para producir un producto, prestar un servicio o vender un bien. Es fundamental tener en cuenta la adquisición de insumos y los gastos derivados de la producción, tales como el pago de mano de obra, gastos administrativos, entre otros.

En este caso particular, la estimación del costo de desarrollo de software es un elemento crítico en el análisis de proyectos informáticos y en la elaboración de la propuesta técnica. Es importante considerar el esfuerzo, costo y tiempo asociados a las actividades que se llevarán a cabo en el proyecto. Al planificar los costos, se deben tener en cuenta los siguientes aspectos:

- El tamaño del software a desarrollar y el tiempo estimado de producción.
- ¿Se realizará el trabajo por módulos?
- ¿Es necesario contratar personal de apoyo para la producción y/o soporte?
- ¿Cómo se proporcionará el soporte técnico? (Duración, presencial o remoto).
- ¿Es necesario adquirir equipos tecnológicos?

6. La propuesta técnica

La propuesta técnica es un documento crucial que se entrega al potencial cliente, donde se detalla la estrategia propuesta para resolver un problema existente. El formato y contenido de la propuesta pueden variar según el tipo de organización solicitante, ya sea privada, gubernamental, fundación u ONG. En estos casos, es importante adaptar la propuesta a los formatos específicos requeridos por cada tipo de organización. Una propuesta debe ser clara, concreta, explícita y fácil de entender.

Estructura

Una vez recopilada y analizada toda la información necesaria, se procede a estructurar la propuesta técnica según el siguiente esquema:

- Saludos y presentación.
- Alcance de la propuesta.
- Opciones de propuestas de trabajo (dos o tres).
- Cotización por cada propuesta presentada.
- Condiciones (forma de pago).
- Conclusión y despedida.

Términos de Referencia (TdR)

Es el documento que elabora una empresa, en el cual identifica oportunidades de mejora, necesidades o problemas y las presenta a otra empresa o a un experto informático para su solución. Los Términos de Referencia (TdR) son esenciales para que las empresas entiendan claramente los servicios que necesitan contratar y, para que la

contraparte conozca las condiciones bajo las cuales ofrecerán dichos servicios, permitiéndoles elaborar una propuesta técnica adecuada.

A continuación, se detallan algunos aspectos importantes que debe tener en cuenta en relación con los Términos de Referencia (TdR):

- **Servicio requerido**

Los Términos de Referencia (TdR) especifican el servicio que necesita la empresa de forma clara y concisa, incluyendo los objetivos que se esperan lograr con la contratación. Detallan las especificaciones técnicas de la solución informática, los requisitos del contratista, el tiempo de ejecución del contrato y las condiciones de pago.

- **Estructura del documento**

Este documento está estructurado con una justificación, alcances del servicio, perfil de la empresa a contratar y condiciones contractuales.

- **Empresa nueva**

Cuando una empresa es nueva y no maneja los TdR, el experto informático debe acordar una reunión con las partes interesadas o stakeholders. En esta reunión, se deben hacer preguntas sobre el funcionamiento de la empresa, los procesos que manejan, los servicios que prestan y en qué sector económico están.

- **Presupuesto**

En la mayoría de las ocasiones, y en búsqueda de un servicio justo, será posible proponer un diálogo sobre el presupuesto proyectado para la solución de software; asimismo, sobre la matriz DOFA de la empresa,

para tener claras sus necesidades y ofrecer los servicios adecuados en una propuesta técnica.

Elaboración de la propuesta técnica

Es fundamental recordar la importancia de disponer de una papelería adecuada que represente al negocio apropiadamente. El formato del documento ya sea físico o digital, debe contar con un diseño sobrio pero atractivo. El documento debe incluir un encabezado con el nombre de la empresa y su NIT o, en el caso de trabajadores independientes, el nombre de la persona. Este encabezado debe estar acompañado del logo de la empresa.

En el pie de página, es necesario incluir la dirección de la empresa junto con datos de contacto tales como números telefónicos, correos electrónicos, y enlaces a redes sociales como WhatsApp, Facebook, Instagram, entre otros. Al elaborar la propuesta, es crucial considerar como insumos principales los Términos de Referencia (TdR) o la información obtenida durante la entrevista inicial.

Explore el recurso que se presenta a continuación para conocer en profundidad los aspectos más importantes sobre la elaboración de la propuesta técnica:

Video 3. Elaboración de la propuesta técnica



[Enlace de reproducción del video](#)

Síntesis del video: Elaboración de la propuesta técnica

Teniendo clara la línea gráfica de la empresa en la papelería de nuestro negocio, es decir, habiendo definido el formato, encabezados o membretes, datos de contacto y todos los aspectos de identificación empresarial o personal; hay que conocer entonces la estructura que se usará en el cuerpo del documento. En el cuerpo del documento ha de aplicarse la siguiente estructura; recordando que se deben usar solo las hojas o páginas que en realidad sean requeridas, sin excederse, excepto cuando ello sea estrictamente necesario:

Fecha: según formato utilizado y ciudad, luego se nomina al remitente, sea este una persona, varias personas, o una organización.

Presentación y saludos: este párrafo deberá ser muy corto, en él se debe ser formal conciso y cordial.

Alcance de la propuesta: aquí se define cuáles son los objetivos reales que la implementación de software tendrá en la empresa y los límites de la propuesta, así es posible asegurarse de que las metas y objetivos se logren sin contratiempos.

Valoración de la situación desde el cliente: en este punto se expresa la situación deseada según la entrevista que anteriormente se haya realizado o teniendo en cuenta los términos de referencia, TdR.

Valoración de la situación desde el propio punto de vista: en este punto se expresan las maneras o particularidades con las que el servicio de software una vez implementado contribuirá a la transformación y mejora de la productividad de la empresa, entre otros beneficios probables, ello puede precisarse en un párrafo o también se puede listar con viñetas.

Opciones de propuesta: se puede presentar de una a tres propuestas dentro de este documento, brindándole al cliente más opciones para elegir, teniendo en cuenta el tamaño del proyecto, es posible hacerlo por fases o de manera sencilla, si se decide hacerlo de manera sencilla se podrá utilizar un título breve pero descriptivo y adicionar un párrafo concreto que describa lo que se pretende llevar a cabo.

Cotización: cada propuesta presentada deberá estar apoyada en una cotización, la misma es un precio fijo y no estimaciones, por lo cual se ha enviado más de una

propuesta para que el cliente tenga más alternativas de decisión acordes a su presupuesto.

Términos y condiciones: detallan cómo deben ser los pagos, adelantos, tiempo y fechas, cantidades o porcentajes, de cada cuota entre otras, todo ello conforme a las fases o entregas, conviene tener una cuenta bancaria activa ya que el medio de pago más utilizado en la actualidad es la transferencia o consignación bancaria.

Presentación: la propuesta debe ser entregada o socializada de acuerdo con los términos acordados con el cliente o según los requerimientos de la licitación, si el cliente solicita que la propuesta sea socializada, hay que contar con recursos o medios audiovisuales que favorezcan una óptima presentación.

6.1. Validación de la propuesta técnica

La validación de la propuesta técnica para servicios de software se rige y determina por varios factores críticos, incluyendo la normatividad, los procesos de contratación o licitación y las garantías de efectividad de la propuesta, entre otros aspectos.

A continuación, se presentan los elementos más relevantes que influyen directamente en el proceso de validación de una propuesta técnica:

- **Contratación**

Se realiza después de que el cliente haya aceptado una de las propuestas técnicas presentadas, o que al ofertante le hayan adjudicado una licitación. Para legalizar este trámite, es necesario cerrar el acuerdo por medio de un contrato escrito, en el cual estén estipulados todos los términos y

condiciones que se acordaron en la propuesta técnica o en el pliego de condiciones.

- **Fundamentos**

El fin de los principios o fundamentos en contratación es garantizar a los ciudadanos un proceso público, transparente y con igualdad, al que pueden acceder todas las personas o empresas que cumplan con los requisitos establecidos en un pliego de condiciones publicado por las entidades estatales. Estas pueden ser SENA, DIAN, Fuerzas Militares, ICBF, DANE, universidades, alcaldías, gobernaciones, etc.

- **Contratación privada**

Se trata del acuerdo al que llegan dos partes, que pueden ser personas naturales o jurídicas, para que se realice un producto, una labor o se deje de hacer algo. En este documento se pactan términos, acuerdos, multas y la forma de remuneración económica.

- **Contratación pública o estatal**

Según el artículo 3 de la Ley 80 de 1993, la contratación estatal tiene como fin la continua y eficiente prestación de los servicios públicos y la efectividad de los derechos e intereses de los administrados que colaboran con ellas, en la consecución de dichos fines. El Estado contrata a particulares para el cumplimiento de sus fines.

- **La licitación**

Método por el cual una entidad del Estado otorga un servicio u obra a una empresa o persona que ofrece las mejores condiciones. La licitación es privada cuando solo pueden participar las empresas a las cuales se hayan

enviado invitaciones. Es pública cuando se abre a todos los participantes, permitiendo que cualquier persona pueda participar en el proceso.

Aspectos de la contratación por licitación

En Colombia, la contratación se divide en dos tipos principales: privada y pública o estatal. Cada uno de estos tipos tiene sus propias particularidades, incluyendo aspectos normativos y legales específicos.

Para participar legítimamente en una licitación estatal, se deben cumplir varios requisitos a nivel personal o empresarial:

- Poseer el RUT (Registro Único Tributario), expedido por la DIAN.
- Tener la empresa inscrita en el registro mercantil, gestionado en la Cámara de Comercio.
- Registrar la empresa o los trabajadores en una ARL (Administradora de Riesgos Laborales).
- Mantener la contabilidad de la empresa de acuerdo con la guía técnica 184, con el apoyo de un Contador Público.
- Asegurar que los trabajadores de la empresa estén afiliados al sistema de seguridad social, incluyendo salud, pensión y caja de compensación.

El proceso de licitación pública requiere seguir algunos pasos de estricto cumplimiento, que usted puede conocer a continuación:

- **Pliego de condiciones**

La entidad estatal, que requiere obras, bienes o servicios, elabora un plan de compras que debe ser aprobado. Posteriormente, elabora y publica un

pliego de condiciones "Borrador" en la página SECOP

<https://colombiacompra.gov.co/> , así como en la página web de la entidad, dentro de 10 días hábiles.

- **Proponentes u oferentes**

Las personas interesadas, también llamadas proponentes u oferentes, pueden solicitar aclaraciones del pliego de condiciones dentro del término de los 10 días hábiles. Con la resolución de apertura del proceso se incluye el cronograma de presentación de propuestas, lo que conlleva a la publicación del pliego de condiciones definitivo.

- **Entrega de propuestas**

En la audiencia de asignación, revisión y distribución de riesgos, la entidad permite que los futuros proponentes aclaren sus dudas y realicen observaciones sobre el proceso. Conforme al cronograma, los proponentes entregarán sus propuestas en el lugar, hora y términos señalados.

- **Etapas de evaluación**

La entidad inicia la verificación de los requisitos mínimos y otorga los puntajes a las ofertas presentadas. Posteriormente, presenta el informe de la evaluación durante un término de cinco días hábiles, durante los cuales los oferentes pueden pronunciarse y presentar observaciones.

- **Observaciones presentadas**

La entidad analiza las observaciones presentadas y cita para definir el proceso en una audiencia pública según el cronograma establecido.

- **Audiencia de adjudicación**

Antes de la audiencia de adjudicación, la entidad pública publicará en el SECOP el borrador del acto administrativo de adjudicación. Si algún oferente cumple con todos los requisitos del pliego y ofrece las condiciones más favorables en términos de costos y calidad, la entidad adjudica el proceso a este oferente; de lo contrario, se declara el proceso desierto.

6.2. Normatividad

La contratación en Colombia, tanto pública como privada, está regulada y regida por la ley. Tener un conocimiento, al menos básico, de esta normatividad beneficia a personas y empresas al permitirles cumplir con lo establecido y alcanzar mejores procesos en la contratación de personal, bienes, servicios y obras.

Algunas leyes cruciales en la regulación de la contratación en Colombia son:

- Ley 80 de 1993: Modalidades de selección, publicidad, selección objetiva.
- Ley 1150 de 2007: Contratación con recursos públicos.
- Ley 1712 de 2014: Ley de Transparencia.
- Ley 1474 de 2011: Estatuto Anticorrupción.
- Ley 4170 de 2011: Agencia Nacional de Contratación Pública.

Acuerdos

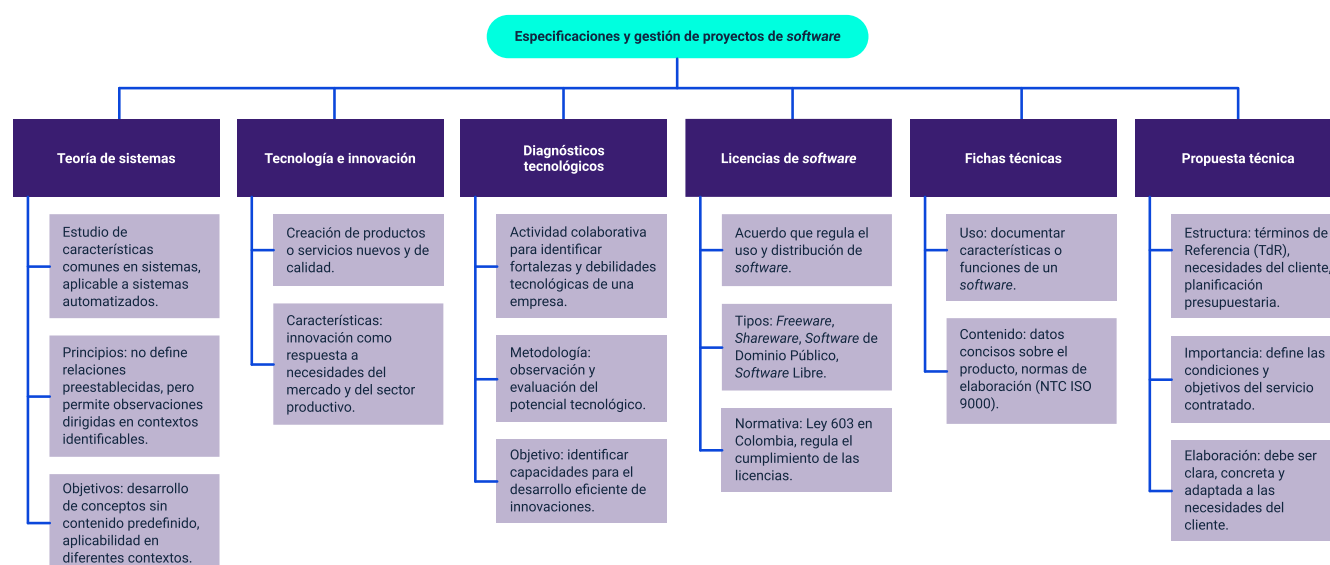
El principal acuerdo que rige el sistema de contratación en Colombia es conocido como “Acuerdo Marco”. Según la página de Colombia Compra Eficiente, el Acuerdo Marco es un instrumento de agregación de demanda. Se trata de un contrato entre Colombia Compra Eficiente, como representante de los compradores públicos, y uno o

más proveedores para adquirir bienes, obras o servicios que pueden ser estandarizados.

El Acuerdo Marco especifica la identificación del bien o servicio, el precio máximo de adquisición, el plazo máximo de entrega, las garantías mínimas y las condiciones bajo las cuales los compradores pueden vincularse al acuerdo.

Síntesis

A continuación, se muestra un mapa conceptual con los elementos más importantes desarrollados en este componente.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Teoría de sistemas	ONGAWA ONGD. (2012, septiembre 11). Tecnología, innovación, ¿desarrollo humano? YouTube.	Video	https://www.youtube.com/watch?v=OrLM7awX5Ls
Teoría de sistemas	Pacheco, L. [Administracion 101]. (2015, marzo 2). Gestión Tecnológica: Conceptos y Dimensiones. YouTube.	Video	https://www.youtube.com/watch?v=iiT9giJhTU
Licencias de software	Ármate Abogados. (2020). Licencia de Software.	Artículo	https://armate.cl/web/2020/11/03/licencia-de-software/
Licencias de software	Evaluando Software. (2021). Las licencias de software.	Artículo	https://www.evaluandosoftware.com/las-licencias-software/
Licencias de software	Bazán, V. (2020). ¿Conoces los tipos de licencia de software? Aquí te desglosamos el tema. OK hosting.	Artículo	https://okhosting.com/blog/tipos-de-licencia-de-software
Derechos de autor	Firma Legal Colombia. (2021, mayo 20). DERECHOS DE AUTOR EN COLOMBIA. YouTube.	Video	https://www.youtube.com/watch?v=JAfXwjut_ig
Derechos de autor	Leancase: Asesoría para emprendedores. (2020, junio 25). Derechos de autor sobre el Software. YouTube.	Video	https://www.youtube.com/watch?v=0pUHYFQIPI
Normatividad	Starfish Host. (2020). Cómo hacer una propuesta de Software - ¡Presenta una	Video	https://www.youtube.com/watch?v=6zI4ZcRS8yg

Tema	Referencia	Tipo de material	Enlace del recurso
	propuesta de software exitosa!. YouTube.		
Normatividad	Ministerio de Comercio, Industria y Turismo. (2013). Contratación Pública.	Artículo	http://www.aplicaciones-mcit.gov.co/cincopasos/c2.html

Glosario

Búsqueda: conjunto de procedimientos y operaciones que un usuario realiza para obtener la información necesaria.

Búsqueda automatizada interactiva: técnica de búsqueda que simula un diálogo entre usuario y computadora mediante preguntas y respuestas alternas.

Gestión de la Información (GI): procesos que controlan el ciclo de vida de la información, desde su creación hasta su disposición final.

GPL (General Public License): licencia que regula los derechos de autor de los programas de software libre.

Hardware: elementos físicos que constituyen una computadora o un sistema informático.

Herramienta digital: paquetes informáticos disponibles en computadoras y dispositivos electrónicos como celulares y tabletas.

Información: datos procesados y ordenados que aportan conocimientos sobre diversos temas.

Innovación: dinámica económica y social para la transformación del conocimiento científico y tecnológico.

Metadatos para la gestión de documentos: información que facilita la creación, gestión y uso de documentos a lo largo del tiempo y entre dominios.

Open source: término de la informática que se refiere a software con código fuente accesible para todos.

Seguridad de la información: medidas preventivas y reactivas para proteger la información y mantener su confidencialidad, disponibilidad e integridad.

Software: conjunto de programas y rutinas que permiten a la computadora realizar determinadas tareas.

Referencias bibliográficas

American Psychological Association (2020). Normas APA: La guía definitiva para presentar trabajos escritos. NORMAS APA. <https://normasapa.in/>

Arnold, M. (1989). Teoría de Sistemas. Nuevos paradigmas: Enfoque de Niklas Luhmann. Revista Paraguaya de Sociología, 26(75), p. 51-72.

Castro, E. (17 de julio de 2020). ¿Cómo hacer una propuesta de desarrollo de software? Anton.agency. <https://anton.agency/como-presentar-una-propuesta-proyecto-software/>

Colombia Compra Eficiente. (2020, 20 noviembre). Acuerdos Marco. DNP. <https://colombiacompra.gov.co/ciudadanos/preguntas-frecuentes/acuerdos-marco>

Coque, J. (2002). La innovación tecnológica en las sociedades cooperativas y otras organizaciones de participación. Revista de Estudios Cooperativos REVESCO, 78, p. 7-25. <https://dialnet.unirioja.es/descarga/articulo/1114242.pdf>

DeConceptos (2021). Conceptos. DECONCEPTOS. <https://deconceptos.com/?s=contrato>

Free Software Foundation Europe. (2005). Campaña “Nosotros Hablamos de Software Libre”. Espacio Linux.

Gaynor, G. (1999). Manual de gestión en tecnología: Una estrategia para la competitividad de la empresa. McGraw-Hill.

Hammond, D. (2003). The Science of Synthesis: Exploring the Social Implications of General Systems Theory. University Press of Colorado. <https://books.google.es/books?hl=es&lr=&id=TBvjCwAAQBAJ&oi=fnd&pg=PA103&dq=>

[+General+Systems+Theory+bertalanffy&ots=Ihf9vKSIAi&sig=ImXibhgxqJHaZ1H1zHsueBiPnaw#v=onepage&q=General%20Systems%20Theory%20bertalanffy&f=false](#)

Johannsen, O. (1975). Introducción a la Teoría General de Sistemas. Universidad de Chile.

Lozano, A. (2019). Teoría de los presupuestos. Gestipolis.

<https://www.gestipolis.com/teoria-de-los-presupuestos>

Malaver, F., Vargas, M. y Cerda, A. (2003). La innovación tecnológica en la industria colombiana: un estudio en dos cadenas industriales. Pontificia Universidad Javeriana.

Ministerio de Comercio, Industria y Turismo (2013). Contratación Pública. APLICACIONES-MCIT. <http://www.aplicaciones-mcit.gov.co/cincopasos/c2.html>

Oberto, A. (2007). Conocimiento tecnológico e innovación en el sector de termoplásticos del estado Zulia. Venezuela. Universidad del Zulia.

Pavón, J. y Goodman, R. (1976). Proyecto MODELTEC. La planificación del desarrollo tecnológico en el caso español. Centro para el Desarrollo Tecnológico Industrial.

Pérez, M. y Fernández, L. (2003). Técnica, Ciencia, Tecnología e Innovación. International Thomson Editores.

Sommerville, I. (2011). Ingeniería de Software. Pearson Educación.

Wiener, N. (1979). Cibernética y Sociedad. Editorial Sudamericana.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Diego Villalobos Barreto	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Juan Daniel Polanco Muñoz	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Alvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Laura Gisselle Murcia Pardo	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia



```
1 group_info = kmalloc(sizeof(*group_info) + nblocks*sizeof(gid_t *), GFP_USER);
2 if (!group_info)
3     return NULL;
4
5 group_info->ngroups = gidsetsize;
6 group_info->nblocks = nblocks;
7 atomic_set(&group_info->usage, 1);
8
9
10 if (gidsetsize <= NGROUPS_SMALL)
11     group_info->blocks[0] = group_info->small_block;
12 else {
13     for (i = 0; i < nblocks; i++) {
14         gid_t *b;
15         b = (void *)__get_free_page(GFP_USER);
16         if (!b)
17             goto
18     }
19 }
20
```

Problems Output Debug Console Terminal

Live link is activate. You can now view guestbook in your browser.

Local: <http://localhost:3000>

On your network: <http://192.80.72.55.91.3000>

Ln 18, Col 22 Spaces: 4 UTF-2 LF

Conceptos Teoría General de Sistemas

Especificaciones y gestión de proyectos de *software*

Servicio Nacional de Aprendizaje - SENA

A continuación, se describen los principales conceptos de la Teoría General de Sistemas, según conceptos de Arnold (1989) y otros autores:

Sistema: se trata de un grupo organizado, compuesto por partes que son independientes, que interactúan y se relacionan formando un todo.

Relación: es básicamente visualizada como la estructuración de una red.

Ambiente: hace referencia a las condiciones y hechos que influyen sobre el comportamiento del sistema.

Atributo: son las propiedades y características funcionales que definen las partes de un sistema.

Energía: la cantidad de energía que se mantiene en un sistema es igual a la de la suma de la energía importada menos la suma de la exportada.

Entropía: se trata de la máxima probabilidad de los sistemas en su consecutiva desorganización y su homogeneización con el ambiente.

Equifinalidad: consiste en que un sistema, iniciando con diferentes condiciones iniciales y por diferentes caminos, llegue a un mismo estado final.

Equilibrio: cuando los estados de equilibrio son alcanzados en sistemas abiertos por diversos caminos, se denomina equifinalidad. Mantener el equilibrio implica importación de recursos del ambiente.

Emergencia: cuando se realiza descomposición de sistemas en unidades menores, hasta el límite, se denomina emergencia.

Elemento: las partes o componentes que constituyen un sistema son llamadas elementos. Por ejemplo, objetos o procesos que son organizados en modelos.

Frontera: es aquella que permite separar al sistema de su entorno, definiendo lo que pertenece y lo que no.

Estructura: cuando las partes más o menos estables tienen interrelaciones que pueden ser identificadas en un momento dado, a esto se le llama estructura del sistema.

Función: el output del sistema es denominado función, enfocada en mantener el sistema mayor en el que está inscrito.

Cibernética: permite la retroalimentación de los procesos de control y de comunicación, igual en máquinas que en seres vivos (Wiener, 1979).

Circularidad: es el proceso de retroalimentación y metamorfosis, donde se refieren procesos de causa, así como cuando A causa a B y B causa C, pero se causa A.

Complejidad: permite indicar el número de elementos de un sistema (complejidad cuantitativa) y sus posibles interacciones (conectividad.)

Conglomerado: se dice que, si la suma de las partes, componentes y atributos en un conjunto es igual al todo, se trata de un conglomerado (Johannsen, 1975).

Homeóstasis: se refiere a los organismos vivos en sistemas adaptables.

Información: se dice que la información es la corriente más importante con la que cuentan los sistemas complejos.

Input/output: se trata de la elaboración de procesos de entradas y elaboraciones de salidas de un sistema.

Input: se refiere a la recolección de recursos de su ambiente para dar inicio a las actividades del sistema.

Output: son las corrientes de salida de un sistema.

Organización: Wiener se refiere a la organización como “una dependencia de las distintas partes organizadas” (Buckley, 1970, p. 127).

Modelo: permite identificar y mensurar relaciones sistemáticas complejas.

Morfogénesis: se caracteriza por modificar o crear sus formas, con el objeto de conservarse viable.

Morfostasis: se trata de los procesos de intercambio que se hacen con el ambiente para preservar una organización en un estado determinado.

Observación: incorpora el problema de la observación de sistemas de observadores.

Recursividad: hace referencia a incluir los resultados de las operaciones de un sistema en el mismo sistema.

Retroalimentación: es cuando un sistema abierto recoge información estudiando los efectos de sus decisiones.

Sinergia: es una consecuencia, un fenómeno que nace de las interacciones entre las partes de un sistema.

Variabilidad: muestra el número máximo de relaciones posibles.

Variedad: cantidad de elementos. Se trata del número de elementos de un sistema.

Viabilidad: se trata de hacer una medición de la capacidad de sobrevivencia y adaptación de un sistema.

Gestión de requisitos

Breve descripción:

Este componente formativo aborda la ingeniería de requisitos y el levantamiento de requerimientos para prever el comportamiento del software. Utiliza casos de uso, historias de usuario y Lenguaje de Modelado Unificado para modelar funcionalidades e interacciones. Además, introduce herramientas CASE, esenciales para visualizar y construir software.

Junio 2024

Tabla de contenido

Introducción	4
1. Características del Lenguaje Unificado de Modelado UML	6
Versión oficial de UML	7
Características generales del UML	7
2. Especificación y análisis de requisitos	10
Diagrama de casos de uso	10
Actor.....	11
2.1. Relaciones de casos de uso.....	16
Especificación de casos de uso	19
2.2. Casos de uso reales (Prototipos)	21
3. Historias de usuario	24
Criterios de aceptación:.....	25
4. Diagrama de actividades.....	31
Elementos de un diagrama de actividades.....	31
Ejemplo de diagrama de actividades	34
5. Diagramas de clase	36
Control de acceso	38
Relaciones entre clases.....	39

5.1.	Asociaciones	39
5.2.	Herencia	43
	Visibilidad en las relaciones de herencia.....	47
5.3.	Agregación.....	47
5.4.	Composición	48
6.	Herramientas CASE.....	50
	Clasificación de las Herramientas CASE.....	51
	Síntesis	54
	Material complementario.....	55
	Glosario	56
	Referencias bibliográficas	57
	Créditos.....	59

Introducción

La especificación de requisitos es uno de los procesos más importantes en las fases de desarrollo de software. Constituye la fase inicial y es crucial para que el proyecto culmine exitosamente. Su objetivo principal es comprender qué esperan los usuarios y clientes del sistema, empleando un lenguaje natural para obtener una perspectiva clara y evitar errores costosos en las fases subsiguientes del desarrollo del producto.

El análisis de requerimientos propone un conjunto de técnicas e instrucciones que facilitan una representación precisa y completa de las funcionalidades que debe ejecutar un sistema para satisfacer los requerimientos de clientes y usuarios. Actualmente, algunas de las técnicas más utilizadas incluyen los diagramas de casos de uso, historias de usuario y diagramas de actividades.

Para la construcción de software, es fundamental la utilización de modelos, ya que representan la dirección y aspecto de una necesidad específica. Esta "necesidad" puede estar ya existente, en desarrollo o en fase de planificación. Los diseñadores del modelo deben levantar los requerimientos del producto, que pueden abarcar áreas como funcionalidad, rendimiento y fiabilidad. El modelo se estructura en varias vistas, cada una describiendo un aspecto específico del producto o sistema en desarrollo. Un modelo permite documentar la estructura y comportamiento de un sistema antes de ser codificado.

Este componente formativo presentará diversas técnicas y diagramas para el análisis y especificación de requisitos, con el fin de organizar adecuadamente los requerimientos. Esto permite una representación efectiva de cada una de las

interacciones del usuario con el sistema, resultando en una especificación eficaz que reduce costos y riesgos en el desarrollo del software, y contribuye a alcanzar un producto de calidad.

1. Características del Lenguaje Unificado de Modelado UML

Con el nacimiento de la Programación Orientada a Objetos (POO), se establecieron una variedad de lenguajes, métodos y simbologías utilizadas para detallar el desarrollo de diferentes sistemas. Indudablemente, esta situación generó desconcierto entre los programadores de software. Para solucionar este inconveniente, surgió el Lenguaje de Modelado Unificado (Unified Modeling Language, UML).

- **El lenguaje de Modelado Unificado UML**

Proporciona un conjunto de herramientas que facilitan el modelado (análisis y diseño) de sistemas orientados a objetos. Establece una serie de normas técnicas que representan de manera gráfica el camino y la estructura de un sistema determinado para programadores, analistas, desarrolladores, diseñadores, etc.

El UML es una de las herramientas más emocionantes en el mundo actual del desarrollo de sistemas. Esto se debe a que permite a los creadores de sistemas generar diseños que capturen sus ideas de forma convencional y fácil de comprender, facilitando la comunicación de estas a otras personas. (Schmuller, 2000, p.24)

- **Evolución del Lenguaje Unificado de Modelado UML**

El UML tuvo su origen a finales de los años 80, inicialmente basado en el método Booch, desarrollado por el diseñador de software Grady Booch. En 1994, el científico de la computación James Rumbaugh implementó el modelado de objetos en este método. En 1995, se añadieron a estas bases

otros principios y metodologías, algunas de ellas introducidas por Ivar Jacobson. (Booch, 2006).

Versión oficial de UML

La primera versión oficial, UML 1.0, se lanzó en 1997 bajo el liderazgo de Booch, Rumbaugh y Jacobson. Esta versión ofrecía una metodología que incluía varios estándares con el principal objetivo de asegurar la coherencia entre todos los diagramas. En 2005, se anunció la versión UML 2.0, cuyo contenido estaba principalmente orientado hacia la visión de la Programación Orientada a Objetos. Desde entonces, se han realizado nuevas versiones basadas en UML 2.0.

- Grady Booch, ingeniero de sistema e investigador, antes del 95.
- Jim Rumbaugh (OMT), antes del 95.
- Ivar Jacobson, UML 0.8 y 0.91 1996.

Características generales del UML

Como se mencionó anteriormente, el UML es un lenguaje que proporciona un conjunto de reglas que facilitan la comunicación de un sistema mediante representaciones gráficas, indicando cómo realizar la creación y lectura de los modelos. Aunque existen muchas características del UML, según Booch (2006), se pueden resumir en las siguientes, las cuales se detallan a continuación:

a) Visualizar

UML facilita la representación de un sistema de forma gráfica, con el fin de que se pueda entender fácilmente por otra persona.

b) Especificar

UML permite detallar cuáles son las necesidades de un sistema previo a su construcción.

c) Construir

La construcción del sistema diseñado se realiza tomando los modelos especificados en el lenguaje UML.

d) Documentar

Cuando se diseñan las notaciones gráficas del sistema desarrollado, estas se utilizan como documentación. Esto permite brindar soporte para nuevas revisiones o empalmes, facilitando que los nuevos desarrolladores comprendan el sistema.

La versión 1.0 de UML fue lanzada en enero de 1997 y desde entonces ha sido utilizado con éxito en la construcción de sistemas para diversas industrias alrededor del mundo, incluyendo hospitales, bancos, comunicaciones, aeronáutica y finanzas, entre otras. UML es actualmente el lenguaje de modelado de sistemas de software más conocido y utilizado, permitiendo visualizar, especificar, construir y documentar sistemas con una perspectiva orientada a objetos (Rumbaugh et al., 2004).

Figura 1. Modelo para la construcción de software UML



UML está compuesto por diversos elementos gráficos que se combinan para conformar diagramas. Entre los tipos de diagramas que ofrece UML, se encuentra el diagrama de clase, que es parte de los diagramas de estructura. Estos últimos enfatizan los elementos que deben existir en el sistema de modelado.

2. Especificación y análisis de requisitos

La función más importante de la especificación de requisitos es servir de intermediario para que los clientes, los analistas de requisitos, los desarrolladores y los usuarios puedan comunicarse eficazmente. En esta fase es fundamental recolectar tanto los requerimientos del usuario y del cliente como los del software a desarrollar, para lograr la satisfacción del cliente. Por ello, es crucial el uso de técnicas que permitan representar de manera visual e intuitiva estos requerimientos.

Cuando se realiza la recolección de requerimientos de un cliente, el UML facilita el modelado de estos a través de sus casos de uso. Los stakeholders (partes interesadas) que están involucrados en el producto son incluidos en el modelado con las funcionalidades requeridas (casos de uso), que a su vez son modelados incluyendo relaciones y asociaciones jerárquicamente organizadas entre ellos.

Los casos de uso y sus actores son quienes especifican los requerimientos de un cliente, detallando las expectativas que cada uno tiene del sistema, sin centrarse únicamente en la funcionalidad que se implementará. Los análisis de requisitos se realizan para diversos procesos, no necesariamente solo para desarrollos de software. Entre los diagramas más relevantes y utilizados en UML se encuentran los diagramas de casos de uso.

Diagrama de casos de uso

Los casos de uso permiten describir cada una de las funciones de las aplicaciones a desarrollar desde el punto de vista del usuario. Su utilidad se centra en expresar las funcionalidades que el producto de software debe ejecutar y, definir a las personas que serán responsables de realizar estas funcionalidades. Esta técnica implica la

creación de un diagrama con su respectiva notación. El caso de uso representa el comportamiento del software en la interacción con el usuario, con el objetivo de que este alcance una meta específica. Por tanto, un caso de uso se compone de los elementos que forman parte de su notación.

- **Componentes de los diagramas de casos de uso**

La notación de un diagrama de caso de uso incluye tres componentes principales que facilitan la representación simbólica de las funcionalidades del sistema, así como de las personas involucradas y las interacciones entre ellas.

Estos componentes son: los actores, que son los usuarios o roles que interactúan con el sistema; los casos de uso, que describen las funciones del sistema; y las relaciones, que especifican las conexiones entre actores y casos de uso.

Actor

Define un rol desempeñado por un usuario, representa a una persona o grupo de personas que interactúan con un software. Se simboliza mediante la figura de un "stick man", junto al nombre del actor, que es obligatorio. Los actores se utilizan para identificar la clase de usuario que participará en el sistema y será responsable de ejecutar determinadas funciones. Por lo general, los nombres de los actores comienzan con mayúscula.

- **Principales**

Son actores especificados para identificar los objetivos de los usuarios, los cuales son guiados por los casos de uso.

- **De apoyo**

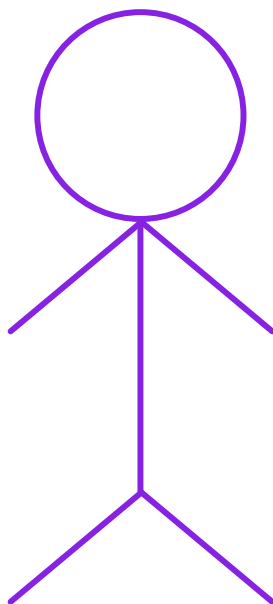
Proveen servicios al sistema informático y son definidos para esclarecer las interfaces externas.

- **Los pasivos**

Se interesan en el funcionamiento de los casos de uso, aunque no son considerados actores principales.

En la figura que se presenta a continuación, se identifica cómo se representa de manera gráfica un actor en un UML, al cual también se le adiciona el nombre del actor correspondiente para dar mayor claridad en el proceso.

Figura 2. Representación gráfica de un actor en UML



Además, es importante considerar que los actores no son necesariamente personas, aunque en los diagramas de casos de uso se representen mediante formas humanas. En este contexto, los sistemas externos también cumplen el rol de actores,

pues pueden requerir información del sistema real o interactuar con él, como ocurre con una base de datos.

Al determinar los casos de uso, resulta esencial identificar al actor. Este término se refiere al usuario cuando desempeña un rol específico, implicando que dicho usuario ejecuta los casos de uso. Dado lo anterior, cada actor tiene la capacidad de participar en diversos casos de uso y, a su vez, cada caso de uso puede involucrar a diferentes actores. Para identificar a un actor, basta con visualizar los roles que interactúan dentro del sistema.

Ejemplo 1:

Para explicar cada uno de los elementos de un caso de uso, se presenta el siguiente requerimiento:

Una empresa solicita un sistema que facilite el proceso de generación de facturas de venta para comerciantes. Estas facturas deben ser enviadas a través de un sistema externo encargado del proceso de facturación.

Los actores identificados en los requisitos del ejemplo 1 son el comerciante y el sistema externo.

a) Relación

La interacción entre un actor y un caso de uso se denota mediante una relación, simbolizada por una línea o una flecha. Se describen varios tipos de relaciones según los actores mencionados anteriormente.

- La relación de comunicación entre un actor y un caso de uso puede ser activa o pasiva.

- Un actor activo inicia la ejecución, con la flecha apuntando hacia el caso de uso correspondiente.
- Un actor pasivo, en cambio, no inicia la ejecución y la flecha apunta hacia él.
- El tipo de comunicación o relación entre actores y casos de uso establece una asociación directa.
- Esta asociación especifica que el actor es el ejecutor del proceso descrito.

Figura 3. Representación gráfica de una relación de caso de uso en UML



Continuando con el ejemplo 1 se identifican las siguientes relaciones de comunicación:

- Entre el comerciante y la generación de la factura.
- Entre el comerciante y el envío de la factura.
- Entre la factura y el sistema externo.

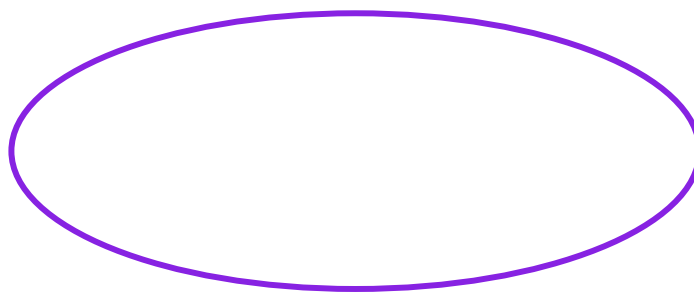
b) Caso de uso

Los casos de uso se identifican mediante los verbos o acciones que el actor ejecuta o ejecutará. Estas acciones se describen y representan de la siguiente manera:

- Se simbolizan con un óvalo, indicando una función que el sistema debe proporcionar.

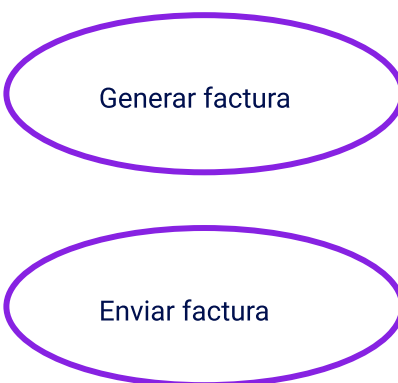
- El caso de uso debe ser de fácil lectura e interpretación, tanto para los desarrolladores como para el cliente. Para el nombre, se recomienda utilizar un verbo en infinitivo acompañado de un complemento.

Figura 4. Representación caso de uso



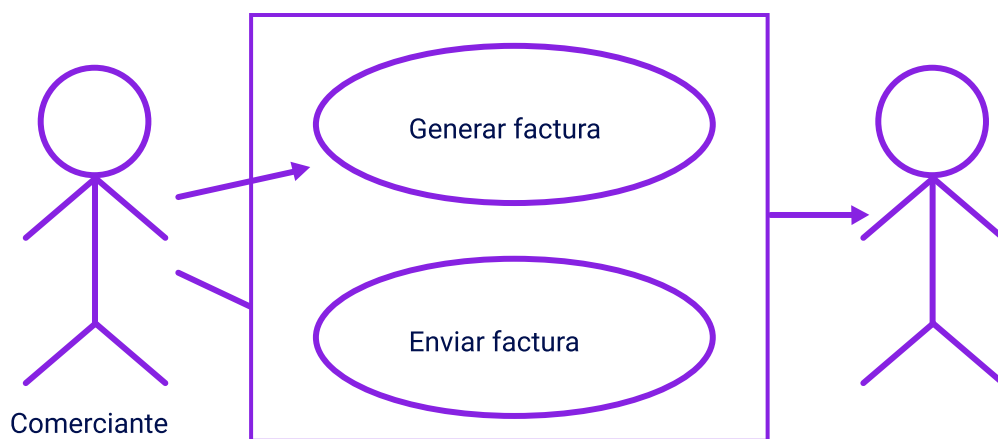
Los casos de uso que se identifican para el ejemplo 1 son: generar factura y enviar factura, los cuales se ven representados gráficamente en la siguiente figura.

Figura 5. Ejemplo caso de uso



A continuación, se presenta un ejemplo de la representación gráfica del diagrama de caso de uso, que incluye los actores, relaciones y el caso de uso del problema planteado en el Ejemplo 1.

Figura 6. Representación ejemplo 1



Por otro lado, los pasos para diseñar el diagrama de casos de uso del sistema de ventas, mostrado en la figura anterior, están detallados a continuación:

- Se identifican los casos de uso según requerimientos, para el caso del ejemplo son: (generar factura, enviar factura).
- Se identifican los posibles actores que interactuaran con el sistema: para el caso del ejemplo: (comerciante y sistema externo).
- Se identifican las relaciones de comunicación entre comerciante y los casos de uso, generar factura y enviar factura.

Además de la relación básica de comunicación mostrada anteriormente, existen otras relaciones en los casos de uso que detallan con mayor precisión las interacciones entre ellos.

2.1. Relaciones de casos de uso

Durante el proceso de desarrollo, se pueden definir diferentes casos de uso que no son independientes entre sí. Es posible establecer relaciones de dependencia mutua entre los casos de uso. Las principales relaciones consideradas por UML son:

- **Inclusión (include)**

Se emplea para extraer comportamientos comunes de un caso de uso. Un caso de uso puede incluir a otro.

- **Extensión (extend)**

Utilizada para ampliar las funcionalidades de un caso de uso existente, es especialmente útil para incorporar nuevos requisitos del sistema.

- **Generalización (generalization)**

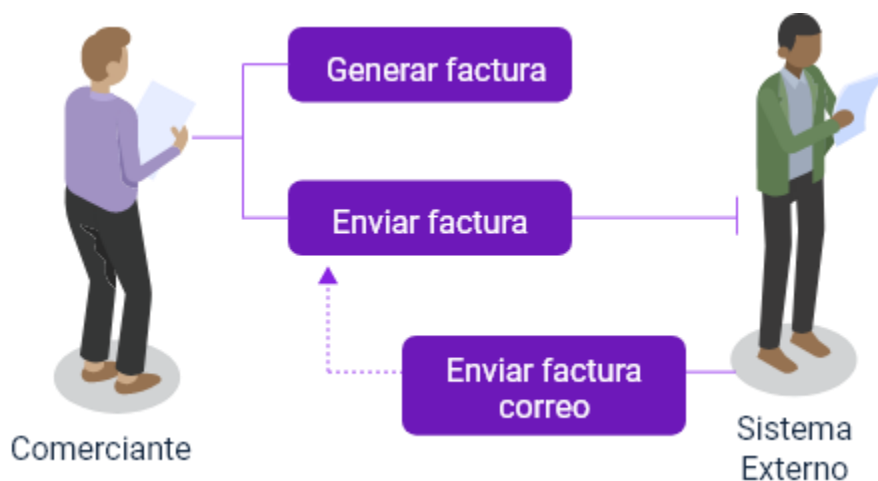
Esta relación ocurre entre un caso de uso general y uno más específico, donde el específico hereda propiedades del caso de uso base.

Tomando como referencia el Ejemplo 1, con el objetivo de perfeccionar el diagrama de casos de uso, se pueden introducir nuevos casos de uso que ilustren las relaciones mencionadas anteriormente.

- **Ejemplo extends**

Se establece la relación extends cuando un caso de uso es similar a otro, pero incluye funciones adicionales. Por ejemplo, en el caso mencionado anteriormente, el caso de uso enviar factura podría ampliarse a enviar factura por correo electrónico. Ambos casos implican la misma acción básica de enviar la factura; sin embargo, en la extensión, además se envía un correo electrónico con la factura. Esto se representaría de la siguiente manera:

Figura 7. Relación extends

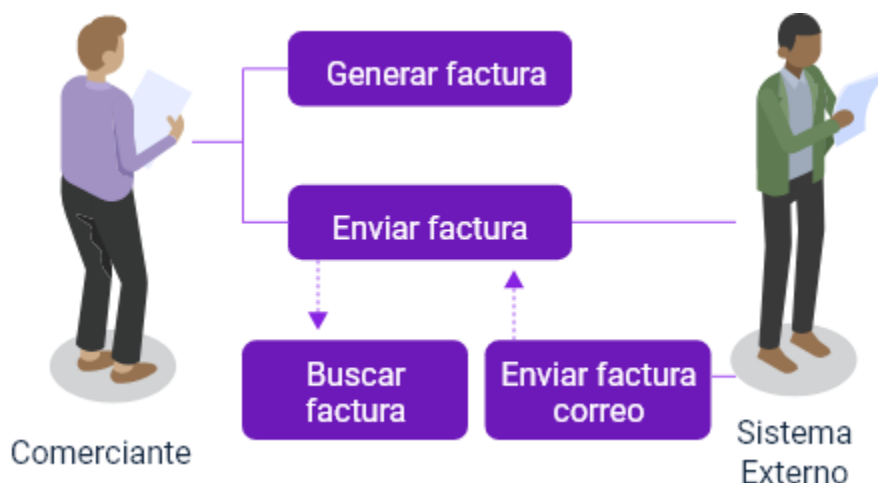


La relación se interpreta así: el caso de uso enviar factura por correo extiende del caso de uso enviar factura.

- **Ejemplo include**

Otro aspecto importante a considerar se presenta en ejemplo include; esta relación se define cuando parte del comportamiento o acción es común en más de un caso de uso, eliminando la necesidad de duplicar dichas acciones. Por ejemplo, en el caso del Ejemplo 1, se añade el siguiente requisito funcional: antes de enviar la factura, el usuario puede realizar una búsqueda de facturas para seleccionar y enviar. Esto hace necesario incluir la relación include. La representación sería la siguiente:

Figura 8. Relación include



Especificación de casos de uso

La especificación de casos de uso se refiere al proceso de descripción detallada de cada caso, incluyendo los flujos y eventos que interactúan con el sistema y los actores involucrados. Generalmente, el analista encargado del diseño identifica los requerimientos y los documenta.

El propósito de la especificación de casos de uso incluye:

- Describir la interacción de un actor con el sistema.
- Explicar el rol que desempeñan los actores en su interacción con el sistema y sus flujos.
- Detallar los casos de uso.

A continuación, se presenta las partes esenciales y las directrices básicas para la documentación de casos de uso.

Tabla 1. Plantilla de casos de uso

Componente	Descripción de una especificación de caso de uso
Nombre del caso de uso	Indica el nombre del caso de uso. Normalmente, el nombre expresa el resultado objetivo y observable del caso de uso, como por ejemplo "Imprimir factura" en el caso de un sistema de ventas.
Autor	Persona que diseña el caso de uso.
Fecha	Aquí se diligencia la fecha en la que se realizó el caso de uso.
Descripción	Se realiza una corta descripción del caso de uso.
Actores	Describe los nombres del actor o actores del caso de uso.
Precondiciones	Son aquellas condiciones que deben existir para que se cumpla el caso de uso.
Flujo normal	Son los pasos normales de ejecución del caso de uso.
Flujo alternativo	Flujos alternativos de ejecución de casos de uso.
PosCondiciones	Son aquellas condiciones que se cumplen cuando finaliza un caso de uso.

A continuación, se presenta un ejemplo de especificación de casos de uso para el caso de uso “Crear Foro” utilizando la plantilla propuesta posteriormente.

Tabla 2. Descripción de una especificación de caso de uso ejecutado

Componente	Descripción de una especificación de caso de uso
Nombre del caso de uso	Crear foro

Componente	Descripción de una especificación de caso de uso
Autor	Mario Vera Cruz
Fecha	20/09/2021
Descripción	Crea un nuevo mensaje.
Actores	Alumno/Docente.
Precondiciones	Al iniciar el usuario debe estar autenticado en el sistema.
Flujo normal	1. El actor da clic en la opción nuevo mensaje. 2. Se muestra un área de texto para escribir el mensaje. 3. El actor introduce el mensaje. 4. El sistema verifica datos y los almacena. 5. El profesor acepta y publica.
Flujo alternativo	4a. Se hace una comprobación de los datos, si no son correctos muestra mensaje de corrección. 4b. El profesor rechaza el mensaje de modo que no es publicado sino devuelto.
PosCondiciones	Se publica y almacena el mensaje satisfactoriamente.

2.2. Casos de uso reales (Prototipos)

El propósito principal de los casos de uso reales es detallar el proceso de un sistema de información descrito mediante un caso de uso, lo que incluye la interacción de objetos y la definición de las transacciones de las interfaces y clases de los diferentes procedimientos secundarios de diseño. Por lo tanto, es fundamental completar los

entornos recolectados del análisis con las clases de diseño correspondientes, sin dejar de lado las restricciones del entorno tecnológico.

Los casos de uso reales representan el diseño preciso del caso de uso desde una tecnología específica. Por ejemplo, si se trata de un diseño gráfico para el usuario, se incluirá el diseño de las ventanas y una descripción de la interacción con los objetos de la interfaz. Los casos de uso reales proporcionan diseños de pantalla y explicaciones sobre la interacción entre los objetos de la interfaz.

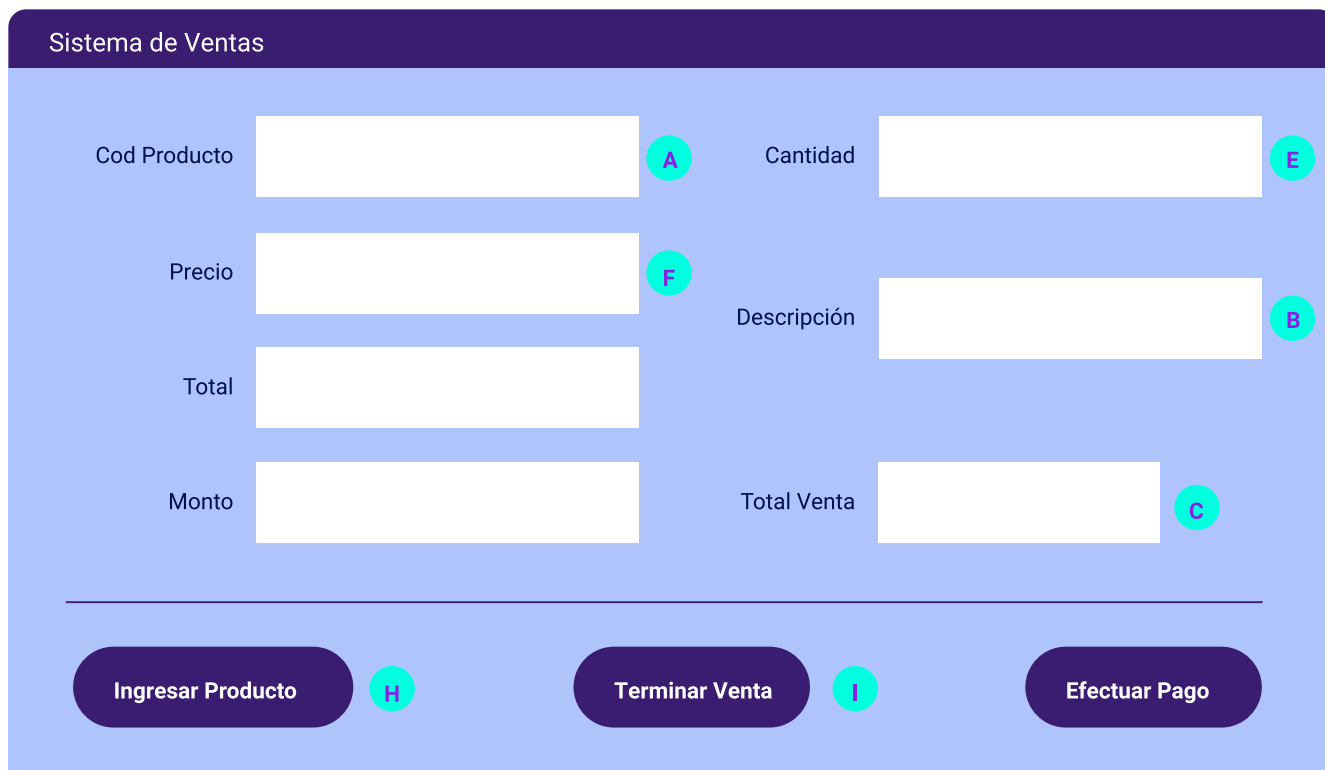
Para aclarar las definiciones anteriores, se utiliza un ejemplo de un sistema de ventas que se presentará posteriormente. En este ejemplo, se realiza el diseño del caso de uso real y se describe la interacción entre los artefactos del diseño. A continuación, se presenta una tabla con los datos de un ejemplo para un sistema de ventas:

Tabla 3. Sistema de ventas

Caso de uso adquirir productos	Adquirir productos
Actores	Cliente (quien inicia el proceso), Cajero.
Tipo	Primario, real.
Descripción	• En el almacén de ropa el cliente se dirige a la caja registradora con los productos a comprar. • El empleado-cajero hace el registro de los productos que el cliente compra y recibe el dinero en efectivo. • Al finalizar el proceso, el cliente se retira con los artículos comprados.

A continuación, se presenta el ejemplo del sistema de ventas:

Figura 9. Ejemplo Sistema de ventas



The image shows a software interface for a sales system. The title bar at the top is dark blue and says "Sistema de Ventas". Below this, on a light blue background, are several input fields arranged in two columns. The left column has fields for "Cod Producto", "Precio", "Total", and "Monto". The right column has fields for "Cantidad", "Descripción", and "Total Venta". Each field is a white rectangle. To the right of the "Cod Producto" field is a small red circle with the letter "A". To the right of the "Cantidad" field is a small red circle with the letter "E". To the right of the "Precio" field is a small red circle with the letter "F". To the right of the "Descripción" field is a small red circle with the letter "B". To the right of the "Total" field is a small red circle with the letter "C". To the right of the "Total Venta" field is a small red circle with the letter "C". At the bottom of the interface, there are three dark blue buttons with white text. The first button is "Ingresar Producto" with a small red circle with the letter "H" to its right. The second button is "Terminar Venta" with a small red circle with the letter "I" to its right. The third button is "Efectuar Pago".

- a) El cliente se acerca a la caja registradora con los artículos que desea llevar.
- b) El cajero ingresa el código del producto en A de la Ventana-1.
 - Opcional: Si el cliente tiene más de un producto, el cajero puede registrar la cantidad en E.
 - Una vez finalizado el registro de todos los productos, se presiona H.
- c) El sistema almacena los productos registrados.
 - La descripción y el precio del producto se muestran en B y en F de la Ventana-1.
- d) El cajero presiona el botón I para dar por finalizada la venta.
- e) El sistema realiza el cálculo y muestra en C el total de la venta.

3. Historias de usuario

Las historias de usuario son una forma sencilla de representar los requisitos de un sistema de información. Se crean en una o dos oraciones y se redactan en un lenguaje común y legible para el usuario. Estas historias, conocidas como User Stories, se han convertido en un estándar ampliamente utilizado en la definición de requisitos. Surgieron en XP (1999) y fueron impulsadas de manera concluyente por Mike Cohn en su libro "User Stories Applied: For Agile Software Development" (2004), estableciendo el patrón para definirlas. La simplicidad es la razón principal de su popularidad. Las historias de usuario están compuestas por tres elementos:

- Cómo [Rol del usuario]
- Quiero [Objetivo]
- Para poder [Beneficio]

A continuación, se hace la descripción de los elementos de una historia de usuario según Cohn, M. (2018):

a) Tarjeta (Card)

Es una frase que sintetiza una necesidad dada en una conversación entre el usuario y el dueño del producto. Escrita en una tarjeta, comúnmente sigue el formato rol-objetivo-beneficio. Su estructura es:

- Como [tipo de usuario] -> ¿quién?
- Quiero [necesidad] -> ¿qué?
- Para [beneficio esperado] -> ¿para qué? Con esta estructura simple de tres líneas se identifican: ¿Quién? (roles de valor), ¿Qué? (problema a resolver) y ¿Para qué? (objetivo a alcanzar).

b) Conversación

Es el proceso de comunicación entre las personas que requieren las necesidades (dueño del producto) y las personas que saben cómo solucionarlas (desarrolladores). Se realiza mediante preguntas y utilizando recursos como gráficas y prototipos para clarificar el contenido de las historias de usuario.

c) Confirmación

Es la verificación del acuerdo con relación a lo que se quiere construir. Permite validar si se terminó o no, o si se cumplió con lo esperado. Esta confirmación recibe el nombre de criterios de aceptación, los cuales ayudan a tener claro el contexto de la historia de usuario y a determinar fácilmente si está terminada o no.

A continuación se presenta un ejemplo, el usuario desea comprar una boleta para una película en el cine. los criterios de aceptación que se pueden establecer son:

Criterios de aceptación:

- a) Cuando se agoten las boletas, se debe mostrar un mensaje informando que no hay disponibilidad para la venta.
- b) El sistema debe validar la edad del comprador para permitir la venta de boletas según la clasificación de la película.
- c) Tras un pago en efectivo, el sistema debe mostrar el valor del cambio a devolver; si no hay cambio, deberá mostrar cero.

Cuando se realizan historias de usuario es muy importante cumplir con las siguientes reglas:

- Ser concisa
- Concreta
- Realizar criterios de aceptación claros y específicos

Así como estas reglas son necesarias, también lo es una buena redacción de las historias de usuario. Por este motivo, a continuación se presenta un método que facilita mejorar la calidad en la escritura de una historia de usuario: el método “Invest”.

Método invest

A continuación, se hace una descripción de cada uno de los elementos que intervienen en este método:

- **Independiente**

Las historias de usuario deben ser lo más independientes posible para evitar dependencias.

- **Negociable**

Las historias de usuario deben ser flexibles y negociables, siempre en acuerdo con el cliente.

- **Valiosa**

Es crucial que cada historia de usuario aporte valor al cliente. Idealmente, el cliente debería participar activamente en su redacción.

- **Estimable**

La estimación de la historia debe ser precisa y realizable, teniendo en cuenta el tamaño y la complejidad, para facilitar la planificación por parte del equipo.

- **Small (Pequeña)**

El tamaño de la historia de usuario debe ser pequeño para simplificar y mejorar la precisión de la estimación. Es ideal que se pueda completar en periodos cortos, como días o semanas.

- **Testeable (Comprobable)**

La historia de usuario debe ser comprobable, con criterios claros de aceptación para considerarse completada.

Al utilizar este método, se garantiza que las historias de usuario sean independientes entre sí, facilitando así su planificación y desarrollo sin sobrecargarlas con detalles técnicos que puedan limitar las negociaciones entre clientes y desarrolladores.

Ejemplos

A continuación, se presentan algunos ejemplos prácticos de historias de usuario aplicando el método invest:

a) Historia de usuario: “Crear Usuario”

En seguida, se presenta dos opciones:

Primera opción:

- **Rol:** como cliente de un supermercado.

- **Objetivo:** quiero hacer compras mediante una aplicación web.
- **Beneficio:** para poder comprar sin salir de casa.
- **Comentario:** en esta historia de usuario, se identifica claramente lo que el cliente desea hacer, aunque no se especifican los detalles de cómo se realizarán las compras en línea. Estos detalles podrían aclararse en reuniones posteriores. Esta historia cumple con las características del método INVEST: es sencilla, clara y satisface una necesidad específica, facilitando así la estimación del tiempo de desarrollo.

Segunda opción:

- **Rol:** como cliente de un supermercado.
- **Objetivo:** quiero acceder al registro mediante un formulario que me permita escribir mi correo electrónico y asignar una contraseña.
- **Beneficio:** para asegurar mi privacidad.
- **Comentario:** esta historia de usuario contiene especificaciones excesivas en la sección de objetivo y no clarifica suficientemente el beneficio. Se recomienda simplificar y aclarar el propósito para alinearse mejor con los criterios de INVEST.

b) Ejemplo de una historia épica

- **Rol:** como cliente de un supermercado.
- **Objetivo:** quiero ingresar a una página web.
- **Beneficio:** para hacer compras en línea.
- **Comentario:** esta historia no es independiente y se considera una Épica, porque es muy amplia y puede subdividirse en múltiples historias más

pequeñas y manejables. Para visualizar mejor una historia de usuario Épica, se recomienda consultar recursos de aprendizaje específicos sobre este tema.

- Hacer un registro
- Un ingreso
- Seleccionar productos
- Pagar, etc.

Plantillas de historias

Para visualizar una historia del usuario épica, lo invitamos a consultar el PDF "**Plantillas de historias**", el cual se encuentra en la carpeta Anexos.

Por lo anterior, tampoco se cumpliría con la característica de estimación, ya que no se podría determinar cuándo terminaría la historia. Como se identificó en los ejemplos anteriores, el método INVEST nos proporciona reglas para optimizar la redacción de las historias de usuario, permitiendo así satisfacer las necesidades reales del cliente.

Para realizar una buena gestión de proyectos, es habitual en las metodologías ágiles utilizar una serie de herramientas para crear las historias de usuario, tales como:

- Trello
- Jira Software
- Taiga
- Excel

c) Plantillas de historias de usuario

Las plantillas de historias de usuario pueden adaptarse según las necesidades específicas sin perder la flexibilidad y sencillez que caracteriza a una historia de usuario.

Los equipos de desarrollo colaboran utilizando herramientas en línea para compartir y trabajar conjuntamente en las plantillas. A continuación, se presenta un ejemplo del uso de una plantilla de Excel para la gestión de productos, descargada de la web.

Tabla 4. Ejemplo plantilla historia de usuario

ID de la historia	Rol	Funcionalidad	Razón/Resultado	Criterio de aceptación
Id.01	Como cliente	Quiero ver una lista de categorías de productos.	Para poder realizar búsquedas de productos por categorías.	Despliegue de listado de categorías a seleccionar con productos asociados.
Id.02	Como cliente	Quiero ver una lista de precios.	Para poder elegir el producto más barato.	Despliegue una lista ordenada de productos por precio de menor a mayor.

4. Diagrama de actividades

Estos son diagramas de comportamiento que se utilizan para representar una sucesión de actividades. Explican el flujo de operaciones desde el punto de inicio hasta el final, definiendo una variedad de caminos de decisiones en el desarrollo de eventos que abarca una actividad. Según Booch, G. (1994), estos diagramas permiten visualizar un caso de uso específico a un nivel más preciso, ilustrando el flujo de actividades definidas en un sistema.

A continuación, se describen los beneficios del diagrama de actividades:

Beneficios:

- Explica la lógica de un algoritmo.
- Representa cada uno de los procesos realizados en los diseños de casos de uso en UML.
- Instruye procesos entre el sistema y los usuarios en un flujo de trabajo o negocios.
- Aporta claridad, optimizando y facilitando los casos de uso complejos.

Elementos de un diagrama de actividades

Un diagrama de actividades contiene fundamentalmente los siguientes elementos:

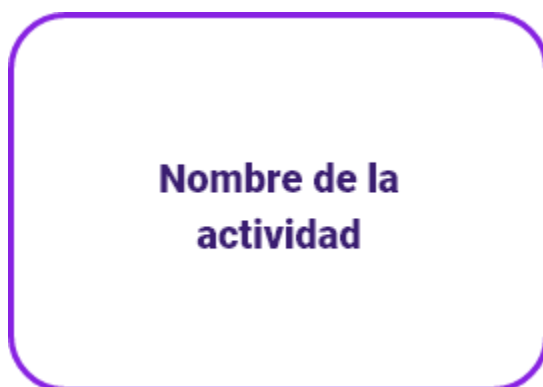
a) Actividad

Es la descripción o el detalle de una sucesión de conductas parametrizadas, simbolizada como un flujo de acciones ordenadas. El modelado de este flujo se realiza mediante nodos de actividad que se conectan a través de

flujos de control. Las actividades pueden generar jerarquías de peticiones, ya sea llamando a diferentes actividades o, como último recurso, solventando operaciones propias. La representación gráfica de las actividades se realiza mediante un rectángulo con bordes redondeados, dentro del cual se especifica el nombre de la actividad.

Para nombrar las actividades se usan verbos del modelo de negocio como, por ejemplo:

Figura 10. Nombre de la actividad



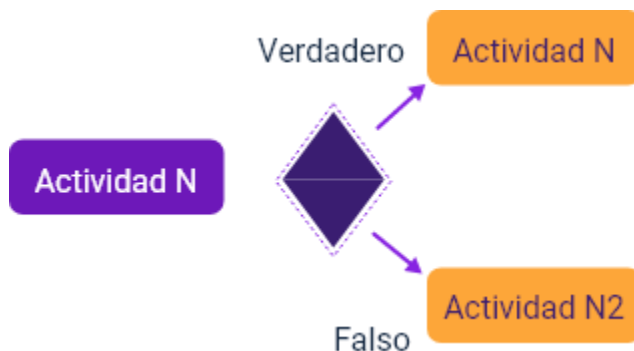
- Buscar factura
- Realizar pago
- Renovar lista

b) Flujo entre actividades

Este permite realizar los enlaces entre objetos y nodos dentro de una actividad, incorporando flujos de objetos y flujos de control. La representación gráfica de estos flujos se realiza mediante una flecha con punta abierta, que indica el orden en la ejecución de las actividades. Frecuentemente, se añade una descripción en la flecha para facilitar la comprensión.

Cuando los flujos de actividades incluyen condiciones para su ejecución, estos se representan mediante un rombo y se denominan nodos de decisión.

Figura 11. Flujo entre actividades



c) Nodo inicial

Como su nombre lo indica, el nodo inicial se encarga de empezar un flujo cuando se llama a una actividad. En cada diagrama existe un solo nodo inicial, el cual se representa con un círculo pequeño.

Figura 12. Nodo inicial



d) Nodo final

También es un nodo de control final. Su función es detener todos los flujos de una actividad. En una actividad pueden existir más de un nodo final. Este se representa con un rectángulo, como se presenta a continuación:

Figura 13. Nodo final

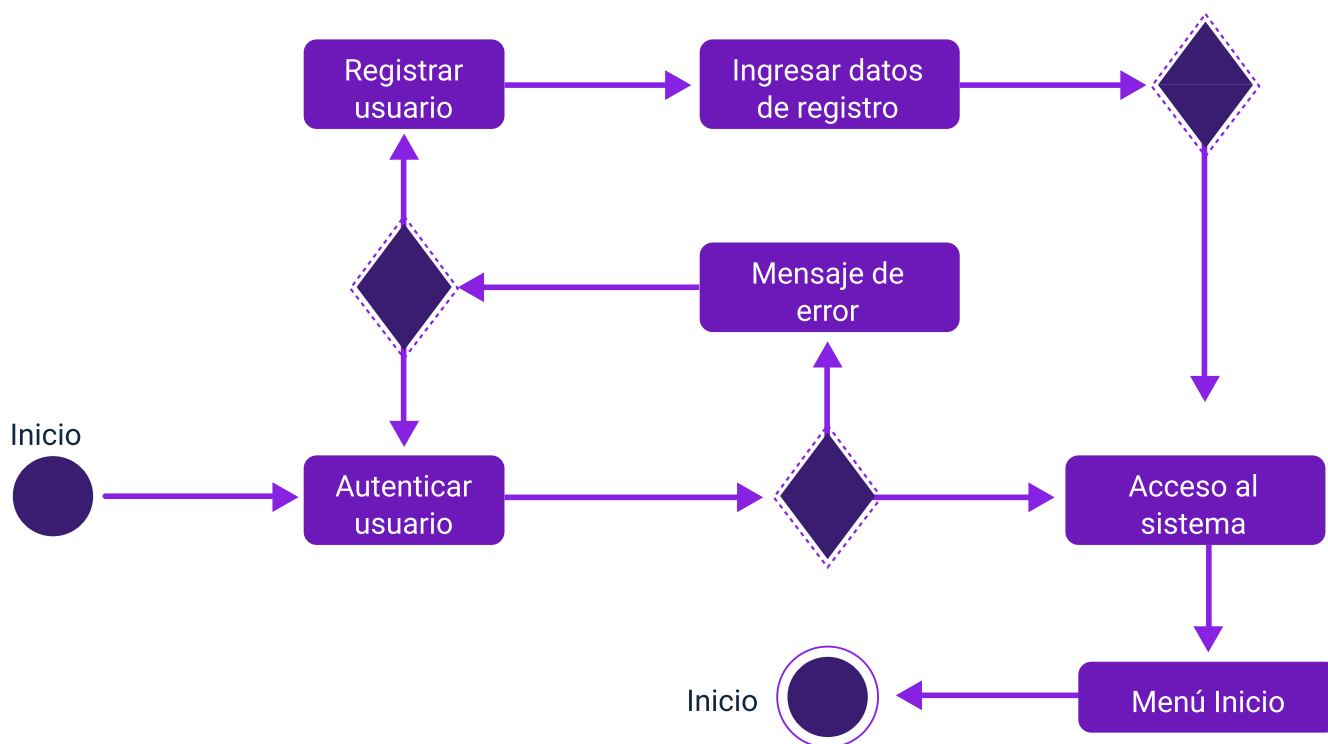


Después de realizar la descripción de los elementos que componen un diagrama de actividades, se presenta el siguiente ejemplo de un diagrama de clase sobre un proceso de registro e inicio de sesión.

Ejemplo de diagrama de actividades

A continuación, se presenta un ejemplo de un diagrama de actividades que incluye las funcionalidades de iniciar sesión y registrar en un sistema.

Figura 14. Ejemplo de diagramas de actividades.



Como se presenta en el ejemplo anterior, el flujo inicia con la actividad de autenticar usuario y pasa por un nodo de decisión que verifica si el usuario existe o no. Si el usuario existe, pasa a un nodo de decisión donde se valida el usuario; si no es

correcto, muestra un mensaje de error; si es correcto, da acceso a la aplicación y muestra el menú de inicio. En caso de no existir el usuario, pasa a la actividad de registrar usuario y después ingresa datos. Luego sigue a un nuevo nodo de decisión, donde válida los datos y da acceso a la aplicación, continuando el flujo de actividad mostrando la interfaz del menú del sistema y finalmente termina la actividad con un nodo final.

5. Diagramas de clase

Un diagrama de clases es un tipo de diagrama estático que describe la estructura de un sistema mostrando sus clases, atributos y las relaciones entre ellas. Se utiliza durante el proceso de análisis y diseño de sistemas para crear el diseño conceptual de la información que se manejará y los componentes que se encargarán de su funcionamiento y la interacción entre ellos. En un diagrama de clases, se pueden distinguir principalmente dos elementos: las clases y sus relaciones (Flores Cueto & Bertolotti Zúñiga, 2013).

El objetivo de un diagrama de clases es representar las clases que conforman el modelo de un sistema determinado. Este diagrama se construye y se refina durante las fases de análisis y diseño, y sirve como referencia en la implementación del sistema.

Los diagramas de clases pueden utilizarse desde tres perspectivas diferentes:

- **Conceptual**

El diagrama de clase representa los conceptos dentro del dominio del problema en estudio.

- **Especificación**

El diagrama de clase muestra las interfaces de las clases sin detallar su implementación. Las clases se asemejan a tipos de datos, representando una interfaz con múltiples implementaciones posibles.

- **Implementación**

Esta vista muestra las clases tal y como se manifiestan en el entorno de implementación.

Los elementos que componen un diagrama de clase son:

- **Clase**

Es la unidad básica que encapsula la información para definir un objeto (siendo un objeto una instancia de una clase). Mediante la clase, es posible modelar entidades del dominio de estudio, como una casa, un auto o una cuenta corriente.

- **Atributos**

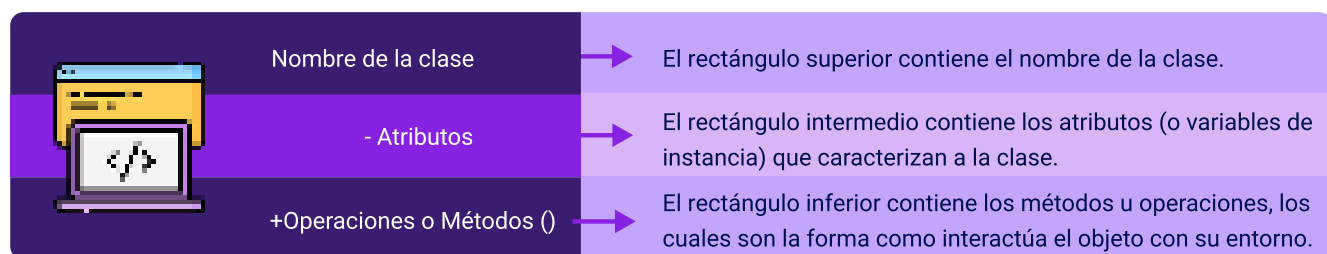
Son las características o propiedades específicas de una clase.

- **Métodos**

Son las operaciones o funciones a través de las cuales una clase interactúa con sus datos.

En UML, una clase es representada por un rectángulo que posee tres divisiones, en donde:

Figura 15. Representación de una clase en UML



Los atributos y los métodos de las clases tienen características que definen su grado de comunicación y visibilidad con el entorno, a lo que se denominan controles de acceso.

Control de acceso

El control de acceso, también conocido como visibilidad, determina la accesibilidad de los atributos o métodos de una clase. Los atributos y métodos pueden ser privados, protegidos o públicos. Cada uno de estos niveles de acceso se representa con los símbolos especificados a continuación:

- **Público**

Se define un atributo o método como público cuando debe ser visible o accesible por todas las clases.

Símbolo: -

- **Privado**

Se define un atributo o método como privado cuando su visibilidad o acceso se restringe exclusivamente a la clase en la que se declara.

Símbolo: +

- **Protegido**

Se define un atributo o método como protegido cuando se desea que sea accesible solamente desde la clase que lo define y desde aquellas clases que heredan de ella.

Símbolo: #

Por ejemplo, se puede representar una clase denominada Producto que contiene tres atributos: código de tipo entero, nombre de tipo String y precio de tipo double. Además, incluye dos métodos: obtenerPrecio y calcularTotal. En esta representación, todos los atributos de la clase son privados, lo cual se indica con el símbolo -, y los métodos son públicos, identificados con el símbolo +.

- Producto
 - codigo: int
 - nombre: String
 - precio: double
 - + obtenerPrecio()
 - + calcularTotal()

Relaciones entre clases

Las relaciones entre las diferentes clases indican cómo interactúan los objetos pertenecientes a esas clases. Existen distintos tipos de relaciones:

- Asociación (conexión entre clases).
- Dependencia (relación de uso).
- Generalización (relaciones de herencia).

5.1. Asociaciones

La relación de asociación se refiere a las conexiones entre clases. Este tipo de relación se representa gráficamente mediante una línea que conecta dos clases y presenta las siguientes características:

- **Nombre de la asociación**

Es necesario asignar un nombre a la asociación, que es obligatorio para establecer el contexto de la relación.

- **Rol**

Cada clase tiene un rol asociado que describe su función en la asociación.

Los roles son opcionales y no se necesitan si el nombre de la clase describe claramente su papel en la relación.

- **Navegabilidad**

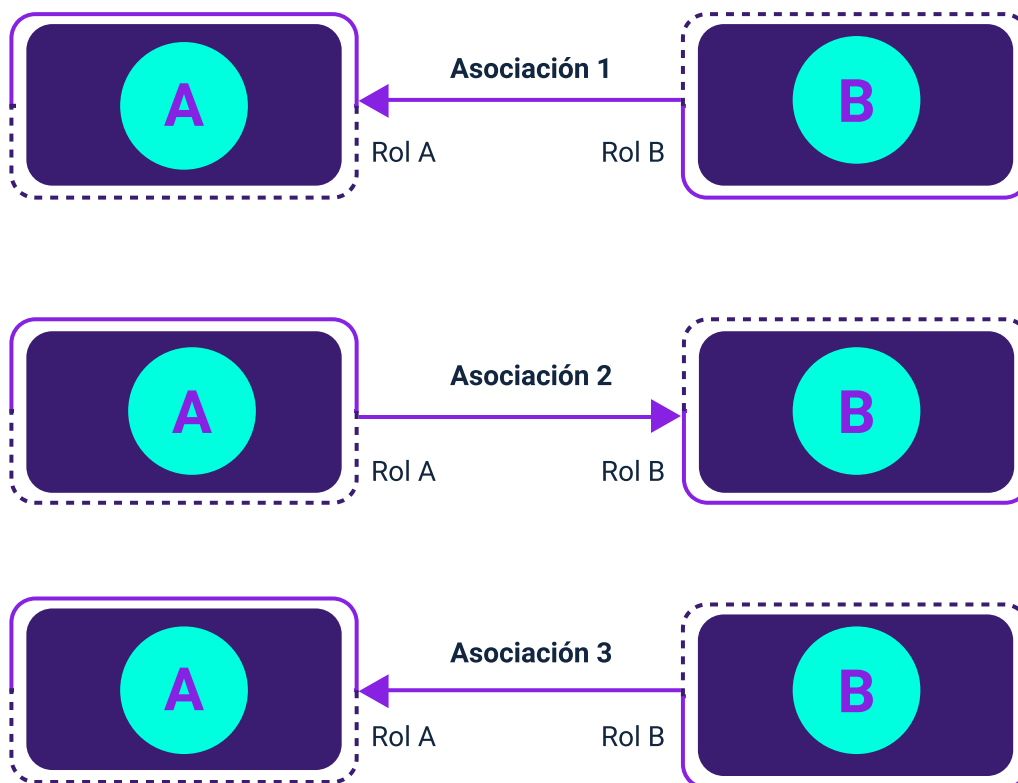
Define si la relación es unidireccional o bidireccional. Se representa gráficamente con flechas que indican la dirección de la asociación.

- **Multiplicidad**

Indica el número de instancias de una clase que pueden asociarse con una instancia específica de la otra clase en la relación.

A continuación, se presentan algunos ejemplos gráficos:

Figura 16. Ejemplo de asociación



En el ejemplo anterior, se pueden identificar tres diferentes ejemplos de asociación entre la clase A y la clase B, en todos los ejemplos la Clase A asume el rol A en el contexto de asociación, mientras que la clase B asume el rol B en el contexto de la relación. Adicionalmente, se identificó que los dos primeros ejemplos presentan navegabilidad unidireccional, lo que indica que la relación se establece en una sola dirección. En el primer caso, la 'Asociación1' vincula la Clase B, como origen, con la Clase A, como destino. En el segundo caso, 'Asociación2' vincula la Clase A, como origen, con la Clase B, como destino. En el último ejemplo, el vínculo es bidireccional; es decir, funciona en ambos sentidos: desde la Clase A hacia la Clase B y viceversa.

Una relación de asociación debe contar, como mínimo, con tres elementos:

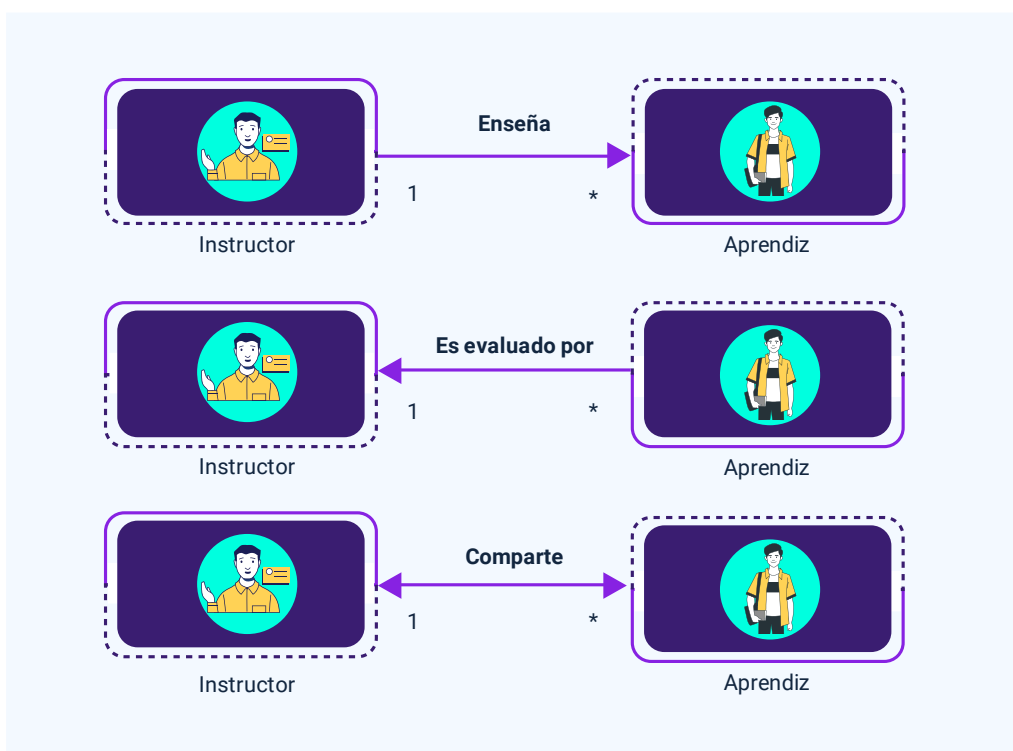
- El nombre de la asociación
- La navegabilidad
- La multiplicidad.

En cuanto a la multiplicidad, esta indica el número de instancias de las clases que participan en la relación. Aunque este número puede variar, es común utilizar el valor de '1' o '*' para denotar múltiples instancias.

Para facilitar la comprensión y la interpretación, es importante considerar la navegabilidad. La lectura se realiza tomando como referencia una instancia de la clase de origen de la asociación con respecto al valor 'N' de instancias de la clase de destino, donde 'N' representa la multiplicidad en el extremo opuesto.

Por ejemplo, se podría representar una clase llamada 'Figura' que contiene dos atributos ('lado1' y 'lado2') y tres métodos (un constructor 'Figura', un método 'área' y un método 'perímetro') de la siguiente manera:

Figura 17. Ejemplo de variaciones de asociación entre clases



En el ejemplo anterior, se presentan tres variaciones de relaciones de asociación entre la clase Instructor y la clase Aprendiz. En los tres casos, la navegabilidad varía en función de lo que se representa en la relación, tal como se indica en su nombre.

- **En el primer caso**

Existe una relación de asociación denominada Enseña, con una navegabilidad que va del Instructor al Aprendiz, indicando que un instructor enseña a varios aprendices.

- **En el segundo caso**

La relación se interpreta de la siguiente manera: un aprendiz es evaluado por un instructor.

- **En el tercer caso**

De esta representación gráfica, se ilustra una relación bidireccional; es decir, se pueden realizar lecturas desde cada extremo. En este ejemplo, un instructor comparte conocimientos con muchos aprendices y un aprendiz comparte experiencias con un instructor.

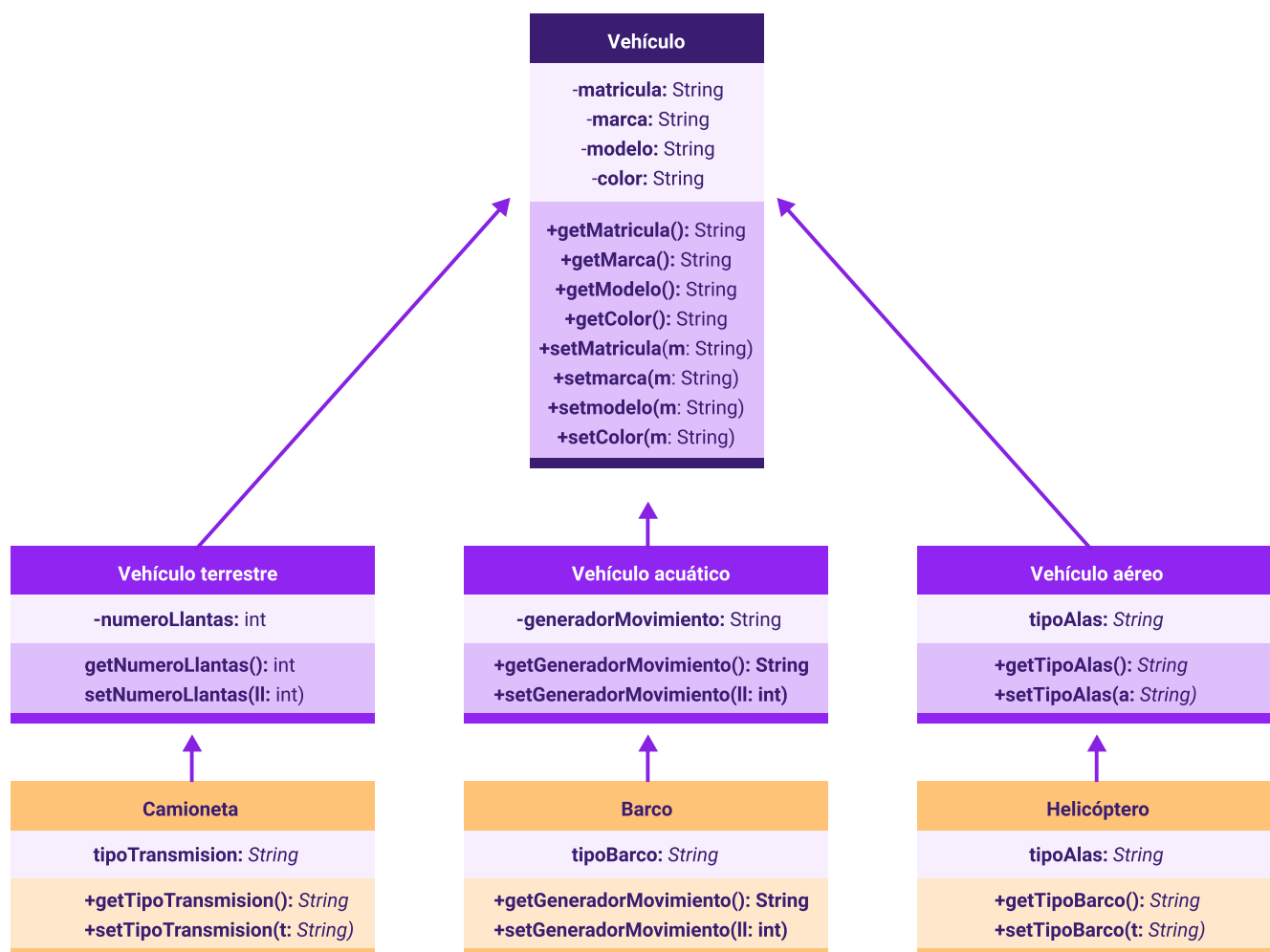
5.2. Herencia

La herencia es una de las relaciones más comunes en la práctica del paradigma orientado a objetos. Como se mencionó anteriormente, se pueden construir relaciones que van desde conceptos (clases) generales a conceptos (clases) específicos (especialización), o de conceptos específicos a generales (generalización), según Schmuller (2001).

La herencia es un tipo especial de asociación que cuenta con clases principales o superclases (que son las más generales en la relación) y clases secundarias o subclases (que son las más especializadas). Este tipo de relación tiene implícito el nombre 'es un' o 'es una'. También, implícitamente, es una relación que vincula una instancia de la superclase con una instancia de la subclase; por lo tanto, no se asignan nombres ni multiplicidad a este tipo de relación.

En una relación de herencia, las subclases heredan las características (atributos) y comportamientos (métodos) de las superclases.

Figura 18. Ejemplo de herencia

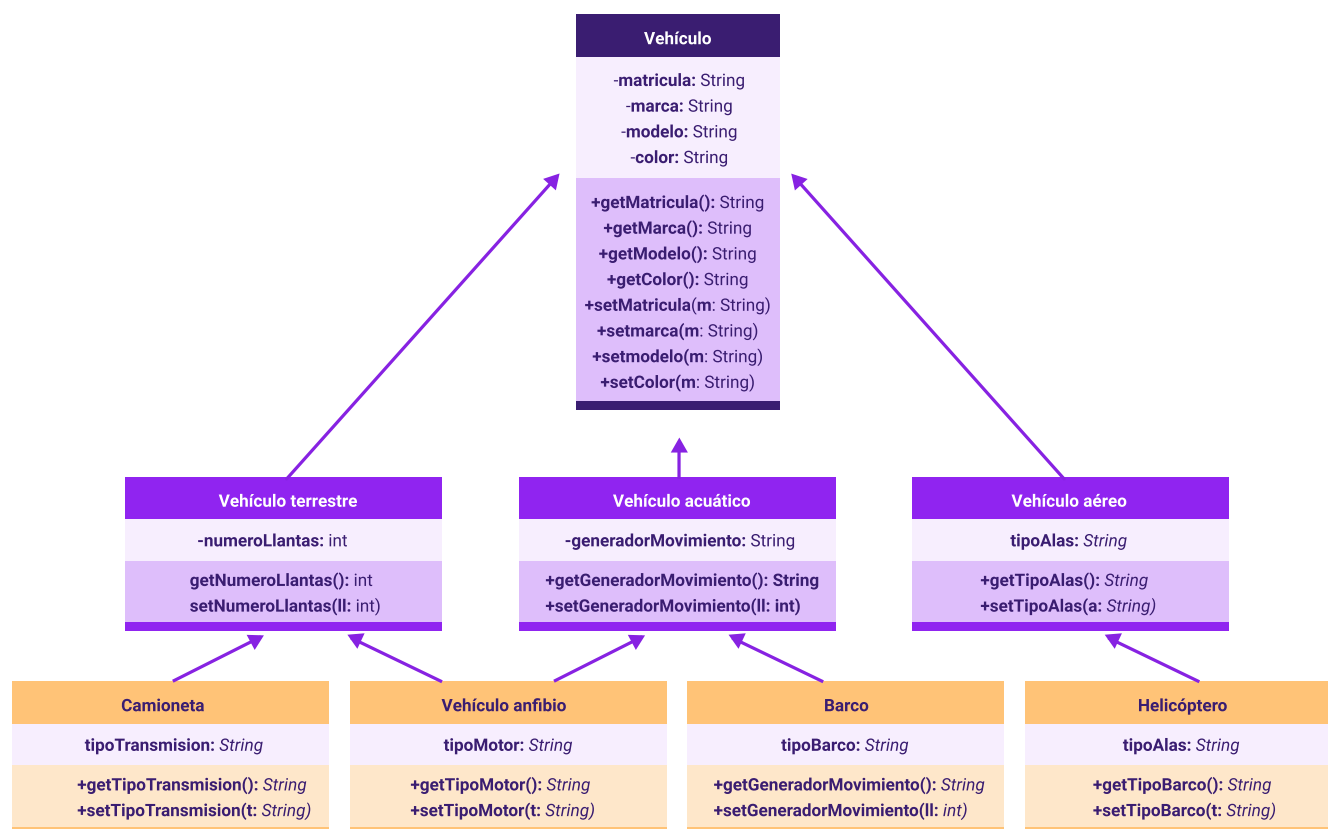


En la figura anterior, se presenta una relación de herencia entre siete (7) clases distintas. Las clases de tipo 'supertipo' están situadas en los niveles superiores de la jerarquía y representan las clases más generales cuyas definiciones serán heredadas por las subclases. En las relaciones de herencia, se pueden establecer varios niveles, y en cada nivel existen la correspondiente superclase y sus subclases. Es importante notar que no se asigna un nombre a estas relaciones porque se entiende implícitamente como una relación del tipo 'es un' o 'es una', la cual se interpreta desde las subclases hacia las superclases y siempre en relaciones uno a uno.

Por ejemplo, una Camioneta es un Vehículo Terrestre, que a su vez es un Vehículo. La clase Vehículo Terrestre hereda todas las características y comportamientos de la clase Vehículo. A su vez, la clase Camioneta hereda todas las características y comportamientos de la clase Vehículo Terrestre. De manera indirecta, la clase Camioneta está heredando las características y comportamientos de la clase Vehículo al estar en la misma línea de herencia. La clase Barco no hereda las características de la clase Vehículo Terrestre porque no se encuentra en la misma línea de herencia.

Hay dos tipos de herencia en función de la cantidad de clases supertipo directas asociadas: si una clase hereda características de un solo supertipo, es una herencia simple; si una clase hereda características de más de un supertipo, es una herencia múltiple.

Figura 19. Ejemplo de herencia múltiple



En el ejemplo de herencia múltiple, el diagrama de clases presenta varias relaciones de herencia. Por ejemplo, la clase Vehículo Anfibio es un caso de herencia múltiple, ya que hereda características de dos clases supertipo directas y diferentes: Vehículo Terrestre y Vehículo Acuático.

La herencia múltiple es admitida solo por algunos lenguajes de programación, siendo Python, Perl y C++ los más conocidos que la implementan. Sin embargo, otros lenguajes orientados a objetos ofrecen funcionalidades similares a la herencia múltiple mediante el uso de interfaces.

Visibilidad en las relaciones de herencia

Para definir la visibilidad de los atributos y métodos de una clase, se utilizan los identificadores de acceso: `private`, `public` y `protected`. En el contexto de las relaciones de herencia, es crucial ser cuidadoso para evitar resultados no deseados, respetando las siguientes normas:

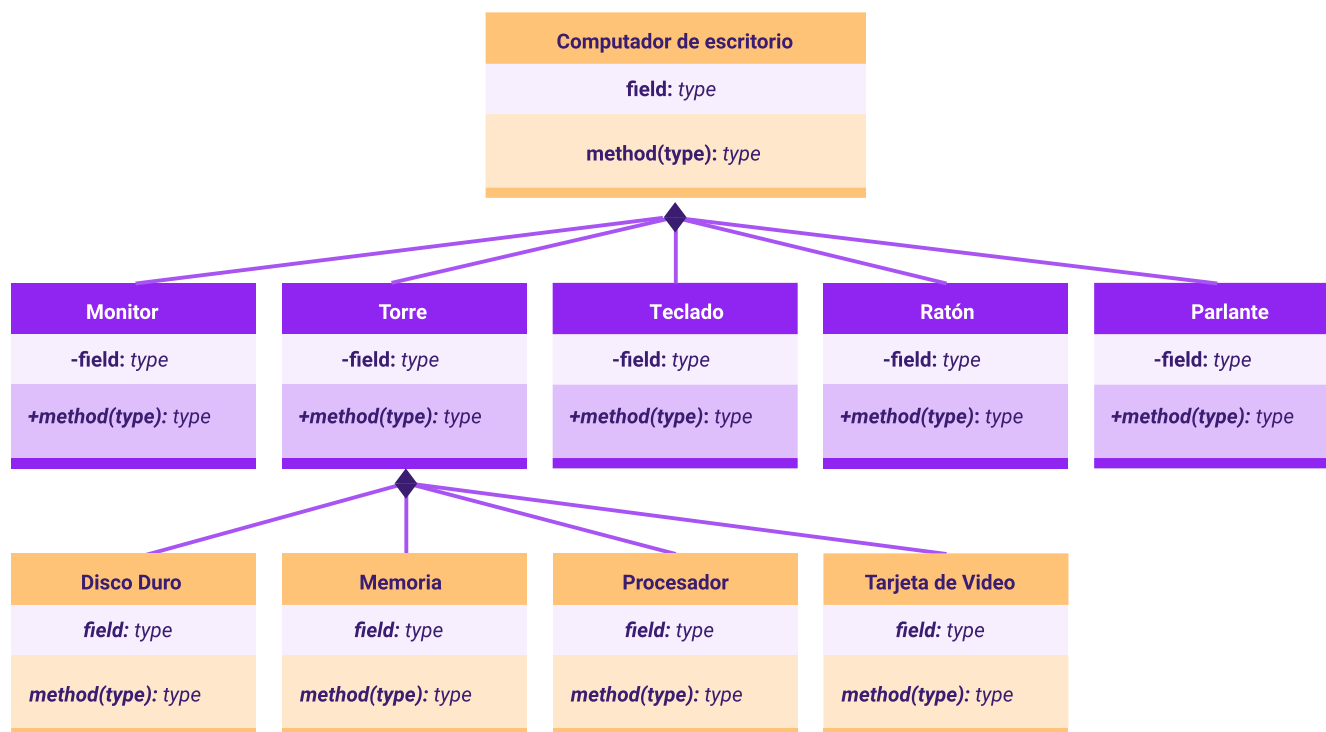
- Un atributo o método `private` en una superclase es inaccesible desde las subclases
- Un atributo o método `public` en una superclase permanece público en las subclases.
- Un atributo o método `protected` en una superclase se mantiene protegido en las subclases si existen más niveles de herencia, o actúa como privado en caso contrario.

5.3. Agregación

La agregación es un tipo de asociación que indica que un conjunto de clases forma un todo. Por ejemplo, existen clases agregadas que simbolizan el todo y están compuestas por un conjunto de clases componentes.

Esta relación se representa con una línea que conecta la clase agregada con sus clases componentes, teniendo un rombo en el lado de la clase agregada que representa la relación de agregación. Aunque esta relación no tiene un nombre explícito, ya que se comprende la existencia de clases agregadas y componentes, es necesario establecer la multiplicidad en el lado de los componentes.

Figura 20. Ejemplo de relación de Agregación



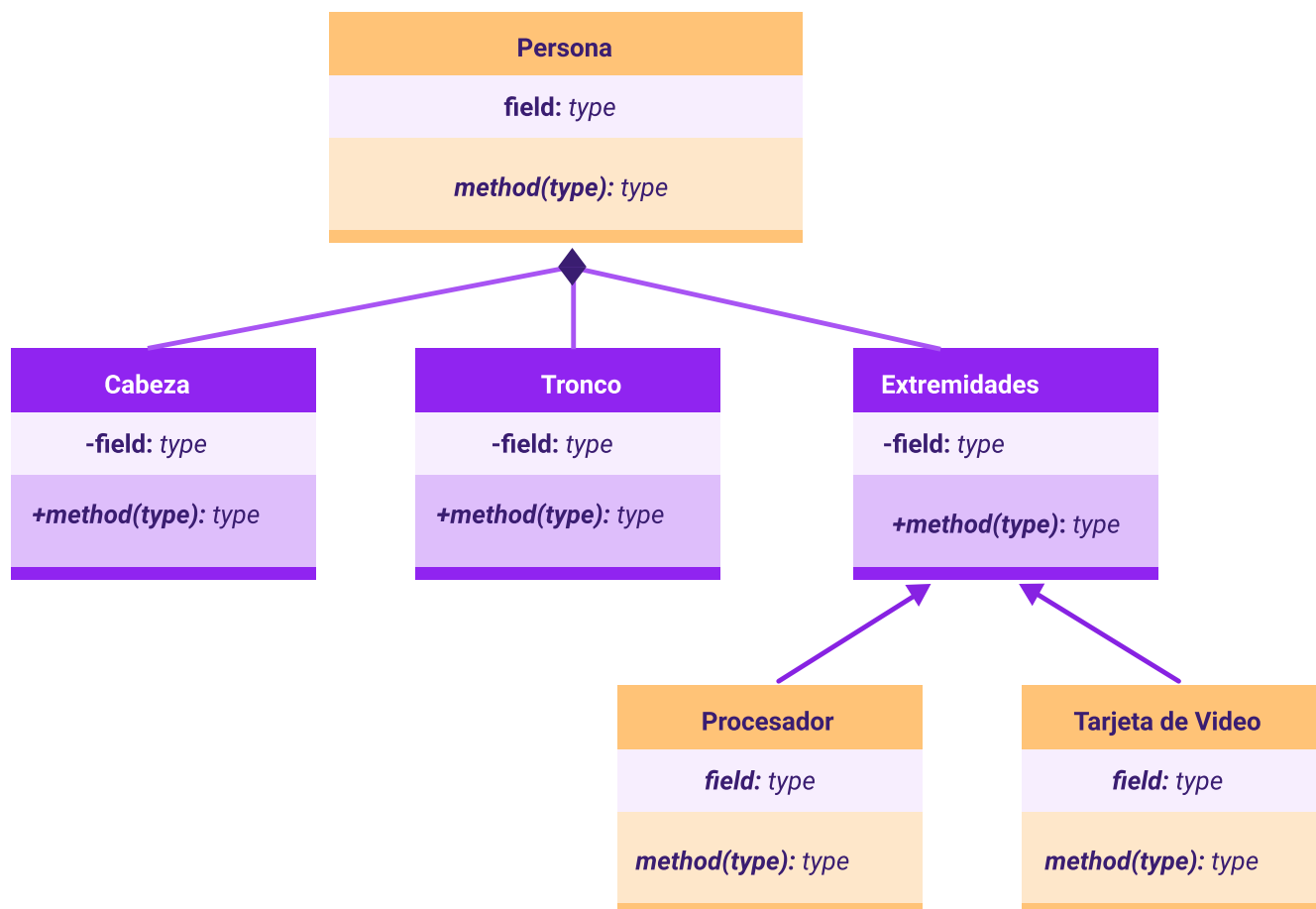
En la figura se presenta un ejemplo de relación de agregación. Como se puede apreciar, la clase Computador de Escritorio está compuesta por un monitor, una torre, un teclado, un ratón y dos parlantes. Además, la clase Torre consta de varios discos duros, memorias, un procesador y una tarjeta de video. Los componentes en este diagrama pueden ser partes de otras clases. Asimismo, una instancia de cualquier componente puede ser sustituida con facilidad, lo cual sugiere que la asociación con la clase agregada no es muy fuerte, característica distintiva de una agregación.

5.4. Composición

La composición es un tipo particular de relación de agregación donde los componentes no pueden pertenecer a más de una relación de agregación; es decir, son exclusivos de la composición establecida. Para distinguir una relación de composición

de una de agregación, el rombo representativo se rellena completamente de negro.
(Schmuller, 2001).

Figura 21. Ejemplo de relación de Composición



En la figura anterior, se identifica un ejemplo de composición. En este caso lo que se representa es una relación mucho más fuerte que la agregación ya que los elementos que componen la clase agregada no pueden formar parte de otra clase.

6. Herramientas CASE

Según Pressman, en el pasado, la ingeniería de software era principalmente una actividad manual, donde las herramientas se utilizaban solo en las etapas finales del proceso. Sin embargo, los ingenieros de software contemporáneos destacan la necesidad de herramientas más sofisticadas que las manuales para satisfacer las demandas de los sistemas informáticos actuales (Pressman, 1998).

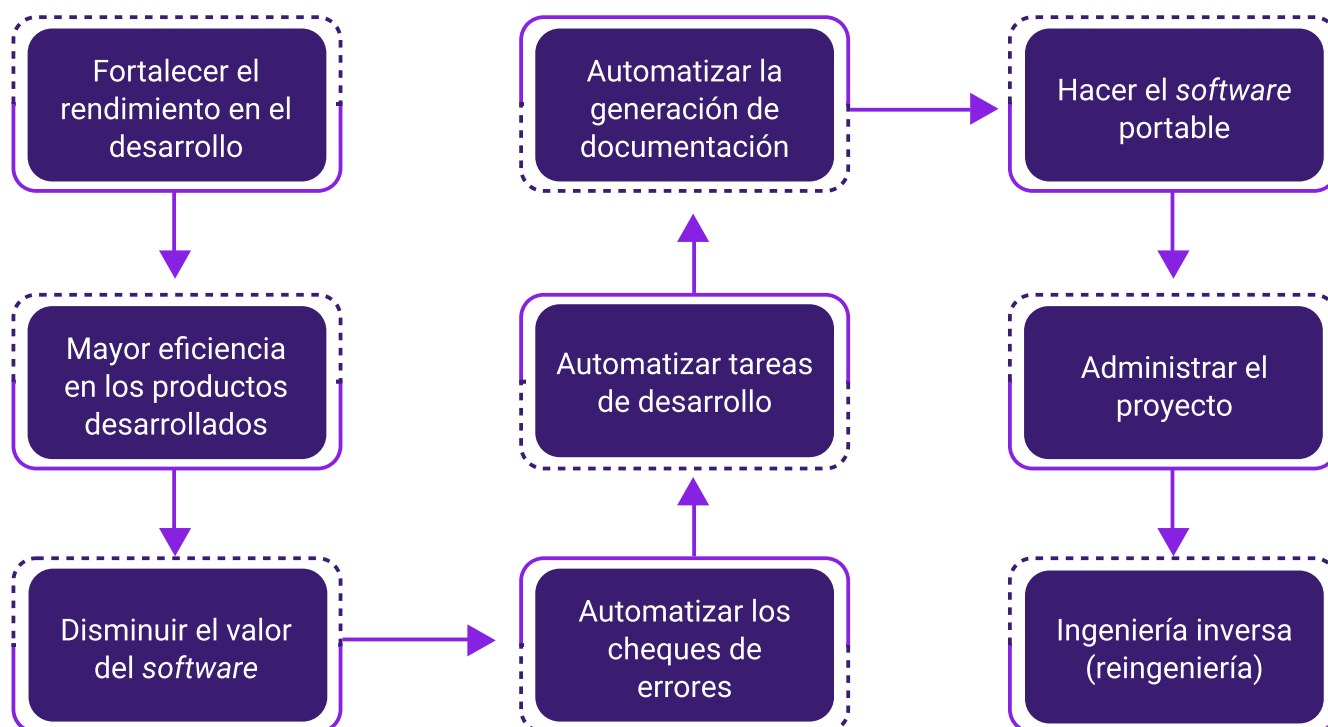
Las herramientas CASE son un complemento en la caja de herramientas de un ingeniero de software proporcionando la capacidad de automatizar tareas que tradicionalmente se ejecutaban de forma manual. Esto ayuda a garantizar que la calidad se integre desde la fase de diseño, mucho antes de proceder a la construcción del producto.

CASE es un acrónimo de "Computer Aided Software Engineering", que traducido al español significa "Ingeniería de Software Asistida por Computadora".

La tecnología CASE se refiere a la ingeniería de Software apoyada por herramientas computacionales. Consiste en un conjunto de herramientas que automatizan las tareas de desarrollo de software, con la finalidad de ofrecer un sistema integrado que conecte y automatice las fases del ciclo de vida del desarrollo (Cuevas, 1991).

De acuerdo con Pressman (1998), los objetivos de las herramientas CASE son:

Figura 22. Objetivos de la herramienta CASE



Clasificación de las Herramientas CASE

Las herramientas CASE en función de las fases del ciclo de vida que abarcan, se pueden agrupar de la siguiente forma:

- **Herramientas integradas, I-CASE (Integrated CASE, CASE integrado)**

Abordan todos los aspectos del ciclo de vida del desarrollo de sistemas.

Son llamadas también CASE de trabajo en equipo.

- **Herramientas de alto nivel, U-CASE (Upper CASE - CASE superior) o front-end**

Se utilizan para la automatización y soporte de las actividades de las etapas tempranas del desarrollo: análisis y diseño.

- **Herramientas de bajo nivel, L-CASE (Lower CASE - CASE inferior) o back-end**

Ayudan a las etapas finales en el ciclo de vida del desarrollo: construcción e implementación.

- **Juegos de herramientas o Tools-Case**

Son el tipo más simple de herramientas CASE. Automatizan una tarea o actividad del ciclo de vida del desarrollo de sistemas que se convierte en herramienta o herramientas independientes, constituyendo un caso de multi-herramienta.

A continuación, se presentan ejemplos de las herramientas CASE por ámbitos:

- **Análisis y diseño**

Workbench

GUI Design Studio

DIA

StarUML

- **Diseño de archivos y base de datos**

Power Designer

Oracle Designer

EasyCase

- **Programación**

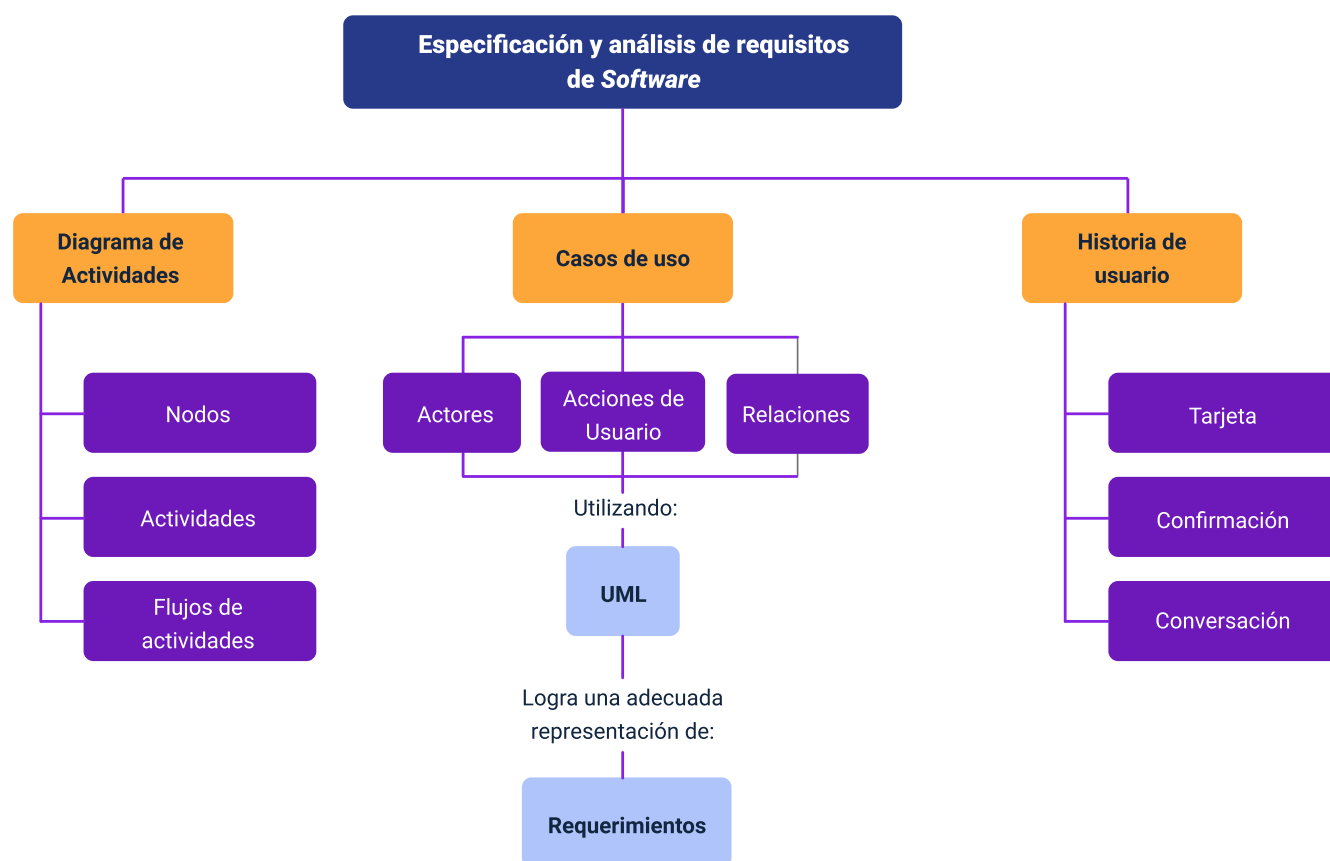
NetBeans

Eclipse/Omondo

Los diagramas de clase se diagraman con las herramientas que hacen parte del grupo de Herramientas de alto nivel, U-CASE, entre las que encontramos StartUML, Lucichart, ArgoUML, MagicDraw.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Diagrama de casos	Nicosiored. (2017, November 22). Diagrama de Casos de Uso II - 5 - Tutorial UML en español. YouTube.	Video	https://youtu.be/DUjBnEvlm1M?feature=shared
Diagrama de actividades	Nicosiored. (2018, January 31). Diagrama de Actividades - 15 - Tutorial UML en español. YouTube.	Video	https://youtu.be/GoYdpOVhDRc?feature=shared
Diagramas de clase	TodoCalls & DataScience (2017).Curso UML Diagrama de Clases. YouTube.	Video	https://www.youtube.com/watch?v=7WRYH2ei5Rw&t=106s
Lista Herramientas Case	Herramientas CASE (2008, July 11). Herramientas CASE más utilizadas.	Página web	https://herramientascase.wordpress.com/las-mas-utilizadas/

Glosario

Ágil: conjunto de tareas o acciones para producir y mantener productos y lograr objetivos de proceso. incluye procedimientos, estándares, políticas y objetivos para crear y modificar productos de trabajo.

CASE: ingeniería de software asistida por computación.

Herramienta CASE: una herramienta software que automatiza una parte de las fases del ciclo del desarrollo de sw.

Método: indica cómo construir técnicamente el software, incluyendo técnicas de modelado y otras técnicas descriptivas.

Metodología: colección de métodos para resolver un tipo de problemas.

Requerimiento: petición de algo que se solicita.

Requisito: condición que debe cumplir algo, en general, esto cumple con el requerimiento.

Stakeholders: individuo u organización que comparte, reclama o le interesa un sistema, o le compete una característica que satisface sus necesidades y expectativas.

Tecnología CASE: conjunto de instrumentos y técnicas software para automatizar una disciplina de la ingeniería, incluyendo metodologías estructuradas y herramientas.

Referencias bibliográficas

Booch, G. (1994). Object-oriented analysis and design. Redwood City.

Booch, G., Rumbaugh, J., Jacobson, I., Molina, J. J. G., & Martínez, J. S. (2006). El lenguaje unificado de modelado: guía del usuario. Addison-Wesley.

Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley Professional.

Cohn, M. (2018). User Stories and User Story Examples by Mike Cohn. [online] Mountain Goat Software. <https://www.mountaingoatsoftware.com/agile/user-stories>

Cuevas Agustín, Gonzalo. (1991). Ingeniería del Software. Práctica de la programación, Editorial RA-MA, 1 impresión.

Flores Cueto, J. J., & Bertolotti Zúñiga, C. (2013). Diagrama de clases en uml. 6.

Gutiérrez, J. (s.f.). Diagramas UML de casos de uso y de requisitos.

Larman, C. (2002). "Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process". 2nd Ed. Prentice Hall.

Norris & Rigby. (1994). "Ingeniería de software explicada", 1 edición Editorial Megabyte-Noriega editores, México.

Pressman, Roger. (1998). Ingeniería del Software. Un enfoque práctico, Editorial Mc. Graw-Hill, 4ta edición, 1998.

Rumbaugh, J., Jacobson, I., & Booch, G. (2004). El Lenguaje Unificado de Modelado. Elements, 30.

Schmuller, J. (2001). Aprendiendo UML en 24 horas. Pearson Educación.

Schmuller, J., & Garza Marín, A. D. (2000). Aprendiendo UML en 24 horas.

Wake, W.C. (2003). "Extreme Programming Explored" y "Refactoring Workbook"

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suárez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Gestión de requisitos

Breve descripción:

Este componente formativo aborda la ingeniería de requisitos y el levantamiento de requerimientos para prever el comportamiento del software. Utiliza casos de uso, historias de usuario y Lenguaje de Modelado Unificado para modelar funcionalidades e interacciones. Además, introduce herramientas CASE, esenciales para visualizar y construir software.

Junio 2024

Tabla de contenido

Introducción	4
1. Características del Lenguaje Unificado de Modelado UML	6
Versión oficial de UML	7
Características generales del UML	7
2. Especificación y análisis de requisitos	10
Diagrama de casos de uso	10
Actor.....	11
2.1. Relaciones de casos de uso.....	16
Especificación de casos de uso	19
2.2. Casos de uso reales (Prototipos)	21
3. Historias de usuario	24
Criterios de aceptación:.....	25
4. Diagrama de actividades.....	31
Elementos de un diagrama de actividades.....	31
Ejemplo de diagrama de actividades	34
5. Diagramas de clase	36
Control de acceso	38
Relaciones entre clases.....	39

5.1.	Asociaciones	39
5.2.	Herencia	43
	Visibilidad en las relaciones de herencia.....	47
5.3.	Agregación.....	47
5.4.	Composición	48
6.	Herramientas CASE.....	50
	Clasificación de las Herramientas CASE.....	51
	Síntesis	54
	Material complementario.....	55
	Glosario	56
	Referencias bibliográficas	57
	Créditos.....	59

Introducción

La especificación de requisitos es uno de los procesos más importantes en las fases de desarrollo de software. Constituye la fase inicial y es crucial para que el proyecto culmine exitosamente. Su objetivo principal es comprender qué esperan los usuarios y clientes del sistema, empleando un lenguaje natural para obtener una perspectiva clara y evitar errores costosos en las fases subsiguientes del desarrollo del producto.

El análisis de requerimientos propone un conjunto de técnicas e instrucciones que facilitan una representación precisa y completa de las funcionalidades que debe ejecutar un sistema para satisfacer los requerimientos de clientes y usuarios. Actualmente, algunas de las técnicas más utilizadas incluyen los diagramas de casos de uso, historias de usuario y diagramas de actividades.

Para la construcción de software, es fundamental la utilización de modelos, ya que representan la dirección y aspecto de una necesidad específica. Esta "necesidad" puede estar ya existente, en desarrollo o en fase de planificación. Los diseñadores del modelo deben levantar los requerimientos del producto, que pueden abarcar áreas como funcionalidad, rendimiento y fiabilidad. El modelo se estructura en varias vistas, cada una describiendo un aspecto específico del producto o sistema en desarrollo. Un modelo permite documentar la estructura y comportamiento de un sistema antes de ser codificado.

Este componente formativo presentará diversas técnicas y diagramas para el análisis y especificación de requisitos, con el fin de organizar adecuadamente los requerimientos. Esto permite una representación efectiva de cada una de las

interacciones del usuario con el sistema, resultando en una especificación eficaz que reduce costos y riesgos en el desarrollo del software, y contribuye a alcanzar un producto de calidad.

1. Características del Lenguaje Unificado de Modelado UML

Con el nacimiento de la Programación Orientada a Objetos (POO), se establecieron una variedad de lenguajes, métodos y simbologías utilizadas para detallar el desarrollo de diferentes sistemas. Indudablemente, esta situación generó desconcierto entre los programadores de software. Para solucionar este inconveniente, surgió el Lenguaje de Modelado Unificado (Unified Modeling Language, UML).

- **El lenguaje de Modelado Unificado UML**

Proporciona un conjunto de herramientas que facilitan el modelado (análisis y diseño) de sistemas orientados a objetos. Establece una serie de normas técnicas que representan de manera gráfica el camino y la estructura de un sistema determinado para programadores, analistas, desarrolladores, diseñadores, etc.

El UML es una de las herramientas más emocionantes en el mundo actual del desarrollo de sistemas. Esto se debe a que permite a los creadores de sistemas generar diseños que capturen sus ideas de forma convencional y fácil de comprender, facilitando la comunicación de estas a otras personas. (Schmuller, 2000, p.24)

- **Evolución del Lenguaje Unificado de Modelado UML**

El UML tuvo su origen a finales de los años 80, inicialmente basado en el método Booch, desarrollado por el diseñador de software Grady Booch. En 1994, el científico de la computación James Rumbaugh implementó el modelado de objetos en este método. En 1995, se añadieron a estas bases

otros principios y metodologías, algunas de ellas introducidas por Ivar Jacobson. (Booch, 2006).

Versión oficial de UML

La primera versión oficial, UML 1.0, se lanzó en 1997 bajo el liderazgo de Booch, Rumbaugh y Jacobson. Esta versión ofrecía una metodología que incluía varios estándares con el principal objetivo de asegurar la coherencia entre todos los diagramas. En 2005, se anunció la versión UML 2.0, cuyo contenido estaba principalmente orientado hacia la visión de la Programación Orientada a Objetos. Desde entonces, se han realizado nuevas versiones basadas en UML 2.0.

- Grady Booch, ingeniero de sistema e investigador, antes del 95.
- Jim Rumbaugh (OMT), antes del 95.
- Ivar Jacobson, UML 0.8 y 0.91 1996.

Características generales del UML

Como se mencionó anteriormente, el UML es un lenguaje que proporciona un conjunto de reglas que facilitan la comunicación de un sistema mediante representaciones gráficas, indicando cómo realizar la creación y lectura de los modelos. Aunque existen muchas características del UML, según Booch (2006), se pueden resumir en las siguientes, las cuales se detallan a continuación:

a) Visualizar

UML facilita la representación de un sistema de forma gráfica, con el fin de que se pueda entender fácilmente por otra persona.

b) Especificar

UML permite detallar cuáles son las necesidades de un sistema previo a su construcción.

c) Construir

La construcción del sistema diseñado se realiza tomando los modelos especificados en el lenguaje UML.

d) Documentar

Cuando se diseñan las notaciones gráficas del sistema desarrollado, estas se utilizan como documentación. Esto permite brindar soporte para nuevas revisiones o empalmes, facilitando que los nuevos desarrolladores comprendan el sistema.

La versión 1.0 de UML fue lanzada en enero de 1997 y desde entonces ha sido utilizado con éxito en la construcción de sistemas para diversas industrias alrededor del mundo, incluyendo hospitales, bancos, comunicaciones, aeronáutica y finanzas, entre otras. UML es actualmente el lenguaje de modelado de sistemas de software más conocido y utilizado, permitiendo visualizar, especificar, construir y documentar sistemas con una perspectiva orientada a objetos (Rumbaugh et al., 2004).

Figura 1. Modelo para la construcción de software UML



UML está compuesto por diversos elementos gráficos que se combinan para conformar diagramas. Entre los tipos de diagramas que ofrece UML, se encuentra el diagrama de clase, que es parte de los diagramas de estructura. Estos últimos enfatizan los elementos que deben existir en el sistema de modelado.

2. Especificación y análisis de requisitos

La función más importante de la especificación de requisitos es servir de intermediario para que los clientes, los analistas de requisitos, los desarrolladores y los usuarios puedan comunicarse eficazmente. En esta fase es fundamental recolectar tanto los requerimientos del usuario y del cliente como los del software a desarrollar, para lograr la satisfacción del cliente. Por ello, es crucial el uso de técnicas que permitan representar de manera visual e intuitiva estos requerimientos.

Cuando se realiza la recolección de requerimientos de un cliente, el UML facilita el modelado de estos a través de sus casos de uso. Los stakeholders (partes interesadas) que están involucrados en el producto son incluidos en el modelado con las funcionalidades requeridas (casos de uso), que a su vez son modelados incluyendo relaciones y asociaciones jerárquicamente organizadas entre ellos.

Los casos de uso y sus actores son quienes especifican los requerimientos de un cliente, detallando las expectativas que cada uno tiene del sistema, sin centrarse únicamente en la funcionalidad que se implementará. Los análisis de requisitos se realizan para diversos procesos, no necesariamente solo para desarrollos de software. Entre los diagramas más relevantes y utilizados en UML se encuentran los diagramas de casos de uso.

Diagrama de casos de uso

Los casos de uso permiten describir cada una de las funciones de las aplicaciones a desarrollar desde el punto de vista del usuario. Su utilidad se centra en expresar las funcionalidades que el producto de software debe ejecutar y, definir a las personas que serán responsables de realizar estas funcionalidades. Esta técnica implica la

creación de un diagrama con su respectiva notación. El caso de uso representa el comportamiento del software en la interacción con el usuario, con el objetivo de que este alcance una meta específica. Por tanto, un caso de uso se compone de los elementos que forman parte de su notación.

- **Componentes de los diagramas de casos de uso**

La notación de un diagrama de caso de uso incluye tres componentes principales que facilitan la representación simbólica de las funcionalidades del sistema, así como de las personas involucradas y las interacciones entre ellas.

Estos componentes son: los actores, que son los usuarios o roles que interactúan con el sistema; los casos de uso, que describen las funciones del sistema; y las relaciones, que especifican las conexiones entre actores y casos de uso.

Actor

Define un rol desempeñado por un usuario, representa a una persona o grupo de personas que interactúan con un software. Se simboliza mediante la figura de un "stick man", junto al nombre del actor, que es obligatorio. Los actores se utilizan para identificar la clase de usuario que participará en el sistema y será responsable de ejecutar determinadas funciones. Por lo general, los nombres de los actores comienzan con mayúscula.

- **Principales**

Son actores especificados para identificar los objetivos de los usuarios, los cuales son guiados por los casos de uso.

- **De apoyo**

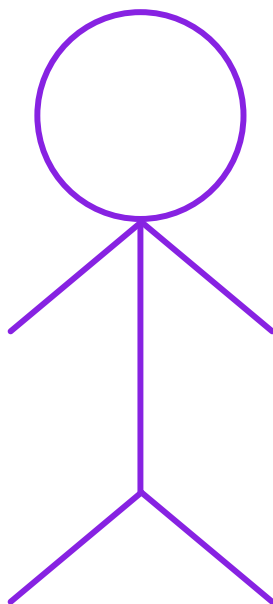
Proveen servicios al sistema informático y son definidos para esclarecer las interfaces externas.

- **Los pasivos**

Se interesan en el funcionamiento de los casos de uso, aunque no son considerados actores principales.

En la figura que se presenta a continuación, se identifica cómo se representa de manera gráfica un actor en un UML, al cual también se le adiciona el nombre del actor correspondiente para dar mayor claridad en el proceso.

Figura 2. Representación gráfica de un actor en UML



Además, es importante considerar que los actores no son necesariamente personas, aunque en los diagramas de casos de uso se representen mediante formas humanas. En este contexto, los sistemas externos también cumplen el rol de actores,

pues pueden requerir información del sistema real o interactuar con él, como ocurre con una base de datos.

Al determinar los casos de uso, resulta esencial identificar al actor. Este término se refiere al usuario cuando desempeña un rol específico, implicando que dicho usuario ejecuta los casos de uso. Dado lo anterior, cada actor tiene la capacidad de participar en diversos casos de uso y, a su vez, cada caso de uso puede involucrar a diferentes actores. Para identificar a un actor, basta con visualizar los roles que interactúan dentro del sistema.

Ejemplo 1:

Para explicar cada uno de los elementos de un caso de uso, se presenta el siguiente requerimiento:

Una empresa solicita un sistema que facilite el proceso de generación de facturas de venta para comerciantes. Estas facturas deben ser enviadas a través de un sistema externo encargado del proceso de facturación.

Los actores identificados en los requisitos del ejemplo 1 son el comerciante y el sistema externo.

a) Relación

La interacción entre un actor y un caso de uso se denota mediante una relación, simbolizada por una línea o una flecha. Se describen varios tipos de relaciones según los actores mencionados anteriormente.

- La relación de comunicación entre un actor y un caso de uso puede ser activa o pasiva.

- Un actor activo inicia la ejecución, con la flecha apuntando hacia el caso de uso correspondiente.
- Un actor pasivo, en cambio, no inicia la ejecución y la flecha apunta hacia él.
- El tipo de comunicación o relación entre actores y casos de uso establece una asociación directa.
- Esta asociación especifica que el actor es el ejecutor del proceso descrito.

Figura 3. Representación gráfica de una relación de caso de uso en UML



Continuando con el ejemplo 1 se identifican las siguientes relaciones de comunicación:

- Entre el comerciante y la generación de la factura.
- Entre el comerciante y el envío de la factura.
- Entre la factura y el sistema externo.

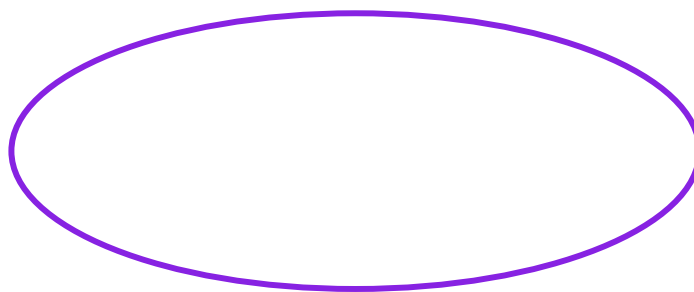
b) Caso de uso

Los casos de uso se identifican mediante los verbos o acciones que el actor ejecuta o ejecutará. Estas acciones se describen y representan de la siguiente manera:

- Se simbolizan con un óvalo, indicando una función que el sistema debe proporcionar.

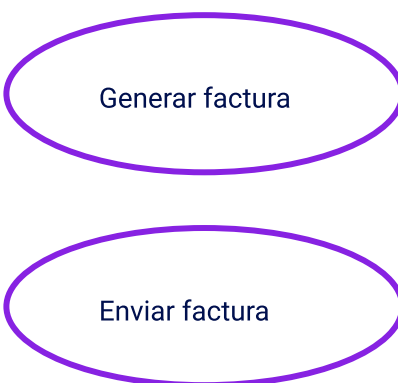
- El caso de uso debe ser de fácil lectura e interpretación, tanto para los desarrolladores como para el cliente. Para el nombre, se recomienda utilizar un verbo en infinitivo acompañado de un complemento.

Figura 4. Representación caso de uso



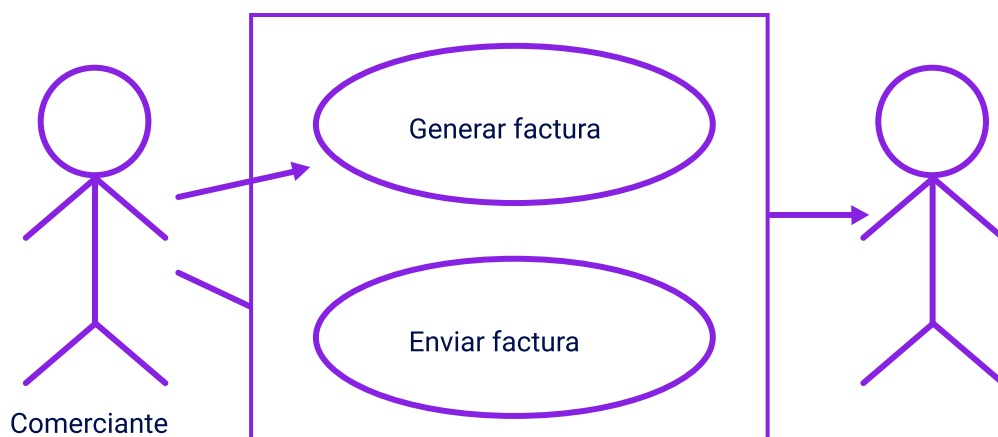
Los casos de uso que se identifican para el ejemplo 1 son: generar factura y enviar factura, los cuales se ven representados gráficamente en la siguiente figura.

Figura 5. Ejemplo caso de uso



A continuación, se presenta un ejemplo de la representación gráfica del diagrama de caso de uso, que incluye los actores, relaciones y el caso de uso del problema planteado en el Ejemplo 1.

Figura 6. Representación ejemplo 1



Por otro lado, los pasos para diseñar el diagrama de casos de uso del sistema de ventas, mostrado en la figura anterior, están detallados a continuación:

- Se identifican los casos de uso según requerimientos, para el caso del ejemplo son: (generar factura, enviar factura).
- Se identifican los posibles actores que interactuaran con el sistema: para el caso del ejemplo: (comerciante y sistema externo).
- Se identifican las relaciones de comunicación entre comerciante y los casos de uso, generar factura y enviar factura.

Además de la relación básica de comunicación mostrada anteriormente, existen otras relaciones en los casos de uso que detallan con mayor precisión las interacciones entre ellos.

2.1. Relaciones de casos de uso

Durante el proceso de desarrollo, se pueden definir diferentes casos de uso que no son independientes entre sí. Es posible establecer relaciones de dependencia mutua entre los casos de uso. Las principales relaciones consideradas por UML son:

- **Inclusión (include)**

Se emplea para extraer comportamientos comunes de un caso de uso. Un caso de uso puede incluir a otro.

- **Extensión (extend)**

Utilizada para ampliar las funcionalidades de un caso de uso existente, es especialmente útil para incorporar nuevos requisitos del sistema.

- **Generalización (generalization)**

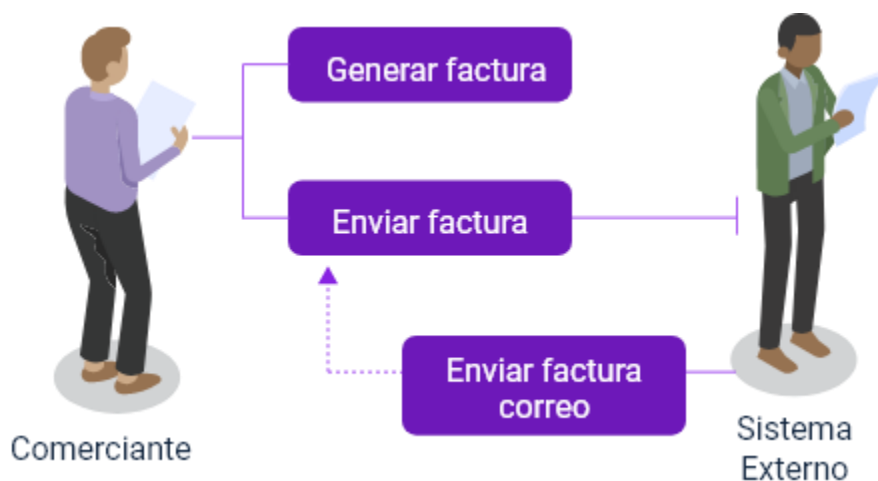
Esta relación ocurre entre un caso de uso general y uno más específico, donde el específico hereda propiedades del caso de uso base.

Tomando como referencia el Ejemplo 1, con el objetivo de perfeccionar el diagrama de casos de uso, se pueden introducir nuevos casos de uso que ilustren las relaciones mencionadas anteriormente.

- **Ejemplo extends**

Se establece la relación extends cuando un caso de uso es similar a otro, pero incluye funciones adicionales. Por ejemplo, en el caso mencionado anteriormente, el caso de uso enviar factura podría ampliarse a enviar factura por correo electrónico. Ambos casos implican la misma acción básica de enviar la factura; sin embargo, en la extensión, además se envía un correo electrónico con la factura. Esto se representaría de la siguiente manera:

Figura 7. Relación extends

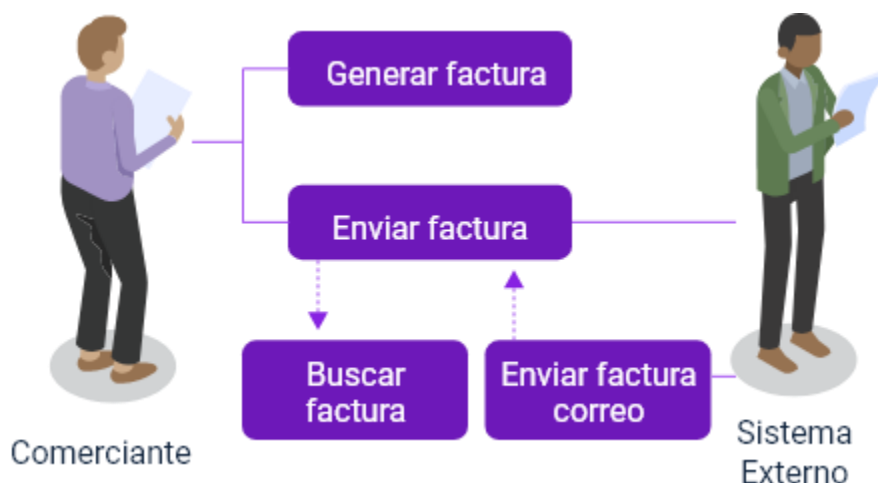


La relación se interpreta así: el caso de uso enviar factura por correo extiende del caso de uso enviar factura.

- **Ejemplo include**

Otro aspecto importante a considerar se presenta en ejemplo include; esta relación se define cuando parte del comportamiento o acción es común en más de un caso de uso, eliminando la necesidad de duplicar dichas acciones. Por ejemplo, en el caso del Ejemplo 1, se añade el siguiente requisito funcional: antes de enviar la factura, el usuario puede realizar una búsqueda de facturas para seleccionar y enviar. Esto hace necesario incluir la relación include. La representación sería la siguiente:

Figura 8. Relación include



Especificación de casos de uso

La especificación de casos de uso se refiere al proceso de descripción detallada de cada caso, incluyendo los flujos y eventos que interactúan con el sistema y los actores involucrados. Generalmente, el analista encargado del diseño identifica los requerimientos y los documenta.

El propósito de la especificación de casos de uso incluye:

- Describir la interacción de un actor con el sistema.
- Explicar el rol que desempeñan los actores en su interacción con el sistema y sus flujos.
- Detallar los casos de uso.

A continuación, se presenta las partes esenciales y las directrices básicas para la documentación de casos de uso.

Tabla 1. Plantilla de casos de uso

Componente	Descripción de una especificación de caso de uso
Nombre del caso de uso	Indica el nombre del caso de uso. Normalmente, el nombre expresa el resultado objetivo y observable del caso de uso, como por ejemplo "Imprimir factura" en el caso de un sistema de ventas.
Autor	Persona que diseña el caso de uso.
Fecha	Aquí se diligencia la fecha en la que se realizó el caso de uso.
Descripción	Se realiza una corta descripción del caso de uso.
Actores	Describe los nombres del actor o actores del caso de uso.
Precondiciones	Son aquellas condiciones que deben existir para que se cumpla el caso de uso.
Flujo normal	Son los pasos normales de ejecución del caso de uso.
Flujo alternativo	Flujos alternativos de ejecución de casos de uso.
PosCondiciones	Son aquellas condiciones que se cumplen cuando finaliza un caso de uso.

A continuación, se presenta un ejemplo de especificación de casos de uso para el caso de uso “Crear Foro” utilizando la plantilla propuesta posteriormente.

Tabla 2. Descripción de una especificación de caso de uso ejecutado

Componente	Descripción de una especificación de caso de uso
Nombre del caso de uso	Crear foro

Componente	Descripción de una especificación de caso de uso
Autor	Mario Vera Cruz
Fecha	20/09/2021
Descripción	Crea un nuevo mensaje.
Actores	Alumno/Docente.
Precondiciones	Al iniciar el usuario debe estar autenticado en el sistema.
Flujo normal	1. El actor da clic en la opción nuevo mensaje. 2. Se muestra un área de texto para escribir el mensaje. 3. El actor introduce el mensaje. 4. El sistema verifica datos y los almacena. 5. El profesor acepta y publica.
Flujo alternativo	4a. Se hace una comprobación de los datos, si no son correctos muestra mensaje de corrección. 4b. El profesor rechaza el mensaje de modo que no es publicado sino devuelto.
PosCondiciones	Se publica y almacena el mensaje satisfactoriamente.

2.2. Casos de uso reales (Prototipos)

El propósito principal de los casos de uso reales es detallar el proceso de un sistema de información descrito mediante un caso de uso, lo que incluye la interacción de objetos y la definición de las transacciones de las interfaces y clases de los diferentes procedimientos secundarios de diseño. Por lo tanto, es fundamental completar los

entornos recolectados del análisis con las clases de diseño correspondientes, sin dejar de lado las restricciones del entorno tecnológico.

Los casos de uso reales representan el diseño preciso del caso de uso desde una tecnología específica. Por ejemplo, si se trata de un diseño gráfico para el usuario, se incluirá el diseño de las ventanas y una descripción de la interacción con los objetos de la interfaz. Los casos de uso reales proporcionan diseños de pantalla y explicaciones sobre la interacción entre los objetos de la interfaz.

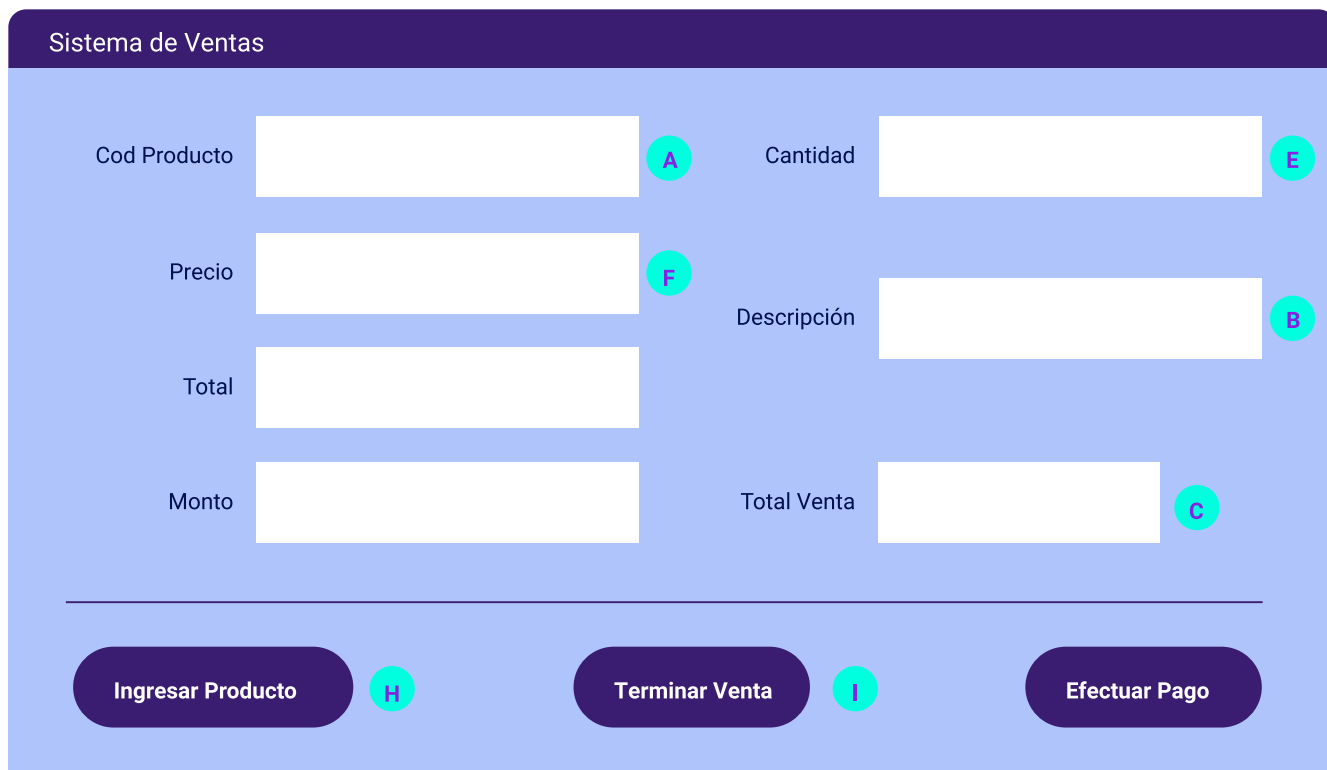
Para aclarar las definiciones anteriores, se utiliza un ejemplo de un sistema de ventas que se presentará posteriormente. En este ejemplo, se realiza el diseño del caso de uso real y se describe la interacción entre los artefactos del diseño. A continuación, se presenta una tabla con los datos de un ejemplo para un sistema de ventas:

Tabla 3. Sistema de ventas

Caso de uso adquirir productos	Adquirir productos
Actores	Cliente (quien inicia el proceso), Cajero.
Tipo	Primario, real.
Descripción	• En el almacén de ropa el cliente se dirige a la caja registradora con los productos a comprar. • El empleado-cajero hace el registro de los productos que el cliente compra y recibe el dinero en efectivo. • Al finalizar el proceso, el cliente se retira con los artículos comprados.

A continuación, se presenta el ejemplo del sistema de ventas:

Figura 9. Ejemplo Sistema de ventas



The screenshot shows a web application titled "Sistema de Ventas". It has a light blue background. At the top, there's a dark blue header with the title. Below the header, there are several input fields arranged in two columns. The left column has fields for "Cod Producto" (labeled A), "Precio" (labeled F), "Total" (labeled C), and "Monto". The right column has fields for "Cantidad" (labeled E), "Descripción" (labeled B), and "Total Venta" (labeled C). At the bottom, there are three buttons: "Ingresar Producto" (labeled H), "Terminar Venta" (labeled I), and "Efectuar Pago".

- El cliente se acerca a la caja registradora con los artículos que desea llevar.
- El cajero ingresa el código del producto en A de la Ventana-1.
 - Opcional: Si el cliente tiene más de un producto, el cajero puede registrar la cantidad en E.
 - Una vez finalizado el registro de todos los productos, se presiona H.
- El sistema almacena los productos registrados.
 - La descripción y el precio del producto se muestran en B y en F de la Ventana-1.
- El cajero presiona el botón I para dar por finalizada la venta.
- El sistema realiza el cálculo y muestra en C el total de la venta.

3. Historias de usuario

Las historias de usuario son una forma sencilla de representar los requisitos de un sistema de información. Se crean en una o dos oraciones y se redactan en un lenguaje común y legible para el usuario. Estas historias, conocidas como User Stories, se han convertido en un estándar ampliamente utilizado en la definición de requisitos. Surgieron en XP (1999) y fueron impulsadas de manera concluyente por Mike Cohn en su libro "User Stories Applied: For Agile Software Development" (2004), estableciendo el patrón para definirlas. La simplicidad es la razón principal de su popularidad. Las historias de usuario están compuestas por tres elementos:

- Cómo [Rol del usuario]
- Quiero [Objetivo]
- Para poder [Beneficio]

A continuación, se hace la descripción de los elementos de una historia de usuario según Cohn, M. (2018):

a) Tarjeta (Card)

Es una frase que sintetiza una necesidad dada en una conversación entre el usuario y el dueño del producto. Escrita en una tarjeta, comúnmente sigue el formato rol-objetivo-beneficio. Su estructura es:

- Como [tipo de usuario] -> ¿quién?
- Quiero [necesidad] -> ¿qué?
- Para [beneficio esperado] -> ¿para qué? Con esta estructura simple de tres líneas se identifican: ¿Quién? (roles de valor), ¿Qué? (problema a resolver) y ¿Para qué? (objetivo a alcanzar).

b) Conversación

Es el proceso de comunicación entre las personas que requieren las necesidades (dueño del producto) y las personas que saben cómo solucionarlas (desarrolladores). Se realiza mediante preguntas y utilizando recursos como gráficas y prototipos para clarificar el contenido de las historias de usuario.

c) Confirmación

Es la verificación del acuerdo con relación a lo que se quiere construir. Permite validar si se terminó o no, o si se cumplió con lo esperado. Esta confirmación recibe el nombre de criterios de aceptación, los cuales ayudan a tener claro el contexto de la historia de usuario y a determinar fácilmente si está terminada o no.

A continuación se presenta un ejemplo, el usuario desea comprar una boleta para una película en el cine. los criterios de aceptación que se pueden establecer son:

Criterios de aceptación:

- a) Cuando se agoten las boletas, se debe mostrar un mensaje informando que no hay disponibilidad para la venta.
- b) El sistema debe validar la edad del comprador para permitir la venta de boletas según la clasificación de la película.
- c) Tras un pago en efectivo, el sistema debe mostrar el valor del cambio a devolver; si no hay cambio, deberá mostrar cero.

Cuando se realizan historias de usuario es muy importante cumplir con las siguientes reglas:

- Ser concisa
- Concreta
- Realizar criterios de aceptación claros y específicos

Así como estas reglas son necesarias, también lo es una buena redacción de las historias de usuario. Por este motivo, a continuación se presenta un método que facilita mejorar la calidad en la escritura de una historia de usuario: el método “Invest”.

Método invest

A continuación, se hace una descripción de cada uno de los elementos que intervienen en este método:

- **Independiente**

Las historias de usuario deben ser lo más independientes posible para evitar dependencias.

- **Negociable**

Las historias de usuario deben ser flexibles y negociables, siempre en acuerdo con el cliente.

- **Valiosa**

Es crucial que cada historia de usuario aporte valor al cliente. Idealmente, el cliente debería participar activamente en su redacción.

- **Estimable**

La estimación de la historia debe ser precisa y realizable, teniendo en cuenta el tamaño y la complejidad, para facilitar la planificación por parte del equipo.

- **Small (Pequeña)**

El tamaño de la historia de usuario debe ser pequeño para simplificar y mejorar la precisión de la estimación. Es ideal que se pueda completar en periodos cortos, como días o semanas.

- **Testeable (Comprobable)**

La historia de usuario debe ser comprobable, con criterios claros de aceptación para considerarse completada.

Al utilizar este método, se garantiza que las historias de usuario sean independientes entre sí, facilitando así su planificación y desarrollo sin sobrecargarlas con detalles técnicos que puedan limitar las negociaciones entre clientes y desarrolladores.

Ejemplos

A continuación, se presentan algunos ejemplos prácticos de historias de usuario aplicando el método invest:

a) Historia de usuario: “Crear Usuario”

En seguida, se presenta dos opciones:

Primera opción:

- **Rol:** como cliente de un supermercado.

- **Objetivo:** quiero hacer compras mediante una aplicación web.
- **Beneficio:** para poder comprar sin salir de casa.
- **Comentario:** en esta historia de usuario, se identifica claramente lo que el cliente desea hacer, aunque no se especifican los detalles de cómo se realizarán las compras en línea. Estos detalles podrían aclararse en reuniones posteriores. Esta historia cumple con las características del método INVEST: es sencilla, clara y satisface una necesidad específica, facilitando así la estimación del tiempo de desarrollo.

Segunda opción:

- **Rol:** como cliente de un supermercado.
- **Objetivo:** quiero acceder al registro mediante un formulario que me permita escribir mi correo electrónico y asignar una contraseña.
- **Beneficio:** para asegurar mi privacidad.
- **Comentario:** esta historia de usuario contiene especificaciones excesivas en la sección de objetivo y no clarifica suficientemente el beneficio. Se recomienda simplificar y aclarar el propósito para alinearse mejor con los criterios de INVEST.

b) Ejemplo de una historia épica

- **Rol:** como cliente de un supermercado.
- **Objetivo:** quiero ingresar a una página web.
- **Beneficio:** para hacer compras en línea.
- **Comentario:** esta historia no es independiente y se considera una Épica, porque es muy amplia y puede subdividirse en múltiples historias más

pequeñas y manejables. Para visualizar mejor una historia de usuario Épica, se recomienda consultar recursos de aprendizaje específicos sobre este tema.

- Hacer un registro
- Un ingreso
- Seleccionar productos
- Pagar, etc.

Plantillas de historias

Para visualizar una historia del usuario épica, lo invitamos a consultar el PDF "**Plantillas de historias**", el cual se encuentra en la carpeta Anexos.

Por lo anterior, tampoco se cumpliría con la característica de estimación, ya que no se podría determinar cuándo terminaría la historia. Como se identificó en los ejemplos anteriores, el método INVEST nos proporciona reglas para optimizar la redacción de las historias de usuario, permitiendo así satisfacer las necesidades reales del cliente.

Para realizar una buena gestión de proyectos, es habitual en las metodologías ágiles utilizar una serie de herramientas para crear las historias de usuario, tales como:

- Trello
- Jira Software
- Taiga
- Excel

c) Plantillas de historias de usuario

Las plantillas de historias de usuario pueden adaptarse según las necesidades específicas sin perder la flexibilidad y sencillez que caracteriza a una historia de usuario.

Los equipos de desarrollo colaboran utilizando herramientas en línea para compartir y trabajar conjuntamente en las plantillas. A continuación, se presenta un ejemplo del uso de una plantilla de Excel para la gestión de productos, descargada de la web.

Tabla 4. Ejemplo plantilla historia de usuario

ID de la historia	Rol	Funcionalidad	Razón/Resultado	Criterio de aceptación
Id.01	Como cliente	Quiero ver una lista de categorías de productos.	Para poder realizar búsquedas de productos por categorías.	Despliegue de listado de categorías a seleccionar con productos asociados.
Id.02	Como cliente	Quiero ver una lista de precios.	Para poder elegir el producto más barato.	Despliegue una lista ordenada de productos por precio de menor a mayor.

4. Diagrama de actividades

Estos son diagramas de comportamiento que se utilizan para representar una sucesión de actividades. Explican el flujo de operaciones desde el punto de inicio hasta el final, definiendo una variedad de caminos de decisiones en el desarrollo de eventos que abarca una actividad. Según Booch, G. (1994), estos diagramas permiten visualizar un caso de uso específico a un nivel más preciso, ilustrando el flujo de actividades definidas en un sistema.

A continuación, se describen los beneficios del diagrama de actividades:

Beneficios:

- Explica la lógica de un algoritmo.
- Representa cada uno de los procesos realizados en los diseños de casos de uso en UML.
- Instruye procesos entre el sistema y los usuarios en un flujo de trabajo o negocios.
- Aporta claridad, optimizando y facilitando los casos de uso complejos.

Elementos de un diagrama de actividades

Un diagrama de actividades contiene fundamentalmente los siguientes elementos:

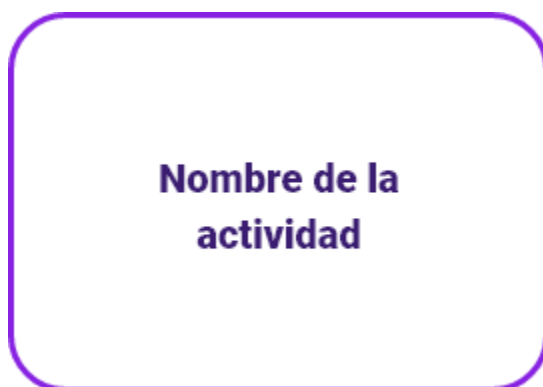
a) Actividad

Es la descripción o el detalle de una sucesión de conductas parametrizadas, simbolizada como un flujo de acciones ordenadas. El modelado de este flujo se realiza mediante nodos de actividad que se conectan a través de

flujos de control. Las actividades pueden generar jerarquías de peticiones, ya sea llamando a diferentes actividades o, como último recurso, solventando operaciones propias. La representación gráfica de las actividades se realiza mediante un rectángulo con bordes redondeados, dentro del cual se especifica el nombre de la actividad.

Para nombrar las actividades se usan verbos del modelo de negocio como, por ejemplo:

Figura 10. Nombre de la actividad



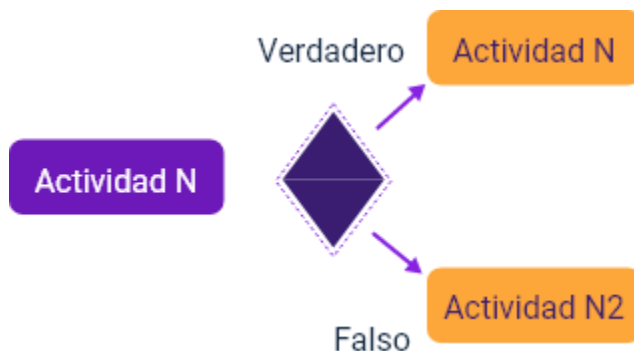
- Buscar factura
- Realizar pago
- Renovar lista

b) Flujo entre actividades

Este permite realizar los enlaces entre objetos y nodos dentro de una actividad, incorporando flujos de objetos y flujos de control. La representación gráfica de estos flujos se realiza mediante una flecha con punta abierta, que indica el orden en la ejecución de las actividades. Frecuentemente, se añade una descripción en la flecha para facilitar la comprensión.

Cuando los flujos de actividades incluyen condiciones para su ejecución, estos se representan mediante un rombo y se denominan nodos de decisión.

Figura 11. Flujo entre actividades



c) Nodo inicial

Como su nombre lo indica, el nodo inicial se encarga de empezar un flujo cuando se llama a una actividad. En cada diagrama existe un solo nodo inicial, el cual se representa con un círculo pequeño.

Figura 12. Nodo inicial



d) Nodo final

También es un nodo de control final. Su función es detener todos los flujos de una actividad. En una actividad pueden existir más de un nodo final. Este se representa con un rectángulo, como se presenta a continuación:

Figura 13. Nodo final

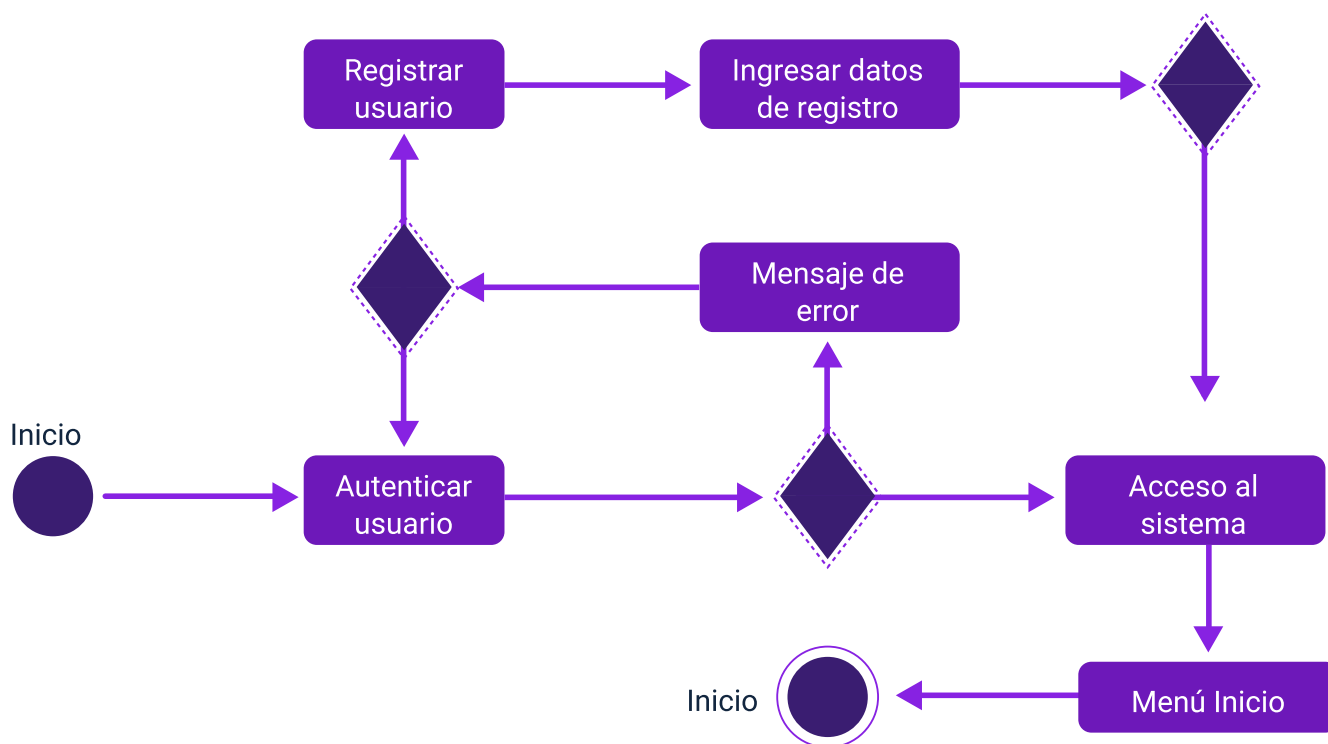


Después de realizar la descripción de los elementos que componen un diagrama de actividades, se presenta el siguiente ejemplo de un diagrama de clase sobre un proceso de registro e inicio de sesión.

Ejemplo de diagrama de actividades

A continuación, se presenta un ejemplo de un diagrama de actividades que incluye las funcionalidades de iniciar sesión y registrar en un sistema.

Figura 14. Ejemplo de diagramas de actividades.



Como se presenta en el ejemplo anterior, el flujo inicia con la actividad de autenticar usuario y pasa por un nodo de decisión que verifica si el usuario existe o no. Si el usuario existe, pasa a un nodo de decisión donde se valida el usuario; si no es

correcto, muestra un mensaje de error; si es correcto, da acceso a la aplicación y muestra el menú de inicio. En caso de no existir el usuario, pasa a la actividad de registrar usuario y después ingresa datos. Luego sigue a un nuevo nodo de decisión, donde válida los datos y da acceso a la aplicación, continuando el flujo de actividad mostrando la interfaz del menú del sistema y finalmente termina la actividad con un nodo final.

5. Diagramas de clase

Un diagrama de clases es un tipo de diagrama estático que describe la estructura de un sistema mostrando sus clases, atributos y las relaciones entre ellas. Se utiliza durante el proceso de análisis y diseño de sistemas para crear el diseño conceptual de la información que se manejará y los componentes que se encargarán de su funcionamiento y la interacción entre ellos. En un diagrama de clases, se pueden distinguir principalmente dos elementos: las clases y sus relaciones (Flores Cueto & Bertolotti Zúñiga, 2013).

El objetivo de un diagrama de clases es representar las clases que conforman el modelo de un sistema determinado. Este diagrama se construye y se refina durante las fases de análisis y diseño, y sirve como referencia en la implementación del sistema.

Los diagramas de clases pueden utilizarse desde tres perspectivas diferentes:

- **Conceptual**

El diagrama de clase representa los conceptos dentro del dominio del problema en estudio.

- **Especificación**

El diagrama de clase muestra las interfaces de las clases sin detallar su implementación. Las clases se asemejan a tipos de datos, representando una interfaz con múltiples implementaciones posibles.

- **Implementación**

Esta vista muestra las clases tal y como se manifiestan en el entorno de implementación.

Los elementos que componen un diagrama de clase son:

- **Clase**

Es la unidad básica que encapsula la información para definir un objeto (siendo un objeto una instancia de una clase). Mediante la clase, es posible modelar entidades del dominio de estudio, como una casa, un auto o una cuenta corriente.

- **Atributos**

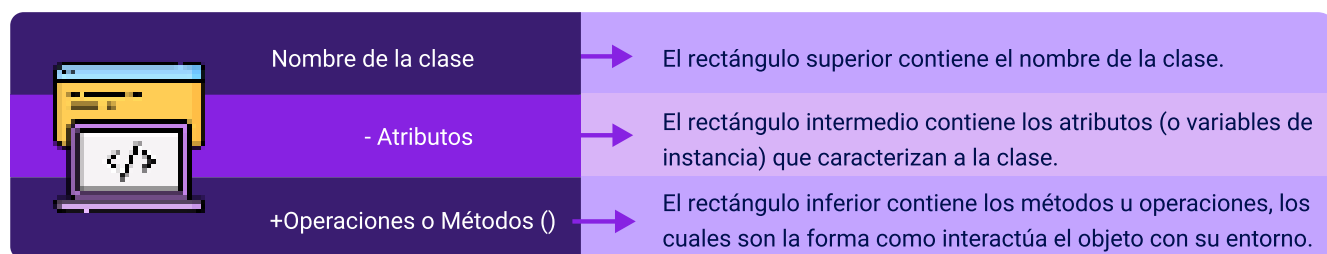
Son las características o propiedades específicas de una clase.

- **Métodos**

Son las operaciones o funciones a través de las cuales una clase interactúa con sus datos.

En UML, una clase es representada por un rectángulo que posee tres divisiones, en donde:

Figura 15. Representación de una clase en UML



Los atributos y los métodos de las clases tienen características que definen su grado de comunicación y visibilidad con el entorno, a lo que se denominan controles de acceso.

Control de acceso

El control de acceso, también conocido como visibilidad, determina la accesibilidad de los atributos o métodos de una clase. Los atributos y métodos pueden ser privados, protegidos o públicos. Cada uno de estos niveles de acceso se representa con los símbolos especificados a continuación:

- **Público**

Se define un atributo o método como público cuando debe ser visible o accesible por todas las clases.

Símbolo: -

- **Privado**

Se define un atributo o método como privado cuando su visibilidad o acceso se restringe exclusivamente a la clase en la que se declara.

Símbolo: +

- **Protegido**

Se define un atributo o método como protegido cuando se desea que sea accesible solamente desde la clase que lo define y desde aquellas clases que heredan de ella.

Símbolo: #

Por ejemplo, se puede representar una clase denominada Producto que contiene tres atributos: código de tipo entero, nombre de tipo String y precio de tipo double. Además, incluye dos métodos: obtenerPrecio y calcularTotal. En esta representación, todos los atributos de la clase son privados, lo cual se indica con el símbolo -, y los métodos son públicos, identificados con el símbolo +.

- Producto
 - codigo: int
 - nombre: String
 - precio: double
 - + obtenerPrecio()
 - + calcularTotal()

Relaciones entre clases

Las relaciones entre las diferentes clases indican cómo interactúan los objetos pertenecientes a esas clases. Existen distintos tipos de relaciones:

- Asociación (conexión entre clases).
- Dependencia (relación de uso).
- Generalización (relaciones de herencia).

5.1. Asociaciones

La relación de asociación se refiere a las conexiones entre clases. Este tipo de relación se representa gráficamente mediante una línea que conecta dos clases y presenta las siguientes características:

- **Nombre de la asociación**

Es necesario asignar un nombre a la asociación, que es obligatorio para establecer el contexto de la relación.

- **Rol**

Cada clase tiene un rol asociado que describe su función en la asociación.

Los roles son opcionales y no se necesitan si el nombre de la clase describe claramente su papel en la relación.

- **Navegabilidad**

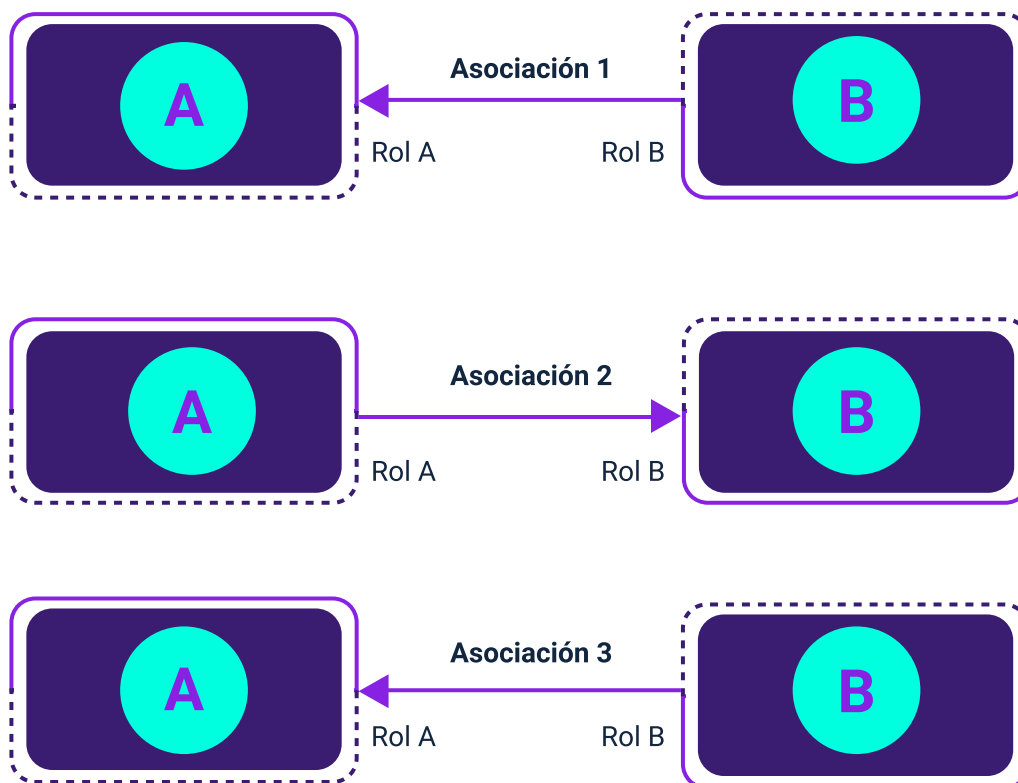
Define si la relación es unidireccional o bidireccional. Se representa gráficamente con flechas que indican la dirección de la asociación.

- **Multiplicidad**

Indica el número de instancias de una clase que pueden asociarse con una instancia específica de la otra clase en la relación.

A continuación, se presentan algunos ejemplos gráficos:

Figura 16. Ejemplo de asociación



En el ejemplo anterior, se pueden identificar tres diferentes ejemplos de asociación entre la clase A y la clase B, en todos los ejemplos la Clase A asume el rol A en el contexto de asociación, mientras que la clase B asume el rol B en el contexto de la relación. Adicionalmente, se identificó que los dos primeros ejemplos presentan navegabilidad unidireccional, lo que indica que la relación se establece en una sola dirección. En el primer caso, la 'Asociación1' vincula la Clase B, como origen, con la Clase A, como destino. En el segundo caso, 'Asociación2' vincula la Clase A, como origen, con la Clase B, como destino. En el último ejemplo, el vínculo es bidireccional; es decir, funciona en ambos sentidos: desde la Clase A hacia la Clase B y viceversa.

Una relación de asociación debe contar, como mínimo, con tres elementos:

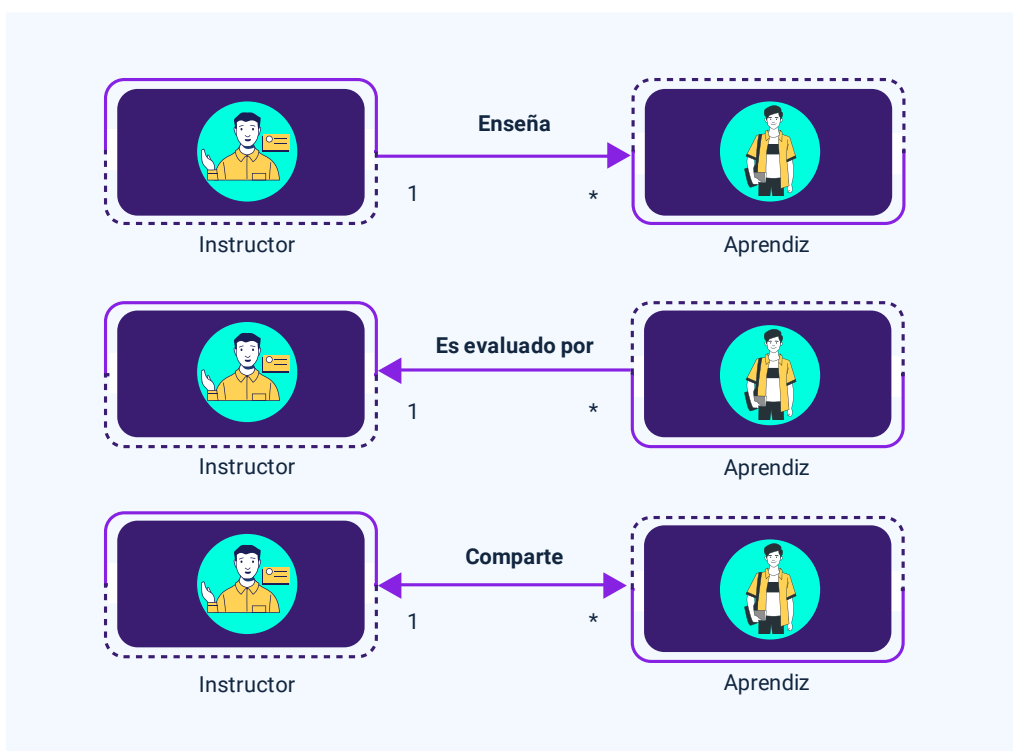
- El nombre de la asociación
- La navegabilidad
- La multiplicidad.

En cuanto a la multiplicidad, esta indica el número de instancias de las clases que participan en la relación. Aunque este número puede variar, es común utilizar el valor de '1' o '*' para denotar múltiples instancias.

Para facilitar la comprensión y la interpretación, es importante considerar la navegabilidad. La lectura se realiza tomando como referencia una instancia de la clase de origen de la asociación con respecto al valor 'N' de instancias de la clase de destino, donde 'N' representa la multiplicidad en el extremo opuesto.

Por ejemplo, se podría representar una clase llamada 'Figura' que contiene dos atributos ('lado1' y 'lado2') y tres métodos (un constructor 'Figura', un método 'área' y un método 'perímetro') de la siguiente manera:

Figura 17. Ejemplo de variaciones de asociación entre clases



En el ejemplo anterior, se presentan tres variaciones de relaciones de asociación entre la clase Instructor y la clase Aprendiz. En los tres casos, la navegabilidad varía en función de lo que se representa en la relación, tal como se indica en su nombre.

- **En el primer caso**

Existe una relación de asociación denominada **Enseña**, con una navegabilidad que va del Instructor al Aprendiz, indicando que un instructor enseña a varios aprendices.

- **En el segundo caso**

La relación se interpreta de la siguiente manera: un aprendiz es evaluado por un instructor.

- **En el tercer caso**

De esta representación gráfica, se ilustra una relación bidireccional; es decir, se pueden realizar lecturas desde cada extremo. En este ejemplo, un instructor comparte conocimientos con muchos aprendices y un aprendiz comparte experiencias con un instructor.

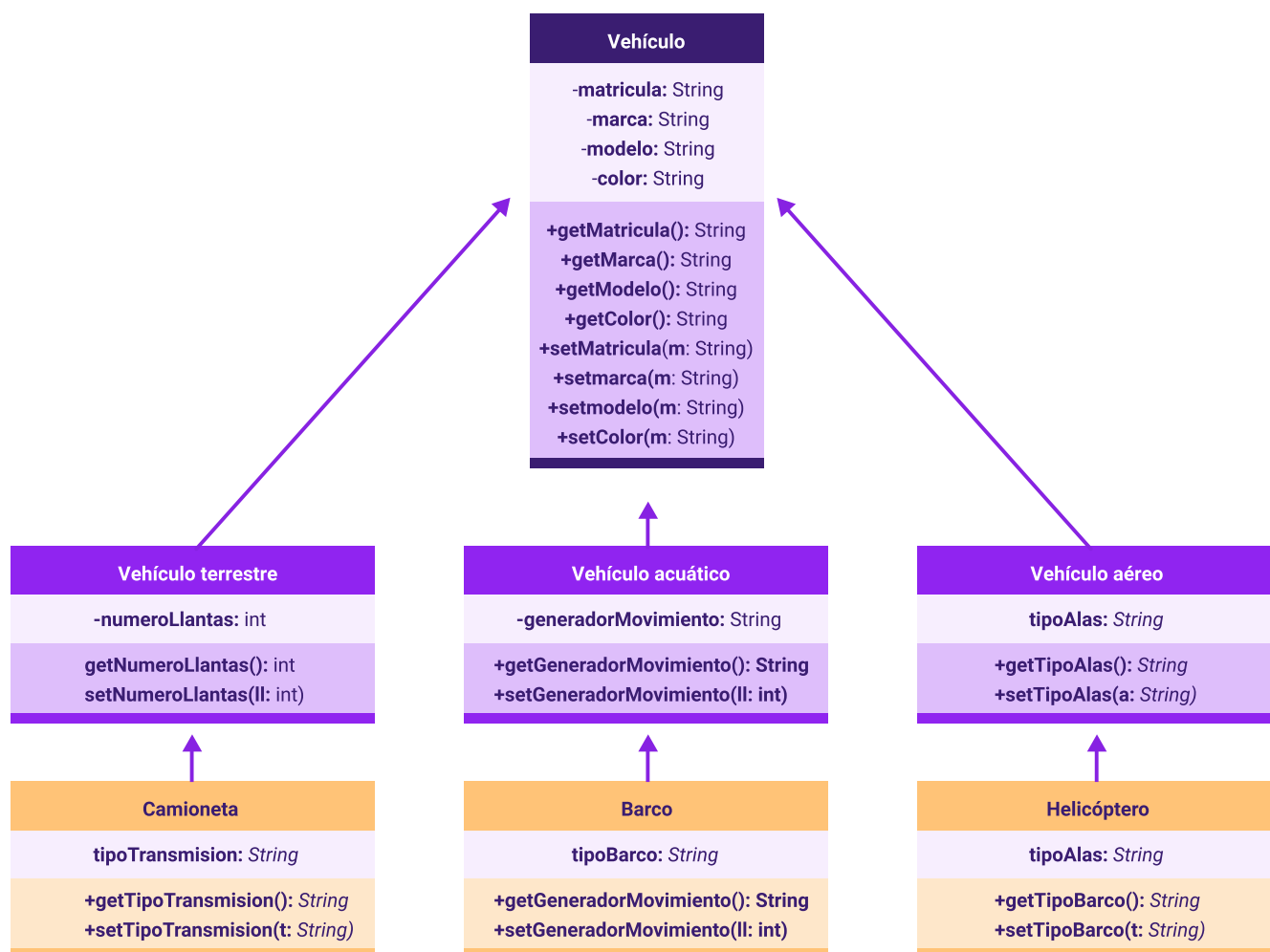
5.2. Herencia

La herencia es una de las relaciones más comunes en la práctica del paradigma orientado a objetos. Como se mencionó anteriormente, se pueden construir relaciones que van desde conceptos (clases) generales a conceptos (clases) específicos (especialización), o de conceptos específicos a generales (generalización), según Schmuller (2001).

La herencia es un tipo especial de asociación que cuenta con clases principales o superclases (que son las más generales en la relación) y clases secundarias o subclases (que son las más especializadas). Este tipo de relación tiene implícito el nombre 'es un' o 'es una'. También, implícitamente, es una relación que vincula una instancia de la superclase con una instancia de la subclase; por lo tanto, no se asignan nombres ni multiplicidad a este tipo de relación.

En una relación de herencia, las subclases heredan las características (atributos) y comportamientos (métodos) de las superclases.

Figura 18. Ejemplo de herencia

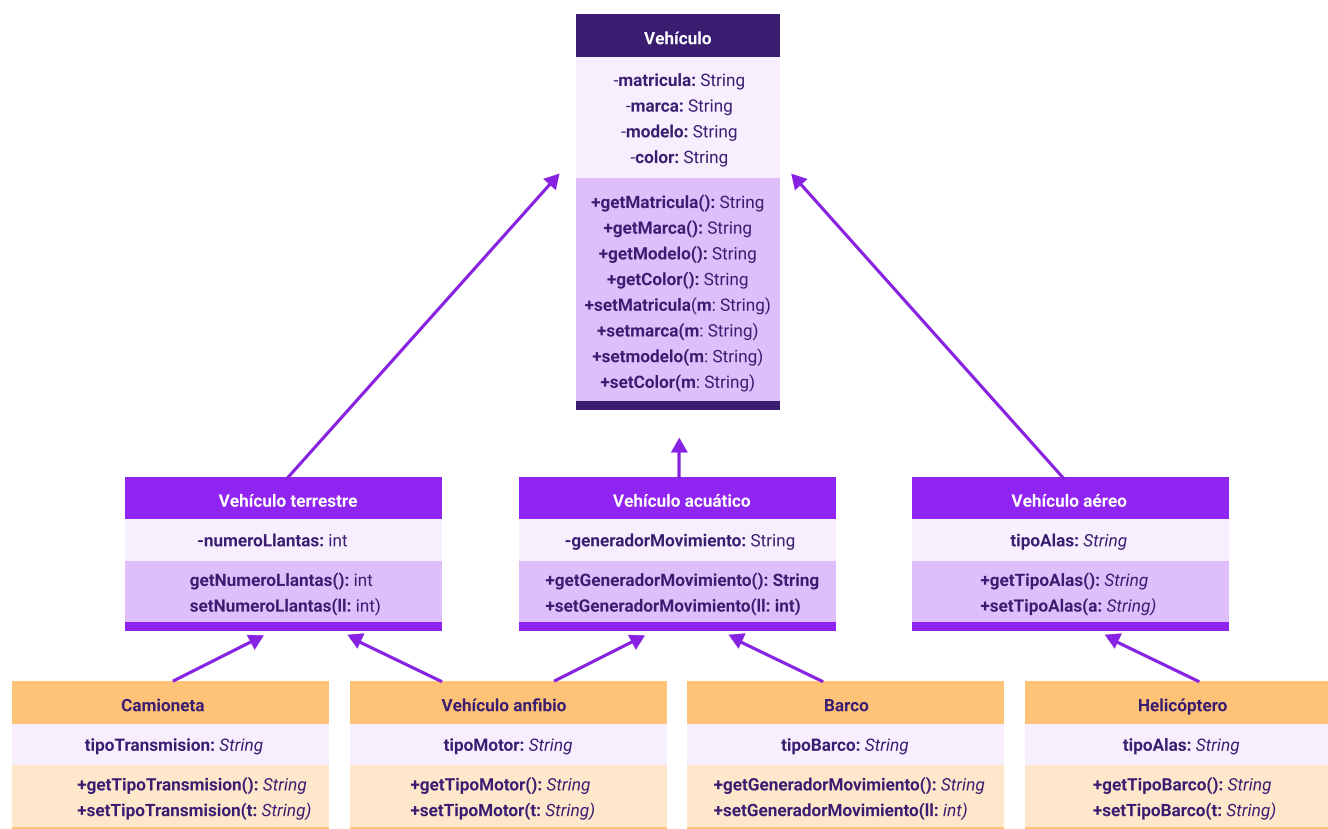


En la figura anterior, se presenta una relación de herencia entre siete (7) clases distintas. Las clases de tipo 'supertipo' están situadas en los niveles superiores de la jerarquía y representan las clases más generales cuyas definiciones serán heredadas por las subclases. En las relaciones de herencia, se pueden establecer varios niveles, y en cada nivel existen la correspondiente superclase y sus subclases. Es importante notar que no se asigna un nombre a estas relaciones porque se entiende implícitamente como una relación del tipo 'es un' o 'es una', la cual se interpreta desde las subclases hacia las superclases y siempre en relaciones uno a uno.

Por ejemplo, una Camioneta es un Vehículo Terrestre, que a su vez es un Vehículo. La clase Vehículo Terrestre hereda todas las características y comportamientos de la clase Vehículo. A su vez, la clase Camioneta hereda todas las características y comportamientos de la clase Vehículo Terrestre. De manera indirecta, la clase Camioneta está heredando las características y comportamientos de la clase Vehículo al estar en la misma línea de herencia. La clase Barco no hereda las características de la clase Vehículo Terrestre porque no se encuentra en la misma línea de herencia.

Hay dos tipos de herencia en función de la cantidad de clases supertipo directas asociadas: si una clase hereda características de un solo supertipo, es una herencia simple; si una clase hereda características de más de un supertipo, es una herencia múltiple.

Figura 19. Ejemplo de herencia múltiple



En el ejemplo de herencia múltiple, el diagrama de clases presenta varias relaciones de herencia. Por ejemplo, la clase Vehículo Anfibio es un caso de herencia múltiple, ya que hereda características de dos clases supertipo directas y diferentes: Vehículo Terrestre y Vehículo Acuático.

La herencia múltiple es admitida solo por algunos lenguajes de programación, siendo Python, Perl y C++ los más conocidos que la implementan. Sin embargo, otros lenguajes orientados a objetos ofrecen funcionalidades similares a la herencia múltiple mediante el uso de interfaces.

Visibilidad en las relaciones de herencia

Para definir la visibilidad de los atributos y métodos de una clase, se utilizan los identificadores de acceso: `private`, `public` y `protected`. En el contexto de las relaciones de herencia, es crucial ser cuidadoso para evitar resultados no deseados, respetando las siguientes normas:

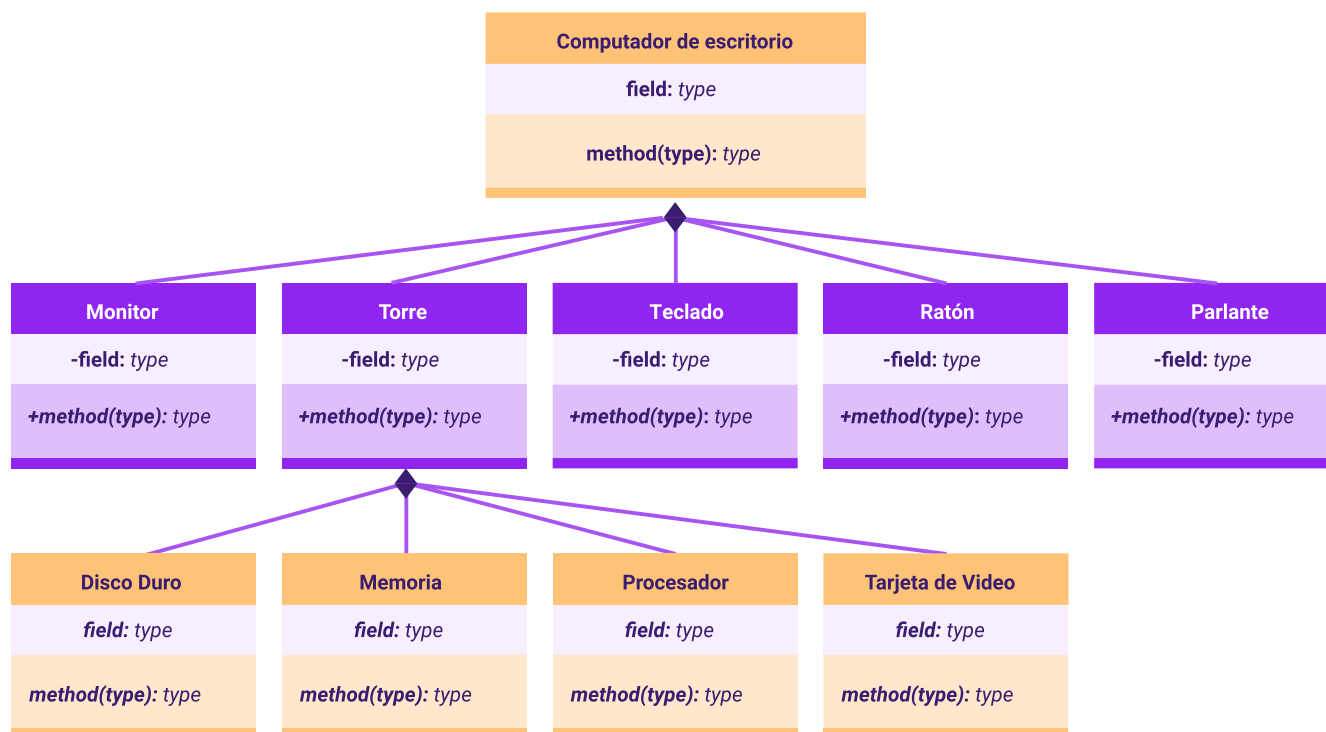
- Un atributo o método `private` en una superclase es inaccesible desde las subclases
- Un atributo o método `public` en una superclase permanece público en las subclases.
- Un atributo o método `protected` en una superclase se mantiene protegido en las subclases si existen más niveles de herencia, o actúa como privado en caso contrario.

5.3. Agregación

La agregación es un tipo de asociación que indica que un conjunto de clases forma un todo. Por ejemplo, existen clases agregadas que simbolizan el todo y están compuestas por un conjunto de clases componentes.

Esta relación se representa con una línea que conecta la clase agregada con sus clases componentes, teniendo un rombo en el lado de la clase agregada que representa la relación de agregación. Aunque esta relación no tiene un nombre explícito, ya que se comprende la existencia de clases agregadas y componentes, es necesario establecer la multiplicidad en el lado de los componentes.

Figura 20. Ejemplo de relación de Agregación



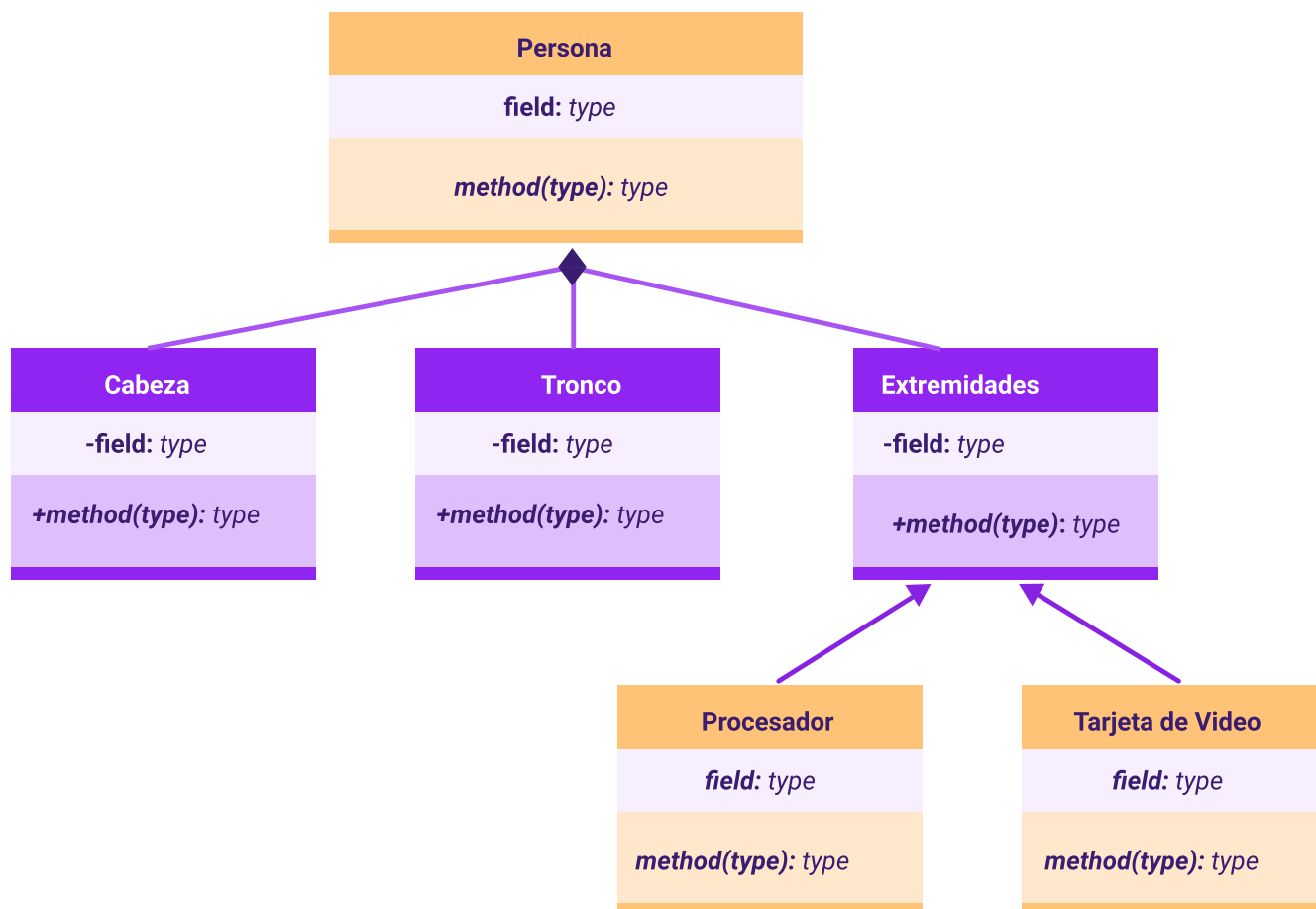
En la figura se presenta un ejemplo de relación de agregación. Como se puede apreciar, la clase Computador de Escritorio está compuesta por un monitor, una torre, un teclado, un ratón y dos parlantes. Además, la clase Torre consta de varios discos duros, memorias, un procesador y una tarjeta de video. Los componentes en este diagrama pueden ser partes de otras clases. Asimismo, una instancia de cualquier componente puede ser sustituida con facilidad, lo cual sugiere que la asociación con la clase agregada no es muy fuerte, característica distintiva de una agregación.

5.4. Composición

La composición es un tipo particular de relación de agregación donde los componentes no pueden pertenecer a más de una relación de agregación; es decir, son exclusivos de la composición establecida. Para distinguir una relación de composición

de una de agregación, el rombo representativo se rellena completamente de negro.
(Schmuller, 2001).

Figura 21. Ejemplo de relación de Composición



En la figura anterior, se identifica un ejemplo de composición. En este caso lo que se representa es una relación mucho más fuerte que la agregación ya que los elementos que componen la clase agregada no pueden formar parte de otra clase.

6. Herramientas CASE

Según Pressman, en el pasado, la ingeniería de software era principalmente una actividad manual, donde las herramientas se utilizaban solo en las etapas finales del proceso. Sin embargo, los ingenieros de software contemporáneos destacan la necesidad de herramientas más sofisticadas que las manuales para satisfacer las demandas de los sistemas informáticos actuales (Pressman, 1998).

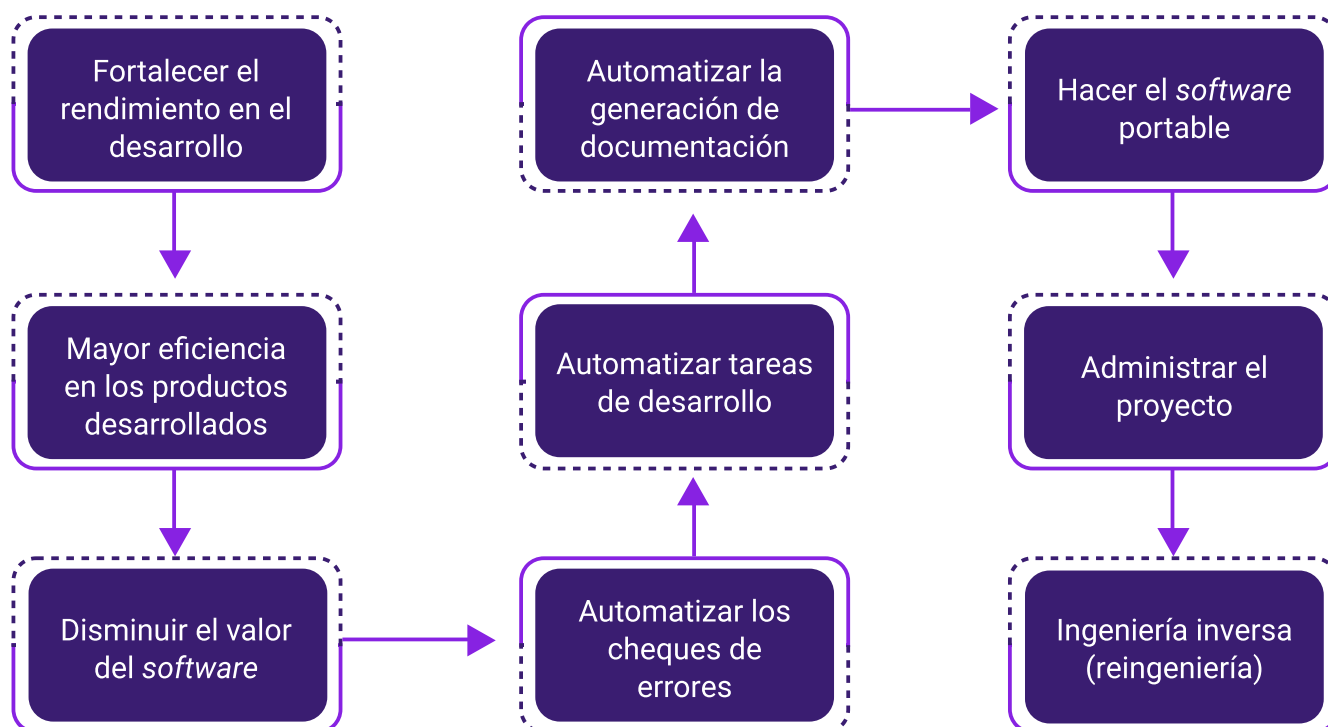
Las herramientas CASE son un complemento en la caja de herramientas de un ingeniero de software proporcionando la capacidad de automatizar tareas que tradicionalmente se ejecutaban de forma manual. Esto ayuda a garantizar que la calidad se integre desde la fase de diseño, mucho antes de proceder a la construcción del producto.

CASE es un acrónimo de "Computer Aided Software Engineering", que traducido al español significa "Ingeniería de Software Asistida por Computadora".

La tecnología CASE se refiere a la ingeniería de Software apoyada por herramientas computacionales. Consiste en un conjunto de herramientas que automatizan las tareas de desarrollo de software, con la finalidad de ofrecer un sistema integrado que conecte y automatice las fases del ciclo de vida del desarrollo (Cuevas, 1991).

De acuerdo con Pressman (1998), los objetivos de las herramientas CASE son:

Figura 22. Objetivos de la herramienta CASE



Clasificación de las Herramientas CASE

Las herramientas CASE en función de las fases del ciclo de vida que abarcan, se pueden agrupar de la siguiente forma:

- **Herramientas integradas, I-CASE (Integrated CASE, CASE integrado)**

Abordan todos los aspectos del ciclo de vida del desarrollo de sistemas.

Son llamadas también CASE de trabajo en equipo.

- **Herramientas de alto nivel, U-CASE (Upper CASE - CASE superior) o front-end**

Se utilizan para la automatización y soporte de las actividades de las etapas tempranas del desarrollo: análisis y diseño.

- **Herramientas de bajo nivel, L-CASE (Lower CASE - CASE inferior) o back-end**

Ayudan a las etapas finales en el ciclo de vida del desarrollo: construcción e implementación.

- **Juegos de herramientas o Tools-Case**

Son el tipo más simple de herramientas CASE. Automatizan una tarea o actividad del ciclo de vida del desarrollo de sistemas que se convierte en herramienta o herramientas independientes, constituyendo un caso de multi-herramienta.

A continuación, se presentan ejemplos de las herramientas CASE por ámbitos:

- **Análisis y diseño**

Workbench

GUI Design Studio

DIA

StarUML

- **Diseño de archivos y base de datos**

Power Designer

Oracle Designer

EasyCase

- **Programación**

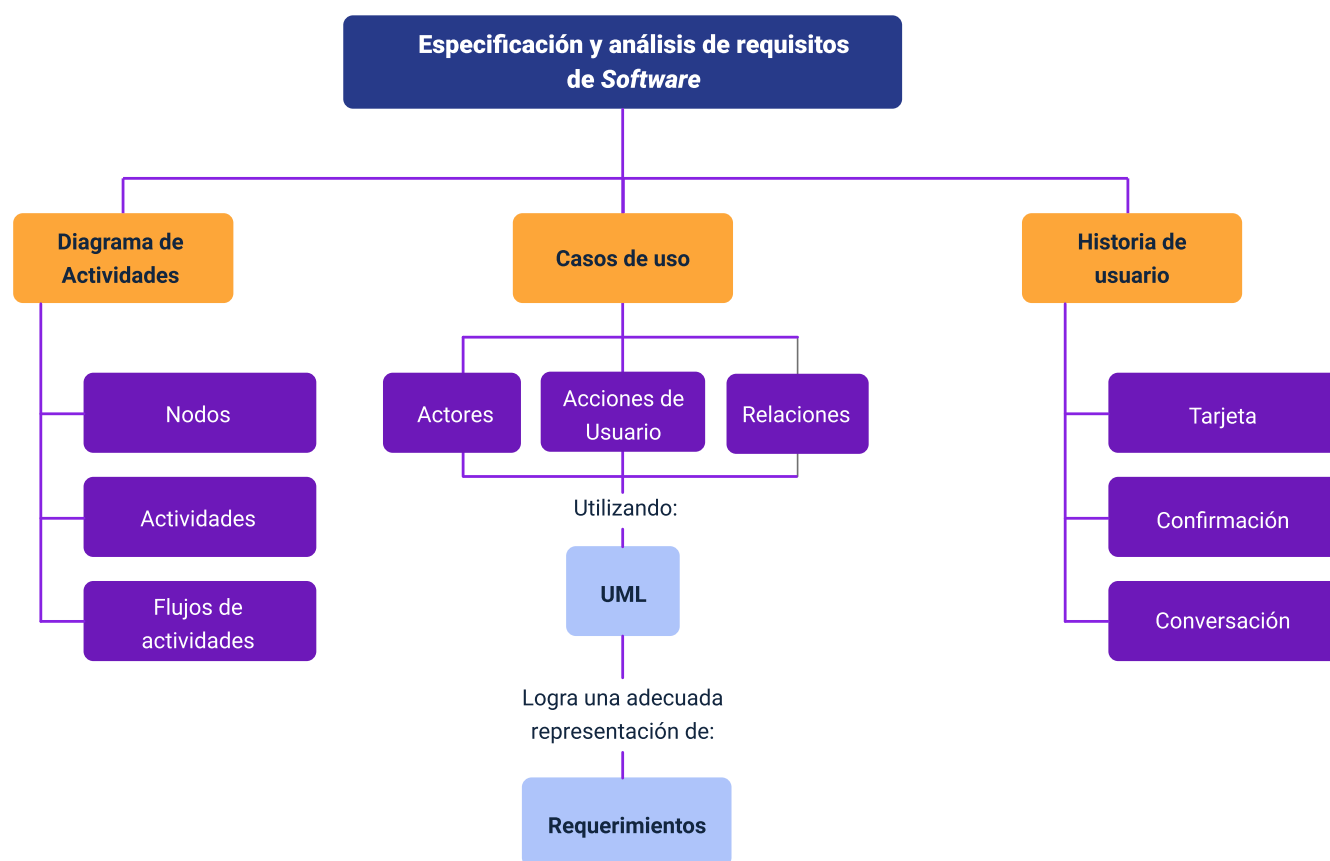
NetBeans

Eclipse/Omondo

Los diagramas de clase se diagraman con las herramientas que hacen parte del grupo de Herramientas de alto nivel, U-CASE, entre las que encontramos StartUML, Lucichart, ArgoUML, MagicDraw.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Diagrama de casos	Nicosiored. (2017, November 22). Diagrama de Casos de Uso II - 5 - Tutorial UML en español. YouTube.	Video	https://youtu.be/DUjBnEvlm1M?feature=shared
Diagrama de actividades	Nicosiored. (2018, January 31). Diagrama de Actividades - 15 - Tutorial UML en español. YouTube.	Video	https://youtu.be/GoYdpOVhDRc?feature=shared
Diagramas de clase	TodoCalls & DataScience (2017).Curso UML Diagrama de Clases. YouTube.	Video	https://www.youtube.com/watch?v=7WRYH2ei5Rw&t=106s
Lista Herramientas Case	Herramientas CASE (2008, July 11). Herramientas CASE más utilizadas.	Página web	https://herramientascase.wordpress.com/las-mas-utilizadas/

Glosario

Ágil: conjunto de tareas o acciones para producir y mantener productos y lograr objetivos de proceso. incluye procedimientos, estándares, políticas y objetivos para crear y modificar productos de trabajo.

CASE: ingeniería de software asistida por computación.

Herramienta CASE: una herramienta software que automatiza una parte de las fases del ciclo del desarrollo de sw.

Método: indica cómo construir técnicamente el software, incluyendo técnicas de modelado y otras técnicas descriptivas.

Metodología: colección de métodos para resolver un tipo de problemas.

Requerimiento: petición de algo que se solicita.

Requisito: condición que debe cumplir algo, en general, esto cumple con el requerimiento.

Stakeholders: individuo u organización que comparte, reclama o le interesa un sistema, o le compete una característica que satisface sus necesidades y expectativas.

Tecnología CASE: conjunto de instrumentos y técnicas software para automatizar una disciplina de la ingeniería, incluyendo metodologías estructuradas y herramientas.

Referencias bibliográficas

Booch, G. (1994). Object-oriented analysis and design. Redwood City.

Booch, G., Rumbaugh, J., Jacobson, I., Molina, J. J. G., & Martínez, J. S. (2006). El lenguaje unificado de modelado: guía del usuario. Addison-Wesley.

Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley Professional.

Cohn, M. (2018). User Stories and User Story Examples by Mike Cohn. [online] Mountain Goat Software. <https://www.mountaingoatsoftware.com/agile/user-stories>

Cuevas Agustín, Gonzalo. (1991). Ingeniería del Software. Práctica de la programación, Editorial RA-MA, 1 impresión.

Flores Cueto, J. J., & Bertolotti Zúñiga, C. (2013). Diagrama de clases en uml. 6.

Gutiérrez, J. (s.f.). Diagramas UML de casos de uso y de requisitos.

Larman, C. (2002). "Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process". 2nd Ed. Prentice Hall.

Norris & Rigby. (1994). "Ingeniería de software explicada", 1 edición Editorial Megabyte-Noriega editores, México.

Pressman, Roger. (1998). Ingeniería del Software. Un enfoque práctico, Editorial Mc. Graw-Hill, 4ta edición, 1998.

Rumbaugh, J., Jacobson, I., & Booch, G. (2004). El Lenguaje Unificado de Modelado. Elements, 30.

Schmuller, J. (2001). Aprendiendo UML en 24 horas. Pearson Educación.

Schmuller, J., & Garza Marín, A. D. (2000). Aprendiendo UML en 24 horas.

Wake, W.C. (2003). "Extreme Programming Explored" y "Refactoring Workbook"

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suárez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia



Validación de requisitos

Breve descripción:

En este componente formativo se abordan los temas de técnicas de validación de requisitos (revisiones, prototipos y casos de prueba) y el tema de los requerimientos duraderos y volátiles.

Mayo 2024

Tabla de contenido

Introducción.....	3
1. Validación de requerimientos	4
1.1. Revisiones de requerimientos	5
1.2. Construcción de prototipos.....	6
1.3. Generación de casos de prueba	9
2. Requerimientos duraderos y volátiles	11
3. Herramientas para la gestión de requisitos	13
Síntesis.....	17
Material complementario	18
Glosario	19
Referencias bibliográficas.....	20
Créditos	22

Introducción

En este componente formativo se exploran técnicas esenciales de validación de requisitos, como las revisiones, la creación de prototipos y la elaboración de casos de prueba, enfocándose en su aplicación práctica para asegurar que los requisitos cumplan con las necesidades y expectativas de las partes interesadas. Además, se distingue entre los requerimientos duraderos, aquellos que permanecen estables a lo largo del tiempo, y los volátiles, que son susceptibles a cambios debido a variaciones en el entorno de mercado, tecnológico o de la propia organización.

La aplicación de revisiones permite un análisis detallado de los requisitos por parte de un equipo, identificando posibles errores o ambigüedades en una fase temprana. Los prototipos ofrecen una visión tangible de cómo será el sistema o producto, facilitando la detección de malentendidos o expectativas no alineadas entre los desarrolladores y las partes interesadas. Los casos de prueba, por su parte, verifican que los requisitos se implementen correctamente, contribuyendo a una evaluación objetiva de la funcionalidad esperada.

Este componente formativo se diseñó para fortalecer las habilidades en el desarrollo y validación de requisitos, con un enfoque práctico que prepara a los profesionales para enfrentar los desafíos de la ingeniería de requisitos en entornos dinámicos y complejos.

¡Les deseamos éxitos en su proceso de aprendizaje y aplicación de estas técnicas fundamentales!

1. Validación de requerimientos

La validación de los requerimientos busca ratificar que los requerimientos realmente están especificando lo que el cliente desea y necesita. Este proceso es muy importante, pues un error en un documento de requerimientos puede ocasionar el desgaste importante de muchos recursos si estos errores son detectados en etapas más avanzadas del proyecto como diseño, construcción o despliegue a producción. Los costos asociados para el arreglo de problemas en los requerimientos siempre van a ser menores que en otras etapas, ya que un error en los requerimientos se propaga en cascada en todas las fases subsiguientes del ciclo de vida.

Según Sommerville (2011), en el proceso de validación de requerimientos se llevan a cabo las siguientes verificaciones:

- **Verificación de validez:** los requerimientos son razonables e identifican realmente todas las funciones necesarias para cumplir con las necesidades del cliente.
- **Verificación de consistencia:** los requerimientos no presentan contradicciones.
- **Verificaciones de completitud:** se incluyen todas las funcionalidades y restricciones definidas por los usuarios del sistema.
- **Verificaciones de realismo:** los requerimientos son realizables de acuerdo con la tecnología existente, el presupuesto y los tiempos definidos.
- **Verificabilidad:** es posible demostrar la realización de cada requerimiento y que hace lo que debe hacer. Es decir, existe una forma clara en la que se le pueda realizar pruebas.

Existen varias técnicas que pueden usarse para la validación de requisitos, como se puede revisar a continuación.

1.1. Revisiones de requerimientos

Las revisiones de los requerimientos son un proceso manual que involucra la participación de personas de parte de la organización constructora del software así como la de los clientes. Por lo general, en este proceso se revisa el documento de requerimientos tratando de encontrar alguna anomalía y/u omisiones en la escritura de estos.

Esta revisión se puede realizar de manera informal o formal (Sommerville, 2011). En una revisión informal, se requiere verificar, con tantos stakeholders como sea posible, el documento generado para recibir confirmación por parte de ellos de que lo escrito sí refleja su deseo. Esta revisión informal permite, de manera muy sencilla, detectar muchos problemas antes de establecer cualquier formalismo de revisión.

En una revisión formal, el equipo de desarrollo debe introducir al cliente por cada uno de los requerimientos establecidos y explicando cada una de sus implicaciones. El equipo encargado de la revisión deberá verificar cada requerimiento de forma individual y realizar el análisis de conjunto. Además, en el proceso de verificación debe resolverse cada una de las siguientes preguntas:

- **Verificabilidad:** ¿Es posible probar este requerimiento en la realidad?
- **Comprensibilidad:** ¿Es claro lo que expresa este requerimiento para las personas que lo van a usar?
- **Rastreabilidad:** ¿Es posible observar el origen del requerimiento y también evaluar los cambios que pueden generar en el sistema?

- **Adaptabilidad:** ¿El cambio en el requerimiento impacta o no a gran escala sobre otros requerimientos?

La validación de los requerimientos busca ratificar que los requerimientos realmente están especificando lo que el cliente desea y necesita. Lo invitamos a consultar el PDF "**Infografía - Validación de requerimientos**", el cual se encuentra en la carpeta Anexos.

1.2. Construcción de prototipos

Haga un chequeo del siguiente video para adentrarse en las generalidades y aspectos clave de la construcción de prototipos:

Video 1. Construcción de prototipos



[Enlace de reproducción del video](#)

Síntesis del video: Construcción de prototipos

Un prototipo es una versión inicial de un sistema que permite probar conceptos, opciones de diseño, flujos y otros aspectos por medio de la interacción directa de los clientes o usuarios finales en las primeras etapas del proceso del software. Un prototipo puede ayudar en la obtención y validación de los requisitos del sistema, permite explorar soluciones y posibles interfaces de usuario, y finalmente ayudan en el proceso de pruebas. Por medio de los prototipos, los clientes y usuarios finales tienen una idea más clara de cómo el sistema puede apoyar sus labores, puede generar nuevas ideas para los requerimientos e identificar fortalezas y debilidades de la forma en cómo hasta el momento se consigue cada requerimiento por parte de los desarrolladores.

También, un prototipo puede revelar errores y omisiones en los requerimientos que no fueron detectados en el momento de la escritura de estos. Cuando se analizan los requerimientos escritos, no es fácil detallar problemas relacionados con las funcionalidades combinadas con otras en un sistema completo, por lo que los prototipos son ideales para encontrar elementos faltantes o sobrantes en las definiciones de los requerimientos.

La construcción de prototipos para procesos de validación de requerimientos tiene los siguientes beneficios: los prototipos son fácilmente modificables y prescindibles, el prototipado mejora la relación entre desarrolladores y usuarios. Puede proporcionar la prueba conceptual necesaria para la consolidación en el proceso, pueden procurar una pronta preparación para los futuros usuarios del sistema. El prototipado puede evitar sorpresas desagradables al destacar los

requerimientos incompletos o inconsistentes y la falta de funcionalidad. Puede reducir los malos entendidos entre desarrolladores y clientes, puede reducir los costes de rediseño si los problemas se detectan pronto y cuando son fáciles de localizar, resultará en un producto que se ajusta mejor a los requerimientos del usuario. El prototipado puede fortalecer la especificación de los requerimientos; los usuarios entienden mejor los prototipos que las especificaciones de papel.

Por otra parte, el uso inadecuado de los prototipos también puede generar algunos riesgos, entre los que se encuentran:

- El prototipado puede estimular un número excesivo de cambios de peticiones.
- Los prototipos operativos pueden inducir a pensar a la directiva y a los clientes que el producto final está prácticamente dispuesto para su salida al mercado.
- Los prototipos pueden encarecer el producto.

Para la fase de verificación de requisitos se recomienda el uso de prototipos de baja fidelidad (presentación de escenarios con maquetas estáticas) y de tipo exploratorio (prototipos no reutilizables, usados únicamente para la clarificación e identificación de requerimientos).

Aunque un prototipo podría ser una sencilla representación gráfica del sistema en papel o la representación de interfaces del sistema usando herramientas de ofimática, existen varias herramientas gratuitas y de pago que facilitan la construcción de prototipos, algunas de ellas solo requieren un registro de correo electrónico para

acceder a sus funcionalidades directamente desde el navegador, y otras requieren la descarga e instalación en el equipo donde se planean construir los prototipos.

Algunas herramientas recomendadas son las siguientes:

- **Adobe XD:** www.adobe.com
- **Marvel App:** www.marvelapp.com
- **Moquos:** www.moqups.com
- **LucidChart:** <https://www.lucidchart.com/pages/>
- **Proto.io:** www.proto.io
- **Balsamiq:** www.balsamiq.com
- **Wirify:** www.wirify.com
- **Pidoco:** www.pidoco.com/en

1.3. Generación de casos de prueba

Desarrollar pruebas para los requerimientos permite evidenciar problemas antes de escribir cualquier línea de código. Normalmente, si es complejo construir un caso de prueba para un requerimiento esto es un indicio de que el requerimiento, puede ser difícil de implementar, por lo que debería considerarse.

La validación de requisitos por casos de prueba requiere el desarrollo de cuatro fases: i) planeación de la prueba. Aquí se define la estrategia a utilizar, el alcance de la prueba a realizar y los tiempos requeridos para el desarrollo de la prueba; ii) diseño de los casos de prueba; iii) ejecución de los casos de prueba y iv) elaboración del informe final de la prueba donde se describen los aspectos más importantes y hallazgos de la ejecución de la prueba.

De otra parte, el diseño de un caso de prueba requiere: i) la construcción de un instrumento donde se debe detallar para cada requerimiento si existen precondiciones, es decir, si se requieren de actividades o valores previos para poder iniciar la ejecución del requerimiento; ii) identificar los pasos a seguir para la validación de un requisito y iii) los resultados esperados de la realización de cada paso.

La siguiente es la estructura que lleva el instrumento de registro de caso de prueba:

Figura 1. Formato de caso de prueba

Encabezado 1	
Objetivo del caso de prueba	
Identificador	
Nombre o identificador del requerimiento	
Precondiciones	
Pasos	Resultados esperados
1	1
2	2
3	3

2. Requerimientos duraderos y volátiles

Los requerimientos de un proyecto inevitablemente sufren variaciones en el tiempo y puede ser por varios motivos, entre los que se podrían destacar cambios en las políticas gubernamentales, sociales, económicas o, sencillamente, por solicitudes de los clientes.

Según Sommerville (2011), dependiendo de la perspectiva evolutiva de los requerimientos, estos se pueden clasificar en dos grupos:

- **Requerimientos duraderos:**

Son relativamente estables y normalmente se derivan de las actividades principales de la organización y están directamente relacionados con el dominio del sistema. Por ejemplo, en un sistema académico los requerimientos relacionados con la gestión de estudiantes, profesores y grupos hacen parte del dominio y, seguramente, el modelo de negocio asociado a estos requerimientos no va a cambiar mucho en el tiempo.

- **Requerimientos volátiles:**

Son los que, muy probablemente, cambian durante el proceso del desarrollo del sistema o después que este entra en funcionamiento. Por ejemplo, en un sistema académico un requerimiento asociado al proceso de pago de pensión podría definirse de forma manual en un principio; es decir, la secretaria una vez reciba el dinero o los recibos de pago registra el pago en el sistema, más adelante si la institución educativa adquiere un servicio de pasarela de pagos en línea el requerimiento podría modificarse totalmente.

Los requerimientos volátiles según Sommerville (2011) se clasifican en:

- **Requerimientos cambiantes:** cambian de acuerdo con el entorno, por ejemplo: una pandemia.
- **Requerimientos emergentes:** surgen como producto de un mejor entendimiento del sistema, por ejemplo, al estructurar el diseño de las interfaces del sistema puede ser que se requieran adicionar componentes que no estaban contemplados.
- **Requerimientos consecuentes:** son resultado de la puesta en marcha de un sistema. Generalmente la adopción de un nuevo sistema involucra el cambio de procesos e incluso roles dentro de la organización que, a la vez, puede influir en cambios en los requerimientos del sistema.
- **Requerimientos de compatibilidad:** dependen de otros sistemas o procesos, cuando estos cambian generalmente producen también cambios en los requerimientos dependientes.

3. Herramientas para la gestión de requisitos

En la actualidad, existe una variedad de herramientas para ser utilizadas específicamente en la gestión de requisitos, la utilización de estas ayuda a mejorar la calidad del desarrollo de un proyecto y permite un mayor control en el mantenimiento, previniendo posibles errores durante la ejecución del proyecto.

Las herramientas de gestión de requisitos variarán según las metodologías de proyecto y los objetivos, la manera en la cual se aborden los requisitos también dependerá según la metodología. Por ejemplo, algunos equipos usan otra palabra para requisitos como “historias de usuarios”, “requisitos de productos” o, simplemente, “características”.

Existen herramientas básicas de requerimientos y herramientas complejas, las cuales se explican a continuación:

- **Herramientas básicas**

Se manejan en la planeación de gestión de requisitos muy básica y se pueden utilizar las hojas de cálculo, o una plantilla de documento en Word; es difícil su utilización cuando se trata de actualización con varias personas y el manejo de las versiones de los documentos.

- **Herramientas complejas**

Para una planificación de gestión de requisitos más compleja, se necesita un sistema de software completo para administrar las relaciones entre los requisitos, analizar el impacto de cualquier cambio, administrar las aprobaciones y demás aspectos.

De acuerdo con el mismo autor, las herramientas de gestión de requisitos se caracterizan por las siguientes propiedades:

- Gestión de requisitos basados en modelos de información, como por ejemplo casos de uso o historias de usuario.
- Organización de requisitos: es fácil agrupar el conjunto de requisitos y determinar su orden.
- Acceso y gestión multiusuario: múltiples personas pueden participar en los procesos de construcción o modificación de requisitos.
- Gestión de la trazabilidad: se puede ver a nivel histórico los cambios realizados en los requisitos con su marca de tiempo y usuarios involucrados.

Las siguientes herramientas principalmente ayudan a documentar, analizar, buscar, priorizar y trazar los requisitos:

Tabla 1. Herramientas de gestión de requisitos

Requirements Management Tool	Company	URL
IBM Rational DOORS	IBM Rational DOORS	https://www.ibm.com/products/requirements-management?mhsrc=ibmsearch_a&mhq=rational%20doors
Visure Requirements	Visure Solutions	https://visuresolutions.com/es/
Reqtify	Dassault Systèmes	https://www.3ds.com/products/catia/reqtify
Jama	Jama Software	https://www.jamasoftware.com/
Accept 360	Accept Software	https://www.acceptmission.com/es/

Requirements Management Tool	Company	URL
RequisitePro	IBM Rational	https://visuresolutions.com/es/rational-requisitepro/

Nota: Tomada de Sevilla (2021).

La norma ISO 24766 (Information Technology - Guide for Requirements Tool Capabilities) ayuda a seleccionar una herramienta adecuada de gestión de requisitos, pues la misma proporciona una orientación sobre los aportes de estas herramientas.

Aparte de las herramientas nombradas anteriormente, existen otras entre las que se encuentran las libres y comerciales; a continuación, se indican las más conocidas.

Herramientas comerciales

Actualmente, se usan distintas herramientas de uso comercial que cubren las necesidades elementales en la gestión de requisitos. Algunas de estas son:

- **IBM Rational.** Aplicación de gestión de requisitos para optimizar la comunicación, la colaboración y la verificación de requisitos.
- **IBM RequisitePro.** Herramienta de administración de requisitos que permite crear y compartir basado en documentos potenciados.
- **Jama Software.** Herramienta de administración de requisitos que permite visibilidad y control.

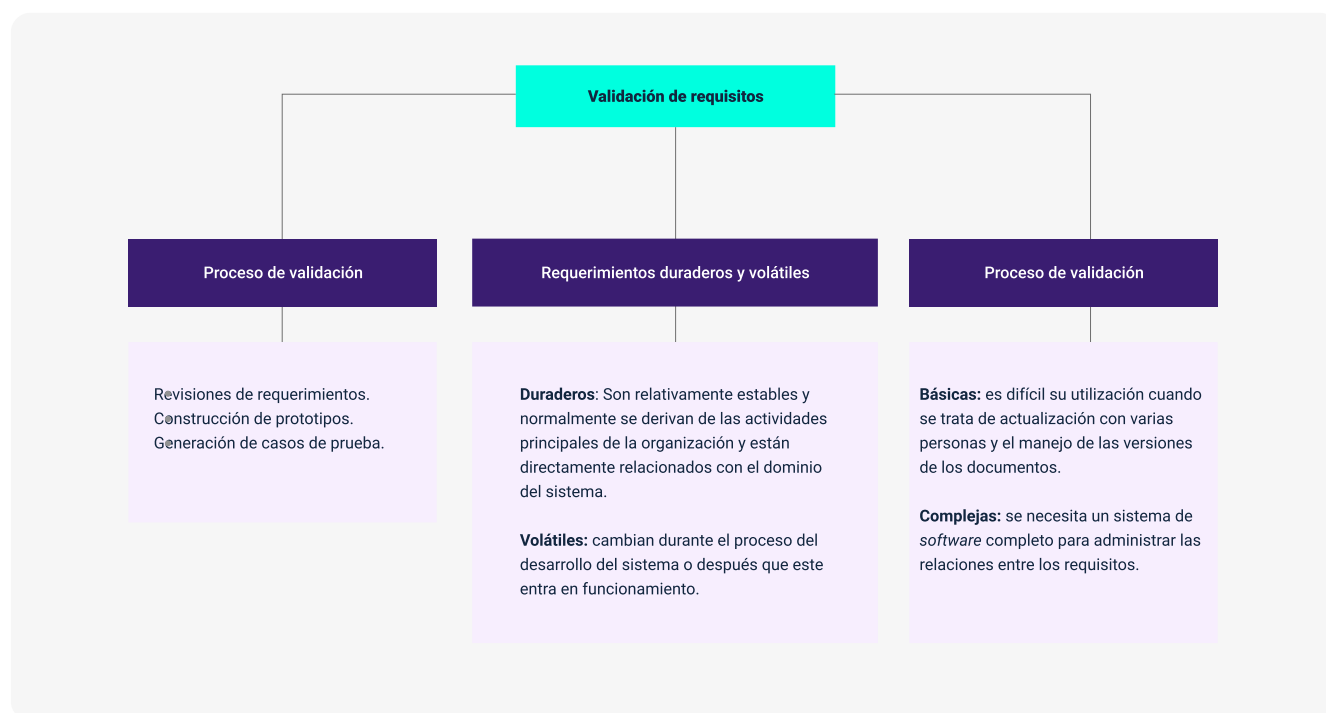
Herramientas libres

También existen herramientas de uso libre para realizar las actividades pertinentes a la gestión de requisitos. Algunas son:

- **Herramienta de requisitos automatizada.**
- **REM.** Totalmente desarrollada para la elicitación de requisitos.
- **OSRMT.** Permite la descripción avanzada de diversos tipos de requisitos y garantiza la trazabilidad entre todos los documentos relacionados con la ingeniería de requisitos (funcionalidades, requisitos, casos de uso, casos de prueba).

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Revisiones de requerimientos	todopmp.com. (s.f.). Guía PMBOK 6 - 49 procesos, entradas, herramientas y salidas.	Página web	https://todopmp.com/cards/
Revisiones de requerimientos	EDAP - Project Business School. (2016). MOOC PMP 302 Identificar Interesados (video). YouTube.	Video	https://www.youtube.com/watch?v=aUkTxgaajBo
Revisiones de requerimientos	Virtual Training Lteam. (2016). Partes Interesadas Stakeholders (video). YouTube.	Video	https://www.youtube.com/watch?v=9AtaIAZEu0c
Construcción de prototipos	Calle, M. (2020). Análisis de Interesados Matriz Poder Interés - PMI (video). YouTube.	Video	https://www.youtube.com/watch?v=hDZ0uu0H1wc
Generación de casos de prueba	Jibaro X. (2019). Tipos de preguntas en una encuesta (video). YouTube.	Video	https://www.youtube.com/watch?v=mwnQuUi9014

Glosario

Ágil: comprende un conjunto de tareas o acciones que se utilizan para producir y mantener productos, así como para lograr los objetivos del proceso. La actividad incluye los procedimientos, estándares, políticas y objetivos para crear y modificar un conjunto de productos de trabajo.

Ciclo de vida software: se refiere a la aplicación de metodologías para el desarrollo del sistema software (AECC, 1986).

Método: indica cómo construir técnicamente el software. Se incluyen técnicas de modelado y otras técnicas descriptivas.

Metodología: colección de métodos para resolver un tipo de problemas.

Requerimiento: el requerimiento se refiere a la petición que se hace de algo que se solicita.

Requisito: es la condición que debe cumplir algo, en general el requisito cumple con lo que se requiere con el requerimiento.

Stakeholders: individuo u organización que comparte, reclama o le interesa un sistema o le compete una característica que satisface sus necesidades y expectativas (ISO 29148).

Referencias bibliográficas

Baar, B. (2006). Using Stakeholder Analysis in Software Project Management. Universidad de Amsterdam.

Braude, J. (2003). Ingeniería de software, una perspectiva orientada a objetos. Alfaomega.

Cohen, L. (2011). Métodos de investigación educativa. La Muralla.

Cohn, M. (2004). User Stories Applied for Agile Software Development. Pearson Education, Inc.

Cox, K., Niazi, M., y Verner, J. (2009). Empirical study of Sommerville and Sawyer's requirements engineering practices. IET Software, 3(5), 339. <https://digital-library.theiet.org/content/journals/10.1049/iet-sen.2008.0076>

Curso de interacción persona-ordenador. (2021). Storyboarding. <https://mpiua.invid.udl.cat/storyboarding/>

Denscombe, M. (2010). The Good Research Guide. McGraw-Hill Education.

Dornyei, Z. (2010). Questionnaires in Second Language Research: Construction, Administration, and Processing. Routledge.

Durán, A., Bernárdez, B., Ruiz, A. y Toro, M. (1999). A Requirements Elicitation Approach Based in Templates and Patterns. https://www.researchgate.net/publication/2890318_A_Requirements_Elicitation_Approach_Based_in_Templates_and_Patterns

Gause, C., & Weinberg, G. M. (1989). Exploring Requirements: Quality Before Design. Dorset House.

Granollers, T., Lorés, J., y Perdrix F. (2002). Prototipado. Capítulo 5: modelo de proceso de la ingeniería de la usabilidad y de la accesibilidad.

Hernández, S., Fernández C., y Baptista L. (2006). Metodología de la investigación. McGraw Hill.

Herrera J., Lizka J. (2003). Ingeniería de requerimientos, ingeniería de software. <http://www.monografias.com/trabajos6/resof/resof.shtml>

La oficina de proyectos de informática (2012). PMOinformatica.com. Plantillas Scrum: historias de usuario y criterios de aceptación. <http://www.pmoinformatica.com/2012/10/plantillas-scrum-historias-de-usuario.html>

Piattini M., Calvo-Manzano J., Cervera J., y Fernández, L. (2004). Análisis y diseño de aplicaciones informáticas de gestión. Una perspectiva de ingeniería de software. Alfaomega-Rama.

Sommerville, I. (2011). Ingeniería de software. Pearson Educación.

Ventura, M. T. (2002). La ingeniería de requerimientos como factor clave para el éxito de los proyectos de desarrollo de software. <http://132.248.9.195/ppt2002/0307178/Index.html>

Wessinger, K., (2012) Project Stakeholder Management. Engineering Management Journal, 14(84), 19-24.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Juan Daniel Polanco Muñoz	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Álvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia



Validación de requerimientos

Busca ratificar que los **requerimientos** realmente están especificando lo que el **cliente desea y necesita**.

Importante

Un error en un documento de requerimientos puede ocasionar el desgaste importante de muchos recursos si estos errores son detectados en etapas más avanzadas del proyecto como **diseño, construcción o despliegue de producción**.



Un error en los requerimientos **se propaga en cascada en todas las fases** subsiguientes del ciclo de la vida.

Para tener en cuenta *

Verificaciones

- ☒ **De validez:** los requerimientos son razonables e identifican realmente todas las funciones necesarias para cumplir las necesidades del cliente.
- ☒ **De consistencia:** los requerimientos no presentan contradicciones.
- ☒ **De completitud:** se incluyen todas las funcionalidades y restricciones definidas por los usuarios del sistema.
- ☒ **De realismo:** los requerimientos son realizables de acuerdo con la tecnología existente, el presupuesto y los tiempos definidos.

Verificabilidad: existe una forma clara en la que se pueda realizar pruebas.





SENA			Pág.1 de2
Número de Documento:	Entrevista Estructurada	Fecha de Creación:	Elaborado por:
Nombre del Documento:	Entrevista estructurada		

ENTREVISTA ESTRUCTURADA

SISTEMA DE DIVULGACIÓN ACADÉMICA DE PROGRAMAS DEL SENA

Entrevistado: Henry Antonio Paruma

Cargo: Coordinador Académico

Introducción: A continuación, se realizarán una serie de preguntas con el objetivo de obtener información acerca del sistema de divulgación académica de los programas de formación del Sena que se quiere desarrollar para el centro. La información que usted nos suministre como parte del personal directivo es de vital importancia para tener éxito en el desarrollo de este.

Tiempo Estimado: 20 minutos

1. ¿Cuál es el tiempo estimado para el desarrollo del proyecto?
2. ¿Cuál es el presupuesto estimado para el desarrollo del proyecto?
3. ¿Tiene definida alguna metodología para el desarrollo del proyecto?
4. ¿Tiene establecida la o las tecnologías en las que desea sea desarrollado el proyecto?
5. ¿Se cuenta con el apoyo de las instancias administrativas y financieras de la universidad?
6. ¿Cuál es el medio de comunicación que piensa es más asertivo con los estudiantes de especialización, maestría y doctorado?
7. ¿Cree necesaria la implementación de otro medio de comunicación destinado únicamente a los estudiantes de postgrado?
8. ¿Cuál tipo de información cree necesaria y obligatoria para los estudiantes de postgrado?
9. ¿Cuánto tiempo dedicaría a la administración de información para el nuevo medio de comunicación a la semana?
10. ¿Quién o quiénes deberían ser los encargados de agregar la información al nuevo medio de comunicación?
11. ¿Desearía tener su propia cuenta o perfil de administración dentro del nuevo medio de comunicación?
12. ¿El medio de comunicación debería tener un canal de conexión directo con usted?
13. ¿El medio de comunicación debería tener un canal de conexión directo con su correo electrónico?
14. ¿El medio de comunicación debería tener un canal de conexión directo con sus redes sociales? ¿Cuáles?
15. ¿El medio de comunicación debería tener un canal de conexión directo con su número de celular?
16. ¿Estaría dispuesto a hacer pruebas en el tiempo de desarrollo de la nueva plataforma de comunicación?
17. ¿Le gustaría administrar la información de este medio de comunicación a través de una aplicación móvil o web o los dos?
18. ¿De poder tener su propia cuenta de administración cómo le gustaría realizar el acceso al nuevo sistema de comunicación?



SENA			Pág.1 de2
Número de Documento:	Entrevista Estructurada	Fecha de Creación:	Elaborado por:
Nombre del Documento:	Entrevista estructurada		

19. ¿Debe haber exclusividad de consulta de la información dentro de la universidad a fuera de ella?
20. ¿Cree usted que finalmente el nuevo medio de comunicación aportaría de forma positiva a todos los procesos en los que el estudiante de postgrado está inmiscuido?

Muchas gracias por la atención.

**Anexo A****Ficha de Observación**

FICHA DE OBSERVACIÓN		
FICHA No.	01	
ELABORA DO POR:		
LUGAR:		
SECTOR:		
TIEMPO	OBSERVADO	
1 día	Empresa Prestadora de servicio de salud; Área: Atención al Afiliado.	
1 día	Empresa Prestadora de servicio de salud; Área: Gestión de Servicios	
1 día	Empresa Prestadora de servicio de salud; Área: Recobros	
PALABRAS CLAVE:		



Anexo B

Guía de observación

GUÍA DE OBSERVACIÓN		
Aspectos a observar y registrar en materia técnica-ejecución	¿Cuáles son los principales procesos que se desempeñan en la compañía?	
	¿Cuáles son los principales actores intervinientes en el proceso y qué labor cumplen?	
	¿Cuáles son los principales problemas que afectan los procesos?	
	¿Qué tipo de problemas con relación a los sistemas de información se observan?	
	¿Cuáles fueron las problemáticas que más le interesaron y por qué?	
	¿Cuáles son los factores que más debilitan –en el manejo articulado del proceso- el resultado final?	

Aspectos a observar en materia comunicativa	¿Qué canales de comunicación se utilizan en el ejercicio de los distintos procesos?	
	¿Cómo ha sido la comunicación de las diferentes áreas intervinientes en los procesos de la compañía?	
	¿Qué incidencia ha tenido la propuesta comunicativa sobre el resultado final del proceso?	
	¿Existen casos exitosos que se deriven del proceso comunicativo, en la articulación de los procesos?	
Aspectos a observar en materia del sistema de información	¿Cuál es el proceso que se sigue a partir del sistema de información?	
	¿Cuáles son los principales inconvenientes en el manejo del sistema de información?	

	¿Qué incidencia ha tenido dichas fallas en el sistema de información en el resultado final del proceso?	
	¿Existen casos exitosos que se en e manejo del sistema de información?	



Anexo C

Entrevista a los principales actores en el proceso

Buenos días y/o tardes:

La presente entrevista es de carácter exclusivamente informativo, la cual se realiza como apoyo al trabajo de investigación de Especialización de Gerencia de la Calidad. De modo que, las apreciaciones acá señaladas no generan compromiso alguno con el tema tratado.

1. ¿Qué sabe sobre los procesos que desempeña la empresa prestadora de servicios de salud?

Rta/

2. ¿Cuáles han sido las acciones ejercidas por la Entidad, en pro de fomentar el resultado eficiente de los servicios a prestar, a partir de la articulación de los procesos en las siguientes áreas?

Rta/

3. ¿Cuál considera ha sido el mayor inconveniente para la generación de resultados eficientes en la articulación de los procesos, por medio de los sistemas de información?

Rta/

4. ¿Cuál ha sido el apoyo de las áreas administrativas en pro de lograr una mejora, en la articulación eficiente de los procesos a través de los sistemas de información?

Rta/

5. ¿Cuál considera usted que puede ser la solución a los principales inconvenientes presentados en la articulación de los procesos por medio de los sistemas de información, en pro de un resultado eficiente por parte de la empresa prestadora de los servicios de salud?

Rta/

6. ¿Cree usted que los actuales sistemas de información se pueden mejorar o se debe contemplar opciones externas?

Rta/



Estándar IEEE 830



Conjunto de prácticas recomendadas para la **redacción de un documento** de especificación de requerimientos mejor conocido como **SRS**.

Secciones del documento



1 Introducción

- Propósito
- Ámbito del sistema
- Definiciones, acrónimos y abreviaturas
- Referencias

2 Descripción

Factores generales que afectan al producto y sus requerimientos.

- Perspectiva del producto
- Funciones del producto
- Características de los usuarios
- Restricciones generales
- Suposiciones y dependencias



3 Especificación de requerimientos

Requerimientos base para guiar los procesos de diseño, implementación y pruebas del software.

- ☒ Corrección
- ☒ Rastreable
- ☒ No ambiguo
- ☒ Verificable
- ☒ Priorizado

Cada requerimiento descrito en esta sección debe cumplir con los siguientes **criterios de calidad**.

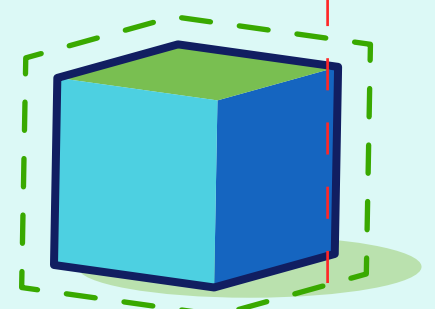
Se subdivide para presentar primero los **requerimientos funcionales** y luego los **no funcionales**.



4 Modelo de Análisis

Se usan en el desarrollo específico de los requerimientos listados en la sección anterior con introducción y descripción narrativa.

- Diagrama de secuencia
- Diagrama de flujo de datos
- Diagrama de transición de estados



5 Identificar y describir

Quién puede enviar cambios, por cuáles medios y cómo pueden ser aprobados estos cambios.



Fundamentos de análisis y diseño de software

Breve descripción:

En este componente formativo se abordan los saberes de ingeniería de requisitos, la fase de elicitación de requisitos y el análisis de requisitos (priorización, descomposición funcional, matriz de trazabilidad) y estándares, y/o guías existentes para la especificación formal de los mismos dependiendo del tipo de marco de trabajo usado (tradición o ágil).

Mayo 2024

Tabla de contenido

Introducción	6
1. Caracterización de procesos	7
1.1. Teoría general de sistemas.....	7
1.2. Enfoque de sistemas.....	8
1.3. Definiendo los sistemas	8
Clasificación de sistemas	9
Sistemas de información.....	12
Datos e información	14
1.4. Análisis de los procesos a nivel de negocio	14
Identificar los procesos.....	17
Determinar el equipo de trabajo	20
Generar los diagramas de los procesos de negocio	20
Realizar un diagnóstico de cada proceso	21
Puntos de mejora	21
Nuevo proceso mejorado	21
2. Ingeniería de requisitos	25
2.1. Ciclo de vida del software	28
Fases.....	29

Paradigmas de los modelos de ciclo de vida del software	30
Modelo Scrum	36
Modelo Kanban	38
Modelo XP o programación extrema	39
2.2. Fase de definición de requisitos.....	41
2.3. Requisitos	42
Importancia de los requisitos	42
Clasificación.....	44
3. La fase de elicitación de requisitos	46
3.1. Planeación	46
Identificar las fuentes de requerimientos	47
Identificar interesados del producto	49
Matriz de stakeholders	52
3.2. Técnicas e instrumentos para elicitar requisitos.....	56
Entrevista	57
Encuesta	60
Observación.....	62
Sesiones grupales	64
3.3. Herramientas para captura de requisitos.....	66

Diagrama de casos de uso	66
Historias de usuario	69
Storyboard	71
3.4. Herramientas de modelado	72
4. Técnicas de análisis y especificación de requisitos	77
4.1. Técnicas de análisis de requisitos	77
Priorización de requisitos	77
Técnica de clasificación de lista	78
Técnica de puntos de historia y valor del negocio	79
Técnica urgente	80
Técnica MoSCoW	82
Juicio de expertos	83
Matriz de priorización	84
Matriz de trazabilidad	85
Descomposición funcional	87
4.2. Especificación de requisitos	88
Estándar IEEE 830	88
Estándar IEEE 29148:2018	90
La especificación de requisitos a través de marcos de trabajo ágiles	92

Scrum y la especificación de requisitos.....	93
Kanban y la especificación de requisitos.....	97
Síntesis	100
Material complementario.....	101
Glosario	103
Referencias bibliográficas	106
Créditos.....	110

Introducción

Bienvenidos al estudio del presente componente formativo, en el cual se estudiará la caracterización de procesos, para continuar con la ingeniería de requisitos, entendiéndola como el conjunto de actividades y tareas del proceso de desarrollo de sistemas software, donde se definen que las características de un sistema software satisfagan las necesidades de negocio de clientes y usuarios y que se integre con éxito en el entorno en el que se trabaje.

Por este motivo, estudiaremos el ciclo de vida del software, sus fases y los diferentes paradigmas de los modelos de vida del software, haciendo énfasis en los requisitos, su importancia y clasificación.

Luego trataremos el tema de la recolección de datos, el cual se refiere al uso de una gran diversidad de técnicas y herramientas que pueden ser utilizadas por el analista para desarrollar los sistemas.

Continuaremos con el análisis de requisitos, documento donde se describe lo que se debe hacer, es el estudio de las necesidades. Allí, se estudiarán los principios básicos sobre los que se fundamenta el proceso de análisis de requisitos.

Conoceremos diferentes formas (planillas y estándares) para realizar el proceso de documentación de requisitos, así como diferentes metodologías ágiles que nos ayudarán en la especificación de requisitos.

Bienvenido, y le deseamos muchos éxitos en el proceso de aprendizaje.

1. Caracterización de procesos

La definición de procesos ayuda al entendimiento del funcionamiento de una organización, el análisis, moldeamiento, diagramación y mejora continua, son actividades esenciales en la estructura de una empresa que quiere aplicar desde un orden lógico las actividades diarias.

1.1. Teoría general de sistemas

Es un campo de la ciencia que pretende examinar las propiedades que definen a los sistemas; es decir, categorías formadas por partes interrelacionadas que llevan al cumplimiento de una acción u objetivo.

Esta teoría se constituyó como un nuevo aporte dentro del conjunto de paradigmas científicos, para el abordaje de problemas que se basan en la correspondencia entre los elementos que conforman los sistemas. Antes de esta propuesta se pensaba que los sistemas, desde el punto de vista de su conformación global, se asemejaban a la sumatoria de sus elementos integrantes y que podían ser entendidos, disgregándolos para realizar un estudio individual de cada componente.

Desde sus inicios, la teoría general de sistemas se ha utilizado en diferentes áreas de la ciencia, la ingeniería, las matemáticas, la biología, la política, las ciencias sociales, las ciencias de la computación, la economía y otras ciencias exactas y sociales, primordialmente en el marco del análisis de las correlaciones que se presentan en sus componentes y su interoperabilidad.

1.2. Enfoque de sistemas

Un puntal básico, unido a los respectivos análisis (simulación, teoría de comportamientos, teoría de colas, teoría de juegos), que se pueden aplicar para la resolución de problemas usando el método científico, es el enfoque de sistemas:

“Se trata de comprender el funcionamiento de una organización u objeto de estudio desde una perspectiva holística e integradora, en donde lo importante son las relaciones entre los componentes. Se llama holismo al punto de vista que se interesa más por el todo que por la suma de las partes. El enfoque sistémico no concibe la posibilidad de explicar un elemento si no es precisamente en su relación con el todo. Metodológicamente, por tanto, el enfoque sistémico es lo opuesto al individualismo metodológico, aunque esto no implique necesariamente que estén en contradicción”. (Centro Tic Junta de Andalucía, 2021).

1.3. Definiendo los sistemas

“Sistema es un conjunto organizado de elementos que interactúan entre sí con estructura lógica o que son interdependientes, formando un todo complejo, identificable y distinto. Por los elementos de un sistema se entiende no solo su conformación física, sino las funciones que estos desempeñan. Algún conjunto de elementos de un sistema puede ser considerado un subsistema si mantienen una relación entre sí que los hace también un conjunto identificable y distinto”. Ludwig Von Bertalanffy (1968).

Tales sistemas se caracterizan por su capacidad de recepción de elementos, los cuales son denominados entradas; a su vez, se pueden tipificar como tipos de energía, información a manera de mensajes, datos y señales, también recursos físicos. Estas

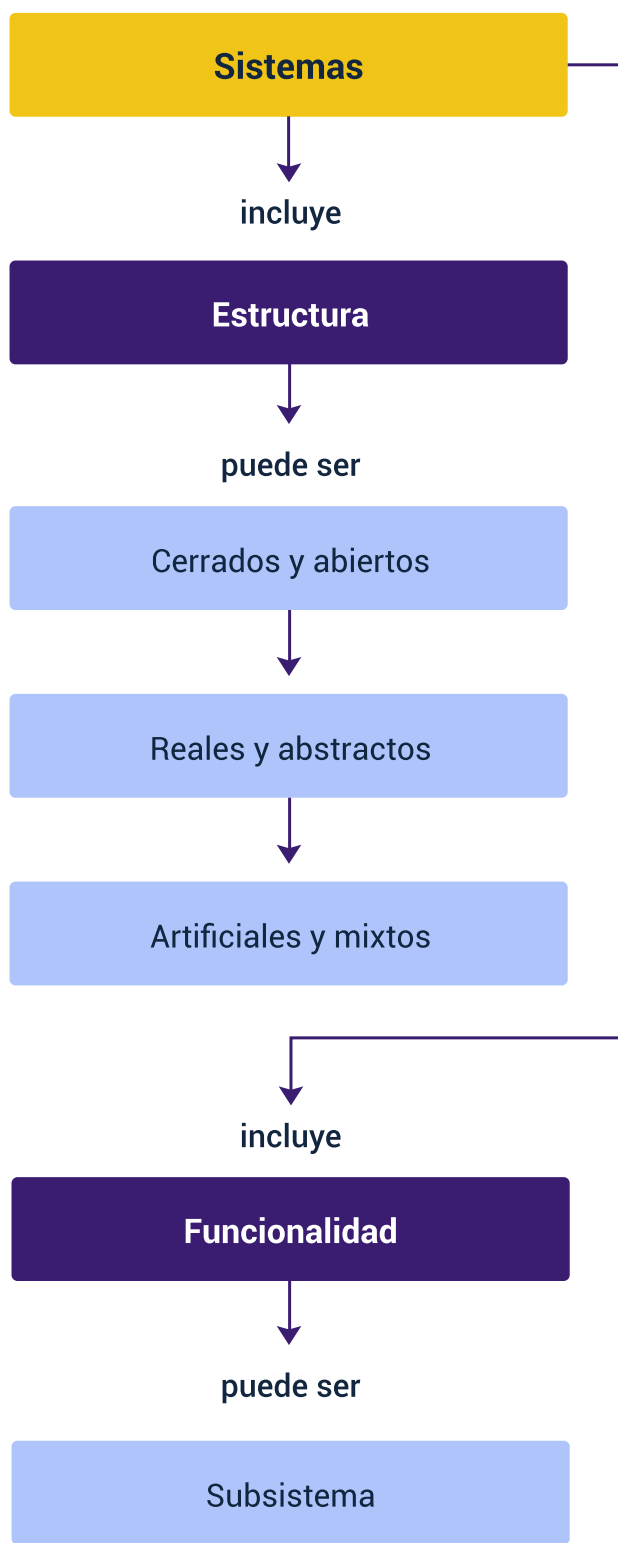
entradas se someten a un conjunto de actividades que alimentan una serie de acciones que las transforman y como consecuencia, se generan unos resultados o salidas; una situación muy interesante se presenta cuando una salida o parte de ella se convierte nuevamente en entrada. Cuando esto sucede, se dice que existe retroalimentación. La retroalimentación ayuda a elevar el grado de perfeccionamiento en las respuestas y en el propio comportamiento del sistema, logrando de manera inmediata altos niveles de control, principio básico de la automatización de estos.

Así mismo, cuando de un subsistema se conocen solo las entradas y las salidas, pero no las actividades internas, se dice que es una caja negra, en ese caso se aconseja utilizar la observación y guiarse por el sentido común para proponer las actividades desde cero, que resuelven el mismo problema.

Clasificación de sistemas

Bertalanffy y otros autores posteriores, han definido distintas formas de clasificar a los sistemas en función de su conformación estructural y sus funcionalidades. La siguiente figura presenta las clasificaciones más importantes:

Figura 1. Definiciones de sistemas



A continuación, se presentan algunos tipos de sistemas:

- **Sistema, supersistema y subsistemas**

Los sistemas se pueden agrupar teniendo en cuenta su grado de complejidad; se componen a su vez de distintos niveles de jerarquía en que pueden dividirse y que interactúan entre ellos intercambiando algún tipo de información, de modo que estos al final no son autónomos unos de otros.

Si se entiende por sistema un conjunto de partes, podemos hablar de subsistemas para referirse a tales elementos; un ejemplo de ello es: la conformación de una familia como un sistema y cada miembro en ella es un subsistema. El supersistema es el contexto externo que rodea al sistema y en el que este se encuentra inmerso; en los grupos o comunidades de humanos es identificable con la sociedad.

- **Reales físicos, ideales y abstracciones**

De acuerdo con su esencia, los sistemas se pueden clasificar en reales físicos, como producto del pensamiento ideal y modelo. Los sistemas reales son aquellos que existen físicamente y que pueden ser objeto de observación y por tanto, objetos de medición, mientras que los sistemas ideales son las representaciones simbólicas extraídas de la abstracción que produce el pensamiento y el lenguaje, que se traducen en diseños y modelos. Las abstracciones y sus correlaciones pretenden representar características reales e ideales, aquí es importante tener en cuenta el conjunto de requerimientos, solicitudes, la definición de límites con las cuales se pretende llegar a los objetivos finales.

- **De la naturaleza, artificiales y mixtos**

Cuando un sistema está inmerso o corresponde únicamente al funcionamiento de la naturaleza, como lo es un lago, un río, la selva, el cuerpo humano, los planetas o las galaxias, entonces hay que referirse a ellos como sistemas naturales. Por el contrario, los sistemas artificiales son aquellos que emergen de la creación del hombre, es decir, como productos de la acción humana.

Los sistemas mixtos se dan por la combinación de elementos naturales y artificiales. Cualquier entorno de orden físico que sufre alteraciones dadas por el ser humano, como lo son los pueblos, las ciudades, los ríos en donde se construyen represas, es considerado un sistema mixto; el grado de participación de elementos naturales y artificiales varía en cada caso concreto en proporciones diferentes.

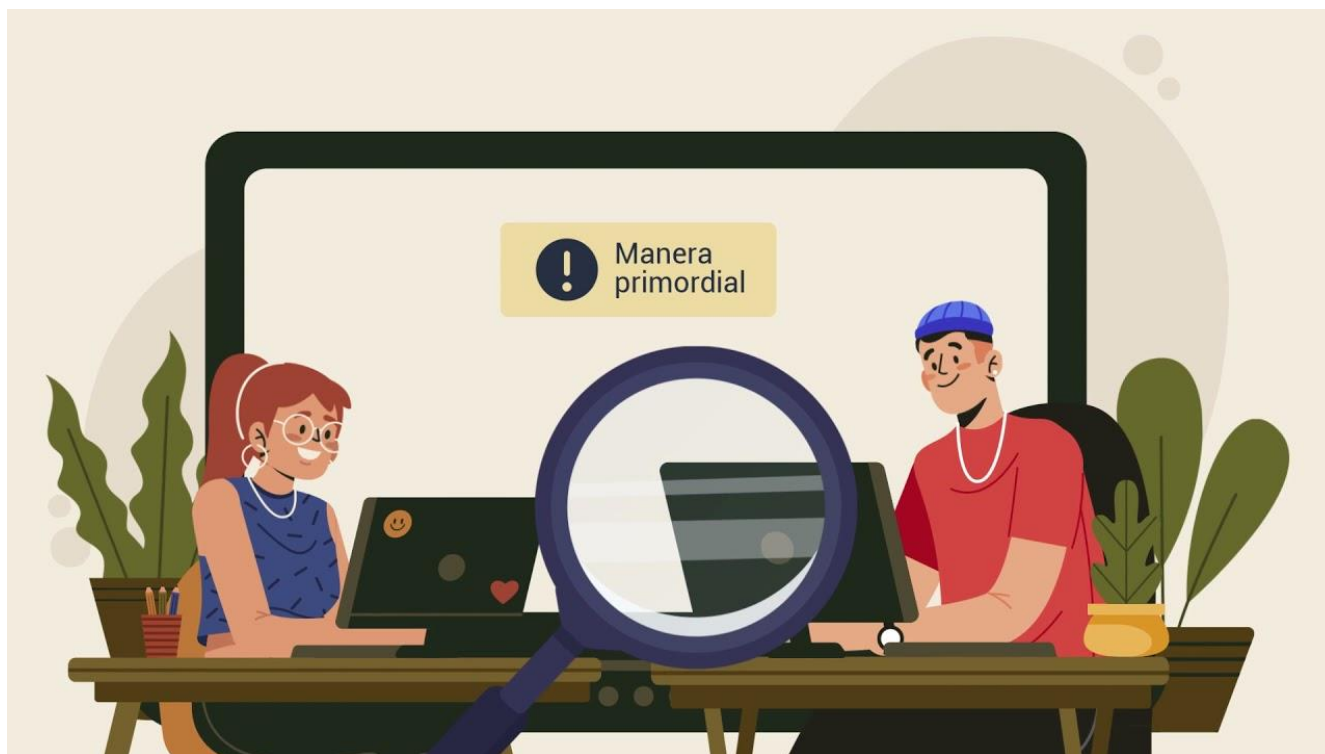
- **Cerrados y abiertos**

Torres (2021) argumenta: “El criterio principal que define a un sistema es el grado de interacción con el supersistema y otros sistemas. Los sistemas abiertos intercambian materia, energía y/o información con el entorno que los rodea, adaptándose a estos e influyendo en él. En cambio, los sistemas cerrados se encuentran teóricamente alejados de las influencias ambientales; en la práctica se habla de sistemas cerrados cuando están altamente estructurados y la retroalimentación es mínima, puesto que ningún sistema es completamente independiente de su suprasistema”.

Sistemas de información

A continuación, se presenta un breve significado de sistemas de información:

Video 1. Sistemas de información



[Enlace de reproducción del video](#)

Síntesis del video: Sistemas de información

El video presenta el concepto de sistemas de información en el contexto de la caracterización de procesos, los cuales son gestionados por procesamiento electrónico de datos que permiten solucionar problemas de la vida real. Convergiendo en las últimas técnicas de procesamiento de datos, big data, inteligencia artificial, minería de datos, domótica entre otros, unido al desarrollo de software, donde es importante analizar su inter-operatibilidad aprovechando funcionalidad y minimizar recursos.

Datos e información

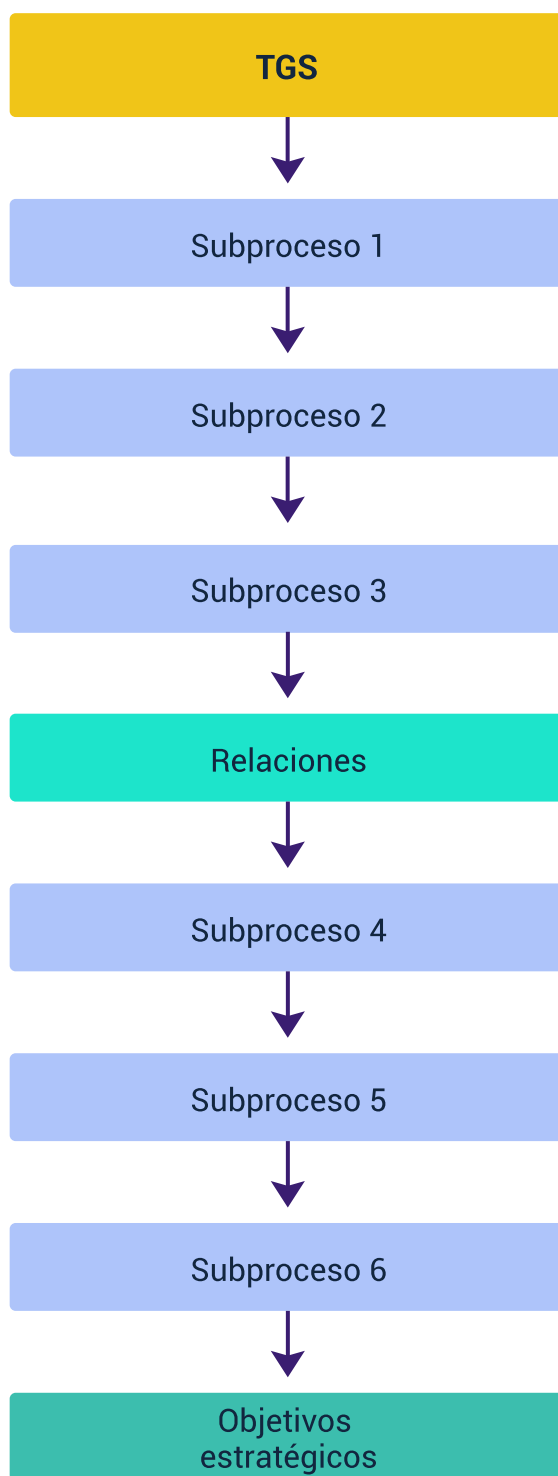
Estos términos tienden a confundirse, pero a la luz de los sistemas que estamos tratando, vale la pena definirlos:

- Un dato no es otra cosa que una representación simbólica de alguna situación o suceso, sin ningún sentido semántico, describiendo un hecho concreto. O lo que es lo mismo, sin transmitir mensaje ninguno, un ejemplo de dato podría ser una letra o un hecho.
- La información es un conjunto de datos, los cuales son adecuadamente procesados, para que puedan proveer un mensaje que contribuya a la toma de decisiones a la hora de resolver un problema o afrontar una situación cualquiera, en la que se requiera de la toma de decisiones de cualquier tipo.

1.4. Análisis de los procesos a nivel de negocio

Con base en los conceptos de la teoría general de sistemas y partiendo de la visión del todo, se debe comprender el funcionamiento real a partir de desestructurar ese todo en procesos, subprocesos, relaciones y actores que intervienen, con el objetivo de identificar, comprender, evaluar y resolver problemas que ayudan a las instituciones a mejorar dichos procesos, reformar las opciones de negocio y brindar a sus consumidores a experimentar una mejor calidad de productos o servicios. Por esto, es necesario que realice de manera constante, frecuente y repetitiva, un análisis de sus procesos de negocio desde la TGS, donde visualicen y comprendan cada uno de los pormenores que transforman el negocio, con el objetivo de mejorar sus operaciones y el servicio al cliente. A continuación, se presenta una gráfica de la explicación.

Figura 2. Análisis del proceso desde la TGS



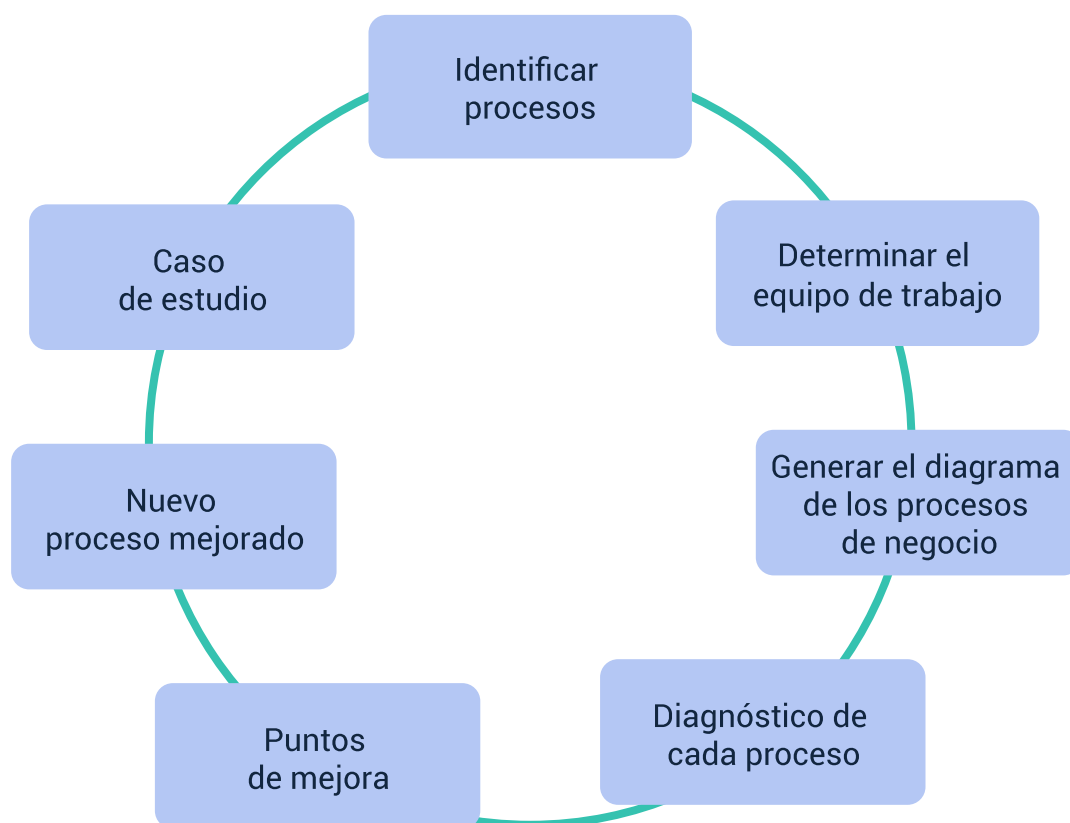
Una de las técnicas que permite conseguir estos objetivos, es el análisis de procesos de negocio, que significa llegar al conocimiento detallado de cada labor y se hace abordando la operación central de valor hasta las que pueden considerarse como labores rama o periféricas, no menos importantes. Ese es el nombre que se le da a la acción que realizan las organizaciones para revisar, documentar y entender sus procesos.

Tiene como objetivo el desglose de cada uno de los componentes en cada proceso:

- Entradas
- Salidas
- Procedimientos
- Controles
- Aplicaciones
- Datos

Es importante comprender las acciones que integran cada uno de los siguientes elementos:

Figura 3. Acciones para el análisis



Ahora, conozcamos de manera detallada, cada proceso:

Identificar los procesos

Para realizar una excelente identificación de procesos, se recomienda realizar las siguientes actividades en dos escenarios:

- **Escenario 1**

Antes: se debe obtener acceso de información sobre la organización, para antes de llegar a esta, se investigue sobre el contexto de negocio en el que se mueve la empresa. Por ejemplo: determinar si se está hablando de un banco o una empresa de energía o una empresa que presta servicios y en ese caso, qué tipo de servicios presta o, por el contrario, si es una empresa

que fabrica productos, qué productos fabrica, es decir, qué hace. Conocer de ella también sus sedes (instalaciones físicas).

- **Escenario 2**

Cuando exista cercanía, acceso o contacto con la entidad, planear entonces la recolección de información, aplicando las técnicas que existen para ello, como son:

- Método de la observación.
- La elaboración de formatos de encuestas.
- El enfoque de trabajo (focus group).
- Las entrevistas.

Como resultado de estas técnicas, se deben establecer claramente y como mínimo los siguientes ítems: roles de los intervinientes, información de contactos con responsabilidades, misión, visión, objetivos estratégicos, estructura organizacional un organigrama de la entidad su distribución de dependencias u oficinas, en donde se puede ubicar de manera ágil a los responsables, que seguramente se convertirán en futuros miembros de los equipos de trabajo donde se presentará interacción.

Con lo anterior dispuesto, se puede dar el primer paso fundamental que corresponde a identificar, conocer, incluso ubicar, los sitios donde suceden en el mundo real, aquellos procesos que necesitan ser mejorados, descritos, analizados o propuestos para generar mejoras.

Figura 4. Pasos para tener en cuenta

Técnicas de recolección de datos

- 1** Encuestas
- 2** Entrevistas
- 3** *Focus group*
- 4** Observación

Determinar



Insumos

- 1** Roles
- 2** Contactos
- 3** Visión
- 4** Misión
- 5** Objetivos
- 6** Organigrama
- 7** Gestión documental

Aproximación inicial



Procesos y subprocesos

- 1** Lugares
- 2** Actores
- 3** Tiempos
- 4** Costos

Determinar el equipo de trabajo

Con base en la información recolectada y a la obtención de los requerimientos que permitieron identificar los procesos que necesitan mejorarse, se procede a formar el equipo de trabajo que los identificará, analizará y documentará. Lo ideal es que los responsables que conocen a detalle los procesos, conformen el equipo.

Por lo tanto, el equipo debería estar integrado por el siguiente grupo humano:

- a) El líder del proyecto o el proceso elegido.
- b) Los empleados vinculados al proceso.
- c) Un alto miembro de la organización que conoce los objetivos estratégicos.
- d) Los trabajadores de otras áreas que cuentan con la actual experiencia y que desean conocer las mejoras que se llevarán a cabo.

Generar los diagramas de los procesos de negocio

Se trata de representar en una secuencia lógica, los elementos expresados en la TGS, nombre del proceso, el cual debe ser un verbo (fabricar, validar, informar) que permite modelar en un dibujo, el proceso de manera rápida, dinámica y sencilla acerca de cómo fluyen las entradas en cada elemento y hacia dónde se deben dirigir sus salidas. En el diagrama es fundamental definir las competencias de cada uno de los integrantes del equipo en cada tarea.

Para ampliar la información sobre este tema, lo invitamos a consultar el PDF **“Generar los diagramas de los procesos de negocio”**, el cual se encuentra en la carpeta Anexos.

Realizar un diagnóstico de cada proceso

Esta labor permite a las empresas detectar, por medio la implementación de estrategias, los problemas de funcionamiento y deficiencias. Estas estrategias son:

- a) Entender las causas propias que maneja los casos de cada proceso.
- b) Ejecutar simulacros de relevamiento de procesos unido a sus responsables.
- c) Apoyarse en herramientas de software para registrar resultados, controles, métricas, análisis de tiempos, costos y materias primas.

Puntos de mejora

Luego del diagnóstico de la situación actual, es importante construir un informe que dé a conocer el estado anterior y el estado actual hacia dónde se quiere llegar con las mejoras. En este informe se recomienda, de manera obligatoria, el poder relacionar los alcances de las soluciones unidos en forma transversal con los objetivos estratégicos de la organización.

Nuevo proceso mejorado

Los resultados obtenidos de la aplicación de cada uno de los pasos anteriores, se deben compilar y almacenar para convertirse en información de salida, con el objetivo de llevar trazabilidad. Al realizar encuentros o reuniones de análisis de dicha información, se genera el diseño de un nuevo proceso, el cual estará destinado al logro de los objetivos estratégicos de la compañía y así llegar a un estadio de mayor eficiencia y eficacia.

Tabla 1. Sistemas de compras en línea

Entradas	Subproceso o actividad	Salidas
Datos del solicitante (nombres, apellidos, identificación, dirección de contacto). Producto que comprar (valor, cantidad).	Recepción de solicitud de compra.	Número único de identificación de la compra. Factura. Envío.
Número único de identificación de la compra.	Almacén.	Producto.
Número único de identificación de la compra.	Almacén.	Producto.
Insumos. Mano de obra.	Fábrica.	Producto elaborado.
Facturas.	Contabilidad	Informes contables.

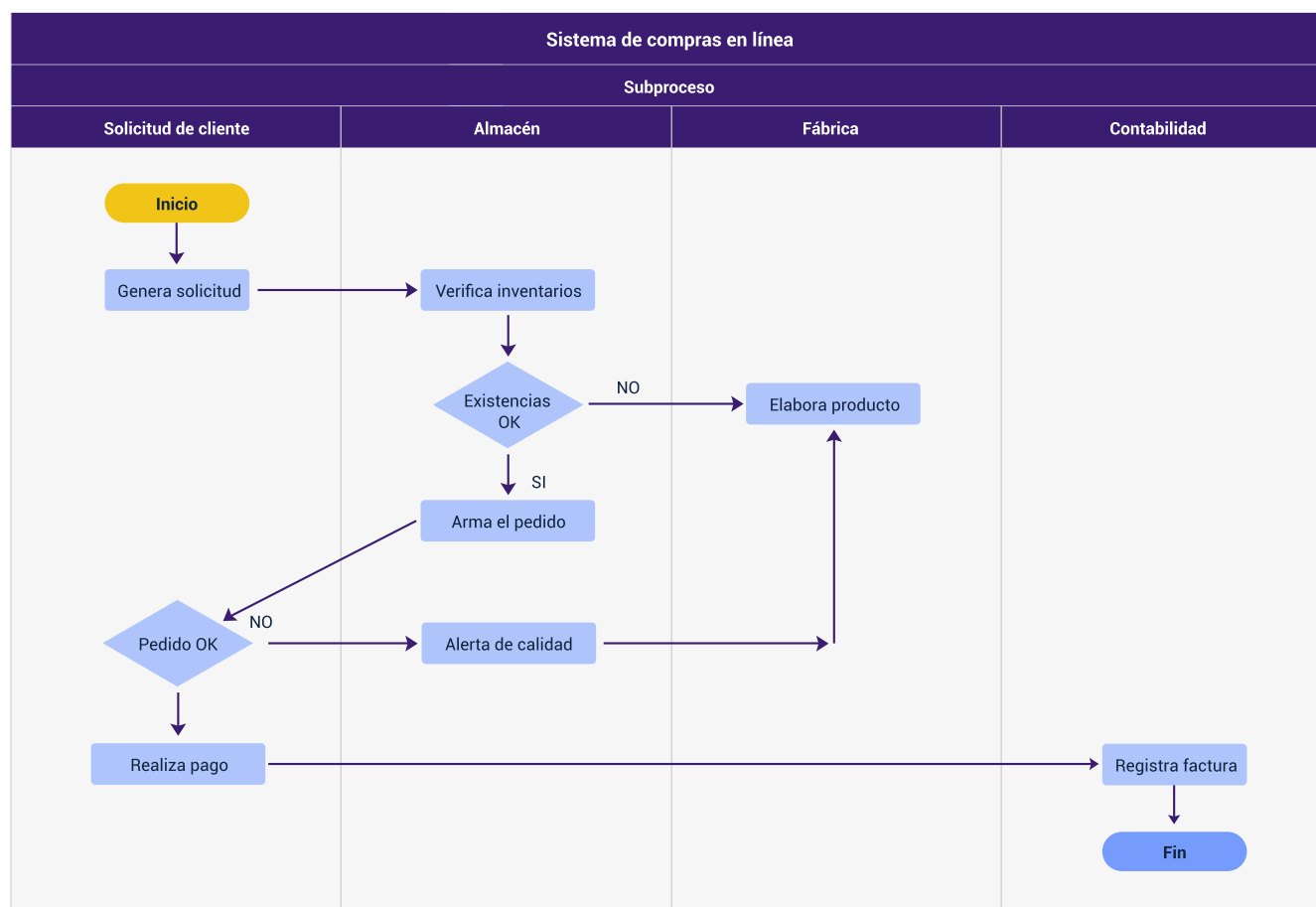
Puesto que el sistema de compras en línea no intercambia elementos de información con el medio ambiente que lo rodea, se considera un sistema cerrado. De lo anterior se concluye:

- **Se pueden inferir las siguientes relaciones:**
 - a) Usuario solicitante contra el proceso de solicitud de compra, en donde un usuario puede generar una o muchas solicitudes de compras.
 - b) El almacén maneja el inventario de productos.
 - c) La fábrica provee de muchos productos al almacén.
 - d) El almacén promociona el producto para la solicitud.
 - e) El usuario realiza el pago del producto.
 - f) Contabilidad registra la factura de compra generada por la solicitud.

- **Actores responsables:**
 - a) Usuario que genera la solicitud.
 - b) Jefe de ventas.
 - c) Administrador del sistema online.
 - d) Jefe de contabilidad.
 - e) Jefe del control de operaciones de fábrica.
 - f) Jefe de inventarios.
 - g) Empleados de la fábrica.
 - h) Gerencia de área.

La siguiente imagen nos presenta el ejemplo del diagrama de procesos.

Figura 5. Ejemplo del diagrama de procesos



2. Ingeniería de requisitos

Las siguientes son definiciones de ingeniería de requisitos de algunos autores.

La ingeniería de requisitos es la disciplina para desarrollar una especificación completa, consistente y no ambigua, la cual servirá como base para acuerdos comunes entre todas las partes involucradas y en dónde se describen las funciones que realizará el sistema. (Boehm, 1979).

La ingeniería de requisitos es el proceso de estudiar las necesidades del usuario para llegar a una definición de requisitos de sistema, hardware o software. (IEEE, 1990).

La ingeniería de requisitos puede considerarse como un proceso de descubrimiento y comunicación de las necesidades de clientes y usuarios y la gestión de los cambios de dichas necesidades. (Amador, 2000).

El término IR “ingeniería de requisitos” ha surgido para englobar los procesos de desarrollo y gestión de requisitos en el ciclo de vida del software, el primer término (ingeniería), se enfoca en las actividades de obtención, análisis, especificación y validación de los requisitos que permitirá alcanzar los objetivos del negocio y el segundo (requisitos), está centrado en la administración de los mismos y tiene como propósito central, la gestión de los cambios y la trazabilidad, de esta forma la IR proporciona el mecanismo apropiado para:

- Entender lo que el cliente quiere.
- Analizar las necesidades.
- Evaluar la factibilidad.
- Negociar una solución razonable.

- Especificar la solución sin ambigüedades.
- Validar la especificación.
- Administrar los requisitos conforme éstos se transforman en un sistema operacional.

Etapas de la ingeniería de requisitos

Existen cuatro (4) etapas en un proceso usual de ingeniería de requisitos y que son utilizadas para el desarrollo de un producto único: elicitación, análisis, especificación y validación de los requisitos.

- **Elicitación**

Actividad involucrada en el descubrimiento de los requisitos del sistema.

Aquí los analistas deben trabajar junto con el cliente para descubrir el problema que el sistema debe resolver, los diferentes servicios que el sistema debe prestar y las restricciones que se pueden presentar.

Los principales objetivos que se deben alcanzar son los siguientes:

- Conocer el dominio del problema, de forma tal que los analistas puedan entenderse con los clientes y usuarios y sean capaces de transmitir dicho conocimiento al resto del equipo.
- Descubrir necesidades reales entre clientes y usuarios, haciendo énfasis en aquellas que la mayor parte de las veces se asumen y toman por implícitas.
- Consensuar los requisitos entre los propios clientes y usuarios hasta obtener una visión común de los mismos.

- **Análisis**

Sobre la base de la obtención realizada previamente, comienza esta fase, la cual tiene como propósito, descubrir problemas con los requisitos del sistema identificados hasta el momento, para ello se basa en los siguientes objetivos:

- Detectar conflicto en los requisitos que suelen provenir de distintas fuentes y presentar contradicciones o ambigüedades debido a su naturaleza informal.
- Profundizar en el conocimiento del dominio del problema puede facilitar el proceso de construir un producto útil para clientes y usuarios (Durán, 2000).

En esta fase, el analista proporciona un sistema de retroalimentación que refina el entendimiento conseguido en la etapa de obtención.

- **Especificación**

Aquí se documentan los requisitos acordados con el cliente, en un nivel apropiado de detalle. En la práctica, esta etapa se realiza conjuntamente con el análisis, por lo que se puede decir que la especificación es el “pasar en limpio” el análisis realizado previamente aplicando técnicas y/o estándares de documentación, como la notación UML (Lenguaje de Modelado Unificado), que es un estándar para el modelado orientado a objetos, por lo que los casos de uso y la obtención de requisitos basada en los casos de uso se utilizan cada vez más para la obtención de requisitos.

- **Validación**

Por último, la validación garantiza que los requisitos, una vez analizados y resueltos los posibles conflictos, correspondan realmente a las necesidades de clientes y usuarios, para evitar que, a pesar de que el producto final sea técnicamente correcto, no sea satisfactorio. La validación puede llevar al analista a reescribir algunas especificaciones de requisitos y, en otros casos, a obtener nuevos, producto de la aparición de necesidades que hasta entonces estaban ocultas, para volver a evaluar el análisis inicial, o para corregir y perfeccionar el conjunto de requisitos documentados.

2.1. Ciclo de vida del software

También conocido como SDLC o Systems Development Life Cycle, es el proceso que se sigue para construir y hacer evolucionar un determinado software. El ciclo de vida permite iniciar una serie de fases mediante las cuales se procede a la validación y al desarrollo del software garantizando que se cumplan los requisitos para la aplicación y verificación de los procedimientos de desarrollo; para ello, se utilizan métodos del ciclo del software, que indican distintos pasos a seguir para el desarrollo de un producto.

Si bien existen diferentes ciclos de desarrollo de software, la normativa ISO/IEC/IEEE 12207:2017 establece que:

Es un marco común para los procesos del ciclo de vida de los programas informáticos, con una terminología bien definida, a la que pueda remitirse la industria del software. Contiene procesos, actividades y tareas aplicables durante la adquisición, el suministro, el desarrollo, el funcionamiento, el mantenimiento o la eliminación de sistemas, productos y servicios informáticos. Estos procesos del ciclo de

vida se llevan a cabo mediante la participación de los interesados, con el objetivo final de lograr la satisfacción del cliente (s.p.).

A continuación, se indican cuáles son los elementos que integran un ciclo de vida:

- **Fases.** Una fase es un conjunto de actividades relacionadas con un objetivo en el desarrollo del proyecto. Se construye agrupando tareas (actividades elementales) que pueden compartir un tramo determinado del tiempo de vida de un proyecto. La agrupación temporal de tareas impone requisitos temporales correspondientes a la asignación de recursos (humanos, financieros o materiales).
- **Entregables.** Son los productos intermedios que generan las fases. Pueden ser materiales o inmateriales (documentos, software). Los entregables permiten evaluar la marcha del proyecto mediante comprobaciones de su adecuación o no a los requisitos funcionales y de condiciones de realización previamente establecidos.

Fases

Las fases del modelo de ciclo del software son: planificación, análisis, diseño, implementación, pruebas y mantenimiento, las cuales se describen a continuación.

- **Fase Planificación:**

En esta primera fase se realiza el planteamiento del problema, se definen alcances y objetivos del software.

Objetivos: estudio de viabilidad, realizar planificación detallada.

- **Fase Análisis (definición de requisitos):**

Esta fase busca definir los requisitos que son los que dirigirán el desarrollo del proyecto de software.

Objetivos: conocer los requisitos, asegurar que los requisitos son alcanzables, formalizar acuerdo con el cliente.

- **Fase Diseño:**

En esta fase se estudian posibles opciones de implementación para el software que hay que construir, estructura general del mismo.

Objetivos: identificar soluciones tecnológicas, asignar recursos materiales, proponer, identificar y seleccionar, establecer métodos de validación, ajustar especificaciones.

- **Fase Pruebas:**

Esta fase busca detectar fallos cometidos en las etapas anteriores para corregirlos.

Objetivos: realizar los ajustes necesarios para corregir posibles errores o inconsistencias.

- **Fase Mantenimiento:**

En esta fase se realizan tres puntos referenciados: mantenimiento correctivo, mantenimiento adaptativo y mantenimiento perfectivo.

Objetivos: operación, asegurar que el uso del proyecto es el que se pretendía, mantenimiento.

Paradigmas de los modelos de ciclo de vida del software

Con la finalidad de proporcionar una metodología común entre el cliente y la empresa de software, se utilizan los modelos de ciclos de vida o paradigmas de

desarrollo de software para plasmar las etapas y la documentación necesaria, de manera que cada fase se valide antes de continuar con la siguiente.

Un modelo de ciclo de vida de software es una vista de las actividades que ocurren durante el desarrollo de software e intenta determinar el orden de las etapas involucradas y los criterios de transición asociados entre estas.

Según la norma 1074 IEEE se define al ciclo de vida del software como “una aproximación lógica a la adquisición, el suministro, el desarrollo, la explotación y el mantenimiento del software”.

La ISO/IEC 12207 Information Technology / Software Life Cycle Processes señala que es:

Un marco de referencia que contiene los procesos, las actividades y las tareas involucradas en el desarrollo, la explotación y el mantenimiento de un producto de software, abarcando la vida del sistema desde la definición de los requisitos hasta la finalización de su uso (2008).

Existen modelos preestablecidos con los cuales se puede elaborar un proyecto; a continuación, se mencionan los diferentes paradigmas de modelos de ciclo de vida para desarrollar software.

- **Paradigma tradicional**

Los paradigmas tradicionales se identifican, fundamentalmente, por ser lineales, es decir, se trata de completar cada proceso de principio a fin, hasta que quede listo para avanzar a la segunda fase del ciclo del software.

Si bien es verdad que las metodologías actuales están basadas en lo que fueron los paradigmas tradicionales, hoy en día se ha evolucionado, sin embargo, los paradigmas tradicionales se mantienen.

Desventaja: Pérdida de tiempo si se encuentran errores en una fase avanzada, porque al devolverse, se debe pasar nuevamente por todas las fases y reestructurar, de acuerdo con las modificaciones.

- **Paradigma orientado a objetos**

Las etapas de desarrollo de software en el paradigma orientado a objetos, se conforma principalmente por la creación de clases, análisis de requisitos y el diseño. Con este paradigma se pretende que el código fuente sea reutilizable para otros proyectos.

- **Paradigma de desarrollo ágil**

El objetivo de este paradigma es el desarrollo de proyectos en poco tiempo, se simplifican procesos tediosos, se agilizan las fases del desarrollo y las interacciones se hacen en corto tiempo.

Una de las principales diferencias con los paradigmas anteriores, es que el cliente se ve involucrado en el proyecto durante el desarrollo de este, así, el cliente sugiere mejoras, propone ideas y se mantiene al tanto del desarrollo del producto, a diferencia del paradigma tradicional y el orientado objeto, donde el cliente únicamente está al principio.

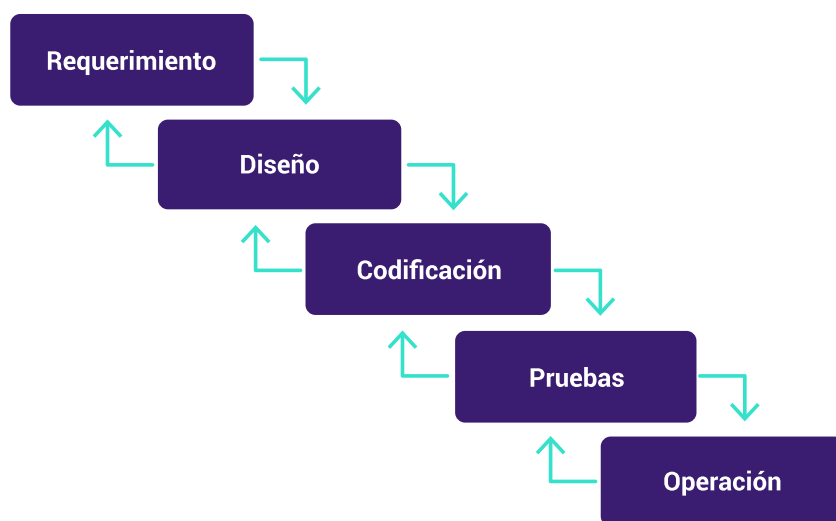
A continuación, se revisan los modelos del paradigma tradicional más utilizados.

- **Modelo en cascada**

Uno de los primeros modelos de ciclo de vida del desarrollo fue establecido por W. Royce en 1970 y es conocido como el “modelo de cascada” (waterfall model).

En su concepción básica, cada una de las actividades genera, como salidas, productos y modelos que son utilizados como entradas para el proceso subsiguiente; esto supone que una actividad debe terminarse (por lo menos, en algún grado) para empezar la siguiente.

Figura 6. Modelo de cascada del ciclo de vida del desarrollo



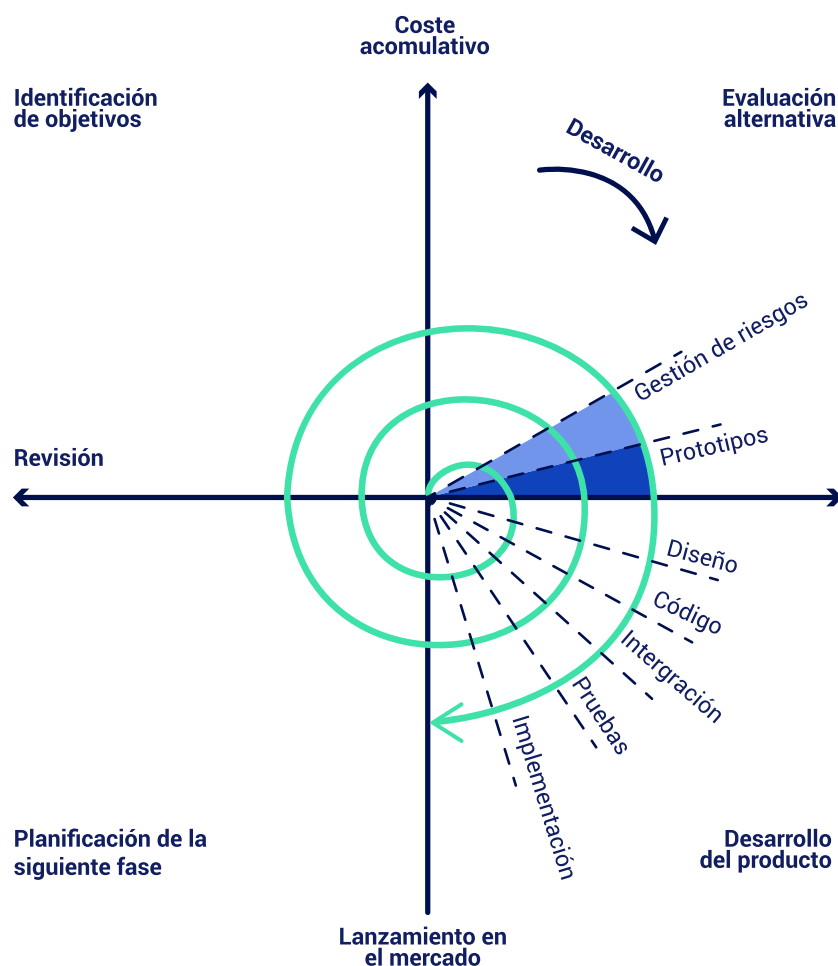
- **Modelo espiral**

Fue diseñado por Boehm en el año 1988 y se basa en una serie de ciclos repetitivos para ir ganando madurez en el producto final. El espiral se

repite las veces que sea necesario hasta que el cliente o el usuario obtenga la satisfacción de sus necesidades.

En este modelo hay 4 actividades que envuelven a las etapas: planificación, análisis de riesgo, implementación y evaluación. Una de sus principales ventajas es que los riesgos van disminuyendo conforme avanzan los ciclos o interacciones.

Figura 7. Modelo espiral del ciclo de vida del desarrollo



Nota. Adaptada de Boehm (2018).

- **Modelo iterativo o por prototipos**

Este modelo consiste en un procedimiento que permite al equipo de desarrollo, diseñar y analizar una aplicación que represente el sistema que será implementado (McCracken y Jackson, 1982).

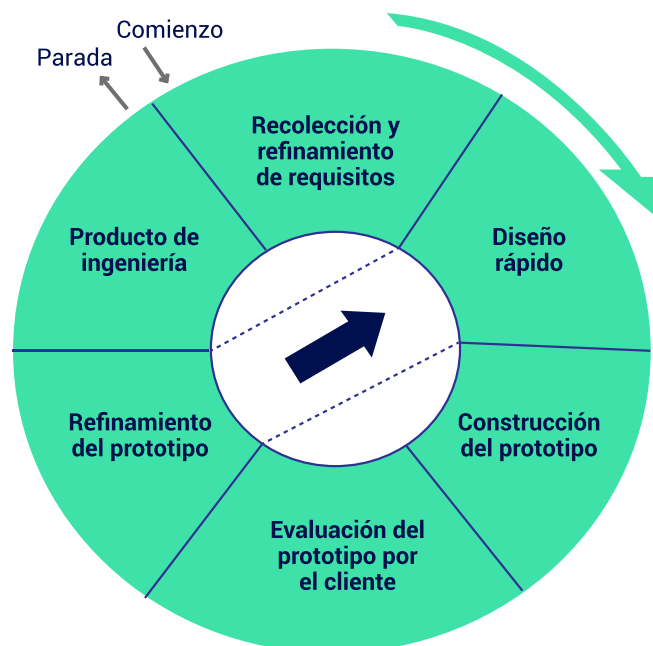
Objetivos

- Son un medio eficaz para aclarar los requisitos de los usuarios e identificar las características de un sistema que deben cambiarse o añadirse.
- Mediante el prototipo se puede verificar la viabilidad del diseño de un sistema.

Las etapas del modelo son:

- Colecta y refinamiento de los requerimientos y proyecto rápido.
 - Análisis.
 - Especificación del prototipo.
- Diseño rápido.
- Construcción del prototipo.
- Evaluación del prototipo por el cliente.
- Refinamiento del prototipo.
 - Diseño técnico.
 - Programación y test.
 - Operación y mantenimiento.
- Producto de ingeniería.

Figura 8. Modelo iterativo o por prototipos



Con respecto a los modelos del ciclo de vida del paradigma ágil, estos se caracterizan por estar basados en etapas del ciclo de vida del software tradicional, pero combinándolas con algunas técnicas, al respecto se pueden revisar los siguientes:

Modelo Scrum

Este modelo se basa en el desarrollo incremental, es decir, conforme pasen las fases y la iteración, mayor será el tamaño del proyecto que se está desarrollando.

Los procesos que utiliza son:

- Product Backlog.
- Sprint Backlog.
- Sprint Planning Meeting.

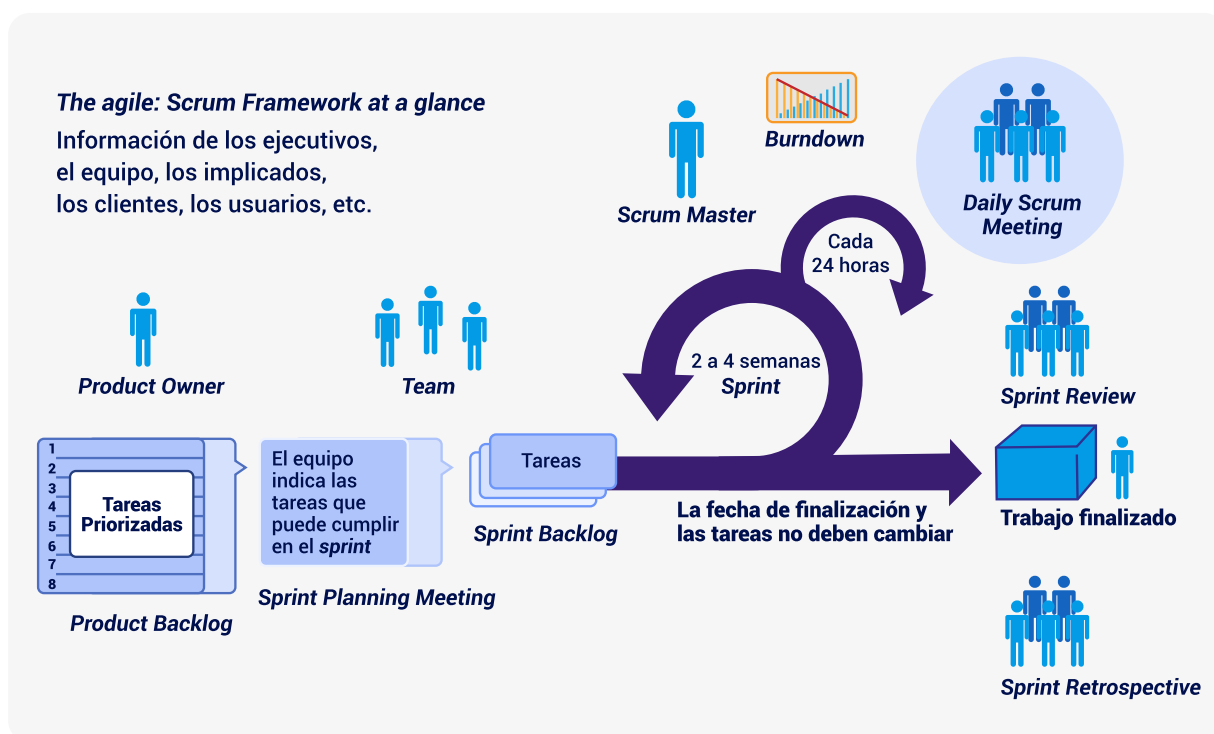
- Daily Scrum.
- Sprint Review.
- Sprint Retrospective.

El Scrum consiste en realizar un análisis de los requerimientos del sistema (Product Backlog), señalar cuáles serán los objetivos a corto o mediano plazo dentro de un sprint, o sea, la fase de desarrollo. Posteriormente, los desarrolladores harán lo suyo, se realizarán algunas pruebas y se retroalimentará de acuerdo con lo conseguido al terminar la última fase.

Ventajas

- **Gestión regular de las expectativas del usuario:** los usuarios participan y proponen soluciones.
- **Resultados anticipados:** no es necesario esperar hasta el final para ver resultados.
- **Flexibilidad y adaptación:** se adapta a cualquier contexto, área o sector.
- **Gestión sistemática de riesgos:** los problemas son gestionados en el mismo momento de su aparición.

Figura 9. Modelo ágil Scrum



Modelo Kanban

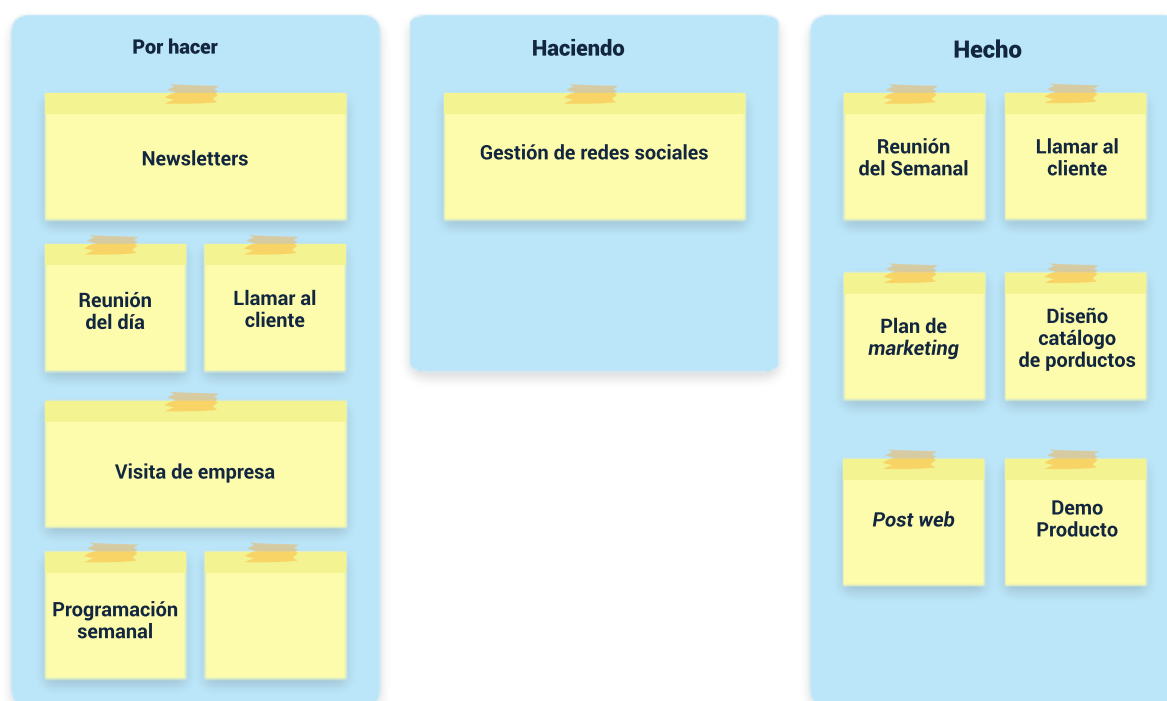
David J. Anderson (reconocido como el líder de pensamiento de la adopción del Lean/Kanban para el trabajo de conocimiento), formuló el método Kanban como una aproximación al proceso evolutivo e incremental y al cambio de sistemas para las organizaciones de trabajo. El método está enfocado en llevar a cabo las tareas pendientes y los principios más importantes pueden ser divididos en cuatro principios básicos y seis prácticas.

El modelo Kanban es uno de los modelos más visuales de las metodologías ágiles; este consiste en la creación de un tablero con etiquetas, donde se seccionan cada una de las fases de su desarrollo, además se clasifican de acuerdo con los equipos de trabajo y se les asignan objetivos a corto, mediano y largo plazo.

Mediante la metodología japonesa Kanban se:

- Define el flujo de trabajo.
- Establecen las fases del ciclo de producción.
- Stop Starting, start finishing.
- Tiene un control.

Figura 10. Modelo ágil Kanban



Nota. Adaptada de Anderson.

Modelo XP o programación extrema

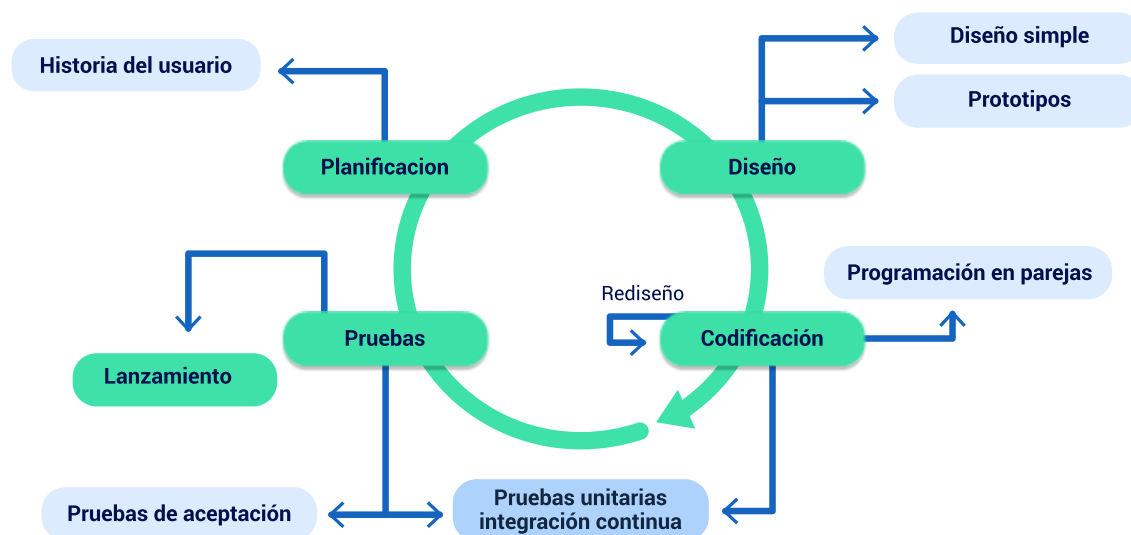
La programación extrema o eXtreme Programming (XP) es un enfoque de la ingeniería de software, formulado por Kent Beck, autor del primer libro sobre este tema: *Extreme Programming Explained: Embrace Change* (1999). Esta metodología es adaptable según las necesidades y requerimientos a implementar,

además, el cliente se encuentra involucrado en el proceso de desarrollo, lo que hace que el producto pueda ser terminado en un menor tiempo.

Características principales de la programación extrema:

- Tipo de desarrollo iterativo e incremental.
- Pruebas unitarias.
- Trabajo en equipo.
- Trabajo junto al cliente.
- Corrección de errores.
- Reestructuración del código.
- El código es de todos.
- El código simple es la clave.

Figura 11. Modelo XP



Nota. Adaptada de Muradas (2020).

2.2. Fase de definición de requisitos

En esta primera fase del ciclo de vida del software, también llamada fase de análisis, se recopila, se examina y se formulan los requisitos del cliente, así como la verificación de las posibles restricciones que se puedan aplicar.

Por eso, la etapa de análisis en el ciclo de vida del software corresponde al proceso a través del cual se intenta descubrir qué es lo que realmente se necesita y se llega a una comprensión adecuada de los requerimientos del sistema (las características que el sistema debe poseer).

La etapa de análisis es esencial, debido a que sin esta no se sabe con precisión qué es lo que se necesita y ningún proceso de desarrollo permitirá obtenerlo. El problema que comúnmente se presenta al inicio de la etapa de desarrollo, es que el cliente no sepa exactamente que necesita, por tanto, se debe averiguar con ayuda de diferentes técnicas.

Por otra parte, la inestabilidad de los requerimientos de un sistema es inevitable, porque se estima que el 25 % de los requerimientos iniciales de un sistema, cambian antes que el sistema comience a utilizarse. Por ello, muchas prácticas resultan efectivas para gestionar adecuadamente los requerimientos de un sistema y, en cierto modo, controlar su evolución.

En la siguiente tabla, se describen las actividades y los artefactos que se realizan en la fase de definición de requisitos.

Tabla 2. Actividades y artefactos de la fase de definición de requisitos

Fase	Actividades	Artefactos
Análisis (definición de requisitos).	Definición del alcance del proyecto.	
Análisis (definición de requisitos).	Identificación del negocio.	Modelo del negocio.
Análisis (definición de requisitos).	Toma de requerimientos.	Análisis y realización de casos de uso.
Análisis (definición de requisitos).	Estudio de procesos de negocio.	Modelo de procesos y actividades de negocio.
Análisis (definición de requisitos).	Calendarización del proyecto.	Cronograma del proyecto.

2.3. Requisitos

Un requisito es una “condición o capacidad que necesita el usuario para resolver un problema o conseguir un objetivo determinado” (IEEE, 1990).

Los requisitos comunican las expectativas de los consumidores de productos software; de otra parte, los requisitos pueden ser obvios o estar ocultos, conocidos o desconocidos, esperados o inesperados, desde el punto de vista del cliente.

Importancia de los requisitos

Los requisitos cobran importancia dentro del ciclo de vida del software, porque:

- Establecen el alcance del trabajo subsecuente, pueden definir estrategias de desarrollo, riesgos, tomar decisiones de negocio (viabilidad de negocio), de proyecto (tiempo, recursos), de sistema (arquitectura).

- Indican al equipo del proyecto qué requieren los usuarios (necesidades de negocio).
- El éxito o fracaso de un proyecto está altamente influenciado por la calidad de los requisitos y el proceso para gestionarlos durante el desarrollo de un producto.

A continuación, se pueden revisar las características que los requisitos deben cumplir, de acuerdo con Pfleeger (2002).

- **Necesario:** si se tiene alguna duda acerca de la necesidad del requerimiento, se puede preguntar “¿Qué sería lo peor de no incluirlo?” Si no se encuentra una respuesta o cualquier consecuencia, entonces es probable que no sea un requerimiento necesario.
- **Completo:** un requerimiento está completo si no necesita ampliar detalles en su redacción, es decir, si se proporciona la información suficiente para su comprensión.
- **Consistente:** un requerimiento es consistente si no es contradictorio con otro requerimiento.
- **Correcto:** acuerdo entre dos partes. Contiene una sola idea.
- **Factible:** el requerimiento deberá de ser totalmente factible y dentro de presupuesto, calendario y otras restricciones, si se tiene alguna duda de su factibilidad, hay que investigar, generar pruebas de concepto para saber su complejidad y factibilidad, si aun así el requerimiento es no factible, hay que revisar la visión del sistema y replantear el requerimiento.
- **Modificable:** los cambios en los requisitos deben hacerse de manera sistemática, y debe tenerse en cuenta su impacto en otros requisitos.

- **Priorizado:** categorizar el requerimiento nos ayuda a saber el grado de necesidad del mismo: esencial/crítico, deseado, opcional, verificable.
- **Verificable:** si un requerimiento no se puede comprobar, entonces, ¿cómo se sabe si se cumplió con él o no? Debe ser posible verificarlo, ya sea por inspección, análisis de prueba o demostración. Cuando se escriba un requerimiento, se deberán determinar los criterios de aceptación.
- **Rastreable:** la especificación se debe organizar de tal forma que cada función del sistema se pueda rastrear hasta su conjunto de requerimientos correspondiente. Facilita las pruebas y la validación del diseño.
- **Claro:** un requerimiento es conciso si es fácil de leer y entender, su redacción debe ser simple y clara para quienes lo consulten en un futuro.

Clasificación

Los requerimientos se pueden definir de distintas maneras, la primera clasificación se encuentra relacionada con el nivel de descripción con la que cuentan estos y dentro de este tipo de clasificación se encuentran:

- **Requerimientos de usuario**

Son declaraciones, en lenguaje natural y en diagramas, de los servicios que se espera que el sistema proporcione y de las restricciones bajo las cuales debe funcionar.

- **Requerimientos de sistema**

Estos requerimientos establecen con detalle las funciones, servicios y restricciones operativas del sistema. El documento de requerimientos del

sistema deberá ser preciso, y definir exactamente lo que se va a desarrollar.

En la siguiente clasificación se observa la que se da a los requerimientos del sistema, la cual se encuentra dividida con base en lo que se va a describir. Las clasificaciones son:

- **Requerimientos funcionales**

Son declaraciones de los servicios que debe proporcionar el sistema, de la manera en que este debe reaccionar a entradas particulares, o también pueden declarar explícitamente lo que el sistema no debe hacer.

- **Requerimientos no funcionales**

Son restricciones de los servicios o funciones ofrecidos por el sistema. Incluyen restricciones de tiempo, sobre el proceso de desarrollo y estándares. Dentro de estos requerimientos se encuentra todo lo referente a la fiabilidad, el tiempo de respuesta y la capacidad de almacenamiento.

En la siguiente tabla se presentan algunos ejemplos sobre requisitos funcionales y no funcionales.

Tabla 3. Requisitos

Funcionales	No funcionales
Se debe ingresar cédula, nombre y teléfono de cada cliente.	Las consultas deben resolverse en menos de 3 segundos.
Se requiere un listado de clientes por zona.	El lenguaje de programación debe ser Java.

Se puede ampliar el tema de requisitos funcionales y no funcionales en los videos que se proponen dentro del material complementario.

3. La fase de elicitación de requisitos

El propósito de la elicitación de requerimientos es ganar conocimientos relevantes del problema, que se utilizarán para producir una especificación formal del software necesario para resolverlo.

“Un problema puede ser definido como la diferencia entre las cosas como se perciben y las cosas como se desean”. (Gause y Weinberg 1989).

Aquí se ve la importancia que tiene una buena comunicación entre desarrolladores y clientes; de esta comunicación con el cliente depende que sus necesidades queden claras. Además, al final de la fase de análisis de requerimientos, el analista podría llegar a tener un conocimiento extenso en el dominio del problema.

La elicitación de requisitos es la actividad que se considera como el primer paso en un proceso de ingeniería de requisitos; su significado hace referencia a la puesta en marcha de técnicas que sirven para recopilar conocimiento o información y los objetivos de esta fase de elicitación, son:

- Conocer el dominio del problema para poder comunicarse con clientes y usuarios y entender sus necesidades.
- Conocer el sistema actual (manual o informatizado) y sus aspectos positivos y negativos.
- Identificar las necesidades, tanto explícitas como implícitas, de clientes y usuarios y sus expectativas sobre el sistema a desarrollar.

3.1. Planeación

La planeación busca definir las tareas a realizar para elegir y planificar las técnicas a emplear durante la actividad de elicitación de la fase de ingeniería de requisitos del

desarrollo de software. En la siguiente tabla se presenta una relación de estas tareas y sus correspondientes procesos.

Tabla 4. Tareas para elicitación de requisitos

Tareas	Proceso
Identificar las fuentes.	Lista de fuentes de requerimientos.
Identificar interesados del producto.	Categorías de los interesados (stakeholders).
Matriz stakeholders (Describir necesidades y criterios de éxito).	Perfil de stakeholders.
Revisar técnicas.	Identificar combinaciones de técnicas, entrevistas, grupos focales, encuestas, prototipos.
Captura de interesados.	Plan de captura de interesados.

Nota: tomado de Durán y Bernárdez (2001)

A continuación, se describen los procesos relacionados con las tareas para elicitación de requisitos:

Identificar las fuentes de requerimientos

Existe un conjunto de fuentes de requisitos en cada proyecto de desarrollo de software, así, usuarios y expertos abastecen de información detallada acerca del problema y necesidades del usuario. Los procesos y sistemas existentes representan, también, fuentes de requisitos; además, la documentación existente como manuales, formularios y reportes, incluso especificaciones de requisitos anteriores, puede proveer información útil acerca de la organización y su entorno.

En el proceso de esta actividad se identifican:

- Interesados relevantes.
- Documentación que se puede usar como fuente de información de los requerimientos.
- Fuentes de información externas.

Las fuentes de requerimientos incluyen los propietarios del problema, los stakeholders, documentos y otros sistemas (Pearson, 2002). En ese sentido, los requerimientos pueden obtenerse en diversas fuentes que pueden clasificarse en gente (people), productos o documentos, pero cualquiera sea la fuente de esos requerimientos deben ser chequeados con los stakeholders.

Estas fuentes de requerimientos, se pueden clasificar en:

- **Fuentes primarias**

Aportan material de primera mano (es protagonista o testigo de los hechos), estas fuentes contienen información original, que ha sido publicada por primera vez y que no ha sido filtrada, interpretada o evaluada por nadie más.

- **Fuentes secundarias**

Toman y reproducen la información que le aportó una fuente primaria. Son las que contienen información primaria, sintetizada y reorganizada y están especialmente diseñadas para facilitar y maximizar el acceso a las fuentes primarias o a sus contenidos. Parten de datos preelaborados, como pueden ser datos obtenidos de anuarios estadísticos, internet, medios de comunicación, bases de datos procesadas con otros fines, artículos y documentos relacionados con un tema, libros, tesis, informes oficiales, etc.

- **Fuentes terciarias**

Son guías físicas o virtuales que contienen información sobre las fuentes secundarias. Forman parte de la colección de referencia de una biblioteca; facilitan el control y acceso a toda la gama de repertorios de referencia, como las guías de obras de referencia, o a un solo tipo, como las bibliografías.

Por otra parte, las fuentes de información, pueden ser orales, escritas o de otro tipo, dependiendo de cómo se transmitan los datos. A continuación, se pueden revisar algunos ejemplos de fuentes de información.

Las fuentes de información o documentación pueden hallarse en diversos soportes.

- Grabaciones audiovisuales y grabaciones auditivas.
- Libros, artículos, prensa escrita y básicamente cualquier tipo de soporte que permita capturar y preservar la información, para recuperarla luego.
- Los testimonios, los relatos, las reseñas, los ensayos, las páginas web, las reflexiones, los listados bibliográficos y los índices.
- Las grabaciones profesionales, accidentales o clandestinas, las fotografías, las filmaciones e incluso ilustraciones.

Identificar interesados del producto

Uno de los primeros pasos en el proceso es el análisis e identificación de todas las personas relevantes que tienen un grado de interés en el proyecto. Los interesados (stakeholders), son los individuos y organizaciones que están relacionados activamente en un proyecto de software; tienen influencia directa o indirecta sobre

los requisitos, o sus intereses se ven afectados por el proyecto (Baar, 2006, Ventura, 2002).

En resumen, son grupos o individuos que están interesados en el producto de software que se está desarrollando y necesitarán estar informados acerca del progreso, conflictos, cambios y prioridades del proceso de desarrollo del producto.

Los stakeholders se dividen en dos grupos:

- **Primarios**

Son aquellas personas indispensables para el correcto funcionamiento de la organización, y tienen una relación económica directa con la empresa. Estos pueden ser sus socios, clientes y accionistas.

- **Secundarios**

Son los entes que no participan directamente de la compañía, pero también son afectados por sus resultados. En este círculo se encuentran los competidores, el mercado y las personas en general.

A continuación, se listan roles más generales de las personas involucradas con sus términos similares, aunque cabe resaltar que existen leves diferencias entre ellos (Sommerville y Sawyer, 2005):

- Líder de proyecto / Administrador de proyecto / Gerente de proyecto.
- Analista / Ingeniero de requisitos.
- Ingeniero de sistemas / Arquitecto.
- Programador / Desarrollador / Ingeniero de software.
- Probador / Asegurador de la calidad.
- Administrador de bases de datos.

En la siguiente tabla se presentan los principales roles involucrados en el proceso de ingeniería de requisitos, así como las actividades en las que tienen mayor participación.

Tabla 5. Roles involucrados en la ingeniería de requisitos

Rol	Descripción
Cliente	Representa a la persona u organización que solicita la creación de un sistema a un área de desarrollo y quien lo paga. Es con quien se negocia el tiempo, costo y alcance del proyecto. Pueden o no ser usuarios del sistema.
Usuario	Son las personas que interactuarán con el sistema. Proporcionan información fundamental para el éxito del proyecto, ya que conocen y conviven con los procesos diarios.
Líder de proyecto	Por parte del equipo de desarrollo, es el representante ante el cliente. Es la persona responsable de completar el proyecto exitosamente con los recursos dados.
Analista	Su labor se enfoca en la ingeniería de requisitos, los identifica, analiza, modela y documenta. Además, establece contacto directo con los usuarios y utiliza diversas técnicas de comunicación y de recopilación de información para lograr su objetivo.
Programador	Con base en los requisitos recibidos de parte de los ingenieros de requisitos, el programador realiza la codificación para producir el sistema deseado.
Asegurador de la calidad	Garantiza el cumplimiento del proceso y de los estándares del producto. Enfocado en los requisitos, los verifica y valida para imprimir la calidad desde las primeras etapas del desarrollo. Paralelamente, prepara planes de prueba para esos requisitos del sistema.
Arquitecto	Es el responsable del diseño de alto nivel y es clave a la hora de precisar los atributos de calidad del producto.

Nota: tomado de Ventura (2002)

Algunas de las técnicas que se pueden emplear para llevar a cabo la labor de análisis de los stakeholders incluyen entrevistas con los expertos, lluvia de ideas en grupo y lista de chequeo. Se espera que este grupo de personas identifiquen y

caractericen a los stakeholders objetivamente, por tal motivo es recomendable involucrar a personas de diferentes contextos (Karisen, 2002 citado en Wessinger, 2012).

Matriz de stakeholders

La utilización de esta herramienta de análisis permite clasificar a los involucrados en el proyecto según sus niveles de interés y poder sobre él, lo que facilita la priorización de los stakeholders más importantes para desarrollar las estrategias de gestión correspondientes.

- **Importancia de la matriz de stakeholders en los proyectos de desarrollo**

En los proyectos de desarrollo, la gestión de los stakeholders es de suma importancia para alcanzar el éxito de los proyectos, ya que el proceso de identificación de los involucrados y la definición de sus niveles de interés e influencia en el proyecto, marcarán el punto de partida para desarrollar estrategias que posibilitan obtener el apoyo requerido para alcanzar los objetivos por los que el proyecto es emprendido. Es por ello, que la matriz de stakeholders es una herramienta indispensable desde el comienzo del proyecto mismo, ya que proveerá de la información necesaria para gestionar, adecuadamente, las expectativas de los involucrados a lo largo del proyecto, maximizando las influencias positivas y mitigando los impactos negativos potenciales derivados de estos. Además, dado el carácter social de los proyectos de desarrollo, involucrar a la sociedad civil no debe ser solo un ejercicio de comunicación unidireccional, sino una oportunidad para lograr su apoyo al proyecto.

- **Proceso de armado de la matriz de stakeholders**

Para desarrollar la matriz de stakeholders es necesario identificar las entradas necesarias que proveerán la información con la que el líder y el equipo de proyecto trabajarán para desarrollar la matriz misma. Tales entradas pueden ser el acta de constitución de proyecto, documentos de adquisición, activos de los procesos y factores ambientales de la organización.

Entradas

- Acta de constitución del proyecto.
- Documentos de adquisición.
- Factores ambientales de la organización.
- Activos de los procesos de la organización.

Herramientas y técnicas

- Análisis de los interesados.
- Juicio de expertos.

Salidas

- Matriz stakeholders (registro y estrategias de gestión).

Para profundizar en detalle, lo invitamos a consultar la Guía PMBOK 6 – 49 procesos, entradas, herramientas y salidas, que se encuentra en el material complementario.

Descripción de los componentes de la matriz de stakeholders

A continuación, se presenta el concepto de cada uno de los componentes que estructuran la matriz de stakeholders.

a) Stakeholder

Es el nombre con el que se identifica al stakeholder.

b) Tipo

Identifica si el stakeholder desempeña un rol interno o externo al proyecto mismo. Los stakeholder pueden ser internos, como el personal de las unidades ejecutoras, el personal administrativo o ejecutivo de la organización, el personal de las entidades financiadoras con alto nivel de poder e influencia en el proyecto y sus recursos; o externos como los beneficiarios del proyecto, las instituciones del sector o las organizaciones de la sociedad civil, quienes serán de un modo u otro, impactados por los resultados del proyecto.

c) Objetivo o resultados

En este campo se enlistan los objetivos o resultados en los que el stakeholder muestra interés o en aquellos en los que puede influir positiva o negativamente con sus acciones. Esta información puede ser suministrada por el acta de constitución de proyectos, la estructura de la organización, la estructura de desglose de trabajo, los diferentes planes que conforman el proyecto, entrevistas a los mismos interesados, etc.

d) Acciones posibles con impacto positivo / negativo

Son las acciones que puede emprender el stakeholder y que pueden influir, negativa o positivamente, en los objetivos del proyecto en los que muestra su interés o en aquellos en los que puede influir debido a su jerarquía, estatus, recursos de los que dispone, entre otros.

e) Estrategias

Es un listado de acciones que se pueden emprender para obtener el apoyo necesario o evitar obstáculos por parte de los stakeholders durante la ejecución y conclusión del proyecto. Las estrategias se desarrollan considerando el tipo de stakeholder, los objetivos en los que está interesado, el nivel de interés y poder que puede ejercer en el proyecto y las acciones posibles que podría emprender para afectar tanto positiva como negativamente al proyecto.

f) Conclusiones

Es la síntesis sobre puntos clave a considerar para gestionar de manera efectiva las expectativas de los stakeholders. Las conclusiones se obtienen de relacionar, analizar y sintetizar toda la información vertida en la matriz de stakeholders.

Categorización de stakeholders y estrategias de gestión de las expectativas:

Como ya se había mencionado anteriormente, la matriz de stakeholders es una herramienta muy útil que permite clasificar a los involucrados en el proyecto según sus niveles de interés e influencia, priorizando a los más importantes y desarrollando así las estrategias correspondientes para gestionar sus expectativas. De la misma manera, su clasificación puede cambiar durante la vida del proyecto. Así, aquellos que fueron inicialmente identificados con un alto nivel de influencia en el proyecto, pueden ser reclasificados a un nivel más bajo durante otras etapas de la vida del proyecto.

La categorización de los stakeholders se lleva a cabo una vez que la información sobre éstos esté completa. Para ello se puede utilizar una matriz de 2x2 en la que se pueda graficar el grado de poder e interés que tiene el involucrado en el proyecto,

coadyuvando así a clasificar a cada stakeholder dentro del grupo para el cual se definen diferentes estrategias.

Figura 12. Ejemplo de matriz interés vs. Influencia



3.2. Técnicas e instrumentos para elicitar requisitos

Hay una variedad de técnicas propuestas para ingeniería de requerimientos (Herrera, 2003. p. 12), por lo que es primordial resaltar que estas técnicas pueden ser aplicables a las distintas fases del proceso de la ingeniería de requerimientos (IR), teniendo en cuenta las características propias del proyecto en particular que se esté desarrollándose para aprovechar al máximo su utilidad.

A continuación, se presentará una serie de técnicas destinadas a facilitar la elicitación correcta y efectiva de los requisitos dentro de un proceso de desarrollo.

Entrevista

La entrevista es una forma de recoger información de otra persona a través de una comunicación interpersonal que se lleva a cabo por medio de una conversación estructurada. (Braude, 2003)

En las entrevistas se pueden identificar tres fases: preparación, realización y análisis (Piattini *et al*, 1996), como se observa a continuación.

a) Preparación

El entrevistador debe documentarse e investigar la situación de la organización, analizando los documentos de la empresa disponible.

- Se debe intentar minimizar el número de entrevistados, considerando las entrevistas de cortesía.
- Analizar el perfil de los entrevistados.
- Definir el objetivo y el contenido de la entrevista.
- Planificar el lugar y la hora en la que se va a desarrollar la entrevista (es conveniente realizarla en un lugar confortable).
- Algunos proponen enviar previamente el entrevistado un cuestionario y un pequeño documento de introducción al proyecto de desarrollo.

b) Realización

Dentro de la realización de las entrevistas se distinguen tres etapas, tal como se expone en Piattini *et al*. (1996):

- **Apertura:** presentarse e informar al entrevistado sobre la razón de la entrevista.

- **Desarrollo:** cumplir las reglas del protocolo, hay que llegar a un acuerdo sobre cómo se va a registrar la información obtenida.

Durante esta fase se pueden emplear distintas técnicas:

- **Preguntas abiertas:** también denominadas de libre contexto (Gause y Weinberg, 1989), estas preguntas no pueden responderse con un "sí" o un "no", permiten una mayor comunicación y evitan la sensación de interrogatorio. Por ejemplo, "¿Qué se hace para registrar un pedido?", "Dígame qué se debe hacer cuando un cliente pide una factura" o "¿Cómo se rellena un recibo?".
- **Utilizar palabras apropiadas:** se deben evitar tecnicismos que no conozca el entrevistado y palabras o frases que puedan perturbar emocionalmente la comunicación (Goleman 1996).
- **Mostrar interés en todo momento:** es fundamental cuidar la comunicación no verbal (Davis, 1985) durante la entrevista: tono de voz, movimiento, expresión facial.
- **Terminación:** se termina recapitulando la entrevista agradeciendo el esfuerzo y dejando abierta la posibilidad de volver a contactar para aclarar conceptos o bien citándole para otra entrevista.

c) Análisis

Consiste en leer las notas, pasarlas en limpio, reorganizar la información, contrastarlas con otras entrevistas o fuentes de información, evaluar cómo ha ido la entrevista.

En estas entrevistas, el equipo de la ingeniería de requerimientos hace preguntas sobre el sistema que utilizan y sobre el sistema a desarrollar. Los requerimientos provienen de las respuestas a estas preguntas.

Las entrevistas se pueden clasificar fundamentalmente, en:

- **Entrevista estructurada**

Las preguntas en esta entrevista se deciden, previamente, de acuerdo con el detalle de información requerida.

- Recoge de forma sistemática y precisa la mayor información sobre los aspectos que quiere explorar.
- Las preguntas son prefijadas y definidas, las respuestas son esperadas e incluso se le dan al entrevistado en forma de varias opciones.
- Las etapas son planificadas.
- La interpretación de las respuestas se realiza de acuerdo con unos criterios establecidos.

Lo invitamos a consultar el PDF "**Entrevista estructurada**", el cual se encuentra en la carpeta Anexos.

- **Entrevista semiestructurada**

Esta presenta un grado mayor de flexibilidad que la estructurada, debido a que parten de preguntas planeadas, que pueden ajustarse a los entrevistados. Su ventaja es la posibilidad de adaptarse a los sujetos con enormes posibilidades para motivar al interlocutor, aclarar términos, identificar ambigüedades y reducir formalismos.

- Las preguntas, desarrollo e interpretación se planifican previamente, pero con un cierto grado de libertad de acción para abordar temas que pueden surgir durante la misma.
- Se suele utilizar un protocolo para facilitar al entrevistador seguir un modelo preestablecido.

- **Entrevista no estructurada**

Las entrevistas no estructuradas suelen describirse como conversaciones mantenidas con un propósito en mente.

- No se estructura ni planifica previamente.
- Es la más ágil y la que proporciona más información en general, pero requiere un cierto dominio por parte del entrevistador.

En el material complementario se pueden revisar ejemplos de entrevistas.

Encuesta

Los cuestionarios son herramientas ampliamente utilizadas para recoger datos de sondeos y pueden ser administradas sin la presencia del investigador. (Cohen, 2011, p. 377).

Pueden variar en cuanto a propósito, diseño y apariencia, y consisten en listas de preguntas escritas. Los individuos participantes en la investigación suelen leer los mismos listados de preguntas, por lo que esto permite consistencia y precisión al analizar las respuestas, además de facilitar el proceso. Una de las ventajas más destacadas de los cuestionarios es que simplifican el proceso de la obtención de datos, preguntando directamente a los individuos participantes para obtener datos de forma rápida y directa y se pueden aplicar a un gran número de sujetos.

Los datos que se obtienen a través de los cuestionarios suelen estar clasificados en dos categorías: hechos y opiniones (Denscombe, 2010, p. 156). La información relacionada con los hechos no requiere el juicio o la actitud personal de los sujetos participantes, pero la información obtenida a través de las opiniones implica creencias, puntos de vista y preferencias de los sujetos participantes.

- **Tipos de preguntas**

La distinción más general entre los tipos de preguntas de los cuestionarios, además de hechos y opiniones, es la de preguntas abiertas y cerradas; las preguntas abiertas son aquellas en las que no se especifica ninguna respuesta para elegir y se deja abierta a la elección del participante para que escriba en ella. Las preguntas cerradas son las que ofrecen ya unas respuestas predeterminadas para su elección.

- **Tipos de respuestas**

Las respuestas de escala son las más comunes en los cuestionarios de investigación, ya que implican al participante en una valoración o evaluación de las respuestas objetivo por medio de varias opciones en las que tienen que marcar dentro de una escala la importancia de cada una. Esa escala de valoración indica diferentes grados en una categoría y puede ser de diversa naturaleza; por ejemplo, puede valorar una categoría indicando si es “bueno” o “malo”, “frecuente” o “infrecuente”, “importante” o “poco importante” o también pueden valorar opiniones: “completamente de acuerdo” o “en desacuerdo”. El número de opciones más común es el de cinco, por ser un número impar, ya que existe una tendencia generalizada a seleccionar la opción intermedia (Dornyei, 2010).

Para ampliar la información, lo invitamos a explorar el video Tipos de preguntas en una encuesta, el cual se encuentra en el material complementario.

Observación

Esta permite la obtención de datos para emprender una investigación de tipo cualitativo, no desde el punto de vista de lo que los sujetos dicen, sino que es la evidencia directa de lo que ve y percibe el investigador en un escenario de primera mano (Denscombe, 2010).

Por su parte, Selltiz (citado por Hernández, Fernández y Baptista, 2006, p. 229), al referirse a la observación, recomienda que para que esta se convierta en una técnica como tal, debe cumplir con cuatro condiciones:

- Debe servir a un objeto formulado de investigación.
- Debe de ser planificada sistemáticamente.
- Debe estar controlada y relacionada con proposiciones generales.
- Debe ser sujeta a comprobaciones y controles de validez y fiabilidad.

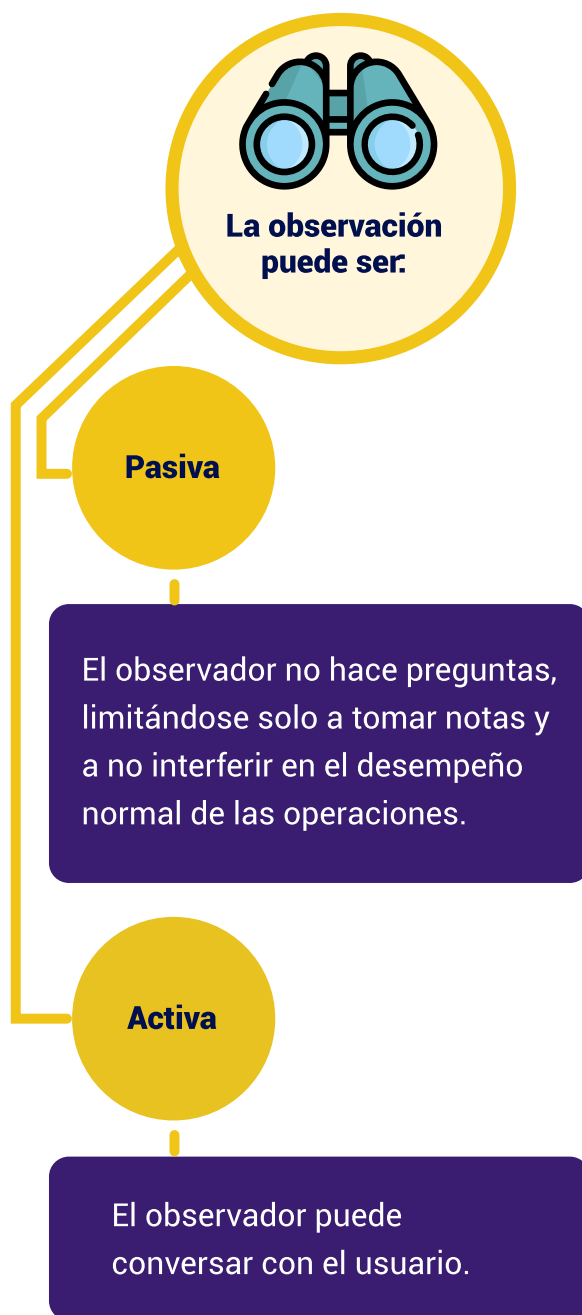
De acuerdo con lo anterior, se puede asumir que la observación:

- Tiene la característica de seguir normas, reglas y procedimientos.

Para el caso de obtención de requerimientos del software la observación nos sirve para estudiar el entorno de trabajo de los usuarios, clientes e interesados de proyecto (stakeholders) y para documentar la situación actual de procesos de negocio.

En la siguiente figura, se pueden revisar los tipos de observación.

Figura 13. Tipos de observación



Ahora bien, para llevar a cabo la observación, el observador puede utilizar como instrumento la guía de observación, la cual le permite situarse de manera sistemática en aquello que realmente es objeto de estudio para la investigación; también es el

medio que conduce la recolección y obtención de datos e información de un hecho o fenómeno.

Tamayo (2004) define a la guía de observación como:

Un formato en el cual se pueden recolectar los datos en sistemática y se pueden registrar en forma uniforme, su utilidad consiste en ofrecer una revisión clara y objetiva de los hechos, agrupa los datos según necesidades específicas, se hace respondiendo a la estructura de las variables o elementos del problema (p. 172).

Para elaborar la guía de observación se ha de diseñar el contenido de la observación; el cual debe incluir por lo menos los siguientes aspectos:

- Datos y características de los sujetos a evaluar.
- Propósitos de la observación o de las observaciones a realizar.
- Temporalidad de la observación.

Lo invitamos a consultar el PDF "**Ficha de observación**" el cual se encuentra en la carpeta Anexos.

Sesiones grupales

Es un proceso por el cual se llevan a cabo reuniones en grupo altamente estructuradas que convocan, en una misma sala, a los usuarios de un sistema, los propietarios del sistema y a los analistas durante un amplio periodo de tiempo. Los objetivos de esta técnica son esencialmente los mismos que los de las entrevistas, con la salvedad de necesitar más analistas para llevarlos a cabo.

Dentro de las sesiones de trabajo en grupo se encuentran técnicas como la lluvia de ideas, las sesiones JAD y el método Delphi.

- **Lluvia de ideas**

También denominada tormenta de ideas o incluso brainstorming.

Faickney (1939) investigó sobre diferentes maneras de generar creatividad.

Se percató de que la mejor manera de ser creativo en una empresa es a través de la interacción y el trabajo en equipo; todos juntos podían dar sus opiniones y sugerencias sobre un tema determinado. Creó de esta manera la lluvia de ideas.

- **Sesiones JAD (Joint Application Design)**

Es un proceso usado para reunir requerimientos en el desarrollo de nuevos sistemas de información para una compañía. El proceso JAD consiste en un taller donde los trabajadores del conocimiento y los especialistas en tecnologías de información se reúnen, algunas veces durante varios días, para definir y revisar los requerimientos de negocio para el sistema. Los asistentes incluyen oficiales de administración de alto nivel, quienes se aseguran de que el producto provea los reportes y la información requerida al final, esto actúa como “un proceso de administración” que permite que los departamentos de servicios de información corporativa trabajen más eficientemente con los usuarios en un marco de tiempo más reducido.

A través de los talleres JAD, los trabajadores del conocimiento y los especialistas en tecnologías de información pueden resolver cualquier dificultad o diferencias entre las posturas referentes al nuevo sistema de información. El taller sigue una detallada agenda para lograr garantizar que todas las incertidumbres entre los grupos sean cubiertas y para ayudar a prevenir cualquier falla en la comunicación, estas fallas de comunicación

pueden provocar repercusiones mucho más serias si no se identifican a tiempo. Al final, este proceso resultará en un nuevo Sistema de Información viable y orientado tanto a diseñadores como a usuarios.

- **Método Delphi**

Es un método de estructuración de un proceso de comunicación grupal que consiste en la selección de un grupo de expertos a los que se les pregunta su opinión frente a ciertas temáticas.

- **Fase uno.** Formulación del problema: se define el campo de investigación.
- **Fase dos.** Elección de expertos: el experto se elige según su preparación y su capacidad de proyección.
- **Fase tres.** Elaboración de cuestionarios: las preguntas deben hacerse de acuerdo con la temática que se quiere obtener.
- **Fase cuatro.** Desarrollo y explotación de resultados: el cuestionario se entrega a los expertos para ser contestado por ellos.

3.3. Herramientas para captura de requisitos

Existen varias herramientas para la captura de requisitos potenciales de un nuevo sistema o una actualización de software, a continuación, se explican las más utilizadas:

Diagrama de casos de uso

Al momento de desarrollar un proyecto se debe pensar en cuáles serán las principales funcionalidades que el software debe permitir llevar a cabo y quiénes serán los que podrán ejecutar dichas funcionalidades. La identificación de estos elementos se puede visualizar de manera efectiva a través de la elaboración de

diagramas de casos de uso; estos diagramas, que son elaborados durante las etapas iniciales de un proyecto, se convierten en referente para cada una de las etapas siguientes del desarrollo del proyecto.

Componentes. En los diagramas de casos de uso, se observan los siguientes componentes.

- **Actor**

Se representa mediante un “hombre de palo”. Este se emplea para indicar el tipo de usuario del sistema que podrá ejecutar alguna función.

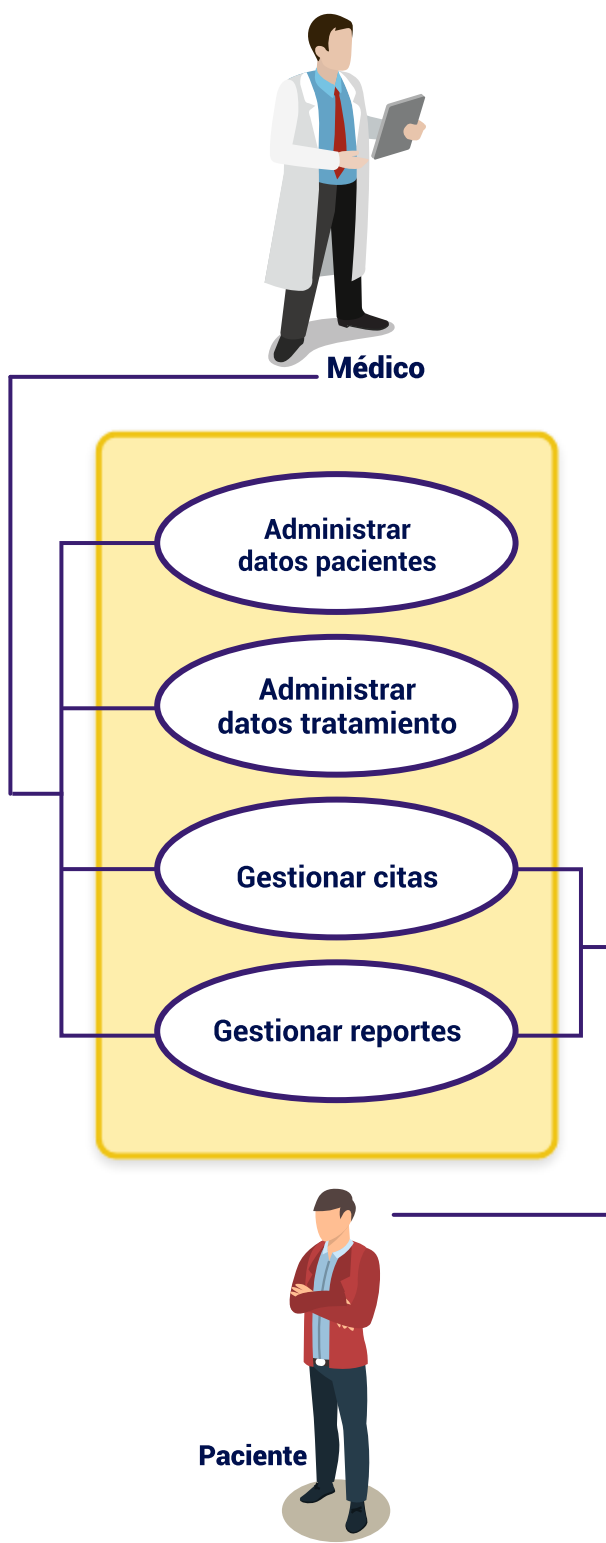
- **Caso de uso**

Se representa mediante un óvalo e indica una función que el sistema debe proveer.

Para ejemplificar un proceso se puede emplear un verbo conjugado en infinitivo y que represente la función a realizar (administrar, gestionar, registrar, entre otros). A continuación, se presenta un ejemplo, en el cual se presenta un diagrama de casos de uso de la sistematización de un centro médico.

- Administrar datos pacientes.
- Administrar datos tratamientos.
- Gestionar citas.
- Generar reportes.

Figura 14. Caso de uso centro médico



- **Identificación de casos de uso:**

En el ejemplo anterior se observan los casos de uso identificados en el sistema, es decir, las funcionalidades que el sistema va a proveer (administrar datos pacientes, administrar datos tratamientos, gestionar citas, generar reportes).

- **Identificación de actores:**

Los actores son los usuarios que podrán ejecutar los casos de uso, en el ejemplo anterior, se identificaron dos actores (médico y empleado).

- **Documentación:**

La técnica de casos de uso requiere además de construir el diagrama de casos de uso, la descripción de estos. Esta descripción permite detallar el flujo de eventos que se da entre el Sistema y el Actor para llevar a cabo el caso de uso.

Documentación caso de uso

Lo invitamos a consultar el PDF “**Caso de uso**”, el cual se encuentra en la carpeta Anexos, y corresponde a un formato diligenciado, de acuerdo con el ejemplo del centro médico.

Historias de usuario

Las historias de usuario son utilizadas en los métodos ágiles para la especificación de requisitos, son una descripción breve de una funcionalidad software tal y como la percibe el usuario (Cohn, 2004).

El formato para las historias de usuario Scrum se basan en una regla de tres palabras:

- **Como** <rol>
- **Quiero** <eventos>
- **Para** <funcionalidades>

Así, el <rol> que se escoja que va a utilizar la aplicación software, requiere de una <Acción> / <evento> que ocurra, porque se desea cubrir una <funcionalidad>.

Corto y conciso, directo y claro.

A continuación, se presentan ejemplos de historias de usuario.

- Como un cliente, quiero consultar para poder encontrar el producto que deseo comprar.
- Como un cliente, quiero consultar para poder encontrar el producto que deseo comprar.

Como un cliente, quiero que los productos seleccionados para la compra queden almacenados en un carrito de compra para poder visualizar todos mis productos y el precio total.

Conversación para explicar mejor la historia de usuario

Como se mencionó anteriormente, las historias de usuario son una frase sencilla y concisa, sin embargo, eso no impide que se pueda abrir un diálogo (conversación) entre todos los miembros del equipo. De hecho, esta conversación se debe llevar a cabo para explicar mejor la propia historia de usuario y conseguir objetivos como:

- Detallar a mayor nivel como se realizará la solución.
- Clarificar aspectos de valor, funcionamiento y técnicos.
- Resolver las dudas que aparezcan.

Estas conversaciones llevarán a alcanzar acuerdos sobre los distintos puntos tratados, que quedarán reflejados en los criterios de aceptación y que permitirán validar cuando una historia de usuario está terminada.

Confirmación de los criterios de aceptación

Los criterios de aceptación, es decir, la confirmación. Se trata de criterios claros y específicos que todo el equipo debe comprender y que permitirán avaluar en el futuro si la implementación que se está desarrollando o las pruebas que se realicen están terminadas.

Storyboard

¿Qué es un Storyboard?

Los storyboards son un tipo de prototipos muy utilizados, consiste básicamente en ir mostrando en una secuencia de imágenes un proceso, acción o ejercicio que se puede realizar en el sistema una vez terminado, las imágenes van mostrando la evolución del sistema conforme el usuario interactúa con el sistema.

Con esta técnica se pretende crear diferentes vistas del sistema en las primeras etapas de su implementación de la manera más rápida y barata posible [SUT02].

Una forma muy común de ejemplificar los storyboards es con las revistas de cómics, ya que van mostrando una secuencia de imágenes en cuadros con un orden establecido que permiten entender la línea de la historia contada. La técnica storyboard permite generar modelos o esquemas visuales como esbozos de interfaces gráficas de usuario (GUI).

A continuación, se detallan las principales características de los storyboards:

- Se preserva el punto de vista del proceso del negocio.
- Se puede validar un escenario.
- Se pueden validar escenarios integradores logrando una visión global.
- Son más fáciles de comprender por el usuario.
- No genera falsas expectativas.
- El usuario sigue trabajando con herramientas conocidas.
- Son fáciles de mantener o adaptar a los cambios.
- Permiten incorporar modificaciones durante la validación.

3.4. Herramientas de modelado

Las herramientas de modelado de sistemas informáticos se emplean para la creación de modelos de sistemas que ya existen o que se desarrollarán; estas herramientas permiten crear un "simulacro" del sistema, a bajo coste y riesgo mínimo. A bajo costo porque, es un conjunto de gráficos y textos que representan el sistema.

El Lenguaje Unificado de Modelado (UML, por sus siglas en inglés, Unified Modeling Language) es el lenguaje de modelado de sistemas de software más conocido y utilizado en la actualidad; UML es un lenguaje gráfico que permite especificar, modelar, construir y documentar los elementos que forman un sistema software.

De otra parte, las herramientas CASE son un conjunto de programas y procesos “guiados”, que ayudan a los analistas, desarrolladores, ingenieros de software y diseñadores en una o todas las etapas que comprende un ciclo de vida, con el objetivo de facilitar el desarrollo de software. El objetivo general de estas herramientas es acelerar el proceso para el que han sido diseñadas, es decir, para automatizar o apoyar

una o más fases del ciclo de vida del desarrollo de sistemas. CASE proporciona un conjunto de herramientas semiautomatizadas y automatizadas que están creando una nueva cultura de ingeniería en muchas empresas. Las herramientas CASE se diseñaron para aumentar la productividad en el desarrollo de software y reducir su costo.

Existen muchas herramientas para modelado tanto en libres como con derechos comerciales; a renglón seguido se listan las herramientas CASE más utilizadas:

- ER win
- ArgoUML
- Easy Case
- Oracle Designer
- Power Designer
- System Architect
- SNAP
- Gliffy
- MagicDraw
- Lucidchart
- Papyrus Uml
- Modelio
- StarUml
- Dia
- Mono Uml

A continuación, se realizará una descripción del top 5 de las más utilizadas.

- **Gliffy**

La aplicación en línea Gliffy es una herramienta de diagramas UML basada en la nube. Apareció por primera vez en 2006 y se trata de una herramienta de modelado que crea todo tipo de diagramas, tales como diagramas de flujo, diagramas de Venn y, por supuesto, diagramas UML. La herramienta en línea fue escrita en HTML5 y es bastante popular gracias a su rapidez de reacción. Es de anotar que antes de que Gliffy pasara por la fase beta en 2007, la empresa homónima cooperó con el grupo de software australiano Atlassian. Ya en 2006, su software de colaboración Confluence integró un plugin de Gliffy y, más tarde, el equipo de Gliffy desarrolló un plugin para Jira. Google Workspace y Drive de Google también integran esta herramienta UML.

- **ArgoUML**

Ha sido durante mucho tiempo una de las herramientas UML gratuitas de código abierto más populares para el escritorio y aunque ya no se mantiene, muchos modeladores continúan usando el programa para tareas más pequeñas. Su software es multiplataforma, cuyo el requisito mínimo es Java 5 ArgoUML soporta todos los tipos de diagramas de la versión 1.4 de UML y perfiles UML. El programa también ofrece algunas formas decorativas que no forman parte del estándar UML.

Además, aunque esta herramienta UML está disponible como descarga gratuita, ArgoUML soporta una amplia gama de lenguajes de programación cuyo código puede generarse a partir de un diagrama. La ingeniería inversa también es posible para Java, C++, PHP, C# y SQL. El programa reconoce otros idiomas como Delphi o Ruby cuando los agrega como extensiones a la carpeta de archivos ArgoULM.

- **MagicDraw**

Esta aplicación de escritorio destaca por su diseño moderno y claro, así como por su variedad de funciones y la facilidad de su uso. Esta herramienta de diagramas UML ofrece además SysML, representación gráfica de procesos de negocio con BPMN (Business Process Model and Notation) y el marco de arquitectura UPDM (United Profile for DoDAF/MODAF).

MagicDraw también ofrece lenguaje de especificación OCL (Object Constraint Language), y XML, que puede usar para exportar diagramas a otros programas sin pérdidas de información.

- **StarUML**

Es una herramienta para el modelamiento de software basado en los estándares UML (Unified Modeling Language) y MDA (Model Driven Architecture).

Da soporte completo al diseño UML mediante el uso de:

- Diagrama de casos de uso.
- Diagrama de clase.
- Diagrama de secuencia.
- Diagrama de colaboración.
- Diagrama de estados.
- Diagrama de actividad.
- Diagrama de componentes.
- Diagrama de despliegue.
- Diagrama de composición estructural (UML 2.0).

- **Lucidchart**

Herramienta para hacer diagramas UML online que permite comprender las complejidades en el código de forma más rápida y sencilla, pues automatiza el proceso de generación de un diagrama de clases.

Simplemente elabora y personaliza los diagramas de secuencia en línea a partir del texto. Al ingresar el marcado en el diálogo emergente, Lucidchart generará automáticamente un diagrama de secuencia que cumple el estándar de PlantUML.

4. Técnicas de análisis y especificación de requisitos

A continuación, abordaremos el análisis de requisitos (priorización, descomposición funcional, matriz de trazabilidad) y estándares, y/o guías existentes para la especificación formal de los mismos dependiendo del tipo de marco de trabajo usado (tradición o ágil).

4.1. Técnicas de análisis de requisitos

El proceso de análisis de requisitos permite, principalmente, el estudio de las necesidades de los usuarios para definir requisitos del sistema por medio de la producción del documento de especificación de requisitos donde se describe lo que el sistema debe hacer, pero no el cómo. Así es como este proceso, además de involucrar un proceso de análisis, también requiere de la síntesis de la información existente.

A continuación, se describen algunas técnicas que pueden ser utilizadas para entender y ordenar los requisitos identificados para su posterior redacción en un artefacto formal dependiendo del marco de desarrollo de software utilizado.

Priorización de requisitos

Esta es una actividad clave para el éxito en la construcción de producto de software y su objetivo es maximizar el valor entregado por el proyecto a sus clientes; es decir, identificar cuáles requisitos se deben priorizar tomando en cuenta factores como la complejidad, las dependencias y el retorno de inversión a los clientes, entre otros indicadores.

La priorización de requisitos, por lo general, es responsabilidad de los gerentes del proyecto o dueños de producto dependiendo del tipo de enfoque utilizado; sin embargo, se requiere de la participación activa de los analistas de requisitos, ya que si

bien es el cliente o su representante quien determina cuáles requisitos son más importantes para su negocio, el analista debe asesorar, facilitar e informarle al cliente para no darle prioridad superior a requisitos que tienen otras dependencias no resueltas; por ejemplo, no se podría priorizar un requerimiento asociado a un carrito de compras sin antes tener resuelto los requerimientos asociados al registro y consulta de productos.

El proceso de priorización es continuo, es decir, la prioridad de un requisito puede variar a lo largo del proyecto, así como los requisitos también pueden variar a lo largo del mismo. La priorización no es un proceso sencillo, por lo que, además de conocer las diferentes técnicas, es importante la toma de decisiones y estas, en el marco de un proyecto, siempre van a generar inconformidades, pero si no se decide respecto a la priorización de requisitos es como si no se fundamentaran las bases sobre las que el equipo de desarrollo va a construir el sistema.

Técnica de clasificación de lista

Esta técnica de clasificación no requiere ningún tipo de entrenamiento o preparación, ya que es una forma de priorización natural que se usa en la vida cotidiana. Este tipo de priorización simple es una de las más utilizadas y consiste en darle un valor numérico a cada requerimiento iniciando por el número 1 y continuando de forma sucesiva con 2, 3 y hasta el número total de requisitos definidos (Porfirio, 2021).

Esta técnica tiene la ventaja de que solo puede existir un número 1, lo cual evita muchos problemas por parte de los interesados o responsables del negocio que quieren que todos los requerimientos tengan prioridad 1; adicionalmente, aporta

claridad y evita confusiones. Cada elemento se prioriza con relación al resto de elementos, lo que simplifica el proceso.

Para usar adecuadamente esta técnica, se requiere de un profundo conocimiento de todos los requerimientos definidos y si bien el proceso parece simple, se requiere de un gran esfuerzo por parte del equipo para situar a cada requerimiento en la posición correcta.

Técnica de puntos de historia y valor del negocio

Esta consiste en realizar el proceso de priorización de acuerdo con factores como el esfuerzo y la opinión de los gerentes de proyecto o dueños de producto, los clientes, el equipo de desarrollo o incluso una combinación de los anteriores. El valor del negocio corresponde a un valor numérico que cuanto más alto sea, más valor representa para el cliente (Porfirio, 2021).

Usar solo el valor del negocio como elemento de priorización puede generar problemas, ya que el valor asociado por el cliente a un requerimiento puede ser muy superfluo y, además, no considera detalles clave como el esfuerzo que se requiere para su desarrollo; por esta razón, también se usa el valor de puntos de historia, que no es más que otro valor numérico asignado por el equipo de desarrolladores a cada requerimiento, en el que se expresa una estimación de esfuerzo. Cuanto más grande sea el número, implica más esfuerzo requerido con miras a realizar el requerimiento.

Para lograr la priorización de los requerimientos, se debe realizar el cálculo del cociente obtenido a partir de los puntos de valor del negocio dividido entre los puntos de historia, quedando entonces una priorización donde estarán en los primeros lugares

los requerimientos más sencillos de resolver por los desarrolladores y que tengan mayor interés por parte del cliente. A continuación, se da un ejemplo.

Tabla 6. Ejemplo de aplicación técnica de puntos de historia y valor del negocio

Requerimientos	Valor del negocio	Puntos de historia	Cociente
R01	6	3	2
R02	7	1	7
R03	9	5	1,8
R04	3	3	1
R05	4	2	2
R06	5	4	1,25
R07	2	8	0,25

Si se trata de un proyecto de siete (7) requerimientos con los valores de negocio, puntos de historia y cocientes como se describe en la tabla anterior, al realizar el proceso de priorización quedaría en el primer lugar el requerimiento R02 ya que tiene el cociente más alto; es decir, representa un requerimiento fácil de construir para el grupo de desarrolladores y, adicionalmente, tiene un alto valor de negocio para el cliente; luego, en el segundo lugar, el requerimiento R01 y R05 y así sucesivamente se registran las prioridades de acuerdo con el valor del cociente de mayor a menor.

Técnica urgente

Aquí se utiliza una tabla de dos dimensiones, donde la horizontal estará determinada por el valor de la urgencia en el requerimiento, el cual corresponde a un valor numérico entre 1 y 5, donde un valor de 5 implica la mayor urgencia y 1 que no

hay tanto apuro en el desarrollo de requerimiento; y la dimensión vertical estará determinada por el valor del negocio, solo que esta vez, a diferencia de la técnica anterior, el valor del negocio también se rige por una escala de 1 a 5, siendo 5 el de mayor valor de negocio posible para un requerimiento (Porfirio, 2021).

Para determinar la prioridad final de un requerimiento, se utiliza una escala de colores que surge a partir de la multiplicación de los valores de las escalas de urgencia y de valor de negocio según la siguiente tabla:

Tabla 7. Referencia para la técnica urgente

Valor del negocio	Urgencia	Urgencia	Urgencia	Urgencia	Urgencia	Urgencia
Valor del negocio	5	5	10	15	20	25
Valor del negocio	4	4	8	12	16	20
Valor del negocio	3	3	6	9	12	15
Valor del negocio	2	2	4	6	8	10
Valor del negocio	1	1	2	3	4	5
Valor del negocio	1	2	3	4	5	

Nota: Tomado de Porfirio (2021).

Luego, se consideran los requerimientos de mayor prioridad que están en el sector de color rojo, después los de color naranja, seguido de los de color amarillo y, por último, los requerimientos del sector de color verde. Para entender mejor este estilo de priorización, observar el siguiente ejemplo:

Tabla 8. Ejemplo de aplicación técnica urgente

Requerimientos	Valor del negocio	Urgencia	Sector
R01	1	4	Verde
R02	2	4	Amarillo
R03	5	5	Rojo
R04	4	3	Amarillo
R05	5	4	Naranja
R06	3	1	Verde
R07	1	3	Verde

Al realizar la multiplicación de los valores de negocio y el valor de la urgencia, se puede establecer en qué sector se encuentra cada requerimiento. Ahora bien, tomando en cuenta los valores del ejemplo de la tabla anterior, se puede concluir que el primer requerimiento a abordar sería el R03 que está en el sector de color rojo, luego el requerimiento R05, que está en el sector de color naranja, y así sucesivamente.

Técnica MoSCoW

Esta técnica se basa en la asignación de etiquetas a cada requerimiento, y las disponibles se relacionan a continuación:

- M: indica una funcionalidad que debe estar (Must).
- S: indica una funcionalidad que debería estar (Should).
- C: indica una funcionalidad que podría estar (Could).
- W: indica una funcionalidad que no estará por ahora, de pronto más adelante (Wont).

Los requerimientos son priorizados utilizando el siguiente orden: primero los que tienen etiqueta M, luego los requerimientos con etiqueta S, después aquellos con C y, finalmente, los etiquetados como W.

Esta técnica requiere de un proceso de consenso sobre el significado de cada una de las etiquetas asignables a cada requerimiento. Los requisitos de tipo M son aquellos obligatorios y que, de no ser abordados, implicaría directamente el fracaso; es importante entonces acordar qué se puede entregar y que sea útil, adicionalmente deben formar parte de un conjunto coherente, ya que si, sencillamente se seleccionan todos los requerimientos de tipo M, automáticamente todos se transforman en requerimientos de tipo M y se pierde la dinámica de la técnica.

Normalmente el proceso de desarrollo de software es iterativo e incremental por lo que dependiendo del momento en que se encuentre un requerimiento, que ahora puede ser W, en la siguiente iteración puede asumir un valor de M.

Juicio de expertos

Es una técnica basada principalmente en función de la complejidad y la exactitud en los resultados, básicamente consiste en realizar el proceso de priorización utilizando como base la opinión del gerente de proyecto o dueño de producto, o de algún stakeholder con conocimientos en la industria asociada al producto a desarrollar (Porfirio, 2021).

Este tipo de técnica funciona muy bien para proyectos pequeños con un dueño de negocio que tiene mucho conocimiento del problema y de la solución que necesita para resolverlo; sin embargo, se debe utilizar esta técnica con cuidado ya que se basa en una visión sesgada del negocio.

Matriz de priorización

Esta técnica consiste en construir una tabla donde cada requerimiento es valorado en una escala de 0 a 10 o de 0 a 100 para cada dimensión alineada con los objetivos del producto. Normalmente cada dimensión tiene un peso porcentual, de modo que cada requerimiento tendrá un valor final a partir de la sumatoria de la multiplicación de cada puntuación por el peso de cada dimensión (Porfirio, 2021).

Por ejemplo, se debe considerar la siguiente tabla:

Tabla 9. Ejemplo de aplicación técnica de matriz de priorización

Requerimiento	Criterio: Conversión 30%	Criterio: Satisfacción de usuario 40%	Criterio: Retención 30%	Resultado: 100%	Ranking
R01	4	3	8	4,8	2
R02	6	4	3	4,3	7
R03	8	7	6	7	1,8
R04	4	2	4	3,2	1

Suponiendo las dimensiones de conversión con un peso porcentual del 30%, la satisfacción del cliente con un peso porcentual del 40% y la retención de clientes con peso porcentual del 30%, cada uno de los requerimientos (R01-R04) son evaluados en una escala de 0 a 10, a los cuales se les aplica el cálculo de acuerdo con el valor porcentual de cada dimensión evaluada y el resultado se obtendría de la sumatoria de los valores parciales de cada dimensión, con lo cual los mayores valores totales obtenidos serán priorizados sobre los de menor valor.

Esta técnica sigue siendo muy subjetiva al igual que la técnica de juicio de expertos, aunque quedan claros cuáles son los criterios de evaluación utilizados para determinar la prioridad de los requisitos.

Matriz de trazabilidad

La matriz de trazabilidad es una herramienta que permite alinear los requisitos del proyecto con los logros de los objetivos, es una tabla que relaciona cada uno de los requerimientos con el entregable solicitado. Es decir, permite identificar qué resultado se alcanza con cada requisito y, a la vez, permite visualizar qué requisitos son necesario cumplir para determinado entregable (Pantaleo, 2018).

Este es un instrumento clave para el responsable del proyecto, en especial en el proceso de seguimiento y control de cambios, ya que permite analizar qué requerimientos, eventualmente, convendría modificar, eliminar o añadir; así, se puede ajustar la planificación de tareas pendientes del proyecto. También permite identificar inconsistencias entre los requerimientos y los beneficios que se esperan alcanzar, tener una visualización rápida de los requisitos que han sido abordados y cuáles están pendientes por realizar y, por lo tanto, qué entregables están próximos a producirse y cuáles todavía requieren de mayor tiempo.

La estructura de la matriz de trazabilidad se puede construir en una hoja de cálculo, en ella se relacionan todos los requisitos y las metas a alcanzar junto con una serie de valores complementarios que aportan información y coherencia a los vínculos generados.

Cada organización es responsable de adaptar la matriz de correlación a sus necesidades particulares; en el siguiente recurso se propone el contenido base de una matriz de correlación dividida por secciones.

- **Identificación**

Identificador: código único para cada requisito.

Código jerárquico: código que permite categorizar cada grupo de requisitos.

Descripción: texto explicativo del requerimiento.

Tipo: categoría del requisito (requerimiento de negocio, requerimiento de los interesados, funcionales y no funcionales, requerimientos del proyecto, requerimiento de calidad).

- **Estado**

Versión: versión del requerimiento (importante para el control de cambios).

Estado: activo, cancelado, diferido, agregado, aprobado, asignado o completado.

Fecha de estado: momento en el que se realizó la última modificación del estado.

Responsable: persona encargada del requisito.

Prioridad: escalas de prioridad usada en el proyecto (alta, media, baja; 1, 2, 3, 4, 5).

Otras características: cualquier otro campo de valor para la organización.

- **Objetivo**

Objetivo: objetivo que pretende alcanzar el requerimiento.

Necesidad de negocio: necesidad de negocio que se pretende cubrir con el objetivo del proyecto y por ende con el requisito.

Entregable: producto.

Ejemplo de la estructura de una matriz de trazabilidad

Al construir una matriz de trazabilidad se deben usar los campos que se consideren útiles para el proyecto, pues no todos los proyectos son iguales y la estructura definida para uno puede no resultar conveniente para otro proyecto. Cuando se usa esta matriz debe permanecer actualizada a lo largo del ciclo de vida de construcción del proyecto.

Lo invitamos a consultar el documento "**Ejemplo estructura matriz trazabilidad**", el cual se encuentra en la carpeta Anexos, como tabla base para una estructuración de trazabilidad.

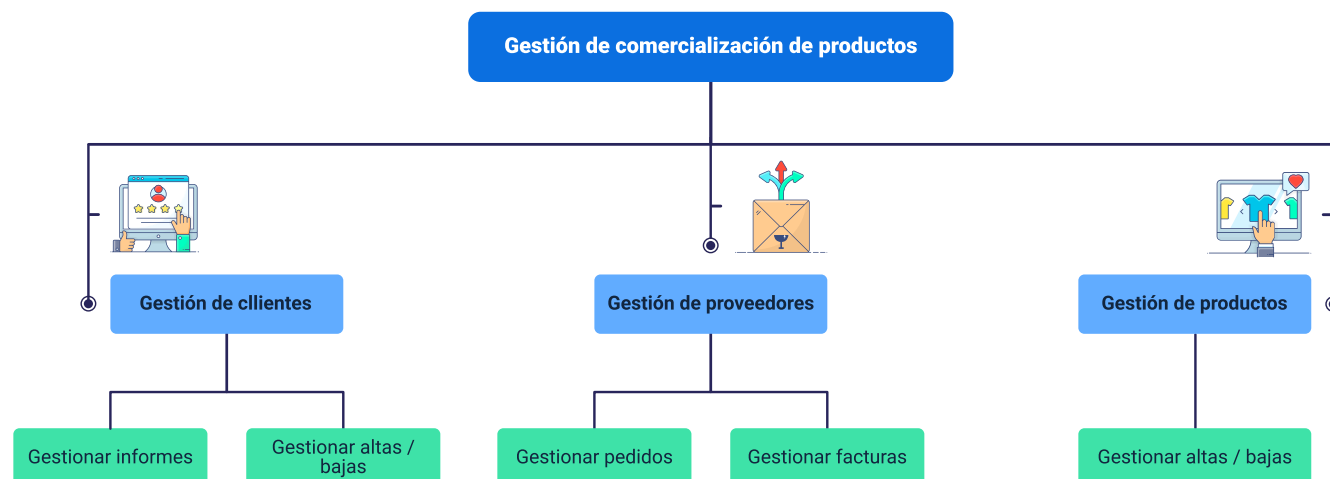
Descomposición funcional

Esta estrategia consiste en definir los requerimientos como una relación de entradas y salidas de un sistema. Normalmente, estos requerimientos se definen de manera muy general y poco detallada y luego empieza a descomponerse en funcionalidades y subfuncionalidades un poco más detallados con el ánimo de analizarlas individualmente hasta lograr un nivel de detalle adecuado para el proyecto. A esta estructura se le conoce como top-down; el resultado es una estructura jerárquica (Pantaleo, 2018).

La descomposición funcional se realiza, por lo general, para identificar y entender los componentes o partes que constituyen un todo; en este proceso es vital identificar

las interacciones entre componentes. A continuación, se expone un ejemplo gráfico sobre esta descomposición funcional.

Figura 15. Ejemplo de descomposición funcional de un sistema de gestión de comercialización de productos (fragmento)



4.2. Especificación de requisitos

En esta sección se describen algunos estándares y/o técnicas que pueden ser usadas por las organizaciones para describir formalmente cada uno de los requisitos del sistema.

Estándar IEEE 830

Este estándar presenta un conjunto de prácticas recomendadas para la redacción de un documento de especificación de requerimientos, mejor conocido como SRS. Este documento está dividido en secciones y cada una de ellas aborda aspectos particulares. A continuación, se describirán de forma general algunos de los elementos que conforman este documento (IEEE 830-1998).

Ejemplo de la estructura de una matriz de trazabilidad

Lo invitamos a consultar el PDF " **Infografía EstandarIEEE830**", el cual se encuentra en la carpeta Anexos.

Este documento está dividido en secciones y cada una de ellas aborda aspectos particulares. A continuación, se describirá de forma general algunos de los elementos que conforman este documento (IEEE 830-1998).

Sumado a esto, se presenta la estructura base de un documento SRS, indicando cuáles son los apartados principales.

Estructura base de un documento SRS

- **Introducción**
 - Propósito
 - Ámbito del sistema
 - Definiciones, acrónimos y abreviaturas
 - Referencias
- **Descripción general**
 - Perspectiva del producto
 - Funciones del producto
 - Características de los usuarios
 - Restricciones
 - Suposiciones y dependencias
- **Requerimientos específicos**
 - Interfaz
 - Requisitos funcionales

- Requerimientos no funcionales
- Otros requisitos
- **Apéndices**

Se sugiere revisar algunos ejemplos de proyectos que se presentan sobre el diligenciamiento del formato SRS, los cuales se encuentran en el material complementario.

Estándar IEEE 29148:2018

Este estándar reemplaza los estándares IEEE 830, IEEE 1233, IEEE 1362, y contiene disposiciones para los procesos y productos relacionados con la ingeniería de requisitos para sistemas, productos y servicios de software a lo largo del ciclo de vida (Penzenstadler, 2021).

Además, define la construcción de un buen requisito, proporciona atributos y características de los requisitos, y analiza la aplicación iterativa y recursiva del proceso de requisitos a lo largo del ciclo de vida. También proporciona orientación adicional en la aplicación de procesos de ingeniería y gestión de requerimientos relacionados con la ingeniería de requisitos, al tiempo que define los elementos de información aplicables a la ingeniería de requisitos y su contenido.

El estándar IEEE 29148:2018 está estructurado de la siguiente forma:

- Introducción, resumen y tabla de contenido.
- Propósito, alcance del estándar, generalidades.
- Explicación de los otros estándares que lo conforman.
- Referencias a normas que lo conforman.

- Clarificación de la terminología, lo que es muy valioso para cuando se quiere establecer nuevos procesos de ingeniería de requerimientos en una empresa.
- Clarificación de los conceptos y procesos.
- Explicación y contenido de los ítems de información que vienen a través de la especificación de requerimientos o que debemos considerar incluir en la especificación de requerimientos.
- Anexos adicionales para mayor detalle.

Esta norma propone un listado de requerimientos mínimos, los cuales son la base de la especificación de requerimientos; en ese sentido, se proponen los siguientes tipos de requerimientos del sistema:

- Requerimientos funcionales: representan las necesidades de los interesados del software.
- Requerimientos de usabilidad: requerimientos que son utilizados directamente por los involucrados en la solución (requerimientos de uso).
- Requerimientos de desempeño: disponibilidad de servicios y procesos transaccionales.
- Interfaces del sistema: interacción entre personas con el software.
- Operaciones del sistema.
- Modos y estados del sistema.
- Características físicas (hardware).
- Condiciones del ambiente (operativas y operacionales).
- Seguridad del sistema.
- Manejo de la información.

- Políticas y regulación: normas y estándares que fundamentan el software.
- Ciclo de vida del sistema: establece las etapas y duración del desarrollo y uso en producción.

La especificación de requisitos a través de marcos de trabajo ágiles

Los marcos de trabajo ágiles promueven la comunicación oral sobre la documentación exhaustiva en la mayoría de los procesos del ciclo de vida, particularmente en los procesos de identificación de necesidades y diseño. Sin embargo, uno de los artefactos presentes para el modelado de requerimientos son las historias de usuario.

Las historias de usuario son una explicación general e informal de una función del software escrita desde la perspectiva del usuario final o cliente. Permiten describir de una manera muy breve un requerimiento, estimar prioridades, alcance y tiempo de realización (Rivadeneira, 2014). En la siguiente tabla, se puede observar la estructura base de un documento de historia de usuario.

La estructura base de un documento de historia de usuario es:

- Número
- Nombre de la historia de usuario
- Usuario
- Prioridad
- Puntos estimados
- Descripción
- Observaciones
- Criterios de aceptación

Las historias de usuario tienen varios beneficios respecto a otros instrumentos de redacción de requerimientos, entre los cuales se pueden listar:

- Las historias de usuario se centran en solucionar problemas a usuarios reales.
- Las historias de usuario permiten la colaboración, ya que como su descripción es corta, se necesita que el equipo colabore para decidir cómo dar solución a la historia para cumplir con la necesidad expresada por el usuario.
- Las historias impulsan la creatividad, ya que fomentan que el equipo piense de forma crítica y creativa sobre cómo solucionar de la mejor manera el objetivo.
- Las historias de usuario motivan a pensar en la mejor solución para una problemática particular, representan retos y pequeñas victorias para el equipo.

Scrum y la especificación de requisitos

El marco de trabajo Scrum está soportado en un proceso de construcción iterativo e incremental evolutivo, en el que se identifican tres roles principales: el equipo de trabajo (team) conformado por los desarrolladores, diseñadores, personal de calidad y de infraestructura requerido para la construcción del producto de software; el scrum master que realizan funciones parecidas a las de un director de proyecto, pero más enfocados en garantizar que el equipo de trabajo tenga todas las herramientas y recursos necesarios para el desarrollo de su trabajo; y, finalmente, el dueño del producto (product owner) que se convierte en un representante del

cliente y quien es el único encargado de la gestión de requisitos del proyecto (ScrumStudy, 2021). En el siguiente video se explica un poco más sobre el Scrum y la especificación de los requisitos.

Video 2. Scrum y la especificación de requisitos



[Enlace de reproducción del video](#)

Síntesis del video: Scrum y la especificación de requisitos

El marco de trabajo scrum está soportado en un proceso de construcción iterativo e incremental evolutivo, en el que se identifican tres roles principales: el equipo de trabajo o team conformado por los desarrolladores, diseñadores, personal de calidad y de infraestructura, requerido para la construcción del producto de software. El Scrum master que realiza funciones parecidas a las de un director de

proyecto, pero más enfocados que el equipo de trabajo tenga todas las herramientas y recursos necesarios para el desarrollo de su trabajo. Y finalmente el dueño de producto o Product owner que se convierte en un representante del cliente, quien es el único encargado de la gestión de requisitos del proyecto.

Scrum establece el concepto de sprint para referirse a una iteración que contempla tiempos fijos entre 2 y 4 semanas dependiendo del equipo de trabajo. Durante este tiempo, se incluye la planeación del sprint, donde se definen los requerimientos a desarrollar en ese periodo de tiempo, una fase de construcción del producto y, finalmente, un proceso de despliegue para poder hacer la respectiva demostración de lo construido al final de iteración en reuniones de revisión. En este marco de trabajo, se redefine el concepto de requerimiento hecho y normalmente va mucho más allá de construir el código, por lo general, se incluyen procesos de validación con pruebas unitarias y pruebas de integración.

El artefacto mediante el cual se condensan todos los requerimientos del sistema se denomina pila de producto (product backlog), la cual es una lista ordenada por prioridad de todos los requerimientos del sistema, generalmente descritos en la forma de historias de usuario.

Todas las historias de usuario se priorizan utilizando números enteros consecutivos de 1 hasta N, donde 1 representa la máxima prioridad. Esta priorización la realiza únicamente el dueño del producto y, para eso, utiliza la información que tiene sobre el negocio y recomendaciones de expertos buscando el mayor retorno de inversión a sus clientes. La priorización de las historias de usuario en la pila de producto

puede variar en el transcurso del tiempo, pero solo la podrá realizar el dueño del producto.

Scrum, siendo un marco de trabajo ágil, no requiere para su funcionamiento que todas las historias de usuario de la pila de producto estén detalladas, pero sí, por lo menos, las de mayor prioridad para poder iniciar el trabajo con el equipo en sus respectivos sprints (ScrumStudy, 2021).

Al inicio de cada sprint se realiza el proceso de planeación que involucra principalmente tres tareas:

- Estimar el valor de esfuerzo requerido para un conjunto de historias de usuario de la pila de producto, trabajo que es realizado únicamente por el equipo de desarrollo.
- Selección de las historias de usuario a desarrollar durante el sprint, tomando como referencia la prioridad y el valor del esfuerzo asociado a cada historia. Esto genera un artefacto llamado pila del sprint (sprint backlog) que lista los requerimientos descritos como historias de usuario a ser realizadas y evaluadas en el sprint.
- Descomposición de cada historia de usuario en tareas y, de ser necesario, asignar responsables a cada tarea.

A continuación, se expone una figura en la que se representan los artefactos generados dentro del marco de trabajo Scrum y que permiten la gestión de los requisitos y el evento desde el cual se construye inicialmente.

Figura 16. Pila de producto vs. pila del Sprint



Kanban y la especificación de requisitos

Kanban es una metodología para gestionar el trabajo que surge del sistema de producción Toyota Production System (TPS) a finales de la década de 1940, el cual representaba un sistema de producción basado en la demanda de los clientes y no en la producción masiva, lo anterior sentó los fundamentos para los sistemas de producción ajustada que consisten en minimizar los desperdicios sin afectar la producción y en crear más valor a los clientes sin generar más gastos.

A principios del siglo XXI, la industria del software adoptó el Kanban para cambiar la forma en la que se producían y entregaban productos y servicios; además, tiene en cuenta los principios de las metodologías ágiles, en especial de Scrum, pero

busca darle más protagonismo al proceso de experimentación y mejora continua (Rivadeneira, 2014).

Kanban en la industria del software se basa en cuatro principios fundamentales:

- Calidad garantizada, todo lo que se produce debe salir bien sin márgenes de errores, prima la calidad sobre la rapidez.
- Reducción del desperdicio: hacer solamente lo justo y necesario, pero hacerlo bien.
- Mejora continua.
- Flexibilidad: se pueden priorizar tareas entrantes según las necesidades del momento.

Además de los principios, Kanban propone seis (6) prácticas:

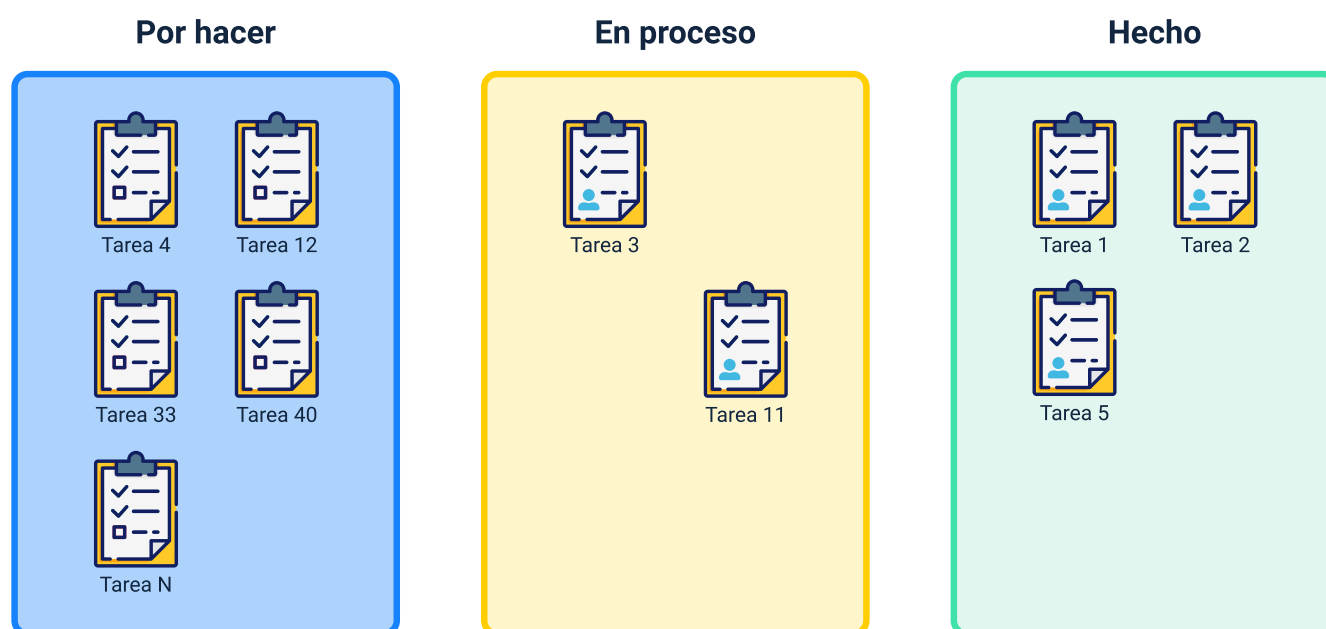
- Visualizar el flujo de trabajo.
- Eliminar interrupciones.
- Gestionar el flujo.
- Hacer políticas explícitas que fomenten la visibilidad.
- Circuitos de realimentación.
- Mejorar colaborando.

Tal vez la herramienta más conocida que permite implementar los principios de Kanban es el tablero Kanban, el cual permite mapear y visualizar el flujo de trabajo. Este se divide en columnas a través de las cuales se pueden visualizar cada una de las fases del proceso; las filas del tablero representan los diferentes tipos de actividades específicas que se desarrollan en el marco del proyecto.

Normalmente, el tablero tiene tres secciones que representan el estado de cada una de las tareas: por hacer, en proceso, hecho.

Cada equipo de trabajo puede realizar un mapeo más detallado de su proceso y agregar tanta sección como considere pertinente, como se muestra en la siguiente figura.

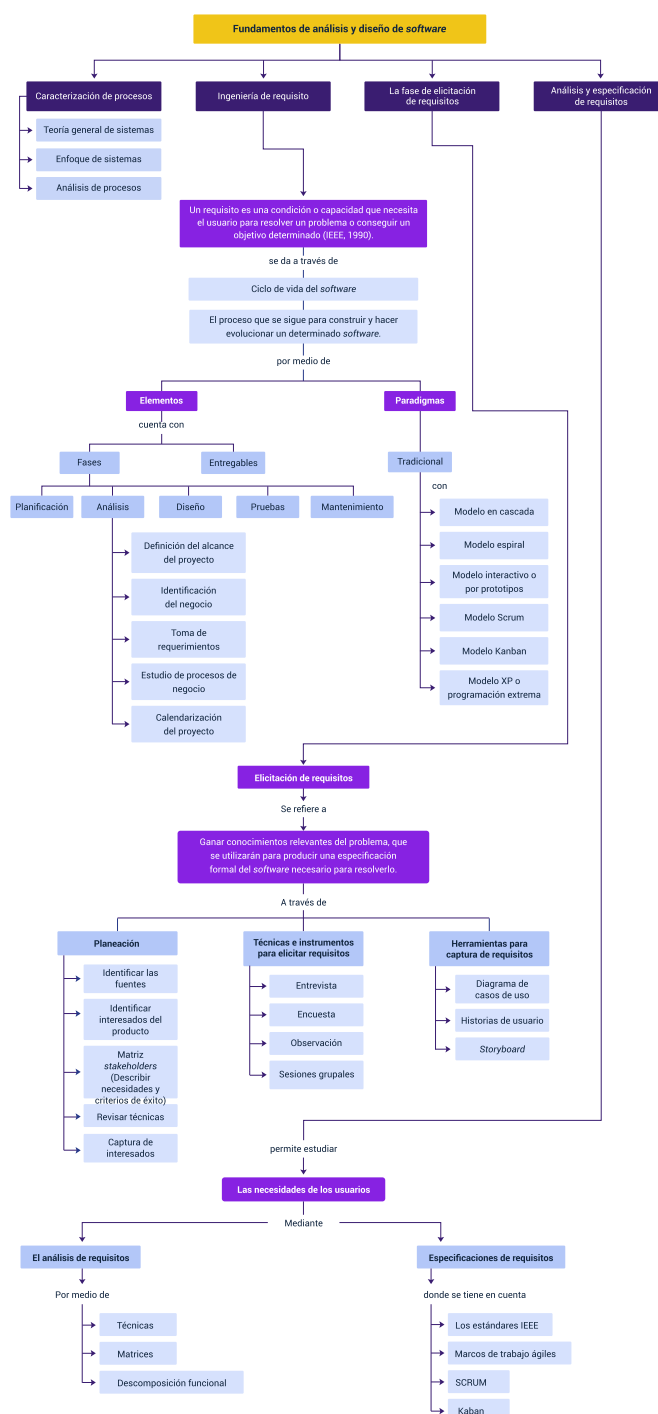
Figura 17. Tablero Kanban



Dependiendo del marco de trabajo, varía la forma en la que se describen cada una de las tareas del tablero Kanban, por ejemplo, dentro de un marco de trabajo como Scrum cada una de las tareas se podría describir en el formato de historias de usuario.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Análisis de los procesos a nivel de negocio	Saber programas. (2021). Cómo crear un DIAGRAMA de FLUJO en WORD paso a paso 2021. [Video]. YouTube.	Video	https://www.youtube.com/watch?v=nJq8A85zNZU
Análisis de los procesos a nivel de negocio	Lucidchart Software. (2021). Lucichart.com.	Página web	https://www.lucidchart.com/pages/es
Análisis de los procesos a nivel de negocio	Dignos. (2013). Elaboración de procesos. Sugerencia de una metodología. [Video]. YouTube.	Video	https://www.youtube.com/watch?v=-U-RQKJ9KKg&abchannel=Dignos
Ingeniería del software - Ciclo de vida	Universidad Católica de Murcia. (2015). Ingeniería del software - Ciclo de vida - Raquel Martínez. YouTube.	Video	https://www.youtube.com/watch?v=4tWmULUzVdE&t=199s
Tipos de requerimientos	Itunes U – UAEH. (2019). Tipos de requerimientos. YouTube.	Video	https://www.youtube.com/watch?v=Lv7XbZtnQ6A
Planeación	todopmp.com. (s.f.). Guía PMBOK 6 – 49 procesos, entradas, herramientas y salidas.	Página web	https://todopmp.com/cards/
Planeación	EDAP – Project Business School. (2016). MOOC PMP 302 Identificar Interesados	Video	https://www.youtube.com/watch?v=aUkTxgaajBo
Planeación	Virtual Training Lteam. (2016). Partes Interesadas Stakeholders	Video	https://www.youtube.com/watch?v=9AtalAZEu0c

Tema	Referencia	Tipo de material	Enlace del recurso
Planeación	Calle, M. (2020). Análisis de Interesados Matriz Poder Interés - PMI	Video	https://www.youtube.com/watch?v=hDZ0uu0H1wc
Técnicas e instrumentos para elicitar requisitos	Jibaro X. (2019). Tipos de Preguntas en una encuesta	Video	https://www.youtube.com/watch?v=mwnQuUi9014
Especificación de requisitos	UMNG. (2019). Elementos de la norma IEEE 830. YouTube.	Video	http://www.foss2serve.org/index.php/RequirementsEngineering,CSULongBeach, Penzenstadler
Especificación de requisitos	UMNG. (2019). Elementos de la norma IEEE 830. YouTube.	Video	https://ieeexplore.ieee.org/document/720574
Estándar IEEE 830	Scrum Certification, Agile Certification Scrum, Agile Training. (n.d.).ScrumStudy.	Video	https://www.scrumstudy.com/

Glosario

Ágil: comprende un conjunto de tareas o acciones que se utilizan para producir y mantener productos, así como para lograr los objetivos del proceso. La actividad incluye los procedimientos, estándares, políticas y objetivos para crear y modificar un conjunto de productos de trabajo.

ANSI: American National Standards Institute.

Ciclo de vida de software: aplicación de metodologías para el desarrollo del sistema software [AECC, 1986].

Estándar: referencia, patrón o modelo que es utilizado a nivel general en un determinado ámbito.

Marcos de trabajo ágiles: conjunto de estándares, metodologías, técnicas, frameworks o guías que rigen un proceso de desarrollo de software basadas en principios y/o valores ágiles como, por ejemplo: Scrum, Lean Software, XP, TDD, entre otros.

Marcos de trabajo tradicionales: conjunto de estándares, metodologías, técnicas, frameworks o guías que rigen un proceso de desarrollo de software basadas en el ciclo de vida tradicional del software como, por ejemplo: RUP, CMMI, ISO 9001, Microsoft Solution Framework, entre otros.

Método: indica cómo construir técnicamente el software. Se incluyen técnicas de modelado y otras técnicas descriptivas.

Metodología: síntesis de un conjunto de técnicas, métodos y procedimientos que se deben seguir durante el desarrollo de un proyecto.

Pert: Program Evaluation Research Task.

Pruebas de integración: prueba que se ejecuta una vez se aprueban las pruebas unitarias y lo que busca es verificar que el conjunto de fragmentos de código funciona junto de forma correcta. Es una prueba de conjunto.

Pruebas unitarias: forma de comprobar el correcto funcionamiento de una unidad de código.

Requerimiento: el requerimiento se refiere a la petición que se hace de algo que se solicita.

Requisito: es la condición que debe cumplir algo, en general el requisito cumple con lo que se requiere con el requerimiento.

Retroalimentación: cuando parte de un mensaje de salida se convierte nuevamente en entrada.

Simulación: corresponde a la posibilidad de conducir experimentos en una computadora.

Sinergia: cuando varios elementos de una organización actúan de manera concentrada.

Stakeholders: individuo u organización que comparte, reclama o le interesa un sistema o le compete una característica que satisface sus necesidades y expectativas (ISO 29148).

Técnica: manera en la que un conjunto de procedimientos es aplicado en una tarea específica, con base en un conocimiento para obtener un resultado específico.

Teoría de colas: teoría que estudia los tiempos de espera dentro de un sistema.

Teoría de comportamientos: orientada al estudio del comportamiento humano en la administración.

Teoría de juegos: corresponde a la aplicación de un modelo matemático juego para entender la toma de decisiones.

TGS: Teoría General de sistemas.

Referencias bibliográficas

830-1998 - IEEE Recommended Practice for Software Requirements Specifications. (1998). IEEE Standard | IEEE Xplore.

<https://ieeexplore.ieee.org/document/720574>

Baar, B. (2006). Using Stakeholder Analysis in Software Project Management. Universidad de Amsterdam.

Bertalanffy, L. (1968). Theory General Systems.

Boehm, B. W. (1979). A Spiral Model of Software Development and Enhancement. ACM Software Engineering Notes, 11(4), 22-42.

Braude, J. (2003). Ingeniería de software, una perspectiva orientada a objetos. Alfaomega.

Centro Tic de Andalucía. (2019). El enfoque sistémico.

Cohen, L. (2011). Métodos de investigación educativa. La Muralla.

Cohn, M. (2004). User Stories Applied for Agile Software Development. Pearson Education, Inc.

Cox, K., Niazi, M., y Verner, J. (2009). Empirical study of Sommerville and Sawyer's requirements engineering practices. IET Software, 3(5), 339.

<https://doi.org/10.1049/iet-sen.2008.0076>

Curso de interacción persona-ordenador. (2021). Storyboarding.

<https://mpiua.invid.udl.cat/storyboarding>

Denscombe, M. (2010). The Good Research Guide. McGraw-Hill Education.

Dornyei, Z. (2010). Questionnaires in Second Language Research: Construction, Administration, and Processing. Routledge.

Durán, A., Bernárdez, B., Ruiz, A. y Toro, M. (1999). A Requirements Elicitation Approach Based in Templates and Patterns.

ESAN Graduate School of Business. (2019). ¿Qué es el análisis de procesos de negocio y cómo aplicarlo en mi empresa?). <https://www.esan.edu.pe/apuntes-empresariales/2019/11/que-es-el-analisis-de-procesos-de-negocio-y-como-aplicarlo-en-mi-empresa/>

Espinal, I., Gimeno, A. y González, F. (1998). El enfoque sistémico en los estudios sobre la familia. <https://www.uv.es/jugar2/Enfoque%20Sistemico.pdf>

Gause, C., & Weinberg, G. M. (1989). Exploring Requirements: Quality Before Design. Dorset House.

Granollers, T., Lorés, J., y Perdrix F. (2002). Prototipado. Capítulo 5: modelo de proceso de la ingeniería de la usabilidad y de la accesibilidad.

Hernández, S., Fernández C., y Baptista L. (2006). Metodología de la investigación. McGraw Hill.

Herrera J., Lizka J. (2003). Ingeniería de requerimientos, ingeniería de software. <http://www.monografias.com/trabajos6/resof/resof.shtml>

McCracken, D., y Jackson, M. A. (1981). "A Minority Dissenting Opinion". En W. W. Cotterman, J. D. Couger, N. L. Enger, F. Harold (Eds.). Systems Analysis and Design: A Foundation for the 1980s (pp. 551-553). Elsevier.

Pantaleo, G., y Rinaudo, L. (2018). Ingeniería de software. Alfaomega.

<https://www.iso.org/obp/ui/#iso:std:iso-iec:12207:ed-2:v1:en>

Pantaleo, L., y Rinaudo. (2018). Ingeniería de software. Alfaomega.

Penzenstadler, B. (s. f.). Requirements Engineering. CSU Long Beach.

<http://www.foss2serve.org/index.php/RequirementsEngineering,CSULongBeach,Penzenstadler>

Pfleeger, Sh. (2002). Ingeniería del software. Teoría y práctica. Prentice Hall.

Piattini M., Calvo-Manzano J., Cervera J., y Fernández, L. (2004). Análisis y diseño de aplicaciones informáticas de gestión. Una perspectiva de ingeniería de software. Alfaomega-Rama.

Porfirio, C. (2021). Técnicas de priorización: el desafío de conseguir un orden para las funcionalidades. <https://www.knowmadmood.com/es/blog/tcnicas-de-priorizacin-el-desafio-de-conseguir-un-orden-para-las-funcionalidades>

Rivadeneira, M., S. (2014). Metodologías ágiles enfocadas al modelado de requerimientos. Informes Científicos Técnicos - UNPA, 5(1), 1-29. <https://doi.org/10.22305/ict-unpa.v5i1.66>

Sommerville I. (2011). Ingeniería del software. Addison-Wesley.

Tamayo. A (1999). Teoría general de sistemas.

Torres. A (2021). La Teoría General de Sistemas, de Ludwig von Bertalanffy. <https://psicologiaymente.com/psicologia/teoria-general-de-sistemas-ludwig-von-bertalanffy>

Ventura, M. T. (2002). La ingeniería de requerimientos como factor clave para el éxito de los proyectos de desarrollo de software.

Wessinger, K., (2012) Project Stakeholder Management. Engineering Management Journal, 14(84), 19-24.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Carlos Hernán Muñoz Carvajal	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Henry Eduardo Bastidas Paruma	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Peter Emerson Pinchao Solis	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Yerson Fabián Zárate Saavedra	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edward Leonardo Pico Cabra	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia

Nombre	Cargo	Centro de Formación y Regional
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Diseño de patrones de “software”

Breve descripción:

Con el estudio de este componente, el aprendiz estará en capacidad de conceptualizar, interpretar y aplicar diseños de patrones de “software”. Así mismo, se afianzará en lo referente a patrones comportamentales, creacionales, estructurales, vistas estáticas, diagramas de despliegue y de componentes, todo ello en el marco general de análisis y desarrollo de “software”.

Abril 2024

Tabla de contenido

Introducción	4
1. Conceptos sobre patrones de diseño.....	5
2. Patrones comportamentales	7
Estrategia.....	7
Comando	8
Iterador	10
3. Patrones creacionales.....	12
4. Patrones estructurales.....	15
4.1. Fachada	15
4.2. “Delegate”	21
5. Vistas estáticas	25
6. Diagrama de despliegue	26
7. Diagrama de componentes.....	29
Síntesis	31
Material complementario.....	32
Glosario.....	33
Referencias bibliográficas	34
Créditos	35



Introducción

Los patrones de diseño surgen, inicialmente, en el área de la arquitectura y la ingeniería civil, refiriéndose a casos particulares a los que se enfrentaron en algún momento y de los cuales se documentó una solución particular. Posteriormente, este concepto se empezó a asociar al diseño orientado a objetos. En lugar de referirse a la forma de colocar paredes, puertas y ventanas, se hizo referencia a la forma en que se construyen clases, objetos e interfaces, y la forma en cómo estos deben interactuar.

Los patrones de diseño del “software” representan un conjunto de formas estandarizadas para resolver un problema particular, los cuales surgen a partir de la experiencia desarrollada en la industria.

Un patrón de diseño debe estar estructurado por cuatro componentes:

- El nombre.
- Problema o contexto de aplicación.
- Solución propuesta.
- Ventajas y desventajas.

En este componente formativo, se abordarán algunos de los patrones de diseño más representativos en la industria del “software”, los cuales le permitirán implementar, a futuro, soluciones más robustas acordes con los altos estándares de calidad ampliamente conocidos en la industria. Este estudio profundizará en cómo estos patrones facilitan la reutilización de soluciones probadas y eficientes, mejorando así la mantenibilidad y la escalabilidad del “software”.

1. Conceptos sobre patrones de diseño

El uso de patrones de diseño en el ámbito de la industria de “software” implica el conocimiento y la aplicación de una serie de criterios que aportan calidad al proceso mismo y efectividad del producto logrado. En ese mismo sentido, se favorece la estandarización del código fuente de determinada aplicación, lo cual facilita su monitoreo y mantenimiento.

Los patrones de diseño tienen las siguientes características:

- Representan soluciones concretas que, aunque se representan de forma genérica, son aplicados para resolver problemas reales.
- Son especificaciones técnicas basadas en los principios de la programación orientada a objetos y dependiendo del lenguaje de programación la forma de implementarlos puede variar.
- Son frecuentemente utilizados ya que se construyen a partir de la experiencia acumulada en la industria del desarrollo de “software”.
- Favorecen la implementación de las características de la programación orientada a objetos como la encapsulación, las jerarquías y el polimorfismo.
- No necesariamente el uso de un patrón implica el uso de palabras claves, reservadas o de librerías especializadas.
- Los patrones generalmente hacen referencia al uso de interfaces, clases y objetos que deben ser ajustados de acuerdo con la solución concreta a desarrollar.

Para comprender mejor la organización y aplicación de los patrones de diseño en el desarrollo de “software”, es importante reconocer su clasificación. Estos suelen dividirse en tres grandes grupos, según su finalidad:

- Patrones comportamentales
- Patrones creacionales
- Patrones estructurales

2. Patrones comportamentales

Los patrones de diseño comportamentales se centran en definir la manera en que los objetos interactúan entre ellos, por medio de mensajes. Son tres los tipos de patrones comportamentales: estrategia, comando e iterador.

Estrategia

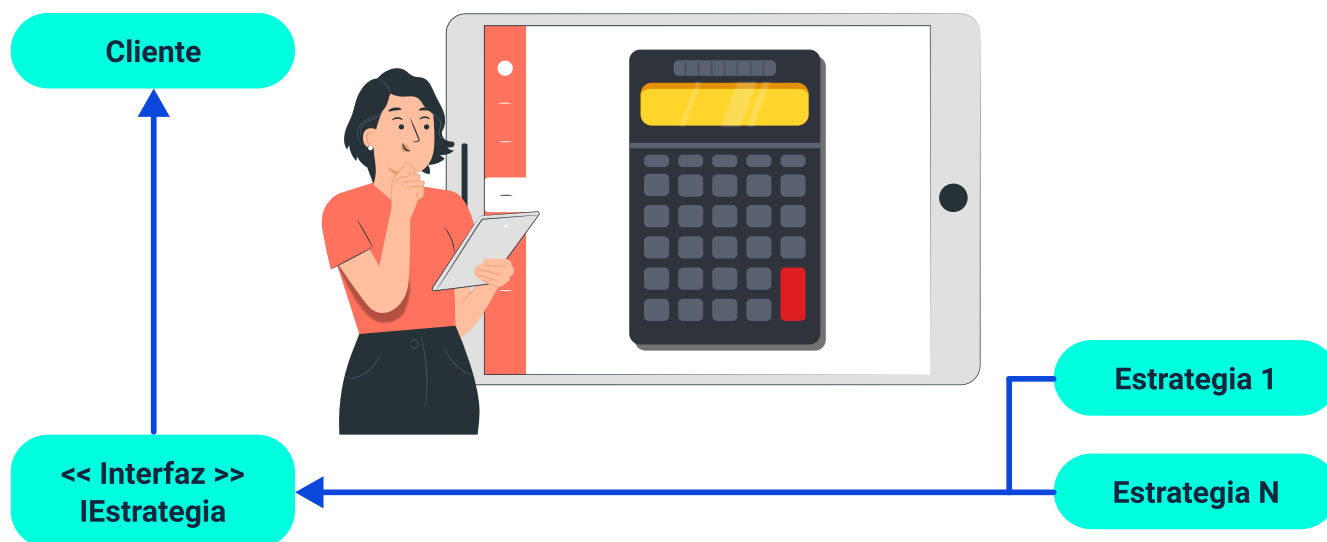
El patrón estrategia permite encapsular un conjunto de algoritmos de manera que puedan ser seleccionados dinámicamente para su ejecución, durante el tiempo de ejecución y de acuerdo con las acciones del cliente. Este patrón es una de las formas en las que se ven reflejadas fácilmente, las características de la programación orientada a objetos, particularmente en lo referente a encapsulamiento y polimorfismo. Landa. (2018).

Este patrón de diseño es útil cuando una misma funcionalidad puede ser provista usando diferentes mecanismos, algoritmos o estrategias, que serán seleccionadas dependiendo de las acciones realizadas por el cliente, en el momento que está ejecutando el programa.

Ejemplo

Suponga que quiere implementar una calculadora, la cual provee un conjunto de operaciones (suma, resta, multiplicación y división) que serán usadas por el cliente según su deseo. Cada una de estas operaciones representan una estrategia diferente y será el cliente quien invocará la ejecución de cada una de ellas dependiendo de su deseo por medio de una interfaz que, usando las propiedades del polimorfismo, se transformará para poder responder a cada solicitud.

Figura 1. Ejemplo patrón comportamental



Comando

El patrón comando permite aislar los objetos que realizan una petición de los objetos concretos encargados de recibir y realizar dicha acción, esto permite entre otras cosas, que las peticiones puedan ser enviadas a varios receptores y si se maneja el estado de las solicitudes, controla acciones de tipo Undo y Redo. El patrón comando necesita la implementación de varios elementos:

- **Comando**
Clase que sirve de puente entre el cliente y los receptores.
- **“Invoker”**
Elemento usado por los clientes y que le solicita al comando llevar a cabo una acción.
- **Cliente**
Invoca la ejecución de las acciones desde el “Invoker”.

- **Icomando**

Interfaz donde se especifican las operaciones a ejecutar.

- **Receptor**

Clase que realiza la acción.

- **Ejecutar**

Operación que necesita ser llevada a cabo.

Ejemplo

Una persona (Cliente) quiere hacer uso del televisor y hace todas las solicitudes de servicios por medio del control remoto ("Invoker"). El control remoto se comunica con una interfaz encargada de responder solicitudes de cada comando, que el usuario puede hacer como, por ejemplo, prender el televisor, apagar el televisor, subir el volumen, entre otros. Cada comando realiza una acción particular sobre el televisor (Receptor).

Figura 2. Ejemplo patrón comportamental Comando



Iterador

Este patrón de diseño está orientado al trabajo con colecciones y facilita el acceso a todos los elementos de la colección sin tener la necesidad de conocer su estructura.

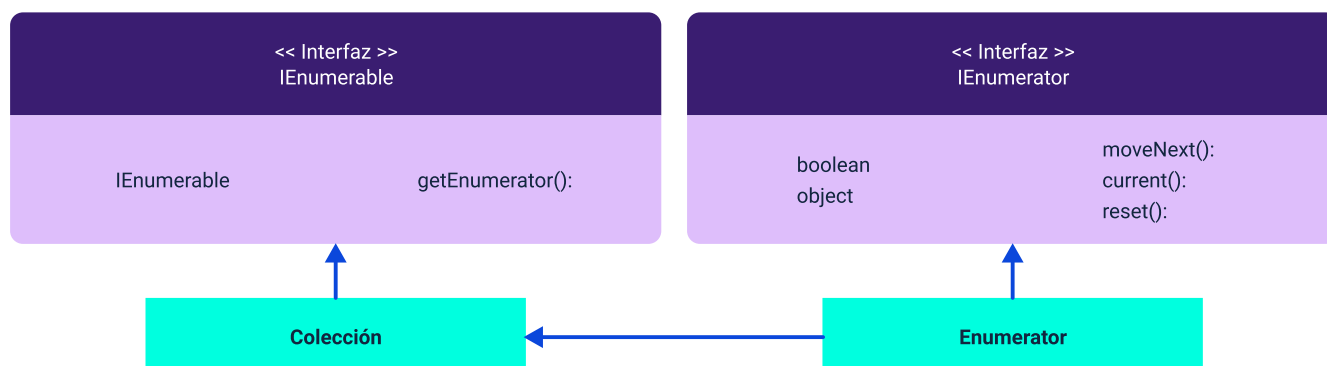
En este patrón se reconocen los enumeradores y los iteradores. Los enumeradores se establecen para conocer el siguiente elemento de la estructura y los iteradores corresponden a un mecanismo para recorrer la estructura, de acuerdo con su secuencia, de inicio a fin. El enumerado se encarga de implementar un conjunto de métodos estándar para poder establecer la secuencia con la que se debe recorrer la estructura.

Los métodos más comunes:

- “MoveNext” () que indica si existe o no un próximo elemento por recorrer.
- Método “Current” () que devuelve el valor actual de la colección según la posición actual en la secuencia.
- Método “Reset” () que permite iniciar nuevamente la secuencia a su punto de partida.

El iterador necesita del enumerador para poder hacer el proceso de recorrido. Este tipo de operaciones son tan comunes en los sistemas actuales, que los lenguajes de programación ya poseen una implementación propia del patrón iterador.

Figura 3. Ejemplo para el patrón comportamental iterador



3. Patrones creacionales

Una de las labores comunes en el proceso de construcción de “software” es precisamente distribuir responsabilidades en un conjunto de módulos o clases, siguiendo los principios definidos en los paradigmas de programación, como por ejemplo la programación orientada a objetos. Sin embargo, al momento de dar solución a un requerimiento particular se requerirá instanciar objetos de diferentes tipos, los cuales bajo sus responsabilidades implementan algún tipo de lógica.

En el recurso que le presentamos a continuación, profundice en los aspectos más importantes de los patrones creacionales:

Video 1. Patrones creacionales



[Enlace de reproducción del video](#)

Síntesis del video: Patrones creacionales

Los patrones de diseño creacionales se encargan de definir diferentes tipos de estrategia que pueden ser usadas al momento de requerir la instancia de un objeto particular. “Singleton”, este patrón de diseño creacional se encarga de definir la forma en que podemos garantizar que exista una única instancia de una clase particular en el contexto de la aplicación, esto es útil para casos en los que por cuestiones de manejo de memoria o de la lógica del negocio se requiere que sea el mismo objeto quien responda todos los mensajes independientemente del contexto actual de la aplicación, este patrón se ve reflejado en el cuerpo de la misma clase que se requiere sea instancia una sola vez, un ejemplo puede ser el manejo de conexiones a bases de datos, en algunos casos no sería conveniente instanciar nuevos objetos de conexión cada vez que se genere un evento, uno de los mecanismos usados para implementar el patrón “singleton” es la definición de constructores privados por lo cual es imposible crear instancias de forma directa, la única forma de instanciar un objeto de la clase es por medio del método “getInstance” el cuál verificará si existe o no una instancia creada de esta clase para determinar si usará el constructor primera invocación o si simplemente devuelve la instancia ya creada para el resto de las veces.

Fabrica abstracta, este factor de diseño permite la instanciación de una familia de objetos relacionados o dependientes sin necesidad de entrar en detalles particulares de su implementación, de forma que tanto el cliente como los elementos creados por la fábrica son totalmente independientes unos de otros. Este patrón define una interfaz de tipo fábrica la cual se refina por medio de la creación de fábricas concretas, las cuales pueden producir diferentes tipos de objetos y en

diferentes combinaciones según las necesidades del cliente, los elementos importantes de este patrón son: interfaz de la fábrica abstracta, la cual relaciona las operaciones de creación de los productos abstractos; fábricas concretas, las cuales implementan las operaciones de creación definidas en el interfaz de fábrica abstracta; interfaz de producto, el cual relaciona los comportamientos de los productos o elementos; productos concretos, los cuales implementan los comportamientos definidos en la interfaz de producto.

4. Patrones estructurales

Los patrones estructurales proveen una orientación relacionada con la forma de definir los componentes de los objetos. Dentro de los patrones estructurales se encuentran:

- **El patrón Fachada**

Utilizado cuando el sistema está compuesto por varios subsistemas.

- **El patrón “Delegate”**

Usado cuando se quiere reutilizar y extender funcionalidades de una clase, sin hacer uso de la herencia.

4.1. Fachada

El patrón fachada se emplea cuando un sistema está conformado por múltiples subsistemas, lo que complica la gestión de las interacciones que el cliente debe realizar con cada uno de ellos. Este patrón permite generar al cliente una vista de alto nivel que simplifica el control y el envío de mensajes a los subsistemas, ocultando los detalles relacionados con la gestión de las clases e instancias.

Existen diferentes variaciones del patrón fachada:

- **Opaco**

Una de las variaciones más utilizadas y en la cual los clientes no pueden acceder a los subsistemas, solo se puede hacer mediante el objeto fachada.

- **Transparente**

El cliente tiene la posibilidad de acceder a los subsistemas por medio de la fachada, pero también puede hacerlo de forma directa.

- **Estática**

En esta variación la fachada se implementa como una clase estática, por lo cual, no se hace necesaria la instancia de un objeto concreto de la fachada.

En este patrón se reconocen tres partes fundamentales. Landa, (2018):

- **Fachada**

Clase que provee las operaciones de alto nivel que serán usadas por el cliente.

- **Subsistemas**

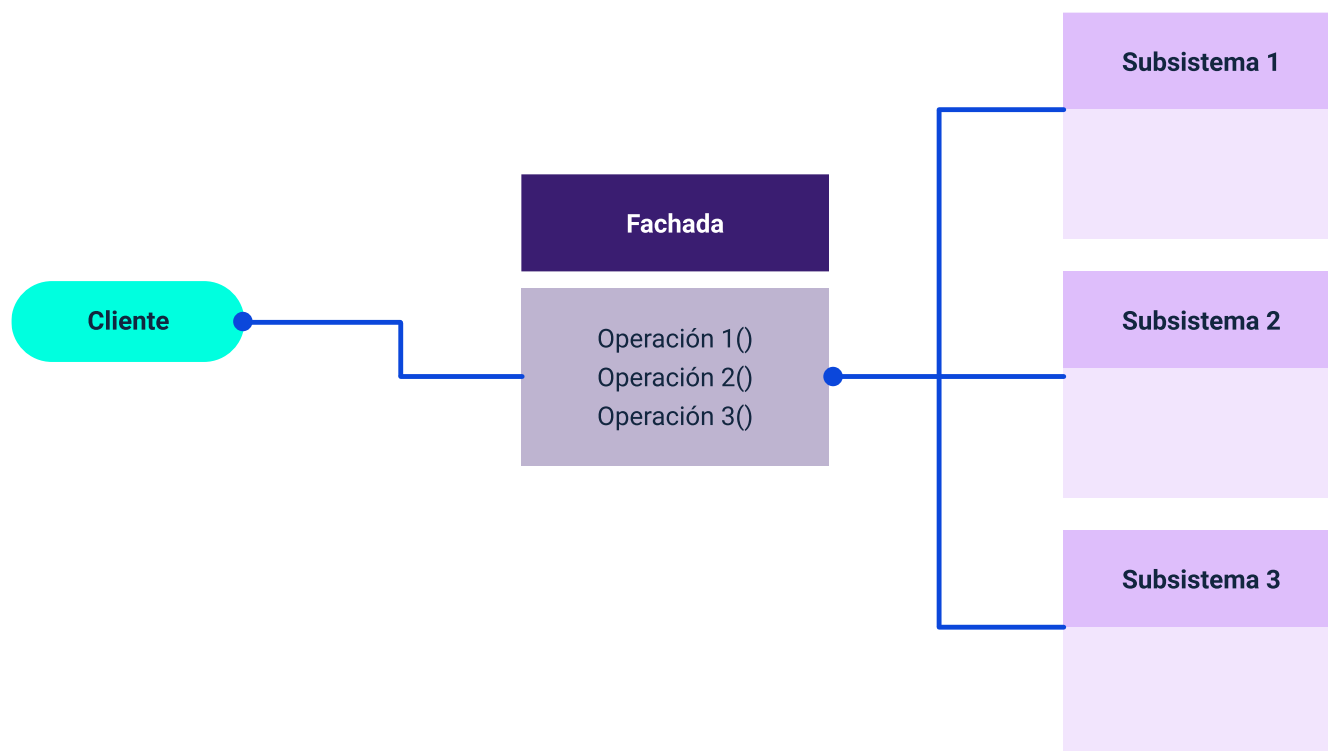
Clases que proveen las funcionalidades que son expuestas por la fachada.

- **Cliente**

Hace uso de las operaciones de alto nivel por medio de la fachada.

Se representa de la siguiente manera:

Figura 4. Diagrama patrón fachada



A continuación, se muestra una implementación en código Java de cada uno de los elementos requeridos para la implementación del patrón fachada en un ejemplo sencillo.

Ejemplo: el cliente quiere realizar una compra, pero este proceso involucra acciones por parte de tres diferentes subsistemas:

- **Subsistema de compras:** verifica la viabilidad de la tarjeta para el pago.
- **Subsistema de inventario:** verifica si hay productos en “stock”.
- **Subsistema de envíos.**

Para disminuir la complejidad, de la cual no debería ocuparse el cliente, se creará una fachada que encapsula las acciones de todos los subsistemas involucrados y

proveer una interfaz simple con la que el cliente podrá interactuar más fácilmente, como se presenta a continuación:

Clase que representa el subsistema de compras

```
public class GestorCompra {  
  
    private Scanner in = new Scanner(System.in);  
  
    public boolean comprar(){  
  
        int numero;  
  
        System.out.println("Ingrese el número de tarjeta para realizar el pago");  
  
        número = in.nextInt();  
  
        if(numero == 4567){  
  
            System.out.println("Procesando la compra");  
  
            System.out.println("-----");  
  
            System.out.println("Pago aceptado");  
  
            return true;  
  
        }else{  
  
            System.out.println("Pago rechazado");  
  
            return false;  
  
        }  
  
    }  
}
```

```
}
```

Clase que representa el subsistema de inventario

```
public class GestorInventario {  
  
    private int stock;  
  
    public GestorInventario() {  
  
        this.stock = 2;  
  
    }  
  
    public boolean retirarStock(){  
  
        if(stock > 0){  
  
            System.out.println("Producto listo para envío");  
  
            stock--;  
  
            return true;  
  
        }else{  
  
            System.out.println("Producto no disponible, no hay existencias, entrega  
reprogramada!");  
  
            return false;  
  
        }  
  
    }  
  
}
```

Clase que representa el subsistema de envíos

```
public class GestorEnvio {  
  
    public void enviarPedido(){  
  
        System.out.println("Envio autorizado y en camino!");  
  
    }  
  
}
```

Clase que representa la fachada de todos los subsistemas

```
public class Fachada {  
  
    //Instancias de los subsistemas que gestiona  
  
    private GestorCompra compra = new GestorCompra();  
  
    private GestorInventario inventario = new GestorInventario();  
  
    private GestorEnvio envio = new GestorEnvio();  
  
    //Metodo de alto nivel que se ofrece al cliente  
  
    public void compra(){  
  
        if(compra.comprar() && inventario.retirarStock()){  
  
            envio.enviarPedido();  
  
        }  
  
    }  
  
}
```


Clase que representa al cliente

```
public class Cliente {  
  
    public static void main(String[] args) {  
  
        Fachada fachada = new Fachada();  
  
        // Se hace uso de las operaciones de alto nivel  
  
        fachada.compra();  
  
        fachada.compra();  
  
        fachada.compra();  
  
    }  
  
}
```

4.2. “Delegate”

El patrón “delegate” se usa cuando se quiere reutilizar y extender funcionalidades de una clase sin hacer uso de la herencia. Este patrón permite de cierta forma implementar algo similar a la herencia múltiple que no es admitido por algunos lenguajes de programación, pero adicionalmente permite tener un control más detallado sobre este proceso, ya que se puede ocultar parte de los elementos heredados o, incluso, compartir elementos que no son posibles de heredar bajo el mecanismo de herencia tradicional.

Este patrón busca evitar la concentración de todas las responsabilidades en una única instancia, optando en cambio, por delegar las tareas en otras instancias especializadas en resolver dichas funciones.

En el siguiente ejemplo se muestra la implementación del patrón de diseño en cuestión, donde se aplica el patrón “Delegate” para integrar en una clase concreta funcionalidades definidas en otras clases a través del uso de interfaces.

Creación de interfaces para comportamientos reutilizables

```
public interface IDisenadora {  
  
    void disenar();  
  
}  
  
public interface ICodificadora {  
  
    void codificar();  
  
}
```

Implementación de interfaces en clases concretas

```
public class ClaseCodificadora implements ICodificadora{  
  
    @Override  
  
    public void codificar() {  
  
        System.out.println("Codificación por la clase codificadora");  
  
    }  
  
}  
  
public class ClaseDisenadora implements IDisenadora{  
  
    @Override  
  
    public void disenar() {
```

```
        System.out.println("Diseñado por clase Diseñadora");  
    }  
}
```

Generación de clase reutilizadora de funcionalidades

```
public class Empleado implements ICodificadora, IDisenadora {  
  
    ICodificadora codificador;  
  
    IDisenadora disenador;  
  
    public Empleado(ICodificadora codificador, IDisenadora disenador) {  
  
        this.codificador = codificador;  
  
        this.disenador = disenador;  
  
    }  
  
    @Override  
  
    public void codificar() {  
  
        codificador.codificar();  
  
    }  
  
    @Override  
  
    public void disenar() {  
  
        disenador.disenar();  
  
    }  
}
```

```
}
```

Instanciación de la clase principal con clases concretasrserán reutilizados

```
public class PatronDelegate {  
  
    public static void main(String[] args) {  
  
        Empleado objEmpleado = new Empleado(new ClaseCodificadora(),new  
ClaseDisenadora());  
  
        objEmpleado.codificar();  
  
        objEmpleado.disenar();  
  
    }  
  
}
```

Cualquier nueva clase concreta que implemente las interfaces originales podrán ser utilizadas por la clase Empleado para su reutilización, simplemente pasando la instancia correspondiente que implementa la nueva lógica al parámetro del constructor.

5. Vistas estáticas

La vista estática está encargada de modelar los conceptos significativos del dominio de la aplicación desde sus propiedades internas y las relaciones existentes. Se denomina vista estática porque no modela el comportamiento del sistema ni muestra las variaciones que se puedan presentar por efecto del tiempo.

Los elementos fundamentales de la vista estática son las clases que describen los conceptos del dominio del problema y las relaciones que pueden ser de tipo:

- Asociación
- Dependencia
- Generalización

Entre los diagramas de Lenguaje Unificado de Modelado (UML) que se utilizan para representar la vista estática del sistema, encontramos (ITCA, 2021):

- Diagrama de clases.
- Diagrama de objetos.
- Diagramas de componentes.

6. Diagrama de despliegue

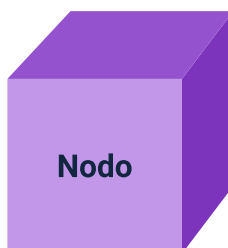
Los diagramas de despliegue, que forman parte de los tipos de diagramas propuestos por UML, tienen como objetivo representar la arquitectura física del sistema, tanto en “hardware” como en “software”, así como las conexiones entre ellos. Estos diagramas son de gran utilidad para el proceso de despliegue del sistema.

Los diagramas de despliegue utilizan un conjunto de elementos gráficos que tienen una representación y significado estandarizado. A continuación, se detalla cada uno de ellos:

- **Nodos**

Representa un elemento que puede ser “hardware” o “software” y se representa por un cubo.

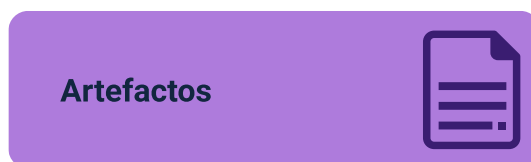
Figura 5. Nodos



- **Artefactos**

Representan elementos concretos generados en el proceso de desarrollo, por ejemplo, bibliotecas, archivos, entre otros.

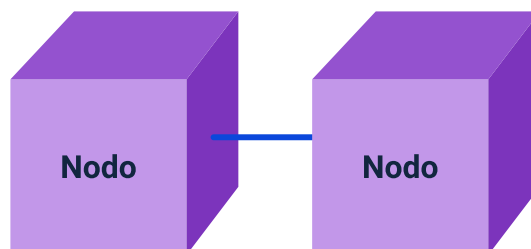
Figura 6. Artefactos



- **Asociación de comunicación**

Representa el camino de comunicación entre nodos, ilustrado mediante una línea continua que une los nodos correspondientes.

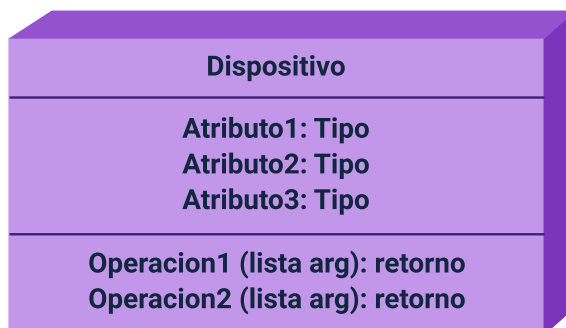
Figura 7. Asociación de comunicación



- **Dispositivos**

Es un tipo especial de nodo que representa un recurso computacional del sistema, por ejemplo, un servidor.

Figura 8. Dispositivos



- **Especificaciones de despliegue**

Representan configuraciones que se deben tener en cuenta para desplegar un artefacto en un nodo.

Figura 9.

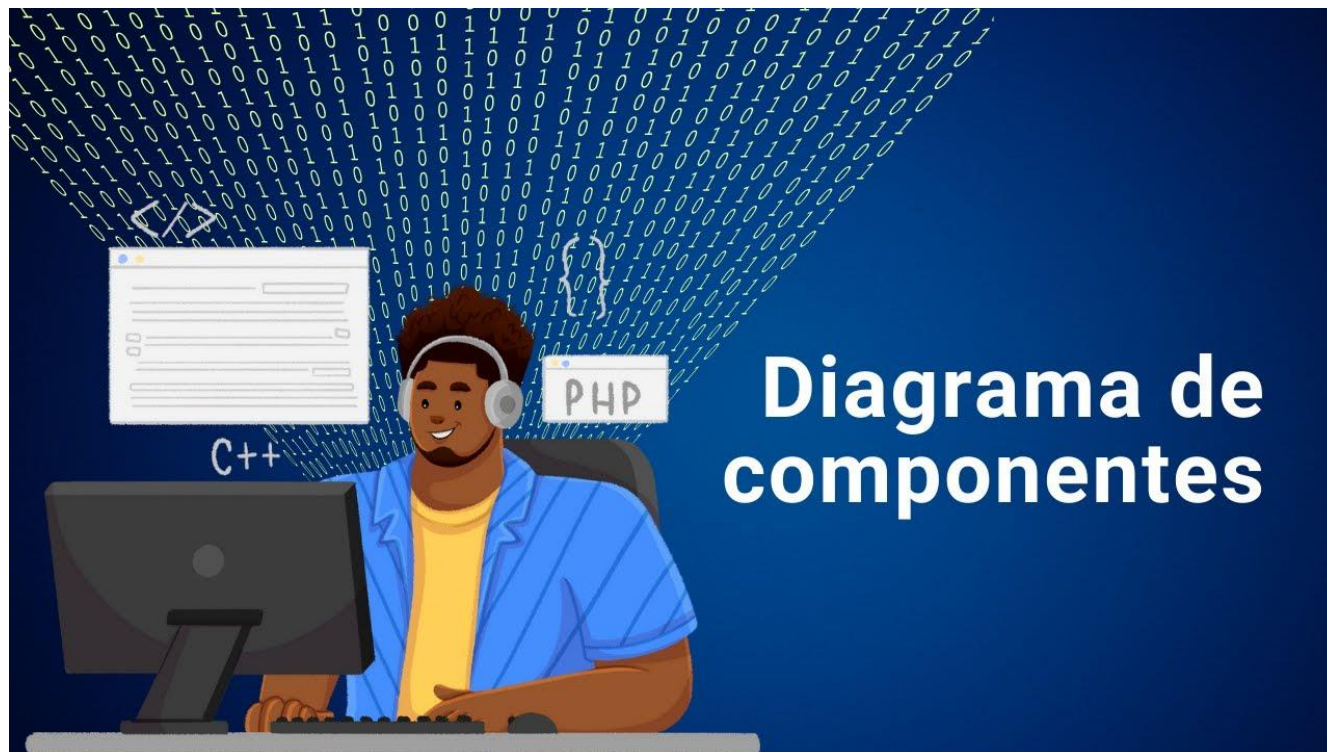
Especificación de despliegue
Atributo1: Tipo
Atributo2: Tipo
Atributo3: Tipo
Atributo4: Tipo
Atributo5: Tipo

7. Diagrama de componentes

El diagrama de componentes, uno de los diagramas sugeridos por UML, muestra una perspectiva estática del sistema de información y pertenece a los diagramas estructurales. Representa cómo se organiza y se relaciona el “software” en términos de componentes lógicos o físicos, como bibliotecas, módulos, paquetes y capas. Ofrece una visión general de los componentes del sistema, resaltando la organización y dependencia entre ellos, lo cual es fundamental para comprender cómo se integran y cooperan para realizar las funcionalidades del sistema.

Le invitamos a revisar los aspectos más característicos e importantes de los diagramas de componentes, explorando el recurso que se muestra a continuación:

Video 2. Diagrama de componentes



[Enlace de reproducción del video](#)

Síntesis del video: Diagrama de componentes

El diagrama de componentes es uno de los diagramas propuestos en UML, que representa una vista estática del sistema de información y hace parte de los diagramas estructurales, este tipo de diagrama proporciona una vista de alto nivel de los componentes dentro del sistema y generalmente se construye posterior a la construcción del diagrama de clases.

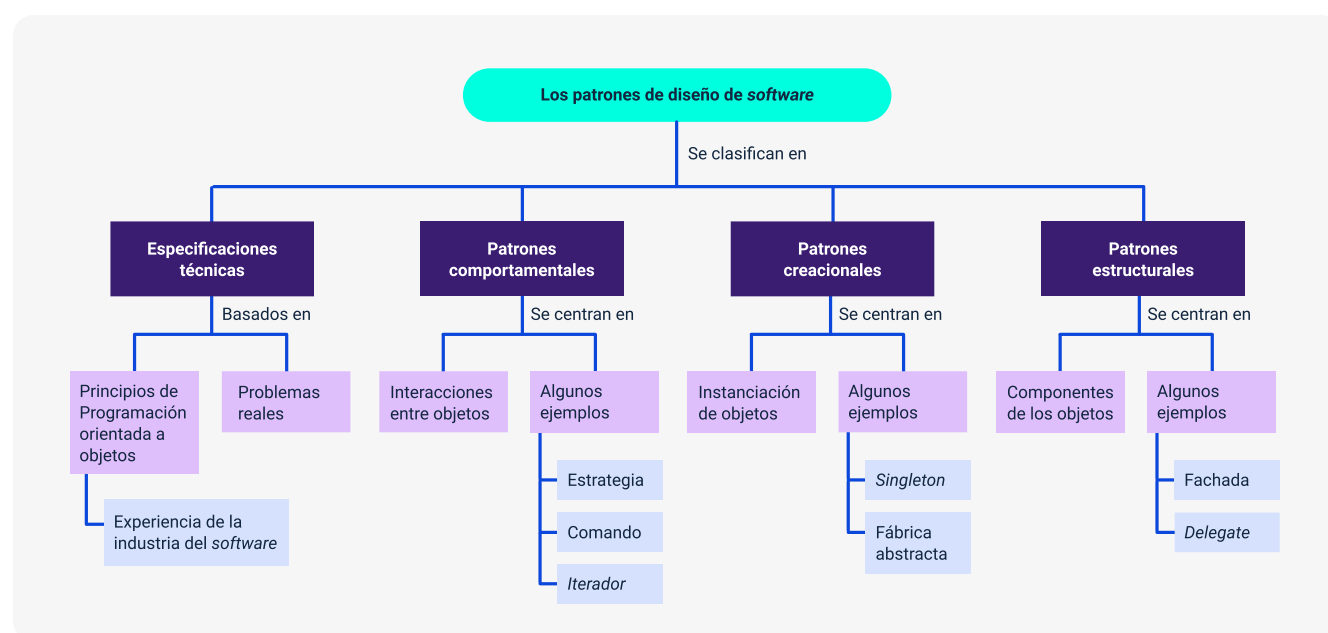
Un componente puede ser “software”, como por ejemplo las bases de datos o una interfaz de usuario, también puede ser “hardware” como un dispositivo o incluso una unidad como la nómina, el inventario, proveedores, etc.

Los diagramas de componentes son muy útiles para arquitecturas orientadas a servicios, permiten mostrar la estructura general del código por lo que puede ser usado para mostrar las funciones del sistema que se construye a cualquier parte interesada.

Los elementos que conforman un diagrama de componentes son:
Componente: es una abstracción de un nivel más alto que las clases y representa unidades lógicas del sistema. Interfaz: las interfaces siempre se asocian a los componentes y representan el lugar que debe ser usado por otros componentes para poder establecer comunicación con el. Relación de dependencia: representa una relación más general entre dos componentes para indicar que un componente requiere de otro, para poder ejecutar su trabajo. Paquetes: también es posible utilizar los paquetes para agrupar lógicamente un conjunto de componentes dentro de un subsistema.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Conceptos sobre patrones de diseño	Leiva, A. (2020). Patrones de diseño “software”: Repaso completo en 10 minutos.	Video	https://www.youtube.com/watch?v=6BHOeDL8vls&feature=youtu.be
Diagrama de despliegue	Nicosiored. (2018). Diagrama de Despliegue - 22 - Tutorial UML en español.	Video	https://www.youtube.com/watch?v=NSB0ATJUavA&feature=youtu.be
Diagrama de componentes	Nicosiored. (2018). Diagrama de Componentes I - 20- Tutorial UML en español.	Video	https://www.youtube.com/watch?v=oOycG_n1ARs&feature=youtu.be

Glosario

Lenguaje Unificado de Modelado (UML): relaciona un conjunto de diagramas estandarizados para la representación de sistemas de información desde diferentes tipos de vista.

Patrones GOF: los patrones GOF, se trata de los patrones de diseño y se denominan de esa manera por el libro “Design Patterns elements of reusable software”, de cuatro autores que descubrieron la manera fundamental de adentrarse en el mundo de la programación. GOF es la sigla de “The Gang of four”, que significa La pandilla de los cuatro.

“Software”: se trata del compendio de rutinas o de programas que favorecen a los dispositivos el cumplimiento y ejecución de determinadas tareas, funciones o aplicaciones.

Referencias bibliográficas

Creately (2021, Enero 15). La Guía Fácil de los Diagramas de Despliegue UML. [Web log post]. Blogspot. <https://creately.com/blog/es/diagramas/tutorial-de-diagrama-de-despliegue/>

DiagramasUML (2013, Diciembre 8). ¿Qué es UML? ¿Qué diagramas componen UML? [Web log post]. Blogspot. <https://diagramasuml.com/componentes/>

EcuRed (2021). Patrones Gof. ECURED. https://www.ecured.cu/Patrones_Gof

Escuela especializada en ingeniería (2021). Diagramas UML estáticos. ITCA-FEPADE. https://virtual.itca.edu.sv/Mediadores/ads/222_diagramas_uml_estticos.html

Gamma, E., Helm, R., Johnson, R., Vlissides, J., & Booch, G. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.

Landa, N. (2018). Patrones de Diseño de Software. [Video]. YouTube. https://www.youtube.com/playlist?list=PLM-p96nOrGcbqbl_A29b0z3KUXdq2_fpn

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Jonathan Guerrero Astaiza	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccionale	Centro de Servicios de Salud - Regional Antioquia
Juan Daniel Polanco Muñoz	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Álvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Casos de uso	Administrar datos pacientes	
Descripción	El comportamiento del sistema deberá describir el paso a paso del caso de uso cuando el personal encargado de gestionar datos del paciente inicie el ingreso de estos.	
Precondición	El paciente se encuentra ingresado al sistema y tiene la documentación necesaria para poder ser ingresado al sistema.	
Secuencia Normal	Paso	
	1	El personal médico ingresa al sistema para registrar el nuevo paciente.
	2	El sistema carga formulario para registro de datos del paciente así: identificación, nombre(s), apellido(s), dirección, teléfono, estrato, tipo de RH, sexo, acudiente
	3	El personal médico ingresa los datos suministrados por el paciente y ejecuta la acción en el sistema.
	4	El sistema almacena los datos suministrados por el personal médico, imprime el carnet del Sistema General de Seguridad Social en Salud, el sistema comunica al personal médico que el proceso ha terminado de manera exitosa.
	5	El personal médico genera reporte del sistema de registro al nuevo paciente, mediante la expedición del carnet o constancia de inscripción al sistema de Seguridad Social en Salud.
Post condición	El paciente se encuentra registrado en el Sistema General de Seguridad Social en Salud, su historial clínico es nuevo.	
Excepciones	Paso	Acción
	1	El sistema detecta la duplicación de un paciente registrado con la identificación que se registra, procede a informar al personal médico, estos deben modificar y/o actualizar la información que sea necesaria y continuar con el caso de uso.
	2	Si el personal médico cancela el registro del paciente se termina el caso de uso.

Generar los diagramas de los procesos de negocio

El diagrama de procesos es también conocido como diagrama de flujo y en el mundo de los sistemas orientados por procesamiento electrónico de datos, obedece a un algoritmo, es decir, al conjunto de pasos lógicos para resolver un problema. Es una de las herramientas más útiles para cualquier entidad que quiere gestionar proyectos de todo tipo tanto a nivel interno como externo a ella. Permite abordar la descomposición y conocimiento de los procesos empresariales en orden lógico, plasmados dentro de un único documento con sus relaciones, visualizar oportunidades de mejora y, en general, para dar importancia y generar control hasta el más mínimo detalle a todos los procesos de una compañía, por pequeños que puedan considerarse. Encontrando con ello, lo siguiente:

a) Diagrama de procesos

Teniendo en cuenta la teoría general de sistemas antes relacionada a un diagrama de procesos, es la forma en la cual se aplica el enfoque sistémico para representar de manera gráfica los componentes de las principales actividades que pueden inferirse a su vez como subsistemas que conforman el todo de la interoperación, con orden lógico, guiados por sus interrelaciones. Muestra el flujo secuencial de interacción de las actividades de un proceso a través de símbolos gráficos para proporcionar una mejor visualización del funcionamiento de este, ayudando a su entendimiento y haciendo su descripción más didáctica, visual e intuitiva.

Así, el diagrama de procesos se convierte en una herramienta primordial para llevar a cabo el análisis de manera global y ver en qué acciones del flujo de trabajo se pueden introducir métricas, controles o descubrir oportunidades para mejorar, especialmente pretende orientar los aspectos que permiten como primera medida, aumentar la productividad de los empleados, delimitar y asignar las competencias para cada tarea, ver cuellos de botella, es decir, posibles atascos en el devenir del flujo y en general, para aclarar el propio flujo de trabajo.

En definitiva, es una de las herramientas que más ayuda en la consecución de una mejor gestión empresarial integral, especialmente en aquellas entidades en donde los procesos suelen ser acoplados, de alta complejidad, bastante rígidos o están muy interrelacionados.

Teniendo en cuenta las labores a ejecutar vs un tiempo estimado o empleado los diagramas de procesos más utilizados para la gestión de proyectos son el diagrama de Gantt y el diagrama de Pert. Los dos diagramas permiten planificar, simplificar, programar actividades, asignar responsabilidades, pesos por valor de actividad de forma sencilla y muy visual que facilitan su entendimiento.

b) Elementos de un diagrama de procesos

A la hora de construir o representar y dibujar un diagrama de procesos, existe un lenguaje de escritura que se conforma por una serie de símbolos comúnmente aceptados que han sido establecidos por el estándar ANSI.

Estos diagramas están compuestos por elementos gráficos como lo son terminadores, símbolos de procesos, de subprocesos y de decisiones, líneas con flechas y conectores. Los principales elementos están representados de las siguientes formas:

1. **Procesos o actividades:** su representación es un rectángulo, se refieren a una acción en un proceso de negocios. Son, posiblemente, los elementos más importantes de un diagrama, en cada uno de estos se recomienda de manera detallada aplicar el análisis sistémico en cada

actividad, son los que siempre deben aparecer, y por eso deben describirse de forma clara y concisa. Deben ser descritos con una frase única de verbo y sustantivo. Por ejemplo: "Redactar un nuevo informe para presentar a la gerencia de la empresa".

2. **Subprocesos:** estos se representan con un rectángulo y líneas dobles a cada lado. Deben ser considerados como partes de un proceso padre de mayor complejidad y sirven para determinar funciones más simples al diagrama de flujo, incluso reutilizables.
3. **Nodos de decisión:** son los que se representan mediante un triángulo doble (diamante) y son los que muestran los puntos de decisión a seguir, bien sea positivo si se responde a la decisión de "sí" o negativo "no".
4. **Conectores:** representados por círculos o cuadros conectores que se deben acompañar con la utilización de etiquetas o nombres de caracteres comúnmente alfanuméricos. Aseguran que todos los procesos están conectados de forma lógica y correcta para la correcta navegación entre páginas.
5. **Líneas de flecha:** utilizadas gráficamente para mantener el hilo conductor del flujo de trabajo la coherencia y claridad en un diagrama de procesos. Se pueden dibujar hacia todas las direcciones, normalmente representa el camino por el que fluirá el diagrama.
6. **Conector para elementos fuera de página:** permite establecer relaciones para elementos que no están en la misma página.
7. **Entrada manual:** permite representar una acción, la cual requiere intervención humana.
8. **Almacenamiento:** representa el sitio en donde normalmente se aloja la información a manera de datos, que comúnmente son de carácter estático.
9. **Múltiples documentos:** representa la necesidad de tener en cuenta varios elementos de información o normas, especialmente son considerados como entradas.
10. **Terminadores:** se representan por un rectángulo con esquinas curvas. Se consideran como los mayores delimitadores del flujo y deben aparecer siempre al inicio y al final de un diagrama de flujo para indicar la parte del documento o gráfico en donde inicia y en donde finaliza un diagrama.

c) Un buen diagrama de procesos

Al momento de crear un diagrama de procesos, se recomienda guiarse por al menos cumplir los siguientes pasos:

1. Ubicar o según sea el caso abstraer los componentes del proceso: en este paso se aclaran cuáles son los parámetros de entrada al proceso y sus salidas, así como las actividades que se desarrollan en el mismo.
2. Ordenar de manera lógica las actividades: dado que el diagrama de procesos es un flujo, es necesario ordenar las actividades para saber cuál es su secuencia.
3. Elegir los símbolos adecuados conforme el tipo de tarea para que cualquier usuario pueda diferenciar los distintos elementos que forman el diagrama.
4. Hacer la conexión y relación entre las actividades, para lo cual se utilizan flechas y líneas de puntos o continuas.

5. Representar el comienzo y el final de cada proceso. Aunque pueda parecer trivial e irrelevante, lo cierto es que este es un punto muy importante para establecer los límites de cada proceso u actividad, de cara a los propietarios de los procesos, gerentes y supervisores.
6. Validar, socializar con los integrantes del equipo y revisar el diagrama de procesos para asegurarse que su representación obedece a un gráfico acorde con lo que se quiere reflejar.

d) Dividendos de los diagramas de procesos

Utilizar un diagrama de flujo le aporta los siguientes beneficios a la empresa:

1. Permite obtener la primera aproximación desde el punto de vista algorítmico de cada actividad o proceso, así como también puntos de interoperabilidad.
2. Favorece la comprensión del proceso que trata de simular. Dado que normalmente están representadas a través de algún elemento gráfico y sus nodos están interconectados entre sí, gracias a un diagrama de este tipo es más fácil saber cuáles son las tareas que se dibujan, así como sus relaciones de dependencia.
3. Permite identificar de una forma rápida los inconvenientes y dónde se producen. Así, es más fácil saber dónde se crean o se presentan posibles cuellos de botella, las responsabilidades de cada área y los puntos de decisión. Asimismo, permite analizar de una forma mucho más exhaustiva el problema, lo cual hace posible que aparezcan nuevas soluciones y alternativas.
4. Permite definir claramente las fronteras de cada proceso, ya que todas las tareas están lo suficientemente delimitadas, incluso aunque estén interrelacionadas. De esta manera, se puede saber qué tareas suponen un mayor valor añadido y cuáles no aportan tanto.
5. También es una buena herramienta de capacitación, instrucción y formación para nuevos empleados que se integran dentro de la empresa facilitando su proceso de adaptación a la organización.

Conceptos generales de bases de datos

Breve descripción:

En la sociedad de la información es imprescindible el empleo de tecnología, técnicas y procedimientos que, a lo largo de las últimas décadas, han consolidado un marco conceptual importante y necesario en todo proceso de administración de datos, por eso, este componente acerca estos conceptos de manera estructurada, para avanzar en los procesos de gestión de la información.

Junio 2024

Tabla de contenido

Introducción	4
1. Conceptos generales de base de datos.....	5
1.1. Tipos de datos y restricción de no nulidad	9
1.2. Tipos de bases de datos	12
1.3. Clasificación de bases de datos	13
1.4. Sistema de gestión de bases de datos	15
2. Modelo entidad relación	16
2.1. Relaciones entre entidades	19
Principio de unicidad	19
Claves	19
Claves candidata, superclave y clave primaria.....	21
2.2. Relaciones de uno a muchos	24
2.3. Relación de muchos a muchos	28
2.4. Relaciones de uno a uno	31
3. Normalización	34
3.1. Dependencias funcionales	37
Definición formal dependencia funcional	40
3.2. Diseño relacional	41

3.3. Reglas de integridad	45
Regla de borrado	48
Regla de edición	49
3.4. Lenguajes de los sistemas administradores de bases de datos	50
Sentencias de definición de datos (DDL)	50
Sentencias de manejo de datos (DML)	51
Ejemplo DML	51
Sentencias de control (DCL)	51
Sentencias de control de transacciones (TCL)	52
4. Sistema gestor de base de datos	53
Síntesis	64
Material complementario	65
Glosario	66
Referencias bibliográficas	67
Créditos	68

Introducción

Una base de datos se puede percibir como un “almacén” de información que se define y se crea una sola vez para guardar grandes cantidades de datos de forma organizada (o estructurada), con el fin de poder encontrarla y utilizarla fácilmente.

Teniendo en cuenta la anterior información, en el presente componente formativo, se presentará el concepto y características de las bases de datos actuales, tomando en cuenta que cada base de datos es diseñada para cumplir con los requisitos de información de una organización o una empresa.

1. Conceptos generales de base de datos

Como concepto general de bases de datos podemos encontrar:

“El conjunto de datos organizado de tal modo que permita obtener con rapidez diversos tipos de información”. (RAE, 2001).

- De acuerdo con esta definición, una hoja de cálculo de Excel puede considerarse una base de datos, o un conjunto de archivos debidamente organizados, o la lista de nombres y teléfonos que está en nuestros smartphones. En principio, es correcto llamarle base de datos a estos ejemplos, sin embargo, en el contexto de desarrollo de software, se referirá a ese conjunto de información que puede ser almacenada en grandes cantidades de forma organizada y es gestionada a través de un Sistema de Gestión de Bases de Datos (SGBD).
- Para organizar y definir la información de forma sistemática, las bases de datos también deben poder almacenar una descripción precisa de los datos que contiene, conocida como metadatos, a los que se le relaciona el tipo de información que es conceptualmente guardada (es decir, si se agrega una explicación de la naturaleza del dato en la empresa), se da origen a lo que conoce como diccionario de datos o catálogo de datos.

Ejemplo 1 - Determinar los metadatos del siguiente problema

Estos diseños están concebidos para emplear Sistemas de Gestión de Bases de Datos (en adelante SGDB), que es un sistema o servicio informático que permite a las personas definir, crear, dar soporte y mantenimiento a las bases de datos, controlando el acceso de forma segura.

Figura 1. Base de datos de llamadas telefónicas

calldate ▼	src ▼	dst ▼	duration ▼	disposition ▼
2021-04-10 08:48:50.0	3176964335	444	00:09:23	ANSWERED
2021-04-10 08:54:14.0	3217212347	444	00:07:47	ANSWERED
2021-04-10 08:57:17.0	3203857798	444	00:05:17	ANSWERED
2021-04-10 09:03:27.0	3226140856	666	00:04:22	ANSWERED
2021-04-10 09:14:13.0	28204400	666	00:00:36	ANSWERED
2021-04-10 09:14:45.0	3148915614	666	00:00:34	ANSWERED
2021-04-10 09:15:29.0	3148915614	666	00:02:27	ANSWERED
2021-04-10 09:18:13.0	3005373259	666	00:01:28	ANSWERED
2021-04-10 09:07:33.0	312523352	444	00:12:12	ANSWERED
2021-04-10 09:13:19.0	7006	444	00:09:06	ANSWERED
2021-04-10 09:21:30.0	3125228302	883108608263	00:00:54	ANSWERED
2021-04-10 09:22:41.0	3203498937	666	00:03:12	ANSWERED
2021-04-10 09:28:04.0	301736892	s	00:00:05	ANSWERED
2021-04-10 09:29:13.0	301736892	tdial	00:00:00	FAILED
2021-04-10 09:29:23.0	7006	tdial	00:02:20	NO ANSWER
2021-04-10 09:30:07.0	7006	883507070131	00:04:32	NO ANSWER
2021-04-10 09:30:28.0	3203498937	883507070131	00:06:57	NO ANSWER
2021-04-10 09:28:35.0	7006	666	00:00:52	ANSWERED
2021-04-10 09:30:58.0	3184155040	883507070131	00:02:07	NO ANSWER
2021-04-10 09:28:32.0	3235767896	666	00:02:15	ANSWERED
2021-04-10 09:27:51.0	3158628495	666	00:052:30	ANSWERED
2021-04-10 09:28:49.0	3214078916	444	00:00:00	ANSWERED
2021-04-10 09:35:45.0	3004668735	666	00:04:32	ANSWERED
2021-04-10 09:35:45.0	3226140856	1	00:06:57	ANSWERED
2021-04-10 09:39:53.0	3046032130	666	00:00:52	ANSWERED
2021-04-10 09:37:02.0	7006	666	00:02:07	ANSWERED
2021-04-10 09:44:57.0	444	883113243787	00:02:15	NO ANSWER

La figura muestra una tabla de datos que donde en cada columna se agrupa un tipo de dato particular, y cada fila es el conjunto de datos que se llama registro y comparten una relación (la relación es que cada fila corresponde a una única llamada telefónica); a esta estructura se le llama bidimensional, aquí filas y columnas son las dos dimensiones.

Los metadatos de la figura anterior serían la definición de cada tipo de dato como se lista a continuación. calldate: tiene la estructura YYYY-MM-DD HH:MI:SS.Z, donde:

- YYYY sería el año un dato numérico entero de cuatro dígitos.
- MM el mes un dato numérico entero de dos dígitos entre 01 y 12.
- DD un dato numérico entero de dos dígitos entre 01 y 31.
- HH un dato numérico de dos dígitos entre 00 y 23.
- MI y SS un dato numérico de dos dígitos entre 00 y 59.
- Z un dato numérico de un dígito (para representar milésimas de segundo).

Como este es el conjunto de datos de la primera columna, puede que use un término que describa lo anterior con una sola palabra que es **TIMESTAMP**, de esta forma esa primera columna puede quedar correctamente descrita como:

calldate **TIMESTAMP(1)** Donde 1 es el número de dígitos de milisegundos (Z).

src y dst: empleado para fuente de la llamada o llamante (src) y destino de la llamada (dst), puede ser un número telefónico o de extensión telefónica. Algunos tienen el código de país seguido del signo más, ejemplo: 57+3155008002. Por lo tanto, se puede describir como una cadena de texto que no superará los 25 caracteres, pero puede tener menos de 25 caracteres, así el metadato iría descrito:

- src **VARCHAR(25)** De esta forma decimos que es una cadena de caracteres.
- dst **VARCHAR(25)** Que varía en longitud de 0 a 25 caracteres.
- duration Tiene la estructura HH:MI:SS y como se vio antes, representa el tiempo en horas, minutos y segundos: src **TIME** **TIME** se usa para esta estructura de dato en particular.

- disposition Indica si la llamada es atendida ANSWER, no atendida NO ANSWER, o si falló la FAIL. Se puede representar con una cadena de 10 caracteres.

calldate *TIMESTAM*(1),

src *VARCHAR*(25),

dst *VARCHAR*(25),

src *TIME*,

disposition *VARCHAR*(10)

Note que cada uno de ellos fue separado por una coma, para que se diferencie de los demás.

Ejemplo 2

Conociendo el objetivo de la base de datos y los metadatos de la figura anterior, crear un diccionario de datos o un catálogo de datos.

Solución:

Tabla 1. Diccionario de datos

Creación: 27/02/2021

Descripción: Registro de las llamadas telefónicas de la PBX de la empresa STD LTDA.

Campo	Tipo dato	Tamaño	Descripción
Calldate	TIMESTAMP	1	Representa el momento exacto en que entra o sale la llamada.

Campo	Tipo dato	Tamaño	Descripción
Src	VARCHAR	25	Fuente (source) de la llamada, número de quien hace la llamada telefónica.
Dst	VARCHAR	25	Número del destino de la llamada telefónica.
Duration	TIME	0	Duración de la llamada telefónica.
Disposition	VARCHAR	10	Estado final en que se considera quedó la llamada.

Como se puede notar en la tabla anterior, un diccionario de datos aporta información de la estructura de los datos y el uso que se da a cada dato en la empresa, organización o sistema de información, pero no sin antes darle un nombre a todo el conjunto de datos (primera fila), una fecha (segunda fila) y una descripción (tercera fila) del conjunto de los datos. En la tabla se puede identificar la independencia que existe entre la lógica de los datos (representado por la columna DESCRIPCIÓN del dato) y el almacenamiento físico (estructura de almacenamiento representado por las columnas TIPO DE DATO y TAMAÑO).

1.1. Tipos de datos y restricción de no nulidad

Existen muchos tipos de datos y varían según el SGDB, porque cada sistema gestor define sus propios tipos de datos, aunque existen las equivalencias notables y otras no tan notables. A continuación, se presenta una tabla que relaciona el tipo de datos según los SGDB más comunes del mercado, como ORACLE, PostgreSQL, MySQL y SQLServer.

Tabla 2. Tipos de datos según bases de datos

Tipo de dato	ORACLE	PostgreSQL	MySQL	SQLServer
Cadena caracteres	VARCHAR2	CHARACTER VARYING	VARCHAR	VARCHAR
Cadena texto	TEXT	TEXT	TEXT	NTEXT
Entero pequeño	SMALLINT	SMALLINT	SMALLINT	SMALLINT
Entero	INTEGER	INTEGER	INT	INT
Fecha	DATE	DATE	DATE	DATE
Fecha y hora	DATE	TIMESTAMP WITH TIME ZONE	DATETIME	DATETIME2
Hora	DATE	TIME	TIME	TIME
Entero con decimales	FLOAT	REAL	FLOAT	FLOAT

Según el tipo de sistema gestor de bases de datos, cada uno define sus tipos de datos que, de alguna forma, son muy parecidos entre ellos, o existen equivalencias; las diferencias son pocas y están relacionadas con aspectos técnicos del almacenamiento. Se pueden consultar más detalles de cada uno de ellos en los sitios oficiales de cada motor de base de datos:

- **ORACLE:**

https://docs.oracle.com/cd/E11882_01/server.112/e41085/sqlqr06002.htm#SQLQR959

- **PostgreSQL:**

<https://www.postgresql.org/docs/13/datatype.html#DATATYPE-TABLE>

- **MySQL:**

<https://dev.mysql.com/doc/refman/8.0/en/data-types.html>

- **SQLServer:**

<https://docs.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver15>

Existen varios tipos de restricciones, por ahora se verá la restricción de longitud, por ejemplo:

src VARCHAR(25)

En él se define que el src, es una columna que no puede exceder los 25 caracteres. De la misma forma, existe una forma de definir que es obligatoria: registrar un dato en esa columna y se llama restricción de no nulidad, es decir, que ninguna fila puede tener valor nulo. Así, en el ejemplo de la base de datos de llamadas telefónicas, si se define que todo y cada uno de los datos son obligatorios (usando las palabras NOT NULL), la metadata que define la estructura de los datos, quedaría así:

calldate TIMESTAM(1) NOT NULL,

src VARCHAR(25)NOT NULL,

dst VARCHAR(25)NOT NULL,

src TIME NOT NULL,

disposition VARCHAR(10) NOT NULL

Algunos sistemas, para abreviar en la decisión de no nulidad, acostumbra usar el acrónimo NN para representar NOT NULL.

1.2. Tipos de bases de datos

Las bases de datos han evolucionado a lo largo del tiempo, usando diferentes tipos de modelos o enfoques, se ha comprobado que algunos de ellos tienen mejor desempeño en distintos tipos de aplicación. La estructura en la que se configuran los datos al interior de las bases de datos presenta una forma de clasificación (Kyocera, 2021):

- **Bases de datos con estructura jerárquica**

Las primeras bases de datos empleadas son las de estructura jerárquica. Las relaciones entre registros forman una estructura en árbol. En la actualidad, un ejemplo de estas bases de datos es la base de datos LDAP, usada para el sistema de directorio de datos tales como usuario, dispositivos, nombres, contraseñas, direcciones, etc. dentro de una red de computadores.

- **Bases de datos con estructura en red**

Esta estructura es un poco más compleja que la anterior, porque contiene otras relaciones de las jerárquicas, admitiendo relaciones de cada registro con otros, de manera tal que puede seguir distintos caminos para acceder a la información.

- **Bases de datos con estructura relacional**

Estas bases de datos son multipropósito, lo que las hace las más extendidas en la industria y más empleadas, pero son las más complejas de aprender, por lo que solo se abordará el diseño e implementación de este tipo de bases de datos. Se basan en tablas como las que se presentó en la

figura "Base de datos de llamadas telefónicas" y en la relación de los datos de unas tablas con otras.

- **Bases de datos con estructura multidimensional**

Es la extensión de las bases de datos de estructuras relacionales. Como se vio en las relacionales, se basan en estructuras bidimensionales (tablas), mientras que la multidimensional en cubos (tres dimensiones), o más complejas con N dimensiones.

- **Bases de datos con estructura orientada a objetos**

Una base de datos orientada a objetos es un sistema de gestión de base de datos mediante el cual representamos la información en forma de objetos, que son utilizados en programación orientada a objetos.

Cada tipo de bases de datos tiene un ámbito de aplicación en el que su desempeño es mejor. Por ejemplo, las bases de datos jerárquicas tienen un mejor desempeño en operaciones de consulta de datos puntuales, mientras que las de datos relacionales son precisas para garantizar la calidad de los datos y la no repetición de los mismos, aunque no muy optimizadas para hacer consultas, y, finalmente, las multidimensionales, son muy útiles para análisis estadístico de datos históricos de volúmenes inmensos de datos. Aparte del tipo de datos, existe otro tipo de clasificaciones, de acuerdo con la naturaleza de los datos.

1.3. Clasificación de bases de datos

Los datos se pueden clasificar según la naturaleza de los datos que se almacenan en la base de datos. La siguiente tabla muestra una compilación de estas clasificaciones.

Tabla 3. Clasificación

Criterio de clasificación	Clasificación	Descripción
Según la variabilidad de los datos	Bases de datos estáticas	Son bases de datos cuyos datos son históricos, es decir, ya no se pueden modificar, se usan comúnmente para estudiar el comportamiento de los datos a través del tiempo. Generalmente, a este tipo de bases de datos se les llama bodega de datos. Muchas de estas bases de datos se almacenan en cubos para su análisis, y se les conoce como OLAP es el acrónimo en inglés de Procesamiento Analítico en Línea (On-Line Analytical Processing).
Según la variabilidad de los datos	Bases de datos dinámicas	Estos datos se almacenan y pueden ser modificados, agregados, borrados y consultados en cualquier momento, por ejemplo: un sistema de facturación. A estos sistemas se les denomina transaccionales, porque cada operación de guardar, borrar o editar, se configura como una transacción. OLTP es la sigla en inglés de Procesamiento de Transacciones en Línea (On-Line Transaction Processing).
Según el contenido	Bases de datos documentales	Permiten la indexación a texto completo, y en líneas generales realizar búsquedas más potentes.
Según el contenido	Base de datos deductivos	Un sistema de base de datos deductivos, es un sistema de base de datos, pero con la diferencia que permite hacer deducciones a través de inferencias. Se basa principalmente en reglas y hechos que son almacenados en la base de datos. También las bases de datos deductivas son llamadas base de datos lógica, a raíz que se basan en lógica matemática.

Pueden existir otras clasificaciones orientadas al ámbito de uso, sin embargo, se presentan las más comunes en el ejercicio del desarrollo de software. Las bases de datos relacionales es de obligatorio dominio en cualquiera de los casos, debido a que los conceptos de estas son reutilizables en casi todos los otros tipos.

1.4. Sistema de gestión de bases de datos

Un sistema de gestión de la base de datos es un programa de computador que permite definir, crear y mantener los datos de una base de datos, controlando el acceso:

- Permiten la definición de la base de datos usando un lenguaje de definición de datos.
- Permiten la inserción, actualización, eliminación y consulta de datos usando un lenguaje de manejo de datos.
- Proporcionan un acceso controlado a la base de datos (con autenticación, roles, niveles de acceso).
- Concurrencia (varios usuarios a la vez accediendo o manipulando los datos) y multitarea.
- Algunos SGDB permiten administrar el catálogo de datos.

Los SGBD son una herramienta muy útil, sin embargo, se podría decir que los SGBD son un poco más complejos, porque los usuarios ven más datos y sus relaciones, de los que realmente se necesitan.

2. Modelo entidad relación

Las bases de datos relacionales, inicialmente no son fáciles de modelar, por lo que se requiere de cierto nivel de abstracción, es decir, se va ocultando la complejidad del diseño de la base de datos, primero analizando el requerimiento desde un punto de vista conceptual, entendido como las necesidades de la empresa o del sistema de información, luego se va analizando desde el punto de vista de la lógica de los datos y, finalmente, se analiza físicamente cómo se almacenarán los datos. A continuación, se presentan algunos conceptos:

- **Entidad**

El modelo de datos entidad-relación (E-R), está basado en una percepción del mundo real que consta de una colección de objetos básicos llamados entidades, y de las relaciones entre estos objetos. Una entidad es una «cosa» u «objeto» en el mundo real que es distinguible de otros objetos. Por ejemplo, cada persona es una entidad, y las cuentas bancarias pueden ser consideradas entidades y se representa con un rectángulo.

- **Atributos**

Las entidades están descritas por un conjunto de atributos, por ejemplo, una persona puede tener estos atributos: nombres, apellidos, edad, documento de identidad (identificación); o la cuenta bancaria puede tener los siguientes atributos: número de cuenta, saldo, fecha de creación; se representan con una elipse.

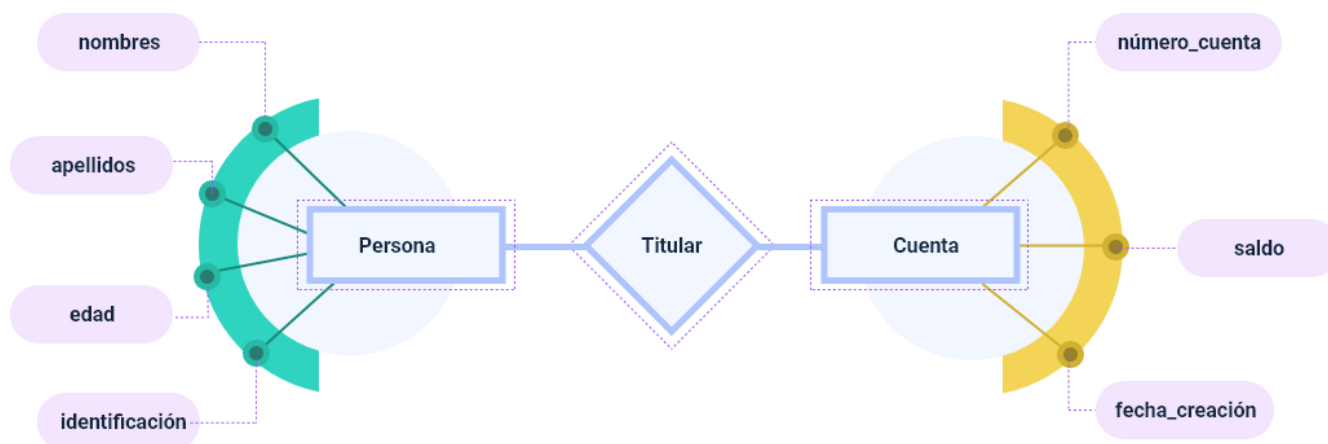
- **Relaciones**

Una relación es una asociación entre varias entidades. Por ejemplo, una relación titular asocia un cliente con cada cuenta que tiene (de la que es titular). Se representa con un rombo.

Se presentará más claro en un diagrama, pero antes se deben considerar los siguientes elementos:

- Rectángulos, que representan conjuntos de entidades.
- Elipses, que representan atributos.
- Rombos, que representan relaciones entre conjuntos de entidades.
- Líneas, que unen los atributos con los conjuntos de entidades y los conjuntos de entidades con las relaciones.

Figura 2. Entidad relación persona cuenta bancaria



Para una base de datos de un sistema bancario consistente en clientes (es decir personas) y sus cuentas bancarias, en la figura anterior se muestra el diagrama Entidad Relación. En ellos se indica que hay dos entidades que son persona y cuenta, con los

atributos descritos anteriormente para cada una. El diagrama también representa la relación titular entre persona y cuenta.

Se debe tener en cuenta que el modelo relacional convierte las entidades en tablas, por lo tanto, se puede poner como ejemplo que las entidades personas se conviertan en tablas, como muestra la siguiente figura.

Figura 3. Tablas de una entidad relación

id_persona	país	identificación	correo	nombres	apellidos	edad
1	Colombia	8.127.346	ana@rmail.com	Ana lis	Méndez	34
2	Colombia	212.394.762	jose@pmail.com	José Omar	Ciano	28
3	Colombia	545.232.546	luis@jmail.com	Luis Darío	Márquez	21
4	Colombia	125.446.732	arm@hmail.com	Armando	Bronca Segura	35
5	Colombia	33.565.236.787	victor@umail.com	Vistor José	Diento Paz	24
6	Colombia	2.343.554.768	josen@pmail.com	José Nicolás	Timado Ruiz	19
7	Perú	45.456.778	andrea@email.com	Andrea	Martinez	36

a) Tabla persona

número_cuenta	títular	saldo	fecha_creacion
19.283.746	7	\$ 2.589.954,00	25/08/98
67.789.901	1	\$ 23.465.212,00	26/08/01
32.112.312	5	\$ 2.343.242,00	27/08/04
41.928.374	2	\$ 2.343.423,00	31/08/13
38.342.657	6	\$ 8.965.211,00	30/08/10
32.443.171	1	\$ 11.172.594,33	22/06/01
34.920.608	5	\$ 14.483.578,83	1/09/16
17.980.455	3	\$ 17.794.563,33	3/09/19
26.754.825	4	\$ 21.105.547,83	12/04/22

b) Tabla cuenta

Las tablas de la figura anterior muestran cada uno de los conjuntos de registros, la “tabla persona” muestra 6 registros y la “tabla cuenta” muestra 9 de ellas; cada uno

de estos registros representa un objeto, es decir, de la tabla persona hay 6 (objetos) o personas. A la agrupación de los datos de cada registro se le llama TUPLA y, en términos sencillos, una TUPLA de una base de datos, es el conjunto de todos los atributos (columnas) de una fila.

2.1. Relaciones entre entidades

Como se representa en las figuras anteriores, una entidad se puede representar con una tabla, sin embargo, la relación titular aún no se sabe cómo representarla y para ello se debe definir algunas propiedades de los atributos.

Principio de unicidad

En una tabla nunca deben existir TUPLAS repetidas, es decir, no debe haber más de una fila con exactamente los mismos valores en cada columna que otra fila de la misma tabla. En otras palabras, cada fila debe diferir al menos en un elemento (columna) en relación con las demás.

En una base de datos relacional, las relaciones permiten evitar datos redundantes. En el ejemplo anterior, una persona puede tener varias cuentas bancarias, pero no es deseable que, por cada una de las cuentas bancarias asociadas a la misma persona, se deban repetir los datos de la misma persona, sino más bien, se debe encontrar un mecanismo para relacionar la información de las dos tablas y para ello se introducen algunos conceptos útiles.

Claves

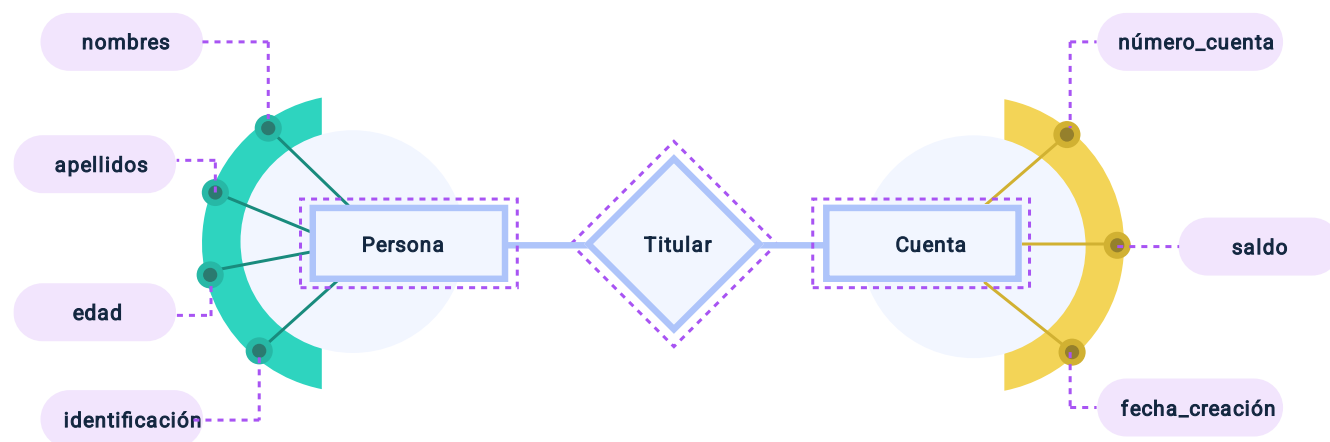
Dado que en una tabla no hay filas repetidas, se pueden distinguir unas de otras, es decir, se pueden identificar de manera única (por el principio de unidad). Por lo

tanto, la manera de identificarlas inequívocamente es según los valores de los atributos. De esta forma, se conoce como claves a los atributos que sirven para identificar un registro de forma inequívoca.

Ejemplo

En la tabla persona de la figura 3, es sabido por todos que no existen dos personas con el mismo número de documento de identidad, por lo tanto, se puede emplear el atributo identificación como clave de la tabla persona. Así mismo, no deben existir dos cuentas bancarias con el mismo número de cuenta, entonces, el atributo numero_cuenta se puede emplear como clave de la tabla cuenta. Esto se puede presentar en un diagrama de entidad relación cuando a los atributos que componen una clave, se les marca con subrayado el nombre del atributo, tal como se muestra en la siguiente figura.

Figura 4. Claves de las entidades persona y cuenta



Nótese como los atributos identificación y numero_cuenta tiene subrayada la palabra identificadora del atributo, representado así que ese atributo es la clave (que hace que se cumpla el criterio de unicidad) de su entidad.

Claves candidata, superclave y clave primaria

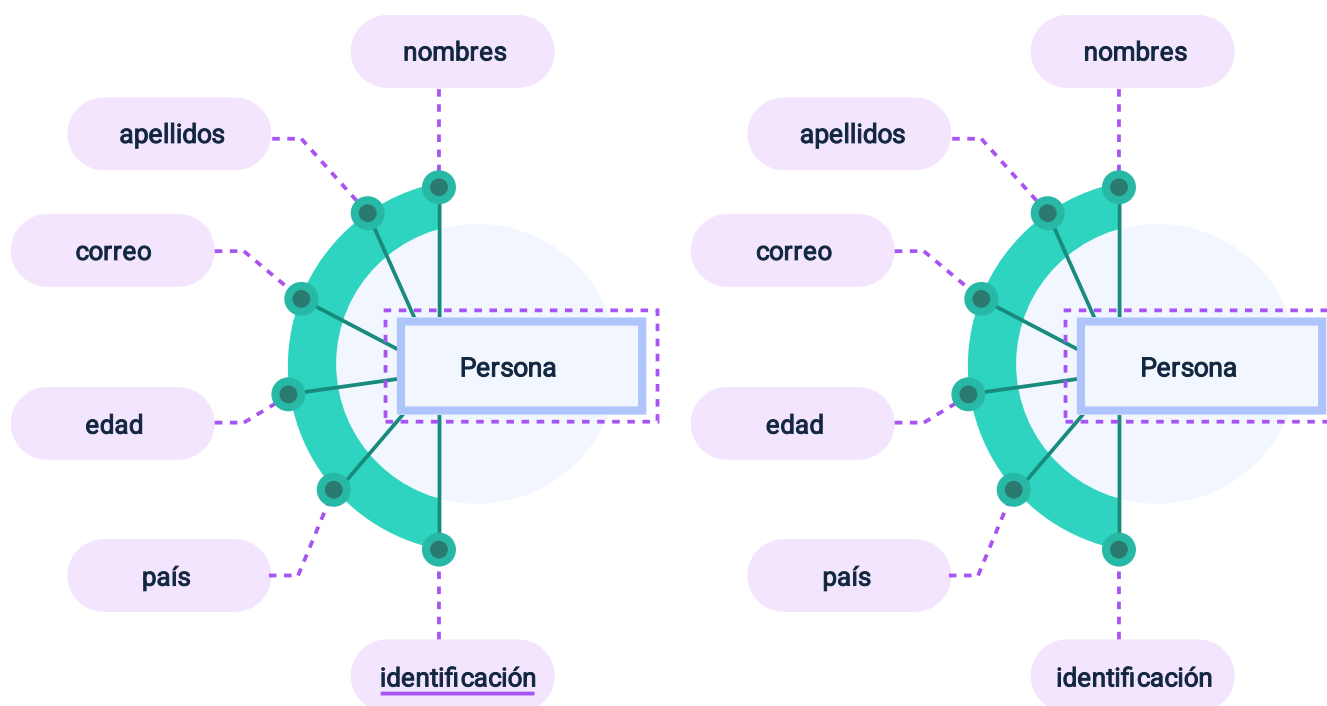
Supongamos el caso que el banco es internacional, que es un banco moderno y solo diseñado para transacciones vía internet o virtuales, por lo tanto, el correo del cliente (persona) es necesario y obligatorio.

En este caso, ocurren un par de asuntos a analizar: al ser internacional el banco, puede existir un cliente en un país con el mismo número de identificación de otro cliente en otro país, por lo tanto, el atributo identificación ya no puede ser una clave de la entidad, sin embargo, es importante y obligatorio saber a qué país corresponde cada cliente, el conjunto de datos identificación y país y si ambos cumplen con el criterio de unidad, porque no existen dos personas en el mismo país con el mismo número de identificación.

Clave compuesta: si se escogiera el conjunto de atributos, identificación y país como clave de la entidad, se está haciendo una clave compuesta, porque es un conjunto de más de un atributo el que se usa para este fin.

Clave compuesta: el otro asunto a considerar es que se puede emplear el correo electrónico como clave, porque no existen dos usuarios con el mismo correo electrónico, esto es válido si se definió como regla del banco que cada cliente tenga su correo electrónico diferente de los demás. Estos dos casos se representan en la siguiente figura.

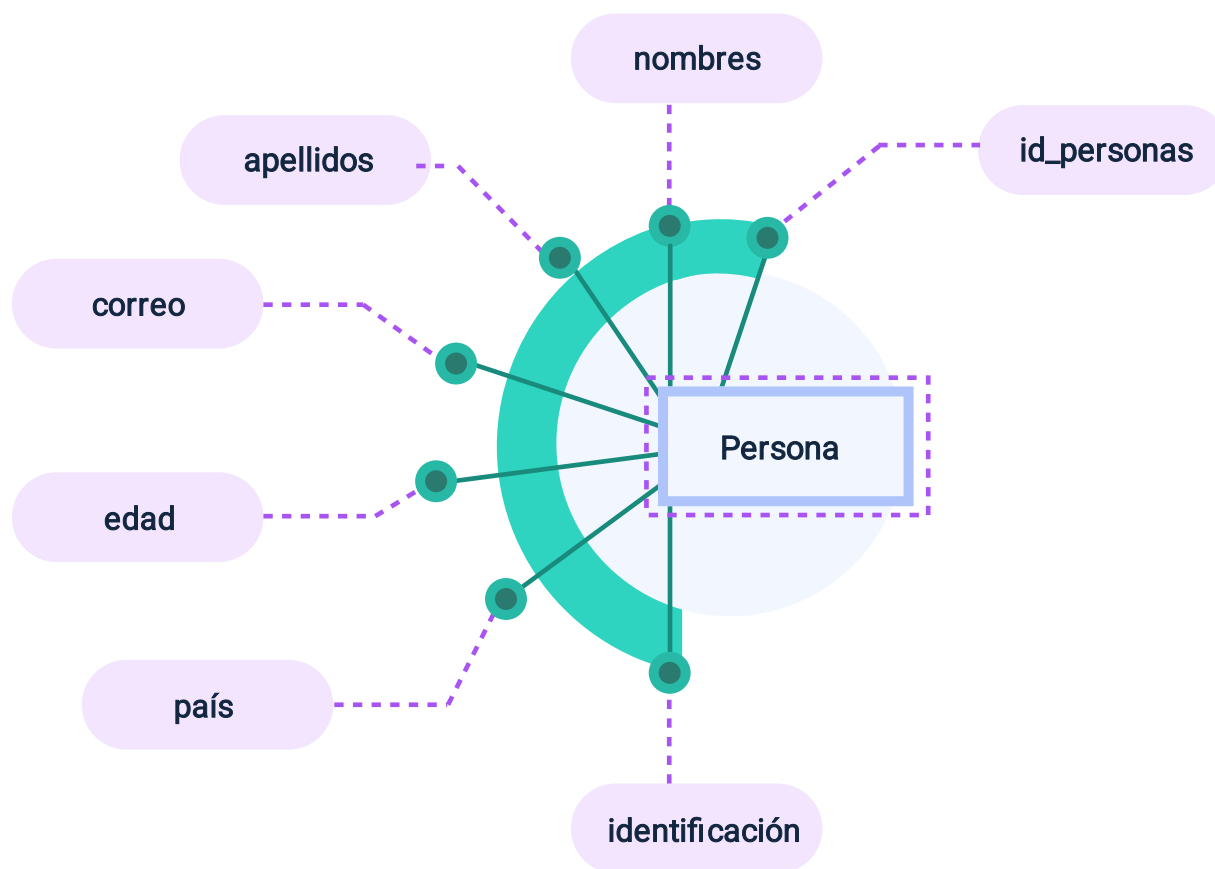
Figura 5. Claves candidata



Clave candidata: los dos caminos propuestos en la figura anterior son válidos, y se denominan claves candidatas porque con ambas se puede lograr el principio de unicidad, y es potestad de cada diseñador o analista de sistemas, determinar cuál de las dos emplear.

Sin embargo, existe otro tipo de solución y es asignarle un atributo identificador a la entidad persona que se encargue exclusivamente de identificar inequívocamente cada tupla o fila, a este atributo se le puede llamar `id_persona` que sería un número que el programa o software para el que se está desarrollando la base, le asignará a cada cliente cada vez que se registren por primera vez, los datos de una persona (generalmente una secuencia). De esta forma, se tendrían 3 claves candidatas; la siguiente figura presenta cómo sería la entidad y la tabla que representa.

Figura 6. Claves candidatas secuencia



Este último enfoque es el más empleado y recomendado por los diseñadores o analistas de software, pues reduce, significativamente, la complejidad del diseño y facilita la relación con otras tablas o entidades. Por lo tanto, se usará cada vez que se tenga oportunidad de hacerlo.

- **Superclave**

Es un conjunto de uno o más atributos que, tomados juntos, permiten identificar de forma única una entidad en el conjunto de entidades. Por ejemplo, el atributo `id_persona` del conjunto de entidades persona, es suficiente para distinguir una persona cliente del banco de las otras. Así, `id_persona` es una superclave. Análogamente, la combinación de

identificación y país es una superclave del conjunto de entidades persona.

El atributo correo de persona es otra superclave, pero el atributo nombres no lo es, porque varias personas podrían tener el mismo nombre.

- **Clave primaria**

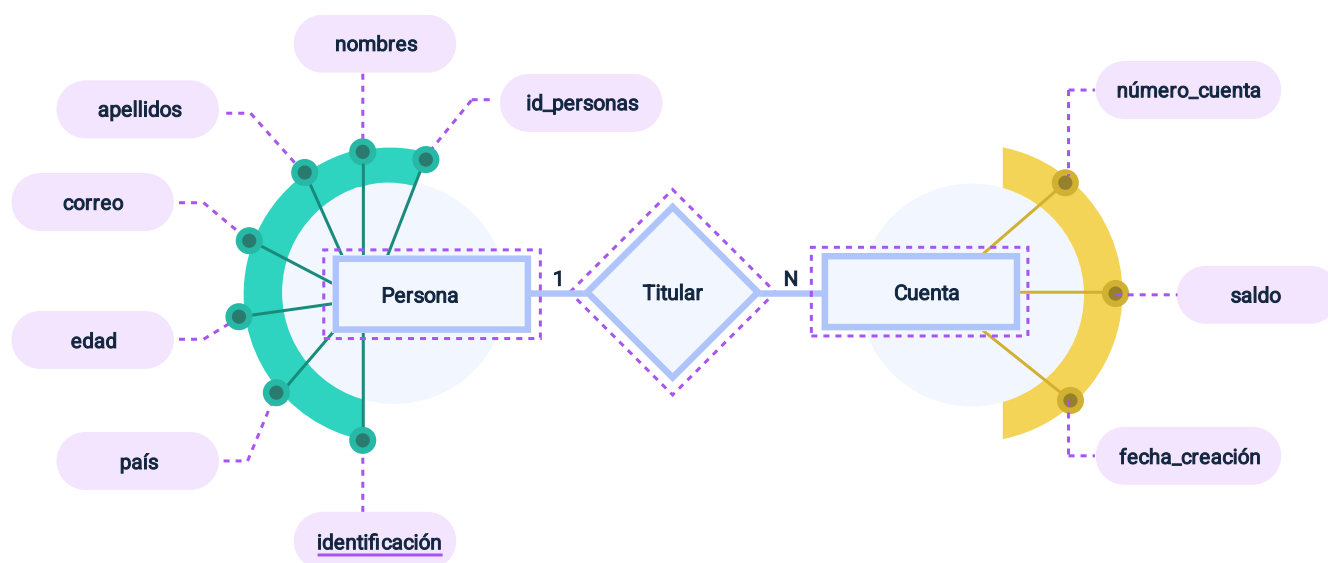
Se emplea el término clave primaria para nombrar una clave candidata que es elegida por el diseñador o analista de la base de datos como clave principal para identificar las tuplas dentro de una entidad. Una clave (primaria, candidata y superclave) es una propiedad del conjunto de entidades, más que de las entidades individuales. Dos tuplas individuales en el conjunto, no pueden tener el mismo valor en sus atributos clave primaria al mismo tiempo. La designación de una clave primaria es en sí una restricción que modela una característica del mundo real.

2.2. Relaciones de uno a muchos

Una relación de uno a varios o de uno a muchos, es el tipo de relación más empleada. En este tipo de relación, una fila de la tabla A puede tener muchas filas coincidentes en la tabla B. Pero una fila de la tabla B solo puede tener una fila coincidente en la tabla A. Por ejemplo, las tablas persona y cuenta tienen una relación de uno a muchos. Es decir, cada persona puede tener más de una cuenta, pero, por ejemplo, cada título proviene de una sola editorial.

En un diagrama de entidad relación esto se representa con lo que se denomina multiplicidad, como se presenta en la siguiente figura.

Figura 7. Relaciones de uno a muchos



A esta representación se le denomina multiplicidad 1:N y se lee de la siguiente forma: UNA persona es titular de N cuentas bancarias, de esta forma una persona puede tener N (que puede ser cero, uno o más de una) cuentas bancarias. Se puede leer en forma inversa que el resultado es el mismo: muchas o varias cuentas bancarias pueden tener un mismo titular.

Cuando una relación es de uno a muchos y se mapea un modelo de base de datos relacional, la relación titular (rombo) se termina convirtiendo una columna de la tabla de los muchos.

Figura 8. Tablas de un modelo relacional 1:N

id_persona	país	identificación	correo	nombres	apellidos	edad
→1	Colombia	8.127.346	ana@rmail.com	Ana lis	Méndez	34
→2	Colombia	212.394.762	jose@pmail.com	José Omar	Ciano	28
3	Colombia	545.232.546	luis@jmail.com	Luis Darío	Márquez	21
4	Colombia	125.446.732	arm@hmail.com	Armando	Bronca Segura	35
5	Colombia	33.565.236.787	victor@umail.com	Vistor José	Diento Paz	24
6	Colombia	2.343.554.768	josen@pmail.com	José Nicolás	Timado Ruiz	19
7	Perú	45.456.778	andrea@email.com	Andrea	Martinez	36

a) Tabla persona

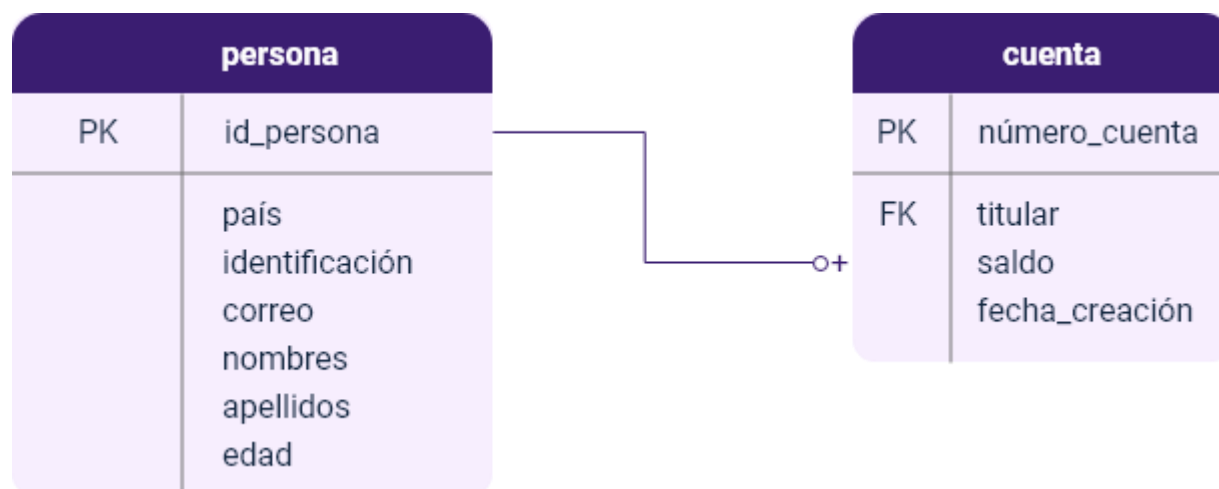
número_cuenta	títular	saldo	fecha_creacion
19.283.746	7	\$ 2.589.954,00	25/08/98
67.789.901	→1	\$ 23.465.212,00	26/08/01
32.112.312	5	\$ 2.343.242,00	27/08/04
41.928.374	→2	\$ 2.343.423,00	31/08/13
38.342.657	6	\$ 8.965.211,00	30/08/10
32.443.171	→1	\$ 11.172.594,33	22/06/01
34.920.608	5	\$ 14.483.578,83	1/09/16
17.980.455	3	\$ 17.794.563,33	3/09/19
26.754.825	4	\$ 21.105.547,83	12/04/22

b) Tabla cuenta

Como se puede comprender en la figura anterior, Ana Lis Méndez, es titular de dos cuentas bancarias (67.789.901 y 32.443.171), y todos los datos de la columna titular en la tabla cuenta se conoce como LLAVE FORÁNEA de la tabla persona.

REGLA DE MAPEO 1:N de Modelo entidad-relación a modelo relacional: una relación de uno a muchos (1:N), se transforma en una columna en la tabla que tiene la multiplicidad de los muchos (cuenta). Y todos los datos de esta columna existen como llave primaria en la tabla que tiene la multiplicidad uno (tabla persona).

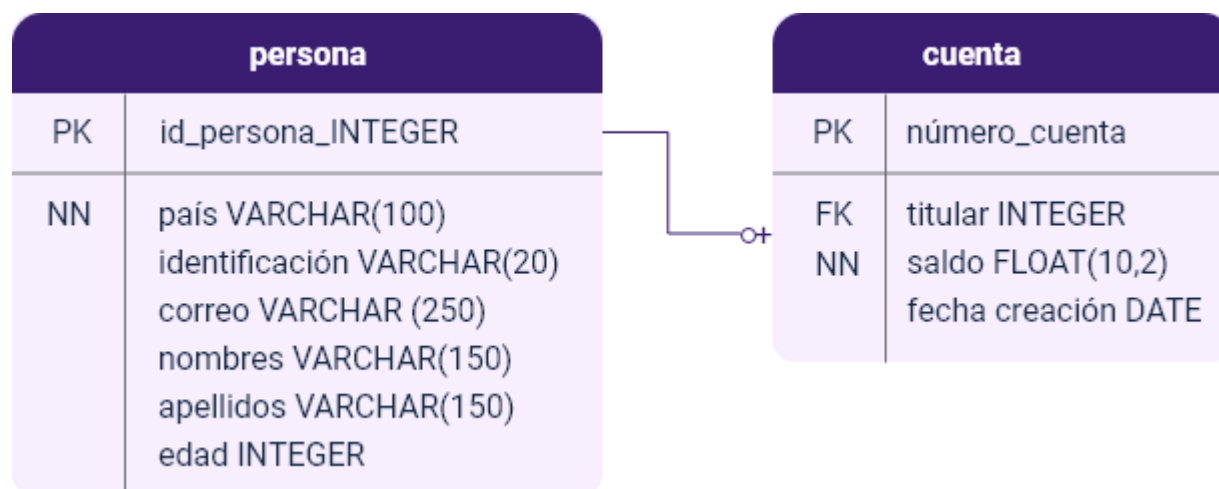
Figura 9. Diagrama relacional 1:N



En la figura se indica el nombre de las entidades (**persona** y **cuenta**), los atributos haciendo especial énfasis en los que representan la llave primaria con el indicador PK (del inglés Primary Key), y también se representa la relación titular (columna de la tabla), indicando que es una clave o llave foránea de la tabla **persona**, lo que se indica con el indicador FK (del inglés Foreign Key).

Algunos diagramas relacionales agregan metadatos para hacer más claro el modelo al diseñador, así como la naturaleza de los datos del modelo.

Figura 10. Diagrama relacional 1:N con metadatos



Conclusión: la relación de uno a muchos se termina convirtiendo en una columna en la tabla que tiene la multiplicidad de los muchos.

2.3. Relación de muchos a muchos

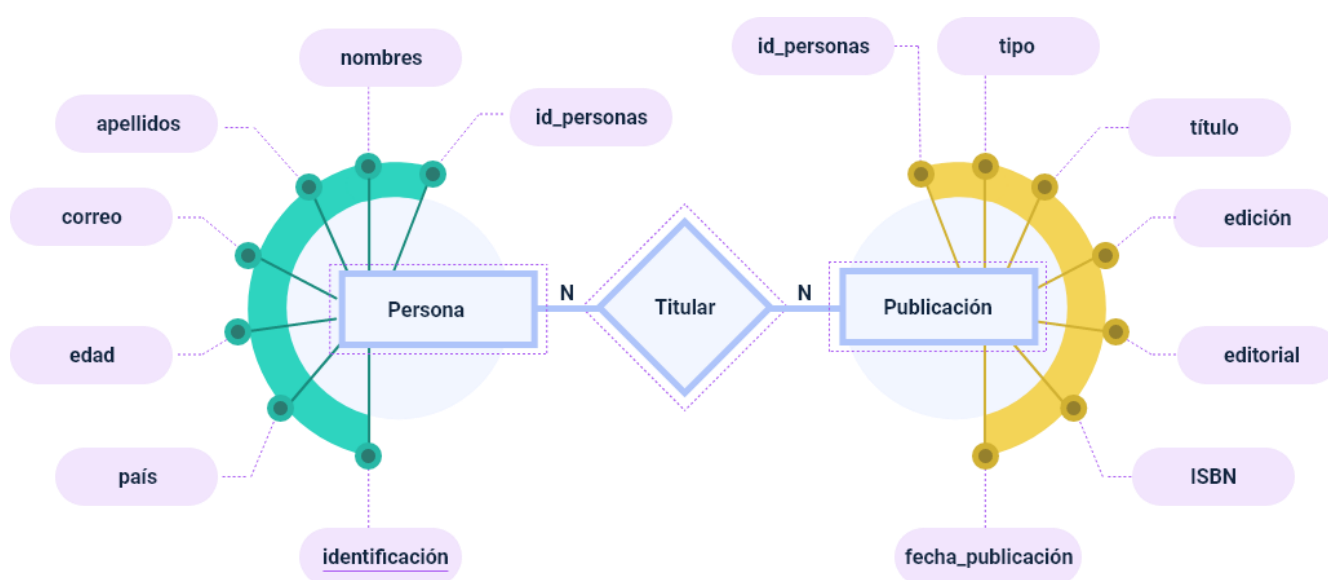
En una relación de muchos a muchos, una fila de la tabla A puede tener relación con muchas filas en la tabla B y también, en sentido contrario, una fila de la tabla B con respecto de varias filas de la tabla A. Esta relación se crea definiendo una tercera tabla denominada tabla de relación. Esta nueva tabla tiene una clave principal de la tabla compuesta de las claves foráneas de la tabla A y de la B.

Ejemplo

Se necesita almacenar la información de unas referencias bibliográficas o publicaciones de libros, tesis, artículos, que escriben los estudiantes de una universidad, y como se sabe, un libro, tesis, artículo o cualquier publicación puede ser elaborada por más de una persona, y a su vez, una persona (docente, estudiante o investigador) puede tener más de una publicación de su autoría.

Como se representa en la siguiente figura, la entidad publicación tiene una llave primaria llamada identificación, y existe una relación llamada autor, y representa que una persona puede ser autor de una o muchas publicaciones, y que una publicación puede ser de autoría de una o muchas personas, la multiplicidad en este caso es de muchos a muchos y se representa como N:N.

Figura 11. Diagrama de entidad de publicaciones bibliográficas



REGLA DE MAPEO N:N de modelo entidad-relación a modelo relacional: en una relación de muchos a muchos (N:N) se transforma en una tabla, cuya llave primaria está compuesta por las llaves primarias de las otras dos tablas.

Como se ilustra en la siguiente figura, la “tabla autor”, dice que la persona Ana Lis, es autora de los libros de título Grandes hazañas y Pellentesque, también que el libro Pellentesque es escrito además de Ana Lis por Luis Darío y Jose Nicolás. Lo anterior haciendo referencia a las llaves primarias de las tablas a) persona y b) publicación.

Figura 12. Tablas de relación N:N

id_persona	país	identificación	correo	nombres	apellidos	edad
→ 1	Colombia	8.127.346	ana@mail.com	Ana lis	Méndez	34
2	Colombia	212.394.762	jose@mail.com	José Omar	Ciano	28
3	Colombia	545.232.546	luis@mail.com	Luis Darío	Márquez	21
4	Colombia	125.446.732	arm@mail.com	Armando	Bronca Segura	35
5	Colombia	33.565.236.787	victor@mail.com	Vistor José	Diento Paz	24
6	Colombia	2.343.554.768	josen@mail.com	José Nicolás	Timado Ruiz	19
7	Perú	45.456.778	andrea@email.com	Andrea	Martinez	36

a) Tabla persona

id_persona	id_publicación
→ 1	1
→ 1	→ 5
6	→ 5
3	5
6	2

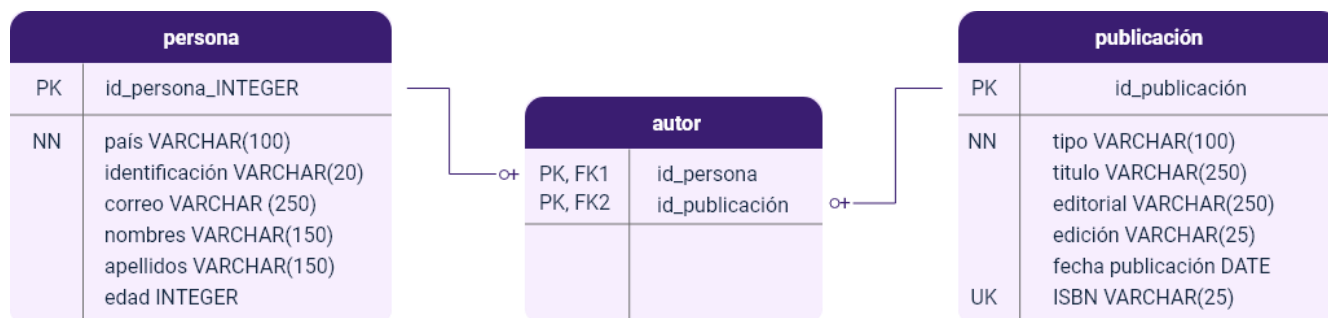
c) Tabla autor

id_publicacion	tipo	titulo	editorial	edición	fecha_publicacion
1	libro	Grandes Hazañas	Universita	6ta	25/08/98
2	tesis	Mediciones en me	Denver	-	26/08/01
3	libro	Consectetur Sem	ISFR	1ta	27/08/04
4	artículo	Consectetur	Sodales	-	31/08/13
→ 5	libro	Pellentesque	Conubia	3ta	30/08/10
6	revista	Praesent	Nascetur	-	22/06/01
7	revista	Fermentum	aptent	-	1/09/16
8	libro	El ruido y la furia	Vulputate	1ta	3/09/19
9	libro	Tempus mauris	Effcitur	5ta	12/04/22

b) Tabla publicación

La representación de la relación N:N con un diagrama relacional se muestra en la figura a continuación, donde se usan los prefijos FK1 y FK2, para indicar llave foránea uno y dos, para las entidades persona y publicación 2, respectivamente. También se indica el prefijo UK (Unique Key) en la tabla publicación, en la columna ISBN, el cual significa que en toda la tabla publicación no puede existir más de una fila con el mismo ISBN, es decir, nunca ninguna publicación tendrá el mismo ISBN que otra.

Figura 13. Diagrama relacional N:N



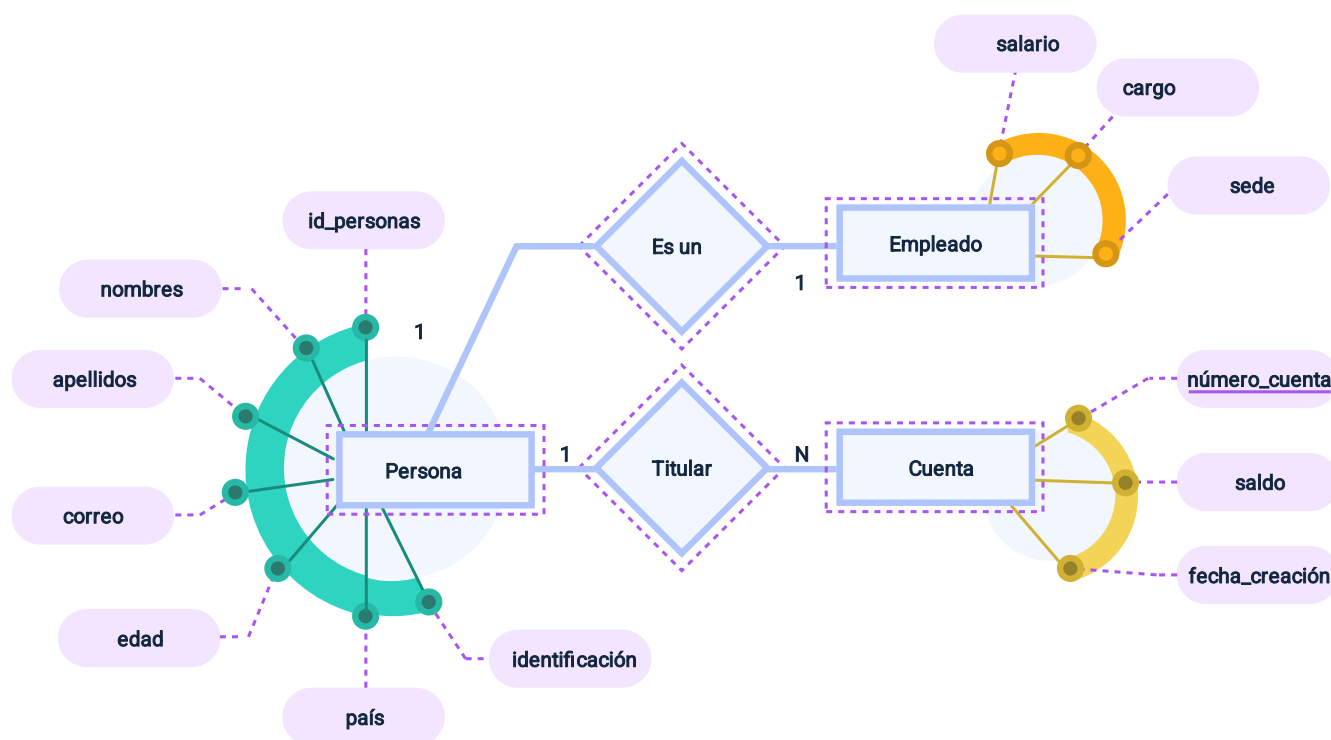
2.4. Relaciones de uno a uno

En una relación uno a uno, una fila de la tabla A solo puede tener una fila coincidente en la tabla B y viceversa. Se crea, entonces, una relación uno a uno si ambas columnas relacionadas son claves principales o tienen restricciones únicas. Este tipo de relación no es común, porque la mayoría de la información que está relacionada de esta manera estaría en una tabla. Se puede usar una relación uno a uno por alguno de estos motivos:

- Dividir una tabla con muchas columnas.
- Aislar parte de una tabla por motivos de seguridad.
- Almacenar datos de corta duración y que podrían eliminarse fácilmente, borrando la tabla periódicamente.
- Almacenar información que solo se aplique a un subconjunto de la tabla principal.

Volviendo al ejemplo del banco, esta vez se supone que la empresa necesita información sobre sus empleados lo que, a la vez, pueden ser clientes del banco.

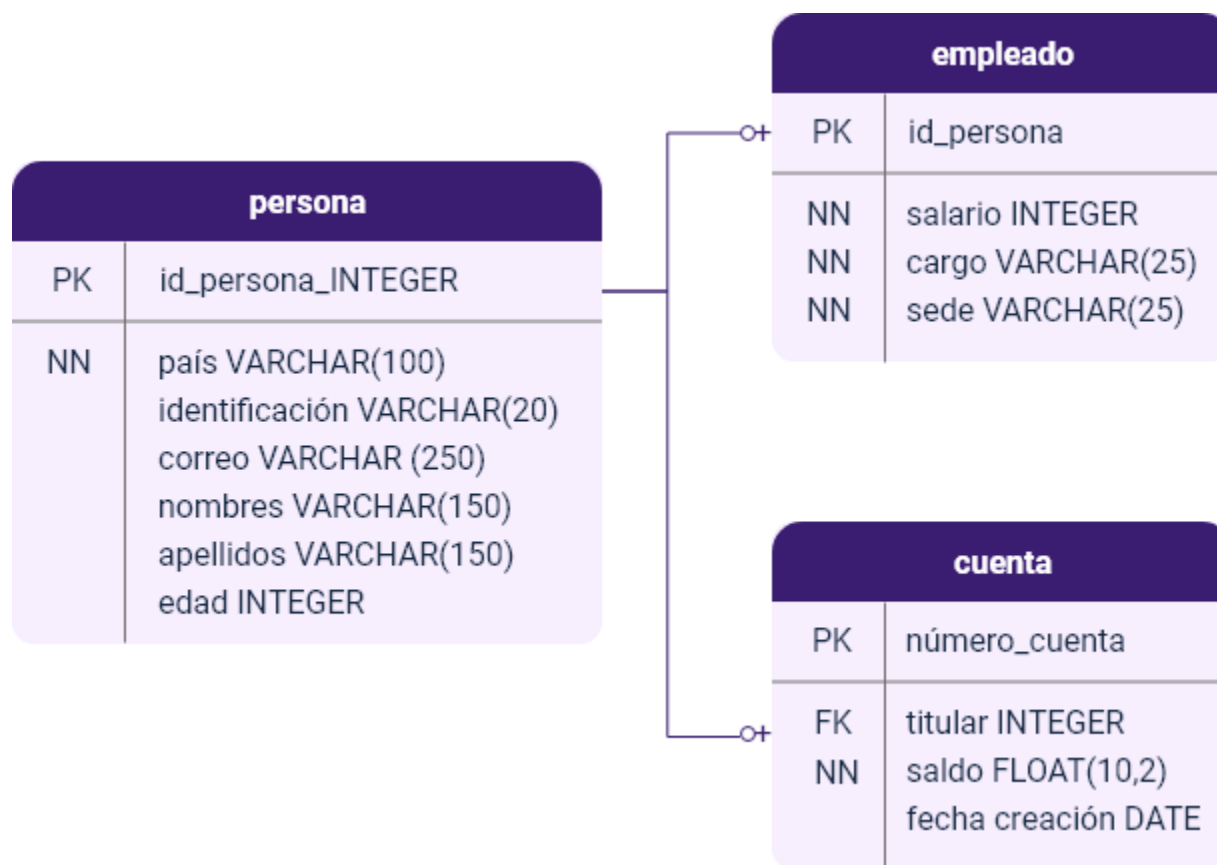
Figura 14. Diagrama entidad relación persona-empleado



REGLA DE MAPEO 1:1 de modelo entidad-relación a modelo relacional: en el momento de convertir la relación o con multiplicidad 1:1 a tablas, la relación uno a uno se convierte en una columna de la tabla empleado con restricción de unicidad (Unique Key o Primary Key), como se muestra en el siguiente diagrama relacional.

Como se puede inferir de la siguiente figura, se crea una tabla donde se relacionan los datos de las personas que son empleados del banco, a través de una columna que se llama *id_persona*, pero es común entre algunos diseñadores ponerle el nombre que los relacione con la tabla a la que pertenece, es decir, *id_empleado*, pero esto queda al criterio del diseñador.

Figura 15. Diagrama relacional 1:1



3. Normalización

La normalización es el procedimiento mediante el cual se aplican las reglas de mapeo o conversión de un modelo entidad-relación a un modelo relacional, es decir, a tablas y relaciones. Los principios rectores de este proceso son: no redundancia de datos (que no se repitan los datos) y que la dependencia de datos sea coherente (una separación lógica de datos en tablas).

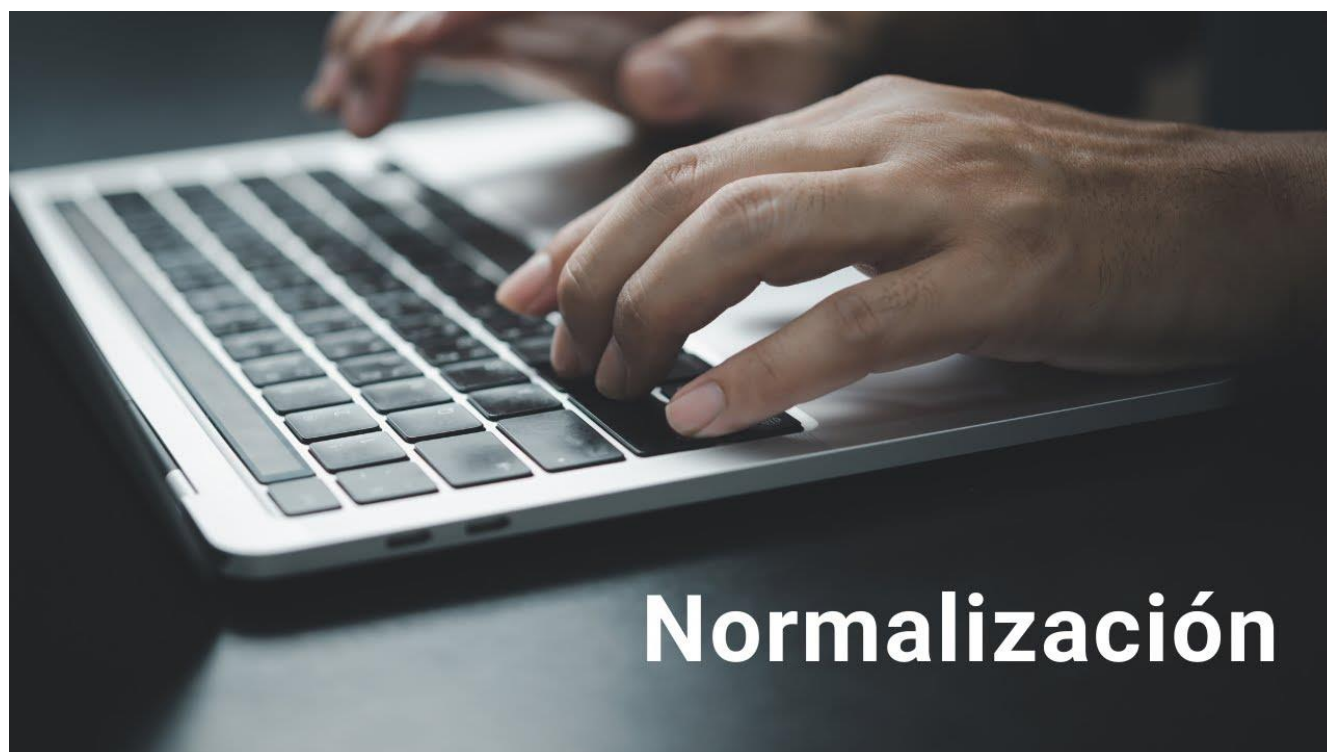
Los datos redundantes desperdician espacio en disco y crean problemas de mantenimiento. Por ejemplo, en la figura 15 del tema anterior, se ve que los datos comunes de los clientes y los bancos están en la tabla persona, mientras que los datos que pertenecen exclusivamente a los empleados están en otra tabla que se relaciona con la de personas.

Ahora, ¿qué es una dependencia incoherente?

Aunque resulta intuitivo para un usuario buscar en la tabla de persona el correo de un cliente en particular, es posible que no tenga sentido buscar en esa columna el salario del empleado que se relaciona con esos datos de la tabla persona, porque el salario del empleado está relacionado con la tabla empleado o depende de él. Las dependencias incoherentes pueden dificultar el acceso a los datos, porque la ruta para encontrar los datos puede faltar o estar dañada.

A continuación, lo invitamos a conciderar el siguiente video sobre normalización.

Video 1. Normalización



Enlace de reproducción del video

Síntesis del video: Normalización

La normalización, es el procedimiento mediante el cual se aplican las reglas de mapeo o conversión de un modelo entidad relación a un modelo relacional es decir a tablas y relaciones, los principios rectores de este proceso son: no redundancia de datos, que no se repitan los datos y la dependencia de los datos, dependencia incoherente, una separación lógica de datos en tablas.

Los datos redundantes desperdician espacio en disco y crean problemas de mantenimiento, por ejemplo, los datos comunes de los clientes y los bancos están en la tabla persona, mientras que los datos que pertenecen exclusivamente a los empleados están en otra tabla que se relaciona con la de personas.

¿Qué es una dependencia incoherente? Aunque resulte intuitivo para un usuario buscar en la tabla de personas el correo de un cliente en particular, es posible que no tenga sentido buscar en esa columna el salario del empleado que se relaciona con esos datos de la tabla persona, pues el salario del empleado está relacionado con la tabla empleados.

Las dependencias incoherentes pueden dificultar el acceso a los datos, ya que la ruta para encontrar los datos puede faltar o estar dañada.

La siguiente tabla nos presenta las tres formas normales y sus principios.

Tabla 4. Formas normales

Forma normal	Principios
Primera forma normal	Eliminar grupos de repetición en tablas individuales. Crear una tabla independiente para cada conjunto de datos relacionados. Identificar cada conjunto de datos relacionados con una clave primaria.
Segunda forma normal	Crear tablas independientes para conjuntos de valores que se aplican a varios registros. Relacionar estas tablas con una clave foránea.
Tercera forma normal	Eliminar los campos que no dependen de la clave.

Hay una cuarta forma normal, también llamada forma normal de Boyce Codd (BCNF), y una quinta forma normal, pero rara vez se consideran en un diseño práctico, solo agregan complejidad al sistema y no aportan un valor funcional que aporte a la solución del problema, por esta razón no será tratada.

3.1. Dependencias funcionales

Una dependencia funcional es un tipo de restricción que termina construyendo una generalización del concepto de clave, como se estudió en el modelo E-R y en el modelo relacional. Pero no es tan fácil localizar las dependencias, debido a que necesitan de un análisis de los atributos (columnas) o, con más precisión, de las interrelaciones entre atributos y, frecuentemente, la intuición no es suficiente a la hora de encontrar y clasificar todas las dependencias. Aunque existe una teoría matemática para realizar este análisis, un ejemplo sencillo puede enseñar cómo analizar estas dependencias de manera intuitiva, tal como se presentará a continuación.

Ejemplo

Un cliente pide que se desarrolle un software para llevar el inventario de una ferretería, los productos, los proveedores y el precio al que cada proveedor vende cada producto, para lo cual suministra la siguiente información, y se deben identificar las dependencias funcionales para poder realizar un modelo relacional.

Figura 16. Proveedores, productos y precios

nit_proveedor	correo	nombres	telefono	código_producto	producto	precio
8127346	ana@rmail.com	Ana Lis Mendez	398589854	MS-028	Masillas FFC	\$ 25.890,00
212394762	jose@pmail.com	Jose Omar	5475698	PS-012	Pistola de silicona	\$ 14.300,00
545232546	luis@jmail.com	Luis Dario	3984558	CT-003	Cutter RDS	\$ 8.700,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 820,00
212394762	jose@pmail.com	Jose Omar	5475698	MS-028	Masillas FFC	\$ 26.000,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 7.900,00
545232546	luis@jmail.com	Luis Dario	36584114	PS-012	Pistola de silicona	\$ 12.990,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 830,00
212394762	jose@pmail.com	Jose Omar	5475698	CT-003	Cutter RDS	\$ 9.200,00

El principio de no repetición de datos, indica que se deben identificar todos los datos que se repiten y se puede examinar en la siguiente figura, teniendo en cuenta los colores de sombreado como se muestra. Es de notar que el nit_proveedor es igual en todos los registros que tienen el mismo valor de correo, nombres y teléfono.

Figura 17. Dependencia de nit_proveedor con datos del proveedor

nit_proveedor	correo	nombres	telefono	código_producto	producto	precio
8127346	ana@rmail.com	Ana Lis Mendez	398589854	MS-028	Masillas FFC	\$ 25.890,00
212394762	jose@pmail.com	Jose Omar	5475698	PS-012	Pistola de silicona	\$ 14.300,00
545232546	luis@jmail.com	Luis Dario	3984558	CT-003	Cutter RDS	\$ 8.700,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 820,00
212394762	jose@pmail.com	Jose Omar	5475698	MS-028	Masillas FFC	\$ 26.000,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 7.900,00
545232546	luis@jmail.com	Luis Dario	36584114	PS-012	Pistola de silicona	\$ 12.990,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 830,00
212394762	jose@pmail.com	Jose Omar	5475698	CT-003	Cutter RDS	\$ 9.200,00

Ahora, en la siguiente figura, la columna código_producto es la misma para la columna producto, pero difiere del precio que cada proveedor ofrece del mismo.

Figura 18. Dependencia de código_producto_ con datos del producto

nit_proveedor	correo	nombres	telefono	código_producto	producto	precio
8127346	ana@rmail.com	Ana Lis Mendez	398589854	MS-028	Masillas FFC	\$ 25.890,00
212394762	jose@pmail.com	Jose Omar	5475698	PS-012	Pistola de silicona	\$ 14.300,00
545232546	luis@jmail.com	Luis Dario	3984558	CT-003	Cutter RDS	\$ 8.700,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 820,00
212394762	jose@pmail.com	Jose Omar	5475698	MS-028	Masillas FFC	\$ 26.000,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 7.900,00
545232546	luis@jmail.com	Luis Dario	36584114	PS-012	Pistola de silicona	\$ 12.990,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 830,00
212394762	jose@pmail.com	Jose Omar	5475698	CT-003	Cutter RDS	\$ 9.200,00

La dependencia código_producto no se cumple para el precio del producto, por lo tanto, dice que para cada proveedor y producto, debe existir un precio distinto, como se presenta en la siguiente figura.

Figura 19. Dependencia de nit_proveedor, código_producto_ con precio del producto

nit_proveedor	correo	nombres	telefono	código_producto	producto	precio
8127346	ana@rmail.com	Ana Lis Mendez	398589854	MS-028	Masillas FFC	\$ 25.890,00
212394762	jose@pmail.com	Jose Omar	5475698	PS-012	Pistola de silicona	\$ 14.300,00
545232546	luis@jmail.com	Luis Dario	3984558	CT-003	Cutter RDS	\$ 8.700,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 820,00
212394762	jose@pmail.com	Jose Omar	5475698	MS-028	Masillas FFC	\$ 26.000,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 7.900,00
545232546	luis@jmail.com	Luis Dario	36584114	PS-012	Pistola de silicona	\$ 12.990,00
8127346	ana@rmail.com	Ana Lis Mendez	398589854	LG-021	Lija C20	\$ 830,00
212394762	jose@pmail.com	Jose Omar	5475698	CT-003	Cutter RDS	\$ 9.200,00

En la siguiente tabla, se listan las dependencias detectadas.

Tabla 5. Dependencias funcionales

Atributo	Es dependencia funcional de:
nit_proveedor	Correo Nombres Teléfono

Atributo	Es dependencia funcional de:
codigo_producto	Producto
nit_proveedor, codigo_producto	Precio

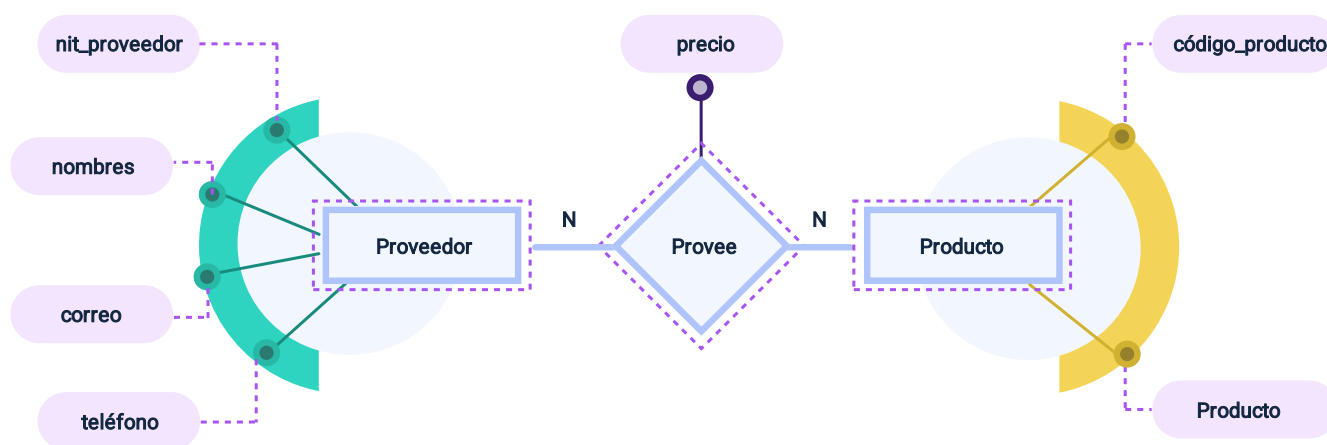
Referencia Tabla - Norma APA

Definición formal dependencia funcional

Se dice que un atributo X de una relación, “depende funcionalmente” de otro atributo o conjunto de atributos Y de la relación, si a todo valor (o valores del conjunto) Y le corresponde siempre el mismo valor de X.

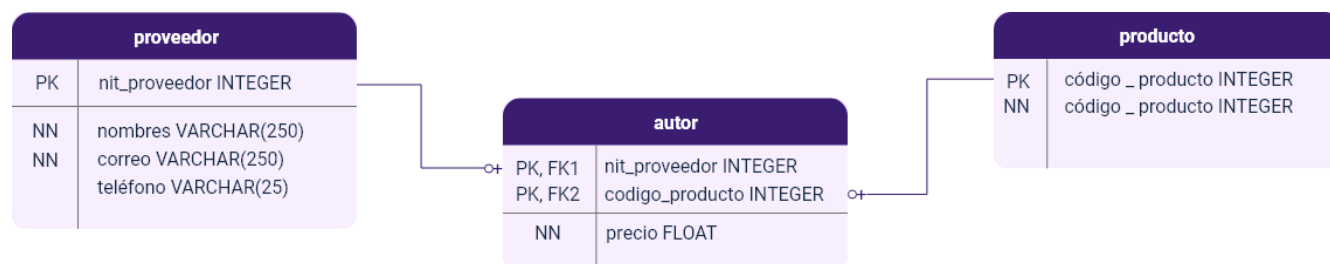
Las dependencias funcionales de la tabla anterior se pueden representar en modelo entidad relación, teniendo en cuenta que un proveedor puede suministrar muchos productos y, a su vez, un producto puede ser proveedor por más de un proveedor, y que por cada producto que es proveído por un proveedor, existe un precio.

Figura 20. Diagrama entidad relación dependencia funcional



Como se puede notar, en la figura anterior, la relación provee, cuando es mapeado a un diagrama relacional, se genera una tabla intermedia provee, que debe tener un atributo adicional “precio”.

Figura 21. Diagrama relacional dependencia funcional



3.2. Diseño relacional

Hasta este momento, se han examinado problemas concretos de las formas normales muy básicas y de la normalización. Ahora, se estudiará el modo en que se encaja la normalización en un proceso general de diseño de bases de datos y se puede hacer de una de las siguientes formas:

- **Opción 1**

Convirtiendo un diagrama entidad relación a un diagrama relacional y de esa forma, obtener las tablas.

- **Opción 2**

Puede tener una tabla con todos los datos que se procesan en una aplicación, e identificado las dependencias funcionales, de manera que nos permite identificar las tablas y las llaves o claves primarias de cada una.

- **Opción 3**

Se puede hacer un diseño ad hoc (para que cumpla el requerimiento) y se debe comprobar luego que satisface la forma normal deseada.

En el mundo práctico y en la medida que se tiene más experticia, las opciones 2 y 3 son las más empleadas.

A continuación, se muestra un ejemplo completo sobre cómo abordar un problema de almacenamiento de datos y transformarlo en un modelo relacional, recogiendo los conceptos vistos hasta ahora, pasando de un diagrama entidad relación a un modelo relacional, que es en todo caso el objetivo general de un diseño de base de datos.

Problema:

Se necesita gestionar una biblioteca escolar y nuestro cliente quiere tener ciertas herramientas a su disposición para controlar publicaciones tales como libros, revistas, folletos y también llevar un registro de los estudiantes y préstamos. Adicionalmente, se necesita un control de los ejemplares de cada libro o publicación, su ubicación y su estado, registrar la retirada de libros por préstamo y restitución de ejemplares, para esto necesita información sobre editoriales datos como dirección, teléfono y nombre de la editorial.

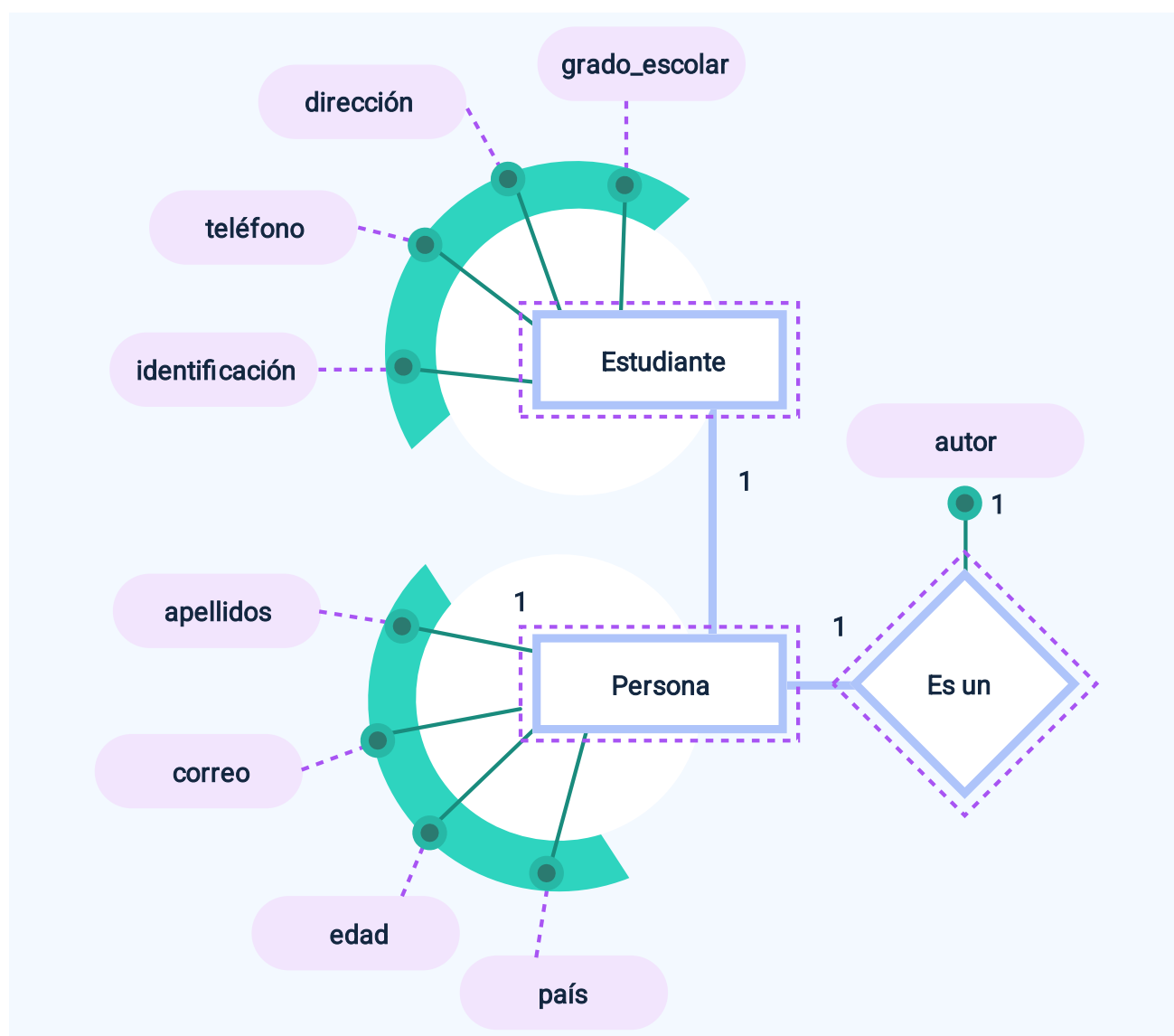
De cada publicación se pueden tener varios ejemplares, por ejemplo: del libro Nacho lee, la biblioteca tiene 5 ejemplares, también de cada publicación se debe conocer, su autor o autores, el tema que trata la publicación (geografía, física, matemática, etc.) y la editorial. Cada libro tiene un título, idioma, formato (digital web, impreso, digital CD), código ISBN y fecha de publicación.

Cada ejemplar tiene un código de barras, cuando un ejemplar ya está muy descargado se debe poder dar de baja; es decir, que ya se puede botar y no usarlo más.

Tanto el estudiante, como el autor de un libro tienen los siguientes datos: nombres, apellidos, correo. Adicional, un estudiante tiene los datos de documento de identidad, teléfono, dirección, grado escolar.

De cada préstamo interesa saber qué ejemplar se prestó, fecha en la que se prestó, la fecha en que se devolvió y alguna nota o comentario.

Figura 22. Análisis de relacionar autor



En este diagrama se presenta una posible solución:

La relación entre persona y estudiante corresponde al caso estudiado en las relaciones de uno a uno (entre persona y empleado del banco), de la figura Diagrama relacional 1:1.

El tema se vuelve complejo, porque se debe analizar si aplica lo mismo para autor.

Al mapear estas entidades a diagrama relacional, resultan 3 tablas (estudiante, persona y autor), pero autor solo tendría una única columna y al no tener atributos adicionales relacionados con el autor, es prácticamente lo mismo que tener la tabla persona, por lo tanto, no es conveniente dejarlo de esta forma y más bien relacionar la publicación directamente con la entidad persona.

Se debe estudiar detenidamente el diagrama entidad relación por varios minutos, asegúrese que comprende el porqué de cada elemento y su multiplicidad.

Para recordar:

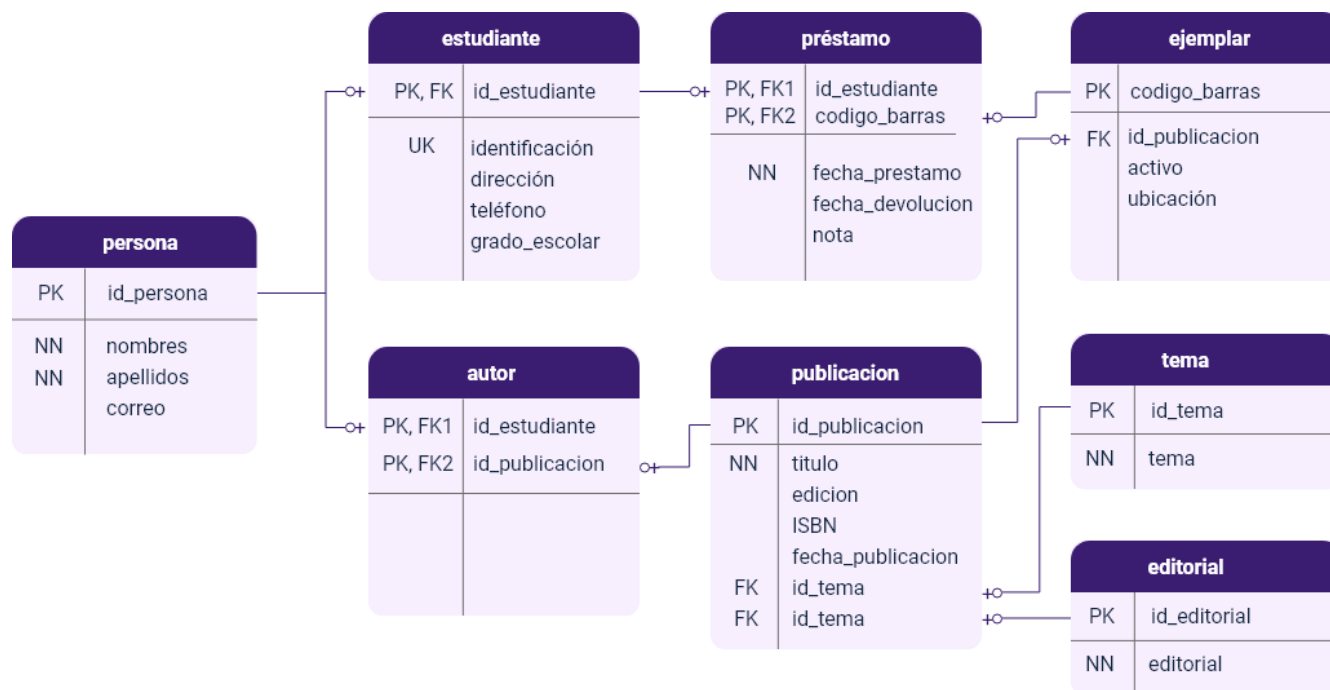
Una relación de muchos a muchos (N:N) se convierte en una tabla cuya llave primaria es computar por las llaves foráneas de las otras dos tablas. Una relación de uno a muchos (1:N) se convierte en una columna (llave foránea) de la tabla que tiene los muchos, y finalmente una relación de uno a uno (1:1) se convierte una columna en cualquier (llave foránea) de las dos tablas.

Modelo entidad biblioteca

Lo invitamos a consultar el PDF “**Modelo entidad relación biblioteca**”, el cual se encuentra en la carpeta Anexos, para conocer cómo queda el diagrama con el ejercicio trabajado.

A continuación, se presenta el modelo racional.

Figura 23. Modelo relacional



Hasta ahora se ha visto cómo distribuir el almacenamiento de los datos en tablas, de manera que no se permita la duplicidad de datos y procurando una estructura lógica, que presente la información del mundo real. Pero aún falta analizar el problema desde el punto de vista que permita garantizar que los datos que se van a insertar, son correctos, para lo cual es importante conocer las restricciones de integridad que puede tener una base de datos.

3.3. Reglas de integridad

Cuando se define un atributo, sin saber, se está escogiendo para él un dominio, es decir, un conjunto de valores que puede tener. Por ejemplo: en MySQL, cuando se elige un atributo de tipo entero (INT), los posibles valores que puede tener esa columna van desde -2147483648 hasta 2147483647, es decir, valores positivos o negativos, de

manera que si se quieren solo valores positivos, se deben definir como entero sin signo (UNSIGNED INT), por lo tanto, los valores van desde 0 hasta 4294967295.

La calidad de los datos garantiza que los datos almacenados en la base de datos cumplan con los estándares y requisitos de la organización. En otras palabras, asegura el mantenimiento de la integridad de los datos en una base de datos. Al hacerlo, aplica un conjunto de reglas a un conjunto de datos completo o específico y se almacena en la base de datos de destino, por lo consiguiente, para garantizar la integridad se debe tener en cuenta:

- **Criterio de NULIDAD**

Cuando en una fila un atributo (columna) es desconocido, se dice que es nulo. Un nulo no representa el valor cero (0) ni una cadena vacía (""), porque estos son valores que tienen un significado o valor. El nulo significa ausencia de información, bien porque al insertar la fila se desconocía el valor del atributo, o tal vez porque para dicha fila el atributo no tiene valor. Debido a que los nulos no son valores, deben tratarse de forma particular, lo que causa problemas de implementación. Actualmente casi todos los SGDB soportan valores nulos.

- **Integridad de entidad**

Ninguno de los atributos o columnas que componen una llave primaria debe ser NULO. Es decir, una clave primaria es irreducible para identificar de modo único una fila, el que sea irreducible significa que ningún subconjunto de la clave primaria sirve para identificar inequívocamente la fila: a la fila o tupla. Si se permitiera valor nulo en una columna que hace parte de la llave primaria, esta estaría contradiciendo la irreductibilidad de

una llave primaria; esto no aplica para claves alternativas, solo para la llave primaria.

- **Integridad referencia**

Para esto se debe analizar el modelo de la figura Modelo relacional y en particular, la relación entre las tablas persona y estudiante. Como se sabe, la tabla estudiante tiene una llave primaria que, a la vez, es una referencia de la tabla persona.

En la siguiente figura, por cada estudiante (tabla estudiante) existe uno y solo un elemento en la tabla persona, de tal forma que un estudiante es la conjunción de ambas tablas por medio de la llave foránea (que a la vez es llave primaria) id_estudiante, pero ¿qué tal que no existiera? Eso significa que la base de datos es inconsistente o que tiene datos basura en la tabla estudiante. Afortunadamente los SGDB permiten definir reglas que impidan que esto ocurra en las tablas, es decir, que no permita registrar un estudiante con id_estudiante con un valor que no existe en la columna id_persona de la tabla persona.

Figura 24. Error de integridad referencial

id_persona	correo	nombres	apellidos	id_estudiante	identificación	dirección	grado_escolar
1	ana@rmail.com	Ana lis	Méndez	3	12.345.565	Carrera 3 # 18-45	6
2	jose@pmail.com	José Omar	Ciano	5	12.490.345	Carrera 8a # 7-88	3
3	luis@jmail.com	Luis Dario	Márquez	1	13.767.234	Calle 15 # 9-56	5
4	arm@hmail.com	Armando	Bronca Segura	7	13.498.543	carrera 74a #85-95	7
5	victor@umail.com	Vistor José	Diento Paz	8	13.242.343	Calle 10 d # 25-85	8
6	josen@pmail.com	José Nicolás	Timado Ruiz	6	19.833.574	Calle 7 # 5-85	7
7	andrea@email.com	Andrea	Martinez				

a) Tabla persona

b) Tabla estudiante

Pregunta de análisis de datos

Ahora, hay que preguntarse lo siguiente: ¿qué pasaría si se borra el registro de la tabla persona que tiene id_persona con valor tres (3), sufrirá algún problema los datos

de la tabla estudiante?, o ¿qué pasaría si se cambia el valor de tres (3) en la columna id_persona, por otro valor como por ejemplo nueve (9)?, antes de continuar, hay que tomarse unos minutos para pensarlo.

Respuesta

La respuesta a las cuestiones planteadas es que en cualquiera de las operaciones de borrado o actualización del dato se incurrirá en una violación de la integridad referencial de la tabla estudiante, porque no habrá manera de conocer los datos complementarios del estudiante que se encuentran en la tabla persona.

Para evitar estos problemas de inconsistencia, los sistemas gestores de bases de datos permiten definir unas reglas cuando se borra o actualiza un dato que es referencia foránea de otra tabla.

Regla de borrado

Define el comportamiento a la pregunta: ¿qué ocurre si se intenta borrar la fila referenciada por la clave foránea de otra tabla? Para lo cual, el SGBD implementa una de las siguientes opciones:

- **Restringir**
No se permite borrar la fila referenciada.
- **Cascada**
Se borra la fila referenciada y también se borran las filas que la referencian, mediante la clave foránea en la otra tabla.
- **Poner null**
Se borra la fila referenciada y las filas que la referenciaban ponen en nulo la clave foránea (solo si acepta nulos).

- **Valor por defecto**

Se borra la fila referenciada y las filas que la referenciaban ponen en la clave foránea el valor por defecto que defina la regla.

Regla de edición

Define el comportamiento a la pregunta: ¿qué sucede si se intenta modificar el valor de la clave primaria de la fila referenciada por la clave foránea por otra tabla? Para lo cual, el SGBD implementa una de las siguientes opciones:

- **Restringir**

No se permite editar la fila referenciada.

- **Cascada**

Se actualiza la fila referenciada y también se actualiza las filas que la referencian mediante la clave foránea en la otra tabla.

- **Poner null**

Se actualiza la fila referenciada y las filas que la referenciaban ponen a nulo la clave foránea (solo si acepta nulos).

- **Valor por defecto**

Se edita la fila referenciada y las filas que la referenciaban ponen en la clave foránea el valor por defecto que defina la regla.

De esta forma, una vez definida la estructura del modelo de datos, se debe determinar el comportamiento de estas operaciones, lo cual generalmente responde a requerimiento del sistema de información.

3.4. Lenguajes de los sistemas administradores de bases de datos

Los sistemas gestores de bases de datos, emplean un lenguaje que se denomina SQL que corresponde al nombre en inglés (Structured Query Language). Generalmente, cuando un SGBD relacional implementa el lenguaje SQL, todas las acciones, y operaciones se llevan a cabo en el sistema, mediante sentencias de este lenguaje. Dentro de SQL hay varios tipos de sentencias que se agrupan en cuatro conjuntos.

Sentencias de definición de datos (DDL)

Son las sentencias que permiten crear tablas, alterar su definición y eliminarlas. En una base de datos relacional existen otros tipos de objetos además de las tablas, como las vistas, los índices y los disparadores. Las sentencias para crear, alterar y eliminar vistas e índices también pertenecen a este conjunto de sentencia y se les denomina DDL del inglés Data Definition Language.

Ejemplo DDL

Para crear la tabla persona de la figura Modelo relacional, quedaría de la siguiente forma:

```
CREATE TABLE persona (  
  
    id_persona INT NOT NULL,  
  
    nombres VARCHAR(250) NOT NULL,  
  
    apellidos VARCHAR(250) NOT NULL,  
  
    correo VARCHAR(250) NOT NULL,  
  
    PRIMARY KEY (id_persona)
```

);

Nota: cada atributo se define en una línea separada por una coma, se agregan restricciones de no nulidad (NOT NULL) al final la sentencia termina con punto y coma. Ahora, así se borraría esa misma tabla:

```
DROP TABLE persona;
```

Sentencias de manejo de datos (DML)

Estas sentencias que permiten insertar datos en las tablas, consultarlos, editarlos y borrarlos, se denomina DML del inglés Data Manipulation Language.

Ejemplo DML

A continuación, se muestra cómo se inserta una fila a la tabla persona, luego cómo se actualiza la fila insertada y posteriormente cómo se borra:

```
INSERT INTO persona (id_persona, nombres, apellidos, correo) VALUES (1, 'Ana  
lis', 'Mendez, 'ana@rmail.com');
```

```
UPDATE persona SET nombres = 'Ana Lis' WHERE id_persona = 1;
```

```
DELETE FROM persona WHERE id_persona = 1;
```

Sentencias de control (DCL)

Son las sentencias empleadas por los administradores de la base de datos para realizar tareas como por ejemplo, crear usuarios y concederles o revocar los privilegios. Se usa el término DCL del inglés Data Control Language.

Ejemplo DCL

En el ejemplo se le da permisos a un usuario llamado userdb de la base de datos de nombre biblioteca_db para que pueda insertar, actualizar, borrar y consultar datos de la tabla persona desde el mismo computador donde está instalada la base de datos (localhost).

```
GRANT INSERT, UPDATE, DELETE, SELECT ON biblioteca_db.persona TO  
userdb@'localhost';
```

Sentencias de control de transacciones (TCL)

Un pequeño grupo de sentencias que permiten procesar en bloque operaciones DML, garantizando que se efectúen todas y cada una de las operaciones o ninguna. TCL del inglés Transaction Control Language.

4. Sistema gestor de base de datos

En el mercado existen muchos sistemas gestores de bases de datos, a continuación, se usará MySQL para realizar las prácticas, pero ello no significa ni que sea el mejor, o el más completo del mercado, sino más bien el más usado para aplicaciones web, de pequeña, mediana y, en algunos casos, de gran complejidad. Es un buen punto de partida y todos los conceptos en MySQL vistos son reutilizables en PostgreSQL, Oracle, u otro motor de bases de datos relacional.

A continuación, se presenta el paso a paso de la descarga e instalación del entorno desarrollo WEB XAMPP:

Video 2. Descarga e instalación de entorno desarrollo WEB XAMPP



[Enlace de reproducción del video](#)

Síntesis del video: Descarga e instalación de entorno desarrollo WEB XAMPP

En este instructivo que presenta el experto temático explica cómo descargar e instalar XAMPP. XAMPP es un paquete de software que integra herramientas para el desarrollo de aplicaciones web.

Desde el navegador de internet y realizando la búsqueda con el texto: XAMPP, el buscador Google tiene como uno de los resultados que XAMPP es un paquete de software libre que consiste principalmente en un sistema gestor de base de datos MySQL.

También instala un servidor web: Apache, el cual es un programa que permite alojar páginas web y responder a las solicitudes de esas páginas que se hacen hacia esas páginas web, también los lenguajes de programación o intérpretes, lenguajes intérpretes como PHP y Perl, el nombre XAMPP viene del acrónimo: X, que significa cualquier sistema operativo, Apache: la A es de Apache, el servidor de aplicaciones web, la M de MariaDB o MySQL que es el motor de base de datos y las dos P son: la P de PHP y la P de Perl, X hace relación a que se puede instalar tanto en Windows porque sería wam o en Linux lam o en Mac.

A continuación, el experto temático realiza detalladamente el proceso de instalación de XAMPP, realizando la descarga desde internet, con la posibilidad de hacer la descarga para: Windows, Linux o Mac. A continuación, se realiza la instalación para Windows, asignando los permisos de instalación, también se indica todo el paso a paso para desempaquetar e instalar el sistema; y finalmente verificar el acceso al administrador del sistema de gestión de base de datos.

Continuemos con la descarga e instalación de herramienta CASE MySQL Workbench:

Video 3. Descarga e instalación de herramienta CASE MySQL Workbench



[Enlace de reproducción del video](#)

Síntesis del video: Descarga e instalación de herramienta CASE MySQL Workbench

En este video tutorial el Experto temático explica de manera práctica cómo instalar MySQL Workbench.

Desde el navegador se escribe MySQL Workbench, en donde arroja esta información: dice que MySQL Workbench es una herramienta visual de diseño de base de datos, integrada a desarrollo de software, es decir que es un id para bases de

datos, permite administrar bases de datos, diseñarla, gestionarla, mantenerla, y está diseñado para MySQL, en el enlace de la página de MySQL.com se encuentra la opción “Download” para descargarla en diferentes sistemas operativos con lo cual se realiza la descarga para Windows. Se debe tener una cuenta que corresponde a la plataforma de Oracle, por lo que se debe tener una cuenta registrando el correo electrónico y una contraseña para esta e ingresar datos de contacto, aceptar recibir información y crear la cuenta, inmediatamente llega un correo electrónico con un link de confirmación y la cuenta quedará activa cuando se dé clic en ese link de confirmación y activación de la cuenta.

Ya con la cuenta creada se dirige al botón de descarga, y una vez termina la descarga, se procede con la instalación, se asigna el permiso de instalación, se avanza en dos ventanas más y al finalizar el proceso permite abrir de una vez el programa.

Proceden las indicaciones para abrir MySQL workbench, el entorno de desarrollo integrado para diseño de bases de datos en MySQL, además del acceso para administrar bases de datos, para administrar comportamiento, para obtener copias de seguridad, restablecer copias de seguridad y esta es la herramienta para hacer modelos de bases de datos, diseños de bases de datos, para importar datos de un Excel a la base de datos, de otras bases de datos a esta base de datos.

Seguidamente están las orientaciones para iniciar con el proceso de diseño y crear una conexión, verificar que la base de datos MySQL esté corriendo, en color verde significa que si está corriendo y que se puede entrar al sistema de administración de base de datos. Para entrar se dá clic en el signo más (+) en donde se puede crear una base de datos. En el caso de una conexión local se debe indicar

local Host o 127.0.0.1 y Root que es el usuario principal, por defecto. XAMPP no asigna contraseña al Root de manera que se puede dejar sin contraseña, Una vez se ha establecido la conexión local, y ya en el modelador se muestra una pestaña para crear un diagrama, diagrama entidad relación, y a continuación se traen las diferentes entidades, las tablas, cómo se hacen relaciones de uno a uno, relaciones de uno a muchos, de uno a uno, de muchos a muchos, varios tipos de relaciones cuyas diferencias se podrán estudiar en un nuevo video.

Ahora, estudiemos el diseño de una base de datos con MySQL Workbench relación de muchos a muchos:

Video 4. Diseño de una base de datos con MySQL Workbench relación de muchos a muchos



[Enlace de reproducción del video](#)

Síntesis del video: Diseño de una base de datos con MySQL Workbench relación de muchos a muchos

En este video tutorial, el experto temático explica cómo se crea una relación de muchos a muchos, lo cual es mucho más sencillo, cuando tenemos dos tablas, y queremos hacer una relación o dos entidades y queremos hacer una relación de uno a muchos, lo que debemos hacer es usar esta herramienta, no hay punteado porque no tiene sentido que no sean nulas las llaves foráneas y seleccionamos estas dos en cualquier orden, al final nos genera una tabla intermedia, la tabla intermedia de la relación de muchos a muchos y él crea como llave primaria unas columnas.

Según el modelo antipersona, antipublicación, se cambia el nombre a la tabla, se asigna autor, según el modelo, ya con la tabla autor y el modelo, se tienen los tres tipos de relaciones.

Como ejercicio, el aprendiz podrá crear las demás relaciones del modelo, para lo cual se adjunta en la guía cómo queda el modelo final, y cada uno lo adjunta, lo termina y lo completa como un ejercicio de clase.

Continuemos con el diseño de una base de datos con MySQL Workbench relación de uno a muchos:

Video 5. Diseño de una base de datos con MySQL Workbench relación de uno a muchos



[Enlace de reproducción del video](#)

Síntesis del video: Diseño de una base de datos con MySQL Workbench relación de uno a muchos

A través de este video tutorial vamos a practicar cómo se crea una relación de uno a muchos.

Para hacer la relación entre publicación y tema, vamos a crear la tabla o la entidad dándole clic aquí y luego le damos clic en cualquier parte de la pizarra, luego le damos doble clic, la tabla, y escribimos el nombre de la entidad que en este caso sería publicación, le damos doble clic y él ya crea una llave primaria por defecto,

antipublicación, primary key y No null. Unsigne, significa que los datos no son, no puede haber datos negativos sino de cero en adelante, sin signo y se asigna el título, una publicación debe tener un título, título: 250; y seleccionar que no sea nulo, es decir, es obligatorio. No puede haber una publicación sin título, luego la edición, el ISBN que es el código que tienen todas las publicaciones y la fecha de publicación, entonces la fecha de publicación podría ser un tipo date o fecha y que sea no nulo, es decir obligatorio, este diagrama ha creado la tabla, los que son obligatorios de color azul, los no de color blanco, la llave primaria con un candado y básicamente ha creado la tabla “publicación”.

A continuación, vienen las indicaciones para crear la otra tabla, la tabla “temas”, donde están los temas: sociales, matemáticas, inglés, como los temas no son muchos se puede escoger un entero pequeño, y para el tema se puede poner 150. Es obligatorio definir el tema. Ahora, para crear una relación de uno a muchos, tienen que estar las llaves primarias ya definidas: id_publicación, id_tema y se crea de uno a muchos, usando el punteado, el que no tiene punta, el que es directo.

Y ya con la llave foránea, o llave no nula, que las pinta de color rosado y cuando no son obligatorias color rojo, esta es la forma de crear la publicación.

Queda como ejercicio que el aprendiz haga la relación de muchos a muchos; de uno a muchos, de editorial, es una relación que es igual a esta.

Y finalicemos con el diseño de una base de datos con MySQL Workbench relación de uno a uno:

Video 6. Diseño de una base de datos con MySQL Workbench relación de uno a uno



[Enlace de reproducción del video](#)

Síntesis del video: Diseño de una base de datos con MySQL Workbench relación de uno a uno

En este último video tutorial se realiza el instructivo para aprender cómo crear un modelo relacional con la herramienta Workbench.

Lo primero es verificar que XAMPP esté funcionando en el sistema gestor de base de datos que se llama MySQL y abrir la herramienta MySQL Workbench, una vez esté abierta ingresar en modelos, para crear un modelo dando clic en el signo más (+), donde abre la pestaña que dice MySQL Model y allí podemos agregar un diagrama, primero hay una pestaña donde está el diagrama que necesitamos, algunas

opciones: de guardar, de volver hacia atrás, cancelar, las tablas que vamos creando, una mano para recoger, para desplazarse, para borrar, para escribir algunas anotaciones.

Para crear una tabla, vamos a ver inicialmente cómo crear una entidad, vamos a hacer el modelo del ejercicio que aprendimos a modelar de la biblioteca escolar y le damos clic a la entidad o tabla luego le damos clic en cualquier parte de modelo donde queremos que quede y luego le damos doble clic a la tabla, al darle doble clic nos abre en la parte inferior una herramienta para editar, le cambiamos el nombre, por ejemplo, la tabla persona o la entidad persona, y empezamos a crear cada una de las columnas entonces ID persona como lo tenemos en el modelo con raya abajo, ID persona de tipo entero que es primary key PK que es la llave primaria, nn significa que no es null, recordemos que una llave primaria nunca puede ser null, y también tenemos la opción autoincrementable, significa que cada vez que voy a insertar un dato en la tabla persona él le va a poner una secuencia o sea 1, 2, 3, 4, en el orden en que vayamos insertando datos él les va asignando automáticamente un número del ID persona, podemos ponerle 150 como longitud y podemos decir que no null, es decir, que cuando vayamos a insertar datos en esta tabla debe ser obligatorio, poner nombres, lo mismo en apellidos pongámosle varchar 150 y no nulo, significa que es obligatorio y para el correo electrónico varchar de 50 por si hay cuentas de correo muy grandes que puede ser nulo si no colocamos not null, significa que puede ser nulo, de esa manera hemos creado la entidad persona.

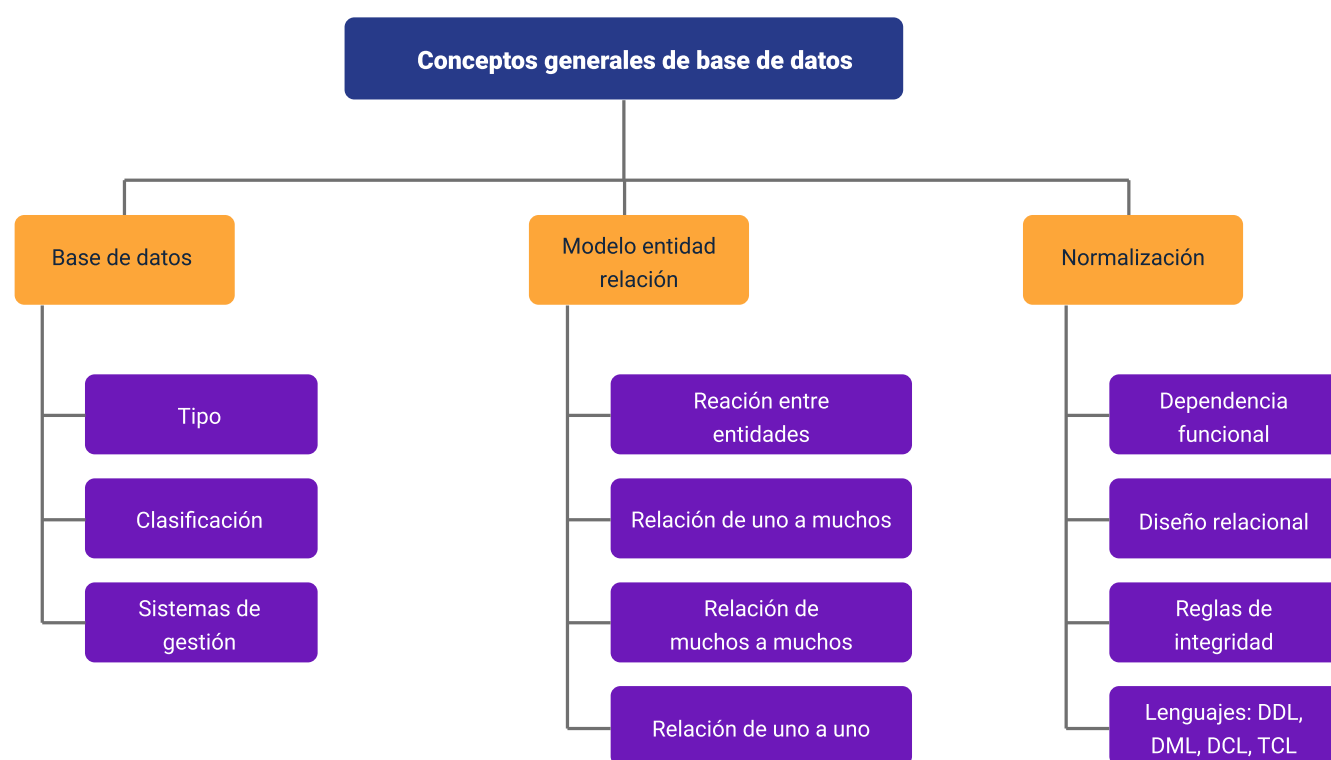
Ahora vamos a crear la entidad estudiante, creo una tabla, al dar doble clic nos permite editar la entidad estudiante, el estudiante tiene datos como la identificación pongámosle varchar a todos los tipos de datos con tamaño 12, no hay dos personas

en esta tabla con la misma identificación, y eso lo logramos con la restricción de unicidad, es decir que no haya más de dos personas con la misma identificación. La dirección es varchar 250 no es obligatorio tener la dirección, pero de pronto el teléfono sí, con longitud de 25, no se permiten datos nulos y el grado escolar, que puede ser un número de 1 a 10 grado escolar, entonces como es un número de 1 a 11 sería un entero muy pequeño, entonces usamos un entero, hay enteros grandes, medianos, reales y enteros muy pequeños, entonces le colocamos entero pequeño, tamaño 3, y puede ser nulo.

Ya tenemos nuestra entidad persona, nuestra entidad estudiante, a continuación, vamos a crear la relación de uno a uno que teníamos en nuestro diagrama, lo hacemos con la relación que tiene la línea continua y luego vamos a darle clic, después que está seleccionada esta herramienta vamos a darle clic a la tabla o a la entidad que va a recibir esa relación o sea que va a tener que va a agregar esa columna que es la tabla estudiante entonces le doy clic y luego le doy clic hacia la tabla de dónde voy a tener la llave foránea, entonces automáticamente él me puso una columna que se llama ID persona y me la convirtió en llave primaria pero yo le quiero cambiar el nombre, le coloco ID estudiante y para que se parezca más al diagrama que tenemos, podemos ordenarla, ir la subiendo en el orden ¿cómo se hace? pues le damos clic derecho mover hacia arriba, clic derecho mover hacia arriba, mover hacia arriba y mover hacia arriba y ya tenemos nuestra tabla estudiante y nuestra tabla persona con MySQL Workbench es la relación de uno a uno.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Nociones y modelos de bases de datos	Espinosa, A. P. (16 de noviembre de 2014). L6 1 Nociones y modelos de bases de datos. [Video] YouTube.	Video	https://www.youtube.com/watch?v=pPATLxijDfw
Fundamentos de bases de datos	Silberschatz, A., Korth, H. F., Sudarshan, S., Pérez, F. S., Santiago, A. I., y Sánchez, A. V. (2006). Fundamentos de bases de datos. McGrawHill.	PDF	https://www.academia.edu/31085120/FUNDAMENTOS DE BASES DE DATOS Cuarta edici%C3%B3n

Glosario

Base de datos: conjunto ordenado y finito de operaciones que permite hallar la solución de un problema.

Diccionario de datos: conjunto de programas, instrucciones y reglas informáticas para ejecutar ciertas tareas en una computadora.

Metadatos: conjunto unitario de instrucciones que permite a una computadora realizar funciones diversas, como el tratamiento de textos, el diseño de gráficos, la resolución de problemas matemáticos, el manejo de bancos de datos.

Referencias bibliográficas

Asale y RAE. (2021). Diccionario de la lengua española.

<https://dle.rae.es/base#CiiOsQO>

Kyocera. (2021). Conceptos sobre base de datos orientada a objetos. Kyocera.

<https://www.kyoceradocumentsolutions.es/es/smarter-workspaces/business-challenges/paperless/conceptos-sobre-base-de-datos-orientada-a-objetos.html#:~:text=Una%20base%20de%20datos%20orientada%20a%20objetos%20e s%20un%20sistema,en%20programaci%C3%B3n%20orient>

López, J. C. (2009). Algoritmos y programación (guía para docentes). Eduteka

<http://eduteka.icesi.edu.co/articulos/GuiaAlgoritmos>

Microsoft. (2021). Fundamentos de la normalización de bases de datos.

Microsoft. <https://docs.microsoft.com/es-es/office/troubleshoot/access/database-normalization-description>

Resnick, M. (2007). Sembrando las semillas para una sociedad más creativa.

Laboratorio de medios de MIT, Massachussets. Eduteka.

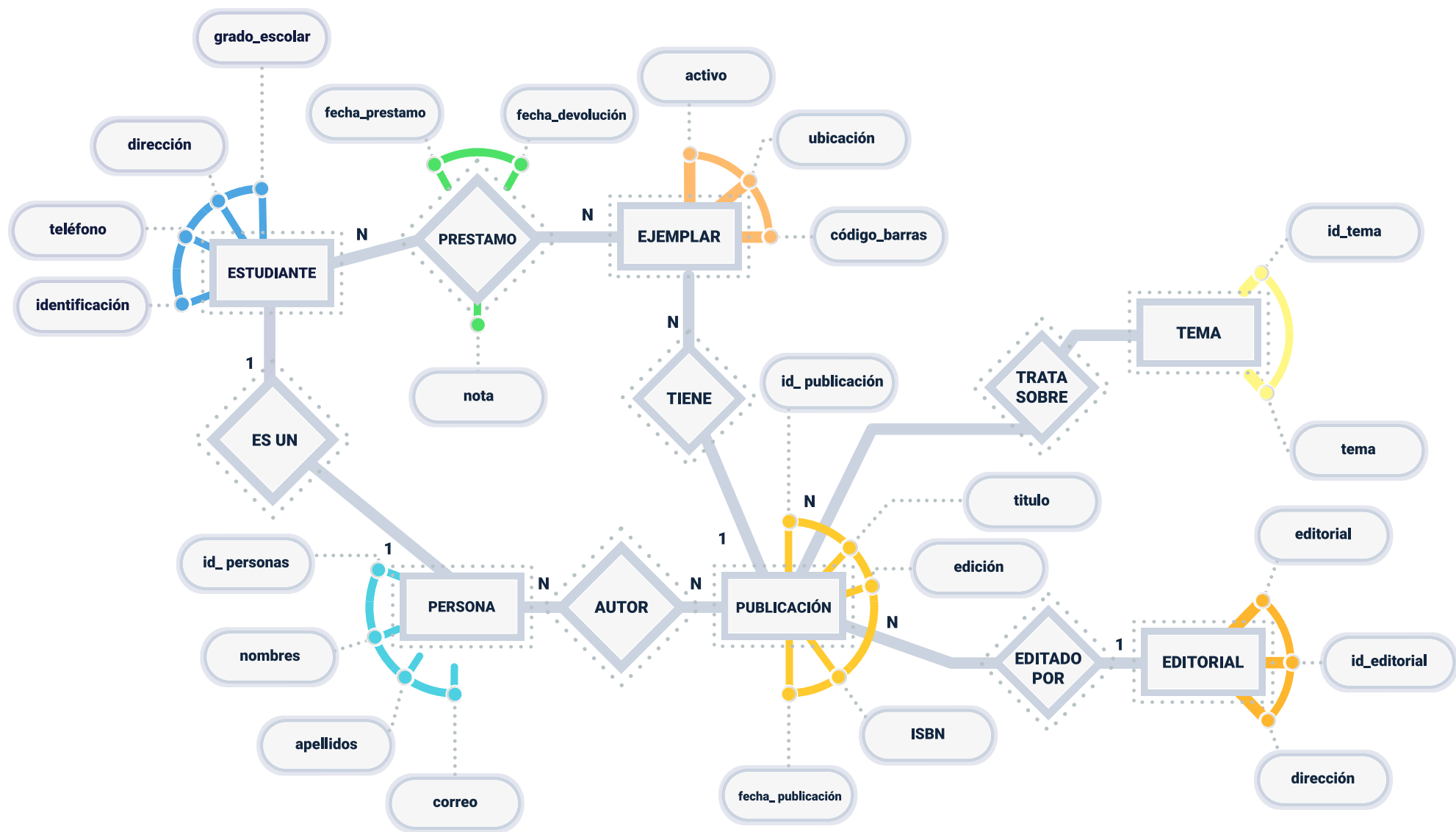
<http://www.eduteka.org/ScratchResnickCreatividad.php>

Silberschatz, A., Korth, H. F., Sudarshan, S., Pérez, F. S., Santiago, A. I., y Sánchez, A. V. (2006). Fundamentos de bases de datos. McGrawHill

<http://biblioteca.univalle.edu.ni/files/original/01aebde3cc06dce33f2538aa2724eb2541cb9473.pdf>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Henry Eduardo Bastidas Paruma	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia



Programación y algoritmia básica

Breve descripción:

El componente formativo en algoritmia enseña la definición y aplicación de algoritmos para resolver problemas de manera eficaz. Incluye el pensamiento algorítmico, fases de diseño, ejemplos prácticos, y herramientas como pseudocódigo y diagramas de flujo, proporcionando una base sólida para la programación y la optimización de procesos computacionales.

Julio 2024

Tabla de contenido

Introducción	6
1. Introducción a la algoritmia	8
Definición de algoritmo	9
1.1. Pensamiento algorítmico	9
1.2. Solución de problemas y programación	11
1.3. Análisis del problema (entenderlo)	14
Formular claramente el problema	14
Precisar elementos de entrada	15
Precisar resultados esperados	16
1.4. Desarrollo de la creatividad: elementos, modelos, fases y sus objetivos	16
2. Lógica matemática	18
2.1. Componentes de la lógica matemática	18
Proposiciones	18
Conectores lógicos y proposiciones compuestas	19
Tablas de verdad	20
2.2. Ejercicios de lógica proposicional	27
3. Metodología de algoritmos	31
3.1. Análisis	32

3.2.	Diseño	33
3.3.	Pruebas de escritorio o trazas.....	36
	Solución.....	37
4.	Diagramas de flujo	41
	Elaborar diagramas de flujo	44
	Herramientas digitales para creación y prueba de algoritmos	45
5.	Uso de identificadores y palabras reservadas	46
6.	Operadores y jerarquía en los operadores	50
	Jerarquía en los operadores.....	51
7.	Estructuras de control secuencial	54
	Ejemplo estructura de control secuencial	55
7.1.	Estructura condicional	58
7.2.	Estructuras de iteración o repetitivas	60
	Estructura PARA (FOR)	61
	Estructura MIENTRAS (WHILE)	67
8.	Estructura de datos.....	71
	Matrices	76
9.	Conceptos básicos de programación.....	81
10.	Sintaxis del lenguaje JavaScript	84

10.1. Tipos de datos, operadores y orden de evaluación	87
a. Tipos de datos.....	88
b. Operadores.....	89
c. Orden de evaluación de operadores	93
10.2. Expresiones y comentarios	94
10.3. Estructuras de selección	95
10.4. Estructuras de repetición.....	99
a. Estructura de repetición FOR	99
b. Estructura de repetición WHILE	103
10.5. Estructuras de datos	105
a. Vectores.....	105
b. Matrices	108
c. Registros	109
10.6. Estructuras de salto	113
10.7. Métodos de ordenamiento y búsqueda.....	117
a. Ordenamiento de cadenas de texto	117
b. Ordenamiento de datos numéricos	120
c. Ordenamiento de registros	123
d. Buscar un elemento.....	124

11. Depuración y fallas de sintaxis	126
12. Manejo de errores y excepciones	128
Síntesis	131
Material complementario.....	132
Glosario	135
Referencias bibliográficas	136
Créditos	139

Introducción

La algoritmia es una disciplina fundamental en la programación de computadoras, ya que permite la resolución precisa y eficiente de problemas mediante la aplicación de una serie de pasos detallados y ordenados. La correcta comprensión y aplicación de algoritmos es esencial para el desarrollo de software, pues un algoritmo bien diseñado no solo garantiza una solución adecuada a un problema, sino que también optimiza el rendimiento y la eficacia del programa resultante. La base de esta práctica radica en la capacidad de descomponer problemas complejos en operaciones más simples y manejables, facilitando así su resolución.

El pensamiento algorítmico es una habilidad crucial que permite abordar problemas desde diferentes enfoques lógicos, llevando siempre a una solución eficiente. A través de la algoritmia, los aprendices desarrollan la capacidad de estructurar secuencias de pasos que, aunque puedan variar en su ejecución, convergen en una solución común al problema planteado. Esta habilidad es especialmente relevante en contextos donde se requiere procesar grandes volúmenes de datos o realizar múltiples operaciones en un corto período, tareas en las que las computadoras sobresalen debido a su capacidad de realizar operaciones matemáticas básicas de manera extremadamente rápida.

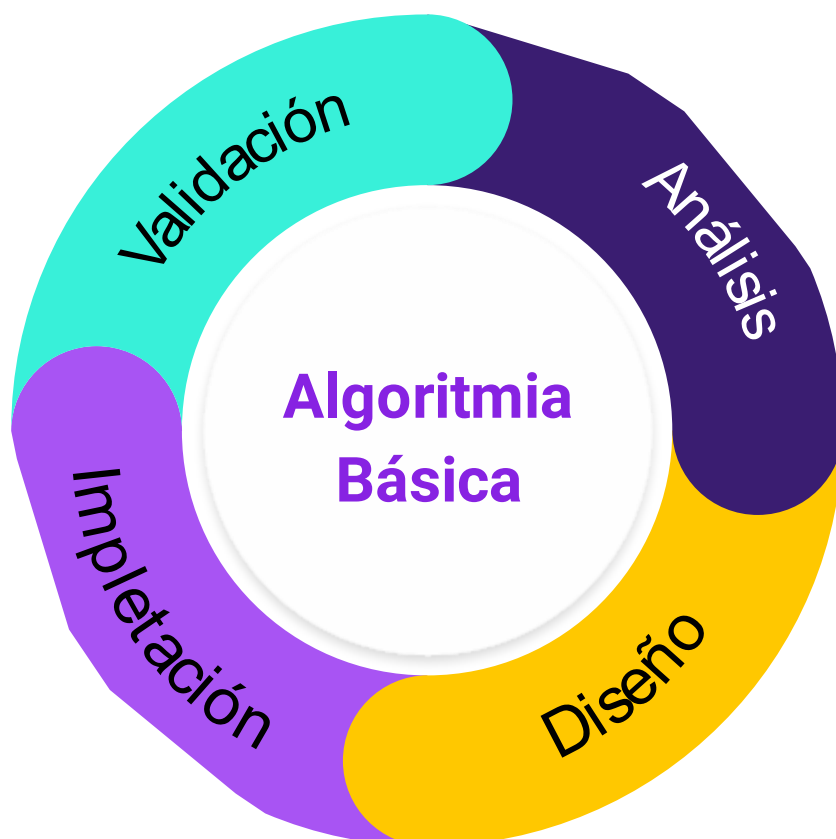
En este componente formativo, se explorarán los conceptos básicos de algoritmos, su definición, y las fases para su diseño y desarrollo. Se analizarán ejemplos prácticos que ilustran cómo los algoritmos pueden aplicarse en situaciones cotidianas, así como en problemas más complejos. Además, se abordarán las herramientas y

metodologías utilizadas en la elaboración de algoritmos, incluyendo el uso de pseudocódigo y diagramas de flujo.

1. Introducción a la algoritmia

La correcta aplicación de una serie de pasos detallados puede garantizar una solución precisa para un problema. Es precisamente en esta práctica donde se basa la programación de computadoras y la construcción de software.

Figura 1. Pasos en la algoritmia básica



Esta figura presenta los cuatro pasos fundamentales en la creación de algoritmos básicos: análisis, diseño, implementación y validación. Cada paso es crucial para garantizar que el algoritmo funcione correctamente y cumpla con los objetivos establecidos.

Definición de algoritmo

Según la Real Academia Española (2020), un algoritmo está definido como un “conjunto ordenado y finito de operaciones que permite hallar la solución de un problema”.

Es importante comprender que computacionalmente, las máquinas están en capacidad de realizar unas pocas operaciones matemáticas muy básicas y sencillas, pero también se considera que computan enormes cantidades de estas operaciones en unidades de tiempo muy corto.

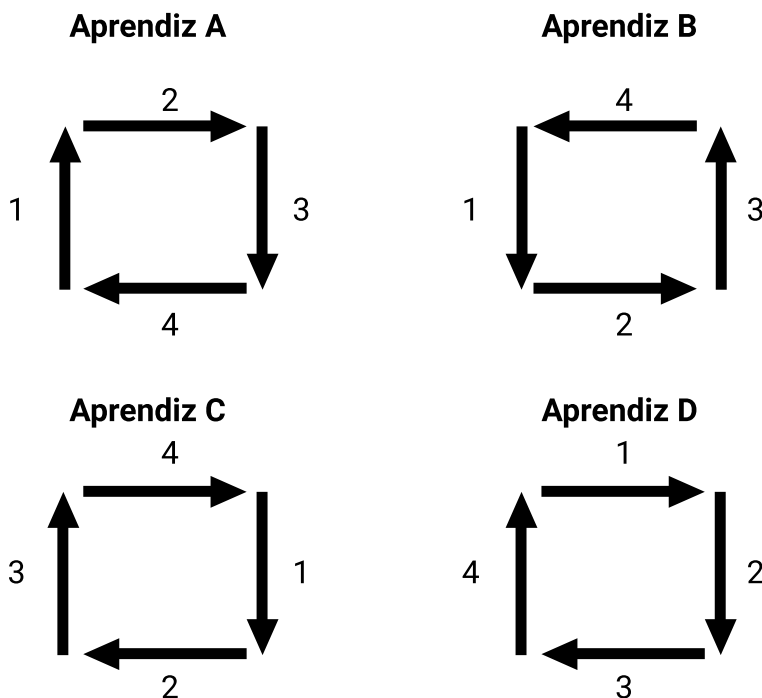
Entonces, la solución de un problema utilizando computadoras fácilmente se puede convertir en una enorme cantidad de pequeñas operaciones matemáticas, tal vez abrumadoras desde el punto de vista humano. Es por eso que se requiere la especialización de conjuntos de operaciones, los cuales se agrupan en pequeños procesos que componen la solución de un problema.

1.1. Pensamiento algorítmico

Así como un problema puede ser abordado de varias maneras y aportar la misma solución, lo que siempre tendrán en común es que para resolverlo se utilizó una secuencia de pasos ordenados.

Por ejemplo, si se entrega una hoja de papel cuadriculado a los aprendices de una clase y se les pide que dibujen un cuadrado sin levantar el lápiz, habrá quienes lo hagan de manera diferente y obtengan el mismo resultado, como se presenta en la siguiente figura:

Figura 2. Pensamiento lógico y procedimental



En la figura, el aprendiz A comenzó realizando una línea vertical hacia arriba, mientras que el aprendiz B trazó una vertical hacia abajo, y el aprendiz D dibujó una horizontal hacia la derecha. Todos ellos llevaron a cabo un proceso que les permitió cumplir con la solución al problema planteado. En términos globales, prácticamente existe una gran cantidad de soluciones al mismo problema.

Al imaginar que el problema es mucho más complejo que el descrito en el ejemplo anterior, se concluye que existen tantas soluciones como el enfoque o lógica empleada por cada aprendiz o persona. Sin embargo, lo que sí es común en todos los casos es que se realizaron unas fases mentales secuencialmente ordenadas, las cuales son:

- Entender el problema enunciado por el instructor (entender el problema).
- Escoger una ruta a seguir del rectángulo (hacer un plan).

- c) Plasmar en el papel cuadriculado ese plan (ejecutar el plan).
- d) Revisar el resultado y ver que se alcanzó el objetivo (revisar).

Cuando el problema es muy complejo de resolver, muchas veces toca volver y analizar el problema, trazar un nuevo plan o plasmar ese nuevo plan una y otra vez, revisando los resultados y volviendo a analizar hasta encontrar la solución del problema.

1.2. Solución de problemas y programación

Los algoritmos son una herramienta que permite describir claramente un conjunto finito de instrucciones.

“En el ámbito de la computación, los algoritmos son una herramienta que permite describir claramente un conjunto finito de instrucciones ordenadas secuencialmente y libres de ambigüedad, que debe llevar a cabo un computador para lograr un resultado previsible. Vale la pena recordar que un programa de computador consiste en una serie de instrucciones muy precisas y escritas en un lenguaje de programación que el computador entiende”. (López, 2019, p.7).

Ejemplo 1: se propone diseñar un algoritmo para apagar una computadora.

Algoritmo: apagar computadora

INICIO

Dar clic en el icono INICIO.

Dar clic en el icono que dice APAGAR.

Esperar que se apague completamente.

Desconectar los cables del enchufe.

FIN

Ejemplo 2: se propone diseñar un algoritmo para apagar una computadora.

Algoritmo: cargar un celular

INICIO

Buscar el cargador.

Agarrar el cargador.

Agarrar el celular.

Poner el enchufe en el celular.

Conectarlo al tomacorriente.

Esperar a que se ponga a cargar.

Desconectarlo del tomacorriente.

Ponerlo en un lugar seguro.

FIN

Con base en los ejemplos anteriormente planteados. Se puede concluir que un algoritmo debe ser:

- **Realizable**

El proceso algorítmico debe terminar después de una cantidad finita de pasos. Se dice que un algoritmo es inaplicable cuando se ejecuta con un

conjunto de datos iniciales y el proceso resulta infinito o durante la ejecución se encuentra con un obstáculo insuperable sin arrojar un resultado.

- **Comprensible**

Debe ser claro lo que hace, de forma que quien ejecute los pasos (ser humano o máquina) sepa qué, cómo y cuándo hacerlo. Debe existir un procedimiento que determine el proceso de ejecución.

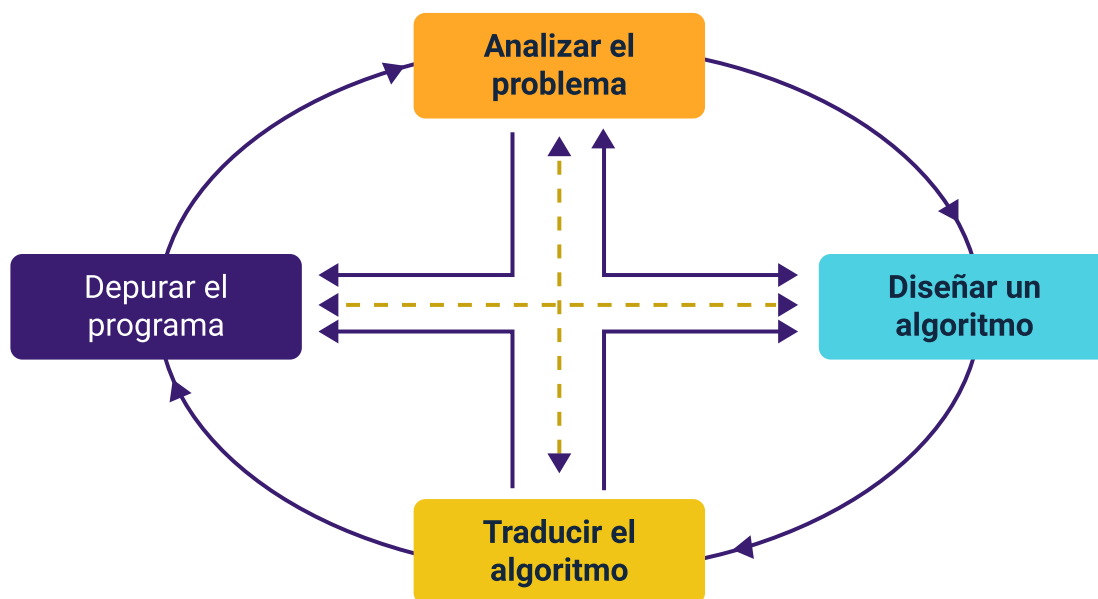
- **Preciso**

El orden de ejecución de las instrucciones debe estar perfectamente indicado. Cuando se ejecuta varias veces, con los mismos datos iniciales, el resultado debe ser el mismo siempre. La precisión implica determinismo.

(López, 2009, p. 22)

La figura presenta las fases y sus relaciones en el proceso de diseño de algoritmos de programas informáticos, que están basados en un algoritmo que posteriormente es escrito en un lenguaje de programación.

Figura 3. Fases para elaborar un programa de computador



Nota. Tomado y adaptado de Arellano et al. (2022).

El diagrama presenta el ciclo de desarrollo de algoritmos, que incluye las etapas de análisis del problema, diseño del algoritmo, traducción del algoritmo a un lenguaje de programación y depuración del programa. Este proceso es iterativo, permitiendo mejoras continuas.

1.3. Análisis del problema (entenderlo)

Como los programas de computador tienen por objetivo resolver problemas muy puntuales, lo primero que se debe realizar para resolverlos es lograr una mejor comprensión posible de estos.

Para realizar esta actividad se debe:

Formular claramente el problema

A continuación, algunas preguntas previas a la formulación del problema:

- ¿Es posible definir de forma más clara el problema?
- ¿Qué palabras no son conocidas o se desconoce su significado?
- ¿Se ha resuelto antes algún problema similar?
- ¿Qué información es importante para resolver el problema?
- ¿Qué información no es importante y se puede omitir?

Ejemplo:

Don Juan necesita decidir cómo comprar un teléfono celular que cuesta \$870.000 de contado o \$980.000 a crédito; él tiene \$630.000 pesos en efectivo.

Luego de revisar las preguntas previas se puede concluir:

Como el efectivo que tiene don Juan no le alcanza para comprar el teléfono celular de contado, entonces él tiene dos opciones:

Comprarlo completamente a crédito o también pagar una parte de contado (como cuota inicial) y el resto diferirlo a crédito.

Para poder resolver este problema se debe conocer el número de cuotas en el que difiere, si desea pagarlo totalmente a crédito o conocer el número de cuotas y el valor total del celular si se da una cuota inicial de \$630.000 pesos.

Precisar elementos de entrada

Otro punto importante en la fase de análisis del problema es determinar cuál es la información disponible, por lo tanto:

- ¿Qué información es importante y necesaria para resolver el problema?
- ¿Qué información no es importante y se puede prescindir?
- ¿Cuáles son los datos de entrada conocidos?

- ¿Cuál es la incógnita o qué se debe calcular?
- ¿Los datos se pueden agrupar en categorías?
- ¿Qué información adicional hace falta para resolver el problema?

Otro elemento a resaltar es el nivel de conocimiento y las limitaciones que se tienen en el ámbito o el contexto del problema que está tratando de resolver. Por ejemplo, si el problema requiere conceptos financieros para entender la naturaleza de los datos, o si es necesario adquirir nueva información y/o poder consultarla, para estar seguros de que es posible tratarla conforme el contexto lo requiera.

Precisar resultados esperados

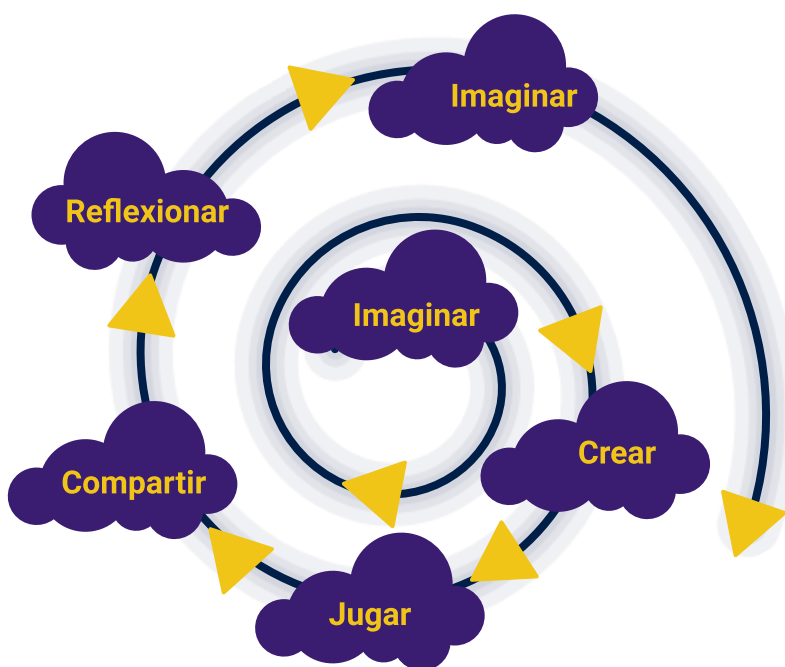
Para determinar con claridad cuál es el resultado final esperado (o producto) que se quiere obtener es necesario determinar o definir qué resultados se solicitan y, en qué tipo o formato deben estar (mostrados en pantalla, diagramación, orden, entre otros.). Para ello, es importante tener en cuenta:

- ¿Qué información están solicitando?
- ¿En qué formato se debe entregar esta información?

1.4. Desarrollo de la creatividad: elementos, modelos, fases y sus objetivos

Resolver algoritmos es una actividad que requiere, en la mayoría de los casos, creatividad por parte del programador. Esta es una habilidad que se va fortaleciendo en la medida en que se desempeña más y más en procesos de desarrollo de software.

Figura 4. Espiral del Pensamiento Creativo diseñada por el Dr. Mitchel Resnick



Nota. Tomado y adaptado de Resnick (2007).

Esta figura representa el ciclo creativo compuesto por las etapas de imaginar, crear, jugar, compartir, reflexionar e imaginar nuevamente. Cada etapa se retroalimenta, promoviendo un proceso continuo de innovación y desarrollo creativo.

“Para estimular el pensamiento creativo, propuesto por Resnick (2007), el programador debe imaginar qué quiere hacer; crear un proyecto basado en sus ideas; jugar y compartir sus ideas y creaciones con otros para reflexionar sobre sus experiencias y permitirse imaginar nuevas ideas y nuevos proyectos. La espiral genera un proceso indefinido de mejoramiento continuo”. (López, 2009, p. 19).

2. Lógica matemática

La lógica matemática es la disciplina que trata de métodos de razonamiento. En un nivel elemental, la lógica proporciona reglas y técnicas para determinar si un argumento dado es válido o no. El razonamiento lógico se emplea en matemáticas para demostrar teoremas; en ciencias de la computación para verificar si los programas son correctos; en las ciencias físicas y naturales para sacar conclusiones de experimentos; y en las ciencias sociales y en la vida cotidiana para resolver una multitud de problemas. Ciertamente, se usa de forma constante el razonamiento lógico para realizar cualquier actividad (EcuRed, s.f.).

2.1. Componentes de la lógica matemática

Todo razonamiento lógico está compuesto por una serie de elementos que se interrelacionan entre sí. La correcta interpretación de estos componentes y de sus relaciones corresponde al análisis deductivo. Estos componentes son las proposiciones y los conectores lógicos, que se describen a continuación:

Proposiciones

Una proposición o enunciado es una oración que puede ser falsa o verdadera, pero no ambas a la vez. La proposición es un elemento fundamental de la lógica matemática.

Las proposiciones pueden ser simples o compuestas. Las primeras están formadas por una sola proposición; por el contrario, las proposiciones compuestas están conformadas por dos o más proposiciones unidas por un conector.

p: la tierra es plana.

q: $-17 + 38 = 21$

r: hola ¿cómo estás?

S: lava el carro por favor.

p y **q** son proposiciones simples válidas porque pueden tomar un valor verdadero o falso; sin embargo, los enunciados **r** y **s** son proposiciones no válidas, porque no pueden tomar un valor de falso o verdadero. Uno de ellos es un saludo y el otro es una orden.

Conectores lógicos y proposiciones compuestas

Existen las proposiciones compuestas que son las que se componen de varias proposiciones, las cuales utilizan para su conexión operadores o conectores lógicos que son:

a) Operador AND (y) conjunción

Conecta dos proposiciones que se deben unir para que se pueda obtener un resultado verdadero, esta se representa por el siguiente símbolo $\wedge$.

Ejemplo

- Nuestra bandera es de color amarillo, azul y rojo.
- El número 9 es mayor que 7 y menor que 12.
- El carro enciende cuando tiene corriente eléctrica y gasolina en el tanque.

b) Operador OR (o) disyunción

Conecta dos proposiciones y se obtiene una proposición compuesta verdadera cuando por lo menos unas de las proposiciones son verdaderas.

Ejemplo

- Puedo ir a Bogotá en avión o en bus.
- Puedo entrar al estadio si compro mi boleta o si tengo un pase de cortesía.
- Voy a desayunar con huevos revueltos o huevos pericos.

c) Operador NOT (no) negación

Su propósito es negar una proposición, es decir, que si tenemos una proposición verdadera al aplicarle este tipo de operador se obtiene la negación, es decir, es falso.

Ejemplo

- No es cierto que Gabriel García Márquez escribió la Ilíada.
- No es cierto que el América de Cali es de la ciudad de Bogotá.
- Es falso que el agua es un hidrocarburo.

Tablas de verdad

Una estrategia de la lógica que tiene como objetivo determinar la validez de varias propuestas en alguna situación es la tabla de verdad. Esta estrategia determina las condiciones necesarias en las que un enunciado es verdadero, representando dichas condiciones en una tabla que establece el valor de verdad de todas las proposiciones que componen la situación. Así como lo explica el principio de bivalencia: una proposición puede ser verdadera o falsa únicamente, nunca ambas.

Algunas consideraciones:

- a) El valor de una proposición verdadera se representa por “V” o un “1”.
- b) El valor de una proposición falsa se representa por “F” o un “0”.

- c) Existe una tabla de verdad para cada uno de los conectores lógicos, lo que se tiene que evaluar para responder si una proposición compuesta es verdadera o falsa es el estatus actual de las proposiciones simples.

A continuación, se presentan las tablas de verdad.

Tabla 1. Tabla de verdad conjunción (Y)

P	Q	$P \wedge Q$
V	V	V
V	F	F
F	V	F
F	F	F

O puede

Tabla 2. Tabla de verdad conjunción (Y)

P	Q	$P \wedge Q$
1	1	1
1	0	0
0	1	0
0	0	0

Tabla 3. Tabla de verdad disyunción (O)

P	Q	$P \vee Q$
V	V	V
V	F	V

P	Q	$P \vee Q$
F	V	V
F	F	F

O puede

Tabla 4. Tabla de verdad disyunción (O)

P	Q	$P \vee Q$
1	1	1
1	0	1
0	1	1
0	0	0

Tabla 5. Tabla de verdad negación (no)

P	$\neg P$
V	F
F	V

O puede

Tabla 6. Tabla de verdad negación (no)

P	$\neg P$
1	0
0	1

Existe otro tipo de conectores que se llaman proposiciones condicionales, aquellas que están formadas por dos proposiciones simples o compuesta por p y q , la cual se representa y se lee de la siguiente manera:

$p \rightarrow q$ se lee “Si p entonces q ”

- **Ejemplo 1**

Las palomas vuelan.

p : todas las palomas vuelan.

q : todas las aves tienen plumas.

r : luego todas las palomas son aves.

Estas proposiciones se pueden escribir utilizando los operadores lógicos de la siguiente manera: $(p \wedge q) \rightarrow r$.

Para el anterior ejercicio se construye la tabla de verdad:

Tabla 7. Tabla de verdad ejemplo 1

P	Q	R	$P \wedge Q$	$\neg R$
V	V	V	V	F
V	V	F	V	V
V	F	V	F	F
V	F	F	F	V
F	V	V	F	F
F	V	F	F	V
F	F	V	F	F

P	Q	R	$P \wedge Q$	$\neg R$
F	F	F	F	V

- **Ejemplo 2**

Si no llueve se arruina la cosecha.

Este tipo de proposición se conoce como la condición “para que”, de manera que para que no se arruine la cosecha se requiere que llueva.

p: si no llueve.

q: se arruina la cosecha.

Al aplicar el operador $p \rightarrow q$ su tabla de verdad quedaría de la siguiente manera:

Tabla 8. Tabla de verdad ejemplo 2

P	Q	$P \rightarrow Q$
V	V	V
V	F	F
F	V	V
F	F	V

O puede

Tabla 9. Tabla de verdad ejemplo 2

P	Q	$P \rightarrow Q$
1	1	1

P	Q	$P \rightarrow Q$
1	0	0
0	1	1
0	0	1

De igual manera, existe una proposición bicondicional que indica que una proposición es verdadera si y solo si las proposiciones simples o compuestas que forman la proposición son verdaderas o son falsas si y solo si ambas proposiciones son falsas, esta se representa y se lee de la siguiente manera: $p \leftrightarrow q$ se lee "p si solo si q"

- **Ejemplo 3**

Es buen estudiante.

Es buen estudiante, si y solo si tiene promedio de cinco, donde se puede concluir lo siguiente:

p: es un buen estudiante.

q: tiene promedio de 5.

al aplicar el operador $p \leftrightarrow q$ su tabla de verdad quedaría de la siguiente manera:

Tabla 10. Tabla de verdad ejemplo 3

P	Q	$P \leftrightarrow Q$
V	V	V
V	F	F

P	Q	$P \leftrightarrow Q$
F	V	V
F	F	V

O puede

Tabla 11. Tabla de verdad ejemplo 3

P	Q	$P \leftrightarrow Q$
1	1	1
1	0	0
0	1	1
0	0	1

Cuando se aplican tablas de verdad se puede llegar a tres tipos de resultado que son:

- **Tautología:** es una expresión lógica que es verdadera para todos los posibles valores de verdad de sus componentes atómicos. Las proposiciones dan el resultado positivo.
- **Contradicción:** son aquellas fórmulas que son falsas para cualquier valoración de los símbolos proposicionales que contiene. Las proposiciones dan el resultado negativo.
- **Contingencia:** son aquellas fórmulas cuyo valor de verdad o falsedad depende de la valoración de los símbolos proposicionales que contiene. Las proposiciones dan un resultado negativo y positivo. (TOMi.digital, 2021).

2.2. Ejercicios de lógica proposicional

A continuación, se plantean a manera de ejemplo algunos ejercicios con su respectiva solución que permiten identificar un problema, reconocer los datos de entrada y los procesos a ser aplicados como estrategia a seguir con la utilización de la lógica proposicional en la identificación y solución de problemas. Para dar solución al enunciado anterior se realiza las siguientes actividades.

Ejercicio 1: se necesita personal femenino que sepa Microsoft Office o Gsuite.

Pasos a seguir:

- a) Identificar el problema.
- b) Descomponer el enunciado en proposiciones atómicas.
- c) Evaluar los posibles casos derivados del enunciado anterior.
- d) Determinar los casos donde se cumpla la condición y hacer la selección de las personas de acuerdo con el requerimiento.

Solución

- **Paso 1**

Identificar el problema

Se trata de buscar o contratar personal femenino que sepa utilizar Microsoft Office o Google Suite.

- **Paso 2**

Descomponer el enunciado

Descomponer en proposiciones atómicas del enunciado (cuando están separadas del contexto del problema se llaman proposiciones atómicas).

p: es personal femenino.

q: sabe utilizar el Microsoft Office.

r: sabe utilizar la suite de Google.

- **Paso 3**

Determinar los casos

Determinar las relaciones lógicas de acuerdo con lo planteado en el enunciado, utilizando las proposiciones atómicas definidas en el punto anterior.

$$p \wedge (q \vee r).$$

Ejercicio 2: Juan sabe utilizar Microsoft Office y Gsuite, Martha sabe utilizar Gsuite, Natalia sabe utilizar Gsuite y Microsoft Office.

Crear la tabla de verdad y aplicar la lógica proposicional para elegir las personas que cumplan con los requisitos exigidos.

Para la construcción de la tabla de verdad se evalúa cada una de las preposiciones atómicas y, se van validando para verificar si se cumple lo que la haría verdadera o si no se cumple, lo cual haría que la proposición sea falsa.

Se analiza cada posible caso por lo que se evalúa a Juan, para la primera proposición sería un valor falso, ya que **p** requiere que sea personal femenino, el cual se pondrá en la columna 2, luego se evalúa si Juan sabe utilizar Microsoft Office o la Suite de Google, el valor va en la columna tres y por último, se evalúa la conjunción para determinar la solución.

Tabla 12. Tabla de verdad ejemplo 2

Nombre	P	$(q \vee r)$	$p \wedge (q \vee r)$
Juan	F	V	F
Martha	V	V	V
Nathalia	V	V	V

Ejercicio 3. Sandra dice adelgazo o no hago dieta.

Para este enunciado además se propone una serie de alternativas que podrían ser válidas o no, dependiendo de la solución propuesta utilizando la lógica matemática. Las siguientes son las posibles respuestas de las cuales se deberá elegir la que se considera es real al problema planteado.

- Si adelgazo, hago dieta.
- Si no adelgazo, hago dieta.
- Si hago dieta, no adelgazo.
- Si hago dieta adelgazo.
- Hago dieta o no adelgazo.

Solución

- **Paso 1**

Problema

Identificar la respuesta correcta al problema presentado por Sandra.

- **Paso 2**

Definir

Definir las proposiciones atómicas:

p: adelgazo.

q: hago dieta.

- **Paso 3**

Determinar

Determinar la relación lógica.

$p \vee \neg q$.

- **Paso 4**

Crear

Crear la tabla de verdad.

Tabla 13. Tabla de verdad ejercicio 3

P	Q	$P \vee \neg Q$
V	V	V
V	F	V
F	V	F
F	F	V

3. Metodología de algoritmos

De manera implícita, en las distintas áreas productivas de la sociedad se usan algoritmos. Un par de ejemplos pueden ser que, en la construcción de una casa, el ingeniero o el arquitecto analiza el terreno, diseña unos planos, realiza maquetas y así continúa siguiendo una secuencia lógica de procesos para la resolución del problema; o un abogado que, antes de defender a un cliente, analiza las pruebas, elabora un plan, practica sus argumentos y sigue un método para lograr la defensa de su cliente. Se puede concluir, entonces, que una solución a la medida de un problema requiere una metodología que lleve a la solución final.

En el proceso de desarrollo de programas informáticos, aparte de conocer esta metodología, se necesita conocer las funciones que puede realizar un computador y la representación de estas.

En general, existen tres fases secuenciales para la elaboración de un algoritmo, de las cuales se abordarán las dos primeras:

a) Análisis

- Planteamiento del problema.
- Identificación de datos.
- Identificación de procesos y operaciones.
- Identificación de salidas y resultados.

b) Diseño

- Desarrollo de algoritmos.
- Verificación de algoritmos.
- Implementación.

c) Codificación de algoritmo

- Prueba y depuración.
- Documentación.
- Mantenimiento.

3.1. Análisis

Como se explicó en componentes anteriores, la fase de análisis trata de responder a la pregunta: ¿Qué es lo que se debe hacer? Esto es necesario para entender el problema. Por ello, es preciso relacionar los datos de entrada, salida o resultados con los procesos a ejecutar con esta información y los datos auxiliares que se puedan requerir en las tareas.

Hasta este punto, parece que esas son las actividades propias del análisis. Sin embargo, el éxito de un algoritmo consiste en reducir a cero la brecha entre lo que el cliente necesita y los resultados que el algoritmo entrega. Para lograrlo en su totalidad, es importante indagar otros aspectos que corresponden a la naturaleza del problema. Las siguientes preguntas modeladoras son de referencia casi obligatoria a tener en cuenta:

- ¿Qué es?
- ¿Quiénes intervienen?
- ¿Cómo interviene o se comporta?
- ¿Cuándo?
- ¿Cómo lo hace?
- ¿Por qué?
- ¿Para qué?

3.2. Diseño

En esta etapa se responde, básicamente, a la pregunta: ¿Cómo se va a hacer?, refiriéndose a la manera en que el sistema va a realizar la tarea solicitada. Lo importante en este punto es que debe quedar bien definido el problema, sin dar espacio a más de una interpretación de la acción o acciones que debe hacer el sistema.

En el proceso de desarrollo del algoritmo, las siguientes actividades son obligatorias, especialmente si se está comenzando en el diseño de soluciones basadas en algoritmos y no se cuenta con mucha experiencia:

Figura 5. Actividades diseño algoritmo



Esta figura detalla las actividades clave en el diseño de algoritmos: definir las partes del algoritmo, representar el algoritmo mediante notación, diagramas o

pseudocódigo. Estas actividades son esenciales para estructurar y comunicar el algoritmo de manera correcta.

Para la verificación del algoritmo, se aplica una técnica denominada pruebas de escritorio o trazas, que trata de la validación y verificación del algoritmo mediante la ejecución de las sentencias u operaciones que lo componen (proceso), y así determinar sus resultados de salida a partir de una base determinada de elementos en la entrada del proceso.

Como el resultado del proceso de diseño es un algoritmo, es obligatorio saber representarlo de alguna manera. Inicialmente, se empleará la notación de pseudocódigo para hacer este tipo de representaciones.

Notación de diseño: pseudocódigo

Para expresar el contenido de un programa de computadora, se necesita un lenguaje de programación que las computadoras puedan interpretar. Sin embargo, se puede expresar ese mismo contenido en el lenguaje natural, empleando expresiones sencillas que lo simplifican y abrevian. Cuando se emplea esta técnica, se dice que está escrito en pseudocódigo.

Ejemplo:

El cliente pide que se desarrolle un programa de computadora que le solicite al usuario un número, luego el sistema debe mostrar en pantalla el número digitado.

A continuación, se aplica la metodología:

- **Análisis**

El programa debe solicitar al usuario que ingrese un número. Luego, recibirá ese número desde el teclado y lo almacenará en una variable. Para ello, se debe definir una variable antes de empezar el algoritmo, reservando un espacio para guardar un número entero.

Una vez definida la variable, se leerá el número ingresado por el usuario y se almacenará en dicha variable. Finalmente, el programa imprimirá el contenido de la variable, concluyendo así el proceso.

- **Diseño**

El anterior análisis se contemplaron los datos de entrada, datos auxiliares (variable), operaciones (impresión en pantalla, lectura de datos e impresión en pantalla de resultados). Pero son muchas las palabras usadas en el análisis por lo tanto lo abreviamos con el siguiente código que, posteriormente, se explicará.

ALGORITMO Mostrar número digitado;

VAR

ENTERO entrada;

INICIO

ESCRIBIR ("Por favor ingrese un número");

LEER (entrada);

ESCRIBIR (entrada);

FIN

En un algoritmo, se indica el nombre del mismo y luego se declaran las variables que serán usadas (con VAR), especificando su tipo de dato (un valor de tipo ENTERO, para este ejemplo).

Entre INICIO y FIN se escriben las diferentes acciones que realizará el algoritmo. Usamos ESCRIBIR() para mostrar un mensaje por pantalla (el texto que se va a mostrar 'tal cual' debe ir entre paréntesis dobles) y LEER() para recibir el dato desde el teclado.

Se debe tener en cuenta que cada línea termina en punto y coma (excepto INICIO y FIN). El punto y coma indica que lo que se ejecuta es una sentencia que hace una única operación, mientras que lo que está entre INICIO y FIN agrupa varias sentencias que se ejecutan en un orden determinado.

3.3. Pruebas de escritorio o trazas

Las pruebas de escritorio son simulaciones de ejecución de un algoritmo que permiten determinar la validez o efectividad del mismo.

Pruebas de escritorio o trazas

Consisten en escribir en una tabla con tantas columnas como variables tiene el algoritmo y seguir las instrucciones, registrando los valores correspondientes.

Lo invitamos a consultar el PDF “**Pruebas de escritorio o trazas**”, el cual se encuentra en la carpeta Anexos.

Ejemplo

Escribir un algoritmo que calcule el cuadrado y el cubo de un número introducido por teclado y, mostrar los resultados para el cubo y el cuadrado.

Solución

- **Análisis**

Nuevamente, el dato de entrada debe ser un número ingresado por el usuario, y al igual que en el ejemplo anterior, se debe solicitar al usuario que lo digite. Antes de realizar las operaciones de elevar al cuadrado y al cubo, se necesita definir las dos variables que recibirán estos datos (el cuadrado y el cubo). Por lo tanto, el algoritmo debe definir las variables antes de realizar las operaciones y asignarles a ellas los resultados.

Una vez realizadas las operaciones, el algoritmo debe mostrar en pantalla el contenido de estas variables, con un mensaje de texto que indique si es el resultado del cuadrado o del cubo.

- **Diseño**

El siguiente código tiene la representación del resultado al análisis anterior.

ALGORITMO Calcular el cuadrado y el cubo;

VAR

ENTERO entrada;

ENTERO cuadrado;

ENTERO cubo;

INICIO

ESCRIBIR ("Por favor ingrese un número");

LEER (entrada);

cuadrado $\leftarrow$ entrada * entrada;

cubo $\leftarrow$ entrada * entrada * entrada;

ESCRIBIR("El Cuadrado " cuadrado);

ESCRIBIR("El Cubo " cubo);

FIN

Las pruebas de escritorio permiten verificar que el pseudocódigo realice lo que se espera al resolver el problema. Es importante que a todos los pseudocódigos se les aplique, al menos, una prueba de escritorio o las que se consideren necesarias.

La mayoría de las pruebas de escritorio se representan en una tabla, donde en cada columna se anotan las variables y condiciones, si las hay, en el orden en que aparecen en el algoritmo. Para entenderlo, se debe recorrer línea a línea el pseudocódigo, identificando tanto las variables como las condiciones e ir anotándolas en la tabla de la prueba de escritorio.

Es importante que en la tabla no aparezcan variables o condiciones repetidas. En la última columna se pueden anotar los datos de salida, identificándolos como "Salida".

Ejecutando línea a línea el algoritmo del ejemplo anterior, se va llenando la tabla:

- **Paso 1**

ESCRIBIR ("Por favor ingrese un número");

Tabla 14. Tabla paso 1

Número	Cuadrado	Cubo	Salida
			Por favor ingrese un número

- **Paso 2**

LEER(entrada);

Tabla 15. Tabla paso 2

Número	Cuadrado	Cubo	Salida
3			Por favor ingrese un número 3

- **Paso 3**

cuadrado $\leq$ entrada * entrada;

Tabla 16. Tabla paso 3

Número	Cuadrado	Cubo	Salida
3	9		Por favor ingrese un número 3

- **Paso 4**

cubo $\leq$ entrada * entrada * entrada;

Tabla 17. Tabla paso 4

Número	Cuadrado	Cubo	Salida
3	9	27	Por favor ingrese un número 3

- **Paso 5**

ESCRIBIR (cuadrado);

Tabla 18. Tabla paso 5

Número	Cuadrado	Cubo	Salida
3	9	27	Por favor ingrese un número 3 El cuadrado 9

- **Paso 6**

ESCRIBIR (cuadrado);

Tabla 19. Tabla paso 6

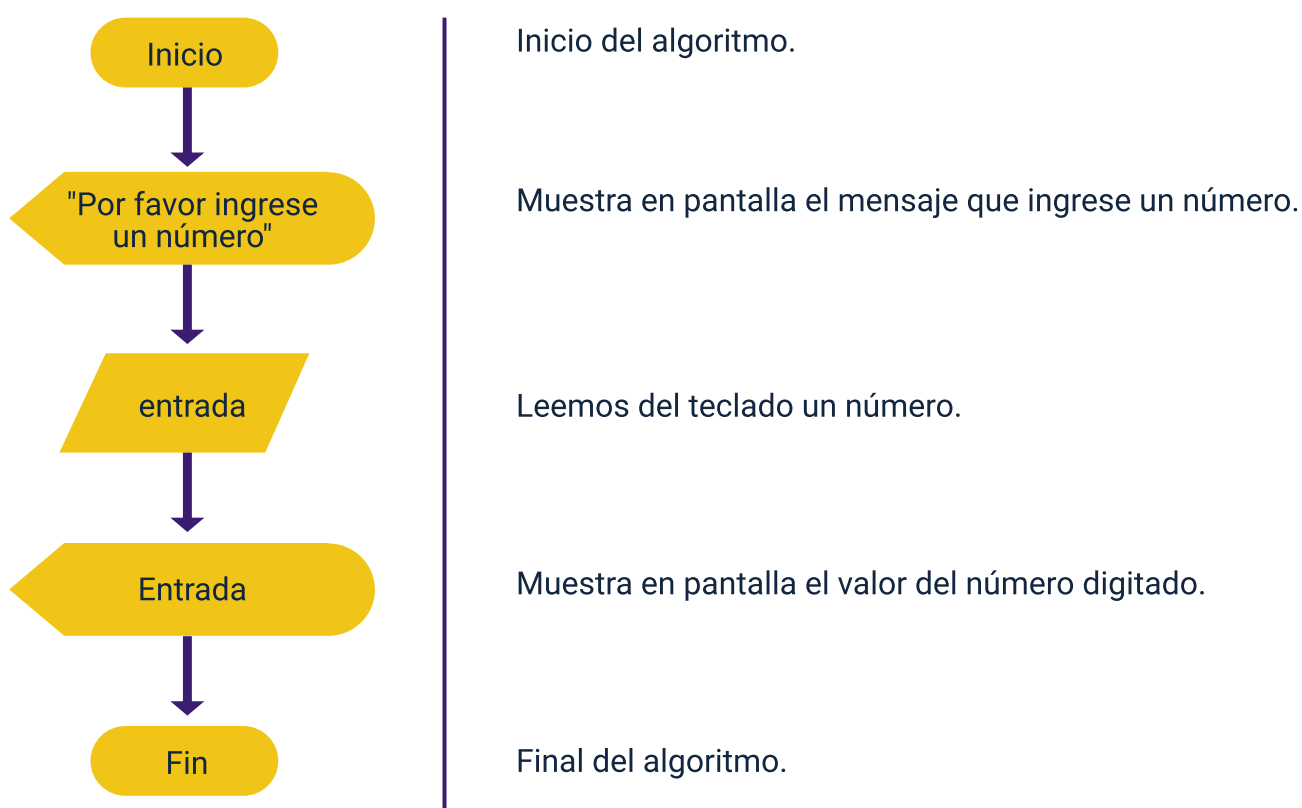
Número	Cuadrado	Cubo	Salida
3	9	27	Por favor ingrese un número 3 El cuadrado 9 El cubo 27

También es posible realizar pruebas de escritorio a otras representaciones de algoritmos, como los diagramas de flujo, expuestos a continuación. Se recomienda realizar la prueba de escritorio correspondiente a cada ejemplo presentado.

4. Diagramas de flujo

Una forma común de presentar los algoritmos es mediante diagramas de flujo, que consisten en una representación gráfica. Mientras que en pseudocódigo las sentencias se representaban con proposiciones, en los diagramas de flujo las proposiciones son reemplazadas por figuras.

Figura 6. Diagrama de flujo imprimir un entero



El diagrama de flujo ilustra los pasos de un algoritmo simple para imprimir un número entero. Comienza con un mensaje solicitando al usuario que ingrese un número, seguido de la lectura del número ingresado y finaliza con la impresión del valor digitado. Este proceso asegura que el usuario introduzca y vea el número correctamente.

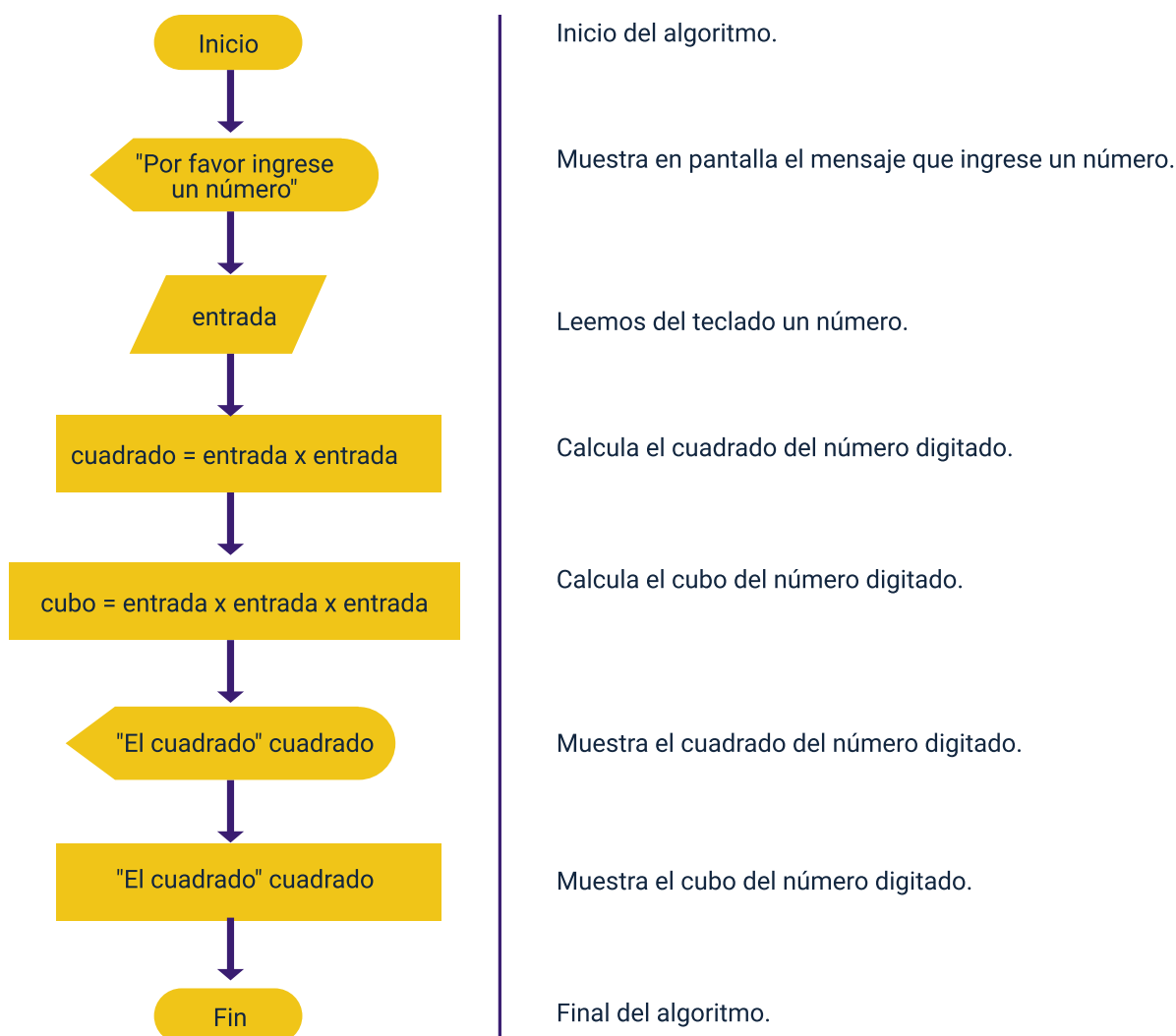
Ventajas de los diagramas de flujo

Es ideal revisar la siguiente infografía sobre las ventajas de la representación gráfica de un algoritmo a través del diagrama de flujo.

Lo invitamos a consultar el PDF “**Ventajas de los diagramas de flujo**”, el cual se encuentra en la carpeta Anexos.

La representación del algoritmo en la figura (algoritmo que calcula el cuadrado y el cubo de un número introducido por teclado y muestra los resultados):

Figura 7. Diagrama de flujo imprimir el cuadrado y el cubo de un entero



Este diagrama de flujo describe un algoritmo que comienza con la solicitud al usuario de ingresar un número. Luego, se calcula el cuadrado y el cubo del número ingresado, y finalmente se presentan estos resultados.

La Organización Internacional para la Estandarización (ISO, por su sigla en inglés) y el Instituto Nacional Americano de Estandarización (ANSI, por su sigla en inglés), estandarizaron los símbolos que han sido mayormente aceptados en 1985. Con el fin de evitar la utilización de símbolos diferentes para representar procesos iguales (López, 2009).

Los siguientes son los principales símbolos para crear diagramas de flujo:

- **Inicio/Final**

Se utiliza para indicar el inicio y el final de un diagrama; del Inicio solo puede salir una línea de flujo y al Final solo debe llegar una línea.

- **Decisión**

Indica la comparación de dos datos y dependiendo del resultado lógico (falso o verdadero) se toma la decisión de seguir un camino del diagrama u otro.

- **Entrada General**

Entrada/Salida de datos en General (en esta guía, solo se usará para la Entrada).

- **Iteración**

Indica que una instrucción o grupo de instrucciones deben ejecutarse varias veces.

- **Entrada por teclado**

Instrucción de entrada de datos por teclado. Indica que el computador debe esperar a que el usuario teclee un dato que se guardará en una variable o constante.

- **Salida Impresa**

Indica la presentación de uno o varios resultados en forma impresa.

- **Llamada a subrutina**

Indica la llamada a una subrutina o procedimiento determinado.

- **Salida en Pantalla**

Instrucción de presentación de mensajes o resultados en pantalla.

- **Acción/Proceso General**

Indica una acción o instrucción general que debe realizar el computador (cambios de valores de variables, asignaciones, operaciones aritméticas).

- **Conector**

Indica el enlace de dos partes de un diagrama dentro de la misma página.

- **Flujo**

Indica el seguimiento lógico del diagrama. También indica el sentido de ejecución de las operaciones.

- **Conector**

Indica el enlace de dos partes de un diagrama en páginas diferentes.

Elaborar diagramas de flujo

Cuando el algoritmo se desea representar como diagrama de flujo, se deben tener en cuenta algunos principios para su elaboración (Rojas y Ñacato, 1980).

Diagramas de flujo

A continuación, se puede conocer cómo se estructura el diagrama de flujo.

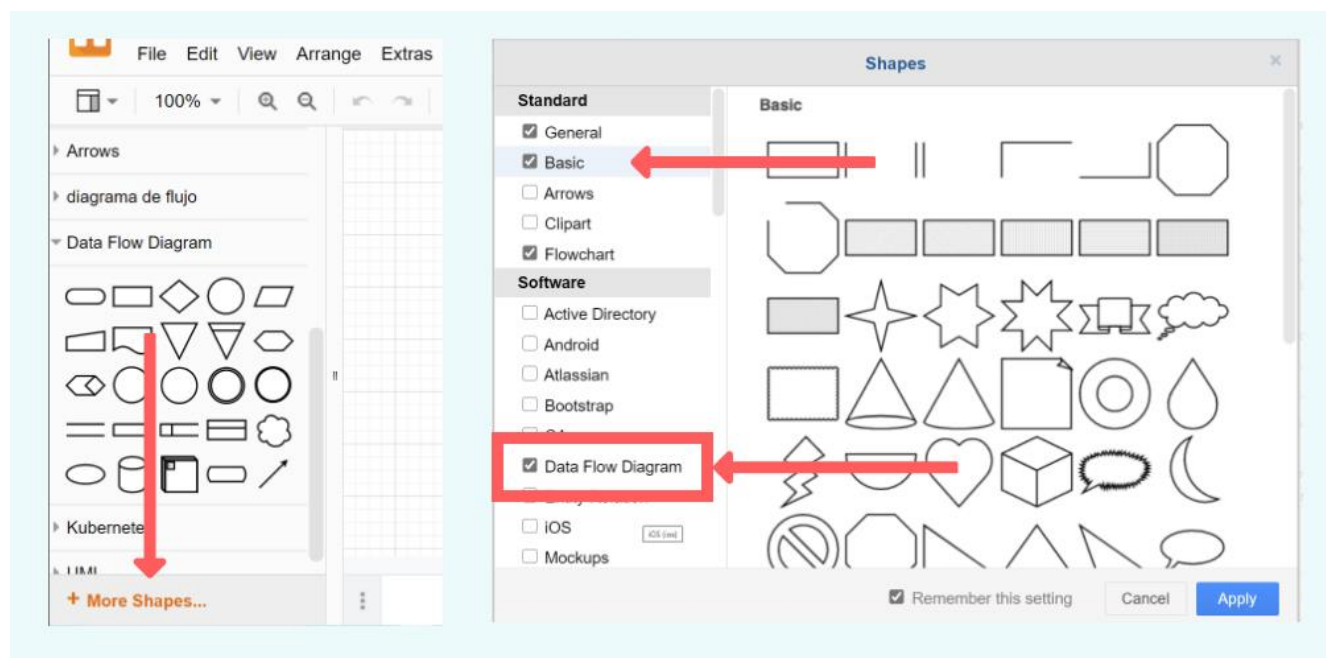
Lo invitamos a consultar el PDF “**Diagramas de flujo**”, el cual se encuentra en la carpeta Anexos.

Herramientas digitales para creación y prueba de algoritmos

Para aprender a reconocer los elementos que componen los diagramas de flujo, se puede hacer uso de la herramienta denominada Draw.io a través del enlace <https://app.diagrams.net/>. Con esta herramienta, puede usar una cuenta de correo de Google o misena.edu.co para acceder y crear un nuevo diagrama.

Es ideal ingresar y seleccionar los diagramas que se presentan en la siguiente figura.

Figura 8. Seleccionar formas



5. Uso de identificadores y palabras reservadas

En la elaboración de diagramas de flujo o algoritmos, muchas veces se necesitan datos auxiliares. A continuación, se definen algunos de estos tipos de datos auxiliares.

Los identificadores son nombres que se dan a los elementos utilizados para resolver un problema y poder diferenciar unos de otros. De acuerdo con López (2009), al asignar nombres (identificadores) a variables, constantes y procedimientos, se deben tener en cuenta algunas reglas, a saber:

- a) Pueden estar formados por una combinación de letras y números (saldoMes, salario, fecha2, etc.).
- b) El primer carácter de un nombre debe ser una letra.
- c) La mayoría de los lenguajes de programación diferencian las mayúsculas de las minúsculas.
- d) Los nombres deben ser nemotécnicos; con solo leerlos se debe entender lo que contienen.
- e) Deben ser muy descriptivos; no utilizar abreviaturas, a menos que se justifique.
- f) Es conveniente utilizar una sola palabra para nombrar páginas, controles, variables, etc.
- g) No utilizar caracteres reservados (% , + , / , > , etc.) ni letras acentuadas (á , é , í , ó , ú).
- h) No utilizar palabras reservadas por los lenguajes de programación.

Ejemplo:

Listar los identificadores usados en los algoritmos previos.

Solución

En el algoritmo de imprimir un número entero, se usa el identificador "entrada" para representar el número digitado. En el algoritmo de imprimir el cuadrado y el cubo de un número entero, se emplean los siguientes identificadores:

- a) "entrada" para representar el número digitado.
- b) "cuadrado" para representar el valor del cuadrado del número de entrada.
- c) "cubo" para almacenar el cubo del número de entrada.

Todos los lenguajes de programación tienen palabras reservadas para nombrar sus comandos, instrucciones y funciones; estas palabras no se pueden usar como identificadores.

- **Variables**

Para los algoritmos con diversos conjuntos de datos iniciales, se debe determinar una independencia entre los datos iniciales de un problema y la estructura de su solución. Esto se hace mediante la utilización de variables (cantidades que se suelen emplear y que durante la ejecución del algoritmo pueden tomar cualquier valor de un intervalo de valores posibles). Las variables son espacios de trabajo (contenedores) dispuestos para almacenar datos (valores). El valor de una variable puede ir cambiando en algún paso del algoritmo o permanecer sin cambios; por lo tanto, el valor de una variable es el último valor o dato asignado a esta.

- **Constantes**

Las constantes se crean de la misma forma que las variables, a diferencia de que se procura su definición o creación al inicio del algoritmo. Las constantes consisten en datos que, luego de ser asignados, no cambian en

ninguna instrucción del algoritmo; pueden contener constantes matemáticas (pi).

- **Contadores**

Un contador es una variable que está en ambos lados de una asignación (a la derecha y a la izquierda) interna, a la que se le suma un valor constante. Es necesario haberla iniciado al principio del algoritmo, ya que va a ser leído su contenido.

Ejemplo: definir un contador llamado contador que se incremente en uno cada vez tres veces.

ALGORITMO Contador;

VAR

ENTERO contador;

INICIO

contador = 0;

contador <= contador + 1;

contador <= contador + 1;

contador <= contador + 1;

ESCRIBIR (contador);

FIN

- **Acumuladores**

Los acumuladores son una versión ampliada de contadores. Tienen las mismas características que un contador, excepto que el valor que se va incrementando no es una constante, sino un valor variable.

6. Operadores y jerarquía en los operadores

Los operadores son símbolos que sirven para manipular datos. Y en un algoritmo es necesario dejar un espacio en blanco a cada lado del signo aritmético. Los operadores y las funciones que realizan se clasifican como se indica a continuación:

- **Aritméticos:**

Permiten realizar operaciones entre datos de tipo numérico y dan como resultado otro valor de tipo numérico.

Ejemplo: producto o multiplicación (*); división (/); suma (+); resta (-); asignación (=).

- **Alfanuméricos:**

Permiten operar con datos de tipo carácter (letras) o cadenas de texto. La mayoría de los lenguajes de programación admiten el operador + para realizar la concatenación (unión) de caracteres o cadenas.

- **Relacionales:**

Permiten la comparación entre datos del mismo tipo de dato y dan como resultado dos valores posibles: verdadero o falso.

Ejemplo: igual a (=); menor que (<); mayor que (>), mayor e igual que (>=).

- **Lógicos:**

Posibilitan la evaluación lógica de dos expresiones de tipo lógico. Dan como resultado uno de dos valores posibles: verdadero o falso.

Ejemplo: negación (no); conjunción (y); disyunción (o).

Jerarquía en los operadores

Las computadoras ejecutan los operadores en un orden establecido. El siguiente es el orden (jerarquía) para ejecutar operadores:

- a) Paréntesis (se ejecutan primero los más internos).
- b) Signo (e.g., -2) si un valor es positivo o negativo.
- c) Potencias (^) y Raíces (sqrt); Productos y Divisiones (* y /), en este mismo orden.
- d) Sumas y Restas (+ y -).
- e) Concatenación (+).
- f) Relacionales (=, <, >).
- g) Negación (no).
- h) Conjunción (y).
- i) Disyunción (o).

El siguiente ejemplo permite comprender mejor la jerarquía de operadores:

Tabla 20. Orden de ejecución de una expresión matemática

Orden	Operación	Explicación y expresión resultante
1	$7 + 8$	Primero que todo, lo que más valor tiene en la jerarquía son los paréntesis, en particular los más interiores. De esta forma, el computador se interesará en resolver primero el contenido de $(7 + 8)$, cuyo resultado es 15. De esta forma, la expresión queda reducida a: $(2 + 15 * 16 + 3 ^ 2) / 4 + 5$
2	$3 ^ 2$	De la expresión resultante luego de la operación 1, se observa que nuevamente hay un paréntesis por reducir, de manera que la computadora intentará resolver su contenido primero, como dentro

Orden	Operación	Explicación y expresión resultante
		del paréntesis está la expresión de elevar al cuadrado el número 3 (3^2) que tiene como resultado 9, la expresión resultante sería: $(2 + 15 * 16 + 9) / 4 + 5$
3	$15 * 16$	De los paréntesis de la expresión resultante la operación de más jerarquía es la multiplicación $15 * 16$ que tiene como resultado 240, la expresión resultante sería: $(2 + 240 + 9) / 4 + 5$
4	$2 + 24 + 9$	De la expresión resultante el que tiene mayor precedencia es el paréntesis, por lo tanto, la suma aritmética de su contenido data como expresión resultante: $251 / 4 + 5$
5	$251 / 4$	De la expresión resultante el operador de mayor jerarquía es la división y da como resultado 62.75 que en la siguiente operación se le sumará el valor de 5 la expresión resultante será: $62,75 + 5$
6	$62,75 + 5$	67,75

A continuación, se revisan dos ejemplos:

- **Ejemplo 1**

Una computadora debe ejecutar las siguientes operaciones. Se debe escribir en el orden que la computadora ejecutará las operaciones de la siguiente expresión matemática: $(2 + (7 + 8) * 16 + 3^2) / 4 + 5$.

Solución:

Primero que todo, lo que más valor tiene en la jerarquía son los paréntesis, en particular los más interiores. De esta forma, la computadora se interesará en resolver primero el contenido de $(7 + 8)$.

- **Ejemplo 2**

Escribir la siguiente ecuación como una expresión que la computadora pueda interpretar en el orden adecuado.

$$\frac{\sqrt{6^2/8^2 + 7}}{8^2} (\sqrt{(6^2/8^2) + 7})/8^2$$

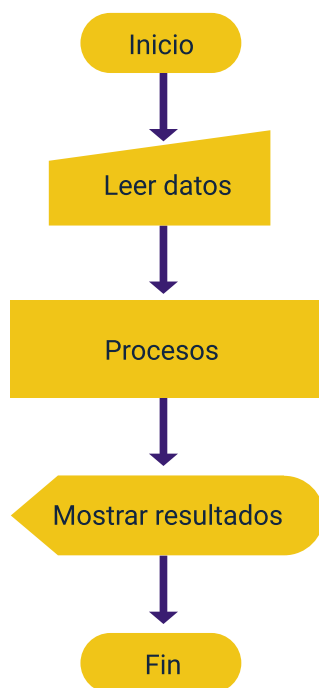
7. Estructuras de control secuencial

Una estructura se define como un esquema con determinada distribución y orden que permite representar una idea de forma simplificada. La estructura de control secuencial es la más sencilla y es conocida como estructura lineal. Se compone de instrucciones que deben ejecutarse de forma consecutiva, una tras otra, siguiendo una línea de flujo.

Los problemas más sencillos pueden resolverse haciendo uso únicamente de esta estructura. Por lo general, la estructura secuencial forma parte de soluciones a problemas complejos, en las que se utilizan otras estructuras iterativas (repetir varias veces un conjunto de instrucciones) y condicionales (López, 2009).

La siguiente figura presenta la estructura general del algoritmo más básico.

Figura 9. Modelo de estructura secuencial



Nota. Tomado y adaptado de The Development Stages (2024).

Este diagrama de flujo detalla un proceso básico que inicia con la lectura de datos, sigue con el procesamiento de los mismos, y termina presentando los resultados obtenidos.

Ejemplo estructura de control secuencial

Escribir un algoritmo para calcular el área de cualquier triángulo rectángulo, en él se debe pedir al usuario que ingrese los valores de la altura y la base del triángulo.

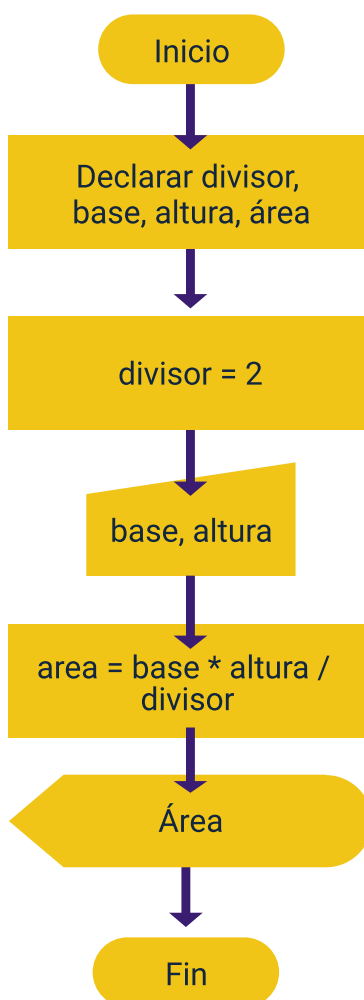
Análisis

- **Formular el problema:** está claramente planteado y consiste en calcular el área de un triángulo rectángulo.
- **Resultados esperados:** se espera que el algoritmo sea capaz de calcular el área del rectángulo y mostrar en la pantalla el resultado de ese cálculo.
- **Datos disponibles:** dos datos números que son la base y la altura del triángulo (se deben solicitar al usuario). El aprendiz debe preguntarse si sus conocimientos actuales de matemáticas le permiten resolver este problema, de no ser así, tendrá que plantear una estrategia para obtener los conocimientos requeridos. Para este ejemplo recordar que un triángulo rectángulo es aquel que uno de sus lados tiene un ángulo recto (90 grados).
- **Restricciones:** los valores de base y altura son variables y se deben solicitar al usuario.
- **Procesos necesarios:** definir variables; asignar el valor 2 a la constante usando para dividir (divisor); solicitar al usuario el valor de la altura del triángulo; solicitar al usuario el valor de la base; aplicar la fórmula de área; mostrar el resultado.

Diseño

El algoritmo resultado del análisis, en la que están representados tanto el diagrama de flujo como en pseudocódigo. Mientras se está aprendiendo a diseñarlos, es importante hacer los algoritmos en diagrama de flujo, y a medida que se va adquiriendo experticia con la representación en pseudocódigo, es más conveniente hacer esta representación debido a la velocidad a la hora de diseñarlo.

Figura 10. Diagrama al área de un triángulo



El diagrama de flujo para calcular el área de un rectángulo comienza declarando las variables necesarias: divisor, base, altura y área. El divisor se asigna un valor de 2.

Luego, se ingresan los valores de la base y la altura del rectángulo. El área se calcula utilizando la fórmula: $\text{área} = (\text{base} * \text{altura}) / \text{divisor}$. Finalmente, se presenta el valor calculado del área y el algoritmo termina. Este proceso asegura que el cálculo del área se realice de manera correcta.

ALGORITMO área de un triángulo;

VAR

ENTERO divisor;

REAL area;

REAL base;

REAL altura;

INICIO

divisor = 2;

ESCRIBIR("Escriba la base ");

LEER (base)

ESCRIBIR("Escriba la altura ");

LEER(altura);

area = base * altura / divisor;

ESCRIBIR(area);

FIN

Del diagrama de flujo y el pseudocódigo se puede notar lo siguiente: el identificador "area" no tiene tilde por la razón antes mencionada. El tipo de dato REAL hace referencia a que es un número que no solamente puede contener un valor entero, sino también cifras decimales.

7.1. Estructura condicional

Es importante prestar atención a las estructuras que se utilizan para resolver problemas y poderlas reconocer para lograr mayor control sobre la solución planteada. La estructura condicional se emplea para indicarle al computador que debe evaluar una condición y, según el resultado, ejecutar el bloque de instrucciones correspondiente.

La forma más común está compuesta por una condición que se evalúa y dos bloques de instrucciones que se ejecutan: uno cuando la condición es verdadera y otro cuando la condición es falsa.

Ejemplo:

Un aprendiz aprueba un examen cuando la calificación de este es mayor o igual a 3. Elaborar un algoritmo que pida al usuario una calificación, aplicar el criterio de aprobación e imprimir “Aprobado” o “Reprobado”, según sea el caso.

Análisis

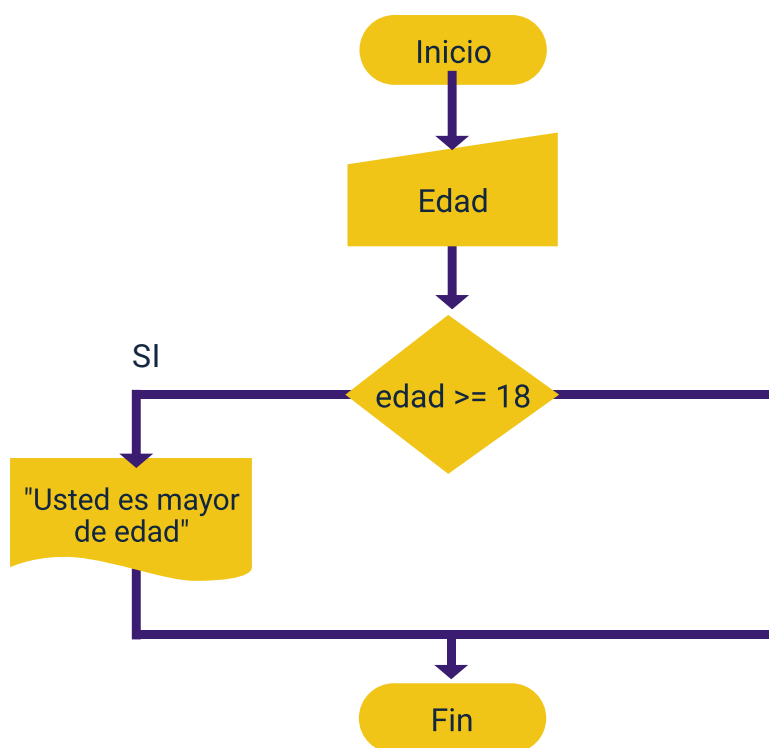
- **Formular el problema:** es un problema sencillo solo consiste en usar una sentencia de repetición.
- **Resultados esperados:** el mensaje “Esto es un algoritmo” 7 veces impreso en pantalla.
- **Datos disponibles:** constante con el número de repeticiones (7).
- **Restricciones:** ninguna.

- **Procesos necesarios:** necesarios: hacer que se imprima en pantalla el mensaje 7 veces.

Diseño

Como se presenta en la figura, no se ejecuta nada en caso de que NO se cumpla la condición. Este tipo de estructura condicional se denomina estructura condicional simple. Además, todo el proceso (pedir la edad, comprobar la condición, mostrar o no el mensaje) se realiza una única vez y finaliza la ejecución del algoritmo. Si se deseara que el programa repitiera estas mismas operaciones más de una vez, se tendría que usar una estructura de iteración, como se verá a continuación.

Figura 11. Estructura condicional simple



Este diagrama presenta un algoritmo que verifica si una persona es mayor de edad. El proceso inicia con la entrada de la edad, y si la edad es mayor o igual a 18, se indica que la persona es mayor de edad.

ALGORITMO Calificación del usuario.

ALGORITMO Edad del usuario;

VAR

REAL edad;

INICIO

ESCRIBIR("Escriba la calificación");

LEER (edad)

SI (edad $\geq$ 18)

ESCRIBIR ("Usted es mayor de edad");

FINSI

FIN

7.2. Estructuras de iteración o repetitivas

Las estructuras iterativas o de repetición permiten ejecutar una o varias instrucciones un número determinado de veces o indefinidamente hasta que se cumpla una condición. Esta estructura ayuda a simplificar los algoritmos.

En programación existen, como mínimo, dos tipos de estructuras repetitivas, las cuales tienen variantes en los diferentes lenguajes de programación. La característica común es que ambos tipos permiten ejecutar una o varias instrucciones:

- Un número conocido de veces (Estructura PARA o FOR en inglés).
- Mientras se cumpla una condición (Estructura MIENTRAS o WHILE).

Estructura PARA (FOR)

Esta estructura se usa cuando se sabe el número de veces que se debe ejecutar un conjunto de operaciones.

Ejemplo:

Escribir un procedimiento que muestre siete (7) veces en pantalla la frase “Esto es un algoritmo”. Mostrar en pantalla la salida del en el computador.

Análisis

- **Formular el problema:** es un problema sencillo solo consiste en usar una sentencia de repetición.
- **Resultados esperados:** el mensaje “Esto es un algoritmo” 7 veces impreso en pantalla.
- **Datos disponibles:** constante con el número de repeticiones (7).
- **Restricciones:** ninguna.
- **Procesos necesarios:** hacer que se imprima en pantalla el mensaje 7 veces.

Diseño

Como se presenta en el algoritmo de la siguiente figura, existe una variable *i* tendrá el valor 1, 2, 3 ... hasta 7, esta variable, se puede usar dentro del el bloque PARA hasta FINPARA.

Figura 12. Diagrama al área de un triángulo



El diagrama muestra un algoritmo que utiliza un bucle 'para' que se ejecuta de 1 a 7. En cada iteración del bucle, se presenta un mensaje indicando que es un algoritmo, y el proceso termina después de completar el bucle.

ALGORITMO Ciclo para;

INICIO

PARA i DESDE 1 HASTA 7

ESCRIBIR("Esto es un algoritmo");

FINPARA

FIN

La salida en pantalla sería:

Esto es un algoritmo

Esto es un algoritmo

Esto es un algoritmo

Esto es un algoritmo

Esto es un algoritmo

Esto es un algoritmo

Esto es un algoritmo

Ahora se presenta un ejemplo un poco más elaborado.

Ejemplo:

Se necesita elaborar un algoritmo que solicite al usuario un número entero y sume todos los números naturales que hay hasta ese número. Por ejemplo, si el usuario digita 3, el programa debe sumar: $1 + 2 + 3$, si el usuario digita 5 el programa debe sumar $1+2+3+4+5$. Al finalizar debe imprimir el resultado.

Análisis

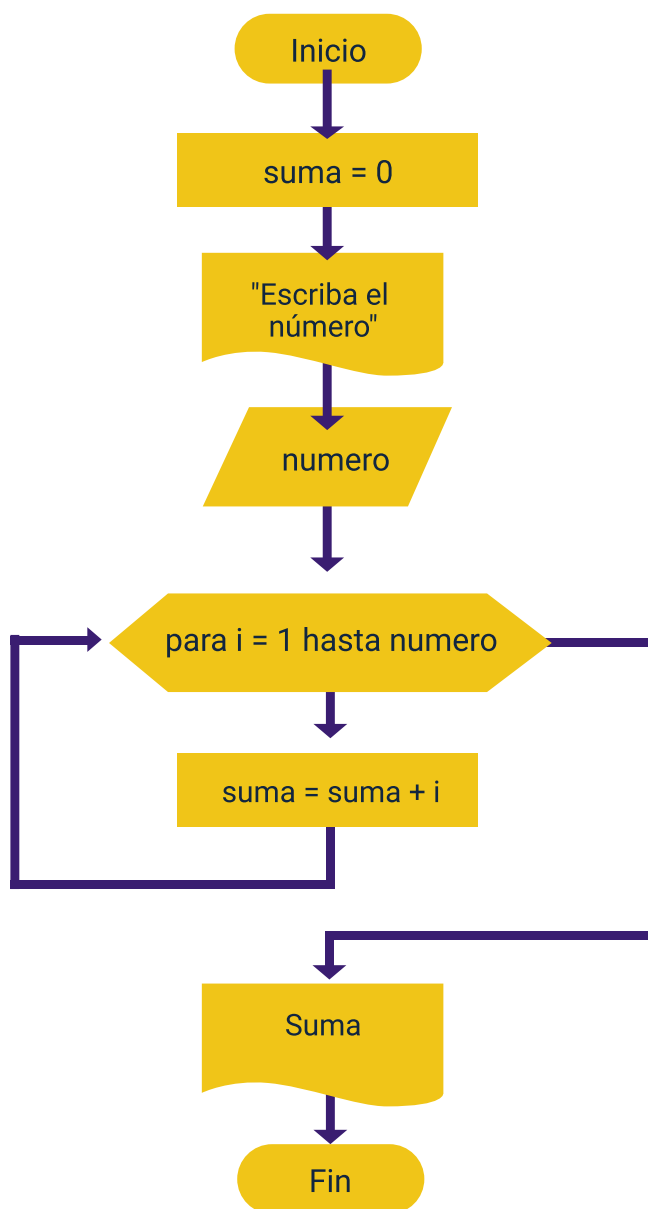
- **Formular el problema:** necesita la sumatoria de los números en secuencia natural.
- **Resultados esperados:** la suma de los números en sucesivos de uno hasta el número digitado por el usuario.
- **Datos disponibles:** el dato entero que digita el usuario es la entrada.

- **Datos adicionales:** se necesita un acumulador donde ir guardando el valor de las sumas sucesivas.
- **Restricciones:** ninguna.
- **Procesos necesarios:** definir un acumulador con valor inicial cero, imprimir para que el usuario sepa que debe digitar un número (“Escriba el número”), Leer del teclado el número del usuario, sumar cada vez el dato que se va incrementando de uno en uno al valor digitado por el usuario.

Diseño

La siguiente figura presenta el algoritmo en sus dos representaciones más comunes, se propone en secciones posteriores hacer una prueba de escritorio a este problema.

Figura 13. Estructura de repetición FOR acumulador suma



Este diagrama describe un algoritmo que suma los números desde 1 hasta un número especificado por el usuario. El algoritmo empieza con la inicialización de la suma en 0, solicita el número al usuario, y luego realiza la suma iterativa, finalizando con la presentación del resultado.

ALGORITMO sumas sucesivas;

VAR

ENTERO número;

ENTERO suma;

INICIO

suma = 0;

ESCRIBIR("Escriba el número");

LEER (número)

PARA i DESDE 1 HASTA numero

suma = suma + i;

FINPARA

ESCRIBIR (suma);

FIN

Estructura MIENTRAS (WHILE)

La instrucción “Mientras”, puede impedir la ejecución de un conjunto de instrucciones, si la evaluación de una condición lógica es falsa. Esto significa que es repetitiva únicamente cuando la evaluación de la condición es verdadera.

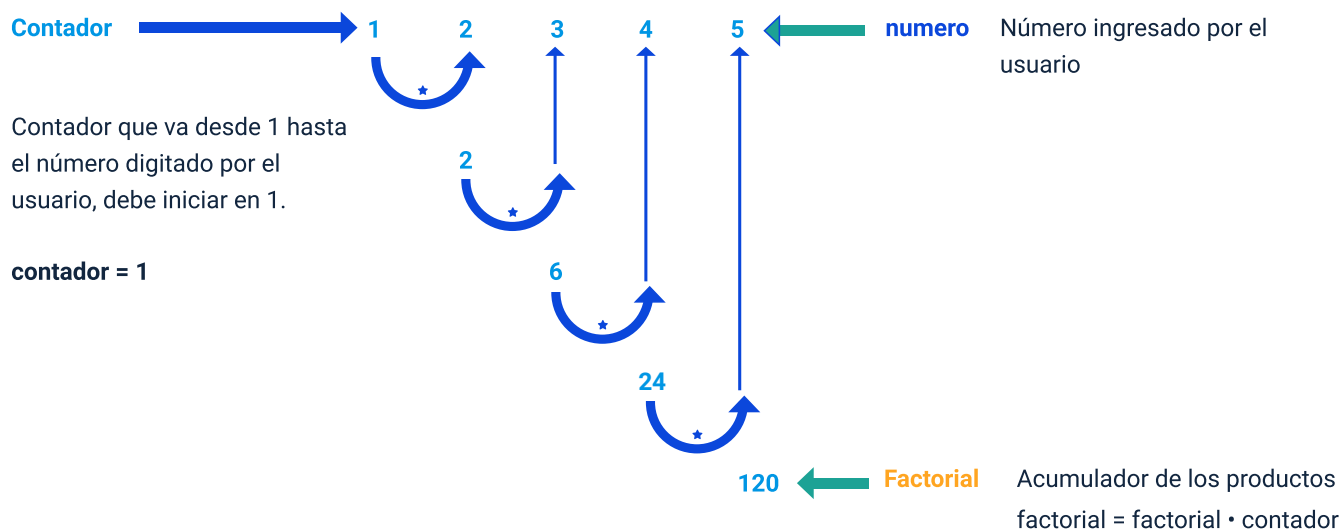
Ejemplo:

Realice un programa que permita obtener la factorial de un número dado por el usuario.

Análisis

- **Formular el problema:** la factorial de un entero positivo n , se define como el producto de todos los números enteros positivos desde 1 hasta n .
- **Ejemplo:** ¡Matemáticamente $5! = 5 * 4 * 3 * 2 * 1$ o a la inversa: $5! = 1 * 2 * 3 * 4 * 5$.
- **Resultados esperados:** la factorial del número digitado por el usuario.

Figura 14. Proceso de factorial



Esta figura ilustra el proceso de cálculo del factorial de un número. El contador comienza en 1 y se incrementa hasta el número ingresado por el usuario, que en este caso es 5. En cada paso, el acumulador del factorial multiplica su valor actual por el valor del contador. El proceso sigue así: 1 , $1 \times 2 = 2$, $2 \times 3 = 6$, $6 \times 4 = 24$, y $24 \times 5 = 120$, resultando en el factorial de 5 que es 120. Este diagrama muestra cómo se realiza la acumulación de los productos a lo largo del proceso, ejemplificando claramente el funcionamiento del cálculo del factorial.

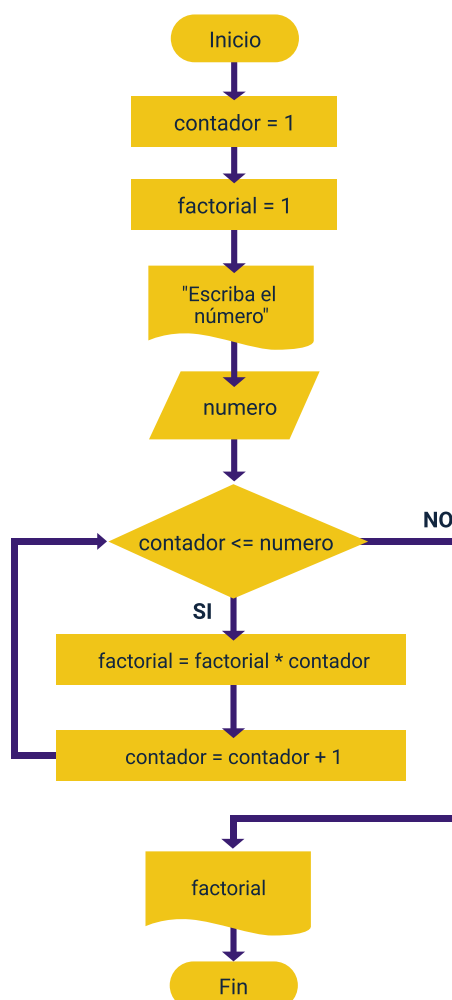
Análisis

- **Datos disponibles:** el dato entero que digita el usuario es la entrada.
- **Datos adicionales:** se requiere de un acumulador donde ir guardando el valor de las multiplicaciones sucesivas, también un contador para ir contando el número secuencial (como lo hace el número i en el ciclo PARA).
- **Restricciones:** la repetición se detiene cuando.
- **Procesos necesarios:** definir un acumulador con valor inicial cero, imprimir para que el usuario sepa que debe digitar un número (“Por favor el número”), leer del teclado el número del usuario, multiplicar cada vez el dato que se va incrementando de uno en uno al valor digitado por el usuario.

Diseño

El diseño es el siguiente:

Figura 15. Algoritmo factorial ciclo mientras



Este diagrama de flujo describe un algoritmo que calcula el factorial de un número ingresado por el usuario. El proceso comienza con la inicialización de las variables contador y factorial en 1. Luego, el usuario ingresa un número. Se utiliza un bucle para multiplicar el valor actual del factorial por el contador, incrementando el contador en 1 en cada iteración, hasta que el contador sea mayor que el número ingresado. Finalmente, se presenta el valor del factorial calculado.

ALGORITMO factorial;

VAR

ENTERO contador;

ENTERO factorial;

INICIO

contador = 1;

factorial = 1;

ESCRIBIR("Escriba el número");

LEER (numero);

MIENTRAS

factorial = factorial * contador;

contador = contador + 1;

FINMIENTRAS

ESCRIBIR(factorial);

FIN

8. Estructura de datos

Es importante conocer el concepto de arreglos e identificar cuándo usarlos en la aplicación de los algoritmos. Asimismo, es crucial ser capaz de resolver problemas básicos mediante diagramas de flujo y pseudocódigo. Existen varios tipos de arreglos, siendo los más comunes los vectores y las matrices.

Vectores

Los arreglos o arrays (en inglés), también conocidos como matrices, arreglos o vectores, se definen como un conjunto finito y ordenado de elementos del mismo tipo. La propiedad "ordenado" implica que el primer elemento, segundo, tercero, ..., enésimo de un array puede ser identificado y accedido. Cuando se dice que los elementos de un arreglo son homogéneos, significa que son del mismo tipo de datos. Un arreglo puede estar compuesto de todos sus elementos de tipo letras, otro puede tener todos sus elementos de tipo entero, etc. Los arreglos se conocen como matrices en matemáticas y tablas en temas financieros (Vázquez, 2012).

Según sus dimensiones, los arreglos se clasifican en:

- **Unidimensionales:**

De una dimensión, comparables con una lista de elementos.

- **Multidimensionales:**

Son arreglos de varias dimensiones o los más comunes son bidimensionales (dos dimensiones o tablas) o arreglos multidimensionales (tridimensionales, por ejemplo).

Figura 16. Matriz unidimensional con 10 elementos

0	1	2	3	4	5	6	7	8	9

Cada vector está compuesto de un número de elementos ya definidos, lo que hace que sea una estructura estática (no puede aumentarse ni reducirse). Cada elemento está referenciado por la posición que ocupa dentro del vector. Estas posiciones son llamadas índices; nótese que empiezan a contarse desde 0, como se presenta en la figura anterior.

Para referirse a un vector se usa un identificador, como se hace con las variables o constantes, pero queda entre corchetes []. Por ejemplo, si el vector se llama "lista" y se quiere acceder al quinto elemento del vector de la figura 17, se debe hacer así: lista[4]. Es importante notar que la quinta posición se accede con el índice 4, porque se empieza a contar desde 0.

Para realizar las operaciones de asignación, lectura/escritura sobre un vector, se recomienda usar las estructuras repetitivas para, a través de ellas, manipular los índices correspondientes a las posiciones del arreglo.

Ejemplo:

Hacer un programa que pida 5 números enteros al usuario y los guarde en un arreglo de números en un arreglo, luego que imprima los números mayores de 100.

A continuación, se revisan los ejemplos:

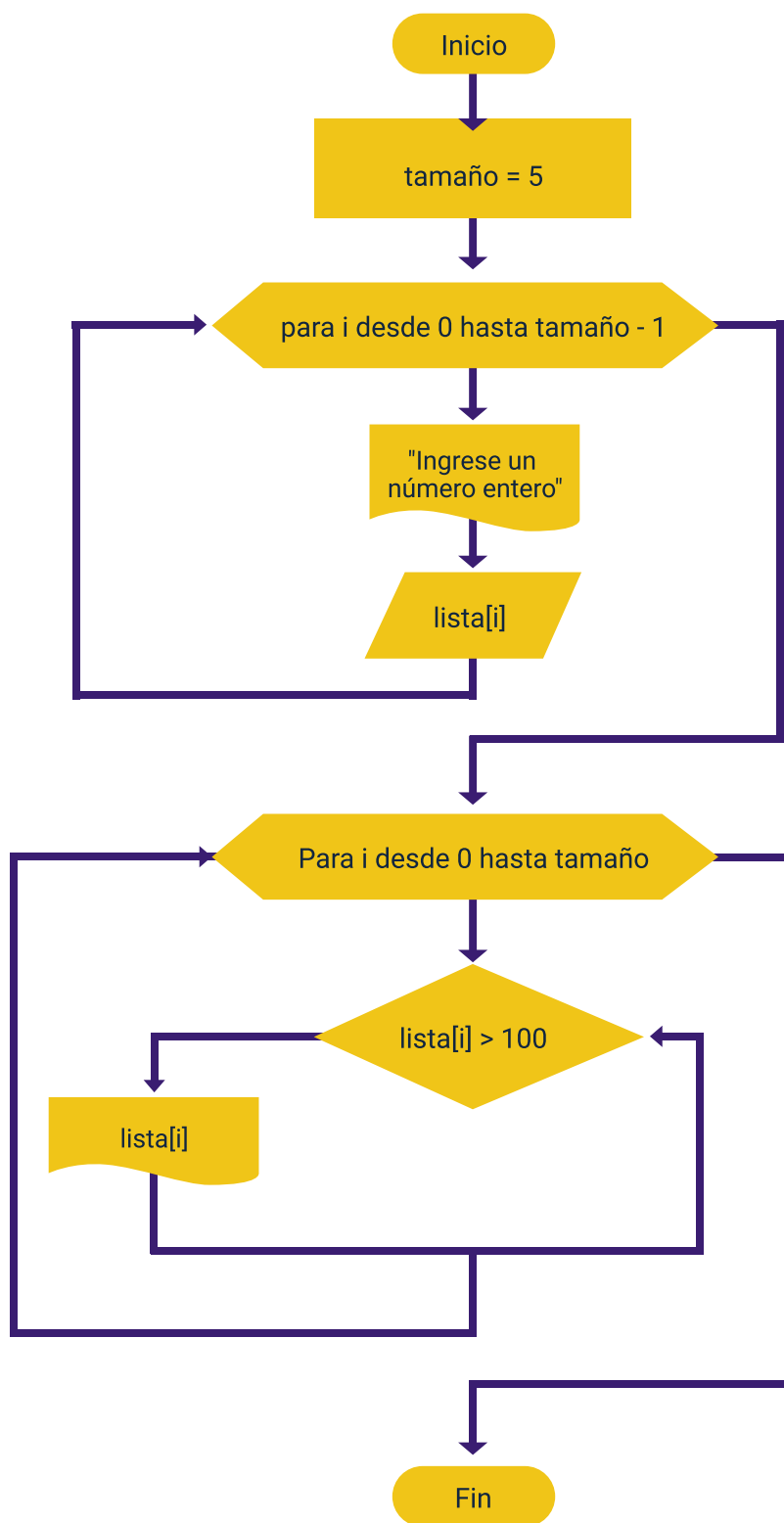
Análisis

- **Formular el problema:** pedirle al usuario 5 números enteros almacenarlos en un arreglo y luego recorrer el arreglo para buscar los números mayores a 100.
- **Resultados esperados:** imprimir en pantalla de los 5 números ingresados por el usuario solamente los mayores a 100.
- **Datos disponibles:** 5 números ingresados por el usuario.
- **Datos adicionales:** por cada uno de esos números validar si son mayores que 100 y mostrar los que lo sean en la pantalla.
- **Restricciones:** ninguna.
- **Procesos necesarios:** se requiere definir un arreglo de tamaño 5, se necesita pedir 5 veces un entero y almacenarlos en el vector. Luego se debe recorrer las 5 posiciones del vector para comparar en cada uno de los datos si es mayor que 100, si el número comparador es lo que se debe imprimir. Como 5 es un número invariante, el programa lo define como una constante de nombre tamaño (no se usa la ñ porque no permite ni acentos latinos ni tildes), se debe usar la sentencia repetitiva PARA porque se sabe el número de veces que se debe repetir cada acción.

Diseño

El diseño es el siguiente:

Figura 17. Vector de 5 elementos



Este diagrama de flujo muestra un algoritmo que permite al usuario ingresar una lista de 5 números enteros. Luego, recorre la lista y verifica si algún número es mayor que 100. Si encuentra un número que cumple esta condición, lo presenta. El proceso termina después de revisar todos los números de la lista.

ALGORITMO Vector de 5 enteros;

VAR

ENTERO tamano;

ENTERO lista[tamano] ;

INICIO

tamano = 5;

PARA i DESDE 0 HASTA tamano -1

ESCRIBIR("Escriba el número");

LEER (lista[i]);

FINPARA

PARA i DESDE 0 HASTA tamano -1

SI (lista[i] > 100)

ESCRIBIR (lista[i]);

FINSI

FINPARA

FIN

Matrices

Las matrices son arreglos de más de una dimensión. En la siguiente tabla se presenta un ejemplo de una matriz de 2 dimensiones, que es de 4 x 5 (correspondiente a 4 columnas y 5 filas).

Tabla 21. Matriz de 4 x 5

Columna 1	Columna 2	Columna 3	Columna 4
0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3
4,0	4,1	4,2	4,3

Lo anterior se aplica cuando se quiere acceder a un elemento de la matriz. El identificador usa corchetes dobles. Por ejemplo, para acceder al último elemento de la tabla, se utilizaría: `matriz[4][3]`.

Para recorrer estas estructuras de datos se requieren ciclos anidados; estas técnicas se abordarán más adelante.

Programación modular

Existe un área denominada ingeniería del software, que se interesa en el proceso de creación o producción de software y concentra sus esfuerzos en aportar herramientas, procedimientos y técnicas para su construcción.

La ingeniería del software propone que la construcción de software esté basada en la descomposición de un problema en un conjunto de subproblemas

independientes entre sí, más sencillos de analizar y resolver; estos subproblemas pueden ser tratados separadamente unos de otros. Esto consiste, básicamente, en la modularidad del software.

Debido a la modularidad, se pueden probar los subprogramas o módulos de manera independiente, depurando errores antes de su uso en el programa principal y almacenándolos para su posterior reutilización cuantas veces sea necesario.

El enfoque básico de modularidad en un algoritmo puede estar compuesto de otros algoritmos que se especializan en una función determinada. Para saber cómo descomponer un algoritmo en pequeños algoritmos, se listan los aspectos que pueden ayudar a definir la división de algoritmos:

- **Según la funcionalidad**

Un algoritmo debe realizar una función específica y no ir más allá de sus responsabilidades. Por ejemplo, si se tiene un módulo que calcula la multiplicación de dos números y devuelve el resultado, no sería adecuado que, además, imprima mensajes en pantalla o haga otra cosa diferente de su propósito.

- **Según su identificación**

Cada módulo tiene un nombre de identificación que permite ejecutarlo y diferenciarlo de otros. Es recomendable que al elegir este nombre sea representativo de la función que realiza, de esa forma resulta intuitivo en el momento de usarlo dentro de un algoritmo.

- **Según sus datos de entrada**

Los módulos comúnmente reciben datos de entrada necesarios para realizar las operaciones requeridas. Se pueden clasificar los módulos de

acuerdo con los parámetros que se requieren para obtener los resultados solicitados. Al momento de usar el módulo, se tienen que agregar todos los parámetros necesarios, en el orden establecido y deben ser del tipo correcto.

- **Parámetro de salida**

También se pueden emplear los resultados a la salida del algoritmo para clasificar los módulos, ya que en el momento de definir un algoritmo se debe indicar el tipo de dato que devuelve al momento de la ejecución del algoritmo.

- **Pruebas de escritorios**

Como se mencionó antes, las pruebas de escritorio sirven para depurar y probar que un algoritmo solucione el problema. En esta sesión se mencionarán los objetivos principales y se ejecutará una prueba a un algoritmo de mayor complejidad para su comprensión.

Ejemplo

Realizar un algoritmo que solicite dos números e imprima cuál es el mayor de ambos, si son iguales, mostrar cualquiera.

Análisis

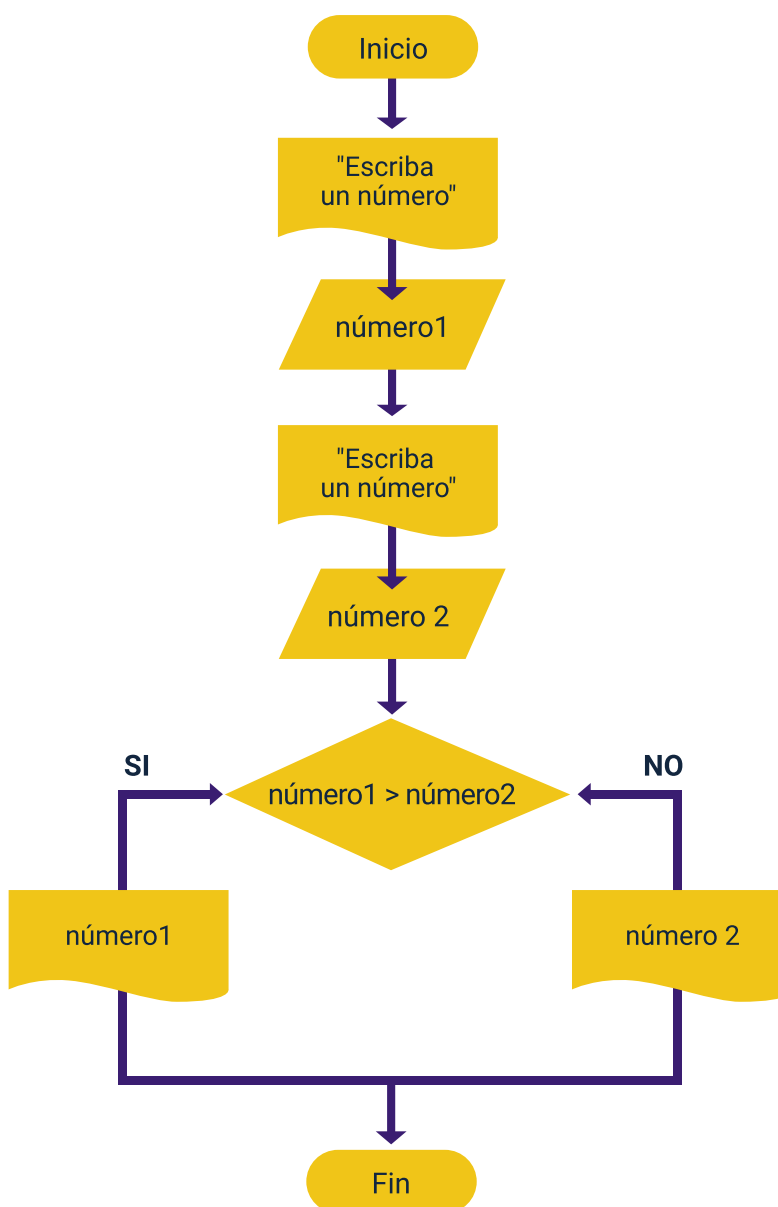
- **Formular el problema:** es un algoritmo que solita y compara dos números para mostrar el mayor.
- **Resultados esperados:** el número que es mayor de ambos.
- **Datos disponibles:** 2 números ingresados por el usuario.
- **Datos adicionales:** ninguno.
- **Restricciones:** validar cuál número es mayor para saber cuál mostrar.

- **Procesos necesarios:** se requiere definir los datos a recibir, pedirle que al usuario que digite los números, comparar para saber cuál es el mayor y mostrarlo.

Diseño:

El diseño es el siguiente:

Figura 18. Estructura Algoritmo el mayor de dos números



Este diagrama de flujo representa un algoritmo que solicita al usuario ingresar dos números. Luego, compara los dos números y determina cuál es el mayor. El resultado, el número mayor, se presenta antes de finalizar el algoritmo.

ALGORITMO El mayor de dos números;

VAR

ENTERO numero1;

ENTERO numero2;

INICIO

ESCRIBIR ("Escriba un número");

LEER (numero1);

ESCRIBIR("Escriba un número");

LEER (numero2);

numero1 > numero2

ESCRIBIR(numero1);

SINO

ESCRIBIR(numero2);

FINSI

FIN

9. Conceptos básicos de programación

Programar es el proceso de crear software escribiendo, probando, depurando y dando mantenimiento a las instrucciones del computador en un lenguaje de programación. A este conjunto de instrucciones se le denomina código fuente del software creado.

Existen centenares de lenguajes de programación, muchos de ellos fueron creados para dar respuesta a problemas particulares o máquinas específicas. No es de interés ahora discutir todos los tipos de lenguajes existentes, sino, más bien, conocer aquellos que pueden ser interpretados por computadores, smartphones o que pueden ser empleados para proveer servicios informáticos a través de internet y su clasificación más general.

Un lenguaje de programación es diferente al lenguaje de códigos que puede entender la máquina (lenguaje de máquina). Los lenguajes de programación pueden dividirse en dos categorías: lenguajes interpretados o lenguajes compilados, a saber:

- El código creado debe traducirse de manera que el procesador del computador o smartphone pueda comprenderlo, a este proceso se le llama compilación.
- El lenguaje compilado necesita que antes de ser ejecutado, se convierta el código fuente escrito a un lenguaje de máquina.

Las características son:

- **Primero**

Para un lenguaje compilado, luego de compilar un programa se deben crear ejecutables para cada uno de los sistemas operativos en los que se

vaya a utilizar. Es decir, un programa compilado en Linux no servirá en Windows o en Mac OS.

- **Segundo**

Cuando se está ejecutando un programa escrito en un lenguaje compilado, es mucho más rápido que uno interpretado, dado que se encuentra en código de máquina.

- **Tercero**

Los lenguajes compilados están optimizados para el momento de la ejecución, pero esto significa más trabajo para el programador (compilarlo).

JavaScript, al igual que cualquier otro lenguaje de programación, tiene algunas características especiales: sintaxis, modelo de datos, etc. JavaScript es un lenguaje esencial tanto para cualquier principiante en programación como para quienes ya saben programar, debido a que las diferencias con otros lenguajes de programación son numerosas. Es un lenguaje necesario para el desarrollo de aplicaciones web y móviles hoy en día.

En adelante, JavaScript será el lenguaje de programación abordado. Es un lenguaje interpretado que inicialmente fue creado para que los navegadores lo interpreten, y hoy en día muchos dispositivos tienen el intérprete que permite correr programas escritos en él. Por lo tanto, será más sencillo el trabajo tanto para la máquina como para el programador.

Entornos de codificación e instalación JavaScript.

En su mayoría, los navegadores (Chrome, Mozilla, Edge) tienen intérprete de JavaScript y, adicionalmente, se puede instalar el intérprete para el sistema operativo de preferencia. (Framebits, 2020)

Dado lo anterior, existen herramientas, manuales y videos que se emplean para el desarrollo de programas en JavaScript.

10. Sintaxis del lenguaje JavaScript

En adelante, se presentan las características básicas de la sintaxis de JavaScript, codificando algoritmos básicos para ejemplificar la estructura propia del lenguaje de programación. Sin embargo, se recomienda que, siempre que se esté codificando algoritmos, se tenga a disposición la documentación técnica oficial del lenguaje y el material complementario propuesto.

En JavaScript se pueden utilizar muchas funciones matemáticas. Por ejemplo, un algoritmo que toma dos números (8 y 6) y les aplica las 4 operaciones básicas puede servir para revisar las características de la sintaxis de JavaScript y conocer las operaciones matemáticas del lenguaje.

Para ello, ingresar en la ruta del navegador a <https://playcode.io/new> y escribir el código de la línea 1 a la 12 en la pestaña script.js, como se presenta a continuación:

1. `let numero1 = 8;`
2. `let numero2 = 6;`
3. `var suma = numero1 + numero2;`
4. `var resta = numero1 - numero2;`
5. `var producto = numero1 * numero2;`
6. `var division = numero1 / numero2;`

```
7. console.log("la suma es: " + suma );  
8. console.log("la resta es: " + resta );  
9. console.log("el producto es: " + producto );  
10. console.log("la división es: " + division );
```

Consola

la suma es: 14

la resta es: 2

el producto es: 48

la división es: 1.3333333333333333

En las líneas 1 y 2 aparece la palabra reservada `let` que sirve para indicar que se define una variable accesible en el contexto donde se crea (cuando se vea programación modular quedará más claro esto del contexto).

Las palabras reservadas `numero1` y `numero2` se les asigna el valor correspondiente. Es de conocer que se definen y se inicializa la variable en una misma línea, esto es una característica del lenguaje.

Luego en las líneas 3 al 6 se crean o se definen unas variables con la palabra reservada `var`, a diferencia de la palabra `let` estas variables se pueden ver desde cualquier parte o módulo del script.

Una vez se crean las variables con identificadores suma, resta, producto, división, también son inicializados sus valores empleando las operaciones matemáticas respectivas.

Una de las formas que JavaScript permite mostrar los resultados es con la función `log` del objeto `console` y se accede `console.log()` (con el operador punto `.`) y entre paréntesis lo que se desea mostrar en la pantalla de la consola.

Validación:

Dentro de los paréntesis `"la suma es: " + suma`, aparece el operador `+`, usado anteriormente para sumar dos números. Sin embargo, en este caso, no hay dos números, sino una cadena de texto y un número. Aquí, el operador `+` concatena o junta el texto `"la suma es:"` y el valor de la variable `suma`. Esta característica de los operadores se llama sobrecarga de operadores, porque según el contexto, el operador `+` puede sumar o concatenar.

Cuando las operaciones matemáticas son más complejas, como la raíz cuadrada, la función exponencial, o las funciones trigonométricas, existe un objeto que agrupa estas funciones llamado `Math`. Puede ver la documentación técnica completa de esta clase en `Math` (MDN, 2021b). A continuación, se presenta un ejemplo de su uso.

Se debe crear un algoritmo que muestre el seno del valor π (3.1416) multiplicado por 3 y que también eleve 6 a la tercera potencia, mostrando su resultado:

```
var angulo = Math.PI * 3;  
  
console.log('seno(3PI) =' + Math.sin(angulo));
```

```
var potencia = Math.pow(6, 3);  
  
console.log('6^3 =' + potencia);
```

Consola

seno(3PI) = -3.6739403974420594e-16

6^3 = 216

Se debe escribir el código fuente, y obtener los mismos resultados en la consola, como se presentan en cada imagen, de esta forma se familiariza con la herramienta de trabajo.

Funciones clase Math

A continuación, puede revisar el listado de funciones de la clase Math.

Lo invitamos a consultar el PDF “**Funciones clase Math**”, el cual se encuentra en la carpeta Anexos.

Todas las funciones matemáticas se pueden combinar con otras operaciones aritméticas o emplear operadores de evaluación de condición. Previo a ello, es preciso conocer los diferentes tipos de datos que existen.

10.1. Tipos de datos, operadores y orden de evaluación

Existen muchos tipos de operadores para JavaScript, cuya función es realizar una operación entre dos o más valores contenidos en variables, constantes o acumuladores:

a. Tipos de datos

JavaScript tiene los siguientes tipos de operadores (se describirán los principales para aprender a programar).

Según sea el tipo que las variables o constantes almacenan, se pueden clasificar en seis (6) tipos de datos primitivos, la referencia del lenguaje dice que son:

- **Undefined**

Indeterminado o indefinido.

- **Boolean**

Tipo booleano los valores posibles son true o false.

- **Number**

Números enteros, o decimales.

- **String**

Cadenas de texto.

- **BigInt**

Números enteros grandes.

- **Symbol**

Referencia a otros datos.

Otros tipos de datos más complejos o abstractos de datos:

- **Null**: tipo primitivo especial que tiene un uso adicional para su valor: si el objeto no se hereda, se muestra null.
- **Object**: tipo estructural especial que no es de datos, pero para cualquier instancia de objeto construido que también se utiliza como estructuras de datos (new Object, new Array, new Map, new Set, new

WeakMap, new WeakSet, new Date y casi todo lo hecho con la palabra clave new).

- **Function**: una estructura sin datos, aunque también responde al operador t.

Cuando se declaran variables se debe considerar los siguientes tipos, ya se han usado dos (2) de ellas:

- **var**: declara una variable, opcionalmente la inicia a un valor.
- **let**: declara una variable local con ámbito de bloque, opcionalmente la inicia a un valor.
- **const**: declara un nombre de constante de solo lectura y ámbito de bloque.

Una variable en JavaScript puede tener cualquier tipo de dato, entero, real, cadena de texto, etc. JavaScript no es un lenguaje fuertemente tipado de manera que queda en el programador la responsabilidad de saber qué tipo de dato está almacenando en cada variable.

b. Operadores

Asignar un valor a una variable también se considera una expresión, aunque de un tipo distinto. Para todas estas operaciones se emplean los denominados "operadores". A continuación, se listan algunos de los operadores que se utilizan con mucha frecuencia:

- **Operadores de asignación**

Un operador de asignación asigna un valor a la variable a la izquierda basándose en el valor de su operando derecho. El operador de asignación

más simple es igual (=), que asigna el valor de su operando derecho a su operando izquierdo. Es decir, $x = y$ asigna el valor de y a x . A continuación, se presenta una lista de operadores más usados.

También hay operadores de asignación compuestos que son una abreviatura de las operaciones enumeradas en la siguiente tabla:

Tabla 22. Operadores de asignación compuestos

Nombre	Operador abreviado	Significado
Asignación.	$x = y$	$x = y$
Asignación de adición.	$x += y$	$x = x + y$
Asignación de resta.	$x -= y$	$x = x - y$
Asignación de multiplicación.	$x *= y$	$x = x * y$
Asignación de división.	$x /= y$	$x = x / y$
Asignación de residuo.	$x \% = y$	$x = x \% y$
Asignación de exponenciación.	$x ** = y$	$x = x ** y$

- **Operadores de comparación**

Son operadores cuya función es comparar dos expresiones y, como resultado de la comparación, devuelven un valor falso o verdadero (booleano) que representa la relación de sus valores comparados. Existen operadores para comparar valores numéricos, pero también operadores para comparar cadenas y otros tipos de datos.

Tabla 23. Operadores de comparación

Nombre	Operador abreviado	Significado
Igual (==)	Devuelve true si los operandos son iguales.	3 == var1 "3" == var1 3 == '3'
No es igual (!=)	Devuelve true si los operandos no son iguales.	var1 != 4 var2 != "3"
Estrictamente igual (===)	Devuelve true si los operandos son iguales y del mismo tipo. Consulta también Object.is y similitud en JS.	3 === var1
Desigualdad estricta (!==)	Devuelve true si los operandos son del mismo tipo, pero no iguales, o son de diferente tipo.	var1 !== "3" 3 !== '3'
Mayor que (>)	Devuelve true si el operando izquierdo es mayor que el operando derecho.	var2 > var1 "12" > 2
Mayor o igual que (>=)	Devuelve true si el operando izquierdo es mayor o igual que el operando derecho.	var2 >= var1 var1 >= 3
Menor que (<)	Devuelve true si el operando izquierdo es menor que el operando derecho.	var1 < var2 "2" < 12
Menor o igual que (<=)	Devuelve true si el operando izquierdo es menor o igual que el operando derecho.	var1 <= var2 var2 <= 5

- **Operadores de aritméticos**

Los operadores aritméticos toman valores numéricos (ya sean literales o variables) como sus operandos y devuelven un solo valor numérico. Los operadores aritméticos básicos son suma (+), resta (-), multiplicación (*) y división (/). Estos operadores funcionan como en la mayoría de los otros

lenguajes de programación cuando se usan con números de punto flotante, teniendo en cuenta que la división por cero produce un error.

Tabla 24. Operadores de aritméticos

Nombre	Operador abreviado	Significado
Residuo (%)	Operador binario. Devuelve el resto entero de dividir los dos operandos.	12 % 5 devuelve 2
Incremento (++)	Operador unario. Agrega uno a su operando. Si se usa como operador prefijo (++x), devuelve el valor de su operando después de agregar uno; si se usa como operador sufijo (x++), devuelve el valor de su operando antes de agregar uno.	Si x es 3, ++x establece x en 4 y devuelve 4, mientras que x++ devuelve 3 y, solo entonces, establece x en 4.
Decremento (--)	Operador unario. Resta uno de su operando. El valor de retorno es análogo al del operador de incremento.	Si x es 3, entonces --x establece x en 2 y devuelve 2, mientras que x-- devuelve 3 y, solo entonces, establece x en 2.
Negación unaria (-)	Operador unario. Devuelve la negación de su operando.	Si x es 3, entonces -x devuelve -3.
Positivo unario (+)	Operador unario. Intenta convertir el operando en un número, si aún no lo es.	+ "3" devuelve 3. + true devuelve 1.
Operador de exponenciación (**)	Calcula la base a la potencia de exponente, es decir, base exponente.	2 ** 3 retorna 8. 10 ** -1 retorna 0.1.

- **Operadores lógicos**

Con los operadores lógicos, se pueden crear condiciones compuestas. Por ejemplo, cuando se deben cumplir dos o más condiciones para elegir las

operaciones a ejecutar. Además, se pueden describir estas combinaciones de condiciones utilizando los operadores lógicos como AND (&&), OR (||), y NOT (!). Estos operadores permiten evaluar múltiples expresiones booleanas en una sola sentencia.

Tabla 25. Operadores lógicos

Nombre	Operador abreviado	Significado
AND Lógico (&&)	expr1 && expr2	Devuelve expr1 si se puede convertir a false; de lo contrario, devuelve expr2. Por lo tanto, cuando se usa con valores booleanos, && devuelve true si ambos operandos son true; de lo contrario, devuelve false.
OR lógico ()	expr1 expr2	Devuelve expr1 si se puede convertir a true; de lo contrario, devuelve expr2. Por lo tanto, cuando se usa con valores booleanos, devuelve true si alguno de los operandos es true; si ambos son falsos, devuelve false.
NOT lógico (!)	!expr	Devuelve false si su único operando se puede convertir a true; de lo contrario, devuelve true.

c. Orden de evaluación de operadores

Es importante recordar que los computadores ejecutan los operadores en un orden establecido. El siguiente es el orden (jerarquía), de acuerdo con los expuestos anteriormente, y deben conservar el orden de precedencia de operadores:

- Paréntesis (se ejecutan primero los más internos).
- Signo (-2), si un valor es positivo o negativo.

- c) Potencias (^) y Raíces y otras funciones (Math.sqrt()); Productos y Divisiones (* y /), en este mismo orden.
- d) Sumas y Restas (+ y -).
- e) Concatenación (+).
- f) Relacionales (=, <, >).
- g) Negación (no).
- h) Conjunción (y).
- i) Disyunción (o).

10.2. Expresiones y comentarios

JavaScript permite poner comentarios en el código fuente, como la mayoría de los lenguajes de programación. Existen dos tipos de comentarios:

```
/*
```

Comentario que puede tener más de una línea

de texto

```
*/
```

```
var var1 = 3;
```

```
var var2 = 4;
```

```
console.log( var2 > var1 ); // Este es un comentario de línea
```

```
console.log( var2 < var1 ); // Este es otro comentario de línea
```

```
console.log( var1 !== var2 );
```

Consola

true

false

true

Los comentarios son importantes porque le dan mantenibilidad al programa, es decir que otro programador, o el mismo, tiempo después puede revisar el código y apoyarse en los comentarios para saber qué hace el algoritmo y como lo hace.

En adelante se procura que exista al menos un comentario en cada código fuente empleado para ejemplificar los elementos constitutivos del lenguaje de programación JavaScript y puede hacer lo mismo durante la creación de los diferentes códigos.

10.3. Estructuras de selección

La estructura de selección, se necesita cuando el código de tu programa ejecuta uno de varios resultados posibles, basado en el valor de una condición.

Ejemplo:

Un aprendiz aprueba un examen cuando la calificación de este es mayor o igual a 3. Elaborar un programa en JavaScript donde que dada una calificación, aplique el criterio de aprobación e imprima “Aprobado” o “Reprobado”, según sea el caso.

Como resultado del análisis a este problema tenemos el diseño del siguiente algoritmo:

ALGORITMO Calificación del usuario;

VAR

REAL calificacion

INICIO

calificacion = 4;

SI (calificación $\geq$ 3)

ESCRIBIR("Aprobo");

SINO

ESCRIBIR("Reprobo");

FINSI

FIN

El mismo algoritmo se puede visualizar, pero ya en lenguaje de programación:

/*

Algoritmo calificaciones

*/

const calificacion = 4;


```
if (calificacion >= 3) {  
    console.log('Aprobó');  
} else {  
    console.log('Reprobó');  
}
```

Consola

Aprobó

Como la línea 1 empieza con `/*`, todo lo que sigue será un comentario hasta que aparezcan los caracteres `*/` (línea 3). En este comentario se debe escribir qué hace el algoritmo y otra información relevante que se verá más adelante.

En la línea 4 se define un valor constante (es decir, no se modificará más en todo el algoritmo). Se ha puesto como identificador la palabra `calificacion`.

Para recordar: una palabra reservada es una palabra que no se puede utilizar como identificador de algún dato.

En la línea 6 ocurren varias cosas que se deben tener en cuenta:

- a) Se usa la palabra reservada `if` y, entre paréntesis, la condición que se quiere comparar.
- b) Se emplea el símbolo de corchete abierto `{`, que se llama abrir corchete.
- c) En la línea 7 se envía la instrucción de imprimir el resultado de que aprobó.

- d) En la línea 8 se cierra un bloque de código con el símbolo de corchete cerrado }, lo que indica que hasta este punto termina el bloque. A todo lo que esté entre estos corchetes se le llamará contexto. En el primer contexto están agrupadas las operaciones que se ejecutarán si la condición `calificacion >= 3` es verdadera.
- e) También en la línea 8 se observa la palabra reservada `else`, que sirve para indicarle al intérprete de JavaScript que se creará un bloque o contexto para cuando la condición evaluada por el `if` no se cumpla. Enseguida se abre el contexto con el carácter de corchete abierto {.
- f) Luego, en la línea 9, se ha puesto la operación de escribir en la consola que ha reprobado.
- g) Para finalizar el contexto, se cierra con el carácter de corchete cerrado }.

Ahora se debe escribir el programa y cambiar los valores de la constante para validar que el algoritmo está bien diseñado.

De la misma forma, se puede realizar el algoritmo que, dada una edad en años, evalúe e imprima si es mayor de edad:

```
/*
```

```
Algoritmo saber si es o no mayor de edad
```

```
*/
```

```
const edad = 41;
```

```
if (edad >= 18) {  
    console.log('Es mayor de edad');  
}
```

Consola

Es mayor de edad

De acuerdo con el código anterior, no existe un contexto si no se cumple la condición de mayoría de edad; a esta estructura se le denomina **CONDICIONAL SIMPLE**.

El flujo de ejecución del algoritmo anterior es básico, y puede haber otros mucho más complejos, como las estructuras de repetición o iterativas, que también tienen una sintaxis particular.

10.4. Estructuras de repetición

Las estructuras de repetición permiten repetir un bloque de instrucciones un número determinado o indeterminado de veces. A continuación, se ejemplifican las más comunes (**FOR**, **WHILE**), que se pueden codificar también en la sintaxis de JavaScript.

a. Estructura de repetición **FOR**

Ejemplo 1:

Escribir un procedimiento que muestre siete (7) veces en pantalla la frase “Esto es un algoritmo”. Mostrar en pantalla la salida del ejercicio.

```
/*  
  
Algoritmo imprimir 7 veces un mensaje  
  
*/  
  
for (let i = 0; i < 7; i++) {  
    console.log('esto es un algoritmo');  
}
```

Consola

esto es un algoritmo

esto es un algoritmo

esto es un algoritmo

esto es un algoritmo

esto es un algoritmo

esto es un algoritmo

esto es un algoritmo

Como se puede identificar en la línea 4, se ha usado la palabra reservada for y dentro del paréntesis aparecen tres instrucciones separadas por punto y coma.

- **Primero**

La primera instrucción `let i = 0;` es la creación e inicialización de una variable con identificador `i`. Como se usó la palabra reservada `let`, se está indicando que la variable `i` solo podrá ser accedida dentro del contexto del `for`.

- **Segundo**

Luego se ve una condición `i < 7;`. Esta condición significa que, mientras `i` tenga valores de 0 a 6, debe ejecutar el bloque o contexto definido en el `for` (desde el corchete abierto hasta el corchete cerrado).

- **Tercero**

Una vez ha terminado de ejecutar el contexto, se ejecutará la tercera sentencia `i++`, que es equivalente a `i = i + 1`. Esto incrementa en 1 la variable `i`. Luego de incrementarla, procede a evaluar nuevamente la condición y, si se cumple, volverá a ejecutar las instrucciones que están dentro del contexto (lo que está dentro de los corchetes).

Es de notar que los valores que toma la variable `i` son 0, 1, 2, 3, 4, 5, 6. En total, 7 valores, que es el número de veces que se necesita que se repita el bucle.

Para recordar:

El carácter punto y coma se usa para decirle al computador, o intérprete, que ha finalizado una instrucción.

La estructura de repetición PARA (FOR) se usa cuando se sabe cuántas veces se debe repetir un grupo de instrucciones.

Ejemplo 2:

Se necesita elaborar un algoritmo en JavaScript que, dado un número entero, sume todos los números naturales que hay hasta ese número. Por ejemplo, si el usuario digita 3, el programa debe sumar: $1 + 2 + 3$. Si el usuario digita 5, el programa debe sumar $1 + 2 + 3 + 4 + 5$. Al finalizar, debe imprimir el resultado.

```
/*  
  
    sumar los números que van desde 1 hasta el valor dado  
  
*/
```

```
const cantidad = 5;
```

```
var suma = 0;
```

```
for (let i = 1; i <= cantidad; i++) {
```

```
    suma += i; // esto es igual a suma = suma + i;
```

```
}
```

```
console.log(suma);
```

```
Consola
```

```
15
```

Como se puede ver, se define una constante con valor 5 y una variable que será un acumulador donde se almacenarán las sumas. También, note que los valores de i van desde 1 hasta exactamente el valor que tiene la constante (es decir, 5).

b. Estructura de repetición WHILE

A través del ejemplo del cálculo de factorial se explicará el uso y comportamiento de la sentencia while. Recordando el algoritmo diseñado:

ALGORITMO cálculo de factorial;

VAR

ENTERO contador;

ENTERO factorial;

INICIO

contador = 1;

factorial = 1;

ESCRIBIR ("Escriba el número");

LEER (numero);

MIENTRAS contador <= numero HACER

factorial = factorial * contador;

contador = contador + 1;

FINMIENTRAS

```
    ESCRIBIR ( factorial );
```

```
FIN
```

De acuerdo con el anterior algoritmo, el resultante para el cálculo de factorial es:

```
/*
```

```
Comentario que puede tener más de una línea de texto
```

```
*/
```

```
var var1 = 3;
```

```
var var2 = 4;
```

```
console.log(var2 > var1); // Este es un comentario de línea
```

```
console.log(var2 < var1); // este es otro comentario de línea
```

```
console.log(var1 !== var2);
```

```
Consola
```

```
true
```

```
false
```

```
true
```


Es ideal transcribir el código usando <https://playcode.io/>. Para ello, se debe escribir la condición del while, es decir, contador <= numero, al final, para que el sistema PlayCode no entre en un ciclo que nunca termina. Una vez funcione correctamente, se propone el código que está entre comentarios (//) para practicar las dos formas de sintaxis.

Es importante recordar que las estructuras de repetición básicas y su sintaxis son comúnmente utilizadas para recorrer estructuras de datos.

10.5. Estructuras de datos

Las estructuras de datos, también conocidas como arreglos, son importantes en el desarrollo de algunos algoritmos y son obligatorias a la hora de hacer aplicaciones web o móviles. Por ello, se explica su sintaxis en JavaScript para los vectores, matrices y registros.

a. Vectores

A través del ejemplo del cálculo de factorial se explicará el uso y comportamiento de la sentencia while. Recordando el algoritmo diseñado:

1. `let frutas = ["Manzana", "Banana", "Pera"];`
- 2.
3. `console.log(frutas);`
4. `console.log(frutas.length);`

Console

```
[ "Manzana", "Banana", "Pera"]
```

```
3
```

- En la línea 1 se ve cómo se define un vector con identificador frutas, de tres (3) elementos, cada uno de ellos es una cadena de texto con una fruta.
- En la línea 3 se manda imprimir en la consola todo el vector con su contenido, como se ve en la consola.
- En la línea 4 se llama al atributo frutas.length, que contiene el tamaño del vector (es decir, 3).

A continuación, se presenta cómo acceder a los elementos de un vector:

1. `let arreglo = [5, 7, 23, 12, 43];`
- 2.
3. `console.log(arreglo[0]);`
4. `console.log(arreglo[2]);`
5. `console.log(arreglo[arreglo.length - 1]);`
6. `console.log(arreglo[arreglo.length]);`

Consola

```
5
```

```
23
```

```
43
```

```
undefined
```

Los índices de los vectores comienzan en cero. En otras palabras, el índice del primer elemento de un vector es 0, y el del último elemento es igual al valor de la propiedad `length` del array restándole 1. Si se utiliza un número de índice no válido, se obtendrá `undefined`.

Ahora, se debe crear un programa que guarde los resultados de la tabla de 5 en un vector, con estos resultados recorrer el vector e imprimir:

```
let resultados = [];  
  
for (let i = 1; i <= 10; i++) {  
    resultados.push(i * 5);  
}  
  
for (let i = 1; i <= 10; i++) {  
    console.log('5 x ' + i + ' = ' + resultados[i - 1]);  
}
```

Consola

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

$$5 \times 6 = 30$$

$$5 \times 7 = 35$$

$$5 \times 8 = 40$$

$$5 \times 9 = 45$$

$$5 \times 10 = 50$$

Es ideal revisar la lista de los métodos más comunes en los arreglos:

Tabla 26. Funciones comunes de un vector

Función	Operador Abreviado
indexOf()	Devuelve el índice del primer elemento del array que sea igual a elemento Buscado, o -1 si no existe.
join()	Concatena en un string todos los elementos de un array.
push()	Añade uno o más elementos al final de un array y devuelve el nuevo valor de su propiedad length.
pop()	Elimina el último elemento de un array, y devuelve dicho elemento.
sort()	Ordena los elementos de un array, modificando este, y devuelve el array ordenado.
shift()	Elimina el primer elemento de un array, y devuelve dicho elemento.

b. Matrices

Las matrices se pueden ver lógicamente como un vector, y cada uno de sus elementos es otro vector, así la representación de la matriz queda codificada como se presenta a continuación:

```
"Manzana" "Pera"      "Papaya"  "Piña"
"Papa"     "Tomate"    "Yuca"    "Ajo"
"Arroz"    "Frijol"     "Cebada"  "Garbanzo"
```

```
let datos = [
  ["Manzana", "Pera", "Papaya", "Piña"],
  ["Papa", "Tomate", "Yuca", "Ajo"],
  ["Arroz", "Frijol", "Cebada", "Garbanzo"]
];
```

```
console.log(datos);
```

c. Registros

Hasta ahora se ha revisado cómo un arreglo es la colección de datos del mismo tipo, pero un registro es una colección de datos de diferente tipo que se relacionan entre sí:

Tabla 27. Registros

Nombre	Correo	Edad	Saldo
Juan	juan@sindato.com	19	36.234
Luis	luis@sindato.com	18	23.234

Nombre	Correo	Edad	Saldo
Andrea	andrea@sindato.com	22	0
Pedro	pedro@sindato.com	12	65.234
Maria	maria@sindato.com	16	123

Exceptuando el encabezado de la tabla 8, cada fila representa un registro y cada dato de un registro puede tener un tipo de dato diferente. Si se está interesado en codificar la tabla 8, en una estructura con JavaScript el código sería como se tiene en:

```
let registros = [
    ["Juan", "juan@sindato.com", 19, 36.234],
    ["Luis", "luis@sindato.com", 18, 23.234],
    ["Andrea", "andrea@sindato.com", 22, 0],
    ["Pedro", "pedro@sindato.com", 12, 65.234],
    ["Maria", "maria@sindato.com", 16, 123]
];
```

Para recorrer todos los registros se precisan dos (2) ciclos for, uno dentro de otro (anidado) como se presenta a continuación:

```
let registros = [
    ["Juan", "juan@sindato.com", 19, 36.234],
    ["Luis", "luis@sindato.com", 18, 23.234],
```

```
["Andrea", "andrea@sindato.com", 22, 0],  
["Pedro", "pedro@sindato.com", 12, 65.234],  
["Maria", "maria@sindato.com", 16, 123]  
];
```

```
var total = 0;
```

```
for (let i = 0; i < registros.length; i++) {  
    console.log(`Registro #${i + 1}`);  
    for (let j = 0; j < registros[i].length; j++) {  
        console.log(" " + registros[i][j]);  
    }  
}
```

Consola

Registro #1

Juan

juan@sindato.com

19

36.234

Registro #2

Luis

luis@sindato.com

18

23.234

Registro #3

Andrea

andrea@sindato.com

22

0

Registro #4

Pedro

pedro@sindato.com

12

65.234

Registro #5

Maria

maria@sindato.com

16

123

Como se puede identificar, se requieren dos ciclos for anidados, y cada ciclo tiene una variable de índice de nombre diferente i y j y la manera de indexar la matriz vista anteriormente en el ejemplo de registros [i][j].

10.6. Estructuras de salto

Estas estructuras son instrucciones que permiten romper la ejecución natural o secuencial de los programas, permitiendo que se salte a otro punto de la ejecución del programa. Estas instrucciones tienen las siguientes palabras reservadas: continue, break y return.

- **La sentencia continue**

La sentencia continue se salta a la siguiente iteración del ciclo for o while, como se presenta en el siguiente ejemplo y teniendo en cuenta los resultados:

```
var i = 0;

while (i < 5) {

    i++;

    console.log(i);

}
```

Consola

1

2

3

4

5

```
var i = 0;
while (i < 5) {
    i++;
    if (i == 3) {
        continue;
    }
    console.log(i);
}
```

Consola

1

2

4

5

Cuando la variable `i` tiene el valor 3, se ejecuta la sentencia `continue` se salta la sentencia donde debe imprimir el valor de 3, por ese motivo no aparece en la consola.

- **Sentencia `break`**

La sentencia `break`, detiene la ejecución del ciclo independientemente de cuántas veces este configurando el ciclo; preste especial atención en la salida de consola.

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
}
```

Consola

1

2

3

4

5

```
for (let i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break;  
    }  
    console.log(i);  
}
```

```
}
```

Consola

1

2

Cuando *i* tiene el valor de 3 ya no se sigue ejecutando más el ciclo, aunque esté programado para ejecutarse 5 veces, solo ejecuta 3 iteraciones y, ya que en el tercero luego de evaluar la condición *i* == 3 ejecuta la sentencia `break` haciendo que termine todo.

- **Sentencia return**

El uso de la sentencia `return` sirve para terminar la ejecución de un bloque de instrucciones y es muy común en la programación modular cuando se quiere retornar un resultado. A continuación, se presenta un ejemplo para ilustrar su uso:

Sin return

```
var divideByFour = function (num) {  
    num / 4;  
};
```

```
var divided = divideByFour(27);  
console.log(divided); // undefined
```

Con return

```
var divideByFour = function (num) {  
    return num / 4;  
};
```

```
var divided = divideByFour(27);  
console.log(divided); // 6.75
```

10.7. Métodos de ordenamiento y búsqueda

Existen varias técnicas de ordenamiento de vectores que son importantes para realizar búsquedas. Al igual que en la vida real, es más fácil buscar en un lugar que esté ordenado que en uno que no lo esté. JavaScript está diseñado para pequeños algoritmos que resuelven problemas específicos y convertirlos en servicios. El uso de estas técnicas puede no tener mucha aplicabilidad en volúmenes de datos pequeños, ya que la diferencia en tiempos computacionales de ejecución es considerable solo cuando se manejan grandes volúmenes de datos. Para volúmenes de datos pequeños, los tiempos computacionales no presentan diferencias significativas.

a. Ordenamiento de cadenas de texto

Para ordenar un vector de mayor a menor valor y teniendo en cuenta que el arreglo se modifica al ordenarlo, es importante hacer una copia del arreglo si se desea mantener el vector original inmutable. A continuación, se presenta un ejemplo que ilustra cómo realizar este proceso en JavaScript:

- **Ordenamiento de números como cadenas de texto**

Por defecto el método `Array.sort()` ordena los elementos del array como si fueran cadenas, inclusive si los datos son de tipo entero, como en este ejemplo:

```
var arg = [10, 53, 12, 56, 120];
```

```
console.log(arg.join(', '));
```

```
arg.sort();
```

```
console.log(arg.join(', '));
```

Consola

10, 53, 12, 56, 120

10, 12, 120, 53, 56

- **Ordenamiento de cadenas de texto**

El arreglo a ordenar en la línea 1, en la línea 2, se presenta en consola el vector, se ha usado la función `join(', ')` que sirve para concatenar los datos del vector o arreglo poniendo entre ellos una coma (,) y espacio (línea 2). Luego se aplica la función `sort()` en la tercera línea de código y, posteriormente, se presenta el vector ya ordenado. Como se presenta, el

resultado en consola es un ordenamiento como si los elementos fueran cadenas de texto.

```
1. var data = ["Montería", "Sincelejo", "Cartagena", "Bogotá"];
2. data.sort();
3. console.log(data);
```

Consola

```
["Bogotá", "Cartagena", "Montería", "Sincelejo"]
```

- **Invertir el orden de los elementos**

En el ejemplo de un ordenamiento de cadenas de texto. También se puede ordenar el arreglo de forma descendente con el método `Array.reverse()`.

```
var data = ["Montería", "Sincelejo", "Cartagena", "Bogotá"];
data.sort().reverse();
console.log(data);
```

Consola

```
["Sincelejo", "Montería", "Cartagena", "Bogotá"]
```

La función `Array.reverse()` no ordena los elementos, simplemente toma los elementos y les invierte el orden, es decir, el primero pasa a ser el último, el segundo, el penúltimo y así hasta que el última pasa al primer puesto. Es por esto que primero se ordena en orden ascendente y luego se invierte para lograr el orden descendente.

b. Ordenamiento de datos numéricos

Como se explica, el ordenamiento con la función `Array.sort()` se realiza considerando los elementos del arreglo como cadenas de texto. Antes de describir cómo se ordenan con datos numéricos, se explicará qué es una función en JavaScript.

Las funciones son uno de los bloques de construcción fundamentales en JavaScript. Una función en JavaScript es similar a un procedimiento o “un conjunto de instrucciones que realiza una tarea o calcula un valor, pero para que un procedimiento califique como función, debe tomar alguna entrada y devolver una salida donde hay alguna relación obvia entre la entrada y la salida”. Para usar una función, se debe definir en algún lugar del ámbito en el que se desea llamarla (MDN, 2021d).

Ejemplo

Se quiere construir una función que reciba dos (2) datos numéricos, hay que multiplicar estos dos números, al resultado se le suma el valor de 100 y, por último, retorna la mitad de este resultado.

```
function operacion(a, b) {  
    return (a * b + 100) / 2;  
}
```

```
var dato1 = operacion(3, 8);
```

```
var dato2 = operacion(2, 4);
```

```
console.log(dato1);
```



```
console.log(dato2);
```

Consola

62

54

En la línea 1, se usa la palabra reservada `function` y luego se elige un identificador para la función, que en este caso es `operacion`. Luego, entre paréntesis, se seleccionan nombres para las variables de entrada (en el ejemplo, `a` y `b`). A continuación, se define un contexto (espacio de código entre corchetes) para indicar cuáles son las instrucciones que ejecutará la función. En la línea 2, se indica que la función retorna la multiplicación de `a * b` sumando 100 y a este resultado dividido entre 2. En la línea 4, se ve cómo se invoca la función `operacion` y se le pasan los parámetros 3 y 8 para las variables `a` y `b` respectivamente, el resultado es almacenado en la variable `dato1`. Y la función puede invocarse más de una vez (las que se necesiten).

```
var arg = [10, 53, 12, 56, 120];
```

```
arg.sort(function(a, b) {
```

```
    return a - b;
```

```
});
```

```
console.log(arg.join(', '));
```

Consola

10, 12, 53, 56, 120

Para ordenar un vector con datos numéricos, se debe pasar como argumento una función al método `Array.sort()`. Esta función indica qué operación debe hacerse sobre los datos a comparar entre dos datos del arreglo. Para comparar si un número es mayor que otro, es muy común usar la resta porque, al restar un número, solo existen tres tipos de respuestas. Cuando a un número a se le resta un número b ($a - b$), si el resultado es negativo, es porque b es mayor que a ; si el resultado es positivo, es porque a es mayor que b ; y si el resultado es cero, es porque a y b son iguales.

Como se puede apreciar, se pasa una función (sin identificador) que recibe dos datos del arreglo (a y b) y los compara con la diferencia (resta) de ellos. A las funciones que no se les pone identificador se les llaman funciones anónimas. También existe otra forma de sintaxis para funciones anónimas, denominada funciones flecha. A continuación, se presenta cómo ordenar un arreglo numérico de manera inversa utilizando la sintaxis de función:

```
var arg = [10, 53, 12, 56, 120];
```

```
arg.sort((a, b) => b - a);
```

```
console.log(arg.join(' '));
```

Consola

120, 56, 53, 12, 10

c. Ordenamiento de registros

Suponiendo que se tienen los registros de la tabla, se desea ordenarlos de mayor edad a menor edad.

Tabla 28. Registros

Usuario	Edad	Rol
Mariela	31	SAC
Eduardo	30	SEO
Andrés	34	Project Manager

En esta oportunidad en las líneas 6 y 8, que se puede usar contexto (corchetes) a pesar de que dentro solo hay una sentencia (línea 7), el resultado a retornar es la diferencia entre b.edad y a.edad para que el ordenamiento sea de mayor a menor.

```

1  var data = [
2    {usuario: 'Mariela', edad: 31, rol: 'SAC'},
3    {usuario: 'Eduardo', edad: 30, rol: 'CEO'},
4    {usuario: 'Andrés', edad: 34, rol: 'Project Manager'}
5  ];
6  data.sort((a, b) => {
7    return b.edad - a.edad;
8  });

```

```
9 console.log(data);
```

Consola

```
[  
  {usuario: "Andrés", edad: 34, rol: "Project Manager"},  
  {usuario: "Mariela", edad: 31, rol: "SAC"},  
  {usuario: "Eduardo", edad: 30, rol: "CEO"}  
]
```

d. Buscar un elemento

El método `find()` devuelve el valor del primer elemento del array que cumple la función de prueba proporcionada (`Array.prototype.find()` - JavaScript, MDN, 2021).

```
const array1 = [5, 12, 8, 130, 44];  
  
const found = array1.find(element => element > 10);  
  
console.log(found);
```

Consola

12

Se utiliza el identificador elemento para referenciar la variable a la que se aplicará el criterio de búsqueda, que en este ejemplo es encontrar el primer elemento mayor que 10. Muchas veces es importante que, en lugar de retornar un solo valor, el resultado sea más de uno. Por ejemplo, si en el arreglo se retornan los valores mayores que 10, se debe retornar otro vector con los valores resultantes. Para esto, existe la función `Array.filter()`, que crea un nuevo arreglo con los datos que cumplan con la condición.

```
const array1 = [5, 12, 8, 130, 44];  
  
const found = array1.filter(element => element > 10);  
  
console.log(found);
```

Consola

```
[ 12, 130, 44]
```

Es importante notar que la consola solo devuelve los datos que cumplen la condición de que el elemento es mayor que 10. A medida que se avanza en el conocimiento de la sintaxis del lenguaje, el código fuente se enriquece con más elementos y, por tanto, aumenta en complejidad. Esto hace necesario conocer herramientas para la depuración de fallos y la corrección de errores en la redacción del código.

11. Depuración y fallas de sintaxis

La principal funcionalidad que se requiere en la depuración del código es la de mostrar salidas parciales de datos para verificar el estado de las variables, o tal vez la necesidad de indicar que se está ejecutando determinada línea de código. Por lo general, la salida de JavaScript está dirigida hacia un navegador o hacia un flujo de datos de un servidor web, dejando la consola para realizar estas depuraciones.

Hasta ahora se ha utilizado la consola para verificar datos de depuración mediante el uso de `console.log()`, que es un método usado para verificar mensajes de confirmación de operaciones o notificaciones de éxito en las operaciones del algoritmo. Sin embargo, también existen otros métodos:

- `console.log()` para enviar y registrar mensajes generales.
- `console.dir()` para registrar un objeto y visualizar sus propiedades.
- `console.warn()` para registrar mensajes de alerta.
- `console.error()` para registrar mensajes de error.

```
const array1 = [5, 12, 8, 130, 44];
```

```
console.dir(array1);
```

```
console.warn("Mensaje de alerta " + array1[3]);
```

```
console.error("Mensaje de alerta " + array1[4]);
```

Consola

```
[5, 12, 8, 130, 44]
```



Mensaje de alerta 130

error: Mensaje de alerta 44

12. Manejo de errores y excepciones

Las excepciones son errores imprevistos que ocurren en la ejecución de un programa; son anomalías que impiden o perturban el comportamiento o el flujo normal de las instrucciones de un programa.

El objetivo principal en el manejo de errores es separar el código de la lógica del programa del código de manejo de excepciones. De esta forma, al producirse la anomalía, si ocurre un error y existe un manejador de excepciones, él toma el control de la ejecución del programa.

En JavaScript, el control de errores resulta muy necesario, tal vez más que en otros lenguajes, debido a la dificultad natural para probar aplicaciones o los cambios de versión de intérprete en múltiples navegadores.

La forma más sencilla de disparar un error es con la palabra reservada `throw`. Este comando permite enviar al intérprete de JavaScript el evento de que ha ocurrido un error, generalmente permite enviar cualquier tipo de dato, pero lo más común es enviar un "Error" como se indica a continuación:

```
throw new Error("No se puede dividir por cero");
```

Existe un tipo de estructura de control en JavaScript que está pendiente de revisar en el flujo de ejecución de un programa para detectar comportamientos inesperados. La estructura es `try...catch`, que consta de un bloque que intenta la ejecución de una sección de código (contexto) a la espera de que pueda ocurrir una posible excepción. Finalizado el contexto, existe otro denominado `catch` que tiene un argumento o variable de contexto donde se captura el elemento disparado.

```
let numerador = 4;
```



```
let denominador = 0;

var resultado = null;

try {

    if (denominador === 0) {

        throw new Error("No se puede dividir por cero");

    } else {

        resultado = numerador / denominador;

    }

} catch (e) {

    console.error(e);

    resultado = null;

}
```

Consola

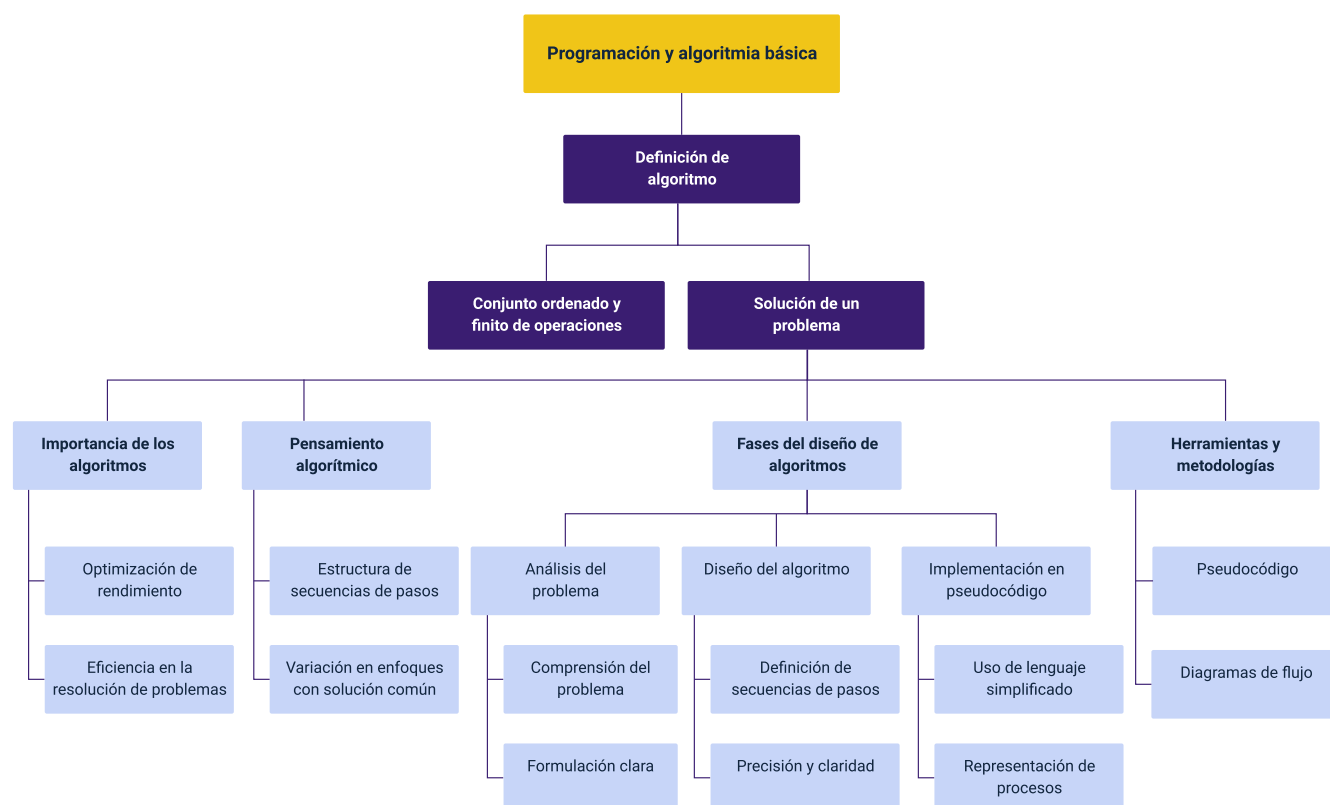
error: Error: No se puede dividir por cero

En el código se describe un algoritmo que divide dos números y, como se sabe, en este proceso se debe validar que no exista una división por 0, ya que no está definida aritméticamente.

Para eso, existe el contexto try que va desde la línea 4 a la línea 10. Dentro de él se verifica el caso de que el divisor, si es igual a 0, debe disparar una excepción. Ella es capturada en la línea 10 por el contexto catch en la variable e. De esta forma, el contexto de catch de la línea 10 a la 13 maneja la excepción, enviando un mensaje de error y asignando a la variable resultado el valor nulo. Así, la excepción es lanzada y capturada, y el programa no se interrumpe abruptamente, sino que es controlado a pesar de los problemas que se puedan presentar.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Introducción a la algoritmia	Magic Markers. (2015). ¿Qué es un algoritmo? [Video]. YouTube.	Video	https://www.youtube.com/watch?v=U3CGMyjzlvM
Introducción a la algoritmia	Calderón, J. P. (2015). Fundamentos de lógica matemática. [Video]. YouTube.	Video	https://www.youtube.com/watch?v=9zYDaZhS7Ac
Diagramas de flujo	Material_Audiovisual_10031 9633. (6 de enero de 2021). App Diagrams.net (básico para diagramas de flujo). [Video]. YouTube.	Video	https://youtu.be/idnqmFp qTvo
Diagramas de flujo	Enrique Olivares. (23 de febrero de 2016). Introducción a utilizar el programa DIA para diagramas de flujo a nivel bachillerato. [Video]. YouTube.	Video	https://youtu.be/DnRx9D7 QWg8
Diagramas de flujo	DiscoDurodeRoer. (27 de abril de 2015). Ejercicios PseInt - Básicos #1 - Empezando por lo básico. [Video]. YouTube.	Video	https://youtu.be/DHli4dca MEc
Diagramas de flujo	Insituto de Informática Uach. (08 de septiembre de 2020). Cómo descargar Draw.io 2 para dibujar diagramas de flujo. [Video]. YouTube.	Video	https://www.youtube.com/watch?v=encz3h8TLWA

Tema	Referencia	Tipo de material	Enlace del recurso
Entornos de codificación e instalación JavaScript	Framebits. (19 de enero de 2020). Descargar e instalar Node Js en Windows 10.	Video	https://youtu.be/v0x1Ku5Tgac
Entornos de codificación e instalación JavaScript	Code Compadre. (30 de junio de 2020). How to Download and Install Visual Studio Code for Windows 10.	Video	https://youtu.be/KpzKPlhHsU
Entornos de codificación e instalación JavaScript	VideoMarketingViral. (11 de enero de 2020). Descargar Google Chrome para PC 2021 (WINDOWS 7/8/10).	Video	https://youtu.be/V-M2rMBhgKc
Entornos de codificación e instalación JavaScript	MDN. (2021). Array - JavaScript.	Documentación técnica	https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math
Entornos de codificación e instalación JavaScript	MDN. (2021). Expresiones y operadores - JavaScript	Documentación técnica	https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Expressions_and_Operators
Entornos de codificación e instalación JavaScript	MDN. (2021). Funciones - JavaScript.	Documentación técnica	https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Functions
Entornos de codificación e instalación JavaScript	MDN. (2021). Array.prototype.find() - JavaScript.	Documentación técnica	https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/find
Entornos de codificación e instalación JavaScript	Choque, R. C. (10 de mayo de 2020). Cómo usar la consola de Google Chrome para JavaScript. [Video] YouTube.	Video	https://youtu.be/Hf3n-p3VYx4

Tema	Referencia	Tipo de material	Enlace del recurso
Entornos de codificación e instalación JavaScript	Autodidacta, F. (11 de mayo de 2017). Depurar o hacer debug a JavaScript con Google Chrome. [Video] YouTube.	Video	https://youtu.be/l388w3wDkjc

Glosario

Algoritmos: conjunto ordenado y finito de operaciones que permite hallar la solución de un problema.

Compilación: proceso de traducción del código fuente de un programa a un lenguaje de bajo nivel ejecutable.

Depuración: proceso de identificar y corregir errores o fallos en un programa informático.

Diseño: proceso de planificar la solución a un problema a través de un algoritmo.

Flujo: secuencia de pasos que determina el orden en que se ejecutan las instrucciones de un programa.

Modularidad: propiedad de un sistema que permite dividirlo en partes más pequeñas y manejables.

Optimización: mejora del rendimiento o eficiencia de un programa mediante la reducción de recursos utilizados o el aumento de la velocidad de ejecución.

Programación: proceso de crear software escribiendo, probando, depurando y manteniendo el código fuente en un lenguaje de programación.

Pseudocódigo: forma simplificada de representar un algoritmo utilizando expresiones del lenguaje natural.

Variables: elementos que almacenan datos que pueden cambiar durante la ejecución de un programa.

Referencias bibliográficas

Arellano, J., Solar, R., Nieva, S., y Canedo, P. (2022). Compilador e intérprete en línea de diagramas de flujo con fines didácticos [Imagen]. RITI Journal, 10(20), 82.

<https://doi.org/10.36825/RITI.10.20.007>

Autodidacta, F. (11 de mayo de 2017). Depurar o hacer debug a JavaScript con Google Chrome. [Video]. <https://www.youtube.com/watch?v=I388w3wDkjc&t=192s>

Choque, R. C. (10 de mayo de 2020). Cómo usar la consola de Google Chrome para JavaScript. [Video]. <https://www.youtube.com/watch?v=Hf3n-p3VYx4>

EduTEKA.org. (2008). Sembrando las semillas para una sociedad más creativa. Dr. Mitchel Resnick. [Traducción]. Universidad ICESI.

<http://www.eduteka.org/ScratchResnickCreatividad.php>

Framebits. (19 de enero de 2020). Descargar e instalar Node Js en Windows 10. [Video]. <https://youtu.be/v0x1Ku5Tgac>

López, J., C. (2009). Educación básica algoritmos y programación. Guía para docentes. Segunda edición. Fundación Gabriel Piedrahita Uribe.
<https://libros.metabiblioteca.org/bitstream/001/169/8/AlgoritmosProgramacion.pdf>

MDN. (2021). Array - JavaScript.
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array
<#>

MDN. (2021a). Expresiones y operadores - JavaScript.
[https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Expressions and Operat
ors](https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Expressions_and_Operators)

MDN. (2021b). Math - JavaScript.

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math

MDN. (2021c). Array.prototype.find() - JavaScript.

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/find

MDN. (2021d). Funciones - JavaScript

<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Functions>

MDN. (2021e). Tipos de datos y estructuras en JavaScript - JavaScript.

<https://developer.mozilla.org/es/docs/Web/>

Medina, H., M. (2018). Estrategias metodológicas para el desarrollo del pensamiento lógico-matemático. Didasc@lia: Didáctica y Educación, 9(1), p. 125-132.

<https://dialnet.unirioja.es/descarga/articulo/6595073.pdf>

Real Academia Española (2020). Algoritmo | Diccionario de la lengua española.

RAE. <https://dle.rae.es/algoritmo>

Real Academia Española (2020). Programa | Diccionario de la lengua española.

RAE. <https://dle.rae.es/programa>

Resnick, M. (2007). Sembrando semillas para una sociedad más creativa.

Resnick, M. (2007). Todo lo que Realmente Necesito Saber (Acerca del Pensamiento Creativo) Lo Aprendí (Estudiando Cómo Aprenden los niños) en el Kindergarten. EDUTEKA. https://eduteka.icesi.edu.co/pdfdir/RESNICK_kindergarten-learning-approach.pdf

Rojas, A., V. y Ñacato C., J. (1980). Técnica de flujogramas I. Editora Andina.

<https://biblioteca.epn.edu.ec/cgi-bin/koha/opac-detail.pl?biblionumber=27333>

Suppes, P., y Hill, S. (2021). Introducción a la lógica matemática. Editorial Reverté.

https://www.reverte.com/libro/introduccion-a-la-logica-matematica_91496/

Tapia, L. (1995). Cómo desarrollar el razonamiento lógico matemático. Editorial

Universitaria. <https://www.worldcat.org/title/como-desarrollar-el-razonamiento-logico-y-matematico/oclc/50414752>

The Development Stages. (2024). PostgreSQL: Ejercicios prácticos.

<https://thedevelopmentstages.com/postgresql-ejercicios-practicos/>

TOMi.digital. (s.f.). Valores de las tablas de verdad. Colegio Nuestra Señora de las

Victorias. https://tomi.digital/es/59552/valores-de-las-tablas-de-verdad?utm_source=google&utm_medium=seo

Vázquez, J. (2012). Análisis y diseño de algoritmos.

https://www.academia.edu/8646162/An%C3%A1lisis_y_dise%C3%B1o_de_algoritmos

Vedia, M. (2018). Cuando usar Return en JavaScript.

<https://medium.com/@mvtercero85/cuando-usar-return-en-javascript-7b80e025eb7f>

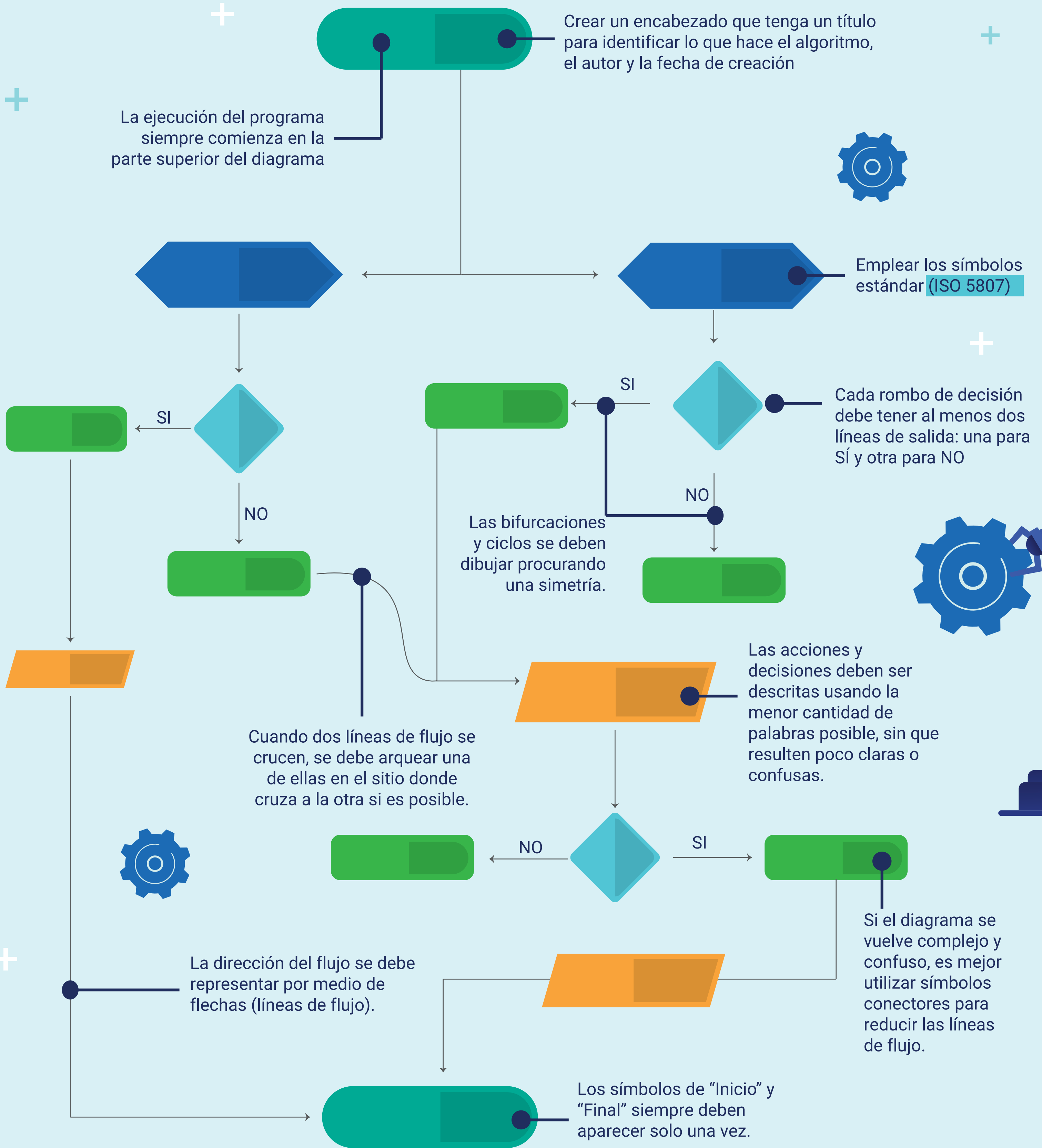
Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Henry Eduardo Bastidas Paruma	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Peter Emerson Pinchao	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluable Instruccional	Centro de Servicios de Salud - Regional Antioquia
Yerson Fabián Zárate Saavedra	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Álvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluable para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

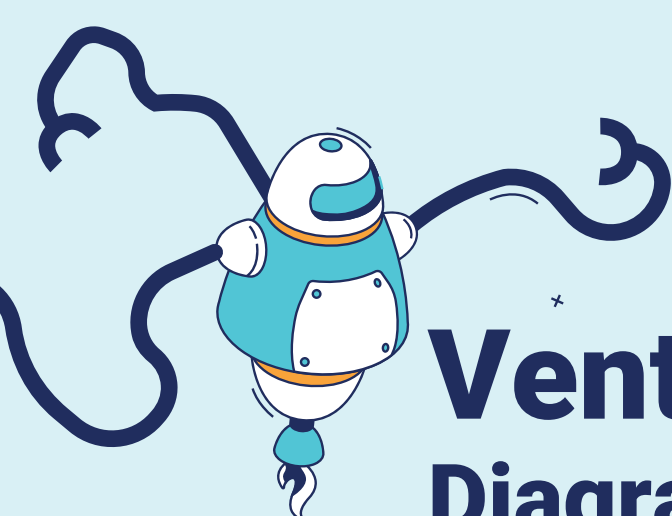


Diagramas de Flujo

Todo el diagrama debe ser claro, ordenado y fácil de recorrer



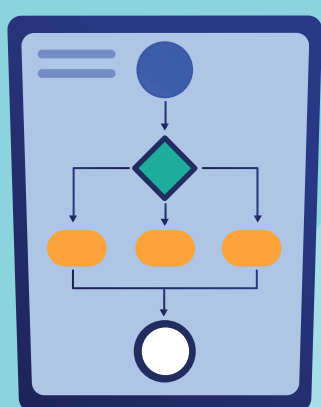
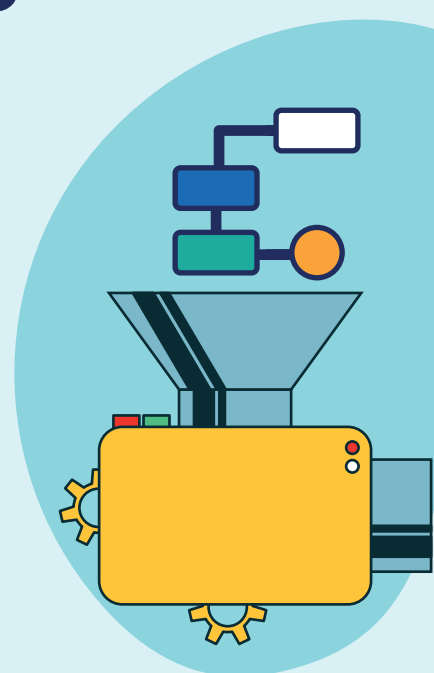
- Todas las líneas deben llegar a un símbolo o a otra línea.
- El diagrama se debe probar recorriéndolo con datos iniciales simples (prueba de escritorio).
- Siempre inicializar las variables que se empleen o permitir la asignación de valores a través de consulta al usuario.



Ventajas Diagramas de flujo

1- Aprender a Programar

Por su representación son construidos por personas que está en procesos de aprender a programar.

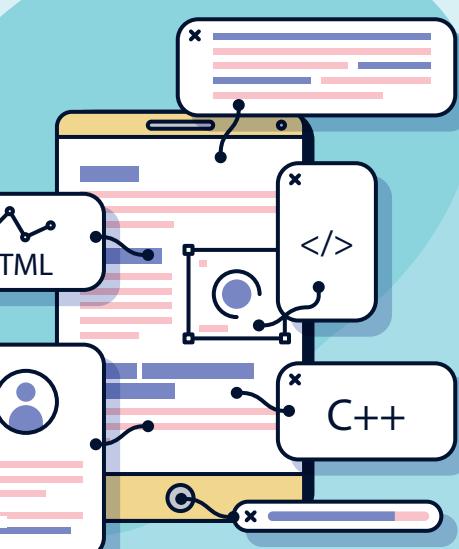


2- Aprendizaje Visual

Es uno de los mejores métodos para enseñar habilidades del pensamiento.

3- Representar Ideas

Las técnicas que utilizan formas gráficas ayudan a clarificar ideas, a procesar, organizar y priorizar la información.



4- Interrelaciones Interdependencias

Muestran patrones, interrelaciones, interdependencias y también estimulan el pensamiento creativo.

5- Representa Instrucciones

de un algoritmo de manera secuencial, este tipo de representación es muy usada y no solamente en algoritmia, sino en otras áreas donde se quiera representar un proceso.



6- Compuesto por decisiones lógicas

por operaciones, ciclos repetitivos, entre otros.



FUNCIONES CLASE MATH

Función	Descripción
Math.abs(x)	Devuelve el valor absoluto de un número.
Math.acos(x)	Devuelve el arco coseno de un número.
Math.acosh(x)	Devuelve el arco coseno hiperbólico de un número.
Math.asin(x)	Devuelve el arco seno de un número.
Math.asinh(x)	Devuelve el arco seno hiperbólico de un número.
Math.atan(x)	Devuelve el arco tangente de un número.
Math.atanh(x)	Devuelve el arco tangente hiperbólico de un número.
Math.atan2(y, x)	Devuelve el arco tangente del cociente de sus argumentos.
Math.cbrt(x)	Devuelve la raíz cúbica de un número.
Math.ceil(x)	Devuelve el entero más pequeño mayor o igual que un número.
Math.clz32(x) (en-US)	Devuelve el número de ceros iniciales de un entero de 32 bits.
Math.cos(x)	Devuelve el coseno de un número.
Math.cosh(x) (en-US)	Devuelve el coseno hiperbólico de un número.
Math.exp(x)	Devuelve E^x , donde x es el argumento, y E es la constante de Euler (2.718...), la base de los logaritmos naturales.
Math.expm1(x)	Devuelve $e^x - 1$.
Math.floor(x)	Devuelve el mayor entero menor que o igual a un número.
Math.fround(x)	Devuelve la representación flotante de precisión simple más cercana de un número.
Math.hypot([x[, y[, ...]]])	Devuelve la raíz cuadrada de la suma de los cuadrados de sus argumentos.
Math.imul(x, y) (en-US)	Devuelve el resultado de una multiplicación de enteros de 32 bits.
Math.log(x)	Devuelve el logaritmo natural (log, también ln) de un número.
Math.log1p(x) (en-US)	Devuelve el logaritmo natural de $x + 1$ (loge, también ln) de un número.
Math.log10(x)	Devuelve el logaritmo en base 10 de x .
Math.log2(x)	Devuelve el logaritmo en base 2 de x .
Math.max([x[, y[, ...]]])	Devuelve el mayor de cero o más números.
Math.min([x[, y[, ...]]])	Devuelve el más pequeño de cero o más números.
Math.pow(x, y)	Las devoluciones de base a la potencia de exponente, que es, <code>baseexponent</code> .
Math.random()	Devuelve un número pseudo-aleatorio entre 0 y 1.
Math.round(x)	Devuelve el valor de un número redondeado al número entero más cercano.
Math.sign(x)	Devuelve el signo de la x , que indica si x es positivo, negativo o cero.
Math.sin(x)	Devuelve el seno de un número.
Math.sinh(x) (en-US)	Devuelve el seno hiperbólico de un número.
Math.sqrt(x)	Devuelve la raíz cuadrada positiva de un número.
Math.tan(x)	Devuelve la tangente de un número.
Math.tanh(x)	Devuelve la tangente hiperbólica de un número.



Conceptos básicos programación orientada a objetos

Breve descripción:

En este componente formativo se abordan los conceptos clave del paradigma orientado a objetos, entre los que se encuentran la abstracción, el encapsulamiento, la modularidad, la jerarquía y el polimorfismo. Adicionalmente, se introducen los conceptos de clases y objetos desde la perspectiva de este paradigma.

Abril 2024

Tabla de contenido

Introducción	3
1. Paradigma orientado a objetos	4
Abstracción	4
Encapsulamiento	5
Modularidad	6
Jerarquía	7
Polimorfismo	8
2. Clases y objetos	10
Atributos	12
Síntesis	14
Material complementario	15
Glosario	16
Referencias bibliográficas	17
Créditos	18

Introducción

¿Sabías que algunos de los elementos fundamentales que se usan para la descripción de un algoritmo en un lenguaje de programación, son los identificadores? Estos pueden ser variables o constantes, dependiendo del comportamiento de los valores que registramos en ellos.

Estos identificadores se componen principalmente de tres elementos: nombre del identificador, valor almacenado y posición física de la memoria donde se encuentra. Para poder utilizar identificadores, se debe realizar el proceso de declaración del identificador, donde se define el tipo de dato asociado, el cual indica el tamaño a reservar en la memoria y el nombre de dicho identificador.

Cuando el problema a desarrollar requiere gran cantidad de información y adicionalmente, la información se compone de diversos tipos de datos, el mecanismo de identificadores se vuelve obsoleto, no solo por la gran cantidad de estructuras de memoria que se deberán construir, sino porque su gestión también será más compleja.

Al tener problemas cada vez más complejos y grandes cantidades de datos, se requieren mejores mecanismos para gestionar los datos y es a partir de este escenario, que surge la programación orientada a objetos, como un paradigma que busca aproximar el manejo de la información, desde la forma como es procesada internamente en los programas a como realmente se representa en la vida cotidiana.

A lo largo del componente formativo, se profundizará sobre los conceptos en los que se fundamenta el paradigma y luego se examinarán en detalle algunos de los elementos centrales que se utilizan para implementar el paradigma. Bienvenido.

1. Paradigma orientado a objetos

El paradigma orientado a objetos es un concepto que inició en los años 70, pero que se logró estandarizar alrededor de 1997 con la aparición del Lenguaje Unificado de Modelado UML, teniendo presente que la palabra objetos no hace referencia necesariamente a componentes gráficos de un sistema de información (Rumbaugh, et al., 2004).

Según Booch et al. (2007), para que un sistema o modelo sea orientado a objetos debe tener los siguientes elementos:

- Abstracción
- Encapsulamiento
- Modularidad
- Jerarquía
- Polimorfismo

Abstracción

La abstracción es un proceso natural del ser humano y consiste esencialmente en la identificación de características claves según el contexto y conocimiento de las personas. De tal forma, se constituye en el medio mediante el cual se enfrentan los problemas complejos para extraer las características esenciales y posteriormente, representar dichas características en objetos.

La abstracción se enfoca en la vista externa de los elementos, lo cual permite separar su comportamiento esencial de su implementación e ignorar detalles internos. Mediante la abstracción hay un énfasis en el qué es y qué hace y no en el cómo debe implementarse o cómo funciona algo. Aguilar, 2021

Analicemos el siguiente ejemplo, sobre el proceso de abstracción.

En diferentes escenarios se aplican ciertos niveles de abstracción, por ejemplo, a la hora de decidir qué teléfono celular se desea comprar.

- **Edad adulta**

Una persona de negocios de edad adulta, puede interesarle la disponibilidad de comunicación por medio de telefonía móvil, así como la conectividad a las redes de datos y la facilidad con la que pueda conectarse a cualquiera de esas redes.

- **Adolescente**

Probablemente una persona más joven, por ejemplo, un adolescente, podría interesarle más las capacidades para soportar diferentes tipos de archivos multimedia y la velocidad con la que se puedan ejecutar juegos.

- **Desarrollador**

Una persona que desarrolla “software” para móviles, podría interesarle más otras características como el tipo de sistema operativo, la operatividad con los sensores disponibles, y cosas de ese estilo.

Podemos concluir que lo que se busca ahora, es llevar ese mismo proceso para entender la esencia de los problemas complejos a desarrollar, y construir, a partir de esas abstracciones, que representan todos los elementos involucrados.

Encapsulamiento

El encapsulamiento es el proceso que permite agrupar datos y también operaciones que relacionan estos datos bajo una misma unidad lógica.

La encapsulación además, trae consigo algo llamado ocultamiento de datos, que consiste en la posibilidad de separar los elementos esenciales con los que se opera algo (interfaz – externa y pública) y de los detalles de su implementación (interna y privada). El encapsulamiento es un proceso que se debe realizar luego del proceso de abstracción. Aguilar, 2021

A continuación, presentamos un ejemplo de encapsulamiento.

Un control remoto, agrupa varios elementos que tienen funcionamientos particulares pero que en conjunto permiten la manipulación a distancia de un televisor.

Adicionalmente, como usuarios de un control remoto interactuamos con él mediante su interfaz externa y pública, es decir, sus botones de colores, cada botón tiene un identificador que es conocido para nosotros y mediante el cual manipulamos el televisor.

Como usuarios de un control remoto no es esencial su implementación (interna y privada), es decir, cómo interactúan las ondas de luz de baja frecuencia o las señales que son decodificadas por los aparatos electrónicos.

Modularidad

La modularidad es la propiedad que permite dividir una aplicación o sistema en partes más pequeñas, idealmente deben ser muy independientes (bajo acoplamiento) y altamente funcionales (alta cohesión).

Dividir un sistema en partes más pequeñas, tiene varios beneficios, entre los cuales se pueden listar:

- Se produce la complejidad. Los problemas más pequeños son más fáciles de resolver que los problemas grandes.
- Cada módulo puede ser ejecutado y probado de forma independiente, lo cual facilita la corrección de errores y permite que un problema pueda ser desarrollado por varias personas en forma paralela.

Como ejemplo de modularidad, tenemos:

Un vehículo se puede construir a partir del desarrollo de cada una de sus partes de forma independiente, e incluso, por fabricantes diferentes: puertas, ventanas, motor, chasis, asientos, llantas, etc.

Luego, en un proceso que la mayoría de las veces es automatizado, se unen estas partes para producir el vehículo.

Cuando el vehículo es reparado u optimizado, se puede cambiar cada una de sus partes, de forma individual.

Jerarquía

Las jerarquías son clasificaciones y ordenaciones de las abstracciones de un problema (Aguilar, 1998), permiten comprender de forma general la estructura de un sistema y favorecen, en algunos casos, la reutilización de características y comportamientos de las abstracciones definidas.

Las jerarquías que se producen desde el punto de vista de la estructura de objetos se les conoce como agregación y composición, y las jerarquías que se producen desde las estructuras de clases se les conoce como generalización y especialización.

Conozcamos la jerarquía de un computador de escritorio:

Figura 1. Jerarquía de un computador de escritorio



Dependiendo del nivel de dependencia que existe entre la abstracción que agrupa y las abstracciones que son partes, se definen relaciones de composición o agregación.

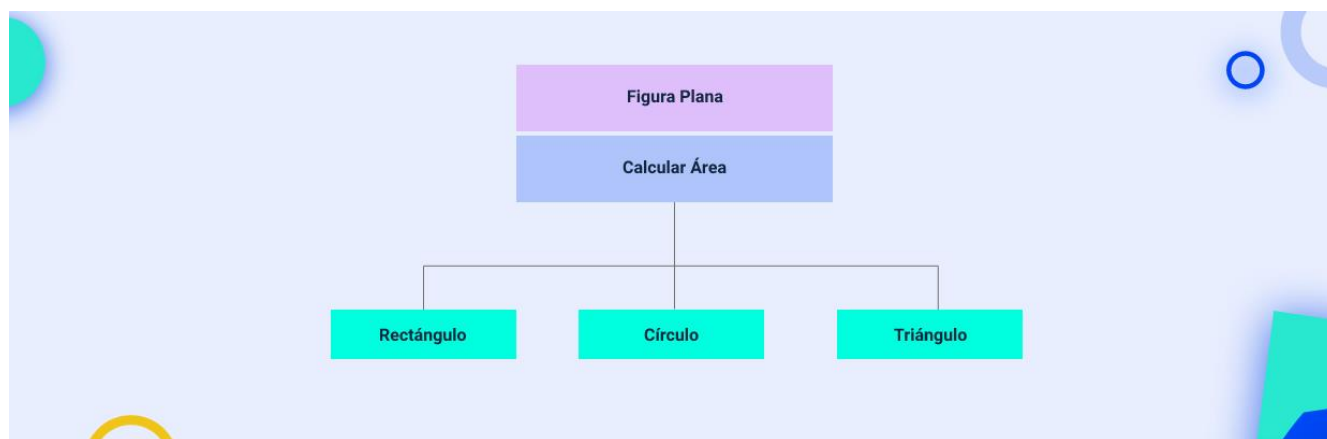
La representación de las uniones entre las abstracciones de una jerarquía usa símbolos diferentes para diferenciar lo que son jerarquías de clases y jerarquías de objetos. Esto se detalla en el tema de diagrama de clases.

Polimorfismo

Es la característica que permite que una abstracción tome diferentes formas o diferentes comportamientos según el contexto. Esta característica se potencia en compañía de las jerarquías vistas anteriormente.

En la siguiente figura, se muestra un ejemplo de una jerarquía sobre figuras planas y adicionalmente, se indica que hay un comportamiento llamado Calcular Área, es decir, todas las figuras planas pueden calcular su área.

Figura 2. Ejemplo de una jerarquía sobre figuras planas



En la jerarquía de clases del ejemplo anterior, las abstracciones rectángulo, círculo y triángulo heredan las características de la abstracción figura plana incluyendo el comportamiento de Calcular Área. Sin embargo, este comportamiento tiene diferentes formas de calcularse dependiendo qué abstracción de la jerarquía es la que la utilice:

- Para el caso de la abstracción rectángulo, el cálculo de área se obtiene luego de multiplicar el valor de la base y la altura de la figura.
- Para la abstracción círculo, se debe multiplicar la constante π (Pi) por el valor del radio al cuadrado.
- Para la abstracción triángulo se calcula diferente, en este caso, el cálculo del área se obtiene luego de multiplicar el valor de la base por la altura de la figura y dividir el valor entre dos.

2. Clases y objetos

Las clases y objetos son el corazón de la programación orientada a objetos y en la construcción de estos elementos, se refleja cada una de las características del paradigma orientado a objetos revisado en el apartado anterior. Algunas veces parece que ambos elementos se traslapan y algunas otras veces se suele hablar de forma indiscriminada de estos dos elementos, por lo cual, a continuación, se ampliará la revisión para comprender sus características y diferencias.

Un objeto en el paradigma orientado a objetos representa un objeto del mundo real, el objeto debe ser especializado y solo se encarga de una tarea.

Un objeto puede ser:

- Cosas tangibles como un avión, un auto o un televisor.
- Roles o papeles como un gerente, un cliente o un vendedor.
- Organizaciones como una empresa o un departamento.
- Interacciones como transacciones o contratos.
- Incidentes como vuelos, sucesos o accidentes.
- Lugares como muelles, carreteras, etc.

Los objetos se comunican con otros objetos por medio de mensajes y su estructura se describe en la siguiente tabla.

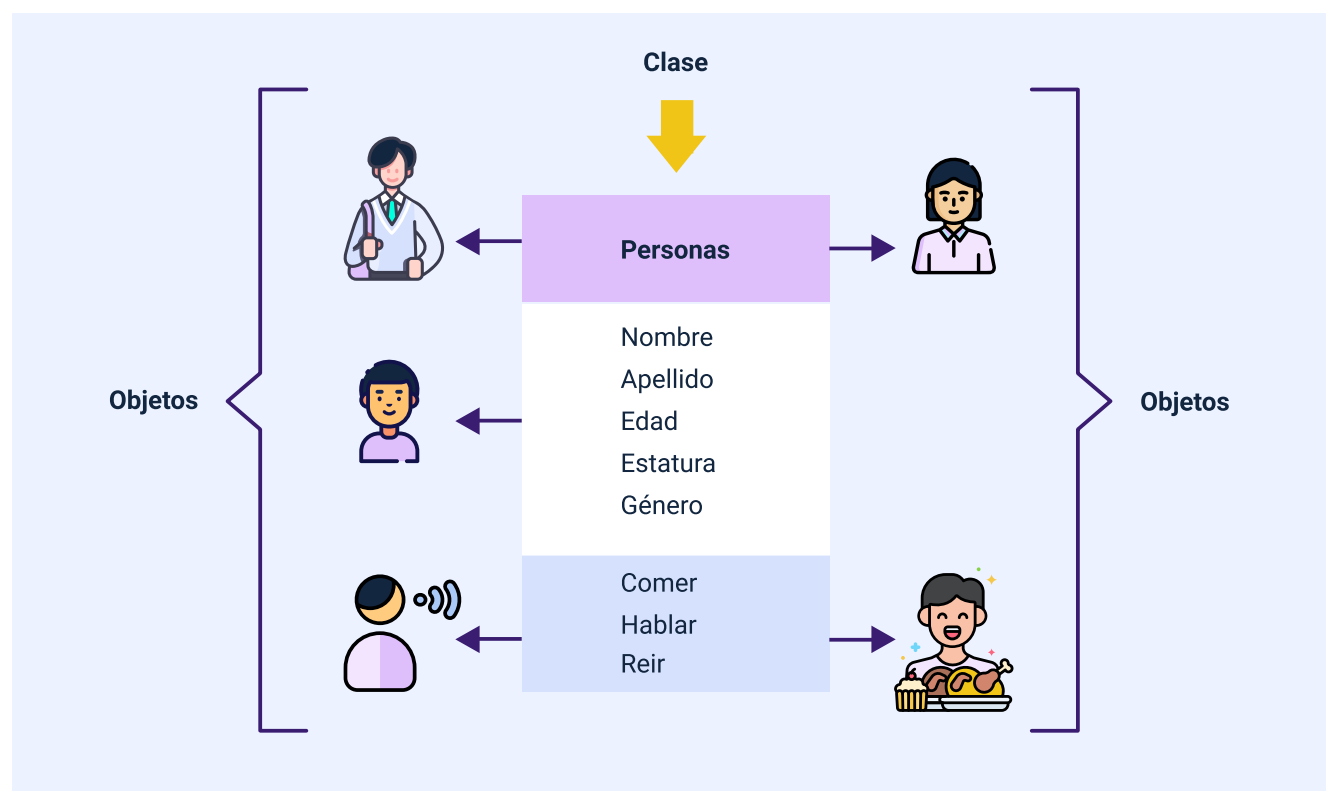
Tabla 1. Estructura de un objeto

Elemento	Descripción
Identidad (nombre).	Representa las características del objeto. Son los parámetros que lo definen y lo diferencian de objetos del mismo tipo.

Elemento	Descripción
Atributos y propiedades (estado).	Representa las características del objeto. Son los parámetros que lo definen y lo diferencian de objetos del mismo tipo.
Métodos.	Representan el comportamiento del objeto. Acciones que realizan o manejan los datos.

Ahora, una clase es una abstracción o visión generalizada de un conjunto de objetos que tienen características (atributos y propiedades) y métodos iguales o similares. Es decir, una clase representa las características comunes de un conjunto de objetos, como se muestra en la figura que se encuentra a continuación.

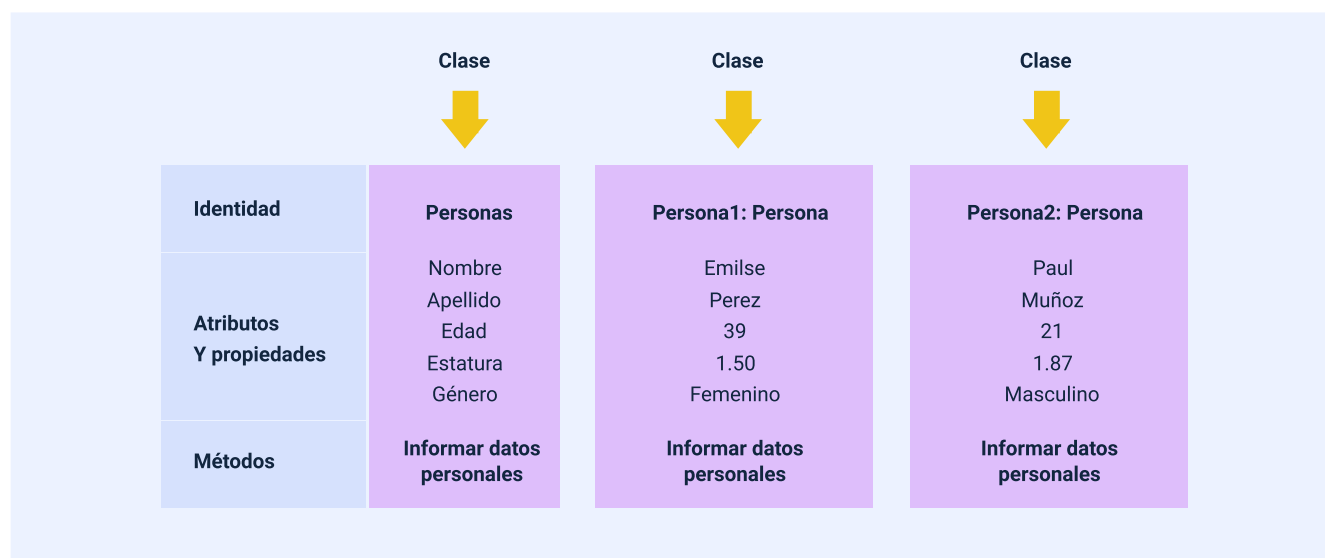
Figura 3. Ejemplo clases y objetos



Una clase es un tipo de dato definido por el usuario, no existe en forma concreta, debido a que es solo una abstracción. A partir de la definición de una clase, se pueden construir objetos. Las clases son equivalentes a los tipos de datos y los objetos son los identificadores que se crean a partir de este tipo particular de dato.

Así, una clase puede servir para definir (instanciar) objetos. Un objeto es una instancia de una clase totalmente independiente de otros objetos de la misma clase. La estructura de los objetos corresponderá a la estructura definida por la clase desde la cual se crean, así como se observa en la siguiente figura.

Figura 4. Estructura de una clase y objetos



Atributos

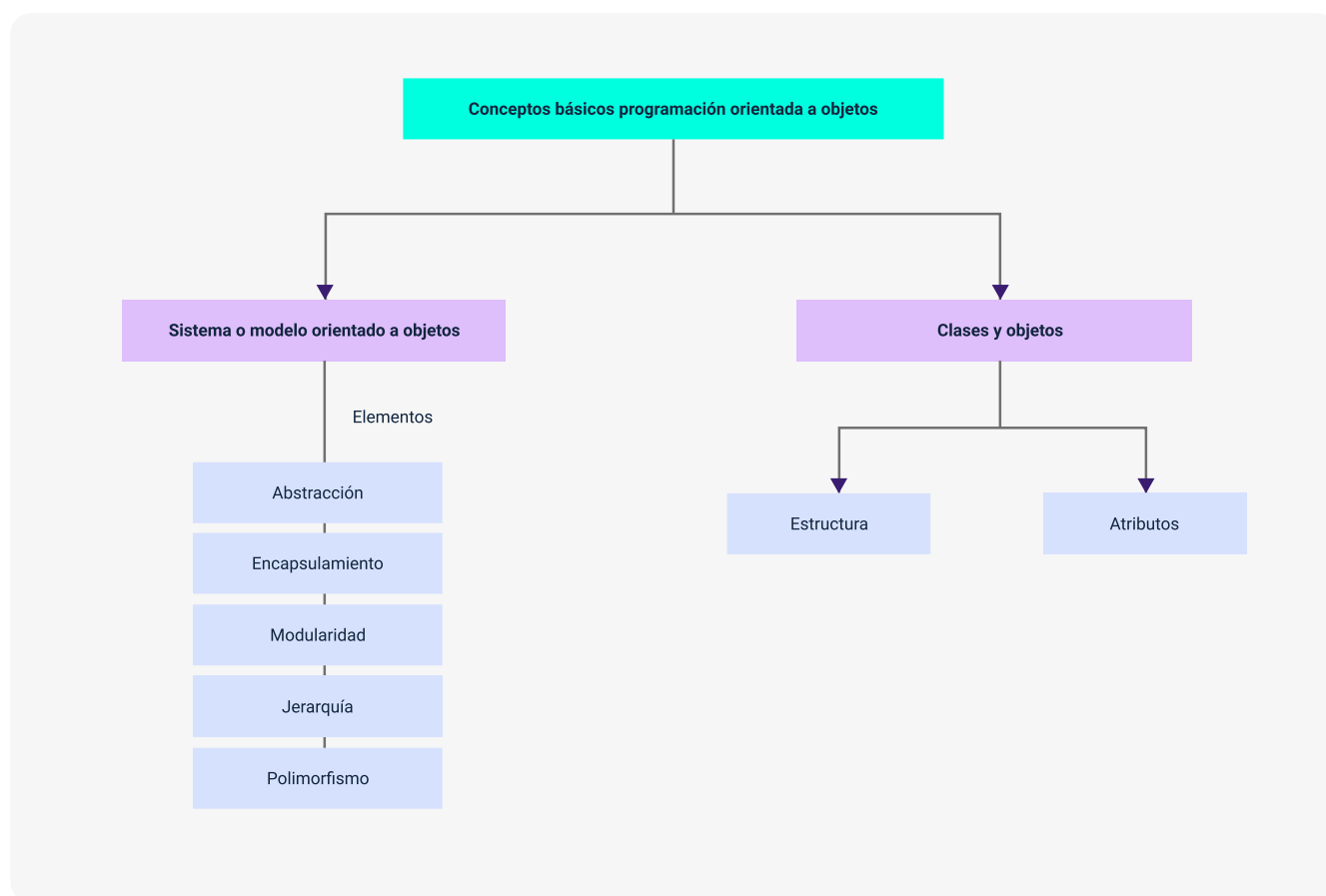
Los atributos de los objetos que se definen en las clases, cuando se describen en lenguaje de modelado formal o en un lenguaje de programación, se estructuran de forma similar a como se definen los identificadores, es decir, cada atributo se expresa con un tipo de dato y nombre del identificador.

Adicionalmente, tanto los atributos como los métodos tendrán una especificación, en la que se indica si corresponden a elementos públicos (interfaces visibles) o elementos privados (implementación interna), lo que representa una aplicación de la propiedad del encapsulamiento.

- a) Lo primero por hacer, es por medio de la abstracción, identificar de la información existente, cuáles son los posibles objetos y las posibles características claves para tener en cuenta al realizar un registro de matrícula y la información esencial relacionada con los estudiantes.
- b) Puede ser útil también, verificar de estudiantes reales el tipo de información que requiere la institución.
- c) Luego de esto, se empieza a construir las plantillas (clases) con la estructura de los elementos identificados. Posteriormente, estas clases definidas, permitirán instanciar objetos que corresponden a la información concreta con la que se manipulará el sistema.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Paradigma orientado a objetos	EDteam. (2019). ¿Qué es la programación orientada a objetos? La mejor explicación en español.	YouTube	https://youtu.be/DlphYPc_HKk
Paradigma orientado a objetos	Soy Dalto. (2019). Programación orientada a objetos explicada en 10 minutos.	YouTube	https://youtu.be/uNlB7141umY

Glosario

Algoritmo: secuencia de pasos ordenados y finitos que describen un conjunto de acciones para resolver un problema.

Paradigma: ejemplo o modelo de algo. En el caso del paradigma orientado a objetos corresponde a un conjunto de características y formas en la que se representa la información.

Referencias bibliográficas

Aguilar, J. L. (2021). Fundamentos de programación. McGraw Hill Education.

Aguilar, L. J. (1998). Programación orientada a objetos. McGraw Hill Education.

Booch, G., Maksimchuk, R. A., Engle, M. W., Conallen, J., Young, B. J. & Houston, K. A. (2007). Object-oriented Analysis and Design with Applications. Addison-Wesley.

Rumbaugh, J., Jacobson, I., & Booch, G. (2004). El lenguaje unificado de modelado. Addison-Wesley.

Samuel. (2020). ¿Qué es la POO?. [Web log post] Fundamentos de las POO.
<http://micanalsamuelc.blogspot.com/2017/03>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Líder del Ecosistema	Dirección General
Olga Constanza Bermudez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Zulema Yidney León Escobar	Experta temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Jonathan Guerrero Astaiza	Experto temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Carlos Julián Ramírez Benítez	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para contenidos inclusivos y accesibles	Centro de Servicios de Salud - Regional Antioquia

Conceptos, tecnologías y arquitectura para el desarrollo web

Breve descripción:

En este componente formativo se abordarán los conceptos relacionados con el funcionamiento de un sistema web, soportado por la arquitectura de internet. A continuación, se revisarán las tecnologías más importantes para el desarrollo web, tanto del lado del cliente como del servidor. Por último, se explorarán los conceptos básicos de una arquitectura orientada a servicios.

Junio 2024

Tabla de contenido

Introducción	4
1. Internet y la web	5
1.1. Fundamentos de la internet	5
Historia de la internet.....	5
Hardware de red	6
Redes de área local LAN	10
Modelo de referencia TCP/IP	11
Proceso de comunicación	13
Funcionamiento del modelo.....	13
Cuatro capas del modelo TCP/IP.....	13
Dirección IP.....	15
Sistema de Nombres de Dominio (DNS)	16
Protocolo HTTP.....	16
1.2. Arquitectura web	17
Arquitectura cliente/servidor	17
Cliente web.....	18
Servidor web	18
2. Introducción a las tecnologías para el desarrollo web	22

2.1. Tecnologías del front-end	23
HTML5	23
CSS3	24
JavaScript (JS)	25
2.2. Tecnologías del back-end	28
Lenguajes del servidor	29
Frameworks del back-end	30
3. Introducción a la arquitectura orientada a servicios	32
SOAP	33
REST	34
Síntesis	35
Material complementario	36
Glosario	37
Referencias bibliográficas	38
Créditos	40

Introducción

En este componente formativo, se abordarán los conceptos fundamentales relacionados con la arquitectura de sistemas web. Se examinará la naturaleza de su comportamiento, así como los componentes físicos y lógicos esenciales para su operación. Además, se identificarán y caracterizarán las tecnologías web del lado del cliente y del servidor, avanzando hacia la comprensión de las tecnologías que facilitan la comunicación y transferencia de información o la creación de servicios con otros sistemas.

Este componente tiene como objetivo fomentar un análisis crítico para la elección de tecnologías específicas, dependiendo de los requerimientos y el alcance de un proyecto estimado. Este análisis se sustenta en la descripción y rasgos distintivos que se detallan en cada sección de este documento.

Se profundizará en la arquitectura de un sistema web, explorando cómo su comportamiento depende tanto de los componentes físicos como lógicos, y cómo estos facilitan la identificación y caracterización de tecnologías web tanto del lado del cliente como del servidor. Estas tecnologías permiten la comunicación y el traspaso de información, así como la generación de servicios con otros sistemas.

En este componente formativo, se desarrollarán conocimientos sobre la arquitectura web y el funcionamiento de internet. También se introducirán las tecnologías para el desarrollo web, como el front-end y el back-end, y la arquitectura orientada a servicios, incluyendo SOAP, REST y API-REST.

1. Internet y la web

El desarrollo de aplicaciones y sistemas web ha sido una tendencia predominante en las últimas tres décadas, y sus mejoras han evolucionado en consonancia con las innovaciones en la infraestructura tecnológica que las respalda. Actualmente, las aplicaciones son notablemente más interactivas, rápidas, seguras y fáciles de usar. Un ejemplo de esto es el acceso a sitios web desde dispositivos móviles, la visualización de videos en YouTube, las conversaciones a través de WhatsApp, o el streaming de series en Netflix. Todos estos avances son posibles gracias a una robusta infraestructura de hardware que permite el despliegue no solo de aplicaciones sino también de servicios que se utilizan diariamente. Por ello, es crucial comprender los componentes fundamentales que enriquecen y otorgan rigurosidad a la construcción de este tipo de sistemas.

1.1. Fundamentos de la internet

Internet emergió ante la necesidad de comunicación e interacción con personas en cualquier parte del mundo. Esta tecnología se considera el pilar de la web y se define como la infraestructura técnica que la hace posible. En su forma más básica, se presenta como una vasta red de computadoras comunicándose simultáneamente.

Historia de la internet

La historia de la internet se remonta a inicios de la década de 1960 con un proyecto de investigación denominado ARPANET, financiado por el ejército de los Estados Unidos. En la década de 1980, se transformó en una infraestructura pública con el apoyo de diversas instituciones del sector público y privado. Las tecnologías que

soportan la internet han evolucionado a lo largo del tiempo para satisfacer la creciente demanda de la población, aunque la forma en que funciona ha cambiado poco.

Internet se presenta como un medio de conectividad entre todos los equipos de cómputo y, debido a su distribución y organización, se busca continuamente la mejor manera de mantener conectados a los usuarios.

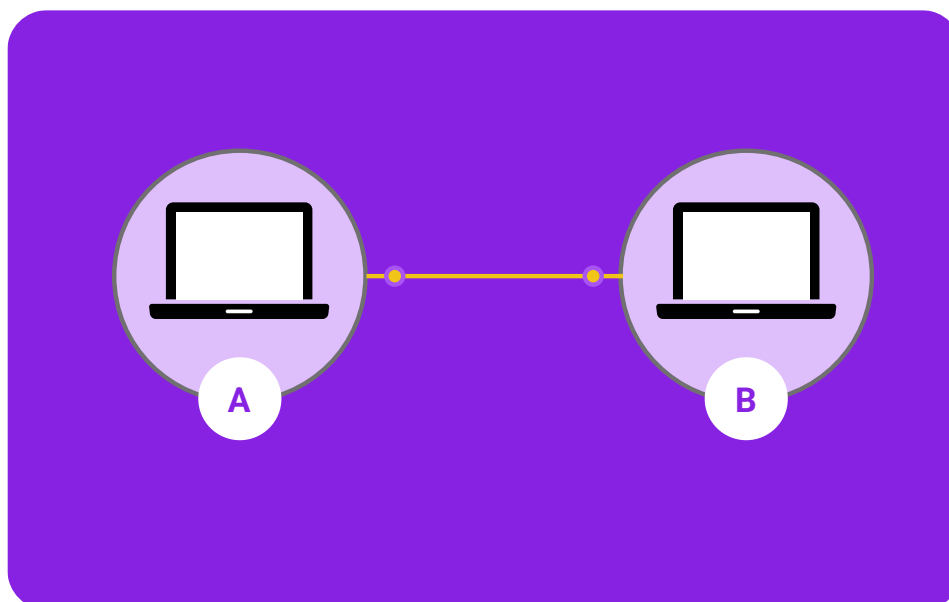
Hardware de red

Cuando dos computadoras necesitan comunicarse, pueden vincularse mediante medios físicos como el cable Ethernet, o de manera inalámbrica utilizando tecnologías como WiFi, Bluetooth, ZigBee, entre otras.

- **Ejemplo de conexión entre dos equipos o terminales**

La mayoría de las computadoras de hoy en día pueden soportar cualquiera de este tipo de conexiones. A continuación, se presenta un ejemplo de conexión vía Ethernet entre dos equipos.

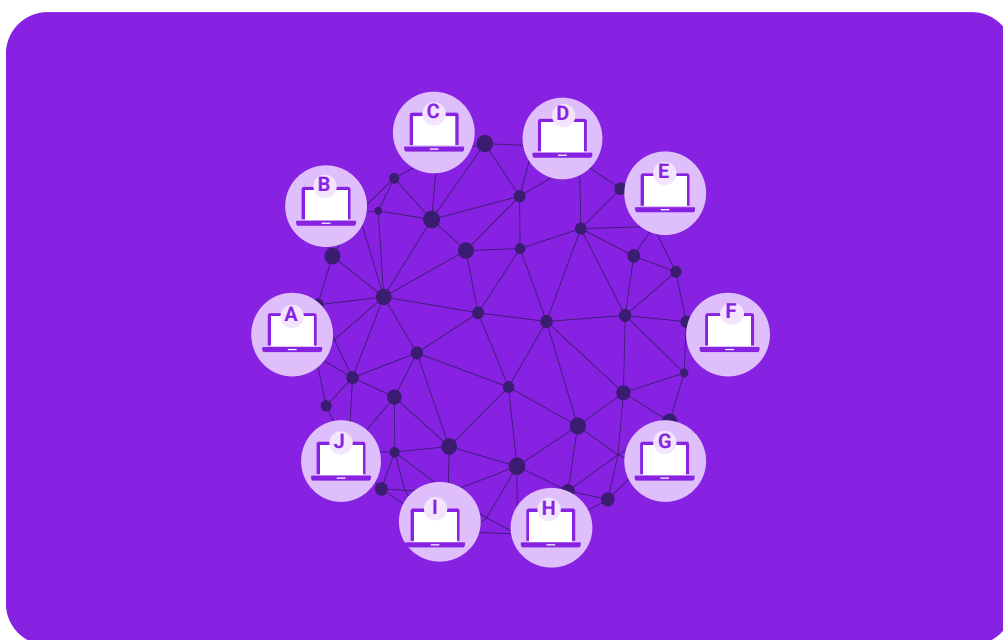
Figura 1. Conexión vía Ethernet entre dos equipos



- **Ejemplo de conexión entre diez equipos o terminales**

No se puede pensar que la red solo se limita a dos ordenadores, se pueden conectar tantos como se deseen, aunque con mayor complejidad. Por ejemplo, si se quieren conectar 10 computadoras, se necesitarían 45 cables y unos nueve conectores por computador, como se alcanza a apreciar en la siguiente figura.

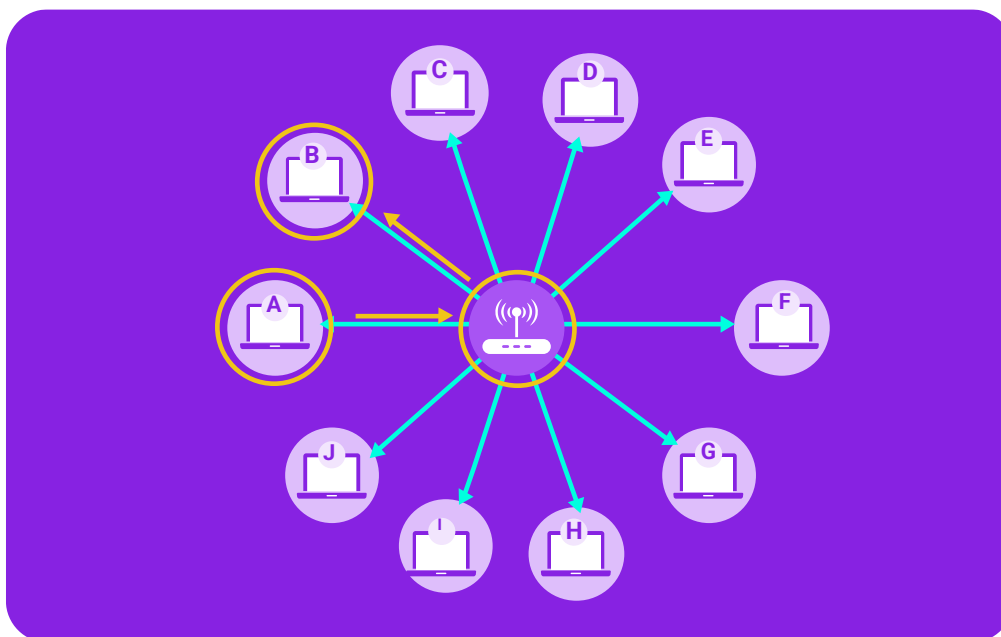
Figura 2. Conexión entre diez equipos o terminales



- **Ejemplo de conexión de diez equipos con un enrutador**

Por ejemplo, para que el terminal B reciba un mensaje desde el terminal A, este debe enviarlo primero al router, quien a su vez lo remite al ordenador B asegurándose que dicho mensaje no sea entregado a otro ordenador. Una vez incorporado un enrutador al sistema, la red de diez terminales solo requiere diez cables: un conector para cada ordenador y un enrutador con 10 conectores, como se aprecia en la siguiente gráfica.

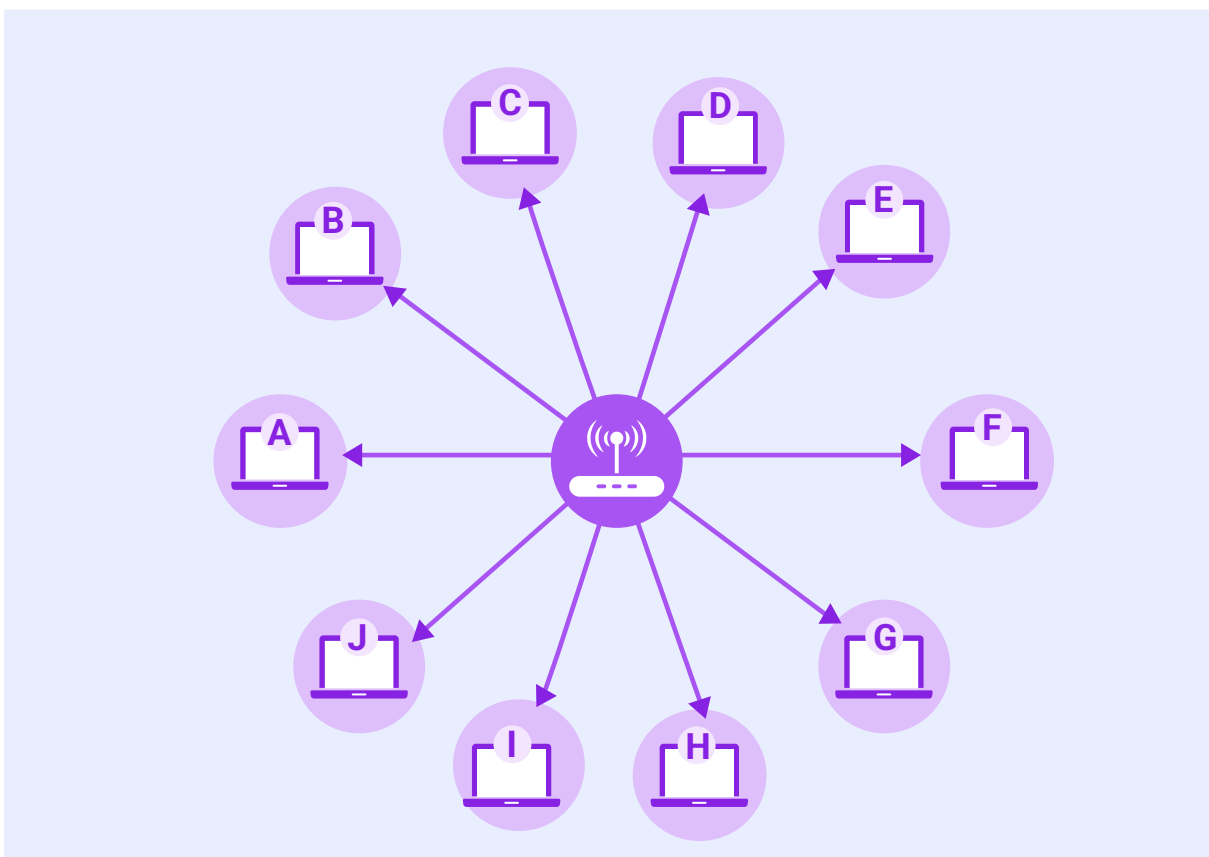
Figura 3. Conexión de diez equipos con un enrutador



Reflexionando sobre el ejemplo anterior, surge la pregunta: ¿qué ocurre al conectar cientos, miles o millones de ordenadores entre sí? Claramente, un solo enrutador no puede gestionar una escala tan amplia. Aunque un enrutador funcione similarmente a un pequeño computador, esto no implica que conectar múltiples enrutadores sea una tarea sencilla.

El proceso de conexión entre varios terminales puede ser muy complejo y laborioso. Por ello, para organizar estas conexiones de manera eficiente, cada ordenador en una red está conectado a un dispositivo especializado denominado enrutador o router. Este dispositivo actúa de manera similar a los señalizadores en las estaciones del sistema integrado de transporte público, dirigiendo el tráfico para garantizar que llegues a tu destino correctamente. Así, el router asegura que los mensajes enviados desde un computador o terminal emisor alcancen el destino adecuado.

Figura 4. Conector para cada ordenador y enrutador con 10 conectores

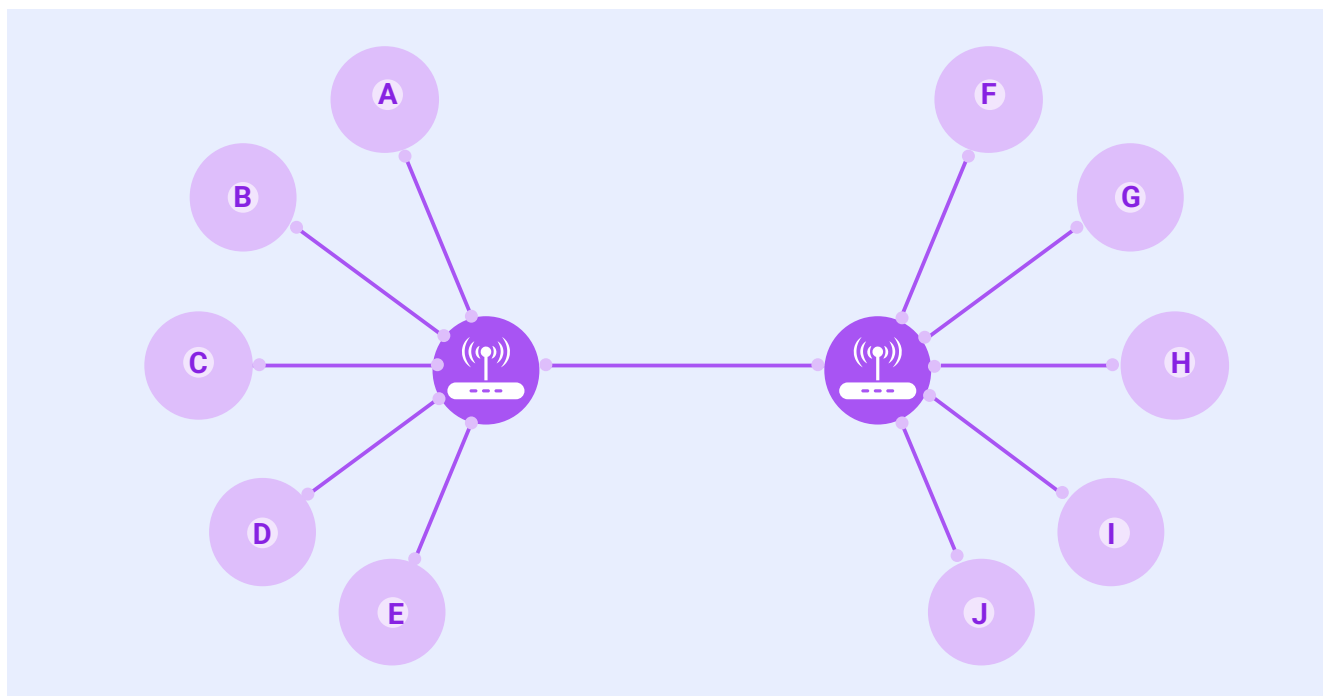


Al incorporar un enrutador al sistema, una red de diez terminales solo necesitaría diez cables: un conector para cada ordenador y un enrutador equipado con 10 conectores.

Ejemplo de conexión de diez equipos con dos enrutadores

En ese sentido, estas conexiones entre terminales generan redes de datos de transmisión como la recepción de información, por lo que es necesario caracterizar estas redes, las cuales son importantes para la comprensión del funcionamiento de un sistema web. Entre ellas se encuentran las redes de área local, una red de área metropolitana y una red de área amplia.

Figura 5. Conexión de diez equipos con dos enrutadores

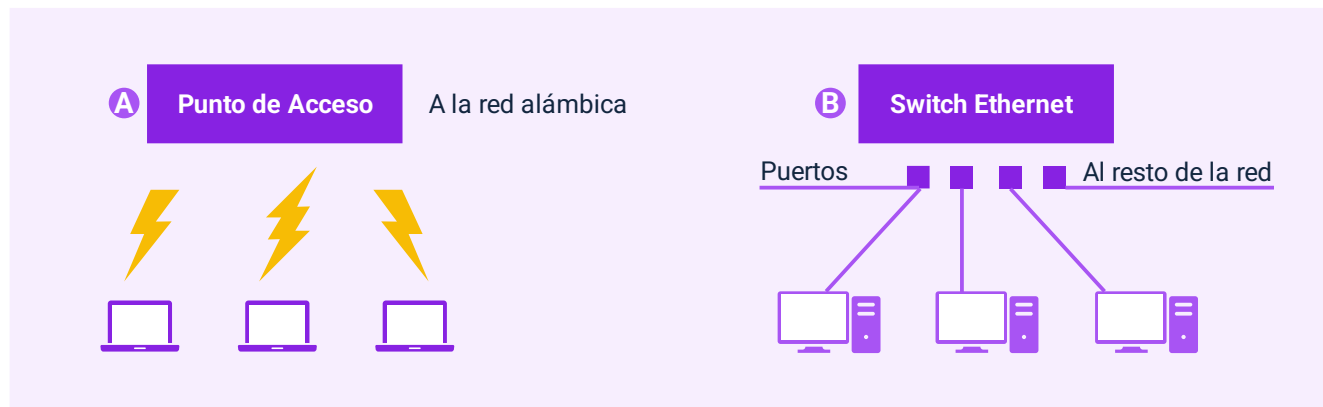


Redes de área local LAN

Generalmente llamadas LAN (Local Area Networks), son redes de naturaleza privada que operan dentro de un solo establecimiento, como una casa, oficina o fábrica. Las redes LAN se utilizan ampliamente para conectar computadoras personales y electrodomésticos, con el fin de compartir recursos (por ejemplo, impresoras) e intercambiar información.

A continuación, se presenta un ejemplo de esta red:

Figura 6. Red de área local inalámbrica y cableada



El diseño físico y lógico de una red, conocido como topología, se basa frecuentemente en enlaces de punto a punto para redes alámbricas. El estándar IEEE 802.3, más conocido como Ethernet, es la forma más común de establecer una LAN (Red de Área Local) alámbrica. La figura en su apartado b ilustra un ejemplo de una topología de Ethernet conmutada, donde cada computadora se comunica a través del protocolo Ethernet y se conecta a un dispositivo conocido como switch mediante un enlace de punto a punto.

En este contexto, un switch dispone de varios puertos, cada uno de los cuales puede conectarse a una computadora. La función principal del switch es transmitir paquetes entre las computadoras que están conectadas a él. Utiliza la dirección incluida en cada paquete para determinar a qué computadora debe enviarlo, tal como se presenta en el ejemplo de la red de área cableada mencionado.

Modelo de referencia TCP/IP

Al igual que los seres humanos requieren de un lenguaje común para comunicarse, es esencial que todos los equipos de cómputo dispongan de un modo

estandarizado para interactuar entre ellos. Para la mayoría de los equipos modernos, este estándar es el TCP/IP. Generalmente, el TCP/IP viene preinstalado en los dispositivos y opera de manera automatizada. Sin embargo, es crucial entender el modelo TCP/IP, especialmente si se necesita configurar un equipo para conectarlo a otro sistema. (Fisher, 2021).

El protocolo TCP/IP, cuyas siglas significan Transmission Control Protocol/Internet Protocol (Protocolo de Control de Transmisión/Protocolo de Internet), constituye un conjunto de reglas estandarizadas que facilitan la comunicación de los equipos en redes como Internet. La eficacia de estas máquinas radica en su velocidad de transmisión y en su capacidad para comunicarse con otras, un aspecto fundamental en la era de la cuarta revolución industrial, caracterizada por la interacción inteligente de equipos, especialmente en contextos de producción.

La dependencia de nuestras actividades diarias, como enviar correos electrónicos, ver Netflix o recibir indicaciones de navegación, se basa en la interacción continua entre distintos dispositivos. Estos equipos pueden ser de diferentes fabricantes y ubicarse en diversas zonas geográficas. Así, tanto las personas como las plataformas de software que los utilizan pueden emplear distintos lenguajes, tanto humanos como informáticos.

El blog de Avast Academy señala que una interacción determinada puede ocurrir entre dos sistemas informáticos o involucrar a cientos de ellos. Sin embargo, al igual que el proceso de pasar una carta o un paquete de mano en mano, cada transacción se realiza entre solo dos equipos a la vez. Para que esta comunicación sea efectiva, ambos equipos deben conocer de antemano cómo se espera que se comuniquen.

Proceso de comunicación

Según (Fishre,2021), la comunicación nace a partir de las siguientes incógnitas:

- ¿Cómo inician la conversación?
- ¿A quién le toca comunicarse?
- ¿Cómo sabe un equipo si su mensaje se ha transmitido correctamente?
- ¿Cómo terminan la conversación?

De esta manera, es evidente que los equipos necesitan reglas, instrucciones o en su término técnico, protocolos para la comunicación y el comportamiento entre esas dos instancias.

Funcionamiento del modelo

El funcionamiento de este protocolo radica en descomponer cada mensaje en paquetes. Porque el objetivo no es volver a enviar de nuevo y desde cero el mensaje completo, toda vez que haya un fallo. Así que estos paquetes se vuelven a ensamblar en el otro extremo. De hecho, cada paquete podría tomar una ruta diferente hasta el equipo de destino, toda vez sí la ruta deja de estar disponible o está muy congestionada. De manera adicional, TCP/IP divide las distintas tareas de comunicación en capas. Así, cada capa tiene una función diferente. Los datos pasan por cuatro capas independientes antes de recibirse en el otro extremo.

Cuatro capas del modelo TCP/IP

En la literatura, se encuentran un sin número de definiciones alrededor de las capas de este modelo, entre las más importante y haciendo un resumen para dar mayor claridad se tienen:

- **Capa de internet**

La capa de internet (también denominada capa de red), es la capa encargada de controlar el movimiento (envío y recepción) de los paquetes alrededor de la red.

- **Capa de aplicación**

La capa de aplicaciones es el grupo de aplicaciones que requiere comunicación de red. Es con lo que el usuario o cliente suele interactuar, como el correo electrónico y la mensajería. Como la capa inferior es la encargada de gestionar los detalles de la comunicación, no tienen que preocuparse por ello.

- **Capa de enlace de datos**

La capa de enlace de datos (igualmente denominada como capa de enlace, capa de interfaz de red o capa física) es la encargada de manejar las partes físicas del envío y recepción de datos mediante el cable Ethernet, la tarjeta de interfaz de red, la red inalámbrica, el controlador del dispositivo en el equipo, entre otros.

- **Capa de transporte**

La capa de transporte es la capa encargada que proporciona una conexión de datos fiable entre dos dispositivos. Divide los datos en paquetes, hace acuse de recibo de los paquetes que recibe del otro dispositivo y además se asegura de que el otro dispositivo haga el acuse de recibo pertinente de los paquetes que recibe a su vez.

La siguiente tabla presenta las capas y sus respectivos protocolos.

Tabla 1. Principales protocolos del Modelo TCP/IP por capas

Modelo TCPS / IP	Suite de protocolos (Principales)	Suite de protocolos (Principales)	Suite de protocolos (Principales)	Suite de protocolos (Principales)
Capa de Aplicación	SSH	FTP	SMTP	DHCP
Capa de aplicación	DNS	RIP	SNMP	HTTP
Capa de transporte	TCP	DCCP	UTP	N/A
Capa de transporte	UDP	ICMP	FCP	N/A
Capa de Internet	IP	ICMP	N/A	N/A
Capa de Internet	IPSEC	IGMP	N/A	N/A
Capa de interfaz de Red	ARP	L2TP	N/A	N/A
Capa de interfaz de Red	NDP	ETHERNET	N/A	N/A

Dirección IP

Para comunicarse con una computadora específica en una red, es necesario especificar su dirección. Por esta razón, cada computadora conectada a una red tiene una dirección única que la identifica de manera lógica y jerárquica, conocida como “dirección IP” (Internet Protocol). Esta dirección consta de una serie de cuatro números separados por puntos, por ejemplo: 192.168.2.19. Aunque para las computadoras este identificador es simple, los humanos pueden tener dificultades para recordarlo y memorizarlo. Para facilitar la asociación de esta serie numérica, se utilizan los nombres de dominio.

Sistema de Nombres de Dominio (DNS)

Los DNS funcionan como una libreta de direcciones de sitios web. Cuando se introduce una dirección web en el navegador, esta consulta los DNS antes de recuperar el sitio web. El navegador necesita determinar en qué servidor se aloja el sitio web para enviar los mensajes HTTP correctamente. Significa que es análogo, el buscar la dirección de una tienda para poder visitarla.

Por ejemplo, google.com es el nombre de dominio que se utiliza para representar la dirección IP 173.194.121.32, facilitando la identificación de un ordenador en internet.

Protocolo HTTP

El Protocolo de Transferencia de Hipertexto (HTTP) es un protocolo de aplicación que establece un lenguaje para la comunicación entre clientes y servidores. Es similar al idioma que se usa para hacer un pedido en una tienda. HTTP permite solicitar datos y recursos, como documentos HTML, y es la base de cualquier intercambio de datos en la web. Funciona con una estructura cliente-servidor, lo que significa que una petición de datos es iniciada por el elemento que recibirá los datos (el cliente), generalmente un navegador web.

Se han abordado conceptos clave sobre los componentes que conforman la arquitectura de internet. Como se ha explicado, internet es una infraestructura técnica que permite la conexión de miles de millones de computadoras. Algunas de estas, conocidas como servidores web, son capaces de enviar mensajes inteligibles a los navegadores. Así, internet constituye la infraestructura, mientras que la web es un

servicio construido sobre ella. Además, existen otros servicios soportados por internet, como el correo electrónico.

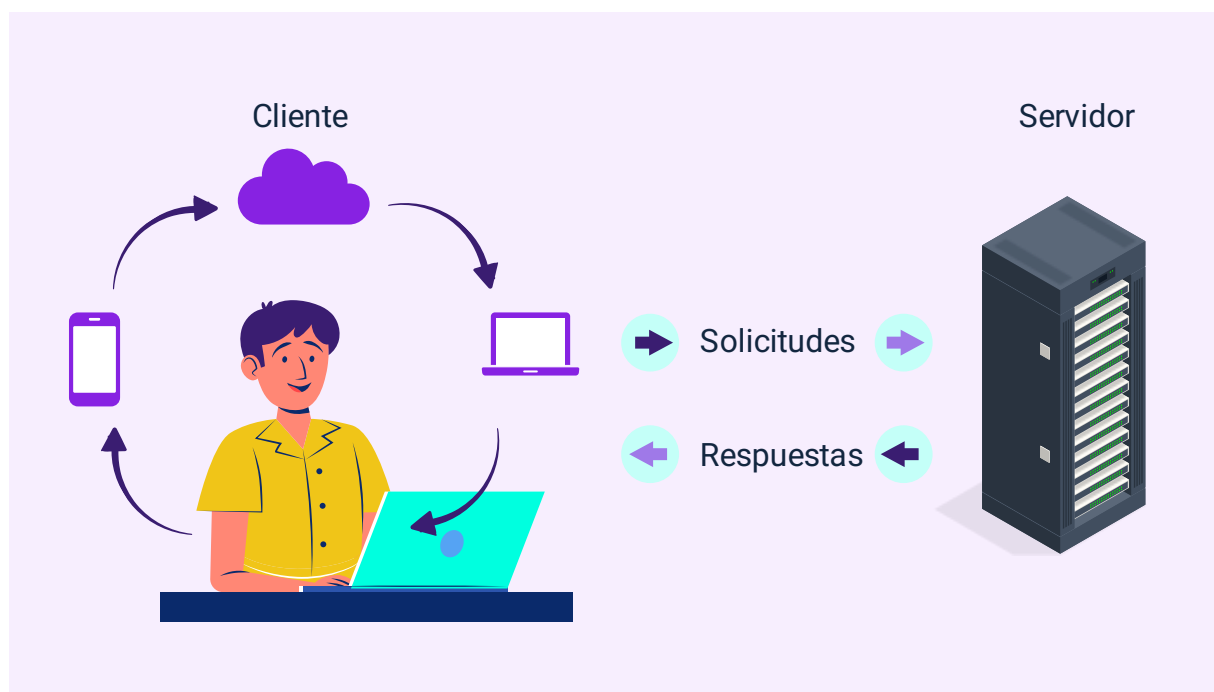
1.2. Arquitectura web

Para comprender el funcionamiento de un sistema, plataforma o aplicación web, es fundamental conocer los conceptos y el comportamiento de las tecnologías que se integran en esta arquitectura. A continuación se detalla:

Arquitectura cliente/servidor

Una aplicación web opera bajo una arquitectura simplificada conocida como cliente/servidor. En este modelo, el "cliente" generalmente se refiere a un navegador web instalado en un dispositivo del usuario, que se encarga de mostrar y solicitar información o documentos a través de una red. Las computadoras conectadas a la web se clasifican como clientes y servidores. Un diagrama simplificado de cómo interactúan se podría visualizar de la siguiente manera:

Figura 7. Arquitectura web



Cliente web

El cliente web es una aplicación ejecutable, usualmente preinstalada en el dispositivo del usuario, que facilita el acceso a internet. Este cliente es capaz de mostrar y solicitar documentos sobre una red, siendo así responsable de la capa de presentación. Los clientes web son dispositivos de usuarios conectados a internet (por ejemplo, un ordenador conectado a la red Wi-Fi o un teléfono conectado a la red de telefonía móvil) y el software disponible que permite acceder a internet en dichos dispositivos (normalmente, un navegador web como Firefox o Chrome).

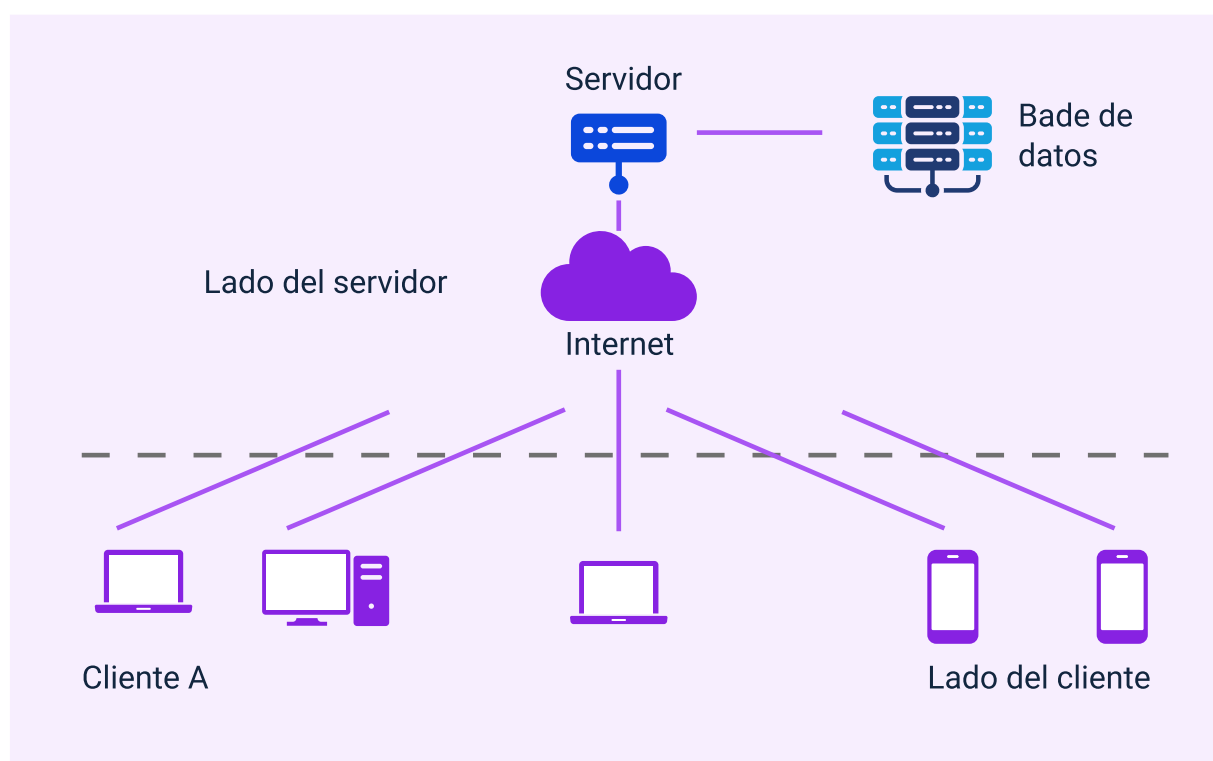
Servidor web

En la arquitectura web, el servidor web juega un papel crucial, pues se encarga de atender las solicitudes enviadas por los clientes a través de los navegadores web. Es en el servidor donde se realiza todo el procesamiento de las aplicaciones y la gestión de los datos. Los servidores son computadoras que almacenan páginas web, sitios o

aplicaciones. Cuando un dispositivo cliente desea acceder a una página web, una copia de esta página se descarga desde el servidor al equipo cliente y se presenta en su navegador web. En esencia, un servidor es una computadora potente que almacena y transmite datos vía internet.

A continuación, se presenta un esquema que facilitará la comprensión de los diversos aspectos relacionados con esta arquitectura.

Figura 8. Arquitectura cliente servidor

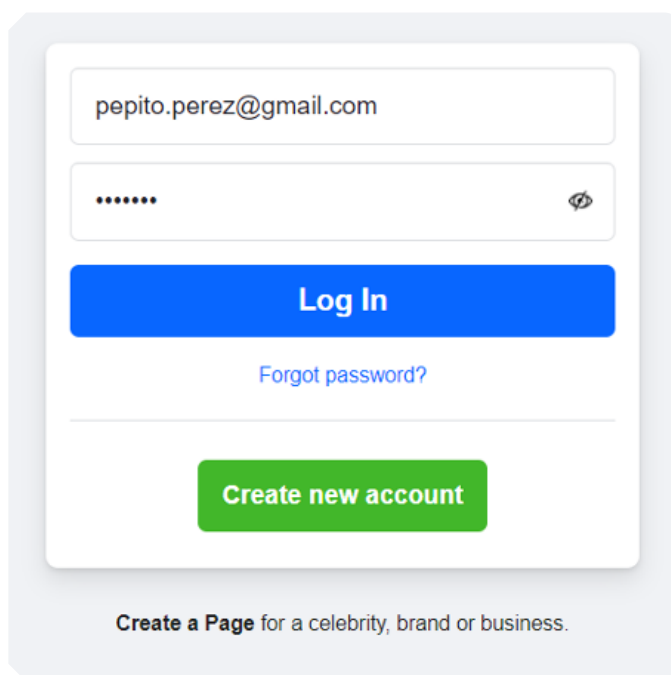


Se inicia con un ejemplo en el que se tiene al cliente A, conforme se presenta en el esquema anterior. Este cliente, que es una computadora o un ordenador de escritorio, cuenta con un explorador web instalado. El usuario, Pepito Pérez, necesita ingresar a Facebook. Para acceder a la red social, es necesario registrar las credenciales, como el login y la contraseña. Al hacer clic en iniciar sesión, se genera inmediatamente

una petición que viaja desde el cliente hasta el servidor, tal como se ilustra en el esquema de la arquitectura.

El servidor, por su parte, procesa los datos de Pepito y verifica la información con otros servidores y bases de datos, como se observa en la figura. Dependiendo de si la validación de los datos es afirmativa o no, el servidor enviará una respuesta al usuario, permitiéndole o negándole el acceso a su espacio en la red social. Esta respuesta del servidor se transmite en un lenguaje y con recursos que son interpretados y manejados por el explorador web para ser mostrados en pantalla. Esto indica que existen tecnologías tanto del lado del cliente como del lado del servidor, por lo que el servidor, además de procesar los datos, debe responder en el formato o lenguaje adecuado para que el cliente pueda interpretarlo correctamente.

Figura 9. Cuadro de login y password de Facebook



The image shows a screenshot of the Facebook login and registration interface. It features a white card with rounded corners on a light gray background. At the top, there is a text input field containing the email address 'pepito.perez@gmail.com'. Below this is a password input field represented by a series of dots, with an eye icon to its right for toggling visibility. A prominent blue 'Log In' button is centered below the password field. Underneath the button is a blue link that says 'Forgot password?'. A horizontal line separates the login section from the registration section, which contains a green 'Create new account' button. At the very bottom of the card, there is a link that says 'Create a Page for a celebrity, brand or business.'

Este ejemplo se puede realizar desde cualquier otro dispositivo, es por eso por lo que la arquitectura web es considerada como un ambiente multiplataforma y distribuido.

Finalmente, es importante tener en cuenta que una web sigue las siguientes pautas:

- **¿Cómo se realiza la transferencia de información?**

Cada solicitud y respuesta se envía utilizando el protocolo HTTP. Es mediante este protocolo que se transmiten los datos.

- **¿Cómo se describe la información apenas llega al cliente como respuesta?**

Para la presentación de la información web se utilizan tecnologías del lado del cliente, es decir, HTML, CSS y JavaScript en sus versiones más recientes. Estos lenguajes de programación y diseño permiten estructurar el contenido, definir su estilo y manejar comportamientos dinámicos en la página.

- **¿Cómo se localiza la información?**

La información se localiza mediante la dirección URL, la cual se compone de tres partes: primero el protocolo, segundo la ruta del servidor y tercero el dominio. La estructura queda de la siguiente forma:

<http://rutadelserveridor/ubicacion/dominio>. Esta estructura asegura que el navegador pueda identificar y acceder a la ubicación exacta del recurso solicitado en la web.

2. Introducción a las tecnologías para el desarrollo web

Las tecnologías de desarrollo web abarcan una amplia gama de lenguajes y herramientas de programación que se utilizan para crear sitios web, aplicaciones dinámicas y plataformas, dotadas de todas las funciones necesarias para satisfacer una diversidad de necesidades y requerimientos de los usuarios. Estas tecnologías están en constante evolución; no es necesario realizar un análisis exhaustivo sobre su estado de transformación para notar el creciente auge de diversos lenguajes de programación que facilitan la realización de acciones más complejas y efectivas en los sitios web. Por esta razón, las tecnologías web son consideradas multiplataforma, una característica que se resalta en la afirmación del grupo de expertos de Ingenio Virtual.

... Los modelos y tecnologías de desarrollo web han evolucionado significativamente en la última década. Existen multitud de aplicaciones, frameworks, librerías, arquitecturas y sistemas de publicación que en sus diferentes versiones reciben actualizaciones y mejoras continuas. El progreso también se ha observado en la administración de sistemas, los servicios de alojamiento, técnicas de escalabilidad, monitorización y gestión de centros de procesos de datos. Esta evolución ha propiciado la convergencia de una gran cantidad de tecnologías, herramientas y estilos arquitectónicos para el desarrollo de sitios web y aplicaciones. (Ingenio Virtual, 2018).

Esta tendencia subraya la importancia de mantenerse actualizado con las últimas tecnologías y enfoques en el campo del desarrollo web, para poder aprovechar al máximo las capacidades que ofrecen estas herramientas modernas.

2.1. Tecnologías del front-end

Las tecnologías de front-end se utilizan para el desarrollo del "lado cliente" de un sitio web o aplicación. Estas tecnologías son responsables de desarrollar los componentes interactivos y producir los elementos visuales con los que los usuarios interactúan, como colores, estilos de texto, imágenes, botones y menús de navegación, entre otros objetos interactivos. Estas tecnologías y lenguajes están implementados en los diferentes navegadores, facilitando así su uso y compatibilidad.

HTML5

HTML, abreviatura de Hyper Text Markup Language, es una de las tecnologías esenciales para el desarrollo de todo tipo de aplicaciones web. Proporciona la estructura base para una página web. El código HTML garantiza que el contenido de un sitio web esté adecuadamente formateado y estructurado, permitiendo que cada navegador de Internet muestre el contenido como está previsto. Sin HTML, los navegadores no podrían mostrar texto, cargar imágenes ni otros elementos esenciales.

HTML5, la versión más reciente de HTML, incluye un conjunto más amplio de tecnologías que permiten a los sitios web y a las aplicaciones ofrecer más diversidad y potencia. Este conjunto es conocido como HTML5 y ofrece una gran variedad de recursos, incluyendo librerías, plugins y APIs que se pueden utilizar junto con lenguajes de programación como JavaScript para crear sitios web más interactivos y dinámicos.

Algunas de las tecnologías más utilizadas en HTML5 incluyen:

- **Canvas**

Es un elemento HTML5 que se utiliza para dibujar imágenes y formas con el fin de manipularlas. También se puede utilizar para casos como gráficos y animaciones de juegos.

- **Almacenamiento web**

Se utiliza para almacenar información directamente en el navegador.

Algunos ejemplos de esto serían: almacenar la información de inicio de sesión del usuario y guardar las preferencias del usuario para un sitio web.

- **Web Workers**

Esta tecnología habilita un script que sigue ejecutándose en segundo plano cuando se abre una página web y se usa principalmente en sitios web con capacidades sin conexión. Hace que las páginas estén disponibles sin conexión y permite el uso de notificaciones como web Push. Puede enviar estas notificaciones incluso cuando su navegador no está abierto.

- **Web Sockets**

Permite una conexión bidireccional persistente entre el usuario y el servidor. Los casos de uso más comunes incluyen chats y notificaciones en aplicaciones web.

CSS3

Hojas de Estilo en Cascada, conocidas por sus siglas en inglés como Cascading Style Sheets (CSS), son el lenguaje de estilos utilizado para describir la presentación de documentos HTML o XML. CSS especifica cómo deben ser renderizados los elementos estructurados, es decir, se encarga de la apariencia del sitio web.

- **Cascading**

Se refiere a que los estilos aplicados a los elementos de una página web se propagan a los elementos contenidos en estos, aplicándose en cascada.

- **Style**

A través de la tecnología CSS, se aplican estilos visuales a los distintos elementos de la página web.

- **Sheets**

Traducido como hojas, porque los estilos de una página web generalmente se añaden en archivos separados. Estos archivos deben tener la extensión .css.

Desde la introducción de CSS3, el alcance de las especificaciones se ha incrementado de forma significativa, y el desarrollo de los distintos módulos de CSS ha comenzado a mostrar varias diferencias. Con CSS3, se ofrece un mayor control sobre los elementos del sitio, gracias a sus nuevas funciones y atributos aplicables a las etiquetas HTML, lo que maximiza las acciones visuales del sitio o plataforma.

JavaScript (JS)

Existen numerosas definiciones de este lenguaje de programación en la literatura. Sin embargo, una definición particularmente notable se encuentra en el blog de Rafa Ramos, que afirma literalmente:

JavaScript es el lenguaje de programación encargado de dotar de mayor interactividad y dinamismo a las páginas web. Cuando JavaScript se ejecuta en el navegador, no necesita de un compilador. El navegador lee directamente el código, sin necesidad de terceros. Por tanto, se le reconoce como uno de los tres lenguajes nativos

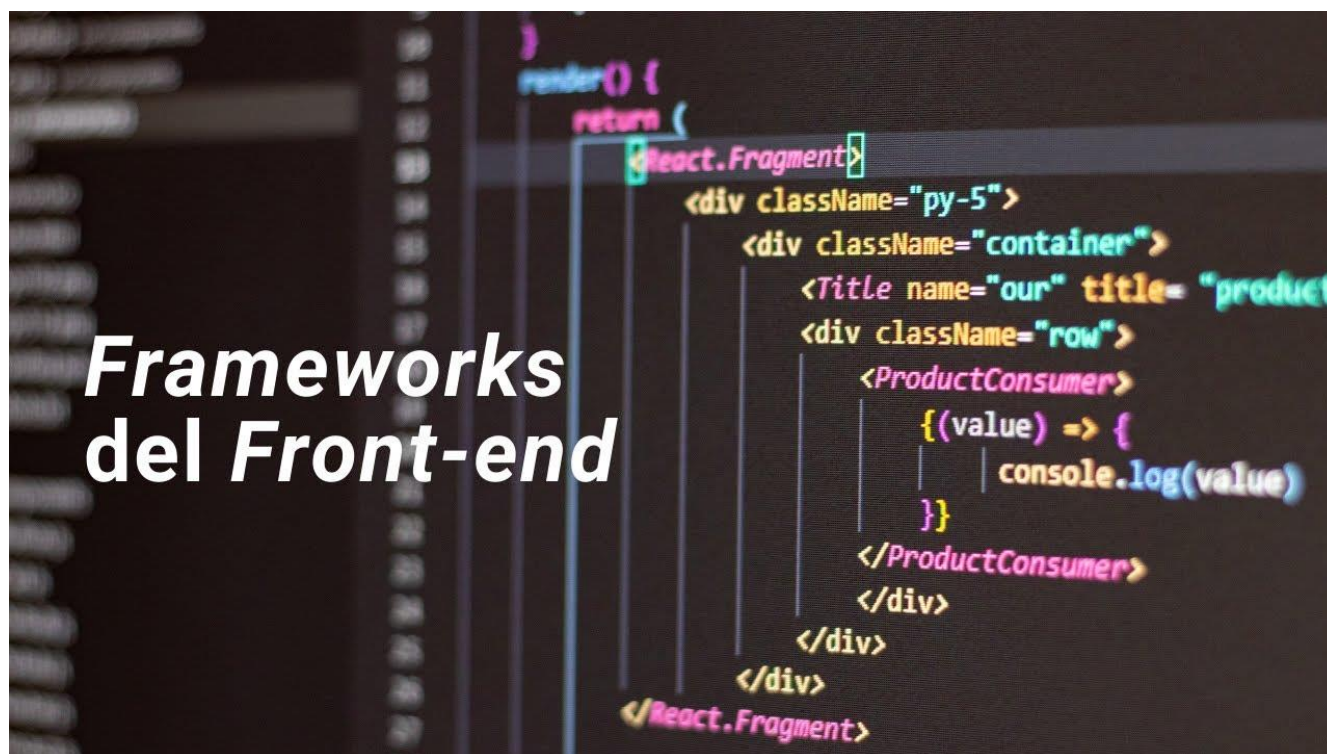
de la web junto a HTML (contenido y su estructura) y a CSS (diseño del contenido y su estructura). (Ramos, 2020).

Además, aunque JavaScript es más conocido como un lenguaje de scripting para páginas web, también se utiliza en muchos entornos fuera del navegador, como en Node.js, Apache CouchDB y Adobe Acrobat.

Para concluir sobre estas tres tecnologías principales, se realiza una analogía con el cuerpo humano: los huesos, que constituyen la estructura ósea, se comparan con HTML, que estructura el contenido de la web; los músculos, que permiten el movimiento y la dinámica del cuerpo, se comparan con JavaScript, que añade interactividad a las páginas web; y la piel, que cubre y da apariencia al cuerpo, se compara con CSS, que estiliza y diseña el contenido web. Esta analogía tiene como objetivo facilitar la comprensión de estas tecnologías del lado del cliente.

Para continuar, lo invitamos a explorar el siguiente video:

Video 1. Frameworks del Front-end



[Enlace de reproducción del video](#)

Síntesis del video: Frameworks del Front-end

Un framework es un tipo de herramienta o recurso que hace que el desarrollo de cualquier aplicativo sea del lado del cliente o servidor sea más rápido fácil y robusto. Este tipo de herramientas incorporan un conjunto de librerías, materiales y adoptan una estructura que van a permitir que todas las aplicaciones tengan un mejor rendimiento; en ese sentido se lista los frameworks más importantes a la fecha con el fin de que se tenga un reconocimiento para usos futuros; este listado es apoyado según la comunidad más grande de gestión de productos llamada Product Coalition:

React, está hecho por un desarrollador de Facebook Jordan Walke, la biblioteca se utilizó por primera vez en el servicio de noticias de Facebook en 2011, luego en instagram en 2012 y en 2013 el código se abrió para todos.

Vue Js, más de 36,000 aplicaciones están escritas en View, entre las empresas que lo eligieron como herramienta de desarrollo se encuentran: Stackoverflow y PlayStation; según lo define la comunidad de Product Coalition.

Angular, es en realidad un proyecto de Google que se utiliza principalmente para el desarrollo de aplicaciones móviles y de escritorio, hay muchas funciones útiles, el sitio oficial de frameworks dice: concéntrese en crear sitios impresionantes en lugar de hacer que su código funcione.

2.2. Tecnologías del back-end

Las tecnologías del back-end se refieren a aquellas utilizadas en el lado del servidor. El servidor emplea estas tecnologías para gestionar y procesar las diferentes peticiones de información que recibe. Además, realiza la gestión de las bases de datos alojadas en el mismo. Una vez que la información es procesada, se envía de vuelta al dispositivo del usuario para que sea visualizada a través de las tecnologías front-end. Entre las tecnologías de back-end más conocidas y utilizadas se encuentran PHP, Java, .NET, Python y MySQL. Estas herramientas son esenciales para el desarrollo y la operación efectiva de aplicaciones web, ya que permiten la creación de funcionalidades complejas y manejo de grandes volúmenes de datos.

Lenguajes del servidor

En el lado del servidor, el back-end es fundamental para el funcionamiento de un sitio web, ya que es en esta instancia donde se realizan los procesos más robustos de manejo de información. Por ello, se requieren lenguajes de programación que ofrezcan una diversidad de funciones para el tratamiento adecuado de esos datos. Además, los programas escritos por desarrolladores de back-end se utilizan para comunicar la información desde la base de datos hasta el navegador. La empresa de base tecnológica Third Rock Techkno señala que:

“En 2021, el desarrollo de Back-End será aún más vital a medida que las empresas busquen expandirse a un ritmo rápido”. Esta empresa también destaca los lenguajes de programación del servidor que están en tendencia, subrayando la importancia de mantenerse actualizado con las tecnologías que lideran el mercado. (Third Rock Techkno, 2021)

Los lenguajes de programación son:

- **Node.JS**

Este lenguaje tiene su propio entorno de ejecución. Es popular entre la comunidad de desarrolladores y se considera uno de los lenguajes más eficientes en la literatura de desarrollo de software. Se prefiere para desarrollar aplicaciones de alta gama para el desarrollo de sitios web, así como para la transmisión de video.

- **Kotlin**

Se presenta como un lenguaje de programación back-end para Java Virtual Machine. Es muy preferido para el desarrollo de Android, contando con el soporte oficial de Google. Según Third Rock Techkno, tiene una

curva de aprendizaje muy fácil, es menos propenso a fallos y facilita la detección de errores.

- **Ruby**

Se estima que este lenguaje jugará un papel fundamental en el futuro, ya que ayuda a reducir el tiempo dedicado a tareas estandarizadas.

- **PHP**

Es un lenguaje de programación de back-end muy flexible con características impecables. Tiene una comunidad en constante actualización y numerosos frameworks basados en este lenguaje. Se considera el futuro del desarrollo back-end en 2021, destacando por una sintaxis muy expresiva que brinda libertad creativa a los desarrolladores.

Frameworks del back-end

Al igual que los frameworks del front-end, los frameworks del back-end proporcionan a los desarrolladores herramientas para realizar su trabajo de manera más eficiente, segura y robusta. Estos frameworks ofrecen estructuras predefinidas y componentes reutilizables que aceleran el proceso de desarrollo, ayudando a manejar aspectos complejos como la conexión con bases de datos, la gestión de sesiones, el procesamiento de transacciones y la seguridad.

Una de las empresas de desarrollo web más destacadas de Canadá, Monocubed, señala la importancia de estos frameworks en el desarrollo actual. Según Mistry (2021), los frameworks de back-end que están marcando tendencia facilitan significativamente la creación de aplicaciones complejas y escalables. Estos frameworks no solo mejoran la eficiencia del desarrollo sino que también aseguran

que las aplicaciones puedan adaptarse y responder a las exigencias del mercado moderno.

- **Laravel**

Este framework tiene una excelente estructura y es capaz de adaptarse a grupos grandes debido a la efectividad de sus recursos, como la llamada toolbox. Basado en PHP, sigue una sintaxis elegante y expresiva. Permite a los desarrolladores web ser flexibles y creativos, manejando los detalles de fondo a diferencia de otros frameworks.

- **Django**

Framework de alto nivel conocido por su enfoque de ‘baterías incluidas’, lo que significa que incluye casi todo lo que necesitaría cualquier desarrollador. Se utiliza principalmente para el desarrollo de aplicaciones web interactivas a gran escala impulsadas por bases de datos. Es altamente personalizable y escalable, con una amplia comunidad y una importante documentación.

- **Ruby on Rails**

Escrito en Ruby, este framework es fácil de probar y compilar. Cuenta con el amplio apoyo de una comunidad activa y una gran cantidad de recursos disponibles, lo que lo convierte en una alternativa viable para los principiantes en desarrollo.

3. Introducción a la arquitectura orientada a servicios

Hasta ahora, se han explorado las tecnologías para el desarrollo web, pero es crucial entender que, además de crear aplicaciones, también es posible desarrollar servicios. Para ello, es esencial abordar el concepto de la Arquitectura Orientada a Servicios, conocida como SOA. SOA es un tipo de arquitectura de software que se fundamenta en la integración de aplicaciones a través de servicios. Según IBM, SOA se define como:

... una manera de hacer que los componentes de software sean reutilizables mediante interfaces de servicio. Estas interfaces utilizan estándares de comunicación que les permiten incorporarse rápidamente a nuevas aplicaciones sin necesidad de una integración profunda cada vez. (IBM, 2021).

En este contexto, es destacable el desarrollo de software en sí, la reutilización de componentes y la comunicación con distintas aplicaciones sin la necesidad de crear desarrollos a medida para cada aplicación con la que se desee conectar. De aquí surge el concepto de servicio y de interoperabilidad.

Dentro de las implementaciones de SOA, se encuentran las arquitecturas REST y SOAP, las cuales siguen los mismos principios de SOA. REST y SOAP son dos enfoques diferentes para la transmisión de datos en línea. Ambos definen cómo diseñar interfaces de programación de aplicaciones (APIS), que actúan como medios para que dos aplicaciones o plataformas se comuniquen entre sí, exponiendo o publicando un conjunto de servicios y reglas sobre cómo acceder a ellos. La principal diferencia entre ellos es que SOAP es un protocolo estructurado, mientras que REST no lo es.

Generalmente, uno de estos dos enfoques registrará las APIS, definiendo así una serie de reglas tanto para la publicación de servicios como para el acceso a los mismos, según el caso práctico y las preferencias del desarrollador.

SOAP

SOAP, acrónimo de Simple Object Access Protocol (Protocolo Simple de Acceso a Objetos), se define como un protocolo o conjunto de reglas estándar diseñado principalmente para facilitar la comunicación entre aplicaciones desarrolladas en diferentes lenguajes de programación y que operan en distintas plataformas tecnológicas.

Debido a su naturaleza de protocolo, SOAP incorpora reglas integradas que, aunque aumentan la seguridad y la estructuración de los mensajes, también pueden incrementar la complejidad y la carga en los servidores, lo que a veces resulta en retrasos en los tiempos de carga de las páginas o en las respuestas. Sin embargo, debido a su robustez en estructura y seguridad, SOAP es ampliamente utilizado en el sector empresarial, donde estos factores son críticos. (RedHat, 2021).

Adicionalmente, el envío de una solicitud de datos a una API de SOAP puede gestionarse a través de diversos protocolos de la capa de aplicación, tales como HTTP, utilizado comúnmente por los navegadores web; SMTP, el protocolo para el envío de correos electrónicos; y TCP, entre otros. Una vez recibida una solicitud, los mensajes SOAP retornan por defecto documentos XML, un lenguaje de etiquetas que es comprensible tanto para máquinas como para personas, facilitando así la interoperabilidad entre sistemas dispares.

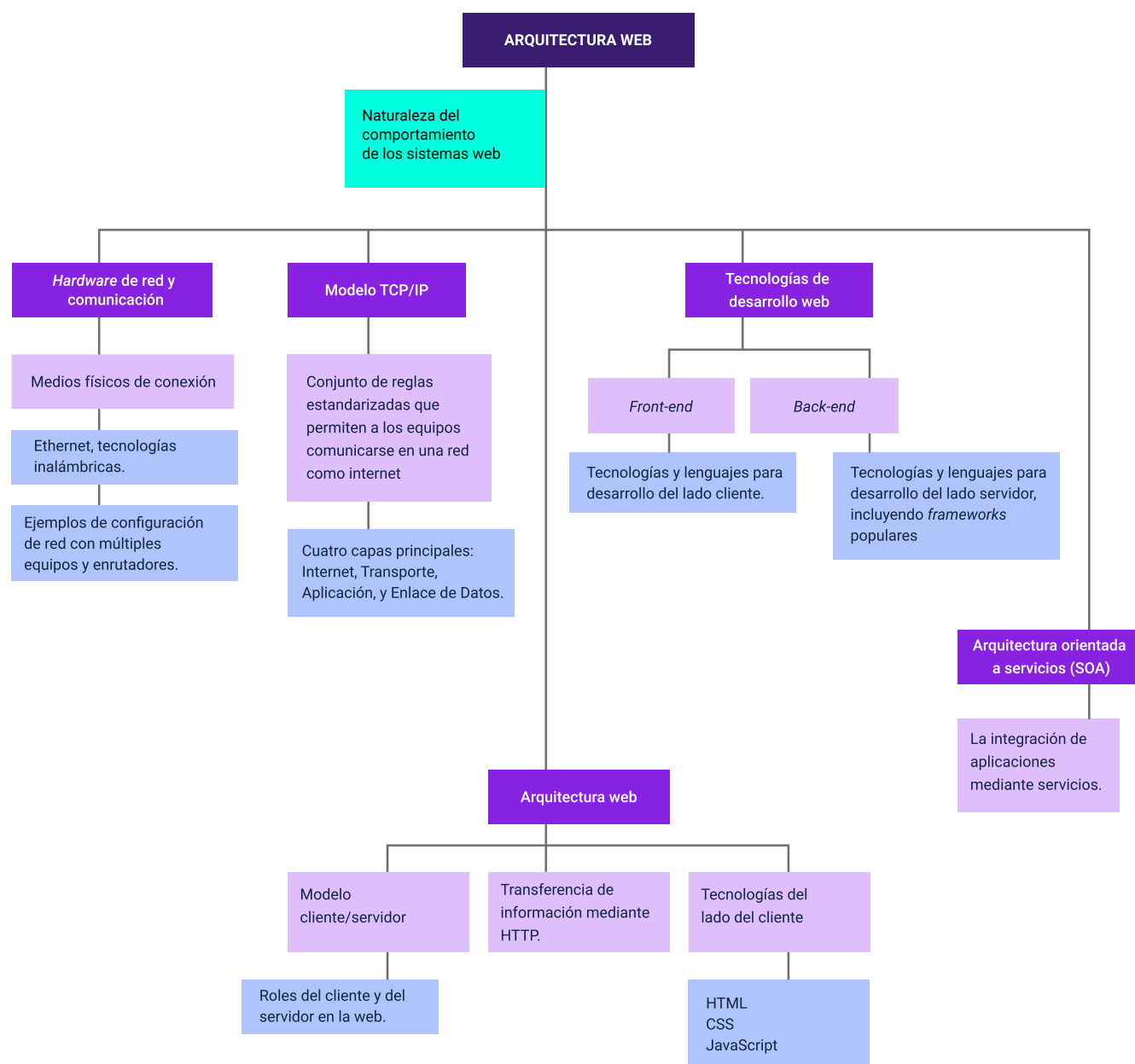
REST

REST, acrónimo de Representational State Transfer (Transferencia de Estado Representacional), se define como un conjunto de principios arquitectónicos que se adaptan a las necesidades de los servicios web, y es especialmente valorado en aplicaciones móviles por la velocidad de transmisión y la ligereza de la información. Al enviar una solicitud de datos a una API REST, se utiliza el protocolo de transferencia de hipertexto, comúnmente conocido como HTTP. Las API diseñadas según los principios de REST (conocidas como API o servicios web RESTFUL) responden a estas solicitudes con mensajes en formatos como HTML, XML, JSON, entre otros. El formato más utilizado para los mensajes es la notación de objetos de JavaScript (JSON), que puede ser interpretado por cualquier lenguaje de programación, es liviano y comprensible tanto para máquinas como para personas. Así, las API RESTFUL son más flexibles y fáciles de configurar. (RedHat, 2021).

En conclusión, al comparar estas dos arquitecturas, las API REST son ideales para los escenarios tecnológicos más modernos, como el internet de las cosas (IoT), el desarrollo de aplicaciones móviles y la computación sin servidor, debido a su ligereza. Por otro lado, los servicios web SOAP ofrecen niveles superiores de seguridad y cumplimiento para operaciones asociadas con la dinámica operativa de las empresas; sin embargo, la rigurosidad de sus reglas puede resultar en una gestión más pesada de la información.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
La internet y la web	ThePRBridge. (2011, 13 octubre). Cómo funciona el internet [Video]. YouTube.	Video	https://www.youtube.com/watch?v=31LE0bPLrhM
La internet y la web	Aaron. (2009, 19 febrero). How the internet Works in 5 Minutes [Video]. YouTube.	Video	https://www.youtube.com/watch?v=7LPdttKXPc
Fundamento de la internet	Saklar. (2017, 25 junio). Topologías de Red [Video]. YouTube.	Video	https://www.youtube.com/watch?v=awLJkNHBoms
Introducción a las tecnologías para el desarrollo web	Code War. (2020, 17 mayo). 2. Frontend y backend, en qué consisten, sus tecnologías, APIs - Aprende a programar sistemas web [Video]. YouTube.	Video	https://www.youtube.com/watch?v=rQX8D4dy1PM

Glosario

Arquitectura software: es una metodología que se refiere a la estructuración de un sistema software que, idealmente, se crea en etapas tempranas de la construcción del sistema.

Página web: es un documento que se puede mostrar en un navegador web como Firefox, Google Chrome, Microsoft Internet Explorer o Edge, o Safari de Apple.

Servidor web: es una colección o conjunto de páginas web que se agrupan y normalmente se conectan de varias maneras. Por lo regular se le conoce como "sitio web" o simplemente "sitio".

Sitio web: conjunto de programas y rutinas que permiten a la computadora realizar determinadas tareas.

W3C: por sus siglas World Wide Web Consortium. Es una comunidad internacional que genera recomendaciones y estándares que aseguran el crecimiento de la WWW a largo plazo.

Web push: son notificaciones que el usuario recibe en su navegador (móvil o de escritorio) sin necesidad de tener una aplicación descargada en su equipo.

Referencias bibliográficas

Facebook. (2021). Facebook - Meld je an of registreer je.

Fisher, S. (2021, 19 de mayo). ¿Qué es TCP/IP y cómo funciona? Avast Academy.

<https://www.avast.com/es-es/c-what-is-tcp-ip>

IBM. (2021, 7 de mayo). SOA (arquitectura orientada a servicios).

<https://www.ibm.com/co-es/cloud/learn/soa>

Ingenio Virtual. (2018a, 11 de febrero). Conceptos básicos sobre tecnologías de desarrollo web. <https://www.ingeniovirtual.com/conceptos-basicos-sobre-tecnologias-de-desarrollo-web/>

Juncosa, M. (2020, 6 de agosto). El modelo TCP/IP capa a capa. Aprende de Redes.

Medium. (2021, 15 de marzo). Front end development trends that will be with us in 2021. <https://productcoalition.com/front-end-development-trends-that-will-be-with-us-in-2021-a0ad023922ea>

Mistry, J. (2021, 29 de mayo). 8 best backend frameworks to use for web development 2021. Monocubed. <https://www.monocubed.com/best-backend-frameworks/>

Mozilla. (2021, 13 de mayo). How does the internet work? - Learn web development. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn/Common_questions/How_does_the_internet_work

Ramos, R. (2020, 18 de noviembre). ¿Qué es JavaScript y para qué sirve? Agencia de Marketing Digital. <https://soyrafaramos.com/que-es-javascript-para-que-sirve/>

RedHat. (2021). REST vs. SOAP.

<https://www.redhat.com/en/topics/integration/whats-the-difference-between-soap-rest>

Tanenbaum, A. S., Wetherall, D. J., & Elizondo, A. V. R. (2012). Redes de computadoras. Pearson Educación.

Third Rock Techkno. (2021, 24 de marzo). Top 5 picks for backend development in 2021. <https://www.thirdrocktechkno.com/blog/top-5-picks-for-backend-development-in-2021/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Santiago Muñoz de la Rosa	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Carlos Julián Ramírez Benítez	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Luis Jesús Pérez Madariaga	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Fundamentos de la calidad del “software”

Breve descripción:

En este componente formativo, el aprendiz identificará que un “software” con calidad implica la utilización de metodologías o procedimientos estándares para el análisis, diseño, programación y prueba, que permitan uniformar la filosofía de trabajo, en aras de lograr una mayor confiabilidad, mantenibilidad y facilidad de prueba, a la vez que eleven la productividad, tanto para la labor de desarrollo como para el control de la calidad del “software”.

Abril 2024

Tabla de contenido

Introducción	5
1. Descripción de la idea de negocio	6
2. Marco de referencia	8
2.1. Proceso de desarrollo de “software”	8
Metodología cascada.....	9
2.2. Estándares ISO/IEC 25000 SQuaRE, ISO/IEC 15504, IEEE	11
Ventajas y desventajas de la ISO 25000	15
2.3. Modelo de desarrollo CMMI (“Capability Maturity Model Integration”)	15
Estructurada por nivel de madurez	16
Estructurada por nivel de capacidad	19
2.4. Calidad en el proceso de desarrollo de “software”	20
Funciones del aseguramiento de la calidad del “software” (SQA)	22
Principios de aseguramiento de la calidad de “software” (SQA)	23
Componentes del aseguramiento de la calidad de “software” (SQA)	25
Proceso del aseguramiento de la calidad de “software” (SQA).....	26
2.5. Modelos de referencia para la calidad en el proceso	27
Modelo Mc Call	27

Modelo FURPS.....	29
Modelo BOHEM.....	31
ISO/IEC 9126.....	32
2.6. Calidad del “software” relacionada con el producto	33
Factores que afectan o determinan la calidad del “software”	35
Costo de los defectos.....	37
2.7. Calidad relacionada con las personas.....	38
PSP/TSP	38
TSP	43
SCRUM	46
Calidad de “software” en metodologías ágiles	51
Valores de la metodología ágil.....	52
Principios de las pruebas ágiles (“Agile Testing”)	53
Cuadrantes de las pruebas en metodologías ágiles.....	54
Metodología XP - Programación Extrema	57
3. Documentar el proceso de calidad de “software”	61
3.1. Diseñar los instrumentos de calidad de “software”	61
Plan de pruebas.....	61
Formatos	63

Artefactos de prueba	63
3.2. Aplicar los instrumentos de calidad de “software”	64
Implementación del plan de pruebas	64
Principales elementos para redactar un caso de prueba.....	66
Principales elementos para redactar un script de prueba	66
Creación del plan de pruebas para proyectos ágiles (“Agile Testing”)	67
3.3. Análisis y entrega de resultados	68
Incidencias detectadas	69
Ciclo de vida de una incidencia	69
Síntesis	71
Material complementario.....	72
Glosario.....	73
Referencias bibliográficas	74
Créditos.....	76

Introducción

Bienvenidos a este componente, en donde trataremos el tema relacionado con los Fundamentos de la calidad del “software”.

Recordemos que implementar buenas prácticas, definir adecuadamente los procesos, ser flexibles ante las diferentes metodologías y herramientas, y tener un personal capacitado en las fases de pruebas y calidad, se convierte en un desafío cada vez más grande, al cual se ven enfrentadas las empresas que construyen “software”.

Dadas las circunstancias actuales, la demanda de construcción de productos de “software” va en auge, teniendo un crecimiento exponencial año por año. Si agregamos las iniciativas de modernización de las tecnologías y la tendencia de sistematizar y facilitar los procesos, implica tener más células de desarrollo operando, y mejoras en las fases de calidad del “software”.

En este componente formativo, usted podrá abordar los fundamentos de la calidad del “software”, entendiendo los valores misionales de las empresas, y cómo estos repercuten en los diferentes procesos de control de la misma. La calidad es una característica que debe ser transversal en todos y cada uno de estos procesos, porque de esta forma podremos asegurar éxito y productividad. Un quipo capacitado, comprometido y creativo es un valor agregado en la industria del “software”, que cada vez va en aumento, pero que también exige compromiso de todos.

1. Descripción de la idea de negocio

La idea de negocio es el servicio o producto que se desea ofrecer al mercado, lo cual se requiere de un plan de negocio que debe incluir misión, visión, objetivos de la empresa y la estrategia para alcanzarlos, así como la estructura organizacional y la inversión para financiar el proyecto. No obstante, todo empieza cuando se tiene una idea de negocio para posteriormente materializarla por escrito en un documento teniendo en cuenta cinco elementos:

- **Estructura ideológica.** Contiene el nombre de la empresa, visión, misión, valores, objetivos y un detalle de las ventajas competitivas del negocio.
- **Estructura del entorno.** Es el arte y la ciencia, contiene un estudio de las fortalezas y las debilidades de la empresa, las tendencias del mercado, clientes potenciales y el comportamiento del sector donde se va a desarrollar.
- **Estructura mecánica.** Se describen las actividades que se deben ejecutar para alcanzar el éxito de la idea de negocio, tales como las ventas, las estrategias de distribución y la publicidad.
- **Estructura financiera.** Se verifica en términos económicos y proyecciones de escenarios la viabilidad de la idea de negocio, con el fin de observar si la utilidad es factible.
- **Recursos humanos.** Determina los puestos de trabajo requeridos, así como las obligaciones y derechos de cada miembro que integra la organización, independientemente que el emprendedor sea una sola persona.

La estructura ideológica corresponde con los valores misionales de cualquier empresa, por lo tanto, es necesario describir en la idea de negocio, la misión, la visión y

los objetivos que se intentan lograr. La presentación ante sus potenciales inversionistas y demás colaboradores, recae de alguna forma en esta claridad, desde lo que es actualmente la empresa y cómo se proyecta a un futuro.

2. Marco de referencia

Un marco de referencia aporta las directrices y saberes teóricos obtenidos con base en un estudio, una investigación y un análisis previo, que permite estimar en este caso, que un proceso de calidad fundamentado a través del estudio e investigación, estaría en la capacidad de plantear mejores prácticas y definir procedimientos que optimicen dicho proceso, para garantizar que este represente una mejora en el proceso de calidad y en la calidad del producto. Por lo anterior, este marco de referencia contiene los fundamentos de la calidad del “software” que permitirá construir un plan de pruebas, teniendo en cuenta el proceso de desarrollo de “software”, los modelos de desarrollo, metodologías de desarrollo tradicionales y ágiles, estándares de calidad, planificación de pruebas de “software” y la documentación que se debe realizar en el proceso de calidad de “software”, tal como se describen a continuación.

2.1. Proceso de desarrollo de “software”

El proceso de desarrollo o ciclo de vida del “software”, requiere de una serie de etapas, con el objetivo de garantizar que el programa desarrollado es fiable, eficiente, seguro y cumple con los requerimientos de los usuarios finales.

Este proceso de desarrollo contiene una serie de fases que permiten validar el proceso de desarrollo del “software”, garantizando que se cumplan con los requerimientos para la verificación y aplicación de los procedimientos de desarrollo, comprobando las metodologías y técnicas usadas. Este proceso es muy aplicable en los casos en donde se utilicen metodologías tradicionales como la metodología en cascada.

Metodología cascada

El desarrollo en cascada es un proceso lineal, por lo que se identifica en separar los procesos del desarrollo de “software” en diferentes fases que son sucesivas en el contexto de la ejecución del proyecto. A diferencia de los modelos iterativos, cada una de las fases se ejecuta una sola vez, y el resultado de cada fase es un insumo para la siguiente. Esta metodología es usada especialmente en el proceso de desarrollo de “software”.

Análisis

Todo proyecto de “software” inicia con una fase de análisis. En el documento de especificación del “software” se determina la viabilidad y la definición de requisitos funcionales y no funcionales.

En la viabilidad se evalúa la rentabilidad, factibilidad y los costos de la construcción, además se define un pliego de condiciones, un plan y una estimación financiera y también la propuesta para el cliente, de ser necesario.

Posteriormente, se realiza la definición detallada de los requisitos, que debe incluir, el análisis de la situación de salida y un concepto. Los análisis de salida definen la problemática, y el concepto define qué funciones y características debe ofrecer el producto “software” para satisfacer las necesidades del cliente.

Diseño

En esta fase se formula una solución específica para la problemática definida o establecida en los requisitos, aquí los desarrolladores de “software” se encargarán de construir un diseño de la arquitectura del sistema, así como un plan de diseño, enfocados en los componentes, interfaces, entornos de trabajos y bibliotecas. Esta fase

da como resultado un estimado del plan de diseño y los planes de prueba para los diferentes componentes.

Implementación

La arquitectura otorgada en la fase de diseño se desarrolla en la fase de implementación, en la que se construye el código fuente que representa dicha arquitectura en un lenguaje de programación, también incluye la búsqueda de errores y pruebas unitarias.

Esta fase genera como resultado un producto “software” que se despliega por primera vez como insumo de la siguiente fase.

Verificación

Esta fase incluye el proceso de integración y despliegue del producto “software” en el entorno seleccionado, por lo general se prueba y envía con una serie de usuarios finales seleccionados. Se aplican las pruebas de aceptación incluidas en la fase de análisis y con ello se puede determinar si el “software” cumple o no con las necesidades definidas.

Los productos que superan estas pruebas, son aquellos que pueden continuar hacia el proceso de liberación hacia el público en general o su “target” específico.

Mantenimiento

Después que la fase de prueba haya finalizado con éxito, se autoriza para que la aplicación pase a un ambiente productivo, en esta última etapa se incluye la entrega, mantenimiento, garantía y mejoras del producto “software”. Cabe destacar que la fase de mantenimiento incluye tres elementos:

- **Mantenimiento correctivo:** eliminar los defectos detectados durante su vida útil.
- **Mantenimiento adaptativo:** adaptar el “software” a nuevas necesidades.
- **Mantenimiento perfectivo:** añadir al “software” nuevas funcionalidades.

2.2. Estándares ISO/IEC 25000 SQaRE, ISO/IEC 15504, IEEE

En un proceso de calidad de “software”, se debe identificar las normas ISO (Organización Mundial para la Estandarización), que proporcionan guías que a su vez aportarán especificaciones, de aspecto internacional, a los servicios, productos y sistemas de una empresa, para garantizar una óptima eficiencia y calidad en cuanto a un producto o servicio y a su funcionamiento y resultados. Por lo tanto, se deben tener en cuenta los estándares descritos en este apartado con el objetivo de seguir patrones que permitan asegurar la calidad a través del análisis de los estándares que se utilizan en la actualidad.

La norma ISO/IEC 25000, también reconocida como “System and Software Quality Requirements and Evaluation” (SQaRE), tiene el objetivo de especificar en un documento el conjunto de guías y patrones que aseguran la óptima utilización del “software” de un negocio. No obstante, este procedimiento se realiza en compañías que realizan sus propias aplicaciones para la organización y gestión interna, en consecuencia, se refuerza la creación de forma correcta y usabilidad en los procesos internos.

La ISO/IEC 25000, evoluciona de anteriores normas tales como las normas ISO/IEC 9126, que describe las características de un modelo de calidad del producto

“software”, e ISO/IEC 14598, que involucraba el proceso de evaluación de productos “software”.

La familia de normas ISO/IEC 25000 está constituida como se aprecia a continuación.

División de Gestión de Calidad - ISO/IEC 2500n

Las normas en este punto, definen todos los modelos, términos y definiciones comunes referenciados por todas las otras normas de la familia 25000. Actualmente esta división se encuentra formada por:

- **ISO/IEC 2500 – “Guide to SQuaRE”**: está compuesta por un modelo de la arquitectura de SQuaRE, la terminología de la familia, un resumen de las partes, los usuarios previstos, las partes asociadas y los modelos de referencia.
- **ISO/IEC 2501 – “Planning and Management”**: determina los requerimientos y disposiciones para gestionar la especificación y evaluación de los requerimientos del producto “software”.

División de Modelo de Calidad - ISO/IEC 2501n

Las normas de esta división presentan modelos detallados de calidad, así como las características para calidad externa, interna y en uso del producto “software”.

Actualmente esta división está conformada por:

- **ISO/IEC 25010 – “System and software quality models”**: define el modelo de calidad para el producto “software”.
- **ISO/IEC 25012 – “Data Quality model”**: describe un modelo general para la calidad de los datos, específicamente a los datos que se encuentran

almacenados de forma estructurada y hacen parte de un sistema de información.

División de Medición de Calidad - ISO/IEC 2502n

Involucran un modelo de referencia de la medición de la calidad del producto, descripciones de medidas de calidad (externa, interna y en uso) así como guías con prácticas para su respectiva aplicación. Actualmente esta división está conformada por:

- **ISO/IEC 25020 – “Measurement reference model and guide”:** representa una explicación introductoria y un modelo de referencia común a los elementos de medición de la calidad.
- **ISO/IEC 25021 – “Quality measure elements”:** especifica un conjunto de métricas base que podrían ser utilizadas en el transcurso de todo el ciclo de vida del desarrollo “software”.
- **ISO/IEC 25022 – “Measurement of quality in use”:** especifica métricas para efectuar la medición de la calidad en uso del producto.
- **ISO/IEC 25023 – “Measurement of system and software product quality”:** especifica métricas para efectuar la medición de la calidad de productos y sistemas “software”.
- **ISO/IEC 25024 – “Measurement of data quality”:** especifica métricas para efectuar la medición de la calidad de datos.

División de Requisitos de Calidad - ISO/IEC 2503n

Las normas de esta división especifican requerimientos de calidad que podrían ser usados en el proceso de elicitación de requisitos de calidad del producto “software” a desarrollar. Actualmente esta división está conformada por:

- ISO/IEC 25030 – “Quality requirements”: contiene una serie de recomendaciones para efectuar la especificación de los requisitos de calidad del producto “software”.

División de Evaluación de Calidad - ISO/IEC 2504n

Esta división involucra normas que aportan guías, recomendaciones y requerimientos para proceder a realizar el proceso de evaluación del producto “software”. Actualmente esta división está conformada por:

- **ISO/IEC 25040 – “Evaluation reference model and guide”**: plantea un modelo de referencia general para la evaluación, por lo tanto, tiene en cuenta las entradas en el proceso de evaluación, las restricciones y los recursos requeridos para alcanzar las adecuadas salidas.
- **ISO/IEC 25041 – “Evaluation guide for developers, acquirers and independent evaluators”**: detalla las recomendaciones y requerimientos para la implementación práctica de la evaluación del producto “software” según las percepciones de los desarrolladores, de los que adquieren el “software” y los respectivos evaluadores independientes.
- **ISO/IEC 25042 – “Evaluation modules”**: define lo que la Norma considera un módulo de evaluación y la documentación, estructura y contenido que se debe utilizar a la hora de definir uno de estos módulos.
- **ISO/IEC 25045 – “Evaluation module for recoverability”**: describe un módulo para la evaluación de la recuperabilidad.

Ventajas y desventajas de la ISO 25000

Cuando se implementa esta norma, es importante reconocer tanto las ventajas como las desventajas de la ISO 25000, que mostrará un panorama para entender su reconocimiento en el sector de la calidad.

Ventajas

- Asegurar que la aplicación, servicios y productos que se vendan o utilicen tienen la confianza y calidad requerida.
- Minimiza la cantidad de errores (Bugs).
- Permite ahorrar tiempo en los procedimientos de entrega.
- Determina la constante calidad a partir de evaluaciones periódicas.
- Lo anterior genera en el cliente un aumento de la satisfacción.

Desventajas

- No se definen desventajas específicamente en cuanto a estas normas, puesto que se crea un marco de calidad a nivel internacional.
- Tener en cuenta estas normas es un proceso tedioso, no obstante, el resultado es óptimo y positivo.

2.3. Modelo de desarrollo CMMI (“Capability Maturity Model Integration”)

Este modelo es uno de los que más utilizan las empresas de desarrollo de “software”, por lo tanto, se debe comprender que si se utiliza este modelo, su objetivo es auditar el cumplimiento de normas de calidad partiendo de la medición con niveles de madurez, además se enfoca en la mejora continua del producto en todo su ciclo de

vida, impulsar la eficiencia y asegurar la calidad, de esta forma se reducen los riesgos en el proceso de desarrollo.

Cuando se desea mejorar algunos procesos de negocio, construir productos de calidad para satisfacer las necesidades del cliente y colaborar en el cumplimiento de los requerimientos de la norma ISO e instaurar prácticas de mejora continua, se debe utilizar este modelo puesto que es un modelo de capacidad y madurez, que además permite efectuar una apreciación de la madurez del proceso de desarrollo de “software” en las empresas.

Los objetivos de CMMI, son:

- Producción de calidad de productos o servicios.
- Creación de valor para los accionistas.
- Mejora continua en la satisfacción de los clientes.
- Aumento en la cuota de mercado.
- Obtener reconocimiento de excelencia en la industria.

CMMI presenta dos modelos, una está estructurada en cinco niveles de madurez y la otra está estructurada en seis niveles de capacidad. Las áreas de proceso están categorizadas en administración de procesos, administración de proyectos, ingeniería y soporte.

Estructurada por nivel de madurez

Un nivel de madurez bien definido con las bases necesarias para evolucionar hacia un proceso de “software” maduro.

Los niveles representados por etapas consideran cinco niveles de madurez, los cuales contienen un conjunto de áreas de proceso predefinidas, las cuales se miden por objetivos generales y específicos, estos se aplican a los conjuntos predefinidos en cada área de proceso. Veamos su distribución:

Nivel de madurez inicial

Los procesos suelen ser ad hoc y caóticos, en el cual se gestionan y crean productos o servicios que no funcionan, excediendo el presupuesto y el calendario de entregas. Las organizaciones en este nivel abandonan los proyectos ante una crisis y no tienen la capacidad de replicar los éxitos y buenas prácticas de procesos previos.

Nivel de madurez administrativo

Los objetivos generales y específicos han sido alcanzados, por decirlo de alguna manera, los proyectos han asegurado que los requisitos son gestionados, los procesos son planificados, y ejecutados de manera controlada. Se garantiza que los procesos se sigan ejecutando bajo presión y alto nivel de estrés, conforme a los planes documentados, y los productos o servicios satisfacen las necesidades del cliente en los requisitos especificados, también los objetivos y normas.

Nivel de madurez definida

La organización ha logrado alcanzar todos los objetivos específicos de las áreas asignadas en los niveles 2 y 3. En este nivel los procesos están bien caracterizados, entendidos y donde se definen normas, procedimientos, herramientas y métodos para su aplicación. Entre el nivel 2 y el 3 hay una diferencia bien marcada desde los estándares, las descripciones de los procesos y los procedimientos dependiendo de las instancias, -por decirlo así- por proyecto en específico. Por decirlo de otra manera en el

nivel 3, los procesos de la organización se adaptan a la ejecución de un proyecto en específico permitiendo de esta manera adaptarse fácilmente a nuevos retos.

Nivel de madurez administrativo cuantitativamente

Se ha logrado con éxito obtener los objetivos específicos de los niveles 2, 3 y 4 y los objetivos genéricos en los niveles 2 y 3. En este nivel se seleccionan los procesos que aportan significativamente al rendimiento del proceso en general, concluyendo subprocesos que se controlan por medio de métricas, reportes y estadísticas que permiten medir la capacidad. Estas métricas posibilitan identificar casos que generen problemas para ser corregidos y no se repitan en el futuro, también para tomar decisiones de negocio que proyectan el crecimiento de la organización.

Una diferencia importante entre el nivel 3 y 4, es que en el nivel 3 los procesos son medidos de forma cualitativamente predecibles, dando paso a la especulación, mientras que en el nivel 4 se evalúa el rendimiento real de los mismos procesos, y esto se determina por medio de técnicas estadísticas y otras técnicas cuantitativas lo previsible dentro de la organización.

Nivel de madurez optimización

En este nivel la organización ha logrado alcanzar todos los “goals of” definidos en los niveles 1, 2, 3, 4 y 5, y los objetivos genéricos en los niveles 2 y 3. Este nivel está centrado en optimizar y en la mejora continua del rendimiento de los procesos de la organización. Una de las diferencias entre el nivel 4 y el 5 es que, en el nivel 4, los procesos están diseñados para causas especiales y proporcionan métricas para ser evaluadas por técnicas estadísticas; mientras que en el nivel 5, los procesos están diseñados para que se encarguen de las causas comunes de la variación y el cambio de

los procesos, conservando las estadísticas para poder prever acciones futuras, este ajuste permite llegar a los objetivos cuantitativos de mejora continua.

Estructurada por nivel de capacidad

Un nivel de capacidad bien definida y establecida sobre una meseta evolutiva, permite describir la capacidad de la organización en relación con un área de proceso determinada, en donde deben tenerse en cuenta las prácticas específicas y genéricas, las mismas que puedan ser extendidas a otras áreas o a otros procesos de la organización relacionados. Cada nivel es una capa en la base que permite una mejora continua con respecto a un proceso particular.

En otras palabras, podemos decir que los niveles de capacidad son acumulativos, los cuales a medida que se está en las capas superiores de los niveles, también se impactan las características de los inferiores.

En CMMI en una representación continua se cuenta con seis niveles de capacidad asignados por los rangos de números del 0 al 5. Esta representación es un conjunto predefinido de áreas de proceso, que se mide por el logro de los objetivos tanto genéricos como específicos, descritos de la siguiente manera:

- **Nivel 0 – Incompleto.** Estar en el nivel de capacidad 0, indica que un proceso se realiza parcialmente o no se realiza, dado que uno o más objetivos no se cumplen y no presenta objetivos genéricos.
- **Nivel 1 – Se ha realizado.**
- **Nivel 2 – Administrativo.** Se denominan procesos administrados, porque se puede contar con una planeación, una trazabilidad y un control en los proyectos individuales o grupales.

- **Nivel 3 – Definido.** Son procesos que están diseñados a partir de un conjunto de procesos estándar en la organización, con productos de trabajo, modelos, medidas y otras características relacionadas a herramientas que faciliten la identificación y ejecución del proceso.
- **Nivel 4 – Cuantitativamente gestionado.** En este nivel se realiza un seguimiento al proceso por medio de métricas estadísticas y otras técnicas cuantitativas, para poder establecer el rendimiento de los procesos, con esto se realiza una gestión en el proceso. En este nivel los atributos de calidad y rendimiento se entienden como términos estadísticos usados para la administración del producto a lo largo de la vida del proceso.
- **Nivel 5 – Optimizado.** Este nivel se centra en la mejora continua del rendimiento de los procesos por medio de mejoras continuas que buscan la innovación. Mientras que en el nivel 4 el enfoque es crear un marco de referencia y las mediciones de rendimiento, el nivel 5 se centra en buscar causas comunes para los problemas y arreglar esos problemas de proceso.

2.4. Calidad en el proceso de desarrollo de “software”

Recordemos que la construcción de “software” está basada en unas fases que permiten incrementar el producto desde sus cimientos hasta obtener un producto base; esto es posible debido a que en el proceso se implementan diferentes estrategias y metodologías, el gran desafío está, en cómo construir un producto que se pueda validar rápidamente y que contenga los niveles adecuados de calidad. La naturaleza de los requisitos de “software” es cambiante, lo cual complica la precisión de los equipos, demorando posiblemente el tiempo, los recursos y el presupuesto asignado.

La calidad de “software” es un área importante en el proceso de desarrollo que busca estar en constante validación de la calidad del producto “software”, por lo general en el diseño de un plan de aseguramiento de calidad. Por ello es de importancia tener en cuenta los siguientes conceptos y su aplicación práctica en el ciclo de desarrollo de “software”:

Calidad

Es un conglomerado de características de una entidad, que son esenciales y que además permiten calificar su valor.

A nivel de empresa, está relacionado con el conjunto de estándares y normas que debe tener un producto o servicio para ser reconocido y percibido en el mercado satisfaciendo las necesidades de los clientes.

Calidad de “software” (QA)

Es el nivel que tienen los clientes para determinar según su percepción que el producto “software” satisface sus necesidades y expectativas.

A nivel empresarial la calidad de “software” hace referencia a las actividades que se establecen de manera común para asegurar que el desarrollo de “software” en todos los proyectos es de calidad.

Aseguramiento de la calidad de “software” (SQA)

Es el conjunto de actividades, métodos y técnicas planificadas y sistemáticas necesarias para asegurar que el producto “software” satisfará los requerimientos dados de calidad.

Se encarga de garantizar que el producto haya sido construido siguiendo los procesos establecidos por la organización.

Propósito del aseguramiento de la calidad de “software” (SQA)

El propósito principal del SQA es detectar diversos problemas en las fases iniciales del desarrollo del “software”, estimando que es menos costosa su corrección.

La calidad del “software” puede medirse hasta la fase de implementación, pero de ser detectados allí los problemas de construcción del producto, implicaría costos extras, por esta razón se aconseja atender y solucionar los posibles errores en las fases iniciales, como en el diseño. Por lo tanto, es necesario tener en cuenta el proceso de SQA durante las etapas iniciales de la construcción del producto, puesto que es donde se definen las características de este y lo que debe garantizar para satisfacer las necesidades implícitas y explícitas. Si el diseño y los requerimientos no están correctamente definidos, es decir que son ambiguos, incompletos o son incoherentes, al momento de implementar el producto va a ser muy costosa su corrección.

Funciones del aseguramiento de la calidad del “software” (SQA)

El rol para SQA es brindar a la metodología de desarrollo de “software”, y a la administración, la seguridad de que procesos oficialmente establecidos están siendo implementados, teniendo en cuenta:

- a) Establecer estándares, procesos y planes, con el objetivo de satisfacer las políticas de la organización y que se ajusten a las necesidades de cada proyecto en particular, para así asegurar la calidad del producto “software”.
- b) Auditar y revisar los productos y actividades desarrolladas para verificar que satisfacen los procesos y estándares definidos.
- c) Proporcionar las herramientas necesarias que den soporte al proceso definido, facilitando su ejecución, visualización, administración y seguimiento.

- d) Proveer al equipo de proyecto y a otros interesados, los resultados sobre las revisiones, auditorías y actividades.
- e) Determinar las funcionalidades esperadas por el cliente.
- f) Escalar problemas presentados dentro del equipo de un proyecto, hacia un nivel apropiado de administración para su resolución.
- g) Garantizar que se cumpla el plan generado para el desarrollo del proyecto y del proceso de desarrollo de “software”.

Principios de aseguramiento de la calidad de “software” (SQA)

Para poder realizar el aseguramiento de la calidad de “software” (SQA), se deben tener en cuenta diversos principios básicos:

- **Proceso.** La calidad debe ser una constante a lo largo de todo el ciclo de vida del “software”. Es una acción que se construye en la medida que el producto se va desarrollando, más no cuando el producto está terminado, es por esta razón que todo el equipo o involucrados en el proceso, deben también enfocarse en la calidad del producto.
- **Equipo.** La calidad del “software” solo se logra con la contribución de todas las personas involucradas, es decir, que la calidad no es un proceso solo del equipo de aseguramiento de la calidad de “software”, sino más bien una actividad sincrónica entre todas las personas que colaboran para el desarrollo del producto.
- **Planificación.** La gestión y la planificación de la calidad debe ser eficaz y previamente definido y ejecutado, por lo tanto, se deben establecer procesos, estándares y una planeación claramente definida para poder asegurar la

calidad del producto, donde contengan puntos de verificación y un proceso claro y definido que incluya qué, quién, y cuándo, va a realizar la verificación o medición de la calidad.

- **Prevención.** Se deben dirigir los esfuerzos a la prevención de defectos, es decir, que una vez que un defecto es detectado además de corregirlo se deben tomar medidas necesarias para que no vuelva a ocurrir. Es en este punto, donde se tiene en cuenta lo que es el análisis de las causas raíces, una vez que se encuentra un defecto se debe analizar por qué se ocasionó y cuál fue la causa que lo originó, por ejemplo, si es un problema en el proceso, o si es un problema con las herramientas que se están utilizando, o si fue una mala definición de los requerimientos, lo que constata que un error no solamente se soluciona o resuelve, sino que hay que indagar más en la causa raíz; porque de no hacerlo en el futuro se va a presentar el mismo problema.
- **Sistemas de detección.** Hay que reforzar los sistemas de detección y eliminación de defectos durante las etapas iniciales, lo que reafirma que eliminando los defectos en las etapas de requerimientos y diseño, estamos asegurando que esos defectos no van a ser replicados en las etapas subsiguientes, por ende es menos costoso detectar y corregir los defectos en las etapas iniciales, por ejemplo si detectamos un defecto en los requerimientos o en el diseño del “software”, lo más posible es que se tenga que corregir documentos o diagramas, pero si se espera a la etapa de desarrollo o cuando el “software” ya está en producción, tenemos que volver hacia atrás en todas las fases del ciclo de vida del desarrollo del “software”.
- **Productividad.** La calidad al igual que la productividad, los costos y los plazos de entrega están al mismo nivel, lo que quiere decir que el cumplimiento de

los plazos de entrega y mantenerse dentro del presupuesto también son variables que se evalúan dentro del proceso de aseguramiento de calidad de “software”, porque de qué sirve un producto sin defectos cuando ha costado mucho más de lo presupuestado y se ha entregado cuando ya no será útil.

- **Participación.** Es esencial la participación de la dirección, que ha de propiciar la calidad, esto quiere decir que la dirección debe ser el primer interesado en asegurar la calidad y de proveer las directrices, herramientas y el apoyo necesario para que se pueda lograr, por lo tanto, si no hay apoyo de la dirección para implementar y ejecutar el proceso de aseguramiento de calidad del “software”, será difícil que se pueda llevar a cabo la calidad y por ende los productos no van a salir con la calidad requerida.

Componentes del aseguramiento de la calidad de “software” (SQA)

El aseguramiento de la calidad de “software” debe incluir los siguientes componentes:

- **Procedimientos e instrucciones de trabajo.** Procesos claramente definidos e instructivos que ayuden al personal a realizar sus actividades con cierto nivel de estandarización.
- **Plantillas y listas de verificación.** Se crean plantillas base para estandarizar la documentación producida y también crear instrumentos que permitan medir la calidad de dichos documentos.
- **Capacitación del personal, la readaptación profesional y la certificación.** El personal debe estar capacitado para ejecutar las actividades que le

corresponden a su rol, por eso la capacitación juega un papel muy importante dentro de SQA.

- **Acciones preventivas y correctivas.** Se deben realizar acciones para prevenir que un problema se presente y acciones para corregir los problemas que ya se presentaron, usualmente estas acciones preventivas y correctivas surgen a raíz de las verificaciones y de las auditorías.
- **Gestión de la configuración.** En la organización se deben de tener procesos que especifiquen cómo serán llevadas a cabo las actividades cotidianas como son la recepción de los requerimientos y movimientos de activos de “software” entre las áreas, tales como el código fuente.
- **Control de la documentación.** Incluye controlar las versiones de los documentos y garantizar la disponibilidad y el acceso a los mismos, por todos los involucrados en el proceso.

Proceso del aseguramiento de la calidad de “software” (SQA)

En el proceso de aseguramiento de la calidad de “software”, se encuentran tres etapas:

- **Planificación.** Se planifica que es lo que se va a probar, es decir, se define el alcance de las pruebas.
- **Construcción.** Se crean los artefactos de prueba y el script de pruebas.
- **Ejecución.** En esta etapa se encuentran los defectos y se ejecutan los scripts de pruebas creados previamente.

2.5. Modelos de referencia para la calidad en el proceso

En la calidad de “software” es importante involucrar el valor de los requerimientos explícitos e implícitos del producto, con el propósito de medir la calidad, los estándares y modelos de calidad existentes.

Estos modelos de calidad tienen un conjunto de factores que pueden ser medidos directamente como unidades de tiempo y errores, pero también indirectamente, como la facilidad de mantenimiento.

Los modelos de calidad son documentos que constituyen una parte importante de las mejores prácticas, plantean aspectos de administración en la cual las organizaciones deben hacer énfasis, además componen diversas prácticas que facilitan la medición de los avances en la calidad.

Los estándares de calidad, además de ser una guía para alcanzar la calidad y la productividad, permiten especificar una serie de criterios de desarrollo para orientar la manera en que se emplea la ingeniería de “software”. A continuación, se nombran los modelos más importantes que evalúan la calidad del producto de “software”.

Modelo Mc Call

Son una serie de preguntas que ponderan un determinado atributo del producto de “software” numéricamente. En la siguiente tabla se establecen los criterios de calidad relacionados con los factores de calidad.

Tabla 1. Modelo Mc Call, factores de calidad del “software”

Perspectivas	Factores	Criterios
Operatividad del producto: factores de calidad que influyen en el grado en que el “software” cumple con su especificación.	Usabilidad: facilidad de uso del “software”.	• Operatividad. • Entrenamiento. • Comunicación
Operatividad del producto: factores de calidad que influyen en el grado en que el “software” cumple con su especificación.	Integridad: protección de programa del acceso no autorizado.	• Control de acceso. • Auditoría de acceso.
Operatividad del producto: factores de calidad que influyen en el grado en que el “software” cumple con su especificación.	Corrección: grado en que una funcionalidad coincide con su especificación.	• Rastreabilidad. • Completitud. • Consistencia.
Operatividad del producto: factores de calidad que influyen en el grado en que el “software” cumple con su especificación.	Fiabilidad – confiabilidad: capacidad de los sistemas de no fallar / la medida en que falla el sistema.	• Simplicidad. • Concreción.
Revisión del producto: factores de calidad que influyen en la capacidad de cambiar el producto “software”.	Mantenibilidad: esfuerzo requerido para localizar y arreglar un fallo en el programa dentro de su entorno operativo.	• Simplicidad. • Concreción.
Revisión del producto: factores de calidad que influyen en la capacidad de cambiar el producto “software”.	Facilidad de prueba: facilidad del programa de realizar pruebas para asegurarse de que está libre de errores y cumple con su especificación.	• Simplicidad. • Instrumentación. • Autodescripción. • Modularidad.
Revisión del producto: factores de calidad que influyen en la	Flexibilidad: facilidad de hacer los cambios necesarios según lo	• Autodescripción.

Perspectivas	Factores	Criterios
capacidad de cambiar el producto “software”.	solicitado en el entorno operativo.	• Capacidad de expansión. • Generalidad. • Modularidad.
Transición del producto: factores de calidad que influyen en la capacidad de adaptar el “software” a los nuevos entornos.	Reusabilidad: facilidad de reutilización de “software” en un contexto diferente.	• Autodescripción. • Generalidad. • Modularidad.
Transición del producto: factores de calidad que influyen en la capacidad de adaptar el “software” a los nuevos entornos.	Interoperabilidad: esfuerzo requerido para acoplar el sistema a otro sistema.	• Modularidad. • Similitud de comunicación. • Similitud de dato. • Independencia del sistema. • Independencia de la máquina.

El modelo de Mc Call es uno de los primeros desarrollados, no obstante, gran parte de los factores descritos son utilizados en la actualidad, además gran variedad de modelos de calidad, incluyendo la Norma ISO 9126 son una estandarización de este modelo.

Modelo FURPS

Este modelo lo desarrolló Hewlett-Packard en el año 1987. Incluye el desarrollo de una serie de factores de calidad de “software”, teniendo en cuenta el acrónimo de FURPS: funcionalidad (“Functionality”), usabilidad (“Usability”), confiabilidad (“Reliability”), desempeño (“Performance”) y capacidad de soporte

(“Supportability”). En la siguiente tabla se establecen los factores y los criterios de calidad.

Tabla 2. Modelo FURPS, factores de calidad del “software”

Factores	Criterios
Funcionalidad	• Características y capacidades del programa. • Generalidad de las funciones. • Seguridad del sistema.
Usabilidad	• Factores humanos. • Factores estéticos. • Consistencia de la interfaz. • Documentación.
Confiabilidad	• Frecuencia y severidad de fallos. • Exactitud de las salidas. • Tiempo medio de fallos. • Capacidad de recuperación ante fallos. • Capacidad de predicción.
Rendimiento	• Velocidad de procesamiento. • Tiempo de respuesta. • Consumo de recursos. • Rendimiento efectivo total. • Eficacia.
Capacidad de soporte	• Extensibilidad. • Adaptabilidad. • Capacidad de prueba.

Factores	Criterios
	• Capacidad de configuración. • Compatibilidad. • Requisitos de instalación.

Modelo BOHEM

Este modelo plantea una jerarquía en forma de árbol con tres ramas y con ciertos niveles, lo cual hace que el “software” sea de gran utilidad, enfocándose en tres aspectos: portabilidad, facilidad de uso y facilidad de mantenimiento. El modelo BOHEM se diseña en los siguientes niveles:

- Aplicaciones primarias.
- Construcciones Intermedias (factores).
- Construcciones primitivas.
- Métricas que establecen valores para los criterios (construcciones primitivas).

En la siguiente tabla se establecen los factores y criterios.

Tabla 3. Modelo BOHEM, factores de calidad del “software”

Factores	Criterios
Portabilidad	• Independencia dispositivos. • Completitud.
Fiabilidad	• Completitud. • Exactitud. • Consistencia.
Eficiencia	• Eficiencia dispositivo.

Factores	Criterios
	• Accesibilidad.
Ingeniería humana	• Accesibilidad. • Comunicatividad. • Estructuración. • Auto-descripción.
Comprensibilidad	• Consistencia. • Estructuración. • Auto-descripción. • Concisión. • Legibilidad. • Expansibilidad.
Modificabilidad	• Estructuración.

ISO/IEC 9126

Es un estándar internacional para la evaluación de la calidad del “software”, que se aplica a diversos tipos de “software”. No obstante, en el 2005 fue reemplazado por el conjunto de normas SQuaRE (“System and Software Quality Requirements and Evaluation”). Sus características se enuncian en la siguiente tabla.

Tabla 4. Modelo ISO/IEC 9126, criterios asociados a factores de calidad.

Factores	Criterios
Funcionalidad	• Adaptabilidad. • Exactitud. • Interoperabilidad.

Factores	Criterios
	• Seguridad.
Usabilidad	• Comprensibilidad. • Aprendizaje. • Operatividad. • Atractivo.
Mantenibilidad	• Análisis. • Cambio. • Estabilidad. • Prueba.
Fiabilidad	• Madurez. • Tolerancia a fallos. • Recuperabilidad.
Eficiencia	• Comportamiento del tiempo. • Uso de los recursos.
Portabilidad	• Adaptabilidad. • Instalación. • Coexistencia. • Reemplazo.

2.6. Calidad del “software” relacionada con el producto

Los clientes determinan según su percepción, que el producto “software” satisface sus necesidades y expectativas a partir de aplicar un proceso de calidad de “software”, puesto que se establecen actividades para asegurar que el desarrollo de “software” en todos los proyectos es de calidad. Por otra parte, el propósito de

asegurar la calidad de un producto es detectar diversos problemas en las fases iniciales del desarrollo del “software”, estimando que es menos costoso su corrección. Cabe aclarar que la calidad de “software” de un producto puede medirse una vez en la fase de implementación o terminado completamente o cuando el producto está en producción, no obstante, este hecho implicaría amplios costos para la compañía de “software” a diferencia de detectar y resolver los problemas en las fases iniciales como el diseño, o en los requerimientos que generaría una reducción de los costos en cuanto a los cambios a realizar.

Una de varias funciones de la calidad de “software” es establecer estándares, procesos y planes, con el objetivo de satisfacer las políticas de la organización y se ajusten a las necesidades de cada proyecto en particular, para asegurar la calidad del producto “software”, por lo tanto, la calidad del “software” está dividida en:

- La calidad del producto obtenido.
- La calidad del proceso de desarrollo.

Las anteriores dependen una de otra, puesto que para alcanzar calidad en un producto es pertinente que haya calidad en el proceso de desarrollo. El objetivo que se determine para la calidad del producto, establecerá los propósitos del proceso de desarrollo. Los requerimientos de calidad más significativos del proceso de “software” son:

- Que genere los resultados esperados.
- Los resultados deberán estar basados en definiciones correctas.
- Los resultados deberán ser mejorados teniendo en cuenta los objetivos del negocio.

Factores que afectan o determinan la calidad del “software”

El objetivo general de la ingeniería del “software”, es la producción de “software” con capacidad para realizar con exactitud las tareas expresadas en su especificación, y esta calidad puede ser considerada desde dos perspectivas diferentes: la óptica del desarrollador y la del cliente o usuario final. Los factores que afectan al desarrollador se denominan externos e interno.

A continuación, se presenta una breve explicación de cada uno de ellos:

- **Corrección.** Nivel en que un “software” satisface las especificaciones y logra los objetivos propuestos por el cliente; lo anterior responde a la pregunta: ¿el “software” hace lo que yo quiero?
- **Fiabilidad.** Nivel en que se puede esperar que un programa realice sus funciones esperadas con la precisión requerida; lo anterior responde a la pregunta: ¿el sistema está apto para usarse?
- **Eficiencia.** Cantidad de recursos de máquina y de código requeridos por un programa para efectuar sus funciones; este factor responde a la pregunta ¿se ejecutará en mi “hardware” con eficiencia?
- **Integridad.** Nivel en el que puede controlarse el acceso al “software” o a los datos por personal que no es autorizado; este factor responde a la pregunta: ¿el “software” es seguro?
- **Seguridad.** Disponibilidad de instrumentos que permiten controlar y proteger los programas y los datos.
- **Reusabilidad.** Nivel de trabajo solicitado para que el “software” o alguna de sus partes pueda reutilizarse en otras aplicaciones. Este factor responde a la

pregunta: ¿podré reutilizar alguna parte del programa para construir otro proyecto?

- **Exactitud.** Nivel de precisión de los cálculos y del control que tiene el programa al ejecutar sus funciones.
- **Compleitud.** Nivel en que se ha obtenido una total implementación de las funciones solicitadas.
- **Flexibilidad.** Trabajo necesario para modificar un programa que ya está en funcionamiento u operación. Este factor responde a la pregunta: ¿puede ser modificado de manera fácil?
- **Portabilidad.** Trabajo necesario para migrar el programa desde un “hardware” o un entorno de sistemas a otro ambiente u otro “hardware”; este factor responde a la pregunta: ¿podré usarlo en otra máquina o con otra configuración distinta a la máquina donde se desarrolló?
- **Consistencia.** Consiste en utilizar un diseño semejante y técnicas de documentación en el transcurso del proyecto de desarrollo de “software”.
- **Facilidad de uso.** Trabajo necesario para aprender a utilizar un programa y que además sabe preparar su entrada e interpretar su salida.
- **Facilidad de mantenimiento.** Trabajo necesario para ubicar y arreglar un error en un programa; este factor responde a la pregunta: ¿permite ser corregido de manera fácil?
- **Facilidad de prueba.** Nivel de trabajo requerido para probar un programa de forma que asegure la ejecución de las funciones solicitadas.

- **Facilidad de interoperabilidad.** Nivel de trabajo necesario para acoplar un sistema a otro; lo anterior responde a la pregunta: ¿podrá interactuar con otros sistemas?
- **Facilidad de auditoría.** Se concentra en la facilidad en que se puede comprobar la conformidad con los estándares.
- **Normalización de las comunicaciones.** Nivel en que se utiliza el ancho de banda, las interfaces estándar y los protocolos.
- **Tolerancia a los errores.** Percance que se ocasiona cuando el programa encuentra un error.

Costo de los defectos

Los costos de calidad son importantes para conseguir productos de calidad. Por lo que se establecen actividades para prevenir, mejorar o detectar la baja calidad que puede haber en los productos “software”.

En la aplicación de la calidad de “software”, generalmente los ingenieros o empresas comienzan a realizar pruebas y a detectar defectos en el momento en que la aplicación ha sido totalmente construida lo cual no es una acción correcta.

Conclusión de los factores que afectan o determinan la calidad del “software”

Cuando calificamos la calidad de un producto se debe elegir los factores que van a ser estimados como requerimientos, por lo tanto, para realizar esta elección se debe tener en cuenta las propiedades particulares de la aplicación a desarrollar o de su ambiente, como ejemplo podemos observar que si la aplicación se desarrolla para un ambiente en el que el “hardware” evoluciona rápidamente entonces debemos tener en cuenta el factor de portabilidad, por otro lado, si se espera que las especificaciones

del sistema cambien frecuentemente entonces el factor flexibilidad va a ser importante. Otro punto que se debe tener en cuenta es el costo de los factores de calidad frente al beneficio que van a proporcionar (análisis de costo/beneficio), es decir que si agregamos todos los factores implicaría un incremento amplio de los costos, en lugar de lo anterior es prudente seleccionar los factores que logren contribuir el mayor valor posible al producto.

2.7. Calidad relacionada con las personas

La mejora de la calidad de “software” contiene procesos que ayudan a que los ingenieros de “software” manejen, controlen y mejoren el trabajo, teniendo también la oportunidad de construir equipos autodirigidos y ser participantes efectivos del mismo, pero se debe tener en cuenta que primero deben tener el conocimiento para controlar y mejorar su trabajo, y después tener el conocimiento para trabajar en equipo. Por lo anterior el ingeniero podrá plantear mucho mejor el trabajo, medir la calidad del producto, mejorar las técnicas y determinar las medidas estándares para desempeño y calidad. En la calidad relacionada con las personas se establecen dos procesos PSP y TSP que proporcionan métodos detallados de planificación y estimación.

PSP/TSP

“Personal Software Process” (PSP) es un proceso diseñado para ayudar a los ingenieros de “software” en el control, manejo y mejora del trabajo que realizan, el cual está basado en la calidad del “software”; para el caso de “Team Software Process” (TSP) es un marco para el desarrollo de “software” que pone igual énfasis en el proceso, producto y trabajo en equipo.

El PSP tiene un proceso definido que ayuda a realizar mejor los trabajos, con el fin de reportar y conseguir datos completos y precisos del trabajo que se efectúa de manera individual, por lo tanto, mejora el desempeño personal, afectando de esta forma el trabajo del equipo. Por otro lado, contiene una serie de prácticas exactas para la mejora de la productividad personal y la gestión del tiempo de los ingenieros de “software” o desarrolladores, en actividades de mantenimiento de sistemas y desarrollo. Está planteado para ser utilizado en modelos de procesos CMMI o ISO 15504 y está orientado tanto a estudiantes como a ingenieros juniors.

El proceso de esta metodología se clasifica en tres fases:

Fase 1 – Planeación

Se realiza el plan que guiará el proceso, por lo tanto, se establecen dos actividades:

- Análisis para obtener los requerimientos del “software”.
- Estimación de recursos, en la cual se realiza un cálculo del tiempo que se requiere para poder desarrollar el programa.

Fase 2 – Desarrollo

En esta fase se tienen en cuenta las siguientes actividades:

- **Diseño:** se hace revisión de los requerimientos para poder producir un diseño que verifique el cumplimiento de los mismos, por otro lado, si es el caso, se debe efectuar un registro de cualquier defecto encontrado en la definición de requerimientos.
- **Codificación:** se implementa el diseño siguiendo el estándar de codificación y se documenta el tiempo en el registro.

Fase 3 – Fase de “postmortem”

Se realiza un reporte que es utilizado para documentar detalles importantes de una actividad. Por ejemplo, que problemas surgieron y cómo se solucionaron.

En esta fase se tienen en cuenta las siguientes actividades:

- Registro de defectos: se hace revisión del resumen del plan de proyecto y se verifica que los defectos encontrados en cada fase de proyecto fueran registrados.
- Consistencia de información de defectos: se hace revisión de la información de cada defecto registrado el cual debe ser preciso y completado.
- Tiempo: se hace revisión de posibles errores u omisiones en el registro del tiempo.

En cada fase se realiza un criterio de entrada, en la cual se describe el problema, el plan del proyecto y registros de tiempo. En los criterios de salida se realizan los reportes y se tiene en cuenta la documentación que se realizó durante el proyecto.

A continuación, se nombran los formatos utilizados en PSP y que se fundamentan generalmente en dos medidas importantes:

- El tiempo que se emplea en cada fase.
- Los defectos que se encuentran en cada fase.

Formato de registro de tiempo

El objetivo de este formato es registrar el tiempo que se emplea en cada fase del proyecto. No obstante, los datos se utilizan para completar el resumen del plan del proyecto. El tiempo se registra en minutos y deben ser exactos.

Un formato de registro de tiempo, debe contener los elementos mostrados en la siguiente figura:

Figura 1. Ejemplo formato de registro de tiempo

Formato del Registro de Tiempo

Estudiante _____ Fecha _____
Instructor _____ Programa # _____

Fecha	Inicio	Trmino	Tiempo de Interrupción	Tiempo Delta	Fase	Comentarios

Formato de registro de defectos

El objetivo de realizar este registro de defectos está en fomentar la mejora continua en cada proyecto que se realice.

Un formato de registro de defectos, debe contener los elementos que se muestran a continuación.

Figura 2. Ejemplo formato de registro de defectos

Tipos de Defectos	
10 Documentación	60 Chequeo
20 Sintáxis	70 Datos
30 Construcción, Empacar	80 Función
40 Asignación	90 Sistema
50 Interfaz	100 Ambiente

Formato del Registro de Defectos

Estudiante		Fecha				
Instructor		Programa #				
Fecha	Número	Tipo	Encontrado	Removido	Tiempo de compostura	Defecto arreglado
Descripción:						

Fecha		Fecha				
Número	Tipo	Encontrado	Removido	Tiempo de compostura	Defecto arreglado	
Descripción:						

Fecha		Fecha				
Número	Tipo	Encontrado	Removido	Tiempo de compostura	Defecto arreglado	
Descripción:						

Fecha		Fecha				
Número	Tipo	Encontrado	Removido	Tiempo de compostura	Defecto arreglado	
Descripción:						

Resumen del plan del proyecto

Este formato define un histórico de todos los proyectos realizados, en él se encuentran los datos que serán útiles en el proyecto siguiente, por lo tanto, es importante que los datos se redacten con precisión y calidad para tener un margen de comparación con futuros proyectos.

Un formato de resumen del plan del proyecto debe contener los elementos mostrados en la siguiente figura:

Figura 3. Ejemplo resumen del plan del proyecto

Resumen del Plan del Proyecto				
Estudiante			Fecha	
Programa			Programa #	
Instructor			Lenguaje	
Tiempo en la Fase (min.)	Plan	Actual	A la fecha	A la fecha %
Planeación				
Diseño				
Codificación				
Compilación				
Pruebas				
Postmortem				
Total				
Defectos Encontrados		Actual	A la fecha	A la fecha %
Planeación				
Diseño				
Codificación				
Compilación				
Pruebas				
Total Desarrollo				
Defectos Removidos		Actual	A la fecha	A la fecha %
Planeación				
Diseño				
Codificación				
Compilación				
Pruebas				
Total Desarrollo				
Después de Desarrollo				

TSP

“Team Software Process” (TSP) es un proceso de desarrollo que está diseñado para orientar a equipos de personas en la planeación, diseño y desarrollo de sistemas de “software” de calidad. Esta metodología administra el desarrollo de los procesos de “software”, garantizando un entorno de trabajo natural y agradable para los equipos.

El TSP contiene una serie de pasos estructurados con indicaciones para realizar las actividades en cada fase del desarrollo del proyecto, además es una herramienta útil

referente a formación de equipos para el desarrollo de “software” de calidad, generando una planificación que permite determinar las responsabilidades y los roles en los equipos de trabajo. Para ello el TSP se basa en el PSP para formar profesionales con condiciones idóneas para la realización de proyectos demasiado grandes, además este marco contiene las características para generar planes detallados, utilizar datos de procesos, medir y gestionar la calidad del producto y detallar procesos operacionales.

Los objetivos del TSP son:

- Conformar equipos de tal forma que tengan la capacidad de registrar y planear su trabajo, constituir metas bien definidas y que tengan la aptitud para mejorar su propio trabajo a través de la medición del mismo.
- Establecer un marco con base a PSP.
- Establecer estándares para medir la calidad y el comportamiento.
- Suministrar métricas para equipos.
- Evaluar los equipos, las responsabilidades y los roles.
- Guías para solución de problemas que se generen en los equipos.
- Establecer una guía para que la mejora continua de procesos esté activa.

A continuación, se presentan las fases del ciclo de vida del TSP, donde se describe unas pautas para realizar un buen desarrollo de “software” en cabeza del equipo de trabajo.

- **Lanzamiento.** En esta etapa se determinan las metas del equipo, se evalúan los objetivos, se establecen los roles y responsabilidades del equipo, se tienen en cuenta los requerimientos del cliente y la creación de estrategia para la finalización del proyecto.

- **Estrategia.** En esta etapa se crea un modelo conceptual de lo que se necesita para generar soluciones óptimas, determinando las estimaciones y los riesgos.
- **Planeación.** En esta etapa se generan los roles y las actividades de cada miembro del equipo.
- **Requerimientos.** En esta fase se determinan entrevistas con el cliente, con el objetivo de concretar lo que es necesario producir en realidad. Desarrollar un plan de pruebas para el producto terminado previamente al análisis de los requerimientos.
- **Diseño.** En esta etapa se desarrolla un plan de pruebas de integración, por lo tanto, se especifica en un diseño, todos los detalles respecto a los procesos del producto.
- **Implementación.** En esta etapa el diseño se transfiere a nivel de código, realizando una revisión con el propósito de encontrar errores. También los módulos y las unidades se compilan y ejecutan con el propósito de analizar su calidad.
- **Pruebas.** En esta etapa el producto está a punto de finalizar, posteriormente se procede a la integración de los módulos y la documentación para los usuarios finales, como son los manuales de uso. También se establecen las pruebas al “software”, con el objetivo de asegurar la calidad y poder así generar una evaluación en el desempeño del equipo de trabajo.
- **“Postmortem”.** En esta etapa se hace una evaluación de los resultados de las pruebas y del trabajo del equipo. Se redacta un reporte del ciclo de vida del proyecto a detalle.

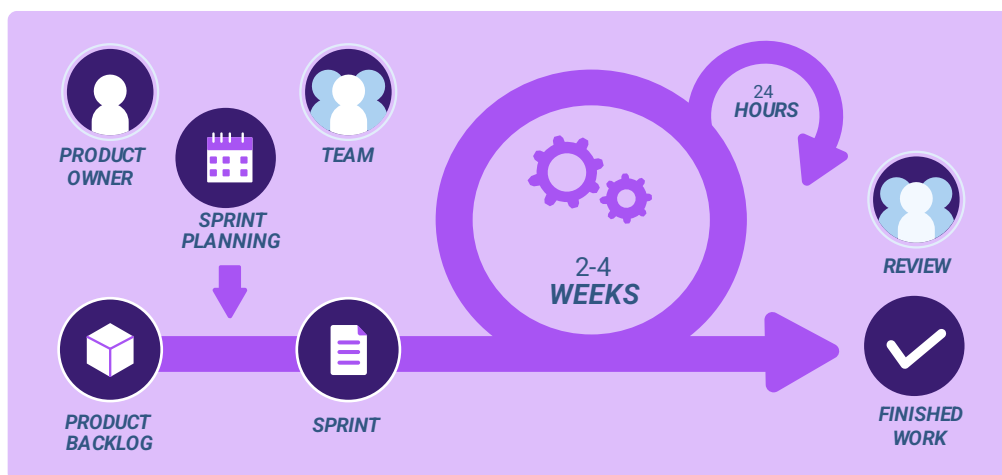
SCRUM

SCRUM es un marco de trabajo ágil de muy amplio uso en la industria del “software” que se fundamenta en los valores y principios ágiles definidos en el Manifiesto Ágil (2001) y donde se definen tres pilares fundamentales (SCRUMstudy, 2013):

- **Transparencia.** Hace referencia a que cualquier proceso de “Scrum” puede ser conocido por cualquiera. Esto es posible por medio de eventos como las reuniones de revisión y reuniones diarias, artefactos como la pila de producto, cronogramas de lanzamiento y documentos de visión del proyecto, e instrumentos de seguimiento como lo son el “BurnDown Chart” o el tablero de “Scrum” (“ScrumBoard”).
- **Inspección.** Permite que cualquiera puede estar enterado de las actividades realizadas por otros y en general conocer el estado actual de los procesos.
- **Adaptación.** Por medio de la transparencia y la inspección es posible fijar actividades de mejoras que permitan mejorar todo tipo de proceso, en pro de lograr más altos estándares de calidad.

Adicionalmente, este marco de trabajo ágil, está estructurado por un conjunto de roles, eventos y artefactos, como se detalla en la siguiente figura.

Figura 4. Roles, eventos y artefactos en el trabajo ágil



Dentro del equipo, hay roles que se dividen en dos categorías fundamentales: los roles centrales que hacen referencia a los requeridos obligatoriamente para la creación de un producto, están altamente comprometidos y de los cuales depende el éxito o no de un proyecto, y también están los roles no centrales que hace referencia a todos el personal interesado en el proyecto, pueden interactuar con el equipo pero no son los responsables del éxito del mismo, dentro de esta categoría entran los “stakeholders”, directivos, gerentes, “marketing”, asesores, etc.

Existen tres roles centrales dentro del marco de trabajo de “Scrum”:

Dueño del producto (“Product Owner”). Persona con amplio conocimiento en el negocio del cliente, sus necesidades y las tendencias del mercado para el área específica. Este rol está encargado de maximizar el valor de negocio entregado al cliente y es el único responsable del control del “Product Backlog” (requerimientos) y su priorización. Este rol también representa al cliente en algunos procesos de demostración de avances y determina cuándo aprobar o no una entrega.

“Scrum Master”. Es un rol que se encarga de facilitar los procesos al interior del equipo de trabajo, removiendo cualquier impedimento y apoyando procesos de empoderamiento personal, debe velar porque los elementos propios del marco de trabajo “Scrum” se apliquen de manera correcta.

Equipo de desarrollo (“Developer team”). Son los responsables de la transformación de los requerimientos en código ejecutable a ser usado por el cliente, pero también son responsables de la planificación de las iteraciones y establecimiento de características para tener en cuenta en la verificación de un requerimiento terminado y presentación de avances a los clientes. Generalmente es un equipo autoorganizado y autogestionado.

Además de los roles, “Scrum” define un conjunto de eventos con participantes y objetivos claros que se desarrollan en momentos particulares del flujo general de “Scrum”, a continuación, se detalla cada uno de estos eventos:

“Sprint”. Es el corazón de “Scrum” y se refiere a una iteración que está acotada generalmente por un lapso entre 2 y 4 semanas, donde se realiza un ciclo completo de actividades de análisis, diseño, construcción y pruebas, para desarrollar una versión del producto potencialmente entregable al cliente.

Planeación de “Sprint”. Reunión realizada justo antes del inicio de un “Sprint”, donde se definen el subconjunto de requerimientos (“Sprint Backlog”) a ser desarrollados en el siguiente “Sprint” y cómo será el proceso requerido para hacer la entrega del siguiente “Sprint”, lo cual incluye detallar los requerimientos en tareas concretas, estimación de tiempos/esfuerzo y distribución inicial de responsabilidades. Dependiendo de la duración del “Sprint” este tiempo de planificación puede variar,

pero la métrica establecida para “Sprint” de 4 semanas corresponde a una planeación de “Sprint” de 8 horas.

Reunión diaria (“daily meeting”). Reunión realizada generalmente al inicio de cada día donde el equipo informa en que ha venido trabajando, qué cosas realizaré en el día y qué problemas se le han presentado. Es una reunión corta que se realiza de pie y que debe tener una duración alrededor de los 15 minutos. Esta reunión se alinea con los pilares de transparencia e inspección.

Revisión del “Sprint”. Reunión realizada al finalizar el “Sprint”, donde el equipo de desarrollo muestra los resultados del “Sprint”. Para “Sprint” de 4 semanas se usa reunión de revisiones 4 horas.

Revisión de retrospectiva. Última realizada luego de la revisión del “Sprint” y tiene como objetivo la autoevaluación personal y del grupo sobre el desempeño del “Sprint” que acaba de finalizar. En esta reunión se identifican y documentan los aprendizajes por medio de diferentes técnicas en las que generalmente se busca dar respuesta a las siguientes preguntas: ¿qué funcionó bien y se debe seguir haciendo?, ¿que no funcionó bien y se debe dejar de realizar? y ¿qué debemos empezar a mejorar? Para un “Sprint” de 4 semanas se utilizan 3 horas para esta reunión.

Finalmente, el marco de trabajo “Scrum” define un conjunto de artefactos que permiten registrar y gestionar información clave, para asegurar los tres pilares fundamentales y proveen información valiosa durante todo el proceso de desarrollo de “software”. Entre los artefactos representativos de “Scrum” encontramos:

Pila de producto (“Product Backlog”)

Lista priorizada de requerimientos generalmente descritos en formato de historias de usuarios, que representa todas las características del sistema a construir.

Pila del “Sprint” (“Sprint Backlog”)

Lista de requerimientos seleccionados desde el “Product backlog” por el equipo de trabajo, para ser desarrollados durante un “Sprint” particular. Este es uno de los artefactos generados en la reunión de planeación del “Sprint”.

“Burndown Chart”.

Es un gráfico de dos ejes que muestra a los equipos la cantidad de trabajo pendiente por completar (eje Y) y el tiempo disponible para hacerlo (eje X).

Este gráfico generalmente se realiza por cada “Sprint”, ubicando la cantidad trabajo a realizar del “Sprint Backlog” (usualmente mide puntos de historia u horas de trabajo) en un tiempo 0 y por cada día finalizado se resta la cantidad de puntos de historia u horas de cada tarea completada, también es posible usar este mismo gráfico para representar el avance general del proyecto, ubicando en el eje Y la cantidad total de horas o esfuerzo del “Product Backlog” y en el eje X la cantidad de “Sprint” proyectados. Cada uno de estos puntos se unen por medio de una línea y es posible determinar visualmente si el flujo de trabajo está en una situación óptima o no respecto al tiempo restante para completar el “Sprint”.

Tablero de “Scrum” (“Scrumboard”)

Es un elemento visual en donde se integra la mayor parte de los elementos del marco de trabajo “Scrum”, en él se indica la carga de trabajo, el estado actual de cada una de las actividades y sus respectivos responsables. Este es un elemento que se

sincroniza de manera permanente y facilita la implementación los pilares de transparencia, inspección y adaptabilidad.

Si bien se aconseja el uso de un tablero existen diferentes tipos de herramientas digitales que permiten la implementación de un tablero de “Scrum”.

Entre los principales beneficios del marco de trabajo “Scrum”, encontramos:

- Es posible gestionar las expectativas del cliente de manera regular ya que este puede y debe participar en las reuniones de revisión por lo que está enterado todo el tiempo del estado actual del proyecto.
- El cliente puede obtener resultados importantes y utilizables desde las primeras iteraciones ya que la lista de producto está priorizada para ofrecer mayor valor en el menor tiempo posible y porque cada finalización de “Sprint” debe tener como resultado una versión totalmente funcional.
- El proyecto puede iniciar con requerimientos de muy alto nivel y es fácil administrar los cambios.
- La participación constante del cliente permite mitigar riesgos del proyecto desde sus primeras etapas.
- Los procesos de retrospectiva permiten establecer actividades permanentes de mejora continua en función de las experiencias del equipo.

Calidad de “software” en metodologías ágiles

La vertiente ágil en el desarrollo de “software” pretende distribuir de forma permanente y continua el proceso de desarrollo distribuido en iteraciones rápidas.

Sin embargo, el término de “metodología ágil” es engañoso dado que implica que el enfoque ágil es la única manera de abordar el desarrollo de “software”. En esta vertiente no se define una serie de pasos o de indicaciones sobre el quehacer en el proceso de desarrollo de “software” por el contrario, trata de una forma de pensar en colaboración y los flujos de trabajo, en el cual se definen una serie de valores que orientan las decisiones respecto a lo que se hace y la forma en que se hace.

Las metodologías ágiles buscan generar una serie de piezas que agregan valor y satisfacción al cliente. En estas metodologías se implementan enfoques flexibles y de trabajo en equipo para mejorar el performance del proceso, con la finalidad de ofrecer mejoras constantes.

Valores de la metodología ágil

La metodología ágil nació aproximadamente en el año 2001, dando respuesta a los proyectos gestionados por medio de metodologías en cascada, por lo cual una serie de desarrolladores redactó el manifiesto ágil. En este manifiesto se describieron cuatro características principales las cuales se deberían priorizar por encima de cualquier otra cosa. Por lo cual, los equipos que trabajan bajo esta vertiente deben valorar lo siguiente:

- Las personas y las interacciones antes que los procesos y las herramientas.
- El “software” en funcionamiento antes que la documentación exhaustiva.
- La colaboración con el cliente antes que la negociación contractual.
- La respuesta ante el cambio antes que el apego a un plan.

En un proyecto ágil vamos a tener iteraciones o “sprints”, cada “sprint” tiene una duración que no es muy extensa aproximadamente dos a cuatro semanas; al final

de cada iteración ya están definidas funcionalidades que pueden ser utilizadas por el cliente. En los proyectos ágiles existen dos tipos de planificaciones: “release”/entrega y “sprint”/iteración; no obstante, estas se pueden dar en un mismo momento, es decir, que puede incluir una sola planificación con actividades tanto del “release” como del “sprint”. Se debe tener en cuenta que en un proyecto ágil las pruebas de “software” se realizan en cada iteración.

La metodología de desarrollo de “software” en cascada utiliza diferentes tipos de pruebas de “software” que probaran tanto requerimientos funcionales como no funcionales de manera secuencial. No obstante, en metodologías ágiles específicamente en las pruebas ágiles, también se consideran varios tipos de pruebas, pero teniendo en cuenta que se utilizan “sprints” o interacciones cortas e integraciones continuas, entre los equipos de desarrollo, diseño y pruebas.

Principios de las pruebas ágiles (“Agile Testing”)

Según los principios del “Agile Testing”, las personas que representan la parte de negocio del producto están involucradas en cada iteración del mismo, y el flujo de retroalimentación continua acorta el tiempo de respuesta y la toma de decisiones para integrar en la mejora del producto, estos principios son:

- **Integración continua.** Los equipos de diseño y desarrollo ejecutan pruebas continuamente, puesto que es una forma de asegurar el avance continuo del producto.
- **Entrega continua.** Se soporta en la integración continua, en este punto los desarrolladores realizan un control y la cantidad de veces que se realiza la entrega continua.

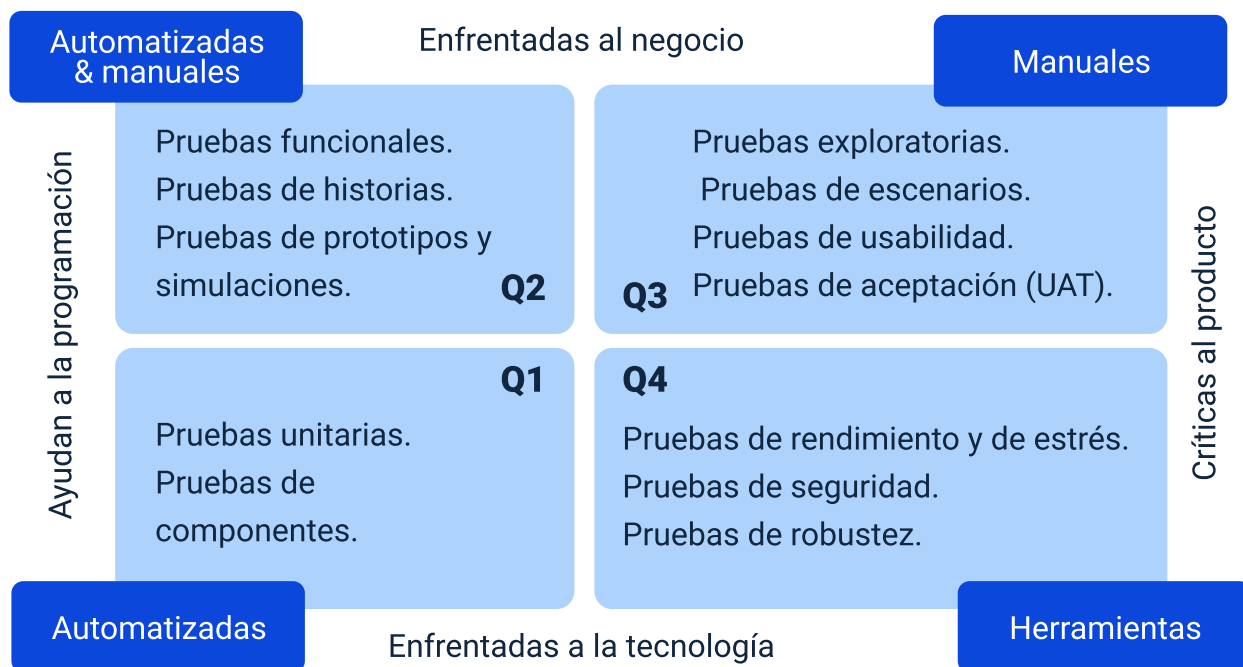
- **Elaborar menos documentación.** Los participantes de los equipos en metodologías ágiles pueden crear listas y tomar notas para centrarse en probar el “software”, lo que permite no enfocarse en detalles secundarios que tomarían mucho tiempo.
- **Responder con velocidad a la retroalimentación.** Los involucrados en el negocio del producto están estrechamente conectados con las interacciones del producto, además el tiempo de respuesta se reduce debido a la realimentación continua en cada parte de la interacción.
- **Proporcionar retroalimentación constante.** En las pruebas ágiles la retroalimentación es continua de modo que el diseño del producto logre los propósitos del negocio requeridos.

Cuadrantes de las pruebas en metodologías ágiles

Para planificar las pruebas ágiles, nos podemos basar en los cuadrantes del “agile testing”, que nos aporta una biblioteca de tipos de pruebas que se pueden usar para cumplir con los requerimientos.

Los cuatro cuadrantes son una clasificación que contribuye al momento de planificar las pruebas ágiles, asegurando que se tengan en cuenta los métodos y recursos para alcanzar productos de “software” de calidad.

Figura 5. Cuadrantes de las pruebas ágiles



Nota. Tomado de <https://lisacrispin.com/2011/11/08/using-the-agile-testing-quadrants/>

Cada cuadrante involucra pruebas específicas, las cuales se distribuyen de acuerdo con los roles ejercidos, es así como se tienen dos tipos de pruebas:

Pruebas de críticas al producto (“Critique the Product”)

Estas pruebas se enfocan en apoyar al equipo desarrollador a través de la construcción (desarrollo) del producto, puesto que le orientan sobre su funcionalidad, posteriormente contribuyen a la introducción de un nuevo código y la refactorización sin que genere resultados imprevistos en el comportamiento del sistema, se encuentran en:

El primer cuadrante (Q1) contiene los siguientes tipos de pruebas

- Pruebas unitarias.
- Pruebas de componente.

Estas pruebas se enfocan en apoyar al equipo desarrollador a través de la construcción (desarrollo) del producto, puesto que le orientan sobre su funcionalidad, posteriormente contribuyen a la introducción de un nuevo código y la refactorización sin que genere resultados imprevistos en el comportamiento del sistema, se encuentran en:

- Pruebas funcionales.
- Pruebas de historias.
- Pruebas de Prototipos y simulaciones.

Las pruebas y las funcionalidades de este cuadrante detallan cada historia de usuario, además también pueden ser automatizadas, no obstante, es necesario que algunas de estas pruebas sean ejecutadas desde la interfaz de usuario o lo que es lo mismo desde el punto de vista del cliente.

En las pruebas de este cuadrante, los diseñadores de la interfaz de usuario crean “wireframes” para realizar en primera instancia, validaciones con el cliente, y posteriormente iniciar con el desarrollo.

Pruebas de apoyo al equipo (“Supporting the Team”)

Estas pruebas pueden tener un carácter positivo, puesto que a partir de ellas se pueden sugerir o realizar mejoras. Cuando se ejecutan, la idea es simular de la forma más fiel, el ambiente real en el cual serán ejecutadas, así mismo estas pruebas

frecuentemente son realizadas por los usuarios finales como pruebas de aceptación (UAT)

El tercer cuadrante (Q3) contiene los siguientes tipos de pruebas:

- Pruebas exploratorias.
- Pruebas de usabilidad.
- Pruebas de escenarios.
- Pruebas de aceptación de usuario.

El cuarto cuadrante (Q4) contiene pruebas técnicas relacionadas con los requerimientos no funcionales, por lo que validan el cumplimiento de estos, analizando la seguridad, el desempeño y la robustez:

- Pruebas de rendimiento y de estrés.
- Pruebas de seguridad.
- Pruebas de robustez.

Metodología XP - Programación Extrema

XP es la abreviación comúnmente utilizada para referirse a “Extreme Programming”, el cual es un marco de desarrollo de “software” ágil que busca producir “software” de alta calidad en contextos con requisitos altamente cambiantes, riesgos que involucran tiempos fijos con tecnologías nuevas y equipos de trabajo pequeños ubicados en un mismo sitio.

XP define cinco valores (Beck & Andres, 2004), tal como se indica a continuación:

Comunicación. El desarrollo de “software” requiere de trabajo en equipo, por lo cual es importante la transferencia de conocimientos y utilizar medios de

comunicación apropiados, se propone la discusión cara a cara con herramientas que permitan dibujar o escribir como por ejemplo, un tablero.

Simplicidad. Se refiere a hacer solo las cosas que sean absolutamente necesarias, evitando el desperdicio, elaborar las cosas de forma que sea fácil entender por otros y abordar solo requisitos conocidos.

Retroalimentación. Permite identificar áreas de mejora y revisión constante de las prácticas implementadas en el proceso de forma, que puedan establecer procesos de mejora permanente.

Coraje. Se refiere al actuar sobre situaciones que pueden ser retadoras para el equipo como por ejemplo, enfrentar problemas organizacionales, intentar implementar funcionalidades de formas diferentes cuando lo convencional no funciona, aceptar comentarios, etc.

Respeto. Los miembros del equipo deben respetarse entre sí para comunicarse y trabajar en equipo. Cada persona contribuye en favor de lograr el objetivo del equipo.

Adicional a los valores, XP se caracteriza por la definición de un conjunto de 12 prácticas de desarrollo de “software” que, aunque pueden ser adoptadas de forma aislada, tiene mayor relevancia cuando son desarrolladas en conjunto (Jeffries, 2011):

- **Planificación.** Consiste en una reunión de planeación que realiza el equipo de desarrollo en conjunto con el cliente, para discutir y probar las características a ser desarrolladas.
- **Pequeños lanzamientos.** Esta práctica sugiere hacer iteraciones cortas con funcionalidades pequeñas que puedan ser probadas con el cliente, de forma que se obtiene realimentación temprana y frecuente.

- **Metáfora.** Los nombres usados para definir cualquier tipo de identificador en el sistema deben ser coherentes. El diseño y la estructura general debe ser comprensible para cualquier persona.
- **Diseño simple.** Hacer código que funcione y que sea a la vez la solución más simple posible, evitando duplicación de código, menor número de métodos y clases.
- **Pruebas.** Sugiere el uso de técnicas donde se enfatice en el proceso de pruebas constantes antes de iniciar la construcción del código, como por ejemplo TTD (Desarrollo basado en pruebas).
- **Refactorización.** Eliminación de funciones innecesarias, desacoplar elementos, eliminar cualquier tipo de redundancia de forma que se mantenga un código lo más limpio posible, que sea fácil de entender y modificar.
- **Programación en pareja.** Esta práctica propone que un código sea desarrollado por el trabajo de dos personas, donde una de ellas codifica y la otra está haciendo revisión, sugerencias y corrección de errores en el mismo momento de escritura.
- **Propiedad colectiva.** Todo el equipo es dueño del sistema, esto promueve la cooperación y la libertad de cada miembro para aportar nuevas ideas.
- **Integración continua.** Implica el uso de sistema de control de versiones de forma que el código pueda ser compartido o reutilizado, pero también cada persona pueda trabajar de forma independiente y en paralelo sobre el mismo proyecto.
- **Semana de 40 horas.** Es importante que los desarrolladores tengan espacios de trabajo apropiados, donde puedan descansar y mantener un equilibrio entre la

vida laboral y personal favoreciendo la eficiencia y puedan sostener la calidad durante todo el tiempo. Propone que no se deben exceder las 45 horas de trabajo semanales.

- **Cliente in situ.** El cliente final debe participar durante todo el proceso de construcción de “software” y estar presente para resolver cualquier pregunta que se pueda generar.
- **Estándar de codificación.** Se deben utilizar los mismos formatos y estilos en la redacción de código.

3. Documentar el proceso de calidad de “software”

La documentación de productos de “software” son artefactos importantes, dado que estos permiten transferir y comunicar aspectos que al revisar o inspeccionar un sistema no se pueden entender rápidamente y suele ser complicado, por tal razón es que se documenta para comprender, compartir y mostrar el comportamiento y la estructura de un sistema o de sus componentes, controlar y visualizar la arquitectura del sistema y controlar el riesgo, además la documentación debe utilizarse para el desarrollo del producto “software” y su mantenimiento en el futuro resaltando que no solo se documenta un proceso respecto a la calidad, sino, también las fases del desarrollo del “software”, requerimientos, análisis, diseño, construcción y pruebas. Sin embargo, cuando se requiere documentar un proceso de calidad de “software” es pertinente tener en cuenta tanto el plan de pruebas como los resultados arrojados que se muestran en los apartados siguientes.

3.1. Diseñar los instrumentos de calidad de “software”

Los instrumentos de calidad de “software”, al ser combinados con otras partes, son útiles al momento de realizar mediciones, observar y almacenar datos que sean verificables y que se puedan utilizar para mejoras continuas en los procesos de calidad de un producto. Entonces para documentar pruebas de “software”, es importante ver el plan de pruebas como un instrumento clave en el diseño y creación de pruebas de “software”.

Plan de pruebas

Un instrumento de gran utilidad para estructurar el proceso de pruebas y la documentación es el plan de pruebas. Por lo tanto, la finalidad es suministrar la

información requerida para planear y controlar las actividades relacionadas con las pruebas en el proceso de desarrollo de un producto “software”. Presenta el enfoque para realizar la verificación de los componentes del producto. Todo proceso de desarrollo de “software” debe contar con la implementación de la fase de pruebas.

El personal que se dedica a ejecutar pruebas de “software” requiere de un plan de pruebas de “software”, cuyo objetivo es comunicar a todos los involucrados del proyecto: los entregables, ítems a ser certificados, criterios de aprobación y fallos, criterios de suspensión y reanudación, las necesidades de ambiente, las capacitaciones necesarias para los integrales del equipo, riesgos y el laboratorio de usabilidad.

El plan se puede aplicar a todo el proyecto y se ajusta a las necesidades de cada empresa, teniendo en cuenta el tamaño del proyecto, el tiempo, el costo, el ciclo de vida del “software”, los involucrados.

Cada entidad puede definir su propio plan de pruebas, basados en buenas prácticas, de acuerdo con las siguientes características:

El plan de prueba. Denota todos los procedimientos y métodos que serán empleados para la certificación del “software” determinando, si cumple con las especificaciones presentadas en los requisitos y dadas por el cliente. Este incluye los objetivos de calidad, los recursos, cronograma, asignaciones, y métodos, entre otros.

Casos de prueba. Listar los ítems que serán certificados y describe detalladamente los pasos que serán ejecutados para evaluar la verificación de las funcionalidades del producto “software”.

Reporte de pruebas. Describe los fallos encontrados en el proceso de ejecución de pruebas y las pruebas exitosas.

Formatos

Recordemos que el estándar ISO/IEC/IEEE 29119-3:2013 es un instrumento que suministra una serie de artefactos validados a nivel internacional por las organizaciones a la vanguardia del proceso de desarrollo de “software”, y a su vez al margen de la ejecución de pruebas de “software”. Los de mayor reconocimiento son:

- Documentación de proceso de pruebas (a escala) organizacional.
- Documentación del proceso de gestión de pruebas.
- Documentación del proceso de pruebas dinámicas.

Artefactos de prueba

Los productos que resultan del proceso de desarrollo de “software” son identificados como artefactos, por ejemplo, el código fuente, defectos, plan y script de pruebas.

La calidad de “software” se compone de artefactos específicos que se forman en las diversas etapas del proceso de SQA, como se describe a continuación.

Planificación

- **Plan de pruebas:** documento que describe un enfoque sistemático para probar un sistema.
- **Informe de resultados de pruebas:** organiza y presenta un análisis resumido de los resultados de las pruebas para su auditoría y evaluación. Este informe debe incluir un listado de defectos detectados, la cantidad y tipos de pruebas elaboradas, así como su complejidad y los tipos de pruebas realizadas.

Construcción

- **Caso de prueba:** conjunto de condiciones con las cuales un “tester” debe determinar si un sistema funciona según lo requerido.
- **“Script” de prueba:** recopilación de instrucciones que se realizan al “software” sujeto a prueba para verificar que el sistema funciona de acuerdo con lo requerido. Hay “scripts” de pruebas automatizadas y manuales.

Ejecución

- **Resultado de la ejecución de pruebas:** recopilación de la información relacionada con los resultados de la ejecución de cada script de prueba, determinando si tuvieron una salida exitosa o un problema, respecto al comportamiento que debería tener.
- **Defectos:** un error produce un defecto y un defecto produce un fallo.

3.2. Aplicar los instrumentos de calidad de “software”

En la elaboración de un plan de pruebas se deben tener claros los requerimientos de usuario que forman la iteración o proyecto, por lo tanto, se debe analizar la información de la especificación de requisitos, la matriz de trazabilidad, especificaciones y diseño funcional, casos de uso, requisitos no funcionales, prototipado e historias de usuario en el caso de metodologías ágiles.

Implementación del plan de pruebas

El plan de pruebas es un documento que describe un conjunto de procedimientos, técnicas y normas para probar un sistema como se indica a continuación:

- a) Debe contener el nombre del sistema a probar, el nombre del documento y la versión.
- b) Especificar en qué consiste el sistema y cuál sistema se va a probar. También se incluye y describe el alcance de las pruebas.
- c) En esta parte se listan los tipos de prueba a realizar y los requerimientos a probar en el proceso. Se especifica y describe las características a probar y las que no se van a probar.
- d) Representa el enfoque recomendado para la comprobación de las aplicaciones.
- e) En esta parte se especifican los criterios de entrada y salida de la aplicación al área de Aseguramiento de Calidad de “Software” (SQA).
- f) Se describen las actividades (plan, diseño, implementación y ejecución de pruebas, así como la evaluación de los resultados) que realizarán el equipo de SQA.
- g) Se describe el informe de resultados de prueba y los defectos detectados durante el esfuerzo de pruebas.
- h) Describe los recursos del sistema.
- i) Describe las características del personal requerido para el esfuerzo de pruebas, así como las necesidades de entrenamiento.
- j) En este punto se plantean las fechas estimadas de inicio y fin de las principales actividades de prueba.
- k) En este punto se plantean los riesgos identificados al momento de la planificación y su estrategia de mitigación.

Principales elementos para redactar un caso de prueba

La estructuración de un caso de prueba se convierte en una actividad sin dificultad, si tenemos la información necesaria para su proceso de elaboración, así mismo al momento de verificar un “software” es de mucho aporte, puesto que se transforma en una herramienta esencial en el proceso de registro, seguimiento y control. A continuación, en la siguiente tabla, se describen los elementos principales que debe contener un caso de prueba.

Tabla 5. Principales elementos de un caso de prueba

Elemento	Descripción
Identificador	Puede ser alfanumérico o numérico.
Nombre	Nombre del caso de prueba de manera concisa.
Descripción	Objetivo del caso de prueba, también describe qué probará; en ciertas ocasiones se incluye el ambiente de pruebas.
Número de orden ejecución	Orden en la cual se ejecuta el caso de prueba, en la situación de que se tengan múltiples casos de prueba.
Requerimiento asociado	Si se plantea un caso de prueba se debe saber a qué requerimiento va asociado para mantener la trazabilidad.
Precondición	Estado en la cual se debe encontrar el sistema antes de comenzar la prueba.
Postcondición	El estado en que debe encontrarse el sistema luego de ejecutar la prueba.
Resultado esperado	Objetivo que debe ser alcanzado posterior a la prueba.

Principales elementos para redactar un script de prueba

Este artefacto es responsabilidad del “tester”, siendo este quien lo escribe y lo ejecuta. A continuación, se nombran los principales elementos que debe contener un “script” de prueba:

- **Nombre.** Identificador único para el “script”.

- **Descripción.** Describe el objetivo del “script” de pruebas.
- **Pasos.** Acciones que se van ejecutando en la aplicación.
- **Puntos de verificación.** Son las observaciones puntuales que se realizan durante la ejecución de las pruebas para verificar que el producto cumple con los requerimientos específicos.
- **Resultado esperado.** Por cada punto de verificación puede haber un resultado esperado asociado.

Creación del plan de pruebas para proyectos ágiles (“Agile Testing”)

Un “reléase” define funcionalidades que ya están disponibles para el cliente. Por lo tanto, una planificación de un “release” tiene ciertos elementos:

- **Alcance.** Se incluye el propósito del “release” y las historias de usuario.
- **Suposiciones.** Se incluyen los supuestos del proyecto.
- **Análisis de riesgos.** Define qué se incluye en la automatización de pruebas.
- **Automatización de pruebas.** Define qué se incluirá en la automatización.
- **Ambiente de pruebas.** Define qué se necesita en relación al ambiente de pruebas.
- **Datos de las pruebas - Resultados de las pruebas.** Se establecen los datos arrojados en cada una de las pruebas, junto con los resultados encontrados.

Estos elementos se describen en un plan de pruebas, sin embargo, es posible que en los equipos se tome la decisión de no diseñarlo, en ese sentido se recomienda que los “tester” tomen notas o documente los factores más importantes relacionados con las pruebas en cada “release”.

Para realizar pruebas de “software” en una metodología ágil, comenzamos por redactar el plan de pruebas, teniendo en cuenta qué se puede actualizar en cada “sprint”.

Un plan de pruebas ágiles debe incluir:

- a) Introducción.
- b) Alcance.
- c) Recursos en este caso el nombre de los “tester”.
- d) Descripción de funcionalidades a probar.
- e) Los tipos de pruebas (pruebas de rendimiento, pruebas de aceptación - UAT) que se van a realizar.
- f) Infraestructura lista.
- g) Suposiciones, plan de riesgos y los entregables que se producirán al final.

3.3. Análisis y entrega de resultados

Una vez ejecutadas las pruebas, se deben analizar los resultados y los fallos detectados, teniendo en cuenta el reporte de defectos y directrices para detectarlos y el informe de resultados de pruebas. Para ello se puede realizar lo descrito a continuación:

- **Impacto.** Analizar el impacto del defecto.
- **Problema.** Investigar el problema, en qué condiciones se produce este fallo.
- **Severidad.** Analizar la severidad, si es alta o baja.
- **Pasos.** Variar los pasos realizados, se pueden ejecutar en orden diferente para observar si existe alguna variante en el resultado esperado.

- **Configuración.** Variar opciones de configuración; si estamos probando una aplicación web podemos probar con diferentes navegadores.
- **Condiciones.** Determinar condiciones específicas bajo las cuales se reproduce el defecto. Si estamos probando entradas de datos, probar con diferentes datos, es decir, se va a tener más información sobre cómo se reproduce el defecto, si por ejemplo, en los campos numéricos se prueba números positivos y negativos.

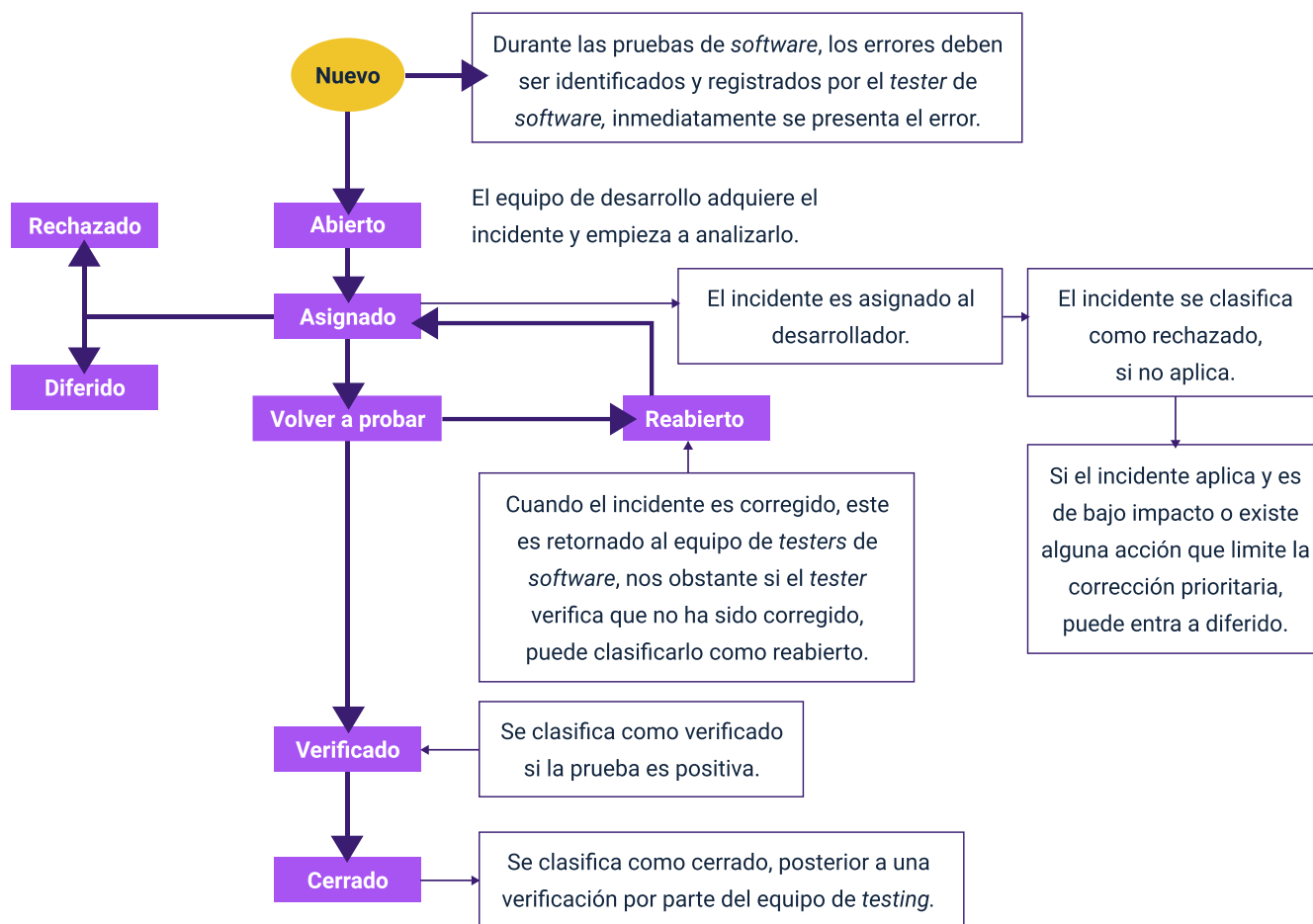
Incidencias detectadas

La gestión de incidencias es un elemento central e importante en el proceso de calidad de “software”, puesto que es en este punto donde se tienen en cuenta los errores (“bugs”). Por lo tanto, el propósito de las incidencias detectadas es precisamente generar su corrección, de tal manera que sea mínimamente probable que el error se repita.

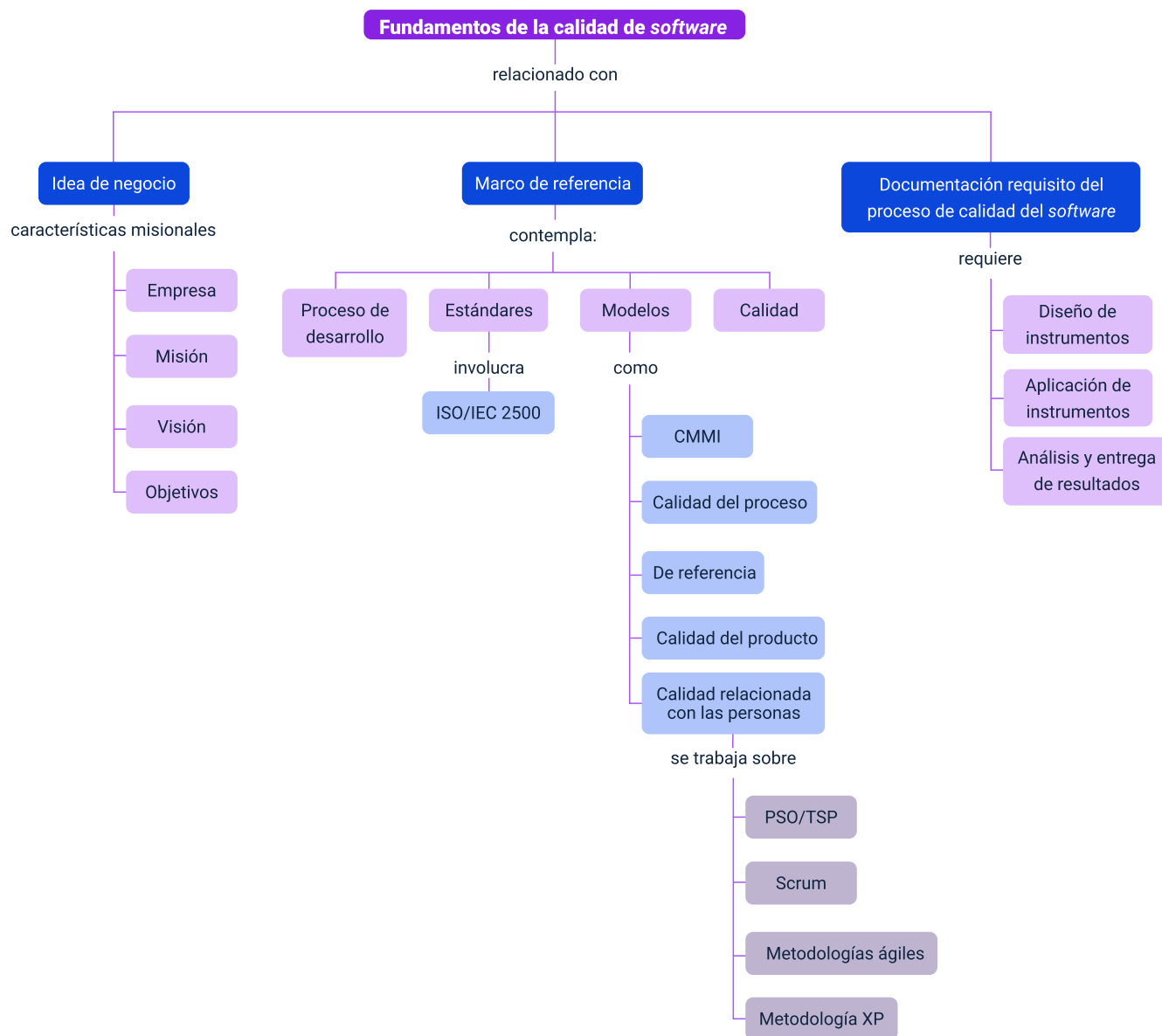
Ciclo de vida de una incidencia

Un sistema de gestión de incidentes debe manejar los errores de “software” teniendo en cuenta el proceso. Este se adapta a proyectos de “software” en fase de desarrollo, más no en ambiente de producción.

Figura 6. Ciclo de vida de una incidencia de “software” (“bug”)



Síntesis



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
PSP/TSP	Callejas-Cuervo, M. & Alarcón-Aldana, A. C. (2017). Modelos de calidad del “Software”, un estado del arte. Entramado, 13(1), pp. 236–250.	Artículo	https://revistas.unilibre.edu.co/index.php/entramado/article/view/428
Modelo CMMI	Chavarría, A., Bayona Oré, S. & Pastor, C. (2016). Aseguramiento de la Calidad en el Proceso de Desarrollo de “Software” utilizando CMMI, TSP y PSp. Risti, 20(12), pp.62-77.	Artículo	https://scielo.pt/pdf/rist/n20/n20a06.pdf
Diseñar los instrumentos de calidad de “software”	Ch Ga, F. (2017). Plan de pruebas de “software”. Mundo Testing.	Página web	https://mundotesting.com/plan-de-pruebas-de-software/

Glosario

Incidencia: suceso que se produce durante una actividad y puede causar, una disminución de calidad de este.

PSP: “Personal Software Personal”.

QA: calidad de “software”.

SQA: aseguramiento de la calidad de “software”.

TSP: “Teams Software Personal”.

Referencias bibliográficas

Beck, K., & Andres, C. (2004b). Extreme Programming Explained: Embrace Change, 2nd Edition (The XP Series) (2nd ed.). Addison-Wesley.

Bustamante Ramírez, J. (2011). Sistema de informes para pruebas de software. <http://bibliotecadigital.iue.edu.co/xmlui/handle/20.500.12717/153>

Clemente, P. J., & Gómez, A. (2014). Aplicación de un proceso de mejora continua en una asignatura de Desarrollo de Software Dirigido por Modelos. <http://hdl.handle.net/2099/15497>

Jeffries, R. (2011). What is Extreme Programming? Ronjeffries.Com. <https://ronjeffries.com/xprog/what-is-extreme-programming/>

Kruchten, P. (2003). The Rational Unified Process: An Introduction (3rd Edition) (3rd ed.). Addison-Wesley Professional.

Maida, EG, Pacienza, J. (2015). Metodologías de desarrollo de software [en línea]. Tesis de Licenciatura en Sistemas y Computación. Facultad de Química e Ingeniería "Fray Rogelio Bacon". Universidad Católica Argentina, 2015 <https://repositorio.uca.edu.ar/bitstream/123456789/522/1/metodologias-desarrollo-software.pdf>

Manifiesto por el Desarrollo Ágil de Software. (2001). Agilemanifesto.Org. <https://agilemanifesto.org/iso/es/manifesto.html>

Martin, J. (1991). Rapid Application Development. Macmillan Coll Div.

Mera Paz, J. A. (19 de 10 de 2016). Análisis del proceso de pruebas de calidad de software. <https://repository.ucc.edu.co/handle/20.500.12494/962>

Royce, W.W. (1970) Managing the Development of Large Software Systems. Proceedings of IEEE WESCON, 26, 328-388.

SCRUMstudy. (2013). A Guide to the Scrum Body of Knowledge (SBOK Guide) (2013th ed.). VMEdU Inc.

Sommerville, I., Galipienso, M. I. A., & Martinez, A. B. (2005). Ingeniería del Software. Pearson Educación.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Líder del Ecosistema	Dirección General
Olga Constanza Bermudez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Ervin Andrade	Experto temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Peter Emerson Pinchao Solis	Experto temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Blanca Flor Tinoco Torres	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Luis Jesús Pérez Madariaga	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Aplicación de pruebas de “software”

Breve descripción:

En este componente, el aprendiz identificará que un “software” de calidad implica la utilización de diferentes tipos de pruebas, metodologías y procedimientos estándares para el análisis, diseño, programación y prueba del “software”. Lo anterior, con el objetivo de lograr una mayor confiabilidad, mantenibilidad y facilidad de prueba, así como de elevar la productividad tanto en la labor de desarrollo como en el control de calidad.

Tabla de contenido

Introducción	1
1. Realización de pruebas de “Software”	2
1.1. Tipos de pruebas	2
Pruebas funcionales	3
Pruebas unitarias.....	4
Pruebas de integración.....	6
Pruebas no funcionales	7
Pruebas de rendimiento	9
Ventajas y desventajas de los tipos de pruebas.	10
Desventajas de los tipos de pruebas	11
Documentos de caso de pruebas	12
Ejecución de las pruebas	14
1.2. “Agile Testing”	16
Principios de las pruebas ágiles (“Agile Testing”).....	18
Cuadrantes de las pruebas ágiles (“Agile Testing”).....	19
“Test Driven Development” (TDD)	23
“Acceptance Test Driven Development” (ATDD).....	25
“Behaviour Driven Development” (BDD)	28

“Testing” exploratorio	32
Automatización de pruebas de regresión	34
Pruebas de exploratorias, usabilidad y aceptación	35
Pruebas desempeño, carga y seguridad	38
1.3. Elaboración de informe de resultados.....	41
2. Elaboración del plan de mejora	46
Identificar el área de mejora	46
2.1. Seleccionar las acciones de mejora	49
2.2. Realizar una planeación	50
2.3. Seguimiento del plan de mejora	51
Síntesis	53
Material complementario	54
Glosario	55
Referencias bibliográficas	57
Créditos	59

Introducción

En este componente formativo, denominado "Aplicación de pruebas de software", se estudiarán los diferentes elementos necesarios para la realización de pruebas. Por lo tanto, se abordarán las siguientes áreas de aprendizaje: tipos de pruebas funcionales y no funcionales, pruebas ágiles ("Agile Testing") y la elaboración de informes de resultados.

Es importante resaltar que, en el desarrollo de aplicaciones web, uno de los principales desafíos es asegurar que la calidad del producto sea óptima y libre de errores. En este contexto, las pruebas de "software" constituyen un componente fundamental del proceso de aseguramiento de la calidad y la adopción de buenas prácticas.

Las pruebas de "software" engloban el conjunto de actividades realizadas para identificar posibles fallos de funcionamiento, configuración o usabilidad en programas o aplicaciones, mediante la evaluación de su comportamiento. La complejidad e interoperabilidad de los sistemas informáticos, programas y aplicaciones han aumentado significativamente, lo que a su vez ha incrementado las posibilidades de defectos.

Para aplicar pruebas de "software" eficazmente, se deben estudiar los diferentes elementos que las componen. Así, se profundizará en los tipos de pruebas funcionales y no funcionales, las pruebas ágiles y la elaboración de informes de resultados. Cabe destacar que, en el contexto del desarrollo de aplicaciones web, verificar la óptima calidad del producto sin errores es esencial, haciendo que las pruebas de "software" sean un pilar fundamental en el proceso de desarrollo.

1. Realización de pruebas de “Software”

Para realizar las pruebas de “software”, es necesario establecer qué tipos de pruebas serán requeridas por el producto. Esto se debe a que no es necesario utilizar todos los tipos de pruebas existentes, sino más bien aquellas que sean necesarias para cada caso específico. Por esta razón, es crucial que el QA o el líder de pruebas determine qué tipos de prueba se adaptan mejor al proyecto en evaluación. A continuación, se especifican y describen los tipos de pruebas que se pueden utilizar.

1.1. Tipos de pruebas

Las pruebas de “software” se pueden clasificar principalmente en dos categorías: pruebas funcionales y pruebas no funcionales. Considerando los diversos aspectos y componentes de un producto de “software”, se requerirán distintos tipos de pruebas, como pruebas unitarias, pruebas de integración, pruebas de estrés, pruebas de rendimiento, pruebas de escalabilidad, entre otras. Cada uno de estos tipos de pruebas determina la calidad de la aplicación, desde el código fuente hasta la interacción y experiencia del usuario.

Pruebas y calidad de “software”

Les invitamos a explorar el video y descubrir los diferentes tipos de pruebas y enriquecer su conocimiento en esta área clave del desarrollo de “software”.

<https://www.youtube.com/watch?v=TLs-a1Cok-w>

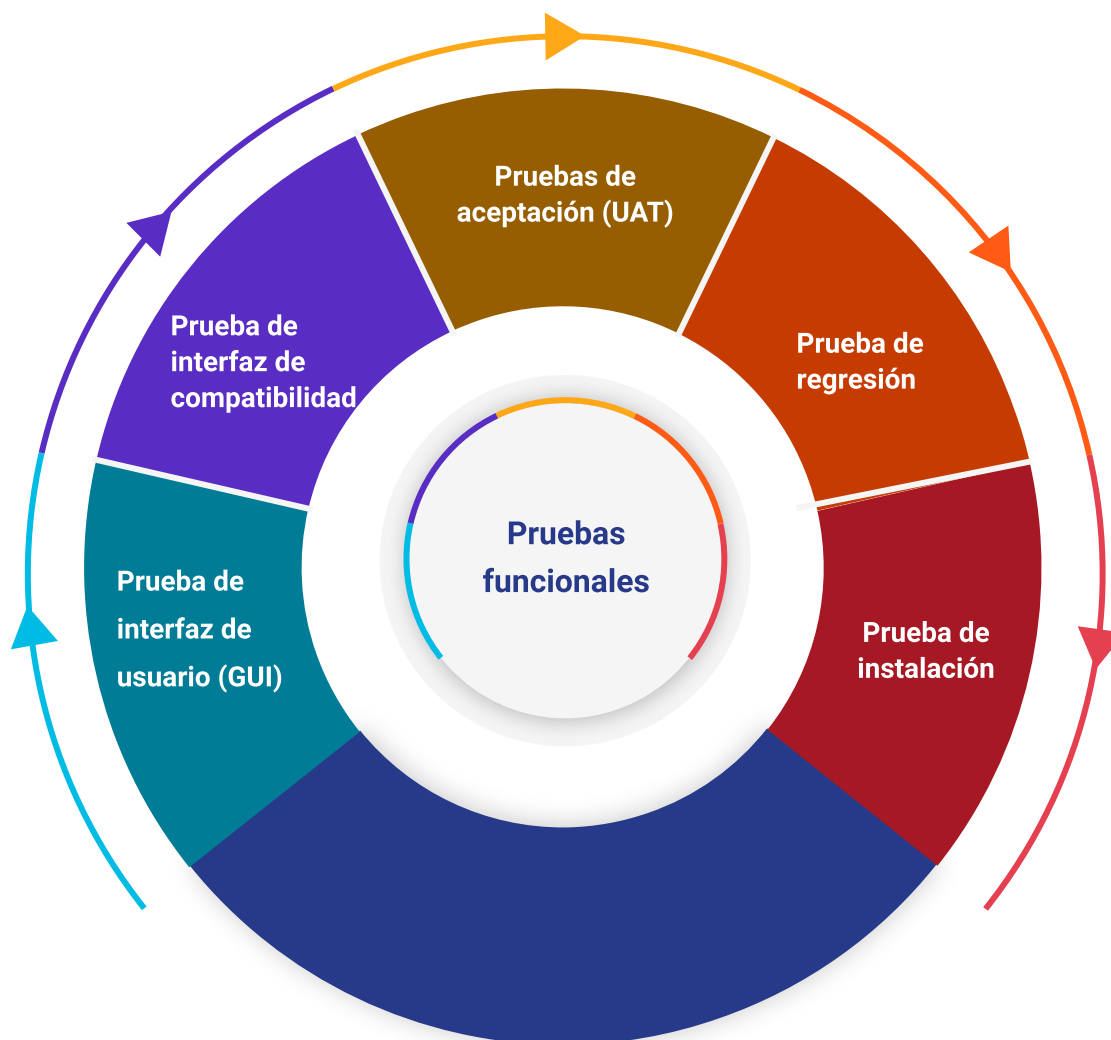
Algunos de los tipos de pruebas más comunes se describen en los siguientes puntos.

Pruebas funcionales

Se realizan para verificar las características cruciales para el negocio, incluyendo la funcionalidad y la usabilidad del “software”. Estas pruebas aseguran que las funcionalidades y características del “software” funcionen y se comporten conforme a lo esperado, sin inconvenientes. Validan la aplicación completa de acuerdo con las especificaciones detalladas en el Documento de Requisitos del “Software” (SRS, por sus siglas en inglés). Dentro de este conjunto de pruebas se incluyen las pruebas unitarias, pruebas de regresión, pruebas de interfaz, entre otras.

IBM (2021) plantea que estas pruebas se basan en la ejecución y revisión de las funciones y en su interoperabilidad con sistemas específicos, se llevan a cabo en todos los niveles de prueba, buscan evaluar cada una de las opciones con las que cuenta el paquete informático, principalmente en el comportamiento externo del producto o aplicativo, también se las identifica como de tipo caja negra.

Figura 1. Tipos de pruebas funcionales



Pruebas unitarias

Estas pruebas se centran en validar elementos como componentes, unidades o piezas de forma individual e independiente dentro de un producto de “software”, siguiendo el principio del Ciclo de Vida del Desarrollo de “Software” (SDLC, por sus siglas en inglés). Cualquier bloque de código, ya sea un procedimiento, función, método, constructor, clase o módulo, puede ser considerado una unidad y, por lo tanto, debe ser objeto de pruebas unitarias. Esto permite determinar su comportamiento y

validar si los resultados obtenidos se alinean con los esperados. Las pruebas unitarias deben ser las primeras en realizarse durante la fase de desarrollo y deben ser implementadas por los desarrolladores.

De acuerdo con Mera (2016) también se las conoce como “unit testing” son pequeños “test” en los cuales se revisa que el comportamiento de un objeto y su lógica funcione adecuadamente.

Generalmente, se llevan a cabo durante la fase de desarrollo de las aplicaciones o “software”. Esta tarea es ejecutada por los programadores, aunque los aseguradores de calidad también pueden participar en su realización. Las razones para desarrollar este tipo de pruebas incluyen:

- Son rápidas de ejecutar, lo que permite realizarlas en gran cantidad.
- Funcionan como documentación de apoyo para el proyecto.
- Demuestran que la lógica del código funciona correctamente en todos los casos.
- Facilitan a los programadores el entendimiento profundo del código base, permitiendo realizar cambios de manera oportuna.
- Contribuyen a obtener un código de alta calidad.

En algunos casos, las pruebas pueden realizarse de manera manual, pero es preferible utilizar herramientas que permitan ejecutar el servicio de la mejor manera posible. Existen muchas herramientas disponibles en el mercado, y estas varían en función del lenguaje de programación que se esté utilizando. A continuación, mencionaremos algunas de las más conocidas:

- **XUnit.**

Es una herramienta de pruebas unitarias para el “framework” .NET.

- **Nunit.**

Es una herramienta de pruebas de plataformas .NET.

- **Junit.**

Es una herramienta de pruebas de aplicaciones Java.

- **PHPUnit.**

Es una herramienta de pruebas de programación PHP.

Para realizar las pruebas unitarias, se deben tener en cuenta los siguientes criterios:

- a) Se debe probar un sólo código a la vez.
- b) Hay que realizar pruebas frecuentemente mientras se programa.
- c) Se debe corregir los “bugs” identificados en las pruebas antes de continuar.
- d) Cualquier cambio que se realice también debe pasar el “test”.

Pruebas de integración

Estas pruebas se centran en validar distintos módulos o componentes de un producto de “software” de tal forma que se verifique su funcionamiento como grupo funcional. Productos como aplicaciones de “software”, sistemas de información o aplicaciones web, están compuestos por diversos componentes, módulos y submódulos distribuidos en distintos niveles y capas, que colaboran y se coordinan para habilitar diferentes funcionalidades.

El objetivo de estas pruebas es asegurar que la integración entre estos componentes sea adecuada y detectar posibles problemas o errores relacionados con la sinergia y comunicación entre ellos.

Según Mera (2016) “Las pruebas de integración se encargan de probar las interfaces entre los componentes o módulos; por ejemplo, el componente validación de usuario con el sistema operativo, el sistema de archivos en integración con el “hardware””

De acuerdo con lo anterior, es importante tener en cuenta los siguientes objetos típicos de prueba:

- Bases de datos de subsistemas.
- Datos de configuración.
- Configuración del sistema.
- Infraestructura.
- Interfaces.

Pruebas no funcionales

Estas pruebas, similares a las pruebas funcionales, se diferencian en que se realizan bajo condiciones de carga de rendimiento para evaluar aspectos como la fiabilidad, usabilidad, escalabilidad, instalación, confiabilidad y seguridad del “software”. Generalmente, se llevan a cabo mediante herramientas o soluciones automatizadas.

Según Mera (2016) en este tipo de pruebas se comprueban los requisitos basados en la operación de un “software”. Para verificar la carga que resiste el

producto “software”, identificando si su rendimiento es el adecuado o si es estable a nivel de contacto con el servidor.

Algunos aspectos, como el rendimiento de una aplicación, pueden ser complejos; sin embargo, esta prueba permite evaluar la calidad del “software” a probar. La calidad se basa principalmente en factores como el tiempo, la precisión, la estabilidad, la corrección y la durabilidad del producto bajo diversas circunstancias adversas. A partir de estos criterios de calidad, es posible explorar distintos tipos de pruebas no funcionales, tales como:

- **Pruebas de carga**

Simulan la demanda esperada que puede tener una aplicación y mide el resultado.

- **Pruebas de rendimiento**

Se calcula la respuesta del programa con diferentes medidas de peticiones del usuario.

- **Pruebas de estrés**

Nos permite saber cuántos usuarios o peticiones puede soportar el programa, con esta prueba determinamos si el sistema es estable.

- **Pruebas de volumen**

Consiste en probar el funcionamiento del programa con ciertos volúmenes de datos.

- **Pruebas de usabilidad**

Se centran en validar que tan fácil se utiliza el programa en cuanto a facilidad de aprendizaje, eficiencia, memorización, errores y satisfacción.

- **Pruebas de robustez**

Son las encargadas de verificar la capacidad del programa para soportar entradas o instrucciones incorrectas. Una de sus características principales es implementar mecanismos de “software” de detección de errores en el “hardware”.

Pruebas de rendimiento

Las pruebas de rendimiento son un tipo de pruebas no funcionales realizadas con el propósito de determinar la estabilidad, escalabilidad y velocidad de una funcionalidad específica en el producto de “software”. El objetivo de estas pruebas es evaluar el comportamiento de la aplicación considerando diferentes parámetros de referencia del sistema y de la red, como la capacidad de la CPU, la velocidad de renderizado o carga de páginas, el manejo del tráfico, la concurrencia, el consumo de recursos del servidor, entre otros. Dentro de esta categoría, podemos distinguir las pruebas de carga y las pruebas de estrés.

Según IBM (2021) estas pruebas sirven para evaluar lo rápido que el programa realiza una tarea en condiciones específicas de trabajo, también evalúa la escalabilidad, fiabilidad y uso de los recursos.

El mismo autor expresa que los objetivos de estas pruebas son:

- **Identificar y localizar problemas de rendimiento.**

Es permitir encontrar un fallo en el rendimiento, y también ayudar a localizar en que parte está ese problema para poder solucionarlo.

- **Verificar el cumplimiento de los SLA (Acuerdos de Nivel de Servicio).**

Se utiliza para medir los tiempos de respuesta del “software” en condiciones y tiempos específicos.

- **Localizar cuellos de botella.**

Ayudar a detallar en qué lugar se generan estos cuellos de botellas si se debe a problemas de “hardware” como CPU, memoria, o es el ancho de banda.

Ventajas y desventajas de los tipos de pruebas.

Aplicar los distintos tipos de pruebas puede asegurar que el “software” funcione conforme a lo esperado en todo momento, garantizando así su calidad. Por lo tanto, es esencial, para su aplicación, identificar las ventajas y desventajas de cada uno, con el fin de determinar las diferencias entre ellos. Podemos observar las ventajas de estos tipos de pruebas en el siguiente recurso.

Pruebas de rendimiento

- Evalúa la velocidad y escalabilidad de la aplicación.
- Determina los cuellos de botella para las mejoras de rendimiento.
- Detecta errores que se pasan por alto en las pruebas funcionales.
- Mejoras de características y optimización del sistema.
- Garantiza la fiabilidad de la aplicación web bajo una gran carga.

Pruebas funcionales

- Se asegura de que la aplicación esté libre de defectos.
- Garantiza el comportamiento requerido de las funcionalidades.

- Verifica que la arquitectura sea la correcta con la seguridad requerida.
- Mejora las funcionalidades generales y la calidad.

Pruebas de integración

- Asegura de que en la aplicación todos los módulos se encuentren bien integrados y funcionen juntos según lo requerido.
- Detecta conflictos y problemas interconectados para solucionarlos antes de crear un problema más grande.
- Verifica la fiabilidad, estabilidad y funcionalidad entre módulos diferentes.

Pruebas unitarias

- Detecta errores en las nuevas funcionalidades desarrolladas de manera temprana.
- Detecta problemas desde el inicio, esto permite reducir los costos de las pruebas.
- Con una mejor refactorización del código, mejora la calidad del código.
- Permite una buena documentación y simplifica la integración.

Desventajas de los tipos de pruebas

Todos los tipos de pruebas mejoran la experiencia del usuario, las funcionalidades y garantizan la calidad del “software”. Esto significa que, más que desventajas, lo que encontramos son ventajas en su aplicación. Sin embargo, el tiempo y el costo asociados a la realización de pruebas de “software” podrían considerarse como desventajas. Llevar a cabo pruebas demanda recursos y esfuerzos, y existe el

riesgo de obtener resultados inexactos. No obstante, omitir las pruebas en las aplicaciones podría acarrear problemas serios al producto.

Documentos de caso de pruebas

Los productos derivados del proceso de desarrollo de “software” son conocidos como artefactos; entre ellos se incluyen el código fuente, los defectos identificados, el plan de pruebas, los casos de prueba y los “scripts” de prueba. El aseguramiento de la calidad del “software” (SQA, por sus siglas en inglés) involucra artefactos específicos que se generan en las distintas fases del proceso de SQA, los cuales se detallan a continuación:

- **Planificación**

En esta etapa se realiza el plan de pruebas como documento que describe un enfoque sistemático para probar un sistema, así como el informe de resultados de pruebas que organiza y presenta un análisis resumido de los resultados de las pruebas para su auditoría y evaluación.

- **Construcción**

En esta etapa se encuentran los casos y “script” de prueba como conjunto de condiciones con las cuales un “tester” debe determinar si un sistema funciona según lo requerido.

- **Ejecución**

En esta etapa se obtiene el resultado de la ejecución de pruebas, recopilando la información relacionada con los resultados de la ejecución de cada “script” de prueba.

En el diseño de las pruebas de “software”, es crucial identificar y describir los casos de prueba. Para documentar un caso de prueba, es necesario comprender que este incluye un conjunto de variables o condiciones bajo las cuales un probador determina si un sistema funciona de acuerdo con lo esperado. Asimismo, es importante destacar que la documentación de un caso de prueba se convierte en una tarea manejable si se dispone de la información necesaria para su elaboración. Al momento de probar un “software”, la documentación de casos de prueba es de gran utilidad, ya que se transforma en una herramienta esencial en el proceso de registro, seguimiento y control.

A continuación, se nombran los elementos principales que debe tener un caso de prueba.

Tabla 1. Elementos de un caso de prueba

Elemento	Caso de prueba
Identificador	Puede ser alfanumérico o numérico.
Nombre	Nombre del caso de prueba de manera concisa.
Descripción	Objetivo del caso de prueba, también describe que probara, en ciertas ocasiones se incluye el ambiente de pruebas.
Número de orden Ejecución	Orden en la cual se ejecuta el caso de prueba, en la situación de que se tengan múltiples casos de prueba.
Requerimiento asociado	Si se plantea un caso de prueba se debe saber a qué requerimiento va asociado para mantener la trazabilidad.
Precondición	Estado en la cual se debe encontrar el sistema antes de comenzar la prueba.

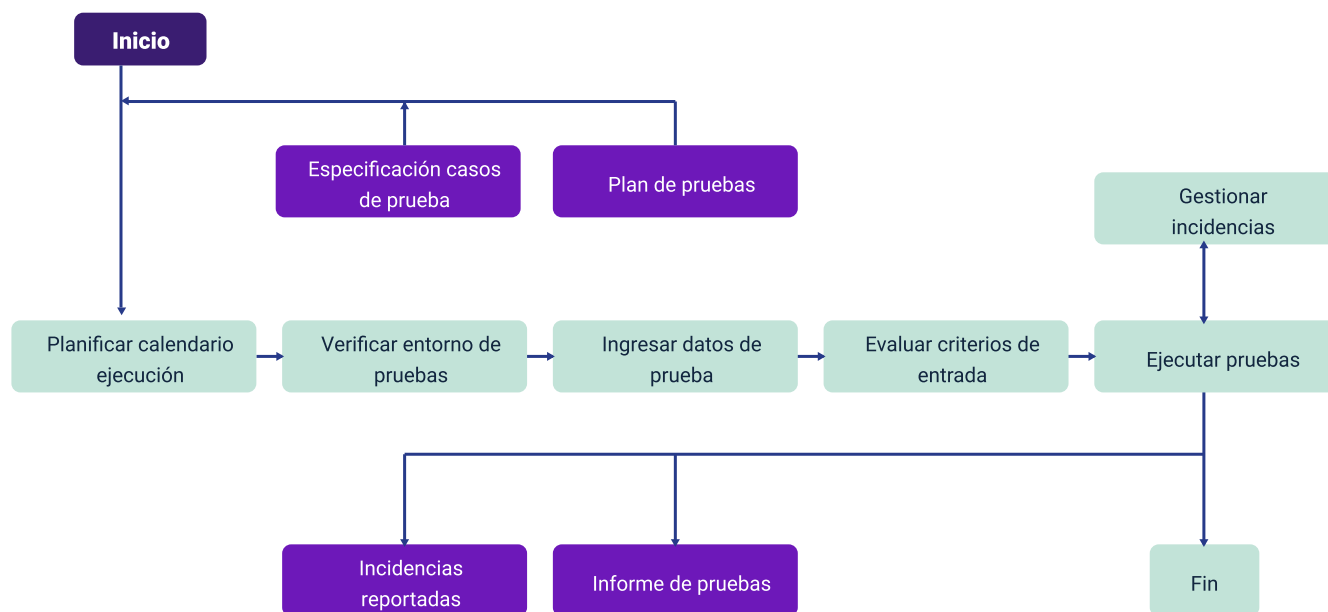
Elemento	Caso de prueba
Postcondición	El estado en que debe encontrarse el sistema luego de ejecutar la prueba.
Resultado esperado	Objetivo que debe ser alcanzado posterior a ejecutarse la prueba.

Identificar, describir y documentar casos de prueba, facilita el trabajo cuando es necesario evaluar y comparar las modificaciones a los requerimientos inicialmente definidos por el cliente a lo largo del tiempo. Este seguimiento permite verificar que los resultados obtenidos estén acordes con lo requerido o solicitado, logrando así la satisfacción del cliente. Esto convierte a los casos de prueba en un aliado principal al momento de garantizar la calidad del “software”.

Ejecución de las pruebas

La ejecución de pruebas comienza con la creación de los datos de prueba necesarios para llevar a cabo los casos de prueba establecidos. Esta ejecución puede ser tanto automatizada como manual; en ambos casos, es fundamental registrar y documentar los fallos detectados en el sistema, así como sus correcciones posteriores. En la siguiente figura se ilustra el flujo de actividades relacionadas con la ejecución de pruebas.

Figura 2. Diagrama Actividades



El diagrama de actividades mencionado anteriormente, detalla el flujo que debe considerarse al momento de ejecutar las pruebas de “software”. Por esta razón, a continuación se describen cada una de las actividades implicadas.

Planificar calendario detallado de ejecución:

- Definir el calendario que especifique la ejecución de los casos de prueba, teniendo en cuenta el plan y los casos de prueba.

Verificar entorno de pruebas:

- Verificar que el ambiente de pruebas tenga un funcionamiento correcto.

Integrar los datos de pruebas:

- Realizar una recolección de los datos de prueba a utilizar en la ejecución de pruebas.

Evaluar criterios de entrada:

- Analizar los criterios de entrada contenidos en el plan de pruebas, de tal forma que se pueda verificar la ejecución de las pruebas.

Ejecutar las pruebas:

- Se incluyen las siguientes actividades:
- Ejecutar casos y “script” de prueba.
- Analizar y registrar los resultados.

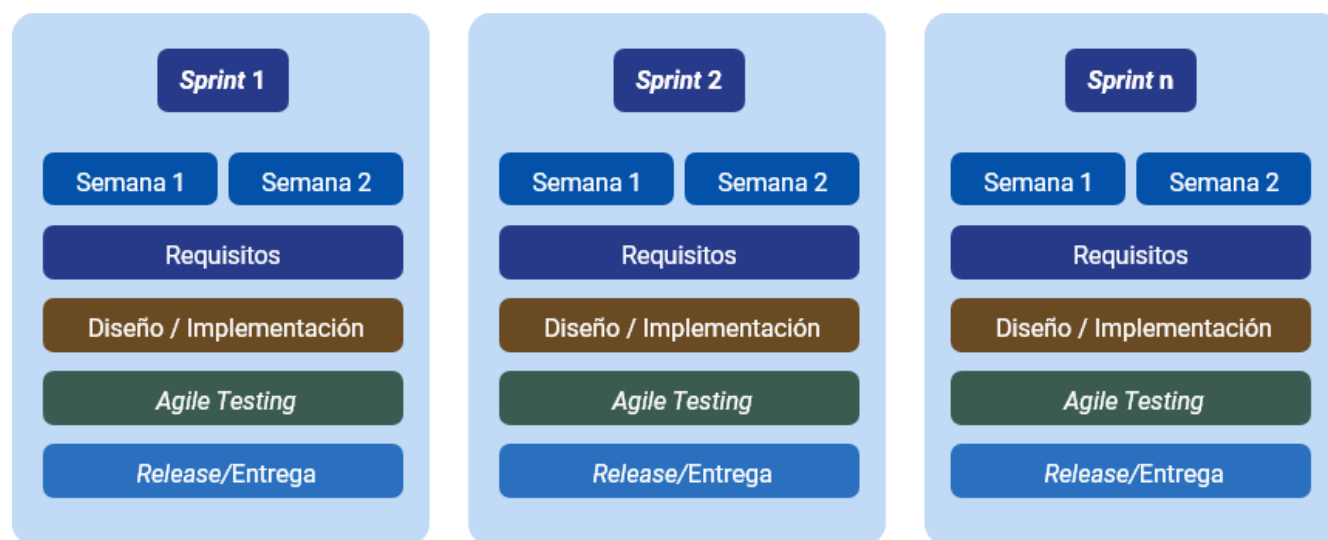
1.2. “Agile Testing”

Las metodologías ágiles proporcionan un conjunto de directrices y principios diseñados para agilizar y priorizar la entrega de productos sobre la documentación exhaustiva, simplificando los procesos e involucrando al cliente final desde las etapas iniciales del proyecto. Es importante destacar que una metodología ágil se compone de iteraciones o “sprints”, cada uno de duración relativamente corta, generalmente de dos a cuatro semanas. Al finalizar cada iteración, se disponen de funcionalidades del “software” que pueden ser entregadas y utilizadas por el cliente.

Recordando las metodologías ágiles, se puede especificar que las pruebas ágiles (“Agile Testing”) son una práctica que se lleva a cabo en cada iteración. Existen dos tipos de planificaciones: la planificación de lanzamiento/entrega, que se refiere a las entregas frecuentes al cliente, es decir, la puesta en producción de nuevas versiones del producto; y la planificación de “sprint”/iteración, que indica que un proyecto se ejecuta en bloques temporales cortos y fijos. Sin embargo, estas planificaciones pueden coincidir en un mismo momento, es decir, puede haber una sola planificación que

incluya actividades tanto de lanzamiento como de “sprint”, tal como se muestra en la siguiente figura:

Figura 3. Actividades “Sprints Release”



Por otro lado, es importante considerar que la metodología de desarrollo de “software” tradicional, conocida como cascada, emplea diferentes tipos de pruebas de “software” que evalúan tanto los requerimientos funcionales como los no funcionales de manera secuencial. Sin embargo, en las metodologías ágiles, y específicamente en las pruebas ágiles, también se contemplan diversos tipos de pruebas. La diferencia radica en que en las metodologías ágiles se utilizan “sprints” o iteraciones cortas y hay una interacción continua entre los equipos de desarrollo, diseño y pruebas.

Las pruebas ágiles se basan en principios que se alinean con el desarrollo ágil de “software”. Además, existen ciertos cuadrantes que se deben tener en cuenta al momento de planificarlas.

A continuación, se describen tanto los principios como los cuadrantes:

Principios de las pruebas ágiles (“Agile Testing”)

El “Agile Testing” se fundamenta en los siguientes principios, aplicables a un desarrollo de “software” exitoso:

- **Integración continua**

Los equipos de diseño y desarrollo ejecutan pruebas continuamente, puesto que es una forma de asegurar el avance continuo del producto.

- **Entrega continua**

Se soporta en la integración continua, en este punto los desarrolladores realizan un control y la cantidad de veces que se realiza la entrega continua.

- **Elaborar menos documentación**

Los participantes de los equipos en metodologías ágiles pueden crear listas y tomar notas para centrarse en probar el “software”, lo que permite no enfocarse en detalles secundarios que tomarían mucho tiempo.

- **Responder con velocidad a la retroalimentación**

Los involucrados en el negocio del producto están estrechamente conectados con las interacciones del producto, además el tiempo de respuesta se reduce debido a la realimentación continua en cada parte de la interacción.

- **Proporcionar retroalimentación constante**

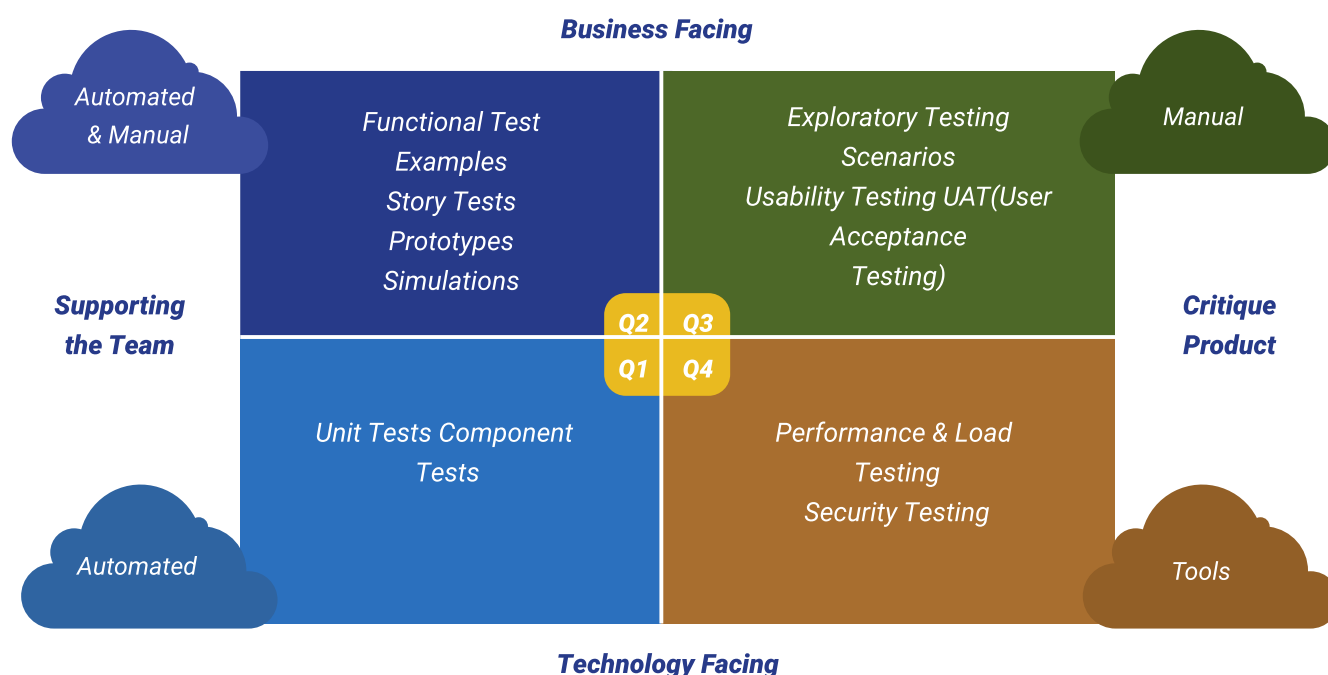
En las pruebas ágiles la retroalimentación es continua, de modo que el diseño del producto logre los propósitos del negocio requeridos.

Cuadrantes de las pruebas ágiles (“Agile Testing”)

Para planificar las pruebas ágiles, podemos basarnos en los cuadrantes de “Agile Testing”, que proporcionan una biblioteca de tipos de pruebas aplicables para satisfacer los requerimientos. Estos cuatro cuadrantes ofrecen una clasificación útil para la planificación de las pruebas ágiles, asegurando que se consideren los métodos y recursos necesarios para lograr productos de “software” de alta calidad.

Las pruebas definidas por los cuadrantes se centran en brindar apoyo al equipo de desarrollo a lo largo del proceso de creación del producto, ya que guían el desarrollo de la funcionalidad. Posteriormente, facilitan la introducción de nuevo código y la refactorización, asegurando que no se produzcan resultados inesperados en el comportamiento del sistema. En la siguiente figura, se pueden observar los cuadrantes de las pruebas ágiles, los cuales se organizan en torno a dos ejes principales: las pruebas que apoyan al equipo y las tecnologías que facilitan el proceso.

Figura 4. Cuadrantes pruebas ágiles



A continuación, se realizará una exploración de los diferentes cuadrantes expuestos:

Pruebas de cara al negocio (“Business Facing”)

Son pruebas del Q2 y Q3, pruebas funcionales y pruebas UAT, se enfoca en realizar pruebas manuales y criticando al producto con las pruebas de aceptación.

Pruebas de cara a la tecnología (“Technology Facing”)

Pueden ser automatizadas y utilizando herramientas, incluye el Q1 y Q4.

Pruebas soportando la programación o soporte del equipo (“Supporting the Team”)

Comprenden pruebas automatizadas y manuales del cuadrante Q2 y Q1 donde se realizan pruebas unitarias y de componentes y pruebas funcionales con prototipos y simuladores.

Pruebas de críticas al producto (“Critique the Product”)

Estas pruebas pueden tener un carácter positivo, puesto que a partir de ellas se pueden sugerir o realizar mejoras. Cuando se ejecutan estas pruebas, la idea es simular lo más que se pueda un ambiente real en el cual serán ejecutadas, así mismo estas pruebas frecuentemente son realizadas por los usuarios finales como pruebas de aceptación (UAT).

El primer cuadrante (Q1)

El primer cuadrante (Q1) contiene los siguientes tipos de pruebas:

- Pruebas unitarias.
- Pruebas de componente.

El segundo cuadrante (Q2)

El segundo cuadrante (Q2) definen las funcionalidades que el cliente solicita:

- Pruebas funcionales.
- Pruebas de historias.
- Pruebas de Prototipos y simulaciones.

El tercer cuadrante (Q3)

El tercer cuadrante (Q3) contiene los siguientes tipos de pruebas:

- Pruebas exploratorias.
- Pruebas de usabilidad.
- Pruebas de escenarios.
- Pruebas de aceptación de usuario.

El cuarto cuadrante (Q4)

El cuarto cuadrante (Q4) contiene pruebas técnicas. Relacionadas con los requerimientos no funcionales, por lo que validan el cumplimiento de estos, analizando la seguridad, el desempeño y la robustez.

- Pruebas de rendimiento y de estrés.
- Pruebas de seguridad.
- Pruebas de robustez.

Ahora, se describen algunas recomendaciones para tener en cuenta al momento de trabajar con pruebas de “software” ágiles (“Agil Testing”).

Gestión de pruebas por “sprint”

- Integrar pruebas unitarias, funcionales y de carga en la planificación desde el principio.

- Identificar y rectificar problemas de forma temprana, antes que se conviertan en problemas complejos que puedan influir el proyecto.

Integración de analistas en desarrollo

- Los “testers” tienen el conocimiento para asegurar que se entregue un producto de calidad y que cumpla los requerimientos del cliente.
- Al integrar a los “testers” al equipo de desarrollo se logra compartir el conocimiento.

Revisión de pruebas unitarias por “testers”

- Los desarrolladores suelen deliberar de forma distinta que los “testers”.
- Para mejorar las pruebas unitarias, se debe permitir que los analistas de pruebas revisen las pruebas unitarias y realicen sugerencias.

Pruebas de carga diarias

- Las pruebas de estrés son cada vez más críticas para el éxito, considerando que el “software” de hoy se diseña para implementaciones masivas que puedan escalar.
- Deben incorporarse pruebas de estrés en todas las construcciones de “software” a diario.

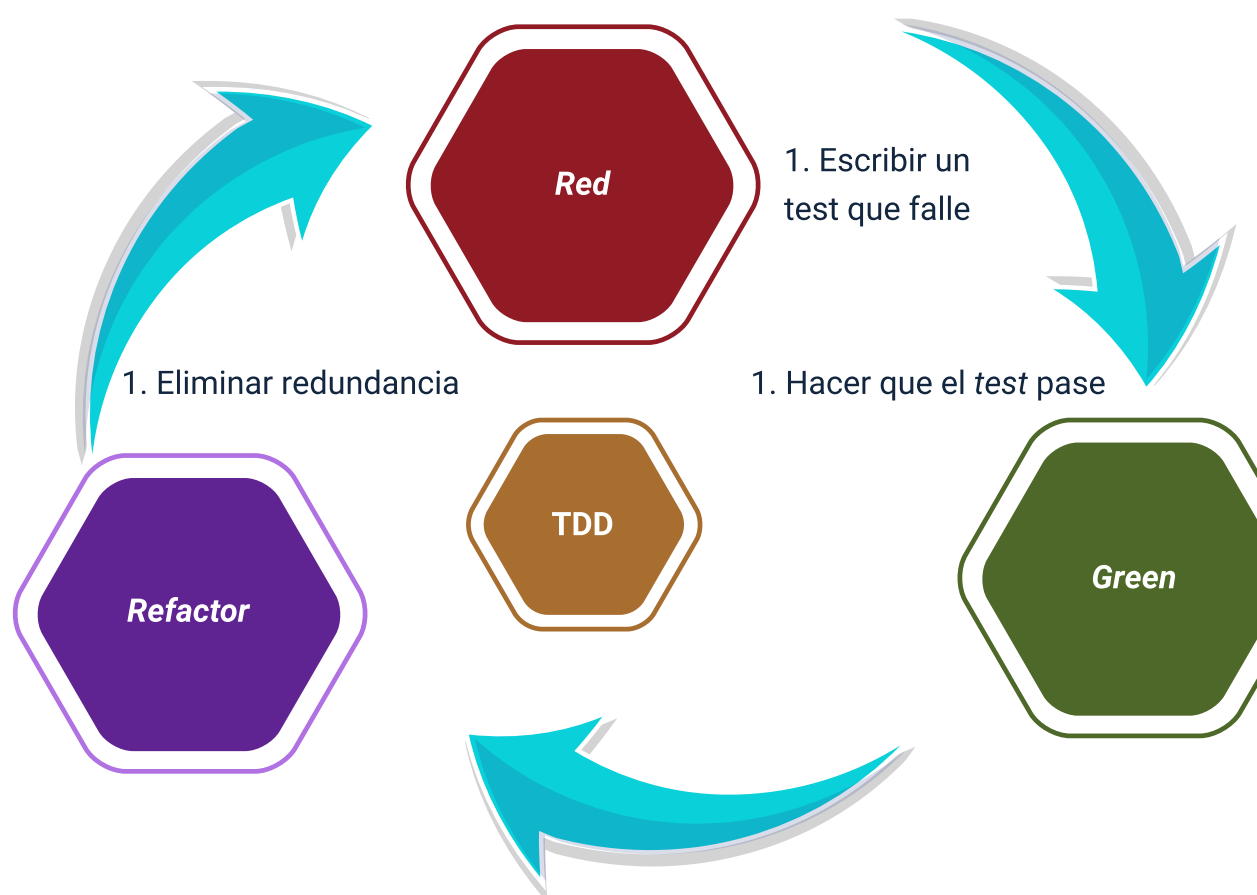
Automatización de pruebas de “software”

- Hacer uso de herramientas de automatización de pruebas. De esta forma será más eficiente y eliminará inexactitudes.
- La automatización de pruebas permitirá ejecutar con mayor eficiencia las pruebas de regresión cada vez que se incorpore actualizaciones.

“Test Driven Development” (TDD)

Es una técnica de desarrollo que permite crear pruebas antes del código de producción, es decir, antes de la implementación final. La esencia del TDD (Desarrollo Guiado por Pruebas) consiste en elaborar una prueba antes de proceder con la implementación, de tal manera que el código desarrollado permita que la prueba se ejecute satisfactoriamente. Además, esta técnica se complementa con la refactorización. Generalmente, se tienen requerimientos que se expresan en historias de usuario como criterios de aceptación o pueden ser descritos como casos de uso. Posteriormente, estos requerimientos se traducen en pruebas automatizadas que, a su vez, orientan el desarrollo del código siguiendo buenas prácticas.

Figura 5. Flujo del TDD



El Desarrollo Guiado por Pruebas (TDD, por sus siglas en inglés) se fundamenta en tres pasos esenciales:

Red: este paso se denomina así porque implica la creación de una prueba que inicialmente falla, lo cual se refleja con letras rojas en la consola al ejecutarla.

Green: el objetivo de este paso es modificar el código para que la prueba anteriormente fallida ahora pase exitosamente. Se llama Green porque, al pasar la prueba, los resultados suelen mostrarse en letras verdes en la consola.

Refactor: una vez que la prueba ha sido superada, se procede a revisar y mejorar el código para optimizarlo, limpiarlo o aplicar mejores prácticas, sin alterar su funcionalidad.

En la siguiente tabla podemos verificar cada uno de los pasos guiados por las pruebas TDD donde se explica cada uno de los pasos con sus ejemplos.

Tabla 2. Flujo del TDD

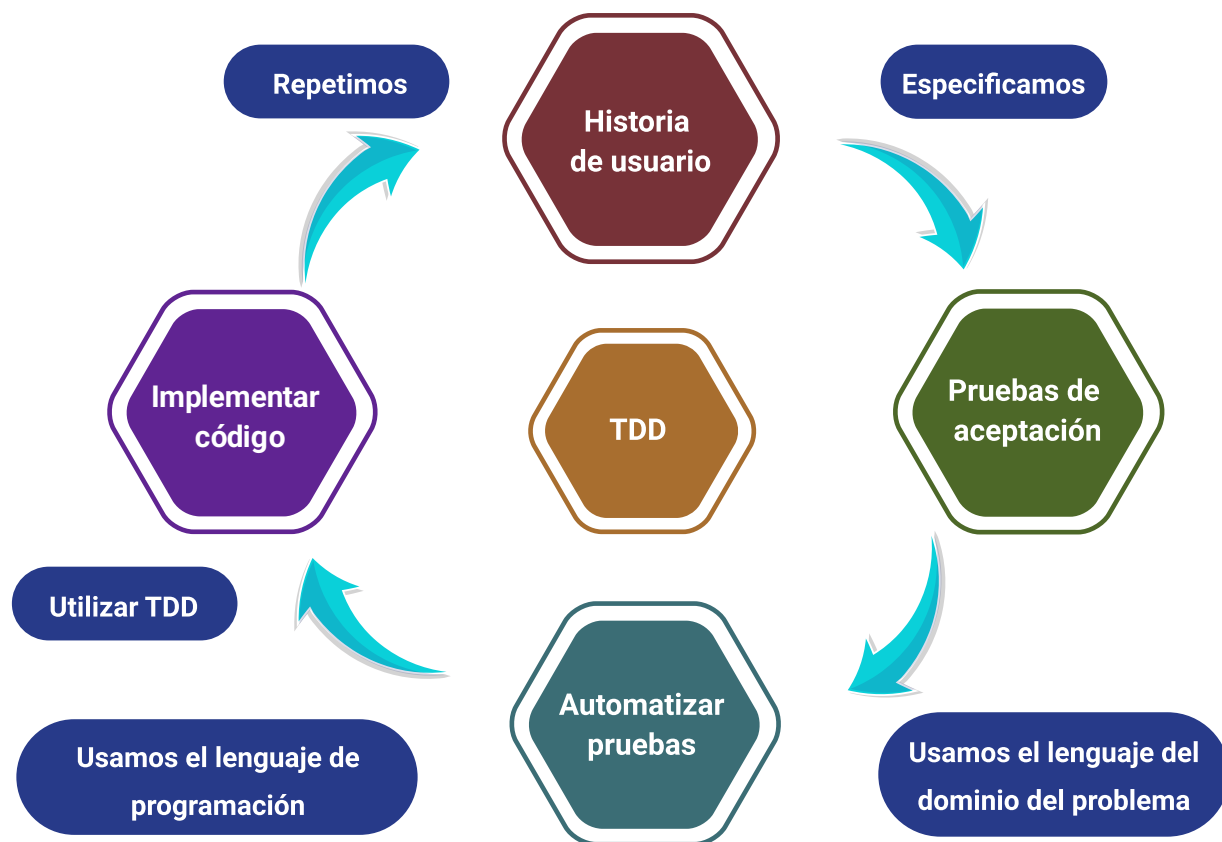
Paso	Pasos	Ejemplo
Red	a) Lo primero es que el cliente escriba una historia de usuario.	Supóngase que el cliente solicita que se desarrolle una aplicación con operaciones matemáticas sencillas que sume y reste.
Red	b) Los criterios de aceptación de esta historia se escriben con el cliente, deben estar bien detallados.	Se define con el cliente que el criterio de aceptación está relacionado con: si se introduce en la aplicación dos números y se suma o resta, entonces la aplicación visualiza el resultado en pantalla.
Red	c) Se debe seleccionar el criterio de aceptación más sencillo y se	De acuerdo a este criterio, se empieza a especificar o definir el funcionamiento del algoritmo para la suma y la resta y se luego se convierte el criterio de aceptación en una prueba específica, por ejemplo, un algoritmo que si se introduce los números 3 y 8 devuelve 11. Ejemplo de

Paso	Pasos	Ejemplo
	traduce en una prueba unitaria.	cómo sería la clase para calcular: <code>public void testSuma() { assertEquals(8, Operaciones.suma(3,8)); }</code>
Red	d) Se verifica que la prueba falla.	RED 4. Se verifica que la prueba falla. Si se intenta pasar este “test” resultara en error, debido a que la clase Operaciones todavía no existe.
Green	e) Se escribe el código que hace pasar la prueba.	Posteriormente el siguiente paso es escribir el código de la clase, en este momento ya se sabe cómo se va a comportar. <code>public class Operaciones { public static int suma (int a, int b) { int c = a + b; return c; } }</code>
Green	f) Se ejecutan todas las pruebas automatizadas.	Posteriormente se ejecuta la prueba y en este momento se tiene el código funcionando con la prueba pasada.
Refactor	g) Se ejecutan todas las pruebas automatizadas.	Cuando esté todo funcionando, se pasa a refactorizar y a eliminar código duplicado.
Refactor	h) Se refactoriza y se limpia el código.	Es necesario pasar todos los “test” después de refactorizar.

“Acceptance Test Driven Development” (ATDD)

El Desarrollo Guiado por Pruebas de Aceptación (ATDD, por sus siglas en inglés) es una metodología de desarrollo que enfatiza la comunicación entre los clientes empresariales, los desarrolladores y los “tester”. Antes de que los desarrolladores empiecen a escribir código, ATDD incorpora las pruebas de aceptación, poniendo especial énfasis en la redacción de estas pruebas. El proceso de ATDD se desarrolla a través de los siguientes pasos:

Figura 6. ATDD



- **Seleccionar la historia de usuario:** en este punto, se asume que se ha realizado una priorización para determinar cuál es la próxima historia de usuario a abordar.
- **Escribir la prueba de aceptación:** en esta etapa, todos los involucrados deben participar de manera colaborativa para definir las pruebas que validen la correcta implementación de la historia.
- **Implementar la historia de usuario - Automatizar las pruebas:** existen varias maneras de llevar a cabo este paso, pero lo crucial es que, a partir de las pruebas de aceptación definidas anteriormente, se obtengan pruebas ejecutables. Estas pruebas permitirán verificar el progreso de la implementación de la funcionalidad.

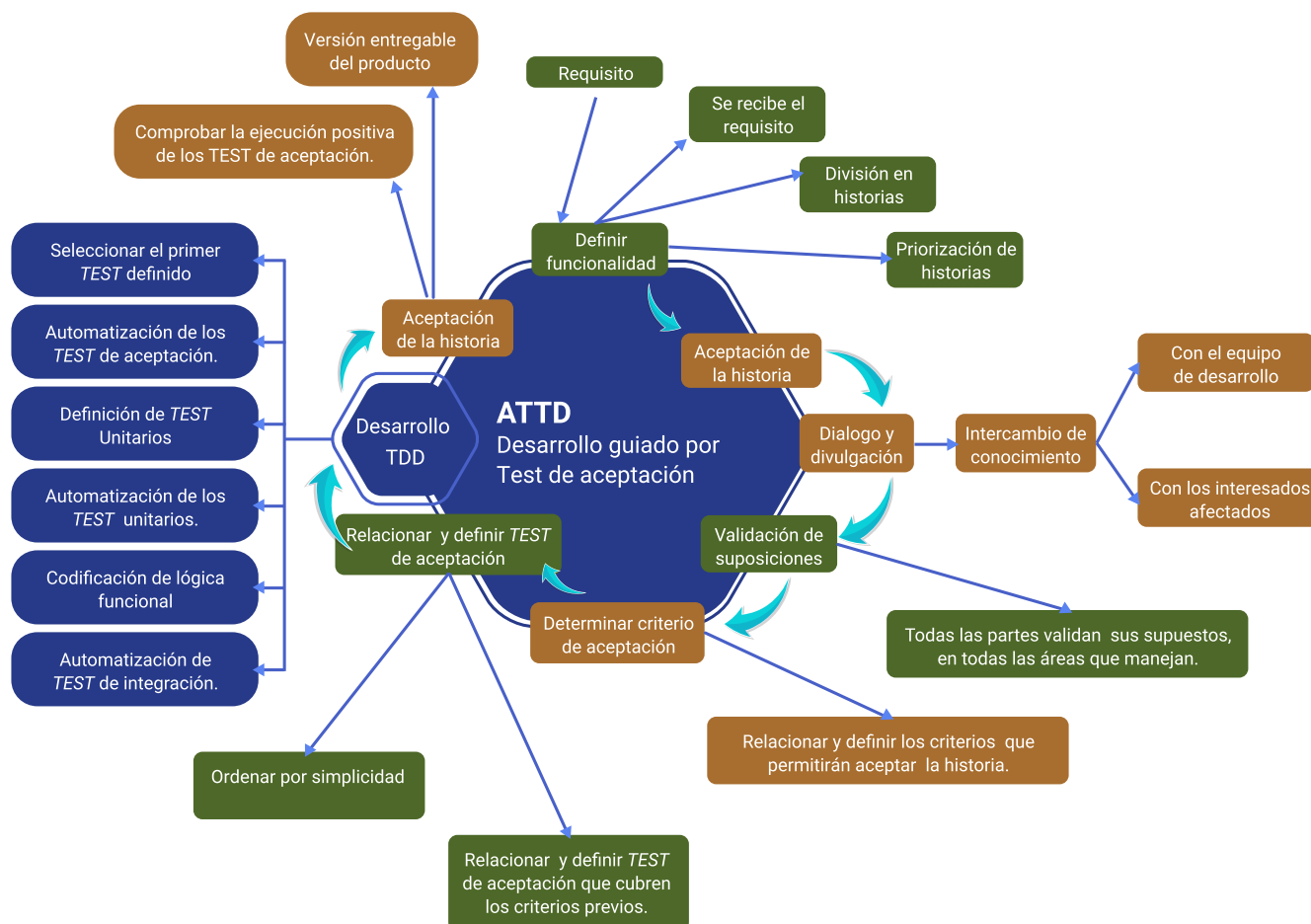
- **Implementar el código:** aunque hay diversas formas de realizar este paso, una de ellas es mediante el uso de TDD (Desarrollo Guiado por Pruebas), que asegura que el código cumpla con los requisitos establecidos en las pruebas de aceptación.
- **Realizar pequeños cambios/Refactorización:** este paso es fundamental tanto en ATDD como en TDD. Se trata de una práctica iterativa que concluye cuando los requerimientos del cliente han sido plenamente satisfechos, optimizando y limpiando el código sin alterar su funcionalidad.

De acuerdo con lo expuesto anteriormente, se inicia desde un desarrollo al cual los programadores han implementado TDD. Si este proceso se ha llevado a cabo de manera adecuada, deberían haber logrado un código limpio y refactorizado, que además ha sido validado mediante pruebas unitarias. Así, el código resultante es de alta calidad; sin embargo, surge la pregunta: ¿ocurre lo mismo con la aplicación en su totalidad?

ATDD no constituye simplemente una técnica de desarrollo o una buena práctica, sino que representa un enfoque más amplio. Se trata de una metodología de trabajo diseñada para el desarrollo de “software”, la cual tiene como objetivo verificar si los procedimientos adoptados durante el desarrollo son los más adecuados para alcanzar los objetivos deseados. Mientras que TDD se enfoca únicamente en confirmar que lo desarrollado esté codificado de forma correcta y libre de errores, lo cual representa una visión limitada centrada exclusivamente en la calidad del código y no en la aplicación como un todo.

Al igual que ocurre en TDD, ATDD consta de una serie de pasos secuenciales que se presentan en la siguiente figura:

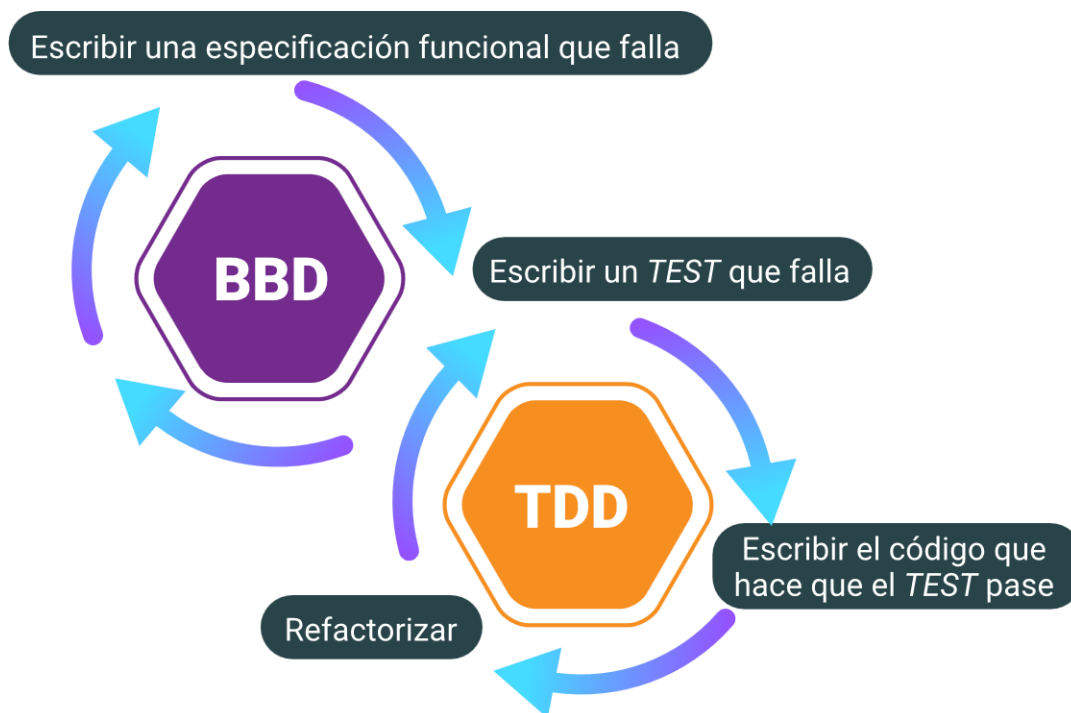
Figura 7. ATDD



“Behaviour Driven Development” (BDD)

El Desarrollo Dirigido por Comportamiento (BDD, por sus siglas en inglés) fue propuesto inicialmente por Dan North como una síntesis y refinamiento de prácticas en ingeniería de “software”, destinadas a asistir a los equipos en la generación y entrega rápida de “software” de mayor calidad. El proceso de BDD es similar al de TDD y sigue los siguientes pasos:

Figura 8. Imagen BDD



- Escribir un escenario.
- Ejecutar el escenario que falla.
- Escribir la prueba que corresponde a las especificaciones del escenario.
- Escribir el código más simple para pasar la prueba y el escenario.
- Refactorizar para eliminar la duplicación.

BDD, basado en la técnica de TDD, aborda ciertas limitaciones de esta última. Un problema fundamental de TDD surge cuando el “software” supera correctamente una prueba pero no cumple con la funcionalidad esperada. Pasar una prueba no garantiza necesariamente que el “software” realice lo que el cliente anticipa.

Ejemplo:

- Inicialmente, se define la característica o funcionalidad que se desea implementar, basándose en una historia de usuario.

Característica:

- Un usuario desea acceder a una plataforma web universitaria para descargar sus apuntes.

A continuación, se presentan dos escenarios derivados de la historia de usuario especificada

El desarrollo dirigido por comportamiento (BDD)

El objetivo de BDD es que las historias de usuario dirijan el desarrollo del proyecto “software”. Además, BDD permite comprobar que el “software” implementado cumple con la funcionalidad requerida.

- Un ejemplo muy sencillo es el siguiente: primero definimos la característica o funcionalidad a implementar en la historia de usuario.
- Característica: un usuario quiere acceder a una plataforma web universitaria para descargar sus apuntes.

Primer escenario

- El usuario entra correctamente.
- Tiene un email y una contraseña.
- Verifica que el nombre de usuario existe.
- Verifica que la contraseña es correcta.

- El usuario puede acceder a su cuenta, su información y puede descargar sus apuntes.

Segundo escenario

- El usuario no recuerda la contraseña.
- Tiene un email.
- Cuando el usuario hace clic en: "No recuerdo la contraseña".
- Luego el usuario inserta su email.
- Posteriormente el email existe.
- Se envía un correo de verificación.
- Y el usuario debe validar el enlace.
- Y el usuario debe insertar nueva contraseña.
- El usuario tiene que volver a entrar con su nueva contraseña.
- Se ejecuta primer escenario.

“Frameworks” de BDD por lenguaje

Una vez definida, BDD utiliza herramientas para convertir esta historia de usuario en código. Para ello se utilizan "contenedores" o “frameworks”; hay tantos contenedores como lenguajes de programación. Normalmente están disponibles para la mayoría de los lenguajes de programación. A continuación, se muestran algunos:

Figura 9. “Frameworks” de BDD por lenguaje

Lenguaje	Framework
Javascript	Concordion, JDave y Easyb
Ruby	Jasmine
Phyton	Cucumber
PHP	BeHave
Java	Behat y Kanlan

“Testing” exploratorio

Las pruebas exploratorias han captado significativamente la atención dentro de la comunidad de pruebas de “software”. Este método de pruebas se caracteriza por una interacción simultánea entre el testeo, el aprendizaje y el diseño de nuevas pruebas.

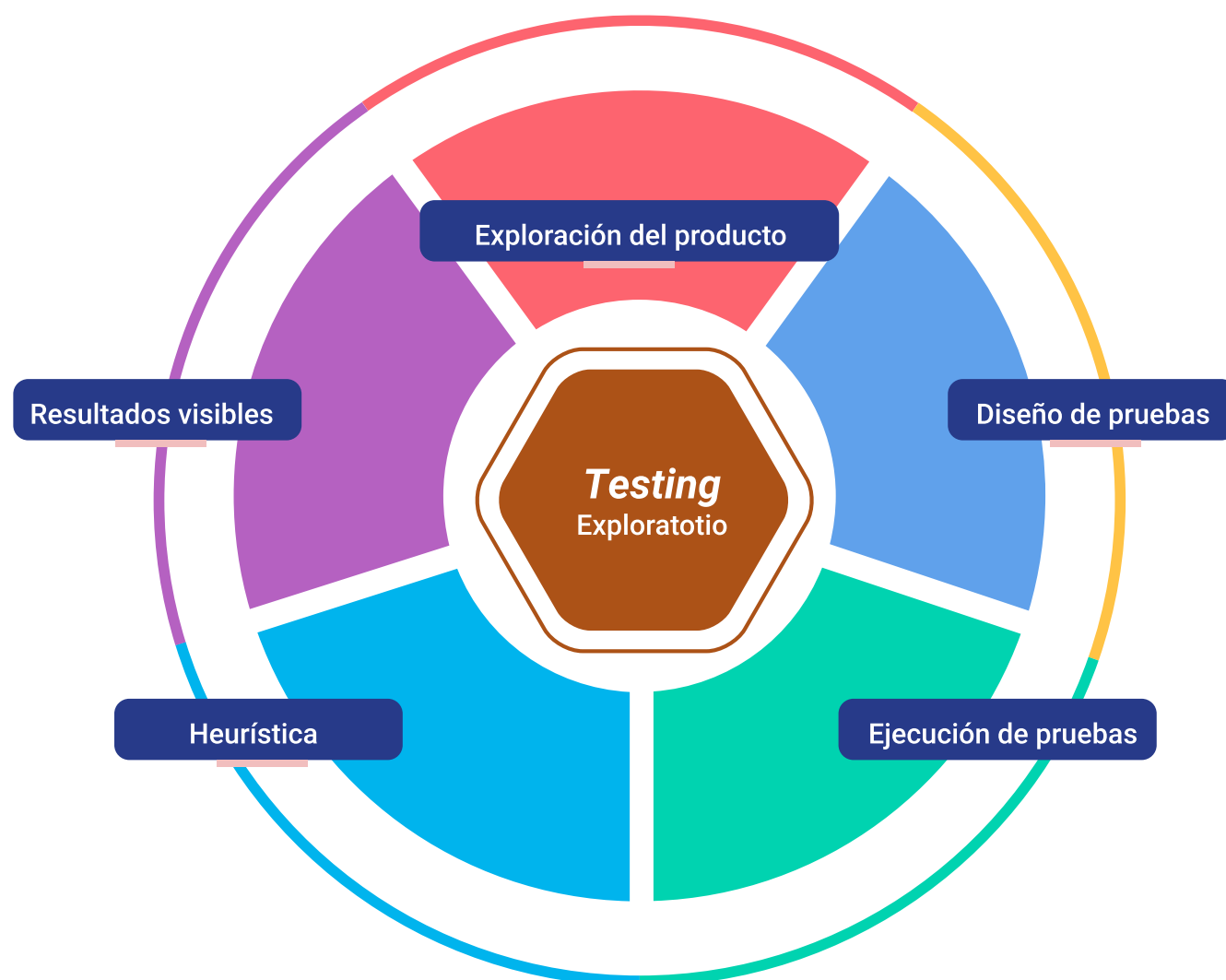
En la práctica, casi todos los “testers” realizan pruebas exploratorias, ya sea de manera consciente o no. Un ejemplo claro de esto son las pruebas de regresión de errores. Para confirmar la corrección de un error, el “tester” podría iniciar siguiendo los pasos detallados en el informe de errores. Sin embargo, una vez que el programa supera esta prueba inicial, es probable que el “tester” lleve a cabo pruebas adicionales para asegurarse de que el error ha sido completamente resuelto.

El “testing” exploratorio integra el diseño, la ejecución y el aprendizaje sobre la aplicación en prueba, fomentando un proceso de exploración y comprensión profunda

del producto. A diferencia de otros enfoques, no se basa en un guion o “script” de pruebas preestablecido, lo que lo hace particularmente valioso en metodologías ágiles.

Después de observar el proceso exploratorio del “software”, se han identificado las siguientes pruebas exploratorias:

Figura 10. Pruebas exploratorias



Exploración del producto: para conocer a fondo cómo cumplir con los requisitos hay que registrar los objetivos, las funciones, los tipos de datos que se procesan y las zonas de inestabilidad del producto.

Diseño de pruebas: crear diferentes estrategias para observar y evaluar por completo el producto.

Ejecución de pruebas: explorar el producto para poder formular una hipótesis de cómo funciona y cuáles pueden ser sus puntos débiles.

Heurística: reglas generales que ayudarán a cómo probar correctamente el producto.

Resultados visibles: cuando se finalicen las pruebas exploratorias, el “tester” debe ser capaz de explicar cualquier aspecto del programa y mostrar cómo se cumplen los requisitos indicados en el procedimiento.

Automatización de pruebas de regresión

Las pruebas de regresión garantizan que los cambios realizados en una aplicación o sistema no introduzcan errores en las funcionalidades existentes. Sin embargo, realizadas de manera manual, estas pruebas pueden ser muy consumidoras de tiempo, llegando a ser más costosas que las pruebas de las nuevas funcionalidades que se están evaluando. Además, la intervención manual incrementa la susceptibilidad a errores debido al factor humano.

Estas pruebas se realizan siguiendo flujos completos, lo que implica ejecutarlas considerando todas las opciones disponibles según la lógica de negocio. Es crucial incluir en las pruebas las partes del código que han sido modificadas, para asegurar que los cambios no afecten negativamente la interacción con el entorno en el que el “software” opera.

Es esencial seleccionar cuidadosamente los flujos a probar, optimizando el proceso para evitar fallos costosos en producción. A pesar de que las pruebas de regresión pueden ser lentas, su automatización permite que sean tanto asertivas como rápidas. Además, la disponibilidad de “scripts” de pruebas facilita que cualquier persona, sin necesidad de un equipo de “testers” especializados, pueda ejecutarlas. Esto es particularmente útil para aplicaciones que han estado en producción durante largo tiempo, ya que reduce el riesgo asociado a la falta de conocimiento sobre los flujos de trabajo por parte del equipo.

Este tipo de pruebas, ya sean lanzadas por los desarrolladores o los “testers”, son susceptibles de automatización. Para su implementación, se utilizan herramientas comunes a las pruebas unitarias y de sistemas. Sin embargo, también se dispone de:

Herramientas especializadas de código abierto diseñadas específicamente para facilitar y optimizar este proceso:

Winrunner, VTest, Canoo, AdventNet y QEngine.

Otras herramientas comerciales para aplicar este tipo de pruebas son:

HP UFT, Seapine QA Wizard Pro, IBM Rational Test Workbench, SmartBear Test Complete, SOASTA, Ranorex UI y OpKey.

Pruebas de exploratorias, usabilidad y aceptación

Las pruebas de usabilidad y las pruebas de aceptación del usuario, aunque pueden presentar semejanzas en términos de resultados y objetivos, difieren significativamente en su enfoque. Las pruebas de usabilidad se centran en identificar errores o deficiencias en el sistema que puedan afectar la experiencia del usuario, con

el objetivo de mejorar el producto. En contraste, las pruebas de aceptación del usuario buscan demostrar que el sistema cumple con los requisitos y expectativas establecidos, señalando así que el producto está listo para su lanzamiento. Por lo tanto, mientras las pruebas de usabilidad se enfocan en perfeccionar la interacción del usuario con el producto, las pruebas de aceptación del usuario se orientan a validar la completitud y adecuación funcional del sistema. Lo invitamos a explorar el siguiente video y conocer más sobre este tipo de pruebas:

Video 1. Pruebas de exploratorias, usabilidad y aceptación



[Enlace de reproducción del video](#)

Síntesis del video: Pruebas de exploratorias, usabilidad y aceptación

En el mundo del desarrollo de “software”, las pruebas juegan un papel crucial. Hoy exploraremos tres tipos fundamentales: pruebas exploratorias, de usabilidad y de aceptación.

Pruebas exploratorias

El “testing” exploratorio es un enfoque dinámico. Combina el aprendizaje, el diseño de pruebas y su ejecución de manera simultánea. Es una técnica flexible que permite a los “testers” adaptar su enfoque en tiempo real, basándose en los hallazgos obtenidos durante el proceso.

Pruebas de usabilidad

La usabilidad trasciende la estética, centrando su valor en la funcionalidad y experiencia del usuario. A través de las pruebas de usabilidad, evaluamos cómo los usuarios interactúan con el producto, enfocándonos en la facilidad de uso e intuitividad. Este método es indispensable para optimizar la experiencia del usuario antes de lanzar el producto al mercado.

Pruebas de aceptación

Las pruebas de aceptación se enfocan en el usuario final. Mediante tareas específicas, evaluamos si el sistema cumple con los requisitos y expectativas del usuario. Estas pruebas son vitales para asegurar que el “software” no solo funcione según lo previsto sino que también resuelva efectivamente las necesidades del usuario.

Las pruebas exploratorias, de usabilidad y de aceptación son esenciales en el ciclo de vida del desarrollo de “software”. Garantizan que el producto final sea robusto, eficiente y, sobre todo, que satisfaga las necesidades y expectativas de los usuarios. Integrar estas pruebas desde las primeras etapas del desarrollo asegura la entrega de soluciones de “software” de alta calidad.

La excelencia en el desarrollo de “software” se logra a través de pruebas rigurosas y continuas. Invitamos a los profesionales del sector a adoptar estos enfoques para mejorar la calidad y la satisfacción del usuario en sus productos.

Pruebas desempeño, carga y seguridad

Concluir la programación de una aplicación marca un momento de éxito; con el paso del tiempo, tener un producto terminado representa un logro significativo. Sin embargo, este hito también señala el momento crucial para probar diversos aspectos del “software”, con el fin de asegurar su correcto funcionamiento y controlar su calidad. Factores como el desempeño, la seguridad y la capacidad de carga son elementos críticos que deben evaluarse en todo producto de “software” desarrollado, los cuales se describen a continuación.

Pruebas de desempeño

Las pruebas de desempeño evalúan cómo se comporta un sistema en términos de estabilidad y rapidez de respuesta bajo ciertas condiciones, como la carga de trabajo, la concurrencia, el ancho de banda, y las características específicas de equipos o servidores, tales como la RAM y la CPU. Además, estas pruebas permiten identificar, medir, investigar, validar o verificar otros aspectos de calidad, incluyendo la seguridad, la escalabilidad y el consumo de recursos. Constituyen un estándar en la ejecución,

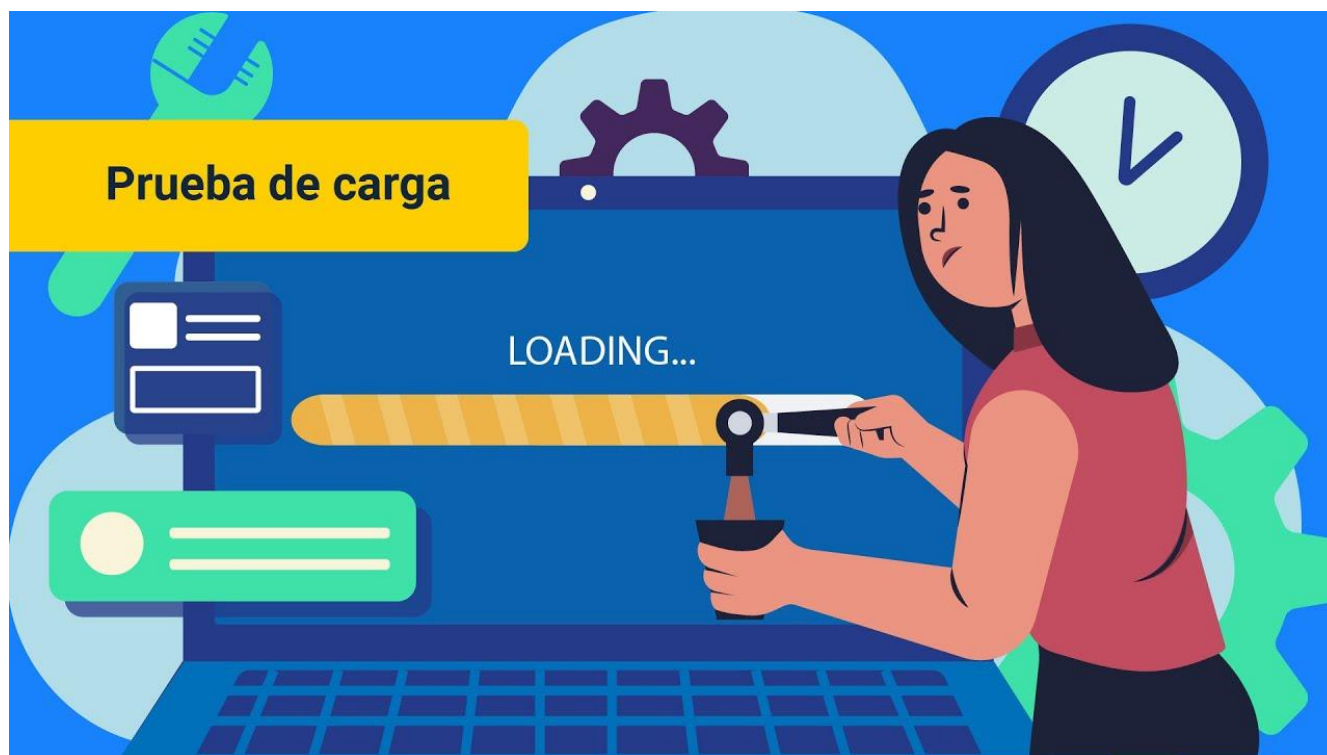
diseño y arquitectura de un producto de “software”. Entre las pruebas de desempeño más importantes se encuentra la prueba de carga, la cual se detallará a continuación.

Pruebas de carga

Las pruebas de carga se enfocan en determinar o validar las características de rendimiento de un sistema o aplicación bajo prueba al enfrentarse a distintos volúmenes de cargas de trabajo previstos durante las operaciones de producción. Estas pruebas son cruciales para comprender cómo se comportará la aplicación en condiciones reales y garantizar que pueda manejar el tráfico esperado sin degradar su rendimiento. (Meier et al., 2007).

Para ilustrar mejor el concepto de las pruebas de carga, se presenta el siguiente video tutorial:

Video 2. Prueba de carga



[Enlace de reproducción del video](#)

Síntesis del video: Prueba de carga

Video tutorial del experto donde se realiza paso a paso la descarga del JDK (Java Development Kit), desde la página de ORACLE, con la creación de reportes y su ejecución.
--

Pruebas de seguridad

Las pruebas de seguridad comprenden un conjunto de actividades ejecutadas con el objetivo de identificar fallas y vulnerabilidades, buscando minimizar el impacto de ataques, el acceso no autorizado a información sensible y la pérdida de datos. Estas pruebas son esenciales porque los productos de “software” deben garantizar constantemente la confidencialidad, disponibilidad e integridad de los datos y funcionalidades que gestionan. La pérdida o corrupción de esta información puede tener consecuencias críticas en el ámbito empresarial. Por tanto, es crucial que estas pruebas se realicen de manera continuada a lo largo de todas las fases del proyecto, asegurando la protección efectiva contra amenazas emergentes.

A continuación, se presenta el siguiente video tutorial de instalación del “software” OWASP para realizar una prueba de seguridad:

Video 3. Prueba de seguridad



[Enlace de reproducción del video](#)

Síntesis del video: Prueba de seguridad

Video tutorial del experto donde se realiza la descarga de OWASP ZAP para hacer una prueba de seguridad, con todas las indicaciones para hacer la instalación según la configuración del equipo, y finalmente hacer las pruebas de seguridad en algunas páginas de ejemplo en donde se ven reflejadas las vulnerabilidades de la página probada.

1.3. Elaboración de informe de resultados

Una vez ejecutadas las pruebas se deben analizar los resultados y los fallos detectados, teniendo en cuenta el reporte de defectos, directrices para detectarlos y el

informe de resultados de pruebas. Para ello se puede realizar lo descrito a continuación:

- a) Analizar el impacto del defecto.
- b) Investigar el problema, en qué condiciones se produce este fallo.
- c) Analizar la severidad, si es alta o baja.
- d) Variar los pasos realizados, se pueden ejecutar en orden diferente para observar si existe alguna variante en el resultado esperado.
- e) Variar opciones de configuración, si estamos probando una aplicación web se puede probar con diferentes navegadores.
- f) Determinar condiciones específicas bajo las cuales se reproduce el defecto, si estamos probando entradas de datos entonces probar con diferentes datos.

El informe de resultados incluye la detección de incidencias o errores de “software”, puesto que es aquí donde se evalúan y corrigen problemas, por lo tanto, a continuación, se describe la gestión de incidencias y su ciclo de vida.

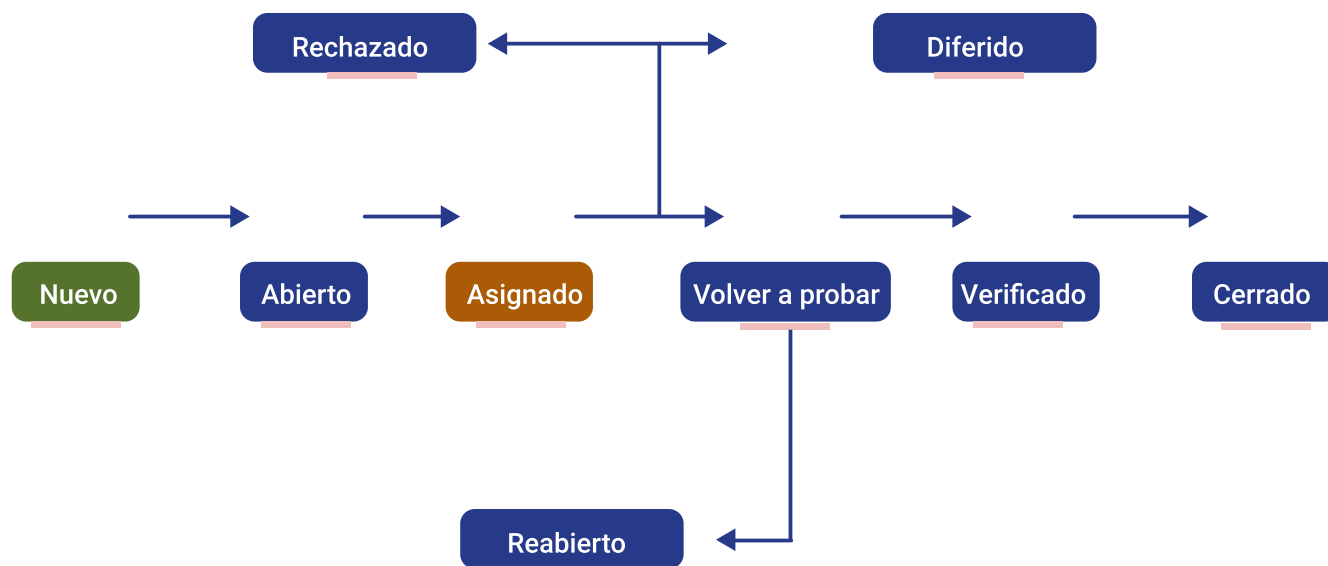
Incidencias detectadas

La gestión de incidencias desempeña un papel crucial en el proceso de aseguramiento de la calidad del “software”, ya que es en esta etapa donde se identifican y abordan los errores (“bugs”). El objetivo principal de tratar las incidencias detectadas es generar las correcciones necesarias para minimizar la posibilidad de que estos errores se repitan. Este proceso asegura que el “software” no solo cumpla con los estándares de calidad requeridos, sino que también mejore continuamente, incrementando su fiabilidad y eficiencia.

Ciclo de vida de una incidencia

Un sistema de gestión de incidentes debe manejar los errores de “software”, teniendo en cuenta el siguiente proceso:

Figura 11. Ciclo de vida de una incidencia de software



- Nuevo: durante las pruebas de “software” los errores de “software” deben ser identificados y registrados por el “tester” de “software” siendo ese instante donde se reporta el error.
- Abierto: el equipo de desarrollo adquiere el incidente y empieza a analizarlo.
- Asignado: el incidente es asignado al desarrollador.
- Diferido: si el incidente aplica y es de bajo impacto.
- Reabierto: cuando el incidente es corregido y es retornado al equipo de “testers” de “software”.
- Verificado: si la prueba es positiva.
- Cerrado: posterior a una verificación por parte del equipo de “testing”.

El proceso mencionado se adecúa a proyectos de “software” en fase de desarrollo, pero no así en ambientes de producción.

Redacción del reporte de Incidencias

El propósito del informe de incidentes es documentar cada incidente, por lo tanto, se debe seguir un procedimiento para gestionar de forma sistémica y ordenada las incidencias de producto halladas a lo largo de la ejecución de los casos de prueba. Este procedimiento tiene las siguientes actividades:

- a) Verificar con las áreas correspondientes las incidencias.
- b) Verificar que el estado de las incidencias este actualizado.
- c) Realizar repuebas y regresión, es decir generar otra vez la ejecución de los casos de prueba necesarios.

Por otra parte, para estructurar el informe de incidentes se requiere tener en cuenta los siguientes puntos:

- Identificador.
- Resumen y descripción del incidente.
- Descripción de datos (fecha/hora, entradas, resultados esperados).
- Impacto que los incidentes tendrán con respecto a las pruebas.

Reporte de defectos

Cuando se confirma la existencia de un defecto, se elabora un reporte. Este documento detalla los fallos identificados durante el proceso de pruebas, subrayando la importancia de que cada defecto detectado cuente con un reporte individual. Esto facilita la trazabilidad y su corrección posterior. A continuación, se describe la estructura típica de un reporte de defectos.

- a) Identificador.
- b) Título.
- c) Reportado por.
- d) Asignado a.
- e) Fecha de creación.
- f) Severidad.
- g) Nombre de la aplicación.
- h) Descripción.
- i) Método de reproducción.
- j) Información adicional.

Informe de resultados de pruebas

El informe de resultados de pruebas presenta y organiza un análisis conciso de los resultados obtenidos en las pruebas, así como las principales métricas utilizadas para la evaluación y verificación. Por tanto, la responsabilidad de este documento recae en el gerente de pruebas. Este informe debe incluir los siguientes puntos:

- a) Alcance de las pruebas, es decir lo que es necesario incluir y no incluir.
- b) Resumen de pruebas.
 - Casos de prueba planificados.
 - Casos de prueba ejecutados.
 - Casos de prueba exitosos.
 - Casos de prueba fallidos.
 - Casos de prueba bloqueados.
- c) Listado de defectos detectados.
- d) Conclusiones.

2. Elaboración del plan de mejora

Cuando una organización alcanza el éxito, suele deberse a su capacidad para integrar la mejora continua en cada uno de sus procesos y actividades diarias. La mejora se logra cuando la organización toma en cuenta tanto sus fortalezas como sus debilidades. Sin embargo, identificar la situación actual de la organización a través de un diagnóstico es también crucial. Una vez realizado el diagnóstico, se puede establecer la estrategia a seguir para que los destinatarios de los servicios perciban de manera significativa la mejora implementada.

Identificar el área de mejora

Una vez elaborado el diagnóstico, se conocen las principales fortalezas y debilidades. Por lo tanto, es importante identificar las áreas de mejora apoyándose en las fortalezas para lograr superar las debilidades, siendo esta una opción de cambio óptima. A continuación, se muestra una tabla como una opción para identificar el área de mejora de acuerdo con las fortalezas y debilidades.

Figura 12. Ejemplo estructura de tabla de fortalezas, debilidades y área de mejora

Fortalezas	Debilidades	Áreas de mejora
1.	1.	1.
2.	2.	2.
3.	3.	3.

Resolver problemas y mejorar un área específica comienza por identificar las causas que los originaron. Por lo tanto, existen varias herramientas metodológicas diseñadas para esta finalidad. A continuación, se mencionan algunas de ellas:

Figura 13. Diagrama de espina de pescado comentado

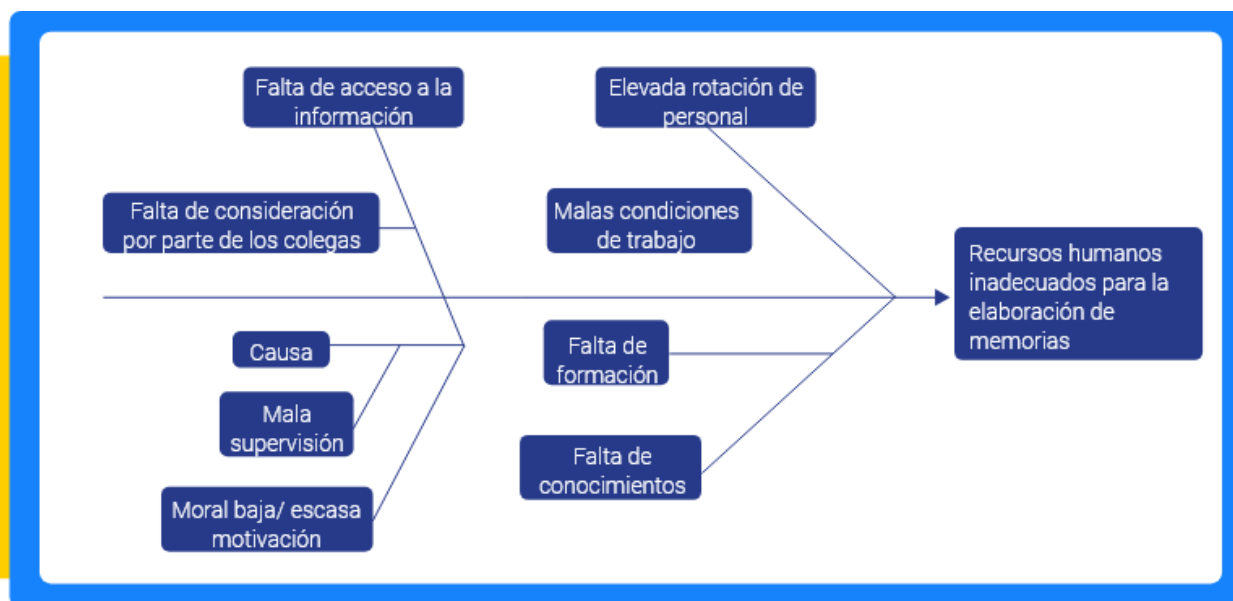


Figura 14. El diagrama de espina (causa-efecto)

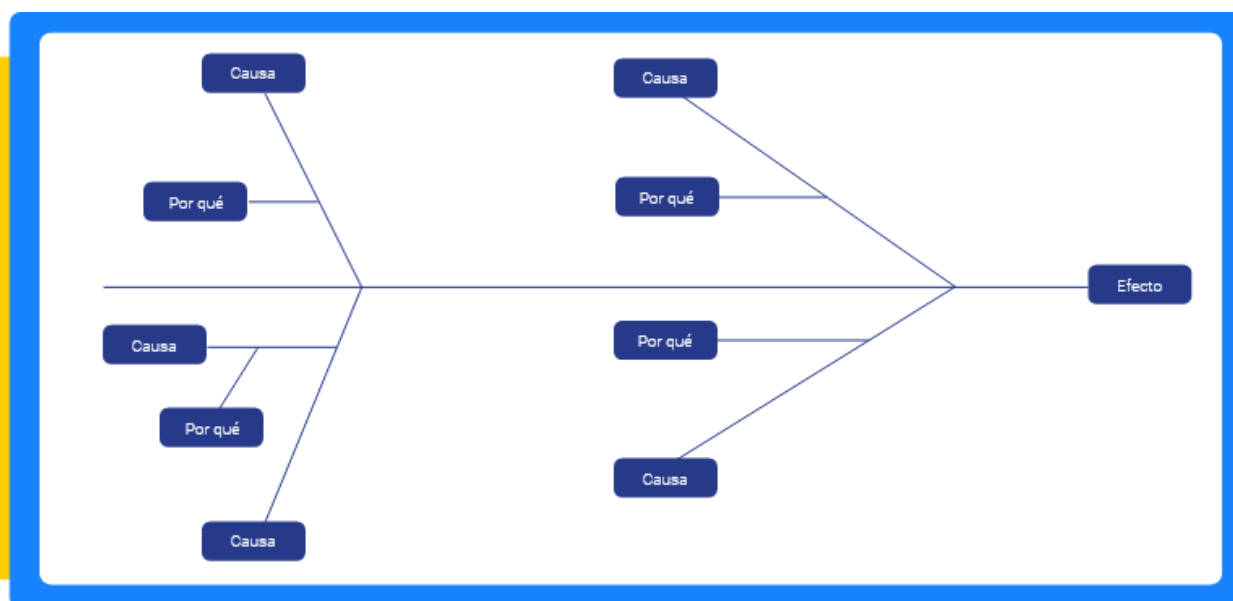


Figura 15. Diagrama de Pareto

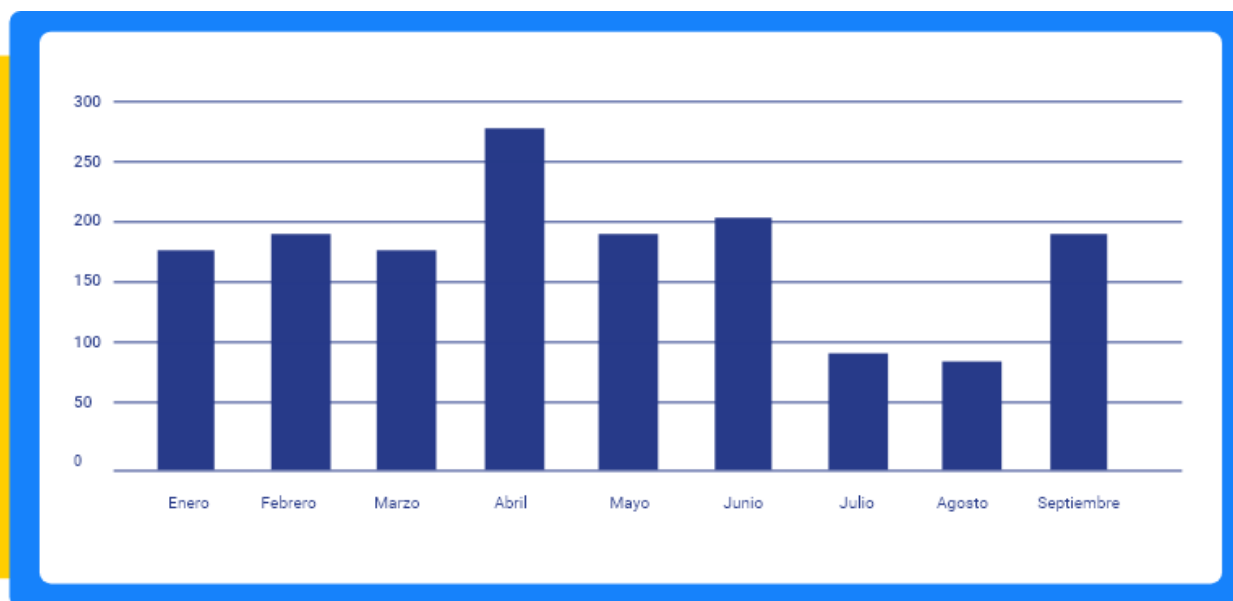


Figura 16. Tormenta de ideas



El uso de estas herramientas contribuye a un análisis más profundo del problema y se convierten en directrices clave al momento de establecer acciones de mejora.

Formular el objetivo

Una vez conocidas e identificadas las causas del problema y las áreas de mejora, se pueden establecer los objetivos y determinar el tiempo para lograrlos. Al momento de escribir los objetivos, estos deben ser:

- Precisos o concretos.
- Expresar de forma indiscutible el resultado que se quiere alcanzar.
- Su redacción debe ser clara.
- Deben ser alcanzables, comprensibles y flexibles, es decir que se puedan cumplir, interpretar y en algún caso extremo modificar.

2.1. Seleccionar las acciones de mejora

En este punto, es necesario seleccionar las opciones de mejora para dar prioridad a las más adecuadas. Para ello, se pueden emplear técnicas como la del grupo nominal y la tormenta de ideas, entre otras. Estas técnicas permiten establecer acciones de mejora que se convierten en soluciones para las debilidades identificadas. Esto significa que, para alcanzar los objetivos establecidos, se debe contar con una lista de las principales actuaciones.

A continuación, se presenta una tabla utilizada para seleccionar las acciones de mejora. Para ello, es imprescindible haber identificado previamente el problema, la causa y su objetivo.

Figura 17. Ejemplo estructura de tabla sobre acciones de mejora

ÁREA DE MEJORA N.º 1:	
Descripción del problema	
Causas que provocan el problema	
Objetivo a conseguir	
Acciones de mejora	
Beneficios esperados	

2.2. Realizar una planeación

El listado de las principales actuaciones se genera sin tener en cuenta un orden de prioridad específico. No obstante, ciertas restricciones inherentes a las acciones pueden limitar su inicio, o incluso llevar a la posible eliminación o aplazamiento del plan de mejora. Por lo tanto, es crucial conocer las restricciones que limitan su viabilidad. Determinar un orden óptimo no se basa únicamente en identificar las acciones relacionadas con los factores de mayor urgencia; otros criterios también se consideran al tomar esta decisión. A continuación, se describen algunos de estos criterios:

Dificultad de la implantación: en este momento se prioriza el grado de dificultad de menor a mayor.

- Mucha
- Bastante
- Poca
- Ninguna

Plazo de implantación: es de resaltar que existen acciones de mejora, cuyo tiempo está establecido y no requiere de un esfuerzo profundo, por lo tanto, pueden

efectuarse a corto plazo. Sin embargo, hay acciones que su realización requiere de un extenso tiempo de implantación.

- Largo
- Medio
- Corto
- Inmediato

Impacto en la organización: este es el resultado de la actuación que se va a implantar, medido por medio del grado de mejora obtenido, puesto que, un cambio fundamental tendrá un impacto más grande que cambios continuos pequeños.

- Ninguno
- Poco
- Bastante
- Mucho

2.3. Seguimiento del plan de mejora

En este momento, se procede a elaborar un cronograma para la implantación y seguimiento de las acciones de mejora. Dicho cronograma organizará de manera ordenada las prioridades junto con los plazos establecidos para su desarrollo.

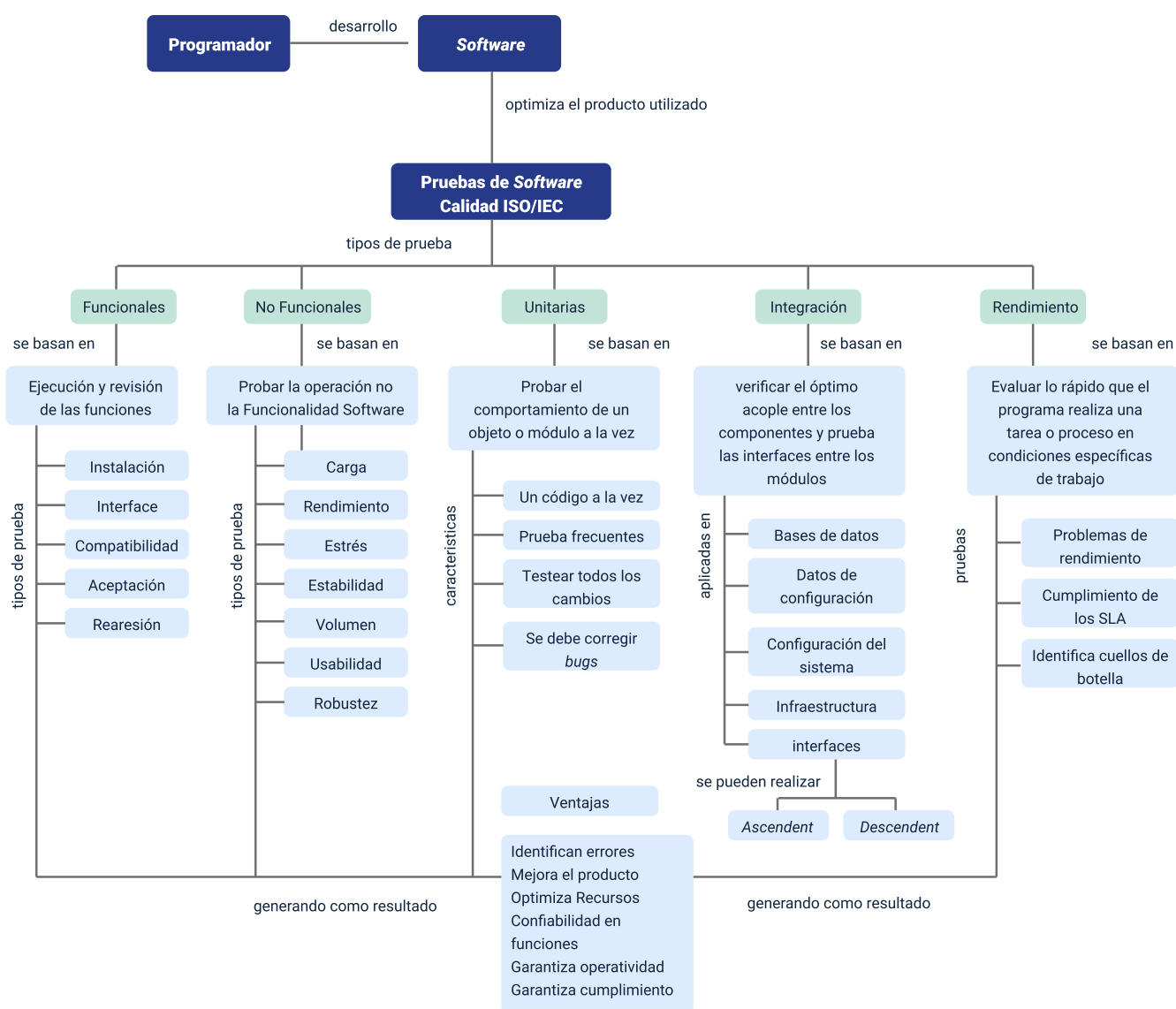
Una vez seleccionadas las acciones por orden de prioridad, se construye el plan de mejoras. Este incluirá los elementos necesarios para realizar un seguimiento detallado del plan, garantizando así su eficiencia y eficacia. Todo ello teniendo en cuenta la tabla presentada en la siguiente tabla:

Figura 18. Estructura plan de mejora

Acciones de mejora	Tareas	Responsable de tarea	Tiempos (inicio-final)	Recursos necesarios	Financiación	Indicador seguimiento	Responsable seguimiento

Síntesis

A continuación, se muestra un mapa conceptual con los elementos más importantes desarrollados en este componente.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
1.1 Tipos de pruebas	SENA, E. d. (24 de 08 de 2021). Tipos de pruebas de “software”.	Video	https://www.youtube.com/watch?v=Uf1Kk52ONYc
1.1.1 Pruebas Funcionales	IBM (2021). Pruebas Funcionales.	Documento	https://www.ibm.com/docs/es/rtw/9.1.0?topic=SSBLQQ_9.1.0/com.ibm.rational.test.ft.doc/topics/Getting_Started_With_Ivory.html
1.1.5 Pruebas de Rendimiento	IBM (2021). Pruebas de rendimiento.	Documento	https://www.ibm.com/docs/es/rtw/9.0.0?topic=phases-performance-testing
1.2 Agile Testing	QA, H.-O. (04 de 09 de 2020). Cuadrantes de prueba.	Video	https://www.youtube.com/watch?v=gz7A7EMZT_4
1.2.1 Test Driven Development (TDD)	Zapata, M. (07 de 06 de 2020). Pruebas unitarias y Test-Driven Development.	Video	https://www.youtube.com/watch?v=YuRdaR6wwWU
1.2.6 Pruebas de Exploratorias, Usabilidad y Aceptación	Silva, F. (2015, 27 mayo). Cómo mejorar la usabilidad de tu diseño web. Blog IDA Chile Estrategia para el éxito de tu negocio.	Blog	https://blog.ida.cl/disenio/como-mejorar-usabilidad-disenio-web/
1.2.6 Pruebas de Exploratorias, Usabilidad y Aceptación	Zamora, A., Benitez, J., & M. M. (25 de 06 de 2020). Test de usabilidad: identificando mejoras con nuestros usuarios.	Workshop	https://www.youtube.com/watch?v=EeRtQUljvOM

Glosario

Aplicación: una aplicación es un programa informático diseñado como una herramienta para realizar operaciones o funciones específicas. Generalmente, son diseñadas para facilitar ciertas tareas complejas y hacer más sencilla la experiencia informática de las personas.

ATDD: desarrollo orientado a pruebas de aceptación (“Acceptance test–driven development”) es una metodología de desarrollo basada en la comunicación entre los clientes comerciales, los desarrolladores y los evaluadores.

Automatización de pruebas: es la práctica que permite controlar la ejecución de un producto “software” de manera automática.

BDD: “Behaviour Driven Development” es una estrategia de desarrollo dirigido por comportamiento, se define en un idioma común entre todos los stakeholders, lo que mejora la comunicación entre equipos tecnológicos y no técnicos.

Incidencia: suceso que se produce durante una actividad y puede causar, una disminución de calidad de este.

Integración Continua: es una práctica de desarrollo de “software” por medio de la cual los desarrolladores combinan los cambios en el código en un repositorio.

QA: calidad de “software” (“Quality Software”) trata los conceptos, los métodos, las técnicas, los procedimientos y los estándares necesarios para producir productos y procesos “software” de alta calidad.

TDD: desarrollo Guiado por Pruebas (“Test Driven Development”) con el TDD se puede agilizar el proceso de creación de código. Este se centra más por el qué y el por qué antes del cómo.

UX: experiencia de usuario (“User Experience”) es el conjunto de factores y elementos relativos a la interacción del usuario con un entorno o dispositivo concretos, dando como resultado una percepción positiva o negativa de dicho servicio, producto o dispositivo.

Referencias bibliográficas

Beck, K., & Andres, C. (2004b). Extreme Programming Explained: Embrace Change, 2nd Edition (The XP Series) (2nd ed.). Addison-Wesley.

Bustamante Ramírez, J. (2011). Sistema de informes para pruebas de “software”. Obtenido de

<http://bibliotecadigital.iue.edu.co/xmlui/handle/20.500.12717/153>

Clemente, P. J., & Gómez, A. (2014). Aplicación de un proceso de mejora continua en una asignatura de Desarrollo de “Software” Dirigido por Modelos. Obtenido de

<http://hdl.handle.net/2099/15497>

Jeffries, R. (2011). What is Extreme Programming? Ronjeffries.Com.

<https://ronjeffries.com/xprog/what-is-extreme-programming/>

Kruchten, P. (2003). The Rational Unified Process: An Introduction (3rd Edition) (3rd ed.). Addison-Wesley Professional.

Maida, EG, Pacienza, J. (2015). Metodologías de desarrollo de “software” [en línea]. Tesis de Licenciatura en Sistemas y Computación. Facultad de Química e Ingeniería “Fray Rogelio Bacon”. Universidad Católica Argentina, 2015.

<https://repositorio.uca.edu.ar/bitstream/123456789/522/1/metodologias-desarrollo-software.pdf>

Manifiesto por el Desarrollo Ágil de “Software”. (2001). Agilemanifesto.Org.

<https://agilemanifesto.org/iso/es/manifesto.html>

Martin, J. (1991). Rapid Application Development. Macmillan Coll Div.

Mera Paz, J. A. (19 de 10 de 2016). Pruebas de Calidad “software”. Obtenido de <https://repository.ucc.edu.co/handle/20.500.12494/962>

Royce, W.W. (1970) Managing the Development of Large Software Systems. Proceedings of IEEE WESCON, 26, 328-388.

SCRUMstudy. (2013). A Guide to the Scrum Body of Knowledge (SBOK Guide) (2013th ed.). VMEdU Inc. Sommerville, I., Galipienso, M. I. A., & Martinez, A. B. (2005). Ingenieria del “Software”. Pearson Educacion.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Ervin Andrade	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Peter Pinchao	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Herrera Roldán	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edwin Sneider Velandia Suárez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Andrés Felipe Guevara Ariza	Locución	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jaime Hernán Tejada Llano	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia

Nombre	Cargo	Centro de Formación y Regional
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia

Pruebas de escritorio o trazas

Son las simulaciones de ejecución de un algoritmo que permite **determinar la validez o efectividad del mismo.**

La mayoría de las pruebas de escritorio **se representan en una tabla**, donde en cada **columna se anotan las variables y condiciones** si las hay, en el orden en que aparecen en el algoritmo.

—	—	—
—		

Para entenderlo se tiene que recorrer **línea a línea el pseudocódigo** y así **identificar tanto las variables como condiciones.**

Es importante que en la tabla **no aparezcan variables o condiciones repetidas.**

Otras representaciones

También es posible realizar pruebas de escritorio a otros algoritmos, como los **diagramas de flujo.**

Validación de documentos

Breve descripción:

La validación de documentos son aquellos instrumentos que permiten verificar el estado de un proceso específico de manera detallada. Son tomados como punto de referencia para tener una acertada toma de decisiones dentro del proceso que se ejecutará o el sistema que se está diseñando.

Junio 2024

Tabla de contenido

Introducción	3
1. Informe de análisis	4
1.1. Proceso de búsqueda de la información, evaluación y síntesis de las fuentes	5
2. Instrumentos de medición o verificación	13
2.1. Lista de chequeo	13
2.2. Cálculo de métricas de software	17
3. Trazabilidad	19
Síntesis	23
Material complementario	24
Glosario	25
Referencias bibliográficas	27
Créditos	29

Introducción

¿Sabías que la falta de calidad de los sistemas que se desarrollan es uno de los mayores contribuyentes a las llamadas crisis del software?

Actualmente, el desarrollo del software ha tenido un crecimiento muy importante y su implementación en diferentes áreas y productos ha crecido a pasos agigantados; por este motivo, en este componente se profundiza en la creación y correcto uso de los instrumentos que permiten la descripción de las características y las funcionalidades de un sistema, verificando el estado de los procesos e identificando si los requisitos que se esperan obtener son los adecuados, lo cual facilita una correcta trazabilidad y se logra una óptima toma de decisiones.

Para lograrlo, estudiaremos qué es el informe de análisis y qué son las fuentes de información, para continuar con los instrumentos de medición o verificación, donde encontramos la lista de chequeo y el cálculo de métricas de software, y finalizaremos con la trazabilidad, uno de los temas trascendentales en el proceso de desarrollo de software, porque permite hacer un seguimiento riguroso al proyecto, desde la idea básica hasta los entregables.

Bienvenido.

1. Informe de análisis

Dorrego (1994), define que el informe de análisis son documentos en los cuales se plasma una serie de procesos, de acuerdo con los requerimientos y objetivos a alcanzar, también conocidos como reporting y su fin es realizar un análisis exhaustivo del sistema que se desea desarrollar y el rendimiento de los procesos, logrando influir directamente en la toma de decisiones.

A continuación, en el siguiente video, se presentan algunos aspectos a tener en cuenta para el desarrollo de un informe de análisis.

Video 1. Informe de análisis



[Enlace de reproducción del video](#)

Síntesis del video: Informe de análisis

El informe de análisis se desarrolla con base en los instrumentos que se utilizan para evaluar el software, el análisis del informe sobre las selecciones, se documenta en listas de chequeo o en escalas de valoración. Estos instrumentos se elaboran a partir del uso y evaluación de las variables, a fin de garantizar su validez.

Con su uso se garantiza la objetividad en las decisiones de los expertos, como los desarrolladores que participan en el proyecto; los datos se pueden analizar directamente a manera de mesa redonda, sin que sea necesario anotarlos; así se discute con los programadores sus opiniones anotadas en los instrumentos, con el fin de escuchar sus sugerencias sobre las posibles soluciones a las fallas que se hayan encontrado.

Evaluar la revisión del objetivo u objetivos terminales puede conducir a su reformulación, modificación o agregar componentes para hacer un software más robusto y completo.

Es importante tener en cuenta la información obtenida y como punto de partida, el informe de requisitos de software, para generar una óptima validación de documentos.

1.1. Proceso de búsqueda de la información, evaluación y síntesis de las fuentes

Al momento de realizar un informe, el principal elemento de trabajo es la información que tenemos o hemos recolectado para tal fin.

Esta información debe ser:

- Confiable
- Objetiva
- Precisa
- Actualizada

Aquí se busca evaluar la calidad de las fuentes como son seguimientos, pruebas, historias de usuario, listas de chequeo, y en este caso específico, el informe de requerimientos de software.

Comencemos hablando de las fuentes de información, las cuales son todos aquellos medios de los cuales procede la información, que satisfacen las necesidades de conocimiento de una situación o problema presentado, que posteriormente será utilizado para lograr los objetivos esperados.

De acuerdo a su origen, se clasifican en: fuentes primarias y fuentes secundarias.

Según Torres et al. (2019):

- Las fuentes primarias, son aquellas en las que los datos provienen directamente del objeto de estudio.
- Las fuentes secundarias son aquellas que parten de datos preelaborados, como pueden ser datos obtenidos de otros proyectos, de Internet y de medios de comunicación.

A su vez, las fuentes primarias pueden subdividirse en:

- **Observación directa**

Cuando el investigador toma directamente los datos de la población, sin necesidad de cuestionarios o entrevistadores. Por ejemplo, cuando un profesor realiza un estudio sobre el avance de sus estudiantes en una materia.

- **De la naturaleza, artificiales y mixtos**

Cuando los datos no son obtenidos directamente, sino por medio de un cuestionario, lista de chequeo u otra fuente.

Tenga en cuenta:

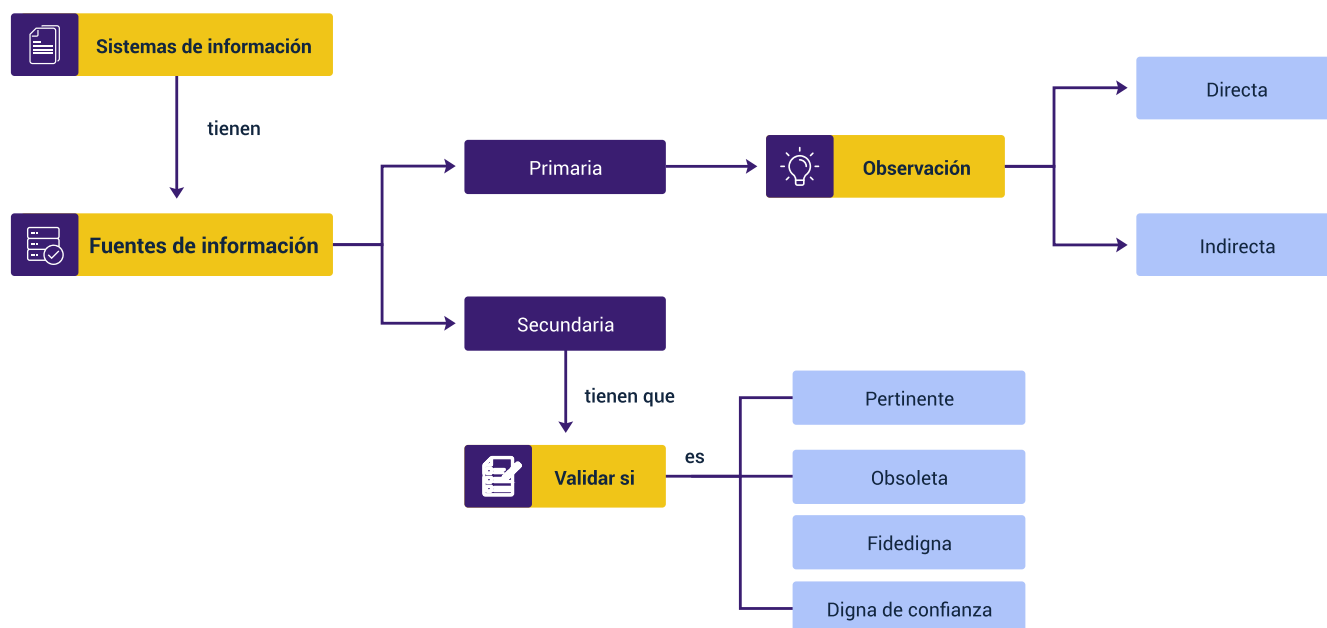
Las fuentes primarias se adquieren por medio de una investigación directa al objeto de estudio, por métodos establecidos con anticipación. Para reunir datos primarios, lo ideal es recurrir a un plan.

Ahora, las fuentes secundarias, para ser utilizadas, deben ser detalladas con cuatro preguntas básicas:

- ¿Es pertinente?
- ¿Es obsoleta?
- ¿Es fidedigna?
- ¿Es digna de confianza?

En el siguiente esquema, se resume de forma general, la forma en que se estructuran los sistemas de información.

Figura 1. Sistemas de información



Teniendo en cuenta la información anterior, debemos tener presente que un informe se debe estructurar de la siguiente manera:

- a) Resumen de no más de 200 palabras describiendo los objetivos y motivación del trabajo.
- b) **Introducción:** una introducción no debe contener más de 2 páginas, se debe detallar el ámbito, objetivos y resultados del trabajo, este debe contener: (Acosta Hoyos, 1972).

- **Contexto**

Descripción del ámbito del trabajo y definiciones generales, longitud máxima de 200 palabras.

- **Motivación**

Por qué es relevante dicho trabajo (usar referencias).

- **Tema del documento**

Sobre qué parte específica del trabajo trata este documento.

- **Objetivo**

Cuál es el objetivo de este documento.

- **Metodología**

Breve resumen del procedimiento seguido para elaborar el trabajo.

- c) **Contenido:** tantas secciones como hagan falta para explicar los contenidos técnicos del trabajo. Se puede ilustrar con imágenes y esquemas si es considerado necesario.

Uno de los componentes medulares del contenido es la redacción, aquí debemos tener en cuenta:

- **Ordena las ideas**

Lo importante es tener claro qué es lo que se quiere expresar y cómo se quiere decir, antes de iniciar a escribir. Para ello, elabora un esquema con las principales ideas.

- **Usar frases cortas**

Al momento de plasmar nuestras ideas, debemos ser directos y concisos, así logramos que los lectores entiendan el mensaje.

- **Adjetivos**

Cuando se usan bien, enriquecen el texto y ayudan a describir, de manera óptima, una idea. Se recomienda no excederse en ellos para no saturar el texto.

- **Revisar el texto**

Es recomendable leer el texto varias veces, porque al hacerlo de manera constante, podemos corregir errores y enriquecer el documento. Se recomienda estar actualizado en cuanto al lenguaje técnico.

- **No escribir como se habla**

La elaboración de la redacción difiere mucho del lenguaje hablado, por lo cual no debemos seguir el patrón de una conversación; cuando estamos redactando, debemos estructurar el texto de la siguiente manera: sujeto, predicado y complementos, expresando de manera concreta la idea principal.

- **Usar puntos y comas**

Es indispensable para expresarnos de manera correcta, dando fluidez al discurso escrito, permitiendo que el lector pueda disfrutar y comprender el texto.

- **Conclusiones y resultados**

Las conclusiones de un informe son el producto de la investigación y el análisis del tema que se trató. Aquí se exponen los resultados obtenidos; esta debe tener unas características, como son:

- Un resumen del texto que aporta una perspectiva general del contenido.
- Debe ser reflexiva sobre el tema tratado.

- Debe expresar de manera clara los resultados obtenidos en la investigación.
- Se debe utilizar un lenguaje formal.
- Hay que evitar redundancias.
- Los resultados deben ser explícitos y concretos.

Nota:

En las conclusiones debemos argumentar cómo hemos cubierto nuestros objetivos.

- d) **Metodología:** es la descripción del proceso seguido para la elaboración del trabajo, es decir, se explica el diseño de la investigación, como: hipótesis, variables, tipo de estudio, diseño utilizado, instrumentos de recolección de información, muestreo y procedimientos.

Acosta Hoyos (1972), afirma que la metodología de investigación tecnológica nos brinda las pautas para resolver problemas de la realidad y tiene base empírica, porque aplica los conocimientos teóricos de la ciencia a la práctica, adoptando el método experimental en la solución de los problemas en forma sistémica.

El tipo de investigación es tecnológico, porque busca resolver un problema práctico para satisfacer las necesidades de una solución de software.

Hipótesis

Según Pájaro (2002), se tienen dos grandes vías a partir de las cuales se descubren hipótesis y por lo tanto, se formulan o redactan:

- A partir de la razón (razonando).

- A partir de la experiencia, usando la inducción.

En ambos casos, es importante recalcar que el programador ha tenido una etapa previa de observación y acumulación de datos relacionados con el problema que está investigando; por lo que la tradición científica permite desprender al menos tres procedimientos que llevan a la formulación de hipótesis, los cuales son:

- a) La información respecto a la observación de un fenómeno es tan clara que conduce fácilmente a una hipótesis.
- b) El programador tiene una serie de convicciones o prejuicios filosóficos, producto de una gran experiencia en torno al problema que aborda, lo cual lo conduce fácilmente a una hipótesis.
- c) Una analogía, con relación al problema de interés, podrá conducir a la hipótesis buscada.

“Las variables son factores que intervienen tanto como causa o como resultado dentro del proceso o fenómeno de la realidad formando parte esencial de la estructura del tema, Las variables intervienen como causa o como efecto en el proceso investigativo, estas quedan identificadas desde el momento en que se define el problema”. (Espinoza Montez 2018)

2. Instrumentos de medición o verificación

Romero Alvarado (2018), plantea que los instrumentos de medición son herramientas necesarias para garantizar la calidad de los productos, sea por medición, verificación o control; es importante identificar si los instrumentos de verificación cumplen con los parámetros establecidos.

Por otro lado, los instrumentos de verificación son formatos de control que se elaboran con el fin de registrar mediciones o el cumplimiento total o parcial de tareas, acciones, funciones o el desempeño de una actividad en particular, bien sea de un sistema, un funcionario, un proceso o procedimiento.

Existen varios instrumentos de verificación como son: lista de chequeo y cálculo de métricas de software.

2.1. Lista de chequeo

La lista de chequeo es el instrumento más utilizado para recolectar información en la etapa de requisitos, también conocida como checklist u hoja de verificación. Es un formato que puede ser analógico o digital, y su función es comprobar, de una forma metódica y sistemática, el cumplimiento o no de los requisitos que contiene la lista, por este motivo, debe ser de fácil uso y muy concretas.

Es preferible redactar un cuestionario fácil de responder, que pueda contemplar todos los sitios necesarios y más fundamentales; por lo general, la checklist tiene varios destinatarios y usuarios, lo cual sirve para hacer un autocontrol o control interno, aunque también sirve para supervisar el trabajo de otros, pudiendo verificar el cumplimiento o no de ciertas funciones o parámetros; a esto lo conocemos con el nombre de control externo.

¿Qué debemos tener en cuenta al momento de elaborar una lista de chequeo?

- Se debe contener toda la información que se necesite.
- Debe ser corta, concisa y entendible.
- Debe estar redactada en un lenguaje sencillo y comprensible y con muy buena ortografía.
- El formato debe ser amigable y con buen diseño.

Debemos seleccionar un nombre clave, el cual identifique para qué sirve el instrumento, por ejemplo, verificación de equipos de cómputo; también se deben escribir las categorías o posibles variables de la información que se espere encontrar, establecer la escala de incidencia y diseñar la cuadrícula, asegurándose que todas las partes de la lista de chequeo, estén claramente descritas.

Esta herramienta busca quitar problemas y que se obtenga un formato claro y fácil de usar, entre más básico mejor.

Los siguientes, son los pasos para elaborar una lista de chequeo adecuada:

- **Determinar el ámbito**

Determinar el ámbito y la información que se desea evaluar u obtener.

Según Bichachi (2004), se debe tener en cuenta el lugar, necesidades tecnológicas, tipo de organización en la cual se aplica el instrumento, se deben realizar un análisis de las características de los procesos y procedimientos, enfocándose en la parte específica que se necesita analizar, esto es importante cuando la compañía cuenta con varias sedes.

- **Diseñar el formato de verificación**

Seleccionar un nombre claro para identificar para qué sirve este instrumento. Ejemplo: Lista de chequeo “Verificación de actualización de equipos de cómputo”. También escribir las categorías o posibles variables en la información que se espera encontrar, estableciendo la escala de incidencias. Se debe diseñar una cuadrícula para asegurarse que todas las partes estén claramente descritas. Esta herramienta busca quitar problemas y que se obtenga un formato claro y fácil de usar, entre más básico, mejor.

- **Diligenciar la información**

Diligenciar la información en el formato de verificación. Es fundamental capacitar al encargado de recopilar los datos en cuanto a la información que se necesita y cómo debe ser el trato con las personas que hacen parte de este proceso, se debe tomar el tiempo necesario para diligenciar los formatos, debido a que el recolectar esta información en ciertos casos requiere de mucho tiempo.

- **Registrar la información**

Registrar la información en una base de datos. Compilar la información en una hoja de cálculo es fundamental para el tratamiento de la información, porque así obtenemos resultados exactos y estadísticos.

Al momento de verificar este proceso, debemos tener en cuenta:

- ¿Tenemos en una lista toda la información que necesitamos?
- ¿Se han seguido los procesos?

- ¿Las métricas, rangos, tiempos, procesos, etc., se encuentran dentro de las tolerancias planeadas?
- ¿Se ha diligenciado el formato de manera correcta?
- ¿Hay observaciones que realizar?
- ¿Hubo acceso a toda la información que se necesitaba?
- ¿El o los productos cumplen con las especificaciones?
- ¿El producto final es conforme?

La siguiente tabla, presenta un ejemplo de lista de chequeo.

Tabla 1. Lista de chequeo Módulo de registro de usuarios

Ítem	Criterio o Actividad	Cumple	No Cumple	No Aplica
1	¿El tiempo de respuesta en los procesos es el esperado?			
2	¿Se especifican los criterios de seguridad en el sistema?			
3	¿El proceso está cuantificado en cantidad de usuarios y ancho de banda?			
4	¿Se especificaron posibles fallas en el sistema?			
5	¿Se definieron contingencias a posibles fallas?			
6	¿Hay estrategias de detección de errores en el sistema?			
7	¿Se especifican requisitos mínimos de hardware para la implementación del sistema?			

También se puede generar una tabla donde se coloque el número del criterio o ítem y una observación.

2.2. Cálculo de métricas de software

La ingeniería del software tiene como objetivo producir un sistema de óptima calidad. Para lograr este objetivo, se deben usar métodos efectivos junto con herramientas y aplicaciones actuales. Al mismo tiempo, un buen desarrollador debe medir si la calidad se va a cumplir en todo aspecto.

La definición estándar de calidad en ISO-8402 es:

“La totalidad de rasgos y características de un producto, proceso o servicio que sostiene la habilidad de satisfacer estados o necesidades implícitas”. (López Bartolomé, 2014)

Pereira et al (2012), refieren que la calidad de una solución de software o aplicación, es tan buena como los requisitos que la detallan y las mediciones que evalúan los siguientes aspectos:

- Requisitos.
- Análisis.
- Modelos de diseño.
- Código fuente.
- Casos de prueba.

Para obtener estas métricas de calidad, el desarrollador debe usar medidas técnicas, que evalúan la calidad con objetividad, y los errores y defectos.

Los anteriores autores, afirman que las métricas que provienen de estas medidas, proporcionan una indicación de la efectividad de las actividades de control, de calidad, errores detectados, arrojando una visión profunda de la eficacia de cada una de las actividades evaluadas en la métrica. A continuación, se presenta los factores y la categorización de los factores en el desarrollo de:

a) Factores calidad de software

Los factores que hacen variable la calidad del software, se pueden categorizar en dos grandes grupos:

- Factores que se miden directamente, cómo son los defectos por puntos de función.
- Factores que se miden indirectamente, cómo es la facilidad de uso o de mantenimiento.

b) Categorización factores

McCall planteó una categorización de factores que perturban la calidad de software, donde se enfocan en tres aspectos relevantes de una solución de software, los cuales son:

- Características operativas.
- Capacidad de cambio.
- Adaptabilidad a nuevos entornos.

3. Trazabilidad

Es uno de los temas trascendentales en el proceso de desarrollo de software, porque permite hacer un seguimiento riguroso al proyecto, desde la idea básica hasta los entregables. Los modelos de trazabilidad facilitan que los desarrolladores realicen una correcta evaluación del costo e impacto de las posibles modificaciones en los modelos y código del software, ayudando a enlazar o relacionar los requisitos con otros elementos del ciclo de vida, principalmente, casos de prueba y código.

En la opinión de Hernández et al (2015), hay que tener en cuenta que el código que se implementa debe ser consecuente con los requisitos, se puede estimar con precisión el esfuerzo que lleva implementar una petición de cambio sobre un requisito. Hay que tener en cuenta que la trazabilidad debe ser bidireccional, de requisitos a código y código a requisitos. Para este fin, se utiliza una matriz de trazabilidad de requerimientos, lo cual asegura un total cumplimiento del proyecto.

Para elaborar esta matriz, hay que tener como insumo fundamental los requisitos de software, entre ellos encontramos:

- **Requisitos del negocio**

Están asociados a objetivos de negocio y del proyecto.

- **Requisitos de los interesados stakeholders**

Pueden ser puntuales, impactando sobre otras áreas de la organización.

- **Requisitos de soluciones**

Pueden ser funcionales o no funcionales, tecnológicos, de cumplimiento de estándares, de apoyo y capacitación, de calidad o de presentación de informes.

- **Requisitos de transición**

Representan la transferencia del producto a la parte operativa, es decir, se trata de informar y proveer del material necesario al personal que tenga que trabajar el producto final.

Entonces, ¿cuál es la finalidad de elaborar una matriz de trazabilidad?

- **Primero**

Ayuda a determinar las mejoras que se puede hacer en los procesos estimados, basándose en las mejores prácticas CMMI.

- **Segundo**

Para informar a las partes interesadas stakeholders sobre cómo van los procesos.

- **Tercero**

Para cumplir con los requisitos contractuales.

¿Y cómo elaboramos una matriz de trazabilidad?

Tradicionalmente, se puede hacer manualmente con lápiz y papel o en una hoja de cálculo, aquí se relacionan los requisitos y los objetivos que se plantearon, adicionalmente, se acompaña de unos campos que aportan información y dan coherencia a esta relación.

Hoy en día existen programas de herramientas de gestión que ayudan para este fin, como Redmine, Trac o similar, aunque hay un gran abanico y de uso libre en el mercado.

A continuación, analicemos un ejemplo de matriz de trazabilidad.

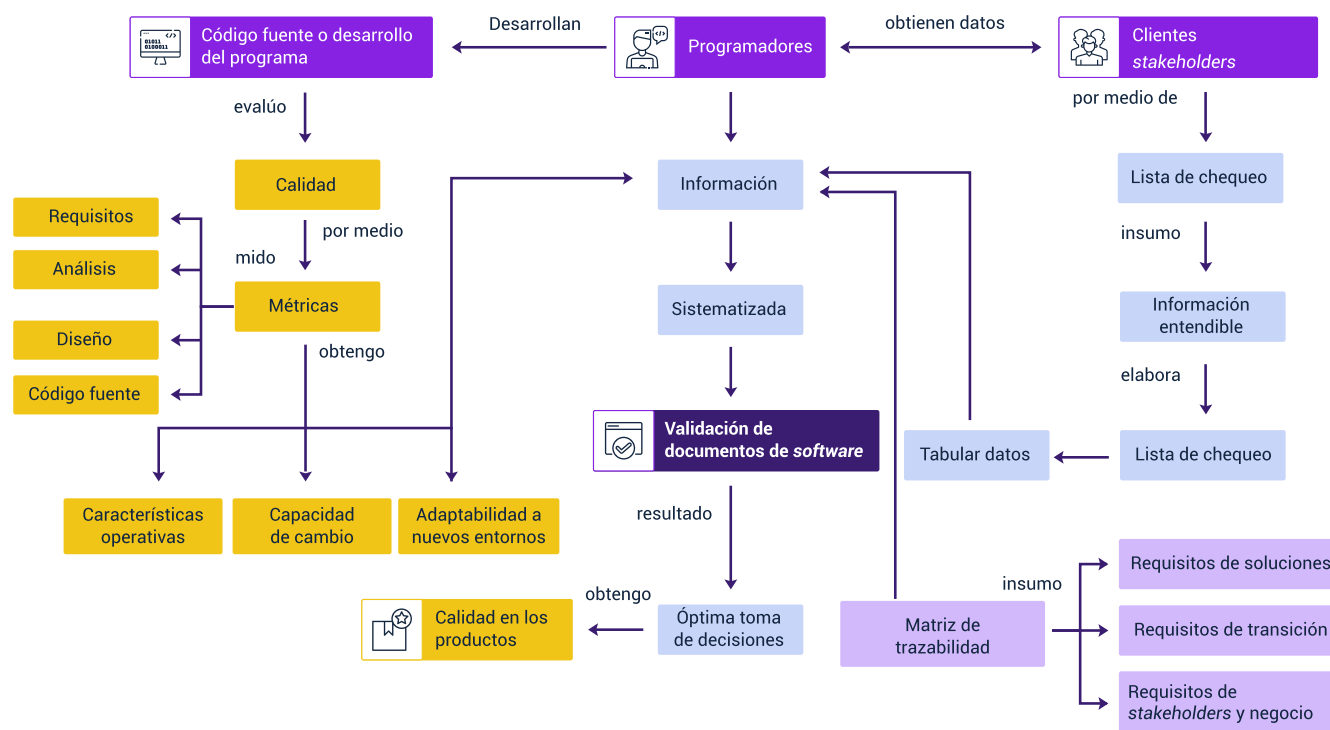
Tabla 2. Ejemplo Matriz de trazabilidad

Ítem	Requisitos	Tipo	Prioridad	Estado	Objetivo	Funcionalidad	Entrega
1	El sistema recoge los siguientes datos: Nombre Apellido Cédula Correo electrónico Dirección Teléfono	Venta	Alta	Activo	Obtener datos del usuario.	Registrar Modificar Eliminar	Entregado
2	Los datos serán registrados por un usuario registrado en el sistema.	Venta	Media	Activo	Garantizar la seguridad de la información.	Registrar Modificar Eliminar	Pendiente
3	El sistema recogerá los datos que el cliente va a adquirir con su código de barras.	Venta	Alta	Activo	Mejorar el control de existencias de productos.	Registrar Modificar Eliminar	Entregado
4	El sistema validará que los datos sean correctamente ingresados.	Venta	Baja	Activo	Validaciones.	Registrar Modificar Eliminar	Pendiente

Ítem	Requisitos	Tipo	Prioridad	Estado	Objetivo	Funcionalidad	Entrega
5	El sistema será capaz de generar la factura con los datos, ingresado haciendo la búsqueda en la base de datos.	Facturación	Alta	Activo	Agilizar el sistema de facturación.	Registrar Modificar Eliminar	Pendiente
6	El sistema será capaz de atender un número limitado de consultas o peticiones.	Venta	Alta	Activo	Control de sentencias SQL.	Registrar Modificar Eliminar	Pendiente

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo:



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Trazabilidad	Datko, C., & Carllinni, Y. (2014). Derivación de requisitos y construcción de trazabilidad entre artefactos del proceso de desarrollo. In XLIII Jornadas Argentinas de Informática e Investigación Operativa (43JAIIO)-XVII Concurso de Trabajos Estudiantiles (EST).	Otro	http://sedici.unlp.edu.ar/bitstream/handle/10915/42037/Documento_completo.pdf?sequence=1

Glosario

Capacidad de expansión: el grado con que se pueden aumentar el diseño arquitectónico, de datos o procedimental.

Complejión: el grado con que se a logrado la implementación total de una función.

Concisión: lo compacto que resulta ser el programa en términos de líneas de código.

Consistencia: el uso de un diseño uniforme y de técnicas de documentación, a través del proyecto de desarrollo del software.

Eficiencia de ejecución: el rendimiento del funcionamiento de un programa.

Estandarización de comunicaciones: el nivel de empleo de estándares de interfaces, protocolos y anchos de banda.

Estandarización de datos: el empleo de estructuras y tipos de datos estándares a lo largo del programa.

Exactitud: la exactitud de los cálculos y del control.

Facilidad de auditoría: la facilidad con la que se puede justificar el cumplimiento de los estándares.

Formación: el grado en que el software ayuda a los nuevos usuarios a manejar el sistema.

Generalidad: la extensión de aplicación potencial de los componentes del programa.

Independencia del hardware: el grado con que se desacopla el software del hardware donde opera.

Instrumentación: el grado con que el programa vigila su propio funcionamiento e identifica los errores que suceden.

Modularidad: la independencia funcional de componentes del programa.

Operatividad: la facilidad de operación de un programa.

Tolerancia al error: el deterioro causado cuando un programa descubre un error.

Trazabilidad: la capacidad de alcanzar una representación del diseño o un componente real del programa, hasta los requisitos.

Referencias bibliográficas

Acosta-Hoyos, L. E. (1972). Guía práctica para la investigación y redacción de informes (Vol. 146). Editorial Paidós.

Ayabaca, L. P., & Bernal, S. M. (2017). Verificación y Validación de Software Verification and Validation. Revista Killkana Técnica. Vol, 1(3).

Bichachi, D. S. (2004). El uso de las listas de chequeo (CHECK-LIST) como herramienta para controlar la calidad de las leyes. Recuperado de:
https://claudiabernazza.ar/ssgp/html/pdf/check_list.pdf

Botta, M., de Roig, N. F., & Izaguirre, I. (2002). Tesis, monografías e informes: nuevas normas y técnicas de investigación y redacción. Biblos.

Dorrego, E. (1994). Modelo para la producción y evaluación formativa de medios instruccionales, aplicado al video y al software. Revista de Tecnología Educativa, 12(3), 313-327.

Espinoza Freire, E. E. (2018). Las variables y su operacionalización en la investigación educativa. Parte I. Conrado, 14, 39-49.

Espinoza Montes, C. (2010). Metodología de investigación tecnológica.

Hernandez, F., Roldán, M. L., Vegetti, M. M., Gonnet, S. M., & Leone, H. P. (2015). TracEDaaS: Captura y Trazabilidad de Artefactos del Proceso de Diseño.

López Bartolomé, C. (2014). El desarrollo de software Open Source analizado desde dentro (Bachelor's thesis).

Pájaro, D. (2002). La formulación de hipótesis. Cinta de Moebio. Revista de Epistemología de Ciencias Sociales, (15).

Pereira, B., Ayaach, F., Quintero, H., Granadillo, I., & Bustamante, J. (2012). Métricas de calidad de software. Calidad Del Software.

Perez Quintero, C. I. (2019). Implementación del modelo Mccall para medir la calidad del software Siplan (Doctoral dissertation).

Romero Alvarado, W. A. (2018). Los instrumentos de verificación y su incidencia en el control de los procesos productivos de la paleta Armijos.

Tabares, M. S., Barrera, A. F., Arroyave, J. D., & Pineda, J. D. (2007). Un método para la trazabilidad de requisitos en el proceso unificado de desarrollo. Revista EIA, (8), 69-82.

Torres, M., Salazar, F. G., & Paz, K. (2019). Métodos de recolección de datos para una investigación.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Mario Fernando Meneses Calvache	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Ana Catalina Córdoba Sus	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Yerson Fabián Zárate Saavedra	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Edward Leonardo Pico Cabra	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Álvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia